GNSS-SDR

0.0.9

Generated by Doxygen 1.8.11

Contents

1 Main Page **1**

1.1 Contents . . . 1

1.2 Overview . . . 2

1.3 Building GNSS-SDR . . . 3

1.3.1 Debug and Release builds . . . 5

1.3.2 Updating GNSS-SDR . . . 5

1.4 Using GNSS-SDR . . . 5

1.5 Control plane . . . 6

1.5.1 Configuration . . . 6

1.5.2 GNSS block factory . . . 7

1.6 Signal Processing plane . . . 7

1.6.1 Signal Source . . . 8

1.6.2 Signal Conditioner . . . 8

1.6.3 Channel . . . 9

1.6.3.1 Acquisition . . . 9

1.6.3.2 Tracking . . . 10

1.6.3.3 Decoding of the navigation message . . . 11

1.6.4 Observables . . . 12

1.6.5 Computation of Position, Velocity and Time . . . 12

1.7 About the software license . . . 13

1.8 Publications and Credits . . . 13

1.9 Ok, now what? . . . 14

2 Reference Documents **15**

2.1 Interface Control Documents . . . 15

2.1.1 GPS . . . 15

2.1.2 GLONASS . . . 15

2.1.3 Galileo . . . 15

2.1.4 BeiDou . . . 16

2.1.5 Satellite Based Augmentation Systems (SBAS) . . . 16

2.2 Other Standards . . . 17

2.2.1 RINEX . . . 17

2.2.2 NMEA . . . 17

2.2.3 KML . . . 17

2.2.4 C++ Standards . . . 18

2.2.5 Positioning protocols in wireless communication networks . . . 18

3 Signal model **19**

3.1 GNSS signal model . . . 19

3.1.1 Global Positioning System (GPS) signal in space . . . 20

3.1.2 GLONASS signal in space . . . 21

3.1.3 Galileo signal in space . . . 22

3.1.4 Reference . . . 25

4 Todo List **27**

5 Hierarchical Index **29**

5.1 Class Hierarchy . . . 29

6 Class Index **33**

6.1 Class List . . . 33

7 File Index **43**

7.1 File List . . . 43

8 Class Documentation **53**

8.1 AcquisitionInterface Class Reference . . . 53

8.1.1 Detailed Description . . . 54

8.2 ArraySignalConditioner Class Reference . . . 54

8.2.1 Detailed Description . . . 55

8.2.2 Constructor & Destructor Documentation . . . 55

8.2.2.1 ArraySignalConditioner(ConfigurationInterface *configuration, std::shared_ptr< GNSSBlockInterface > data_type_adapt, std::shared_ptr< GNSSBlockInterface > in_filt, std::shared_ptr< GNSSBlockInterface > res, std::string role, std::string implementation) . . . 55

8.2.2.2 ~ArraySignalConditioner() . . . 55

8.2.3 Member Function Documentation . . . 55

8.2.3.1 implementation() . . . 55

8.3 beamformer Class Reference . . . 56

8.3.1 Detailed Description . . . 56

8.4 BeamformerFilter Class Reference . . . 56

8.4.1 Detailed Description . . . 57

8.4.2 Member Function Documentation . . . 57

8.4.2.1 implementation() . . . 57

8.5 byte_x2_to_complex_byte Class Reference . . . 57

8.5.1 Detailed Description . . . 58

8.6 ByteToShort Class Reference . . . 58

8.6.1 Detailed Description . . . 59

8.6.2 Member Function Documentation . . . 59

8.6.2.1 implementation() . . . 59

8.7 Channel Class Reference . . . 59

8.7.1 Detailed Description . . . 60

8.7.2 Constructor & Destructor Documentation . . . 60

8.7.2.1 Channel(ConfigurationInterface *configuration, unsigned int channel, std::shared_ptr< GNSSBlockInterface > pass_through, std::shared_ptr< AcquisitionInterface > acq, std::shared_ptr< TrackingInterface > trk, std::shared_ptr< TelemetryDecoderInterface > nav, std::string role, std::string implementation, boost::shared_ptr< gr::msg_queue > queue) . . . 60

8.7.2.2 ~Channel() . . . 60
8.7.3 Member Function Documentation . . . 61
8.7.3.1 implementation() . . . 61
8.7.3.2 set_signal(const Gnss_Signal &gnss_signal_) . . . 61
8.7.3.3 start_acquisition() . . . 61
8.8 channel_msg_receiver_cc Class Reference . . . 61
8.8.1 Detailed Description . . . 62
8.8.2 Constructor & Destructor Documentation . . . 62
8.8.2.1 ~channel_msg_receiver_cc() . . . 62
8.9 ChannelFsm Class Reference . . . 62
8.9.1 Detailed Description . . . 63
8.10 ChannelInterface Class Reference . . . 63
8.10.1 Detailed Description . . . 63
8.11 cl_fft_plan Struct Reference . . . 64
8.11.1 Detailed Description . . . 64
8.12 clFFT_Complex Struct Reference . . . 64
8.12.1 Detailed Description . . . 64
8.13 clFFT_Dim3 Struct Reference . . . 65
8.13.1 Detailed Description . . . 65
8.14 clFFT_SplitComplex Struct Reference . . . 65
8.14.1 Detailed Description . . . 65
8.15 complex_byte_to_float_x2 Class Reference . . . 65
8.15.1 Detailed Description . . . 66
8.16 complex_float_to_complex_byte Class Reference . . . 66
8.16.1 Detailed Description . . . 66
8.17 concurrent_map< Data > Class Template Reference . . . 67
8.17.1 Detailed Description . . . 67
8.18 concurrent_queue< Data > Class Template Reference . . . 67
8.18.1 Detailed Description . . . 68
8.19 ConfigurationInterface Class Reference . . . 68

8.19.1 Detailed Description . . . 69
8.20 ControlMessage Class Reference . . . 69
8.20.1 Detailed Description . . . 69
8.21 ControlMessage_ Struct Reference . . . 69
8.21.1 Detailed Description . . . 70
8.22 ControlMessageFactory Class Reference . . . 70
8.22.1 Detailed Description . . . 70
8.22.2 Constructor & Destructor Documentation . . . 70
8.22.2.1 ControlMessageFactory() . . . 70
8.22.2.2 ∼ControlMessageFactory() . . . 70
8.23 ControlThread Class Reference . . . 71
8.23.1 Detailed Description . . . 71
8.23.2 Constructor & Destructor Documentation . . . 71
8.23.2.1 ControlThread() . . . 71
8.23.2.2 ControlThread(std::shared_ptr< ConfigurationInterface > configuration) . . . 71
8.23.2.3 ∼ControlThread() . . . 72
8.23.3 Member Function Documentation . . . 72
8.23.3.1 flowgraph() . . . 72
8.23.3.2 run() . . . 72
8.23.3.3 set_control_queue(boost::shared_ptr< gr::msg_queue > control_queue) . . . 72
8.24 cpu_multicorrelator Class Reference . . . 73
8.24.1 Detailed Description . . . 73
8.25 cpu_multicorrelator_16sc Class Reference . . . 73
8.25.1 Detailed Description . . . 74
8.26 cshort_to_float_x2 Class Reference . . . 74
8.26.1 Detailed Description . . . 74
8.27 cuda_multicorrelator Class Reference . . . 74
8.27.1 Detailed Description . . . 75
8.28 direct_resampler_conditioner_cb Class Reference . . . 75
8.28.1 Detailed Description . . . 76

8.29 direct_resampler_conditioner_cc Class Reference . . . 76
8.29.1 Detailed Description . . . 76
8.30 direct_resampler_conditioner_cs Class Reference . . . 77
8.30.1 Detailed Description . . . 77
8.31 DirectResamplerConditioner Class Reference . . . 77
8.31.1 Detailed Description . . . 78
8.31.2 Member Function Documentation . . . 78
8.31.2.1 implementation() . . . 78
8.32 Fast_Correction Struct Reference . . . 78
8.32.1 Detailed Description . . . 79
8.33 FileConfiguration Class Reference . . . 79
8.33.1 Detailed Description . . . 79
8.33.2 Constructor & Destructor Documentation . . . 80
8.33.2.1 ∼FileConfiguration() . . . 80
8.34 FileSignalSource Class Reference . . . 80
8.34.1 Detailed Description . . . 80
8.34.2 Member Function Documentation . . . 81
8.34.2.1 implementation() . . . 81
8.35 FirFilter Class Reference . . . 81
8.35.1 Detailed Description . . . 81
8.35.2 Constructor & Destructor Documentation . . . 82
8.35.2.1 FirFilter(ConfigurationInterface ∗configuration, std::string role, unsigned int in_↩ streams, unsigned int out_streams) . . . 82
8.35.2.2 ∼FirFilter() . . . 82
8.35.3 Member Function Documentation . . . 82
8.35.3.1 implementation() . . . 82
8.36 FlexibandSignalSource Class Reference . . . 82
8.36.1 Detailed Description . . . 83
8.36.2 Member Function Documentation . . . 83
8.36.2.1 implementation() . . . 83
8.37 FreqXlatingFirFilter Class Reference . . . 83

8.37.1 Detailed Description . . . 84
8.37.2 Member Function Documentation . . . 84
8.37.2.1 implementation() . . . 84
8.38 FrontEndCal Class Reference . . . 84
8.38.1 Detailed Description . . . 84
8.38.2 Member Function Documentation . . . 85
8.38.2.1 estimate_doppler_from_eph(unsigned int PRN, double TOW, double lat, double lon, double height) . . . 85
8.38.2.2 get_ephemeris() . . . 85
8.38.2.3 GPS_L1_front_end_model_E4000(double f_bb_true_Hz, double f_bb_meas_↩ Hz, double fs_nominal_hz, double ∗estimated_fs_Hz, double ∗estimated_f_if_Hz, double ∗f_osc_err_ppm) . . . 85
8.38.2.4 set_configuration(std::shared_ptr< ConfigurationInterface > configuration) . . . 85
8.39 Galileo_Almanac Class Reference . . . 85
8.39.1 Detailed Description . . . 87
8.39.2 Constructor & Destructor Documentation . . . 87
8.39.2.1 Galileo_Almanac() . . . 87
8.40 galileo_e1_dll_pll_veml_tracking_cc Class Reference . . . 87
8.40.1 Detailed Description . . . 88
8.40.2 Member Function Documentation . . . 88
8.40.2.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 88
8.41 galileo_e1_ls_pvt Class Reference . . . 88
8.41.1 Detailed Description . . . 89
8.41.2 Member Data Documentation . . . 89
8.41.2.1 d_nchannels . . . 89
8.41.2.2 galileo_ephemeris_map . . . 89
8.42 galileo_e1_observables_cc Class Reference . . . 89
8.42.1 Detailed Description . . . 90
8.43 galileo_e1_pvt_cc Class Reference . . . 90
8.43.1 Detailed Description . . . 90
8.43.2 Constructor & Destructor Documentation . . . 91

8.43.2.1 ~galileo_e1_pvt_cc() . . . 91
8.43.3 Member Function Documentation . . . 91
8.43.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 91
8.44 Galileo_E1_Tcp_Connector_Tracking_cc Class Reference . . . 91
8.44.1 Detailed Description . . . 92
8.45 galileo_e1b_telemetry_decoder_cc Class Reference . . . 92
8.45.1 Detailed Description . . . 93
8.45.2 Member Function Documentation . . . 93
8.45.2.1 forecast(int noutput_items, gr_vector_int &ninput_items_required) . . . 93
8.45.2.2 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 93
8.45.2.3 set_channel(int channel) . . . 93
8.45.2.4 set_decimation(int decimation) . . . 93
8.45.2.5 set_satellite(Gnss_Satellite satellite) . . . 93
8.46 Galileo_E5a_Dll_Pll_Tracking_cc Class Reference . . . 93
8.46.1 Detailed Description . . . 94
8.47 galileo_e5a_noncoherentIQ_acquisition_caf_cc Class Reference . . . 94
8.47.1 Detailed Description . . . 95
8.47.2 Constructor & Destructor Documentation . . . 95
8.47.2.1 ~galileo_e5a_noncoherentIQ_acquisition_caf_cc() . . . 95
8.47.3 Member Function Documentation . . . 96
8.47.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 96
8.47.3.2 init() . . . 96
8.47.3.3 mag() . . . 96
8.47.3.4 set_active(bool active) . . . 96
8.47.3.5 set_channel(unsigned int channel) . . . 96
8.47.3.6 set_doppler_max(unsigned int doppler_max) . . . 97
8.47.3.7 set_doppler_step(unsigned int doppler_step) . . . 97
8.47.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 97
8.47.3.9 set_local_code(std::complex< float > *code, std::complex< float > *codeQ) . . . 97

8.47.3.10 set_state(int state) . . . 98
8.47.3.11 set_threshold(float threshold) . . . 98
8.48 galileo_e5a_telemetry_decoder_cc Class Reference . . . 98
8.48.1 Detailed Description . . . 99
8.48.2 Member Function Documentation . . . 99
8.48.2.1 forecast(int noutput_items, gr_vector_int &ninput_items_required) . . . 99
8.48.2.2 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 99
8.48.2.3 set_channel(int channel) . . . 99
8.48.2.4 set_satellite(Gnss_Satellite satellite) . . . 99
8.49 Galileo_Ephemeris Class Reference . . . 99
8.49.1 Detailed Description . . . 102
8.49.2 Member Function Documentation . . . 102
8.49.2.1 Galileo_System_Time(double WN, double TOW) . . . 102
8.49.2.2 satellitePosition(double transmitTime) . . . 102
8.49.2.3 serialize(Archive &archive, const unsigned int version) . . . 102
8.49.2.4 sv_clock_drift(double transmitTime) . . . 102
8.49.2.5 sv_clock_relativistic_term(double transmitTime) . . . 102
8.49.3 Member Data Documentation . . . 102
8.49.3.1 A_1 . . . 102
8.49.3.2 af0_4 . . . 103
8.49.3.3 af1_4 . . . 103
8.49.3.4 af2_4 . . . 103
8.49.3.5 BGD_E1E5a_5 . . . 103
8.49.3.6 BGD_E1E5b_5 . . . 103
8.49.3.7 C_ic_4 . . . 103
8.49.3.8 C_is_4 . . . 103
8.49.3.9 C_rc_3 . . . 103
8.49.3.10 C_rs_3 . . . 104
8.49.3.11 C_uc_3 . . . 104
8.49.3.12 C_us_3 . . . 104

8.49.3.13 d_satpos_X . . . 104
8.49.3.14 d_satpos_Y . . . 104
8.49.3.15 d_satpos_Z . . . 104
8.49.3.16 d_satvel_X . . . 104
8.49.3.17 d_satvel_Y . . . 105
8.49.3.18 d_satvel_Z . . . 105
8.49.3.19 delta_n_3 . . . 105
8.49.3.20 E1B_DVS_5 . . . 105
8.49.3.21 E1B_HS_5 . . . 105
8.49.3.22 E5a_DVS . . . 105
8.49.3.23 E5a_HS . . . 105
8.49.3.24 E5b_DVS_5 . . . 105
8.49.3.25 E5b_HS_5 . . . 106
8.49.3.26 e_1 . . . 106
8.49.3.27 Galileo_dtr . . . 106
8.49.3.28 i_0_2 . . . 106
8.49.3.29 i_satellite_PRN . . . 106
8.49.3.30 iDot_2 . . . 106
8.49.3.31 M0_1 . . . 106
8.49.3.32 OMEGA_0_2 . . . 106
8.49.3.33 omega_2 . . . 107
8.49.3.34 OMEGA_dot_3 . . . 107
8.49.3.35 t0c_4 . . . 107
8.49.3.36 t0e_1 . . . 107
8.49.3.37 TOW_5 . . . 107
8.49.3.38 WN_5 . . . 107
8.50 Galileo_Fnav_Message Class Reference . . . 107
8.50.1 Detailed Description . . . 110
8.50.2 Member Data Documentation . . . 110
8.50.2.1 flag_all_almanac . . . 110

8.50.2.2 flag_all_ephemeris . . . 110
8.50.2.3 flag_almanac_1 . . . 111
8.50.2.4 flag_almanac_2 . . . 111
8.50.2.5 flag_ephemeris_1 . . . 111
8.50.2.6 flag_ephemeris_2 . . . 111
8.50.2.7 flag_ephemeris_3 . . . 111
8.50.2.8 flag_iono_and_GST . . . 111
8.50.2.9 flag_TOW_set . . . 111
8.50.2.10 flag_utc_model . . . 112
8.51 Galileo_Iono Class Reference . . . 112
8.51.1 Detailed Description . . . 112
8.51.2 Constructor & Destructor Documentation . . . 113
8.51.2.1 Galileo_Iono() . . . 113
8.51.3 Member Data Documentation . . . 113
8.51.3.1 ai0_5 . . . 113
8.51.3.2 ai1_5 . . . 113
8.51.3.3 ai2_5 . . . 113
8.51.3.4 Region1_flag_5 . . . 113
8.51.3.5 Region2_flag_5 . . . 113
8.51.3.6 Region3_flag_5 . . . 113
8.51.3.7 Region4_flag_5 . . . 114
8.51.3.8 Region5_flag_5 . . . 114
8.51.3.9 TOW_5 . . . 114
8.51.3.10 WN_5 . . . 114
8.52 Galileo_Navigation_Message Class Reference . . . 114
8.52.1 Detailed Description . . . 118
8.52.2 Member Data Documentation . . . 119
8.52.2.1 A_0G_10 . . . 119
8.52.2.2 A_1 . . . 119
8.52.2.3 A_1G_10 . . . 119

8.52.2.4 af0_4 . . . 119
8.52.2.5 af1_4 . . . 119
8.52.2.6 af2_4 . . . 119
8.52.2.7 ai0_5 . . . 119
8.52.2.8 ai1_5 . . . 120
8.52.2.9 ai2_5 . . . 120
8.52.2.10 BGD_E1E5a_5 . . . 120
8.52.2.11 BGD_E1E5b_5 . . . 120
8.52.2.12 C_ic_4 . . . 120
8.52.2.13 C_is_4 . . . 120
8.52.2.14 C_rc_3 . . . 120
8.52.2.15 C_rs_3 . . . 120
8.52.2.16 C_uc_3 . . . 121
8.52.2.17 C_us_3 . . . 121
8.52.2.18 delta_n_3 . . . 121
8.52.2.19 E1B_DVS_5 . . . 121
8.52.2.20 E1B_HS_5 . . . 121
8.52.2.21 E5b_DVS_5 . . . 121
8.52.2.22 E5b_HS_5 . . . 121
8.52.2.23 e_1 . . . 121
8.52.2.24 flag_all_almanac . . . 122
8.52.2.25 flag_all_ephemeris . . . 122
8.52.2.26 flag_almanac_1 . . . 122
8.52.2.27 flag_almanac_2 . . . 122
8.52.2.28 flag_almanac_3 . . . 122
8.52.2.29 flag_almanac_4 . . . 122
8.52.2.30 flag_ephemeris_1 . . . 122
8.52.2.31 flag_ephemeris_2 . . . 122
8.52.2.32 flag_ephemeris_3 . . . 123
8.52.2.33 flag_ephemeris_4 . . . 123

8.52.2.34 flag_iono_and_GST . . . 123
8.52.2.35 flag_TOW_set . . . 123
8.52.2.36 flag_utc_model . . . 123
8.52.2.37 Galileo_dtr . . . 123
8.52.2.38 galileo_satpos_X . . . 123
8.52.2.39 galileo_satpos_Y . . . 124
8.52.2.40 galileo_satpos_Z . . . 124
8.52.2.41 galileo_satvel_X . . . 124
8.52.2.42 galileo_satvel_Y . . . 124
8.52.2.43 galileo_satvel_Z . . . 124
8.52.2.44 i_0_2 . . . 124
8.52.2.45 iDot_2 . . . 124
8.52.2.46 IOD_nav_1 . . . 125
8.52.2.47 IOD_nav_2 . . . 125
8.52.2.48 M0_1 . . . 125
8.52.2.49 OMEGA_0_2 . . . 125
8.52.2.50 omega_2 . . . 125
8.52.2.51 OMEGA_dot_3 . . . 125
8.52.2.52 Region1_flag_5 . . . 125
8.52.2.53 Region2_flag_5 . . . 125
8.52.2.54 Region3_flag_5 . . . 126
8.52.2.55 Region4_flag_5 . . . 126
8.52.2.56 Region5_flag_5 . . . 126
8.52.2.57 t0c_4 . . . 126
8.52.2.58 t0e_1 . . . 126
8.52.2.59 t_0G_10 . . . 126
8.52.2.60 WN_0G_10 . . . 126
8.53 galileo_pcps_8ms_acquisition_cc Class Reference . . . 127
8.53.1 Detailed Description . . . 128
8.53.2 Constructor & Destructor Documentation . . . 128

8.53.2.1 ~galileo_pcps_8ms_acquisition_cc() . . . 128
8.53.3 Member Function Documentation . . . 128
8.53.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 128
8.53.3.2 init() . . . 128
8.53.3.3 mag() . . . 128
8.53.3.4 set_active(bool active) . . . 128
8.53.3.5 set_channel(unsigned int channel) . . . 129
8.53.3.6 set_doppler_max(unsigned int doppler_max) . . . 129
8.53.3.7 set_doppler_step(unsigned int doppler_step) . . . 129
8.53.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 129
8.53.3.9 set_local_code(std::complex< float > *code) . . . 130
8.53.3.10 set_state(int state) . . . 130
8.53.3.11 set_threshold(float threshold) . . . 130
8.54 Galileo_Utc_Model Class Reference . . . 130
8.54.1 Detailed Description . . . 131
8.54.2 Constructor & Destructor Documentation . . . 131
8.54.2.1 Galileo_Utc_Model() . . . 131
8.54.3 Member Function Documentation . . . 131
8.54.3.1 GST_to_UTC_time(double t_e, int WN) . . . 131
8.54.4 Member Data Documentation . . . 131
8.54.4.1 t0t_6 . . . 131
8.54.4.2 WNot_6 . . . 132
8.55 GalileoE1BTelemetryDecoder Class Reference . . . 132
8.55.1 Detailed Description . . . 132
8.55.2 Member Function Documentation . . . 133
8.55.2.1 implementation() . . . 133
8.56 GalileoE1DllPllVemlTracking Class Reference . . . 133
8.56.1 Detailed Description . . . 134
8.56.2 Member Function Documentation . . . 134
8.56.2.1 implementation() . . . 134

8.56.2.2 set_channel(unsigned int channel) . . . 134
8.56.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 134
8.57 GalileoE1Observables Class Reference . . . 134
8.57.1 Detailed Description . . . 135
8.57.2 Member Function Documentation . . . 135
8.57.2.1 implementation() . . . 135
8.57.2.2 item_size() . . . 135
8.58 GalileoE1Pcps8msAmbiguousAcquisition Class Reference . . . 136
8.58.1 Detailed Description . . . 137
8.58.2 Member Function Documentation . . . 137
8.58.2.1 implementation() . . . 137
8.58.2.2 init() . . . 137
8.58.2.3 mag() . . . 137
8.58.2.4 reset() . . . 137
8.58.2.5 set_channel(unsigned int channel) . . . 137
8.58.2.6 set_doppler_max(unsigned int doppler_max) . . . 138
8.58.2.7 set_doppler_step(unsigned int doppler_step) . . . 138
8.58.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 138
8.58.2.9 set_local_code() . . . 138
8.58.2.10 set_threshold(float threshold) . . . 138
8.59 GalileoE1PcpsAmbiguousAcquisition Class Reference . . . 139
8.59.1 Detailed Description . . . 140
8.59.2 Member Function Documentation . . . 140
8.59.2.1 implementation() . . . 140
8.59.2.2 init() . . . 140
8.59.2.3 mag() . . . 140
8.59.2.4 reset() . . . 140
8.59.2.5 set_channel(unsigned int channel) . . . 140
8.59.2.6 set_doppler_max(unsigned int doppler_max) . . . 141
8.59.2.7 set_doppler_step(unsigned int doppler_step) . . . 141

8.59.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 141
8.59.2.9 set_local_code() . . . 141
8.59.2.10 set_state(int state) . . . 141
8.59.2.11 set_threshold(float threshold) . . . 141
8.60 GalileoE1PcpsCccwsrAmbiguousAcquisition Class Reference . . . 142
8.60.1 Detailed Description . . . 143
8.60.2 Member Function Documentation . . . 143
8.60.2.1 implementation() . . . 143
8.60.2.2 init() . . . 143
8.60.2.3 mag() . . . 143
8.60.2.4 reset() . . . 143
8.60.2.5 set_channel(unsigned int channel) . . . 143
8.60.2.6 set_doppler_max(unsigned int doppler_max) . . . 144
8.60.2.7 set_doppler_step(unsigned int doppler_step) . . . 144
8.60.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 144
8.60.2.9 set_state(int state) . . . 144
8.60.2.10 set_threshold(float threshold) . . . 144
8.61 GalileoE1PcpsQuickSyncAmbiguousAcquisition Class Reference . . . 145
8.61.1 Detailed Description . . . 146
8.61.2 Member Function Documentation . . . 146
8.61.2.1 implementation() . . . 146
8.61.2.2 init() . . . 146
8.61.2.3 mag() . . . 146
8.61.2.4 reset() . . . 146
8.61.2.5 set_channel(unsigned int channel) . . . 146
8.61.2.6 set_doppler_max(unsigned int doppler_max) . . . 147
8.61.2.7 set_doppler_step(unsigned int doppler_step) . . . 147
8.61.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 147
8.61.2.9 set_local_code() . . . 147
8.61.2.10 set_state(int state) . . . 147

8.61.2.11 set_threshold(float threshold) . . . 147
8.62 GalileoE1PcpsTongAmbiguousAcquisition Class Reference . . . 148
8.62.1 Detailed Description . . . 149
8.62.2 Member Function Documentation . . . 149
8.62.2.1 implementation() . . . 149
8.62.2.2 init() . . . 149
8.62.2.3 mag() . . . 149
8.62.2.4 reset() . . . 149
8.62.2.5 set_channel(unsigned int channel) . . . 149
8.62.2.6 set_doppler_max(unsigned int doppler_max) . . . 150
8.62.2.7 set_doppler_step(unsigned int doppler_step) . . . 150
8.62.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 150
8.62.2.9 set_local_code() . . . 150
8.62.2.10 set_state(int state) . . . 150
8.62.2.11 set_threshold(float threshold) . . . 150
8.63 GalileoE1Pvt Class Reference . . . 151
8.63.1 Detailed Description . . . 151
8.63.2 Member Function Documentation . . . 151
8.63.2.1 implementation() . . . 151
8.63.2.2 item_size() . . . 152
8.64 GalileoE1TcpConnectorTracking Class Reference . . . 152
8.64.1 Detailed Description . . . 153
8.64.2 Member Function Documentation . . . 153
8.64.2.1 implementation() . . . 153
8.64.2.2 set_channel(unsigned int channel) . . . 153
8.64.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 153
8.65 GalileoE5aDllPllTracking Class Reference . . . 153
8.65.1 Detailed Description . . . 154
8.65.2 Member Function Documentation . . . 154
8.65.2.1 implementation() . . . 154

8.65.2.2 set_channel(unsigned int channel) . . . 154
8.65.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 155
8.66 GalileoE5aNoncoherentIQAcquisitionCaf Class Reference . . . 155
8.66.1 Detailed Description . . . 156
8.66.2 Member Function Documentation . . . 156
8.66.2.1 implementation() . . . 156
8.66.2.2 init() . . . 156
8.66.2.3 mag() . . . 156
8.66.2.4 reset() . . . 156
8.66.2.5 set_channel(unsigned int channel) . . . 157
8.66.2.6 set_doppler_max(unsigned int doppler_max) . . . 157
8.66.2.7 set_doppler_step(unsigned int doppler_step) . . . 157
8.66.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 157
8.66.2.9 set_local_code() . . . 157
8.66.2.10 set_state(int state) . . . 157
8.66.2.11 set_threshold(float threshold) . . . 158
8.67 GalileoE5aObservables Class Reference . . . 158
8.67.1 Detailed Description . . . 159
8.67.2 Member Function Documentation . . . 159
8.67.2.1 implementation() . . . 159
8.67.2.2 item_size() . . . 159
8.68 GalileoE5aTelemetryDecoder Class Reference . . . 159
8.68.1 Detailed Description . . . 160
8.68.2 Member Function Documentation . . . 160
8.68.2.1 implementation() . . . 160
8.69 GenSignalSource Class Reference . . . 160
8.69.1 Detailed Description . . . 161
8.69.2 Constructor & Destructor Documentation . . . 161
8.69.2.1 GenSignalSource(GNSSBlockInterface *signal_generator, GNSSBlockInterface *filter, std::string role, boost::shared_ptr< gr::msg_queue > queue) . . . 161
8.69.2.2 ~GenSignalSource() . . . 161

8.69.3 Member Function Documentation . . . 161
8.69.3.1 implementation() . . . 161
8.70 GeoJSON_Printer Class Reference . . . 162
8.70.1 Detailed Description . . . 162
8.71 Gn3sSignalSource Class Reference . . . 162
8.71.1 Detailed Description . . . 163
8.71.2 Member Function Documentation . . . 163
8.71.2.1 implementation() . . . 163
8.72 Gnss_Satellite Class Reference . . . 163
8.72.1 Detailed Description . . . 164
8.72.2 Constructor & Destructor Documentation . . . 164
8.72.2.1 Gnss_Satellite() . . . 164
8.72.2.2 Gnss_Satellite(const std::string &system_, unsigned int PRN_) . . . 164
8.72.2.3 ~Gnss_Satellite() . . . 164
8.72.3 Member Function Documentation . . . 164
8.72.3.1 get_block() const . . . 164
8.72.3.2 get_PRN() const . . . 164
8.72.3.3 get_system() const . . . 164
8.72.3.4 get_system_short() const . . . 165
8.72.3.5 what_block(const std::string &system_, unsigned int PRN_) . . . 165
8.72.4 Friends And Related Function Documentation . . . 165
8.72.4.1 operator<< . . . 165
8.72.4.2 operator== . . . 165
8.73 gnss_sdr_supl_client Class Reference . . . 165
8.73.1 Detailed Description . . . 166
8.73.2 Member Function Documentation . . . 166
8.73.2.1 load_ephemeris_xml(const std::string file_name) . . . 166
8.73.2.2 load_iono_xml(const std::string file_name) . . . 166
8.73.2.3 load_ref_location_xml(const std::string file_name) . . . 166
8.73.2.4 load_ref_time_xml(const std::string file_name) . . . 166

8.73.2.5 load_utc_xml(const std::string file_name) . . . 166
8.73.2.6 save_ephemeris_map_xml(const std::string file_name, std::map< int, Gps_↩ Ephemeris > eph_map) . . . 166
8.73.2.7 save_iono_map_xml(const std::string file_name, std::map< int, Gps_Iono > iono_map) . . . 167
8.73.2.8 save_ref_location_map_xml(std::string file_name, std::map< int, Gps_Ref_↩ Location > ref_location_map) . . . 167
8.73.2.9 save_ref_time_map_xml(const std::string file_name, std::map< int, Gps_Ref_↩ Time > ref_time_map) . . . 167
8.73.2.10 save_utc_map_xml(const std::string file_name, std::map< int, Gps_Utc_Model > utc_map) . . . 167
8.74 gnss_sdr_valve Class Reference . . . 167
8.74.1 Detailed Description . . . 168
8.75 Gnss_Signal Class Reference . . . 168
8.75.1 Detailed Description . . . 168
8.75.2 Member Function Documentation . . . 168
8.75.2.1 get_satellite() const . . . 168
8.75.2.2 get_signal_str() const . . . 169
8.75.3 Friends And Related Function Documentation . . . 169
8.75.3.1 operator<< . . . 169
8.75.3.2 operator== . . . 169
8.76 Gnss_Synchro Class Reference . . . 169
8.76.1 Detailed Description . . . 170
8.76.2 Member Data Documentation . . . 170
8.76.2.1 Acq_delay_samples . . . 170
8.76.2.2 Acq_doppler_hz . . . 170
8.76.2.3 Acq_samplestamp_samples . . . 171
8.76.2.4 Carrier_Doppler_hz . . . 171
8.76.2.5 Carrier_phase_rads . . . 171
8.76.2.6 Channel_ID . . . 171
8.76.2.7 CN0_dB_hz . . . 171
8.76.2.8 correlation_length_ms . . . 171
8.76.2.9 d_TOW . . . 171

8.76.2.10 Flag_preamble . . . 171
8.76.2.11 Flag_valid_acquisition . . . 172
8.76.2.12 Flag_valid_symbol_output . . . 172
8.76.2.13 Flag_valid_word . . . 172
8.76.2.14 PRN . . . 172
8.76.2.15 Prn_timestamp_at_preamble_ms . . . 172
8.76.2.16 Prn_timestamp_ms . . . 172
8.76.2.17 Prompt_I . . . 172
8.76.2.18 Prompt_Q . . . 172
8.76.2.19 Rem_code_phase_secs . . . 173
8.76.2.20 Signal . . . 173
8.76.2.21 System . . . 173
8.76.2.22 Tracking_timestamp_secs . . . 173
8.77 GNSSBlockFactory Class Reference . . . 173
8.77.1 Detailed Description . . . 174
8.78 GNSSBlockInterface Class Reference . . . 174
8.78.1 Detailed Description . . . 175
8.79 GNSSFlowgraph Class Reference . . . 175
8.79.1 Detailed Description . . . 175
8.79.2 Constructor & Destructor Documentation . . . 176
8.79.2.1 GNSSFlowgraph(std::shared_ptr< ConfigurationInterface > configuration, boost::shared_ptr< gr::msg_queue > queue) . . . 176
8.79.2.2 ~GNSSFlowgraph() . . . 176
8.79.3 Member Function Documentation . . . 176
8.79.3.1 apply_action(unsigned int who, unsigned int what) . . . 176
8.79.3.2 connect() . . . 176
8.79.3.3 send_telemetry_msg(pmt::pmt_t msg) . . . 176
8.79.3.4 start() . . . 176
8.79.3.5 stop() . . . 176
8.80 Gps_Acq_Assist Class Reference . . . 177
8.80.1 Detailed Description . . . 177

8.80.2 Constructor & Destructor Documentation . . . 177
8.80.2.1 Gps_Acq_Assist() . . . 177
8.80.3 Member Data Documentation . . . 178
8.80.3.1 Azimuth . . . 178
8.80.3.2 Code_Phase . . . 178
8.80.3.3 Code_Phase_int . . . 178
8.80.3.4 Code_Phase_window . . . 178
8.80.3.5 d_Doppler0 . . . 178
8.80.3.6 d_Doppler1 . . . 178
8.80.3.7 d_TOW . . . 178
8.80.3.8 dopplerUncertainty . . . 179
8.80.3.9 Elevation . . . 179
8.80.3.10 GPS_Bit_Number . . . 179
8.80.3.11 i_satellite_PRN . . . 179
8.81 Gps_Almanac Class Reference . . . 179
8.81.1 Detailed Description . . . 180
8.81.2 Constructor & Destructor Documentation . . . 180
8.81.2.1 Gps_Almanac() . . . 180
8.81.3 Member Data Documentation . . . 180
8.81.3.1 d_A_f0 . . . 180
8.81.3.2 d_A_f1 . . . 181
8.81.3.3 d_e_eccentricity . . . 181
8.81.3.4 d_M_0 . . . 181
8.81.3.5 d_OMEGA . . . 181
8.81.3.6 d_OMEGA0 . . . 181
8.81.3.7 d_OMEGA_DOT . . . 181
8.81.3.8 d_sqrt_A . . . 181
8.81.3.9 d_Toa . . . 181
8.81.3.10 i_satellite_PRN . . . 182
8.82 Gps_CNAV_Ephemeris Class Reference . . . 182

8.82.1 Detailed Description . . . 184
8.82.2 Constructor & Destructor Documentation . . . 184
8.82.2.1 Gps_CNAV_Ephemeris() . . . 184
8.82.3 Member Function Documentation . . . 185
8.82.3.1 satellitePosition(double transmitTime) . . . 185
8.82.3.2 serialize(Archive &archive, const unsigned int version) . . . 185
8.82.3.3 sv_clock_drift(double transmitTime) . . . 186
8.82.3.4 sv_clock_relativistic_term(double transmitTime) . . . 186
8.82.4 Member Data Documentation . . . 186
8.82.4.1 b_alert_flag . . . 186
8.82.4.2 b_antispoofing_flag . . . 186
8.82.4.3 b_integrity_status_flag . . . 186
8.82.4.4 d_A_DOT . . . 186
8.82.4.5 d_A_f0 . . . 187
8.82.4.6 d_A_f1 . . . 187
8.82.4.7 d_A_f2 . . . 187
8.82.4.8 d_Cic . . . 187
8.82.4.9 d_Cis . . . 187
8.82.4.10 d_Crc . . . 187
8.82.4.11 d_Crs . . . 187
8.82.4.12 d_Cuc . . . 187
8.82.4.13 d_Cus . . . 188
8.82.4.14 d_DELTA_A . . . 188
8.82.4.15 d_DELTA_DOT_N . . . 188
8.82.4.16 d_Delta_n . . . 188
8.82.4.17 d_DELTA_OMEGA_DOT . . . 188
8.82.4.18 d_dtr . . . 188
8.82.4.19 d_e_eccentricity . . . 188
8.82.4.20 d_i_0 . . . 188
8.82.4.21 d_IDOT . . . 189

8.82.4.22 d_M_0 . . . 189
8.82.4.23 d_OMEGA . . . 189
8.82.4.24 d_OMEGA0 . . . 189
8.82.4.25 d_satClkDrift . . . 189
8.82.4.26 d_satpos_X . . . 189
8.82.4.27 d_satpos_Y . . . 189
8.82.4.28 d_satpos_Z . . . 190
8.82.4.29 d_satvel_X . . . 190
8.82.4.30 d_satvel_Y . . . 190
8.82.4.31 d_satvel_Z . . . 190
8.82.4.32 d_TGD . . . 190
8.82.4.33 d_Toc . . . 190
8.82.4.34 d_Toe1 . . . 190
8.82.4.35 d_Toe2 . . . 191
8.82.4.36 d_Top . . . 191
8.82.4.37 d_TOW . . . 191
8.82.4.38 d_URA0 . . . 191
8.82.4.39 d_URA1 . . . 191
8.82.4.40 d_URA2 . . . 191
8.82.4.41 i_GPS_week . . . 191
8.82.4.42 i_signal_health . . . 191
8.82.4.43 i_URA . . . 192
8.83 Gps_CNAV_Iono Class Reference . . . 192
8.83.1 Detailed Description . . . 193
8.83.2 Constructor & Destructor Documentation . . . 193
8.83.2.1 Gps_CNAV_Iono() . . . 193
8.83.3 Member Function Documentation . . . 193
8.83.3.1 serialize(Archive &archive, const unsigned int version) . . . 193
8.83.4 Member Data Documentation . . . 193
8.83.4.1 d_alpha0 . . . 193

8.83.4.2 d_alpha1 . . . 193
8.83.4.3 d_alpha2 . . . 193
8.83.4.4 d_alpha3 . . . 194
8.83.4.5 d_beta0 . . . 194
8.83.4.6 d_beta1 . . . 194
8.83.4.7 d_beta2 . . . 194
8.83.4.8 d_beta3 . . . 194
8.83.4.9 valid . . . 194
8.84 Gps_CNAV_Navigation_Message Class Reference . . . 194
8.84.1 Detailed Description . . . 195
8.84.2 Constructor & Destructor Documentation . . . 196
8.84.2.1 Gps_CNAV_Navigation_Message() . . . 196
8.84.3 Member Function Documentation . . . 196
8.84.3.1 get_ephemeris() . . . 196
8.84.3.2 get_iono() . . . 196
8.84.3.3 get_utc_model() . . . 196
8.84.3.4 have_new_ephemeris() . . . 196
8.84.3.5 have_new_iono() . . . 196
8.84.4 Member Data Documentation . . . 196
8.84.4.1 b_flag_iono_valid . . . 196
8.84.4.2 d_satpos_X . . . 196
8.84.4.3 d_satpos_Y . . . 197
8.84.4.4 d_satpos_Z . . . 197
8.84.4.5 d_satvel_X . . . 197
8.84.4.6 d_satvel_Y . . . 197
8.84.4.7 d_satvel_Z . . . 197
8.84.4.8 satelliteBlock . . . 197
8.85 Gps_CNAV_Utc_Model Class Reference . . . 197
8.85.1 Detailed Description . . . 198
8.85.2 Constructor & Destructor Documentation . . . 198

8.85.2.1 Gps_CNAV_Utc_Model() . . . 198
8.85.3 Member Data Documentation . . . 198
8.85.3.1 d_A0 . . . 198
8.85.3.2 d_A1 . . . 199
8.85.3.3 d_DeltaT_LS . . . 199
8.85.3.4 d_DeltaT_LSF . . . 199
8.85.3.5 d_t_OT . . . 199
8.85.3.6 i_DN . . . 199
8.85.3.7 i_WN_LSF . . . 199
8.85.3.8 i_WN_T . . . 199
8.86 Gps_Ephemeris Class Reference . . . 200
8.86.1 Detailed Description . . . 202
8.86.2 Constructor & Destructor Documentation . . . 202
8.86.2.1 Gps_Ephemeris() . . . 202
8.86.3 Member Function Documentation . . . 202
8.86.3.1 satellitePosition(double transmitTime) . . . 202
8.86.3.2 serialize(Archive &archive, const unsigned int version) . . . 203
8.86.3.3 sv_clock_drift(double transmitTime) . . . 204
8.86.3.4 sv_clock_relativistic_term(double transmitTime) . . . 204
8.86.4 Member Data Documentation . . . 204
8.86.4.1 b_alert_flag . . . 204
8.86.4.2 b_antispoofing_flag . . . 204
8.86.4.3 b_fit_interval_flag . . . 204
8.86.4.4 b_integrity_status_flag . . . 204
8.86.4.5 b_L2_P_data_flag . . . 205
8.86.4.6 d_A_f0 . . . 205
8.86.4.7 d_A_f1 . . . 205
8.86.4.8 d_A_f2 . . . 205
8.86.4.9 d_Cic . . . 205
8.86.4.10 d_Cis . . . 205

8.86.4.11 d_Crc . . . 205
8.86.4.12 d_Crs . . . 205
8.86.4.13 d_Cuc . . . 206
8.86.4.14 d_Cus . . . 206
8.86.4.15 d_Delta_n . . . 206
8.86.4.16 d_dtr . . . 206
8.86.4.17 d_e_eccentricity . . . 206
8.86.4.18 d_i_0 . . . 206
8.86.4.19 d_IDOT . . . 206
8.86.4.20 d_IODC . . . 206
8.86.4.21 d_IODE_SF2 . . . 207
8.86.4.22 d_IODE_SF3 . . . 207
8.86.4.23 d_M_0 . . . 207
8.86.4.24 d_OMEGA . . . 207
8.86.4.25 d_OMEGA0 . . . 207
8.86.4.26 d_OMEGA_DOT . . . 207
8.86.4.27 d_satClkDrift . . . 207
8.86.4.28 d_satpos_X . . . 207
8.86.4.29 d_satpos_Y . . . 208
8.86.4.30 d_satpos_Z . . . 208
8.86.4.31 d_satvel_X . . . 208
8.86.4.32 d_satvel_Y . . . 208
8.86.4.33 d_satvel_Z . . . 208
8.86.4.34 d_sqrt_A . . . 208
8.86.4.35 d_TGD . . . 208
8.86.4.36 d_Toc . . . 209
8.86.4.37 d_Toe . . . 209
8.86.4.38 d_TOW . . . 209
8.86.4.39 i_AODO . . . 209
8.86.4.40 i_code_on_L2 . . . 209

8.86.4.41 i_GPS_week . . . 209
8.86.4.42 i_SV_accuracy . . . 209
8.86.4.43 satelliteBlock . . . 210
8.87 Gps_Iono Class Reference . . . 210
8.87.1 Detailed Description . . . 211
8.87.2 Constructor & Destructor Documentation . . . 211
8.87.2.1 Gps_Iono() . . . 211
8.87.3 Member Function Documentation . . . 211
8.87.3.1 serialize(Archive &archive, const unsigned int version) . . . 211
8.87.4 Member Data Documentation . . . 211
8.87.4.1 d_alpha0 . . . 211
8.87.4.2 d_alpha1 . . . 211
8.87.4.3 d_alpha2 . . . 211
8.87.4.4 d_alpha3 . . . 212
8.87.4.5 d_beta0 . . . 212
8.87.4.6 d_beta1 . . . 212
8.87.4.7 d_beta2 . . . 212
8.87.4.8 d_beta3 . . . 212
8.87.4.9 valid . . . 212
8.88 gps_l1_ca_dll_pll_c_aid_tracking_cc Class Reference . . . 213
8.88.1 Detailed Description . . . 213
8.89 gps_l1_ca_dll_pll_c_aid_tracking_sc Class Reference . . . 213
8.89.1 Detailed Description . . . 214
8.90 Gps_L1_Ca_Dll_Pll_Tracking_cc Class Reference . . . 214
8.90.1 Detailed Description . . . 215
8.91 Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc Class Reference . . . 215
8.91.1 Detailed Description . . . 216
8.92 gps_l1_ca_ls_pvt Class Reference . . . 216
8.92.1 Detailed Description . . . 217
8.92.2 Member Data Documentation . . . 217

8.92.2.1 d_nchannels . . . 217
8.92.2.2 gps_ephemeris_map . . . 217
8.93 gps_l1_ca_observables_cc Class Reference . . . 217
8.93.1 Detailed Description . . . 218
8.94 gps_l1_ca_pvt_cc Class Reference . . . 218
8.94.1 Detailed Description . . . 218
8.94.2 Constructor & Destructor Documentation . . . 219
8.94.2.1 ~gps_l1_ca_pvt_cc() . . . 219
8.94.3 Member Function Documentation . . . 219
8.94.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 219
8.94.3.2 get_GPS_L1_ephemeris_map() . . . 219
8.95 Gps_L1_Ca_Tcp_Connector_Tracking_cc Class Reference . . . 219
8.95.1 Detailed Description . . . 220
8.96 gps_l1_ca_telemetry_decoder_cc Class Reference . . . 220
8.96.1 Detailed Description . . . 221
8.96.2 Member Function Documentation . . . 221
8.96.2.1 forecast(int noutput_items, gr_vector_int &ninput_items_required) . . . 221
8.96.2.2 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 221
8.96.2.3 set_channel(int channel) . . . 221
8.96.2.4 set_decimation(int decimation) . . . 221
8.96.2.5 set_satellite(Gnss_Satellite satellite) . . . 221
8.97 gps_l2_m_dll_pll_tracking_cc Class Reference . . . 221
8.97.1 Detailed Description . . . 222
8.98 gps_l2c_telemetry_decoder_cc Class Reference . . . 222
8.98.1 Detailed Description . . . 223
8.98.2 Member Function Documentation . . . 223
8.98.2.1 forecast(int noutput_items, gr_vector_int &ninput_items_required) . . . 223
8.98.2.2 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 223
8.98.2.3 set_channel(int channel) . . . 223

8.98.2.4 set_satellite(Gnss_Satellite satellite) . . . 223
8.99 Gps_Navigation_Message Class Reference . . . 223
8.99.1 Detailed Description . . . 227
8.99.2 Constructor & Destructor Documentation . . . 227
8.99.2.1 Gps_Navigation_Message() . . . 227
8.99.3 Member Function Documentation . . . 227
8.99.3.1 get_ephemeris() . . . 227
8.99.3.2 get_iono() . . . 227
8.99.3.3 get_utc_model() . . . 227
8.99.3.4 satellitePosition(double transmitTime) . . . 228
8.99.3.5 subframe_decoder(char *subframe) . . . 228
8.99.3.6 sv_clock_correction(double transmitTime) . . . 228
8.99.3.7 utc_time(const double gpstime_corrected) const . . . 228
8.99.4 Member Data Documentation . . . 228
8.99.4.1 almanacHealth . . . 228
8.99.4.2 b_alert_flag . . . 228
8.99.4.3 b_antispoofing_flag . . . 228
8.99.4.4 b_fit_interval_flag . . . 228
8.99.4.5 b_integrity_status_flag . . . 229
8.99.4.6 b_L2_P_data_flag . . . 229
8.99.4.7 d_A0 . . . 229
8.99.4.8 d_A1 . . . 229
8.99.4.9 d_A_f0 . . . 229
8.99.4.10 d_A_f1 . . . 229
8.99.4.11 d_A_f2 . . . 229
8.99.4.12 d_alpha0 . . . 230
8.99.4.13 d_alpha1 . . . 230
8.99.4.14 d_alpha2 . . . 230
8.99.4.15 d_alpha3 . . . 230
8.99.4.16 d_beta0 . . . 230

8.99.4.17 d_beta1 . . . 230
8.99.4.18 d_beta2 . . . 230
8.99.4.19 d_beta3 . . . 230
8.99.4.20 d_Cic . . . 231
8.99.4.21 d_Cis . . . 231
8.99.4.22 d_Crc . . . 231
8.99.4.23 d_Crs . . . 231
8.99.4.24 d_Cuc . . . 231
8.99.4.25 d_Cus . . . 231
8.99.4.26 d_Delta_n . . . 231
8.99.4.27 d_DeltaT_LS . . . 231
8.99.4.28 d_DeltaT_LSF . . . 232
8.99.4.29 d_e_eccentricity . . . 232
8.99.4.30 d_i_0 . . . 232
8.99.4.31 d_IDOT . . . 232
8.99.4.32 d_IODC . . . 232
8.99.4.33 d_M_0 . . . 232
8.99.4.34 d_OMEGA . . . 232
8.99.4.35 d_OMEGA0 . . . 232
8.99.4.36 d_OMEGA_DOT . . . 233
8.99.4.37 d_satpos_X . . . 233
8.99.4.38 d_satpos_Y . . . 233
8.99.4.39 d_satpos_Z . . . 233
8.99.4.40 d_satvel_X . . . 233
8.99.4.41 d_satvel_Y . . . 233
8.99.4.42 d_satvel_Z . . . 233
8.99.4.43 d_sqrt_A . . . 234
8.99.4.44 d_t_OT . . . 234
8.99.4.45 d_TGD . . . 234
8.99.4.46 d_Toa . . . 234

8.99.4.47 d_Toc . . . 234
8.99.4.48 d_Toe . . . 234
8.99.4.49 d_TOW . . . 234
8.99.4.50 d_TOW_SF1 . . . 234
8.99.4.51 d_TOW_SF2 . . . 235
8.99.4.52 d_TOW_SF3 . . . 235
8.99.4.53 d_TOW_SF4 . . . 235
8.99.4.54 d_TOW_SF5 . . . 235
8.99.4.55 flag_iono_valid . . . 235
8.99.4.56 flag_utc_model_valid . . . 235
8.99.4.57 i_AODO . . . 235
8.99.4.58 i_code_on_L2 . . . 236
8.99.4.59 i_DN . . . 236
8.99.4.60 i_GPS_week . . . 236
8.99.4.61 i_SV_accuracy . . . 236
8.99.4.62 i_WN_A . . . 236
8.99.4.63 i_WN_LSF . . . 236
8.99.4.64 i_WN_T . . . 236
8.99.4.65 satelliteBlock . . . 237
8.100 Gps_Ref_Location Class Reference . . . 237
8.100.1 Detailed Description . . . 237
8.100.2 Constructor & Destructor Documentation . . . 237
8.100.2.1 Gps_Ref_Location() . . . 237
8.100.3 Member Function Documentation . . . 238
8.100.3.1 serialize(Archive &archive, const unsigned int version) . . . 238
8.101 Gps_Ref_Time Class Reference . . . 238
8.101.1 Detailed Description . . . 238
8.101.2 Constructor & Destructor Documentation . . . 238
8.101.2.1 Gps_Ref_Time() . . . 238
8.101.3 Member Function Documentation . . . 239

8.101.3.1 serialize(Archive &archive, const unsigned int version) . . . 239
8.102 Gps_Utc_Model Class Reference . . . 239
8.102.1 Detailed Description . . . 240
8.102.2 Constructor & Destructor Documentation . . . 240
8.102.2.1 Gps_Utc_Model() . . . 240
8.102.3 Member Function Documentation . . . 240
8.102.3.1 utc_time(double gpstime_corrected, int i_GPS_week) . . . 240
8.102.4 Member Data Documentation . . . 240
8.102.4.1 d_A0 . . . 240
8.102.4.2 d_A1 . . . 240
8.102.4.3 d_DeltaT_LS . . . 240
8.102.4.4 d_DeltaT_LSF . . . 240
8.102.4.5 d_t_OT . . . 241
8.102.4.6 i_DN . . . 241
8.102.4.7 i_WN_LSF . . . 241
8.102.4.8 i_WN_T . . . 241
8.103 GpsL1CaDllPllCAidTracking Class Reference . . . 241
8.103.1 Detailed Description . . . 242
8.103.2 Member Function Documentation . . . 242
8.103.2.1 implementation() . . . 242
8.103.2.2 set_channel(unsigned int channel) . . . 242
8.103.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 243
8.104 GpsL1CaDllPllTracking Class Reference . . . 243
8.104.1 Detailed Description . . . 244
8.104.2 Member Function Documentation . . . 244
8.104.2.1 implementation() . . . 244
8.104.2.2 set_channel(unsigned int channel) . . . 244
8.104.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 244
8.105 GpsL1CaDllPllTrackingGPU Class Reference . . . 244
8.105.1 Detailed Description . . . 245

8.105.2 Member Function Documentation . . . 245
8.105.2.1 implementation() . . . 245
8.105.2.2 set_channel(unsigned int channel) . . . 245
8.105.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 246
8.106 GpsL1CaObservables Class Reference . . . 246
8.106.1 Detailed Description . . . 246
8.106.2 Member Function Documentation . . . 247
8.106.2.1 implementation() . . . 247
8.106.2.2 item_size() . . . 247
8.107 GpsL1CaPcpsAcquisition Class Reference . . . 247
8.107.1 Detailed Description . . . 248
8.107.2 Member Function Documentation . . . 248
8.107.2.1 implementation() . . . 248
8.107.2.2 init() . . . 249
8.107.2.3 mag() . . . 249
8.107.2.4 reset() . . . 249
8.107.2.5 set_channel(unsigned int channel) . . . 249
8.107.2.6 set_doppler_max(unsigned int doppler_max) . . . 249
8.107.2.7 set_doppler_step(unsigned int doppler_step) . . . 249
8.107.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 250
8.107.2.9 set_local_code() . . . 250
8.107.2.10 set_state(int state) . . . 250
8.107.2.11 set_threshold(float threshold) . . . 250
8.108 GpsL1CaPcpsAcquisitionFineDoppler Class Reference . . . 250
8.108.1 Detailed Description . . . 251
8.108.2 Member Function Documentation . . . 251
8.108.2.1 implementation() . . . 251
8.108.2.2 init() . . . 252
8.108.2.3 mag() . . . 252
8.108.2.4 reset() . . . 252

8.108.2.5 set_channel(unsigned int channel) . . . 252
8.108.2.6 set_doppler_max(unsigned int doppler_max) . . . 252
8.108.2.7 set_doppler_step(unsigned int doppler_step) . . . 252
8.108.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 253
8.108.2.9 set_threshold(float threshold) . . . 253
8.109 GpsL1CaPcpsAssistedAcquisition Class Reference . . . 253
8.109.1 Detailed Description . . . 254
8.109.2 Member Function Documentation . . . 254
8.109.2.1 implementation() . . . 254
8.109.2.2 init() . . . 255
8.109.2.3 mag() . . . 255
8.109.2.4 reset() . . . 255
8.109.2.5 set_channel(unsigned int channel) . . . 255
8.109.2.6 set_doppler_max(unsigned int doppler_max) . . . 255
8.109.2.7 set_doppler_step(unsigned int doppler_step) . . . 255
8.109.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 256
8.109.2.9 set_threshold(float threshold) . . . 256
8.110 GpsL1CaPcpsMultithreadAcquisition Class Reference . . . 256
8.110.1 Detailed Description . . . 257
8.110.2 Member Function Documentation . . . 257
8.110.2.1 implementation() . . . 257
8.110.2.2 init() . . . 257
8.110.2.3 mag() . . . 258
8.110.2.4 reset() . . . 258
8.110.2.5 set_channel(unsigned int channel) . . . 258
8.110.2.6 set_doppler_max(unsigned int doppler_max) . . . 258
8.110.2.7 set_doppler_step(unsigned int doppler_step) . . . 258
8.110.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 258
8.110.2.9 set_local_code() . . . 259
8.110.2.10 set_threshold(float threshold) . . . 259

8.111 GpsL1CaPcpsOpenClAcquisition Class Reference . . . 259
8.111.1 Detailed Description . . . 260
8.111.2 Member Function Documentation . . . 260
8.111.2.1 implementation() . . . 260
8.111.2.2 init() . . . 260
8.111.2.3 mag() . . . 261
8.111.2.4 reset() . . . 261
8.111.2.5 set_channel(unsigned int channel) . . . 261
8.111.2.6 set_doppler_max(unsigned int doppler_max) . . . 261
8.111.2.7 set_doppler_step(unsigned int doppler_step) . . . 261
8.111.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 261
8.111.2.9 set_local_code() . . . 262
8.111.2.10 set_threshold(float threshold) . . . 262
8.112 GpsL1CaPcpsQuickSyncAcquisition Class Reference . . . 262
8.112.1 Detailed Description . . . 263
8.112.2 Member Function Documentation . . . 263
8.112.2.1 implementation() . . . 263
8.112.2.2 init() . . . 263
8.112.2.3 mag() . . . 264
8.112.2.4 reset() . . . 264
8.112.2.5 set_channel(unsigned int channel) . . . 264
8.112.2.6 set_doppler_max(unsigned int doppler_max) . . . 264
8.112.2.7 set_doppler_step(unsigned int doppler_step) . . . 264
8.112.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 264
8.112.2.9 set_local_code() . . . 265
8.112.2.10 set_state(int state) . . . 265
8.112.2.11 set_threshold(float threshold) . . . 265
8.113 GpsL1CaPcpsTongAcquisition Class Reference . . . 265
8.113.1 Detailed Description . . . 266
8.113.2 Member Function Documentation . . . 266

8.113.2.1 implementation() . . . 266
8.113.2.2 init() . . . 267
8.113.2.3 mag() . . . 267
8.113.2.4 reset() . . . 267
8.113.2.5 set_channel(unsigned int channel) . . . 267
8.113.2.6 set_doppler_max(unsigned int doppler_max) . . . 267
8.113.2.7 set_doppler_step(unsigned int doppler_step) . . . 267
8.113.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 268
8.113.2.9 set_local_code() . . . 268
8.113.2.10 set_state(int state) . . . 268
8.113.2.11 set_threshold(float threshold) . . . 268
8.114 GpsL1CaPvt Class Reference . . . 268
8.114.1 Detailed Description . . . 269
8.114.2 Member Function Documentation . . . 269
8.114.2.1 implementation() . . . 269
8.114.2.2 item_size() . . . 269
8.115 GpsL1CaSubframeFsm Class Reference . . . 270
8.115.1 Detailed Description . . . 271
8.115.2 Constructor & Destructor Documentation . . . 271
8.115.2.1 GpsL1CaSubframeFsm() . . . 271
8.115.3 Member Function Documentation . . . 271
8.115.3.1 Event_gps_word_invalid() . . . 271
8.115.3.2 Event_gps_word_preamble() . . . 271
8.115.3.3 Event_gps_word_valid() . . . 271
8.115.3.4 gps_subframe_to_nav_msg() . . . 271
8.115.3.5 gps_word_to_subframe(int position) . . . 271
8.115.4 Member Data Documentation . . . 271
8.115.4.1 almanac . . . 271
8.115.4.2 d_nav . . . 272
8.115.4.3 ephemeris . . . 272

8.115.4.4 i_channel_ID . . . 272
8.115.4.5 i_satellite_PRN . . . 272
8.115.4.6 iono . . . 272
8.115.4.7 utc_model . . . 272
8.116 GpsL1CaTcpConnectorTracking Class Reference . . . 273
8.116.1 Detailed Description . . . 273
8.116.2 Member Function Documentation . . . 273
8.116.2.1 implementation() . . . 273
8.116.2.2 set_channel(unsigned int channel) . . . 274
8.116.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 274
8.117 GpsL1CaTelemetryDecoder Class Reference . . . 274
8.117.1 Detailed Description . . . 275
8.117.2 Member Function Documentation . . . 275
8.117.2.1 implementation() . . . 275
8.118 GpsL2CObservables Class Reference . . . 275
8.118.1 Detailed Description . . . 276
8.118.2 Member Function Documentation . . . 276
8.118.2.1 implementation() . . . 276
8.118.2.2 item_size() . . . 276
8.119 GpsL2CTelemetryDecoder Class Reference . . . 276
8.119.1 Detailed Description . . . 277
8.119.2 Member Function Documentation . . . 277
8.119.2.1 implementation() . . . 277
8.120 GpsL2MDllPllTracking Class Reference . . . 277
8.120.1 Detailed Description . . . 278
8.120.2 Member Function Documentation . . . 278
8.120.2.1 implementation() . . . 278
8.120.2.2 set_channel(unsigned int channel) . . . 278
8.120.2.3 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 279
8.121 GpsL2MPcpsAcquisition Class Reference . . . 279

8.121.1 Detailed Description . . . 280
8.121.2 Member Function Documentation . . . 280
8.121.2.1 implementation() . . . 280
8.121.2.2 init() . . . 280
8.121.2.3 mag() . . . 280
8.121.2.4 reset() . . . 281
8.121.2.5 set_channel(unsigned int channel) . . . 281
8.121.2.6 set_doppler_max(unsigned int doppler_max) . . . 281
8.121.2.7 set_doppler_step(unsigned int doppler_step) . . . 281
8.121.2.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 281
8.121.2.9 set_local_code() . . . 281
8.121.2.10 set_state(int state) . . . 282
8.121.2.11 set_threshold(float threshold) . . . 282
8.122 GPU_Complex Struct Reference . . . 282
8.122.1 Detailed Description . . . 282
8.123 GPU_Complex_Short Struct Reference . . . 283
8.123.1 Detailed Description . . . 283
8.124 hybrid_ls_pvt Class Reference . . . 283
8.124.1 Detailed Description . . . 284
8.124.2 Member Data Documentation . . . 284
8.124.2.1 d_nchannels . . . 284
8.124.2.2 galileo_ephemeris_map . . . 284
8.124.2.3 gps_ephemeris_map . . . 284
8.125 hybrid_observables_cc Class Reference . . . 285
8.125.1 Detailed Description . . . 285
8.126 hybrid_pvt_cc Class Reference . . . 285
8.126.1 Detailed Description . . . 286
8.126.2 Constructor & Destructor Documentation . . . 286
8.126.2.1 ~hybrid_pvt_cc() . . . 286
8.126.3 Member Function Documentation . . . 286

8.126.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) 286

8.126.3.2 get_GPS_L1_ephemeris_map() 286

8.127 HybridObservables Class Reference 287

8.127.1 Detailed Description 287

8.127.2 Member Function Documentation 287

8.127.2.1 implementation() 287

8.127.2.2 item_size() 288

8.128 HybridPvt Class Reference 288

8.128.1 Detailed Description 288

8.128.2 Member Function Documentation 289

8.128.2.1 implementation() 289

8.128.2.2 item_size() 289

8.129 lbyteToCbyte Class Reference 289

8.129.1 Detailed Description 290

8.129.2 Member Function Documentation 290

8.129.2.1 implementation() 290

8.130 lbyteToComplex Class Reference 290

8.130.1 Detailed Description 291

8.130.2 Member Function Documentation 291

8.130.2.1 implementation() 291

8.131 lbyteToCshort Class Reference 291

8.131.1 Detailed Description 292

8.131.2 Member Function Documentation 292

8.131.2.1 implementation() 292

8.132 lgp Struct Reference 292

8.132.1 Detailed Description 292

8.133 lgp_Band Struct Reference 293

8.133.1 Detailed Description 293

8.134 INIReader Class Reference 293

8.134.1 Detailed Description 293

8.134.2 Constructor & Destructor Documentation . . . 294
8.134.2.1 INIReader(std::string filename) . . . 294
8.134.3 Member Function Documentation . . . 294
8.134.3.1 Get(std::string section, std::string name, std::string default_value) . . . 294
8.134.3.2 GetInteger(std::string section, std::string name, long default_value) . . . 294
8.134.3.3 ParseError() . . . 294
8.135 InMemoryConfiguration Class Reference . . . 294
8.135.1 Detailed Description . . . 295
8.136 interleaved_byte_to_complex_byte Class Reference . . . 295
8.136.1 Detailed Description . . . 295
8.137 interleaved_byte_to_complex_short Class Reference . . . 296
8.137.1 Detailed Description . . . 296
8.138 interleaved_short_to_complex_short Class Reference . . . 296
8.138.1 Detailed Description . . . 297
8.139 IshortToComplex Class Reference . . . 297
8.139.1 Detailed Description . . . 298
8.139.2 Member Function Documentation . . . 298
8.139.2.1 implementation() . . . 298
8.140 IshortToCshort Class Reference . . . 298
8.140.1 Detailed Description . . . 299
8.140.2 Member Function Documentation . . . 299
8.140.2.1 implementation() . . . 299
8.141 kernel_info_t Struct Reference . . . 299
8.141.1 Detailed Description . . . 299
8.142 Kml_Printer Class Reference . . . 299
8.142.1 Detailed Description . . . 300
8.143 Kml_Printer_gpstk Class Reference . . . 300
8.143.1 Detailed Description . . . 300
8.144 Long_Term_Correction Struct Reference . . . 301
8.144.1 Detailed Description . . . 301

8.144.2 Member Data Documentation . . . 301
8.144.2.1 d_daf0 . . . 301
8.144.2.2 d_daf1 . . . 301
8.144.2.3 d_dpos . . . 301
8.144.2.4 d_dvel . . . 302
8.144.2.5 d_trx . . . 302
8.144.2.6 i_tapp . . . 302
8.144.2.7 i_vel . . . 302
8.145 Ls_Pvt Class Reference . . . 302
8.145.1 Detailed Description . . . 303
8.145.2 Member Function Documentation . . . 303
8.145.2.1 bancroftPos(const arma::mat &satpos, const arma::vec &obs) . . . 303
8.145.2.2 leastSquarePos(const arma::mat &satpos, const arma::vec &obs, const arma::vec &w_vec) . . . 303
8.146 Nmea_Printer Class Reference . . . 303
8.146.1 Detailed Description . . . 304
8.146.2 Constructor & Destructor Documentation . . . 304
8.146.2.1 Nmea_Printer(std::string filename, bool flag_nmea_tty_port, std::string nmea_dump_filename) . . . 304
8.146.2.2 ~Nmea_Printer() . . . 304
8.146.3 Member Function Documentation . . . 304
8.146.3.1 Print_Nmea_Line(const std::shared_ptr< Pvt_Solution > &position, bool print_average_values) . . . 304
8.147 NsrFileSignalSource Class Reference . . . 304
8.147.1 Detailed Description . . . 305
8.147.2 Member Function Documentation . . . 305
8.147.2.1 implementation() . . . 305
8.148 ObservablesInterface Class Reference . . . 305
8.148.1 Detailed Description . . . 306
8.149 OsmosdrSignalSource Class Reference . . . 306
8.149.1 Detailed Description . . . 307
8.149.2 Member Function Documentation . . . 307

8.149.2.1 implementation() . . . 307
8.150 Pass_Through Class Reference . . . 307
8.150.1 Detailed Description . . . 308
8.150.2 Member Function Documentation . . . 308
8.150.2.1 implementation() . . . 308
8.151 pcps_acquisition_cc Class Reference . . . 308
8.151.1 Detailed Description . . . 309
8.151.2 Constructor & Destructor Documentation . . . 309
8.151.2.1 ∼pcps_acquisition_cc() . . . 309
8.151.3 Member Function Documentation . . . 309
8.151.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 309
8.151.3.2 init() . . . 310
8.151.3.3 mag() . . . 310
8.151.3.4 set_active(bool active) . . . 310
8.151.3.5 set_channel(unsigned int channel) . . . 310
8.151.3.6 set_doppler_max(unsigned int doppler_max) . . . 310
8.151.3.7 set_doppler_step(unsigned int doppler_step) . . . 311
8.151.3.8 set_gnss_synchro(Gnss_Synchro ∗p_gnss_synchro) . . . 311
8.151.3.9 set_local_code(std::complex< float > ∗code) . . . 311
8.151.3.10 set_state(int state) . . . 311
8.151.3.11 set_threshold(float threshold) . . . 311
8.152 pcps_acquisition_fine_doppler_cc Class Reference . . . 312
8.152.1 Detailed Description . . . 313
8.152.2 Constructor & Destructor Documentation . . . 313
8.152.2.1 ∼pcps_acquisition_fine_doppler_cc() . . . 313
8.152.3 Member Function Documentation . . . 313
8.152.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const↩ _void_star &input_items, gr_vector_void_star &output_items) . . . 313
8.152.3.2 init() . . . 313
8.152.3.3 mag() . . . 313

8.152.3.4 set_active(bool active) . . . 313
8.152.3.5 set_channel(unsigned int channel) . . . 314
8.152.3.6 set_doppler_max(unsigned int doppler_max) . . . 314
8.152.3.7 set_doppler_step(unsigned int doppler_step) . . . 314
8.152.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 314
8.152.3.9 set_local_code(std::complex< float > *code) . . . 315
8.152.3.10 set_threshold(float threshold) . . . 315
8.153 pcps_acquisition_sc Class Reference . . . 315
8.153.1 Detailed Description . . . 316
8.153.2 Constructor & Destructor Documentation . . . 316
8.153.2.1 ~pcps_acquisition_sc() . . . 316
8.153.3 Member Function Documentation . . . 317
8.153.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 317
8.153.3.2 init() . . . 317
8.153.3.3 mag() . . . 317
8.153.3.4 set_active(bool active) . . . 317
8.153.3.5 set_channel(unsigned int channel) . . . 317
8.153.3.6 set_doppler_max(unsigned int doppler_max) . . . 318
8.153.3.7 set_doppler_step(unsigned int doppler_step) . . . 318
8.153.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 318
8.153.3.9 set_local_code(std::complex< float > *code) . . . 318
8.153.3.10 set_state(int state) . . . 318
8.153.3.11 set_threshold(float threshold) . . . 319
8.154 pcps_assisted_acquisition_cc Class Reference . . . 319
8.154.1 Detailed Description . . . 320
8.154.2 Constructor & Destructor Documentation . . . 320
8.154.2.1 ~pcps_assisted_acquisition_cc() . . . 320
8.154.3 Member Function Documentation . . . 320
8.154.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 320

8.154.3.2 init() . . . 321
8.154.3.3 mag() . . . 321
8.154.3.4 set_active(bool active) . . . 321
8.154.3.5 set_channel(unsigned int channel) . . . 321
8.154.3.6 set_doppler_max(unsigned int doppler_max) . . . 321
8.154.3.7 set_doppler_step(unsigned int doppler_step) . . . 322
8.154.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 322
8.154.3.9 set_local_code(std::complex< float > *code) . . . 322
8.154.3.10 set_threshold(float threshold) . . . 322
8.155 pcps_cccwsr_acquisition_cc Class Reference . . . 323
8.155.1 Detailed Description . . . 324
8.155.2 Constructor & Destructor Documentation . . . 324
8.155.2.1 ~pcps_cccwsr_acquisition_cc() . . . 324
8.155.3 Member Function Documentation . . . 324
8.155.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 324
8.155.3.2 init() . . . 324
8.155.3.3 mag() . . . 324
8.155.3.4 set_active(bool active) . . . 324
8.155.3.5 set_channel(unsigned int channel) . . . 325
8.155.3.6 set_doppler_max(unsigned int doppler_max) . . . 325
8.155.3.7 set_doppler_step(unsigned int doppler_step) . . . 325
8.155.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 325
8.155.3.9 set_local_code(std::complex< float > *code_data, std::complex< float > *code_pilot) . . . 326
8.155.3.10 set_state(int state) . . . 326
8.155.3.11 set_threshold(float threshold) . . . 326
8.156 pcps_multithread_acquisition_cc Class Reference . . . 326
8.156.1 Detailed Description . . . 327
8.156.2 Constructor & Destructor Documentation . . . 327
8.156.2.1 ~pcps_multithread_acquisition_cc() . . . 327

8.156.3 Member Function Documentation . . . 328
8.156.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 328
8.156.3.2 init() . . . 328
8.156.3.3 mag() . . . 328
8.156.3.4 set_active(bool active) . . . 328
8.156.3.5 set_channel(unsigned int channel) . . . 328
8.156.3.6 set_doppler_max(unsigned int doppler_max) . . . 329
8.156.3.7 set_doppler_step(unsigned int doppler_step) . . . 329
8.156.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 329
8.156.3.9 set_local_code(std::complex< float > *code) . . . 329
8.156.3.10 set_state(int state) . . . 329
8.156.3.11 set_threshold(float threshold) . . . 330
8.157 pcps_opencl_acquisition_cc Class Reference . . . 330
8.157.1 Detailed Description . . . 331
8.157.2 Constructor & Destructor Documentation . . . 331
8.157.2.1 ~pcps_opencl_acquisition_cc() . . . 331
8.157.3 Member Function Documentation . . . 331
8.157.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 331
8.157.3.2 init() . . . 332
8.157.3.3 mag() . . . 332
8.157.3.4 set_active(bool active) . . . 332
8.157.3.5 set_channel(unsigned int channel) . . . 332
8.157.3.6 set_doppler_max(unsigned int doppler_max) . . . 332
8.157.3.7 set_doppler_step(unsigned int doppler_step) . . . 333
8.157.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 333
8.157.3.9 set_local_code(std::complex< float > *code) . . . 333
8.157.3.10 set_state(int state) . . . 333
8.157.3.11 set_threshold(float threshold) . . . 333
8.158 pcps_quicksync_acquisition_cc Class Reference . . . 334

8.158.1 Detailed Description . . . 335
8.158.2 Constructor & Destructor Documentation . . . 335
8.158.2.1 ~pcps_quicksync_acquisition_cc() . . . 335
8.158.3 Member Function Documentation . . . 335
8.158.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 335
8.158.3.2 init() . . . 335
8.158.3.3 mag() . . . 335
8.158.3.4 set_active(bool active) . . . 335
8.158.3.5 set_channel(unsigned int channel) . . . 336
8.158.3.6 set_doppler_max(unsigned int doppler_max) . . . 336
8.158.3.7 set_doppler_step(unsigned int doppler_step) . . . 336
8.158.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 336
8.158.3.9 set_local_code(std::complex< float > *code) . . . 337
8.158.3.10 set_state(int state) . . . 337
8.158.3.11 set_threshold(float threshold) . . . 337
8.159 pcps_tong_acquisition_cc Class Reference . . . 337
8.159.1 Detailed Description . . . 338
8.159.2 Constructor & Destructor Documentation . . . 339
8.159.2.1 ~pcps_tong_acquisition_cc() . . . 339
8.159.3 Member Function Documentation . . . 339
8.159.3.1 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 339
8.159.3.2 init() . . . 339
8.159.3.3 mag() . . . 339
8.159.3.4 set_active(bool active) . . . 339
8.159.3.5 set_channel(unsigned int channel) . . . 339
8.159.3.6 set_doppler_max(unsigned int doppler_max) . . . 340
8.159.3.7 set_doppler_step(unsigned int doppler_step) . . . 340
8.159.3.8 set_gnss_synchro(Gnss_Synchro *p_gnss_synchro) . . . 340
8.159.3.9 set_local_code(std::complex< float > *code) . . . 340

8.159.3.10 set_state(int state) . . . 341
8.159.3.11 set_threshold(float threshold) . . . 341
8.160 Pvt_Solution Class Reference . . . 341
8.160.1 Detailed Description . . . 343
8.160.2 Member Function Documentation . . . 343
8.160.2.1 cart2geo(double X, double Y, double Z, int elipsoid_selection) . . . 343
8.160.2.2 compute_DOP() . . . 343
8.160.2.3 togeod(double *dphi, double *dlambda, double *h, double a, double finv, double X, double Y, double Z) . . . 343
8.160.2.4 topocent(double *Az, double *El, double *D, const arma::vec &x, const arma::vec &dx) . . . 344
8.160.2.5 tropo(double *ddr_m, double sinel, double hsta_km, double p_mb, double t_kel, double hum, double hp_km, double htkel_km, double hhum_km) . . . 344
8.160.3 Member Data Documentation . . . 345
8.160.3.1 d_averaging_depth . . . 345
8.160.3.2 d_avg_height_m . . . 345
8.160.3.3 d_avg_latitude_d . . . 345
8.160.3.4 d_avg_longitude_d . . . 345
8.160.3.5 d_height_m . . . 345
8.160.3.6 d_latitude_d . . . 345
8.160.3.7 d_longitude_d . . . 345
8.160.3.8 d_rx_dt_s . . . 346
8.160.3.9 d_valid_observations . . . 346
8.160.3.10 d_visible_satellites_Az . . . 346
8.160.3.11 d_visible_satellites_CN0_dB . . . 346
8.160.3.12 d_visible_satellites_Distance . . . 346
8.160.3.13 d_visible_satellites_El . . . 346
8.160.3.14 d_visible_satellites_IDs . . . 346
8.161 PvtInterface Class Reference . . . 347
8.161.1 Detailed Description . . . 347
8.162 RawArraySignalSource Class Reference . . . 347
8.162.1 Detailed Description . . . 348

8.162.2 Member Function Documentation . . . 348
8.162.2.1 implementation() . . . 348
8.163 Rinex_Printer Class Reference . . . 348
8.163.1 Detailed Description . . . 351
8.163.2 Constructor & Destructor Documentation . . . 351
8.163.2.1 Rinex_Printer(int version=0) . . . 351
8.163.2.2 ~Rinex_Printer() . . . 351
8.163.3 Member Function Documentation . . . 351
8.163.3.1 compute_Galileo_time(const Galileo_Ephemeris &eph, const double obs_time) . . . 351
8.163.3.2 compute_GPS_time(const Gps_Ephemeris &eph, const double obs_time) . . . 351
8.163.3.3 compute_GPS_time(const Gps_CNAV_Ephemeris &eph, const double obs_time) 351
8.163.3.4 compute_UTC_time(const Gps_Navigation_Message &nav_msg) . . . 351
8.163.3.5 log_rinex_nav(std::fstream &out, const std::map< int, Gps_Ephemeris > &eph↩ _map) . . . 351
8.163.3.6 log_rinex_nav(std::fstream &out, const std::map< int, Gps_CNAV_Ephemeris > &eph_map) . . . 352
8.163.3.7 log_rinex_nav(std::fstream &out, const std::map< int, Galileo_Ephemeris > &eph_map) . . . 352
8.163.3.8 log_rinex_nav(std::fstream &out, const std::map< int, Gps_Ephemeris > &gps↩ _eph_map, const std::map< int, Galileo_Ephemeris > &galileo_eph_map) . . . 352
8.163.3.9 log_rinex_obs(std::fstream &out, const Gps_Ephemeris &eph, double obs_time, const std::map< int, Gnss_Synchro > &observables) . . . 352
8.163.3.10 log_rinex_obs(std::fstream &out, const Gps_CNAV_Ephemeris &eph, double obs_time, const std::map< int, Gnss_Synchro > &observables) . . . 352
8.163.3.11 log_rinex_obs(std::fstream &out, const Gps_Ephemeris &eph, const Gps_CNA↩ V_Ephemeris &eph_cnav, double obs_time, const std::map< int, Gnss_Synchro > &observables) . . . 352
8.163.3.12 log_rinex_obs(std::fstream &out, const Galileo_Ephemeris &eph, double obs↩ _time, const std::map< int, Gnss_Synchro > &observables, const std::string galileo_bands=""1B"") . . . 352
8.163.3.13 log_rinex_obs(std::fstream &out, const Gps_Ephemeris &gps_eph, const Galileo_Ephemeris &galileo_eph, const double gps_obs_time, const std::map< int, Gnss_Synchro > &observables) . . . 352
8.163.3.14 log_rinex_sbs(std::fstream &out, const Sbas_Raw_Msg &sbs_message) . . . 352
8.163.3.15 rinex_nav_header(std::fstream &out, const Gps_Iono &iono, const Gps_Utc_↩ Model &utc_model) . . . 353

8.163.3.16 rinex_nav_header(std::fstream &out, const Gps_CNAV_Iono &iono, const Gps_CNAV_Utc_Model &utc_model) . . . 353

8.163.3.17 rinex_nav_header(std::fstream &out, const Galileo_Iono &iono, const Galileo_Utc_Model &utc_model, const Galileo_Almanac &galileo_almanac) . . . 353

8.163.3.18 rinex_nav_header(std::fstream &out, const Gps_Iono &gps_iono, const Gps_Utc_Model &gps_utc_model, const Galileo_Iono &galileo_iono, const Galileo_Utc_Model &galileo_utc_model, const Galileo_Almanac &galileo_almanac) . . . 353

8.163.3.19 rinex_obs_header(std::fstream &out, const Gps_Ephemeris &eph, const double d_TOW_first_observation) . . . 353

8.163.3.20 rinex_obs_header(std::fstream &out, const Gps_CNAV_Ephemeris &eph, const double d_TOW_first_observation) . . . 353

8.163.3.21 rinex_obs_header(std::fstream &out, const Gps_Ephemeris &eph, const Gps_CNAV_Ephemeris &eph_cnav, const double d_TOW_first_observation) . . . 353

8.163.3.22 rinex_obs_header(std::fstream &out, const Galileo_Ephemeris &eph, const double d_TOW_first_observation, const std::string bands=""1B"") . . . 353

8.163.3.23 rinex_obs_header(std::fstream &out, const Gps_Ephemeris &gps_eph, const Galileo_Ephemeris &galileo_eph, const double d_TOW_first_observation, const std::string galileo_bands=""1B"") . . . 354

8.163.3.24 rinex_sbs_header(std::fstream &out) . . . 354

8.163.3.25 to_date_time(int gps_week, int gps_tow, int &year, int &month, int &day, int &hour, int &minute, int &second) . . . 354

8.164 Rtcm Class Reference . . . 354

8.164.1 Detailed Description . . . 356

8.164.2 Member Function Documentation . . . 356

8.164.2.1 print_MSM_1(const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages) . . . 356

8.164.2.2 print_MSM_2(const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages) . . . 357

8.164.2.3 print_MSM_3(const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages) . . . 357

8.164.2.4 print_MSM_4(const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages) . . . 357

8.164.2.5 print_MSM_5(const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages) . . . 357

8.164.2.6 print_MSM_6(const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages) . . . 357

8.164.2.7 print_MSM_7(const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages) . . . 357

8.164.2.8 print_MT1001(const Gps_Ephemeris &gps_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id) . . . 357

8.164.2.9 print_MT1002(const Gps_Ephemeris &gps_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id) . . . 358

8.164.2.10 print_MT1003(const Gps_Ephemeris &ephL1, const Gps_CNAV_Ephemeris &ephL2, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id) . . . 358

8.164.2.11 print_MT1004(const Gps_Ephemeris &ephL1, const Gps_CNAV_Ephemeris &ephL2, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id) . . . 358

8.164.2.12 print_MT1005(unsigned int ref_id, double ecef_x, double ecef_y, double ecef_z, bool gps, bool glonass, bool galileo, bool non_physical, bool single_oscillator, unsigned int quarter_cycle_indicator) . . . 358

8.164.2.13 print_MT1006(unsigned int ref_id, double ecef_x, double ecef_y, double ecef_z, bool gps, bool glonass, bool galileo, bool non_physical, bool single_oscillator, unsigned int quarter_cycle_indicator, double height) . . . 358

8.164.2.14 print_MT1008(unsigned int ref_id, const std::string &antenna_descriptor, unsigned int antenna_setup_id, const std::string &antenna_serial_number) . . . 358

8.164.2.15 print_MT1019(const Gps_Ephemeris &gps_eph) . . . 358

8.164.2.16 print_MT1029(unsigned int ref_id, const Gps_Ephemeris &gps_eph, double obs_time, const std::string &message) . . . 358

8.164.2.17 print_MT1045(const Galileo_Ephemeris &gal_eph) . . . 358

8.164.2.18 read_MT1005(const std::string &message, unsigned int &ref_id, double &ecef_x, double &ecef_y, double &ecef_z, bool &gps, bool &glonass, bool &galileo) . . . 359

8.164.2.19 read_MT1019(const std::string &message, Gps_Ephemeris &gps_eph) . . . 359

8.164.2.20 read_MT1045(const std::string &message, Galileo_Ephemeris &gal_eph) . . . 359

8.165 Rtcm_Printer Class Reference . . . 359

8.165.1 Detailed Description . . . 360
8.165.2 Constructor & Destructor Documentation . . . 360
8.165.2.1 Rtcm_Printer(std::string filename, bool flag_rtcm_server, bool flag_rtcm_tty_port, unsigned short rtcm_tcp_port, unsigned short rtcm_station_id, std::string rtcm↩ _dump_filename, bool time_tag_name=true) . . . 360
8.165.2.2 ∼Rtcm_Printer() . . . 360
8.166 rtl_tcp_dongle_info Class Reference . . . 360
8.166.1 Detailed Description . . . 361
8.167 rtl_tcp_signal_source_c Class Reference . . . 361
8.167.1 Detailed Description . . . 361
8.168 RtlTcpSignalSource Class Reference . . . 362
8.168.1 Detailed Description . . . 362
8.168.2 Member Function Documentation . . . 362
8.168.2.1 implementation() . . . 362
8.169 Sbas_Ephemeris Class Reference . . . 363
8.169.1 Detailed Description . . . 363
8.169.2 Member Data Documentation . . . 363
8.169.2.1 b_sv_do_not_use . . . 363
8.169.2.2 d_acc . . . 364
8.169.2.3 d_af0 . . . 364
8.169.2.4 d_af1 . . . 364
8.169.2.5 d_pos . . . 364
8.169.2.6 d_vel . . . 364
8.169.2.7 i_prn . . . 364
8.169.2.8 i_sv_ura . . . 364
8.170 Sbas_Ionosphere_Correction Class Reference . . . 365
8.170.1 Detailed Description . . . 365
8.170.2 Member Function Documentation . . . 365
8.170.2.1 apply(double sample_stamp, double latitude_d, double longitude_d, double azimut_d, double evaluation_d, double &delay, double &var) . . . 365
8.171 sbas_l1_telemetry_decoder_cc Class Reference . . . 366
8.171.1 Detailed Description . . . 366

8.171.2 Member Function Documentation . . . 366
8.171.2.1 forecast(int noutput_items, gr_vector_int &ninput_items_required) . . . 366
8.171.2.2 general_work(int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items) . . . 367
8.171.2.3 set_channel(int channel) . . . 367
8.171.2.4 set_satellite(Gnss_Satellite satellite) . . . 367
8.172 Sbas_Raw_Msg Class Reference . . . 367
8.172.1 Detailed Description . . . 367
8.172.2 Member Function Documentation . . . 368
8.172.2.1 get_sample_stamp() . . . 368
8.173 Sbas_Satellite_Correction Class Reference . . . 368
8.173.1 Detailed Description . . . 368
8.174 Sbas_Telemetry_Data Class Reference . . . 369
8.174.1 Detailed Description . . . 369
8.174.2 Constructor & Destructor Documentation . . . 369
8.174.2.1 Sbas_Telemetry_Data() . . . 369
8.174.2.2 ~Sbas_Telemetry_Data() . . . 369
8.175 Sbas_Time Class Reference . . . 369
8.175.1 Detailed Description . . . 370
8.176 Sbas_Time_Relation Class Reference . . . 370
8.176.1 Detailed Description . . . 370
8.177 SbasL1TelemetryDecoder Class Reference . . . 370
8.177.1 Detailed Description . . . 371
8.177.2 Member Function Documentation . . . 371
8.177.2.1 implementation() . . . 371
8.178 short_x2_to_cshort Class Reference . . . 371
8.178.1 Detailed Description . . . 372
8.179 signal_generator_c Class Reference . . . 372
8.179.1 Detailed Description . . . 373
8.179.2 Friends And Related Function Documentation . . . 373
8.179.2.1 signal_make_generator_c . . . 373

8.180 SignalConditioner Class Reference . . . 373
8.180.1 Detailed Description . . . 374
8.180.2 Constructor & Destructor Documentation . . . 374
8.180.2.1 SignalConditioner(ConfigurationInterface *configuration, std::shared_ptr< GNSSBlockInterface > data_type_adapt, std::shared_ptr< GNSSBlockInterface > in_filt, std::shared_ptr< GNSSBlockInterface > res, std::string role, std::string implementation) . . . 374
8.180.2.2 ~SignalConditioner() . . . 374
8.180.3 Member Function Documentation . . . 374
8.180.3.1 implementation() . . . 374
8.181 SignalGenerator Class Reference . . . 375
8.181.1 Detailed Description . . . 375
8.181.2 Member Function Documentation . . . 375
8.181.2.1 implementation() . . . 375
8.182 SpirFileSignalSource Class Reference . . . 376
8.182.1 Detailed Description . . . 376
8.182.2 Member Function Documentation . . . 376
8.182.2.1 implementation() . . . 376
8.183 StringConverter Class Reference . . . 377
8.183.1 Detailed Description . . . 377
8.184 tcp_communication Class Reference . . . 377
8.184.1 Detailed Description . . . 377
8.185 tcp_packet_data Class Reference . . . 378
8.185.1 Detailed Description . . . 378
8.186 TelemetryDecoderInterface Class Reference . . . 378
8.186.1 Detailed Description . . . 379
8.187 tlm_dump_reader Class Reference . . . 379
8.187.1 Detailed Description . . . 379
8.188 Tracking_2nd_DLL_filter Class Reference . . . 379
8.188.1 Detailed Description . . . 380
8.188.2 Constructor & Destructor Documentation . . . 380
8.188.2.1 Tracking_2nd_DLL_filter(float pdi_code) . . . 380

8.188.3 Member Function Documentation . . . 380
8.188.3.1 get_code_nco(float DLL_discriminator) . . . 380
8.188.3.2 initialize() . . . 380
8.188.3.3 set_pdi(float pdi_code) . . . 380
8.189 Tracking_2nd_PLL_filter Class Reference . . . 381
8.189.1 Detailed Description . . . 381
8.189.2 Member Function Documentation . . . 381
8.189.2.1 initialize() . . . 381
8.189.2.2 set_pdi(float pdi_carr) . . . 381
8.190 tracking_dump_reader Class Reference . . . 381
8.190.1 Detailed Description . . . 382
8.191 Tracking_FLL_PLL_filter Class Reference . . . 382
8.191.1 Detailed Description . . . 382
8.192 Tracking_loop_filter Class Reference . . . 383
8.192.1 Detailed Description . . . 383
8.193 tracking_true_obs_reader Class Reference . . . 383
8.193.1 Detailed Description . . . 384
8.194 TrackingInterface Class Reference . . . 384
8.194.1 Detailed Description . . . 385
8.195 TwoBitCpxFileSignalSource Class Reference . . . 385
8.195.1 Detailed Description . . . 385
8.195.2 Member Function Documentation . . . 386
8.195.2.1 implementation() . . . 386
8.196 TwoBitPackedFileSignalSource Class Reference . . . 386
8.196.1 Detailed Description . . . 387
8.196.2 Member Function Documentation . . . 387
8.196.2.1 implementation() . . . 387
8.197 UhdSignalSource Class Reference . . . 387
8.197.1 Detailed Description . . . 388
8.197.2 Member Function Documentation . . . 388
8.197.2.1 implementation() . . . 388
8.198 unpack_2bit_samples Class Reference . . . 388
8.198.1 Detailed Description . . . 389
8.199 unpack_byte_2bit_cpx_samples Class Reference . . . 389
8.199.1 Detailed Description . . . 389
8.200 unpack_byte_2bit_samples Class Reference . . . 390
8.200.1 Detailed Description . . . 390
8.201 unpack_intspir_1bit_samples Class Reference . . . 390
8.201.1 Detailed Description . . . 391
8.202 Viterbi_Decoder Class Reference . . . 391
8.202.1 Detailed Description . . . 391
8.202.2 Member Function Documentation . . . 391
8.202.2.1 decode_block(const double input_c[], int *output_u_int, const int LL) . . . 391

9 File Documentation **393**

9.1 src/algorithms/acquisition/adapters/galileo_e1_pcps_8ms_ambiguous_acquisition.h File Reference 393

9.1.1 Detailed Description . . . 393

9.2 src/algorithms/acquisition/adapters/galileo_e1_pcps_ambiguous_acquisition.h File Reference . . . 394

9.2.1 Detailed Description . . . 394

9.3 src/algorithms/acquisition/adapters/galileo_e1_pcps_cccwsr_ambiguous_acquisition.h File Reference 395

9.3.1 Detailed Description . . . 395

9.4 src/algorithms/acquisition/adapters/galileo_e1_pcps_quicksync_ambiguous_acquisition.h File Reference . . . 395

9.4.1 Detailed Description . . . 396

9.5 src/algorithms/acquisition/adapters/galileo_e1_pcps_tong_ambiguous_acquisition.h File Reference 396

9.5.1 Detailed Description . . . 397

9.6 src/algorithms/acquisition/adapters/galileo_e5a_noncoherent_iq_acquisition_caf.h File Reference . 397

9.6.1 Detailed Description . . . 398

9.7 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition.h File Reference . . . 398

9.7.1 Detailed Description . . . 399

9.8 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition_fine_doppler.h File Reference . . 399

9.8.1 Detailed Description . . . 400

9.9 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_assisted_acquisition.h File Reference . . . 400

9.9.1 Detailed Description . . . 401

9.10 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_multithread_acquisition.h File Reference . . . 401

9.10.1 Detailed Description . . . 402

9.11 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_opencl_acquisition.h File Reference 402

9.11.1 Detailed Description . . . 403

9.12 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_quicksync_acquisition.h File Reference 403

9.12.1 Detailed Description . . . 404

9.13 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_tong_acquisition.h File Reference 404

9.13.1 Detailed Description . . . 405

9.14 src/algorithms/acquisition/adapters/gps_l2_m_pcps_acquisition.h File Reference 405

9.14.1 Detailed Description . . . 406

9.15 src/algorithms/acquisition/gnuradio_blocks/galileo_e5a_noncoherent_iq_acquisition_caf_cc.h File Reference . . . 406
9.15.1 Detailed Description . . . 407
9.16 src/algorithms/acquisition/gnuradio_blocks/galileo_pcps_8ms_acquisition_cc.h File Reference . . . 407
9.16.1 Detailed Description . . . 408
9.17 src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_cc.h File Reference . . . 408
9.17.1 Detailed Description . . . 409
9.18 src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_sc.h File Reference . . . 410
9.18.1 Detailed Description . . . 410
9.19 src/algorithms/acquisition/gnuradio_blocks/pcps_assisted_acquisition_cc.h File Reference . . . 411
9.19.1 Detailed Description . . . 412
9.20 src/algorithms/acquisition/gnuradio_blocks/pcps_cccwsr_acquisition_cc.h File Reference . . . 412
9.20.1 Detailed Description . . . 413
9.21 src/algorithms/acquisition/gnuradio_blocks/pcps_multithread_acquisition_cc.h File Reference . . . 414
9.21.1 Detailed Description . . . 414
9.22 src/algorithms/acquisition/gnuradio_blocks/pcps_opencl_acquisition_cc.h File Reference . . . 415
9.22.1 Detailed Description . . . 416
9.23 src/algorithms/acquisition/gnuradio_blocks/pcps_quicksync_acquisition_cc.h File Reference . . . 417
9.23.1 Detailed Description . . . 417
9.24 src/algorithms/acquisition/gnuradio_blocks/pcps_tong_acquisition_cc.h File Reference . . . 418
9.24.1 Detailed Description . . . 419
9.25 src/algorithms/channel/adapters/channel.h File Reference . . . 420
9.25.1 Detailed Description . . . 420
9.26 src/algorithms/channel/libs/channel_fsm.h File Reference . . . 421
9.26.1 Detailed Description . . . 421
9.27 src/algorithms/channel/libs/channel_msg_receiver_cc.h File Reference . . . 421
9.27.1 Detailed Description . . . 422
9.28 src/algorithms/conditioner/adapters/array_signal_conditioner.h File Reference . . . 422
9.28.1 Detailed Description . . . 423
9.29 src/algorithms/conditioner/adapters/signal_conditioner.h File Reference . . . 423
9.29.1 Detailed Description . . . 424

9.30 src/algorithms/data_type_adapter/adapters/byte_to_short.h File Reference 424
9.30.1 Detailed Description 424
9.31 src/algorithms/data_type_adapter/adapters/ibyte_to_cbyte.h File Reference 425
9.31.1 Detailed Description 425
9.32 src/algorithms/data_type_adapter/adapters/ibyte_to_complex.h File Reference 425
9.32.1 Detailed Description 426
9.33 src/algorithms/data_type_adapter/adapters/ibyte_to_cshort.h File Reference 426
9.33.1 Detailed Description 427
9.34 src/algorithms/data_type_adapter/adapters/ishort_to_complex.h File Reference 427
9.34.1 Detailed Description 427
9.35 src/algorithms/data_type_adapter/adapters/ishort_to_cshort.h File Reference 428
9.35.1 Detailed Description 428
9.36 src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_byte.h File Reference 428
9.36.1 Detailed Description 429
9.37 src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_short.h File Reference 429
9.37.1 Detailed Description 430
9.38 src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_short_to_complex_short.h File Reference 430
9.38.1 Detailed Description 431
9.39 src/algorithms/input_filter/adapters/beamformer_filter.h File Reference 431
9.39.1 Detailed Description 432
9.40 src/algorithms/input_filter/adapters/fir_filter.h File Reference 432
9.40.1 Detailed Description 433
9.41 src/algorithms/input_filter/adapters/freq_xlating_fir_filter.h File Reference 433
9.41.1 Detailed Description 434
9.42 src/algorithms/input_filter/gnuradio_blocks/beamformer.h File Reference 434
9.42.1 Detailed Description 435
9.43 src/algorithms/libs/byte_x2_to_complex_byte.h File Reference 435
9.43.1 Detailed Description 436
9.44 src/algorithms/libs/complex_byte_to_float_x2.h File Reference 436

9.44.1 Detailed Description . . . 437

9.45 src/algorithms/libs/complex_float_to_complex_byte.h File Reference . . . 437

9.45.1 Detailed Description . . . 438

9.46 src/algorithms/libs/cshort_to_float_x2.h File Reference . . . 438

9.46.1 Detailed Description . . . 439

9.47 src/algorithms/libs/galileo_e1_signal_processing.h File Reference . . . 439

9.47.1 Detailed Description . . . 440

9.47.2 Function Documentation . . . 440

9.47.2.1 galileo_e1_cboc_gen(std::complex< float > *_dest, int *_prn, char _Signal[3]) . . . 440

9.47.2.2 galileo_e1_code_gen_complex_sampled(std::complex< float > *_dest, char _Signal[3], bool _cboc, unsigned int _prn, signed int _fs, unsigned int _chip_shift, bool _secondary_flag) . . . 440

9.47.2.3 galileo_e1_code_gen_complex_sampled(std::complex< float > *_dest, char _Signal[3], bool _cboc, unsigned int _prn, signed int _fs, unsigned int _chip_shift) 440

9.47.2.4 galileo_e1_code_gen_int(int *_dest, char _Signal[3], signed int _prn) . . . 440

9.47.2.5 galileo_e1_sinboc_11_gen(std::complex< float > *_dest, int *_prn, unsigned int _codeLength) . . . 440

9.47.2.6 galileo_e1_sinboc_61_gen(std::complex< float > *_dest, int *_prn, unsigned int _codeLength) . . . 441

9.48 src/algorithms/libs/gnss_sdr_valve.h File Reference . . . 441

9.48.1 Detailed Description . . . 441

9.49 src/algorithms/libs/gnss_signal_processing.h File Reference . . . 442

9.49.1 Detailed Description . . . 442

9.49.2 Function Documentation . . . 442

9.49.2.1 complex_exp_gen(std::complex< float > *_dest, double _f, double _fs, unsigned int _samps) . . . 442

9.49.2.2 complex_exp_gen_conj(std::complex< float > *_dest, double _f, double _fs, unsigned int _samps) . . . 443

9.49.2.3 hex_to_binary_converter(int *_dest, char _from) . . . 443

9.49.2.4 resampler(std::complex< float > *_from, std::complex< float > *_dest, float _fs_in, float _fs_out, unsigned int _length_in, unsigned int _length_out) . . . 443

9.50 src/algorithms/libs/gps_l2c_signal.h File Reference . . . 443

9.50.1 Detailed Description . . . 443

9.50.2 Function Documentation . . . 444

9.50.2.1 gps_l2c_m_code_gen_complex(std::complex< float > *_dest, unsigned int _prn) 444

9.50.2.2 gps_l2c_m_code_gen_complex_sampled(std::complex< float > *_dest, unsigned int _prn, signed int _fs) . . . 444

9.51 src/algorithms/libs/gps_sdr_signal_processing.h File Reference . . . 444

9.51.1 Detailed Description . . . 444

9.51.2 Function Documentation . . . 445

9.51.2.1 gps_l1_ca_code_gen_complex(std::complex< float > *_dest, signed int _prn, unsigned int _chip_shift) . . . 445

9.51.2.2 gps_l1_ca_code_gen_complex_sampled(std::complex< float > *_dest, unsigned int _prn, signed int _fs, unsigned int _chip_shift, unsigned int _ncodes) . 445

9.51.2.3 gps_l1_ca_code_gen_complex_sampled(std::complex< float > *_dest, unsigned int _prn, signed int _fs, unsigned int _chip_shift) . . . 445

9.52 src/algorithms/libs/pass_through.h File Reference . . . 445

9.52.1 Detailed Description . . . 445

9.53 src/algorithms/libs/short_x2_to_cshort.h File Reference . . . 446

9.53.1 Detailed Description . . . 446

9.54 src/algorithms/observables/adapters/galileo_e1_observables.h File Reference . . . 447

9.54.1 Detailed Description . . . 447

9.55 src/algorithms/observables/adapters/galileo_e5a_observables.h File Reference . . . 447

9.55.1 Detailed Description . . . 448

9.56 src/algorithms/observables/adapters/gps_l1_ca_observables.h File Reference . . . 448

9.56.1 Detailed Description . . . 449

9.57 src/algorithms/observables/adapters/gps_l2c_observables.h File Reference . . . 449

9.57.1 Detailed Description . . . 449

9.58 src/algorithms/observables/adapters/hybrid_observables.h File Reference . . . 450

9.58.1 Detailed Description . . . 450

9.59 src/algorithms/observables/gnuradio_blocks/galileo_e1_observables_cc.h File Reference . . . 450

9.59.1 Detailed Description . . . 451

9.60 src/algorithms/observables/gnuradio_blocks/gps_l1_ca_observables_cc.h File Reference . . . 451

9.60.1 Detailed Description . . . 452

9.61 src/algorithms/observables/gnuradio_blocks/hybrid_observables_cc.h File Reference . . . 452

9.61.1 Detailed Description . . . 453

9.62 src/algorithms/PVT/adapters/galileo_e1_pvt.h File Reference 453
9.62.1 Detailed Description 454
9.63 src/algorithms/PVT/adapters/gps_l1_ca_pvt.h File Reference 454
9.63.1 Detailed Description 455
9.64 src/algorithms/PVT/adapters/hybrid_pvt.h File Reference 455
9.64.1 Detailed Description 455
9.65 src/algorithms/PVT/gnuradio_blocks/galileo_e1_pvt_cc.h File Reference 456
9.65.1 Detailed Description 456
9.66 src/algorithms/PVT/gnuradio_blocks/gps_l1_ca_pvt_cc.h File Reference 457
9.66.1 Detailed Description 457
9.67 src/algorithms/PVT/gnuradio_blocks/hybrid_pvt_cc.h File Reference 458
9.67.1 Detailed Description 458
9.68 src/algorithms/PVT/libs/galileo_e1_ls_pvt.h File Reference 459
9.68.1 Detailed Description 459
9.69 src/algorithms/PVT/libs/geojson_printer.h File Reference 459
9.69.1 Detailed Description 460
9.70 src/algorithms/PVT/libs/gps_l1_ca_ls_pvt.h File Reference 460
9.70.1 Detailed Description 461
9.71 src/algorithms/PVT/libs/kml_printer.h File Reference 461
9.71.1 Detailed Description 462
9.72 src/algorithms/PVT/libs/ls_pvt.h File Reference 462
9.72.1 Detailed Description 463
9.73 src/algorithms/PVT/libs/nmea_printer.h File Reference 463
9.73.1 Detailed Description 464
9.74 src/algorithms/PVT/libs/pvt_solution.h File Reference 464
9.74.1 Detailed Description 465
9.75 src/algorithms/PVT/libs/rinex_printer.h File Reference 465
9.75.1 Detailed Description 466
9.76 src/algorithms/PVT/libs/rtcm_printer.h File Reference 466
9.76.1 Detailed Description 467

9.77 src/algorithms/resampler/adapters/direct_resampler_conditioner.h File Reference 467
9.77.1 Detailed Description 468
9.78 src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cb.h File Reference 468
9.78.1 Detailed Description 469
9.79 src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cc.h File Reference 469
9.79.1 Detailed Description 470
9.80 src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cs.h File Reference 470
9.80.1 Detailed Description 471
9.81 src/algorithms/signal_generator/adapters/signal_generator.h File Reference 471
9.81.1 Detailed Description 472
9.82 src/algorithms/signal_generator/gnuradio_blocks/signal_generator_c.h File Reference 472
9.82.1 Detailed Description 473
9.82.2 Function Documentation 473
9.82.2.1 signal_make_generator_c(std::vector< std::string > signal1, std::vector< std::string > system, const std::vector< unsigned int > &PRN, const std::vector< float > &CN0_dB, const std::vector< float > &doppler_Hz, const std::vector< unsigned int > &delay_chips, const std::vector< unsigned int > &delay_sec, bool data_flag, bool noise_flag, unsigned int fs_in, unsigned int vector_length, float BW_BB) 473
9.83 src/algorithms/signal_source/adapters/file_signal_source.h File Reference 474
9.83.1 Detailed Description 474
9.84 src/algorithms/signal_source/adapters/gen_signal_source.h File Reference 474
9.84.1 Detailed Description 475
9.85 src/algorithms/signal_source/adapters/gn3s_signal_source.h File Reference 475
9.85.1 Detailed Description 476
9.86 src/algorithms/signal_source/adapters/nsr_file_signal_source.h File Reference 476
9.86.1 Detailed Description 477
9.87 src/algorithms/signal_source/adapters/osmosdr_signal_source.h File Reference 477
9.87.1 Detailed Description 478
9.88 src/algorithms/signal_source/adapters/raw_array_signal_source.h File Reference 478
9.88.1 Detailed Description 479
9.89 src/algorithms/signal_source/adapters/rtl_tcp_signal_source.h File Reference 479
9.89.1 Detailed Description 480

9.90 src/algorithms/signal_source/adapters/spir_file_signal_source.h File Reference 480
9.90.1 Detailed Description 481
9.91 src/algorithms/signal_source/adapters/two_bit_cpx_file_signal_source.h File Reference 481
9.91.1 Detailed Description 482
9.92 src/algorithms/signal_source/adapters/two_bit_packed_file_signal_source.h File Reference 482
9.92.1 Detailed Description 483
9.93 src/algorithms/signal_source/adapters/uhd_signal_source.h File Reference 483
9.93.1 Detailed Description 484
9.94 src/algorithms/signal_source/gnuradio_blocks/rtl_tcp_signal_source_c.h File Reference 484
9.94.1 Detailed Description 485
9.95 src/algorithms/signal_source/gnuradio_blocks/unpack_2bit_samples.h File Reference 485
9.95.1 Detailed Description 486
9.96 src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_cpx_samples.h File Reference . . 487
9.96.1 Detailed Description 487
9.97 src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_samples.h File Reference 488
9.97.1 Detailed Description 488
9.98 src/algorithms/signal_source/gnuradio_blocks/unpack_intspir_1bit_samples.h File Reference . . . 489
9.98.1 Detailed Description 489
9.99 src/algorithms/signal_source/libs/rtl_tcp_commands.h File Reference 490
9.99.1 Detailed Description 490
9.99.2 Enumeration Type Documentation 491
9.99.2.1 RTL_TCP_COMMAND 491
9.99.3 Function Documentation 491
9.99.3.1 rtl_tcp_command(RTL_TCP_COMMAND id, unsigned param, boost::asio::ip↩ ::tcp::socket &socket) 491
9.100 src/algorithms/signal_source/libs/rtl_tcp_dongle_info.h File Reference 491
9.100.1 Detailed Description 491
9.101 src/algorithms/telemetry_decoder/adapters/galileo_e1b_telemetry_decoder.h File Reference 492
9.101.1 Detailed Description 492
9.102 src/algorithms/telemetry_decoder/adapters/galileo_e5a_telemetry_decoder.h File Reference 492
9.102.1 Detailed Description 493

9.103 src/algorithms/telemetry_decoder/adapters/gps_l1_ca_telemetry_decoder.h File Reference 493
9.103.1 Detailed Description 494
9.104 src/algorithms/telemetry_decoder/adapters/gps_l2c_telemetry_decoder.h File Reference 494
9.104.1 Detailed Description 494
9.105 src/algorithms/telemetry_decoder/adapters/sbas_l1_telemetry_decoder.h File Reference 495
9.105.1 Detailed Description 495
9.106 src/algorithms/telemetry_decoder/gnuradio_blocks/galileo_e1b_telemetry_decoder_cc.h File Reference 495
9.106.1 Detailed Description 496
9.107 src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l1_ca_telemetry_decoder_cc.h File Reference 496
9.107.1 Detailed Description 497
9.108 src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l2c_telemetry_decoder_cc.h File Reference 497
9.108.1 Detailed Description 498
9.109 src/algorithms/telemetry_decoder/gnuradio_blocks/sbas_l1_telemetry_decoder_cc.h File Reference 498
9.109.1 Detailed Description 499
9.110 src/algorithms/telemetry_decoder/libs/convolutional.h File Reference 499
9.110.1 Detailed Description 500
9.111 src/algorithms/telemetry_decoder/libs/gps_l1_ca_subframe_fsm.h File Reference 500
9.111.1 Detailed Description 501
9.112 src/algorithms/telemetry_decoder/libs/viterbi_decoder.h File Reference 501
9.112.1 Detailed Description 501
9.113 src/algorithms/tracking/adapters/galileo_e1_dll_pll_veml_tracking.h File Reference 502
9.113.1 Detailed Description 502
9.114 src/algorithms/tracking/adapters/galileo_e1_tcp_connector_tracking.h File Reference 502
9.114.1 Detailed Description 503
9.115 src/algorithms/tracking/adapters/galileo_e5a_dll_pll_tracking.h File Reference 503
9.115.1 Detailed Description 504
9.116 src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_c_aid_tracking.h File Reference 504
9.116.1 Detailed Description 505
9.117 src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking.h File Reference 505

9.117.1 Detailed Description . . . 506
9.118 src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking_gpu.h File Reference . . . 506
9.118.1 Detailed Description . . . 507
9.119 src/algorithms/tracking/adapters/gps_l1_ca_tcp_connector_tracking.h File Reference . . . 507
9.119.1 Detailed Description . . . 508
9.120 src/algorithms/tracking/adapters/gps_l2_m_dll_pll_tracking.h File Reference . . . 508
9.120.1 Detailed Description . . . 509
9.121 src/algorithms/tracking/gnuradio_blocks/galileo_e1_dll_pll_veml_tracking_cc.h File Reference . . . 509
9.121.1 Detailed Description . . . 510
9.122 src/algorithms/tracking/gnuradio_blocks/galileo_e1_tcp_connector_tracking_cc.h File Reference . . . 510
9.122.1 Detailed Description . . . 511
9.123 src/algorithms/tracking/gnuradio_blocks/galileo_e5a_dll_pll_tracking_cc.h File Reference . . . 511
9.123.1 Detailed Description . . . 512
9.124 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_cc.h File Reference . . . 512
9.124.1 Detailed Description . . . 513
9.125 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_sc.h File Reference . . . 514
9.125.1 Detailed Description . . . 514
9.126 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_cc.h File Reference . . . 515
9.126.1 Detailed Description . . . 515
9.127 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_gpu_cc.h File Reference . . . 516
9.127.1 Detailed Description . . . 516
9.128 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_tcp_connector_tracking_cc.h File Reference . . . 517
9.128.1 Detailed Description . . . 517
9.129 src/algorithms/tracking/gnuradio_blocks/gps_l2_m_dll_pll_tracking_cc.h File Reference . . . 518
9.129.1 Detailed Description . . . 518
9.130 src/algorithms/tracking/libs/cpu_multicorrelator.h File Reference . . . 519
9.130.1 Detailed Description . . . 519
9.131 src/algorithms/tracking/libs/cpu_multicorrelator_16sc.h File Reference . . . 519
9.131.1 Detailed Description . . . 520
9.132 src/algorithms/tracking/libs/cuda_multicorrelator.h File Reference . . . 520

9.132.1 Detailed Description . . . 521
9.133 src/algorithms/tracking/libs/lock_detectors.h File Reference . . . 521
9.133.1 Detailed Description . . . 521
9.133.2 Function Documentation . . . 522
9.133.2.1 carrier_lock_detector(gr_complex *Prompt_buffer, int length) . . . 522
9.133.2.2 cn0_svn_estimator(gr_complex *Prompt_buffer, int length, long fs_in, double code_length) . . . 522
9.134 src/algorithms/tracking/libs/tcp_communication.h File Reference . . . 523
9.134.1 Detailed Description . . . 523
9.135 src/algorithms/tracking/libs/tcp_packet_data.h File Reference . . . 523
9.135.1 Detailed Description . . . 524
9.136 src/algorithms/tracking/libs/tracking_2nd_DLL_filter.h File Reference . . . 524
9.136.1 Detailed Description . . . 525
9.137 src/algorithms/tracking/libs/tracking_2nd_PLL_filter.h File Reference . . . 525
9.137.1 Detailed Description . . . 525
9.138 src/algorithms/tracking/libs/tracking_discriminators.h File Reference . . . 526
9.138.1 Detailed Description . . . 526
9.138.2 Function Documentation . . . 527
9.138.2.1 dll_nc_e_minus_l_normalized(gr_complex early_s1, gr_complex late_s1) . . . 527
9.138.2.2 dll_nc_vemlp_normalized(gr_complex very_early_s1, gr_complex early_s1, gr_complex late_s1, gr_complex very_late_s1) . . . 527
9.138.2.3 fll_four_quadrant_atan(gr_complex prompt_s1, gr_complex prompt_s2, double t1, double t2) . . . 527
9.138.2.4 pll_cloop_two_quadrant_atan(gr_complex prompt_s1) . . . 527
9.138.2.5 pll_four_quadrant_atan(gr_complex prompt_s1) . . . 528
9.139 src/algorithms/tracking/libs/tracking_FLL_PLL_filter.h File Reference . . . 528
9.139.1 Detailed Description . . . 528
9.140 src/algorithms/tracking/libs/tracking_loop_filter.h File Reference . . . 528
9.140.1 Detailed Description . . . 529
9.141 src/core/interfaces/acquisition_interface.h File Reference . . . 529
9.141.1 Detailed Description . . . 530
9.142 src/core/interfaces/channel_interface.h File Reference . . . 530

9.142.1 Detailed Description . . . 531
9.143 src/core/interfaces/configuration_interface.h File Reference . . . 531
9.143.1 Detailed Description . . . 532
9.144 src/core/interfaces/gnss_block_interface.h File Reference . . . 532
9.144.1 Detailed Description . . . 533
9.145 src/core/interfaces/observables_interface.h File Reference . . . 533
9.145.1 Detailed Description . . . 534
9.146 src/core/interfaces/pvt_interface.h File Reference . . . 534
9.146.1 Detailed Description . . . 535
9.147 src/core/interfaces/telemetry_decoder_interface.h File Reference . . . 535
9.147.1 Detailed Description . . . 536
9.148 src/core/interfaces/tracking_interface.h File Reference . . . 536
9.148.1 Detailed Description . . . 537
9.149 src/core/libs/gnss_sdr_supl_client.h File Reference . . . 537
9.149.1 Detailed Description . . . 538
9.150 src/core/libs/ini.h File Reference . . . 538
9.150.1 Detailed Description . . . 539
9.151 src/core/libs/INIReader.h File Reference . . . 539
9.151.1 Detailed Description . . . 540
9.152 src/core/libs/string_converter.h File Reference . . . 540
9.152.1 Detailed Description . . . 541
9.153 src/core/receiver/concurrent_map.h File Reference . . . 541
9.153.1 Detailed Description . . . 542
9.154 src/core/receiver/concurrent_queue.h File Reference . . . 542
9.154.1 Detailed Description . . . 542
9.155 src/core/receiver/control_message.h File Reference . . . 543
9.155.1 Detailed Description . . . 543
9.156 src/core/receiver/control_message_factory.h File Reference . . . 543
9.156.1 Detailed Description . . . 544
9.156.2 Typedef Documentation . . . 544

9.156.2.1 ControlMessage . . . 544
9.157 src/core/receiver/control_thread.h File Reference . . . 544
9.157.1 Detailed Description . . . 545
9.158 src/core/receiver/file_configuration.h File Reference . . . 545
9.158.1 Detailed Description . . . 546
9.159 src/core/receiver/gnss_block_factory.h File Reference . . . 546
9.159.1 Detailed Description . . . 547
9.160 src/core/receiver/gnss_flowgraph.h File Reference . . . 547
9.160.1 Detailed Description . . . 548
9.161 src/core/receiver/in_memory_configuration.h File Reference . . . 548
9.161.1 Detailed Description . . . 549
9.162 src/core/system_parameters/galileo_almanac.h File Reference . . . 549
9.162.1 Detailed Description . . . 549
9.163 src/core/system_parameters/Galileo_E1.h File Reference . . . 550
9.163.1 Detailed Description . . . 554
9.163.2 Variable Documentation . . . 555
9.163.2.1 GALILEO_C_m_ms . . . 555
9.163.2.2 GALILEO_C_m_s . . . 555
9.163.2.3 Galileo_E1_B_CODE_LENGTH_CHIPS . . . 555
9.163.2.4 Galileo_E1_B_SYMBOL_RATE_BPS . . . 555
9.163.2.5 Galileo_E1_C_SECONDARY_CODE_LENGTH . . . 556
9.163.2.6 Galileo_E1_CODE_CHIP_RATE_HZ . . . 556
9.163.2.7 Galileo_E1_CODE_PERIOD . . . 556
9.163.2.8 Galileo_E1_FREQ_HZ . . . 556
9.163.2.9 Galileo_E1_SUB_CARRIER_A_RATE_HZ . . . 556
9.163.2.10 Galileo_E1_SUB_CARRIER_B_RATE_HZ . . . 556
9.163.2.11 GALILEO_F . . . 556
9.163.2.12 GALILEO_GM . . . 556
9.163.2.13 GALILEO_INAV_PAGE_PART_SYMBOLS . . . 557
9.163.2.14 GALILEO_INAV_PAGE_SYMBOLS . . . 557

9.163.2.15 GALILEO_OMEGA_EARTH_DOT . . . 557
9.163.2.16 GALILEO_PI . . . 557
9.163.2.17 GALILEO_TWO_PI . . . 557
9.164 src/core/system_parameters/galileo_ephemeris.h File Reference . . . 557
9.164.1 Detailed Description . . . 558
9.165 src/core/system_parameters/galileo_fnav_message.h File Reference . . . 558
9.165.1 Detailed Description . . . 559
9.166 src/core/system_parameters/galileo_iono.h File Reference . . . 559
9.166.1 Detailed Description . . . 559
9.167 src/core/system_parameters/galileo_navigation_message.h File Reference . . . 560
9.167.1 Detailed Description . . . 560
9.168 src/core/system_parameters/galileo_utc_model.h File Reference . . . 561
9.168.1 Detailed Description . . . 561
9.169 src/core/system_parameters/gnss_satellite.h File Reference . . . 562
9.169.1 Detailed Description . . . 562
9.170 src/core/system_parameters/gnss_signal.h File Reference . . . 562
9.170.1 Detailed Description . . . 563
9.171 src/core/system_parameters/gnss_synchro.h File Reference . . . 563
9.171.1 Detailed Description . . . 563
9.172 src/core/system_parameters/gps_acq_assist.h File Reference . . . 564
9.172.1 Detailed Description . . . 564
9.173 src/core/system_parameters/gps_almanac.h File Reference . . . 565
9.173.1 Detailed Description . . . 565
9.174 src/core/system_parameters/gps_cnav_ephemeris.h File Reference . . . 565
9.174.1 Detailed Description . . . 566
9.175 src/core/system_parameters/gps_cnav_iono.h File Reference . . . 566
9.175.1 Detailed Description . . . 566
9.176 src/core/system_parameters/gps_cnav_navigation_message.h File Reference . . . 567
9.176.1 Detailed Description . . . 567
9.177 src/core/system_parameters/gps_ephemeris.h File Reference . . . 567

9.177.1 Detailed Description . . . 568
9.178 src/core/system_parameters/gps_iono.h File Reference . . . 568
9.178.1 Detailed Description . . . 569
9.179 src/core/system_parameters/GPS_L1_CA.h File Reference . . . 569
9.179.1 Detailed Description . . . 573
9.179.2 Function Documentation . . . 573
9.179.2.1 ALPHA_0({{69, 8}}) . . . 573
9.179.2.2 T_OA({{69, 8}}) . . . 573
9.179.3 Variable Documentation . . . 573
9.179.3.1 F . . . 573
9.179.3.2 GM . . . 574
9.179.3.3 GPS_C_m_ms . . . 574
9.179.3.4 GPS_C_m_s . . . 574
9.179.3.5 GPS_CA_TELEMETRY_RATE_BITS_SECOND . . . 574
9.179.3.6 GPS_CA_TELEMETRY_RATE_SYMBOLS_SECOND . . . 574
9.179.3.7 GPS_L1_CA_CHIP_PERIOD . . . 574
9.179.3.8 GPS_L1_CA_CODE_LENGTH_CHIPS . . . 574
9.179.3.9 GPS_L1_CA_CODE_PERIOD . . . 575
9.179.3.10 GPS_L1_CA_CODE_RATE_HZ . . . 575
9.179.3.11 GPS_L1_FREQ_HZ . . . 575
9.179.3.12 GPS_PI . . . 575
9.179.3.13 GPS_SUBFRAME_BITS . . . 575
9.179.3.14 GPS_SUBFRAME_LENGTH . . . 575
9.179.3.15 GPS_SUBFRAME_MS . . . 575
9.179.3.16 GPS_SUBFRAME_SECONDS . . . 575
9.179.3.17 GPS_TWO_PI . . . 576
9.179.3.18 GPS_WORD_BITS . . . 576
9.179.3.19 GPS_WORD_LENGTH . . . 576
9.179.3.20 MAX_TOA_DELAY_MS . . . 576
9.179.3.21 OMEGA_EARTH_DOT . . . 576

9.180 src/core/system_parameters/GPS_L2C.h File Reference . . . 576
9.180.1 Detailed Description . . . 579
9.180.2 Variable Documentation . . . 580
9.180.2.1 GPS_L2_C_m_ms . . . 580
9.180.2.2 GPS_L2_C_m_s . . . 580
9.180.2.3 GPS_L2_CNAV_DATA_PAGE_BITS . . . 580
9.180.2.4 GPS_L2_CNAV_DATA_PAGE_BITS_EXTENDED_BYTES . . . 580
9.180.2.5 GPS_L2_F . . . 580
9.180.2.6 GPS_L2_FREQ_HZ . . . 580
9.180.2.7 GPS_L2_GM . . . 580
9.180.2.8 GPS_L2_L_CODE_LENGTH_CHIPS . . . 581
9.180.2.9 GPS_L2_L_CODE_RATE_HZ . . . 581
9.180.2.10 GPS_L2_L_PERIOD . . . 581
9.180.2.11 GPS_L2_M_CODE_LENGTH_CHIPS . . . 581
9.180.2.12 GPS_L2_M_CODE_RATE_HZ . . . 581
9.180.2.13 GPS_L2_M_PERIOD . . . 581
9.180.2.14 GPS_L2_OMEGA_EARTH_DOT . . . 581
9.180.2.15 GPS_L2_PI . . . 581
9.180.2.16 GPS_L2_TWO_PI . . . 582
9.180.2.17 GPS_L2C_M_INIT_REG . . . 582
9.181 src/core/system_parameters/gps_navigation_message.h File Reference . . . 582
9.181.1 Detailed Description . . . 583
9.182 src/core/system_parameters/gps_ref_location.h File Reference . . . 583
9.182.1 Detailed Description . . . 584
9.183 src/core/system_parameters/gps_ref_time.h File Reference . . . 584
9.183.1 Detailed Description . . . 584
9.184 src/core/system_parameters/gps_utc_model.h File Reference . . . 585
9.184.1 Detailed Description . . . 585
9.185 src/core/system_parameters/MATH_CONSTANTS.h File Reference . . . 585
9.185.1 Detailed Description . . . 588

9.185.2 Variable Documentation . . . 588
9.185.2.1 PI_TWO_N19 . . . 588
9.185.2.2 PI_TWO_N23 . . . 588
9.185.2.3 PI_TWO_N31 . . . 588
9.185.2.4 PI_TWO_N38 . . . 588
9.185.2.5 PI_TWO_N43 . . . 589
9.185.2.6 TWO_N10 . . . 589
9.185.2.7 TWO_N11 . . . 589
9.185.2.8 TWO_N14 . . . 589
9.185.2.9 TWO_N15 . . . 589
9.185.2.10 TWO_N16 . . . 589
9.185.2.11 TWO_N19 . . . 589
9.185.2.12 TWO_N2 . . . 589
9.185.2.13 TWO_N20 . . . 590
9.185.2.14 TWO_N21 . . . 590
9.185.2.15 TWO_N24 . . . 590
9.185.2.16 TWO_N25 . . . 590
9.185.2.17 TWO_N27 . . . 590
9.185.2.18 TWO_N29 . . . 590
9.185.2.19 TWO_N30 . . . 590
9.185.2.20 TWO_N31 . . . 590
9.185.2.21 TWO_N32 . . . 591
9.185.2.22 TWO_N33 . . . 591
9.185.2.23 TWO_N34 . . . 591
9.185.2.24 TWO_N35 . . . 591
9.185.2.25 TWO_N38 . . . 591
9.185.2.26 TWO_N43 . . . 591
9.185.2.27 TWO_N44 . . . 591
9.185.2.28 TWO_N46 . . . 591
9.185.2.29 TWO_N48 . . . 592

9.185.2.30TWO_N5 . . . 592
9.185.2.31TWO_N50 . . . 592
9.185.2.32TWO_N51 . . . 592
9.185.2.33TWO_N55 . . . 592
9.185.2.34TWO_N57 . . . 592
9.185.2.35TWO_N59 . . . 592
9.185.2.36TWO_N60 . . . 592
9.185.2.37TWO_N8 . . . 593
9.185.2.38TWO_N9 . . . 593
9.185.2.39TWO_P11 . . . 593
9.185.2.40TWO_P12 . . . 593
9.185.2.41TWO_P14 . . . 593
9.185.2.42TWO_P16 . . . 593
9.185.2.43TWO_P19 . . . 593
9.185.2.44TWO_P31 . . . 593
9.185.2.45TWO_P32 . . . 594
9.185.2.46TWO_P4 . . . 594
9.185.2.47TWO_P56 . . . 594
9.185.2.48TWO_P57 . . . 594
9.186src/core/system_parameters/rtcm.h File Reference . . . 594
9.186.1 Detailed Description . . . 595
9.187src/core/system_parameters/sbas_ephemeris.h File Reference . . . 595
9.187.1 Detailed Description . . . 595
9.188src/core/system_parameters/sbas_ionospheric_correction.h File Reference . . . 596
9.188.1 Detailed Description . . . 596
9.189src/core/system_parameters/sbas_satellite_correction.h File Reference . . . 597
9.189.1 Detailed Description . . . 597
9.190src/core/system_parameters/sbas_telemetry_data.h File Reference . . . 597
9.190.1 Detailed Description . . . 599
9.191src/core/system_parameters/sbas_time.h File Reference . . . 599
9.191.1 Detailed Description . . . 600
9.192src/tests/common-files/signal_generator_flags.h File Reference . . . 600
9.192.1 Detailed Description . . . 601
9.193src/tests/unit-tests/signal-processing-blocks/libs/tlm_dump_reader.h File Reference . . . 601
9.193.1 Detailed Description . . . 601
9.194src/tests/unit-tests/signal-processing-blocks/libs/tracking_dump_reader.h File Reference . . . 602
9.194.1 Detailed Description . . . 602
9.195src/tests/unit-tests/signal-processing-blocks/libs/tracking_true_obs_reader.h File Reference . . . 602
9.195.1 Detailed Description . . . 603
9.196src/utils/front-end-cal/front_end_cal.h File Reference . . . 603
9.196.1 Detailed Description . . . 604

Index **605**

Chapter 1

Main Page



Figure 1.1 GNSS-SDR logo

Welcome to GNSS-SDR!

GNSS-SDR is an open-source GNSS software receiver freely available to the research community. This project provides a common framework for GNSS signal processing which can operate in a variety of computer platforms. This tool is intended to foster collaboration, increase awareness, and reduce development costs in the field of GNSS receiver design and customized use of GNSS signals.

For details about GNSS-SDR and using it, please see the **main project page** or browse the code at the **Sourceforge project page**. You could be also interested in **subscribing to the mailing list**.

1.1 Contents

- Overview
- Building GNSS-SDR
- Using GNSS-SDR
- Control plane
- Signal Processing plane
- About the software license
- Publications and Credits
- Ok, now what?

More details on GNSS-SDR signal processing blocks:

- Signal Source
- Signal Conditioner
- Channel
  - Acquisition
  - Tracking
  - Decoding of the navigation message
- Observables
- Computation of Position, Velocity and Time

1.2 Overview

GNSS-SDR provides an interface to different suitable RF front-ends and implements all the receiver chain up to the navigation solution. Its design allows any kind of customization, including interchangeability of signal sources, signal processing algorithms, interoperability with other systems, output formats, and offers interfaces to all the intermediate signals, parameters and variables. The goal is to write efficient and truly reusable code, easy to read and maintain, with fewer bugs, and producing highly optimized executables in a variety of hardware platforms and operating systems. In that sense, the challenge consists of defining a gentle balance within level of abstraction and performance. GNSS-SDR runs in a personal computer and provides interfaces through USB and Ethernet buses to a variety of either commercially available or custom-made RF front-ends, adapting the processing algorithms to different sampling frequencies, intermediate frequencies and sample resolutions. This makes possible rapid prototyping of specific receivers intended, for instance, to geodetic applications, observation of the ionospheric impact on navigation signals, GNSS reflectometry, signal quality monitoring, or carrier-phase based navigation techniques.

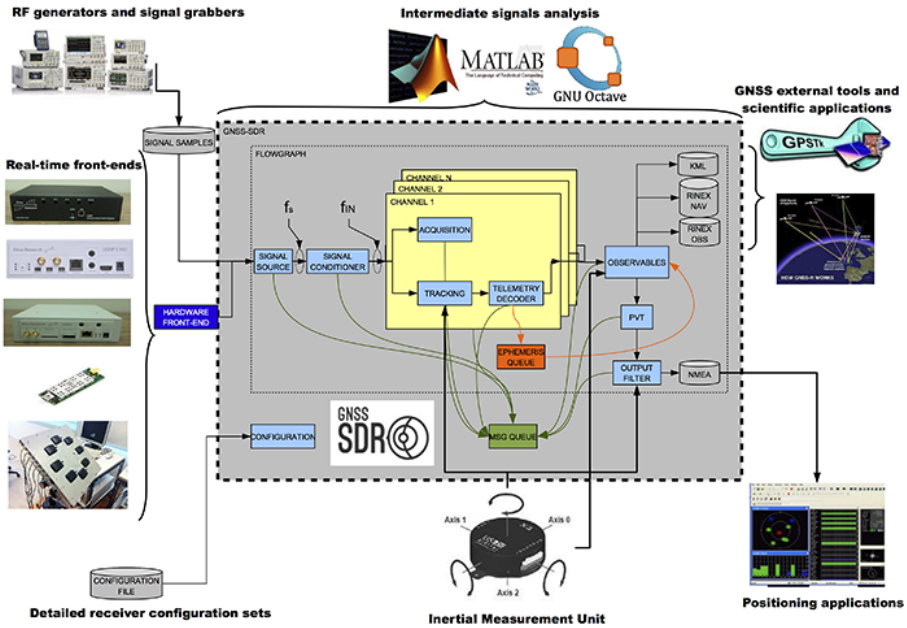


Figure 1.2 Overview

As signal inputs, it accepts:

- Raw data file captured with a data grabber (digitized at some intermediate frequency or directly at baseband).

- Any suitable RF configuration that can be driven by the Universal Software Radio Peripheral Hardware Driver (UHD). This includes all current and future Ettus Research products. The USRP1 + DBSRX 2.2 daughterboard is an example of working configuration for GPS L1 C/A and Galileo E1B and E1C signals.
- The GN3S v2 USB dongle (GN3S v3 might work with small modifications).
- Experimentally, with some USB DVB-T dongles based on the Realtek RTL2832U chipset.
- For mathematical representations of the targeted signals, check out the Signal model page.

As outputs, it provides:

- Dump of intermediate signals (configurable by the user)
- The processing is logged at a system temporary folder (usually, /tmp)
- Observables in form of RINEX file (experimental)
- Navigation message data in form of RINEX file
- Position, Velocity and Time solution in KML format and NMEA

1.3 Building GNSS-SDR

In principle, GNSS-SDR can be built in any Unix-like system. In practice, it depends on being able to install all the required dependencies. See the building guide page for details about the project's dependencies and build process. Mainly, it consists on installing GNU Radio plus some few more libraries:

- Gflags, a library that implements commandline flags processing,
- Glog, a library that implements application-level logging,
- Armadillo, a C++ linear algebra library,
- Googletest, Google's framework for writing C++ tests (requires definition of the GTEST_DIR variable),

and, optionally,

- Gperftools, which provides fast, multi-threaded malloc() and performance analysis tools.

After all dependencies are installed, clone the GNSS-SDR repository:

```
$ git clone git://git.code.sf.net/p/gnss-sdr/cttc gnss-sdr
```

This will create a folder named gnss-sdr with the following structure:

```
|-gnss-sdr
|---build       <- where gnss-sdr is built
|---cmake       <- CMake-related files
|---conf        <- Configuration files. Each file represents one receiver.
|---data        <- Populate this folder with your captured data.
|---docs        <- Contains documentation-related files
|---install     <- Executables
|---src         <- Source code folder
|-----algorithms
|-------PVT
|-------acquisition
|-------channel
|-------conditioner
|-------data_type_adapter
|-------input_filter
|-------libs
|-------observables
|-------resampler
|-------signal_source
|-------telemetry_decoder
|-------tracking
|-----core
|-------interfaces
|-------libs
|-------receiver
|-------system_parameters
|-----main
|-----tests
|-----utils     <- some utilities (e.g. Matlab scripts)
```

You are now ready to build GNSS-SDR by using CMake as building tool:

```
$ cd gnss-sdr/build
$ cmake ../
$ make
```

If everything goes well, three new executables will be created at `gnss-sdr/install`, namely `gnss-sdr`, `volk_gnsssdr_profile` and `run_tests`. You can run them from that folder, but if you prefer to install gnss-sdr on your system and have it available anywhere else, do:

```
$ sudo make install
```

This will make a copy of the conf/ folder into /usr/local/share/gnss-sdr/conf for your reference. We suggest to create a working directory at your preferred location and store your own configuration and data files there.

You can create the documentation by doing:

```
$ make doc
```

from the `gnss-sdr/build` folder. In both cases, Doxygen will generate HTML documentation that can be retrieved pointing your browser of preference to `gnss-sdr/docs/html/index.html`.

There are two more extra targets available. From the `gnss-sdr/build` folder:

```
$ make doc-clean
```

will remove the content of previously-generated documentation and, if a LaTeX installation is detected in your system,

```
$ make pdfmanual
```

will create a PDF manual at `gnss-sdr/docs/GNSS-SDR_manual.pdf`. Please note that the PDF generation requires some fonts to be installed on the host system. In Ubuntu 12.10, those fonts do not come by default. You can install them by doing:

```
$ sudo apt-get install texlive-fonts-recommended
```

and then run `cmake ../` and `make pdfmanual` again.

1.3.1 Debug and Release builds

By default, CMake will build the Release version, meaning that the compiler will generate a faster, optimized executable. This is the recommended build type when using a RF front-end and you need to attain real time. If working with a file (and thus without real-time constraints), you may want to obtain more information about the internals of the receiver, as well as more fine-grained logging. This can be done by building the Debug version, by doing:

```
$ cd gnss-sdr/build
$ cmake -DCMAKE_BUILD_TYPE=Debug ../
$ make
$ sudo make install
```

1.3.2 Updating GNSS-SDR

If you checked out GNSS-SDR some days ago, it is possible that some developer had updated files at the Git repository. You can update your local copy by doing:

```
$ git checkout next
$ git pull origin next
```

Before rebuiling the source code, it is safe (and recommended) to remove the remainders of old builds:

```
$ cd gnss-sdr/build
$ sudo make uninstall
$ rm -rf *
```

You can also check The Git Book for more information about Git usage.

1.4 Using GNSS-SDR

With GNSS-SDR, you can define you own receiver, work with captured raw data or from a RF front-end, dump into files intermediate signals, or tune every single algorithm used in the Signal Processing plane. All the configuration is done in a single file. Those configuration files reside at the `gnss-sdr/conf` folder. By default, the executable `gnss-sdr` will read the configuration available at `gnss-sdr/conf/gnss-sdr.conf`. You can edit that file to fit your needs, or even better, define a new `my_receiver.conf` file with your own configuration. This new receiver can be done by invoking gnss-sdr with the `-config_file` flag pointing to your configuration file:

```
$ gnss-sdr --config_file=../conf/my_receiver.conf
```

You can see a guide of available implementations at `gnss-sdr/conf/master.conf`. That folder contains other working examples as well. If you have a working configuration and want to share it will others, please email it to the **GNSS-SDR developers mailing list** and we will be happy to upload it to the server.

You can use a single configuration file for processing different data files, specifying the file to be processed with the `-signal_source` flag:

```
$ gnss-sdr --config_file=../conf/my_receiver.conf --signal_source=../data/my_captured_data.dat
```

This will override the `SignalSource.filename` specified in the configuration file.

You can get a complete list of available commandline flags by doing:

```
$ gnss-sdr --help
```

For general usage of commandline flags, see how to use Google Commandline Flags.

1.5 Control plane

GNSS-SDR's main method initializes the logging library, processes the command line flags, if any, provided by the user and instantiates a ControlThread object. Its constructor reads the configuration file, creates a control queue and creates a flowgraph according to the configuration. Then, the program's main method calls the run() method of the instantiated object, an action that connects the flowgraph and starts running it. After that, and until a stop message is received, it reads control messages sent by the receiver's modules through a safe-thread queue and processes them. Finally, when a stop message is received, the main method executes the destructor of the ControlThread object, which deallocates memory, does other cleanup and exits the program.

The GNSSFlowgraph class is responsible for preparing the graph of blocks according to the configuration, running it, modifying it during run-time and stopping it. Blocks are identified by its role. This class knows which roles it has to instantiate and how to connect them. It relies on the configuration to get the correct instances of the roles it needs and then it applies the connections between GNU Radio blocks to make the graph ready to be started. The complexity related to managing the blocks and the data stream is handled by GNU Radio's `gr::top_block` class. GNSSFlowgraph wraps the `gr::top_block` instance so we can take advantage of the GNSS block factory, the configuration system and the processing blocks. This class is also responsible for applying changes to the configuration of the flowgraph during run-time, dynamically reconfiguring channels: it selects the strategy for selecting satellites. This can range from a sequential search over all the satellites' ID to smarter approaches that determine what are the satellites most likely in-view based on rough estimations of the receiver position in order to avoid searching satellites in the other side of the Earth.

The Control Plane is in charge of creating a flowgraph according to the configuration and then managing the modules. Configuration allows users to define in an easy way their own custom receiver by specifying the flowgraph (type of signal source, number of channels, algorithms to be used for each channel and each module, strategies for satellite selection, type of output format, etc.). Since it is difficult to foresee what future module implementations will be needed in terms of configuration, we used a very simple approach that can be extended without a major impact in the code. This can be achieved by simply mapping the names of the variables in the modules with the names of the parameters in the configuration.

1.5.1 Configuration

Properties are passed around within the program using the ConfigurationInterface class. There are two implementations of this interface: FileConfiguration and InMemoryConfiguration. FileConfiguration reads the properties (pairs of property name and value) from a file and stores them internally. InMemoryConfiguration does not read from a file; it remains empty after instantiation and property values and names are set using the set property method. FileConfiguration is intended to be used in the actual GNSS-SDR application whereas InMemoryConfiguration is intended to be used in tests to avoid file-dependency in the file system. Classes that need to read configuration parameters will receive instances of ConfigurationInterface from where they will fetch the values. For instance, parameters related to SignalSource should look like this:

```
SignalSource.parameter1=value1
SignalSource.parameter2=value2
```

The name of these parameters can be anything but one reserved word: implementation. This parameter indicates in its value the name of the class that has to be instantiated by the factory for that role. For instance, if our signal source is providing data already at baseband and thus we want to use the implementation Pass_Through for module SignalConditioner, the corresponding line in the configuration file would be

```
SignalConditioner.implementation=Pass_Through
```

Since the configuration is just a set of property names and values without any meaning or syntax, the system is very versatile and easily extendable. Adding new properties to the system only implies modifications in the classes that will make use of these properties. In addition, the configuration files are not checked against any strict syntax so it is always in a correct status (as long as it contains pairs of property names and values in `INI format`).

1.5.2 GNSS block factory

Hence, the application defines a simple accessor class to fetch the configuration pairs of values and passes them to a factory class called GNSSBlockFactory. This factory decides, according to the configuration, which class needs to be instantiated and which parameters should be passed to the constructor. Hence, the factory encapsulates the complexity of blocks' instantiation. With that approach, adding a new block that requires new parameters will be as simple as adding the block class and modifying the factory to be able to instantiate it. This loose coupling between the blocks' implementations and the syntax of the configuration enables extending the application capacities in a high degree. It also allows to produce fully customized receivers, for instance a testbed for acquisition algorithms, and to place observers at any point of the receiver chain.

1.6 Signal Processing plane

GNU Radio's class `gr::basic_block` is the abstract base class for all signal processing blocks, a bare abstraction of an entity that has a name and a set of inputs and outputs. It is never instantiated directly; rather, this is the abstract parent class of both `gr::hier_block2`, which is a recursive container that adds or removes processing or hierarchical blocks to the internal graph, and `gr::block`, which is the abstract base class for all the processing blocks.

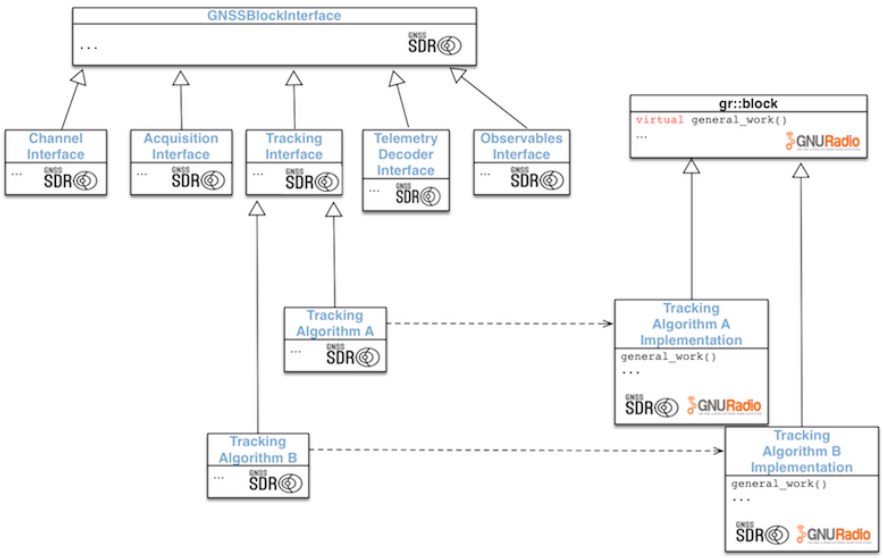


Figure 1.3 Class hierarchy of signal processing blocks

A signal processing flow is constructed by creating a tree of hierarchical blocks, which at any level may also contain terminal nodes that actually implement signal processing functions.

Class `gr::top_block` is the top-level hierarchical block representing a flowgraph. It defines GNU Radio runtime functions used during the execution of the program: run(), start(), stop(), wait(), etc. A a subclass called GNSSBlockInterface is the common interface for all the GNSS-SDR modules. It defines pure virtual methods, that are required to be implemented by a derived class.

Subclassing GNSSBlockInterface, we defined interfaces for the GNSS receiver blocks depicted in the figure above. This hierarchy provides the definition of different algorithms and different implementations, which will be instantiated according to the configuration. This strategy allows multiple implementations sharing a common interface, achieving the objective of decoupling interfaces from implementations: it defines a family of algorithms, encapsulates each one, and makes them interchangeable. Hence, we let the algorithm vary independently from the program that uses it.

1.6.1 Signal Source

The input of a software receiver are the raw bits that come out from the front-end's analog-to-digital converter (ADC). Those bits can be read from a file stored in the hard disk or directly in real-time from a hardware device through USB or Ethernet buses.

The Signal Source module is in charge of implementing the hardware driver, that is, the portion of the code that communicates with the RF front-end and receives the samples coming from the ADC. This communication is usually performed through USB or Ethernet buses. Since real-time processing requires a highly optimized implementation of the whole receiver, this module also allows to read samples from a file stored in a hard disk, and thus processing without time constraints. Relevant parameters of those samples are the intermediate frequency (or baseband I&Q components), the sampling rate and number of bits per sample, that must be specified by the user in the configuration file.

This module also performs bit-depth adaptation, since most of the existing RF front-ends provide samples quantized with 2 or 3 bits, while operations inside the processor are performed on 32- or 64-bit words, depending on its architecture. Although there are implementations of the most intensive computational processes (mainly correlation) that take advantage of specific data types and architectures for the sake of efficiency, the approach is processor-specific and hardly portable. We suggest to keep signal samples in standard data types and letting the compiler select the best library version (implemented using SIMD or any other processor-specific technology) of the required routines for a given processor.

Example: FileSignalSource

The user can configure the receiver for reading from a file, setting in the configuration file the data file location, sample format, and the sampling frequency and intermediate frequency at what the signal was originally captured.

```
;######### SIGNAL_SOURCE CONFIG ############
SignalSource.implementation=File_Signal_Source
SignalSource.filename=/home/user/gnss-sdr/data/my_capture.dat
SignalSource.item_type=gr_complex
SignalSource.sampling_frequency=4000000 ; Sampling frequency in [Hz]
SignalSource.freq=1575420000 ; RF front-end center frequency in [Hz]
```

Example: UhdSignalSource

The user may prefer to use a UHD-compatible RF front-end and try real-time processing. For instance, for a USRP1 + DBSRX daughterboard, use:

```
;######### SIGNAL_SOURCE CONFIG ############
SignalSource.implementation=UHD_Signal_Source
SignalSource.item_type=gr_complex
SignalSource.sampling_frequency=4000000 ; Sampling frequency in [Hz]
SignalSource.freq=1575420000 ; RF front-end center frequency in [Hz]
SignalSource.gain=60 ; Front-end gain in dB
SignalSource.subdevice=B:0 ; UHD subdevice specification (for USRP1 use A:0 or B:0)
```

Other examples are available at `gnss-sdr/conf`.

1.6.2 Signal Conditioner

The signal conditioner is in charge of resampling the signal and delivering a reference sample rate to the downstream processing blocks, acting as a facade between the signal source and the synchronization channels, providing a simplified interface to the input signal. In case of multiband front-ends, this module would be in charge of providing a separated data stream for each band.

1.6.3 Channel

A channel encapsulates all signal processing devoted to a single satellite. Thus, it is a large composite object which encapsulates the Acquisition, Tracking and Decoding of the navigation message modules. As a composite object, it can be treated as a single entity, meaning that it can be easily replicated. Since the number of channels is selectable by the user in the configuration file, this approach helps improving the scalability and maintainability of the receiver.

This module is also in charge of managing the interplay between acquisition and tracking. Acquisition can be initialized in several ways, depending on the prior information available (called cold start when the receiver has no information about its position nor the satellites almanac; warm start when a rough location and the approximate time of day are available, and the receiver has a recently recorded almanac broadcast; or hot start when the receiver was tracking a satellite and the signal line of sight broke for a short period of time, but the ephemeris and almanac data is still valid, or this information is provided by other means), and an acquisition process can finish deciding that the satellite is not present, that longer integration is needed in order to confirm the presence of the satellite, or declaring the satellite present. In the latter case, acquisition process should stop and trigger the tracking module with coarse estimations of the synchronization parameters. The mathematical abstraction used to design this logic is known as finite state machine (FSM), that is a behavior model composed of a finite number of states, transitions between those states, and actions. For the implementation, we used the `Boost.Statechart library`, which provides desirable features such as support for asynchronous state machines, multi-threading, type-safety, error handling and compile-time validation.

The abstract class ChannelInterface represents an interface to a channel GNSS block. Check Channel for an actual implementation.

1.6.3.1 Acquisition

The first task of a GNSS receiver is to detect the presence or absence of in-view satellites. This is done by the acquisition system process, which also provides a coarse estimation of two signal parameters: the frequency shift with respect to the nominal IF frequency, and a delay term which allows the receiver to create a local code aligned with the incoming code. AcquisitionInterface is the common interface for all the acquisition algorithms and their corresponding implementations. Algorithms' interface, that may vary depending on the use of information external to the receiver, such as in Assisted GNSS, is defined in classes referred to as *adapters*. These adapters wrap the GNU Radio blocks interface into a compatible interface expected by AcquisitionInterface. This allows the use of existing GNU Radio blocks derived from `gr::block`, and ensures that newly developed implementations will also be reusable in other GNU Radio-based applications. Moreover, it adds still another layer of abstraction, since each given acquisition algorithm can have different implementations (for instance using different numerical libraries). In such a way, implementations can be continuously improved without having any impact neither on the algorithm interface nor the general acquisition interface.

Check GpsL1CaPcpsAcquisition and GalileoE1PcpsAmbiguousAcquisition for examples of adapters from a Parallel Code Phase Search (PCPS) acquisition block, and pcps_acquisition_cc for an example of a block implementation. The source code of all the available acquisition algorithms is located at:

```
|-gnss-sdr
|---src
|-----algorithms
|-------acquisition
|---------adapters          <- Adapters of the processing blocks to an AcquisitionInterface
|---------gnuradio_blocks   <- Signal processing blocks implementation
```

The user can select a given implementation for the algorithm to be used in each receiver channel, as well as their parameters, in the configuration file:

```
;######### ACQUISITION GLOBAL CONFIG ############

;#dump: Enable or disable the acquisition internal data file logging [true] or [false]
Acquisition.dump=false
;#filename: Log path and filename
Acquisition.dump_filename=./acq_dump.dat
;#item_type: Type and resolution for each of the signal samples. Use only gr_complex in this version.
Acquisition.item_type=gr_complex
;#if: Signal intermediate frequency in [Hz]
Acquisition.if=0
;#sampled_ms: Signal block duration for the acquisition signal detection [ms]
Acquisition.sampled_ms=1
;#implementation: Acquisition algorithm selection for this channel:
Acquisition.implementation=GPS_L1_CA_PCPS_Acquisition
;#threshold: Acquisition threshold
Acquisition.threshold=0.005
;#pfa: Acquisition false alarm probability. This option overrides the threshold option.
;Only use with implementations: [GPS_L1_CA_PCPS_Acquisition] or [Galileo_E1_PCPS_Ambiguous_Acquisition]
Acquisition.pfa=0.0001
;#doppler_max: Maximum expected Doppler shift [Hz]
Acquisition.doppler_max=10000
;#doppler_max: Doppler step in the grid search [Hz]
Acquisition.doppler_step=500

;######### ACQUISITION CHANNELS CONFIG ######
;#The following options are specific to each channel and overwrite the generic options


;######### ACQUISITION CH 0 CONFIG ############
;Acquisition0.implementation=GPS_L1_CA_PCPS_Acquisition
;Acquisition0.threshold=0.005
;Acquisition0.pfa=0.001
;Acquisition0.doppler_max=10000
;Acquisition0.doppler_step=250

;#repeat_satellite: Use only jointly with the satellite PRN ID option. The default value is false
;Acquisition0.repeat_satellite = false

;######### ACQUISITION CH 1 CONFIG ############
;Acquisition1.implementation=GPS_L1_CA_PCPS_Acquisition
;Acquisition1.threshold=0.005
;Acquisition1.pfa=0.001
;Acquisition1.doppler_max=10000
;Acquisition1.doppler_step=250
;Acquisition1.repeat_satellite = false
```

1.6.3.2 Tracking

When a satellite is declared present, the parameters estimated by the acquisition module are then fed to the receiver tracking module, which represents the second stage of the signal processing unit, aiming to perform a local search for accurate estimates of code delay and carrier phase, and following their eventual variations.

Again, a class hierarchy consisting of a TrackingInterface class and subclasses implementing algorithms provides a way of testing different approaches, with full access to their parameters. Check GpsL1CaDllPllTracking or Galileo↩ E1DllPllVemlTracking for examples of adapters, and Gps_L1_Ca_Dll_Pll_Tracking_cc for an example of a signal processing block implementation. There are also available some useful classes and functions for signal tracking; take a look at Correlator, lock_detectors.h, tracking_discriminators.h or tracking_2nd_DLL_filter.h.

The source code of all the available tracking algorithms is located at:

```
|-gnss-sdr
|---src
|-----algorithms
|-------tracking
|---------adapters          <- Adapters of the processing blocks to a TrackingInterface
|---------gnuradio_blocks   <- Signal processing blocks implementation
|---------libs              <- libraries of tracking objects (e.g. correlators, discriminators, and so on)
```

The user can select a given implementation for the algorithm to be used in all the tracking blocks, as well as its parameters, in the configuration file:

```
;######### TRACKING GLOBAL CONFIG ############

;#implementation: Selected tracking algorithm
Tracking.implementation=GPS_L1_CA_DLL_PLL_Tracking
;#item_type: Type and resolution for each of the signal samples. Use only [gr_complex] in this version.
Tracking.item_type=gr_complex

;#sampling_frequency: Signal Intermediate Frequency in [Hz]
Tracking.if=0

;#dump: Enable or disable the Tracking internal binary data file logging [true] or [false]
Tracking.dump=false

;#dump_filename: Log path and filename. Notice that the tracking channel will add "x.dat" where x is the chann
Tracking.dump_filename=./tracking_ch_

;#pll_bw_hz: PLL loop filter bandwidth [Hz]
Tracking.pll_bw_hz=50.0;

;#dll_bw_hz: DLL loop filter bandwidth [Hz]
Tracking.dll_bw_hz=2.0;

;#fll_bw_hz: FLL loop filter bandwidth [Hz]
Tracking.fll_bw_hz=10.0;

;#order: PLL/DLL loop filter order [2] or [3]
Tracking.order=3;

;#early_late_space_chips: correlator early-late space [chips]. Use [0.5]
Tracking.early_late_space_chips=0.5;
```

1.6.3.3 Decoding of the navigation message

Most of GNSS signal links are modulated by a navigation message containing the time the message was transmitted, orbital parameters of satellites (also known as ephemeris) and an almanac (information about the general system health, rough orbits of all satellites in the network as well as data related to error correction). Navigation data bits are structured in words, pages, subframes, frames and superframes. Sometimes, bits corresponding to a single parameter are spread over different words, and values extracted from different frames are required for proper decoding. Some words are for synchronization purposes, others for error control an others contain actual information. There are also error control mechanisms, from parity checks to forward error correction (FEC) encoding and interleaving, depending on the system. All this decoding complexity is managed by a finite state machine implemented with the Boost.Statechart library.

The common interface is TelemetryDecoderInterface. Check GpsL1CaTelemetryDecoder for an example of the GPS L1 NAV message decoding adapter, and gps_l1_ca_telemetry_decoder_cc for an actual implementation of a signal processing block. Configuration example:

```
;######### TELEMETRY DECODER CONFIG ############
TelemetryDecoder.implementation=GPS_L1_CA_Telemetry_Decoder
TelemetryDecoder.dump=false
```

See the Reference Documents for more information about the signal format.

1.6.4 Observables

GNSS systems provide different kinds of observations. The most commonly used are the code observations, also called pseudoranges. The *pseudo* comes from the fact that on the receiver side the clock error is unknown and thus the measurement is not a pure range observation. High accuracy applications also use the carrier phase observations, which are based on measuring the difference between the carrier phase transmitted by the GNSS satellites and the phase of the carrier generated in the receiver. Both observables are computed from the outputs of the tracking module and the decoding of the navigation message. This module collects all the data provided by every tracked channel, aligns all received data into a coherent set, and computes the observables.

The common interface is ObservablesInterface.

Configuration example:

```
;######### OBSERVABLES CONFIG ############
;#implementation: Use [GPS_L1_CA_Observables] for GPS L1 C/A.
Observables.implementation=GPS_L1_CA_Observables

;#dump: Enable or disable the Observables internal binary data file logging [true] or [false]
Observables.dump=false

;#dump_filename: Log path and filename.
Observables.dump_filename=./observables.dat
```

1.6.5 Computation of Position, Velocity and Time

Although data processing for obtaining high-accuracy PVT solutions is out of the scope of GNSS-SDR, we provide a module that can compute a simple least square solution and leaves room for more sophisticated positioning methods. The integration with libraries and software tools that are able to deal with multi-constellation data such as GPSTk or gLAB appears as a viable solution for high performance, completely customizable GNSS receivers.

The common interface is PvtInterface. For instance, in order to use the implementation GpsL1CaPvt, add to the configuration file:

```
;######### PVT CONFIG ############
PVT.implementation=GPS_L1_CA_PVT

;#nmea_dump_filename: NMEA log path and filename
PVT.nmea_dump_filename=./gnss_sdr_pvt.nmea;

;#flag_nmea_tty_port: Enable or disable the NMEA log to a serial TTY port (Can be used with real hardware or v
PVT.flag_nmea_tty_port=true;

;#nmea_dump_devname: serial device descriptor for NMEA logging
PVT.nmea_dump_devname=/dev/pts/4

;#dump: Enable or disable the PVT internal binary data file logging [true] or [false]
PVT.dump=false
```

This implementation allows tuning of the following parameters:

```
PVT.averaging_depth=10 ; Number of PVT observations in the moving average algorithm
PVT.flag_averaging=true ; Enables the PVT averaging between output intervals (arithmetic mean) [true] or [fals
PVT.output_rate_ms=100 ; Period in [ms] between two PVT outputs
PVT.display_rate_ms=500 ; Position console print (std::out) interval [ms].
PVT.dump=false ; Enable or disable the PVT internal binary data file logging [true] or [false]
PVT.dump_filename=./PVT ; Log path and filename without extension.
```

1.7 About the software license

GNSS-SDR is released under the General Public License (GPL) v3, thus securing practical usability, inspection, and continuous improvement by the research community, allowing the discussion based on tangible code and the analysis of results obtained with real signals. The GPL implies that:

- Copies may be distributed free of charge or for money, but the source code has to be shipped or provided free of charge (or at cost price) on demand. The receiver of the source code has the same rights meaning he can share copies free of charge or resell.
- The licensed material may be analyzed or modified.
- Modified material may be distributed under the same licensing terms but **do not** have to be distributed.

That means that modifications only have to be made available to the public if distribution happens. So it is perfectly fine to take the GNSS-SDR source code, modify it heavily and use it in a not distributed application / library. This is how companies like Google can run their own patched versions of Linux for example.

But what this also means is that non-GPL code cannot use GPL code. This means that you cannot modify / use GNSS-SDR, blend it with non-GPL code, and make money with the resulting software. You cannot distribute the resulting software under a non-disclosure agreement or contract. Distributors under the GPL also grant a license for any of their patents practiced by the software, to practice those patents in GPL software. You can sell a device that runs with GNSS-SDR, but if you distribute the code, it has to remain under GPL.

1.8 Publications and Credits

If you use GNSS-SDR to produce a research paper or Thesis, we would appreciate if you reference any of these articles to credit the GNSS-SDR project:

- C. Fernández-Prades, J. Arribas, L. Esteve, D. Pubill, P. Closas, *An Open Source Galileo E1 Software Receiver*, in Proc. of the 6th ESA Workshop on Satellite Navigation Technologies (NAVITEC 2012), ESTEC, Noordwijk, The Netherlands, Dec. 2012.
- J. Arribas, *GNSS Array-based Acquisition: Theory and Implementation*, PhD Thesis, Universitat Politècnica de Catalunya, Barcelona, Spain, June 2012.
- C. Fernández-Prades, J. Arribas, P. Closas, C. Avilés, and L. Esteve, *GNSS-SDR: an open source tool for researchers and developers*, in Proc. of the ION GNSS 2011 Conference, Portland, Oregon, Sept. 19-23, 2011.
- C. Fernández-Prades, C. Avilés, L. Esteve, J. Arribas, and P. Closas, *Design patterns for GNSS software receivers*, in Proc. of the 5th ESA Workshop on Satellite Navigation Technologies (NAVITEC'2010), ESTEC, Noordwijk, The Netherlands, Dec. 2010. DOI:10.1109/NAVITEC.2010.5707981

For LaTeX users, these are the BibTeX cites for your convenience:

```
@INPROCEEDINGS{GNSS-SDR12
  author = {C.~{Fern\'{a}ndez--Prades} and J.~Arribas and L.~Esteve and D.~Pubill and P.~Closas},
  title = {An Open Source {G}alileo {E1} Software Receiver},
  booktitle = {Proc. of the 6th ESA Workshop on Satellite Navigation Technologies (NAVITEC'2012)},
  year = {2012},
  address = {ESTEC, Noordwijk, The Netherlands},
  month = {Dec.} }
```

```
@PHDTHESIS{Arribas12,
  author = {J.~Arribas},
  title = {{GNSS} Array-based Acquisition: Theory and Implementation},
  school = {Universitat Polit\'{e}cnica de Catalunya},
  year = {2012},
  address = {Barcelona, Spain},
  month = {June} }


@INPROCEEDINGS{GNSS-SDR11,
 AUTHOR = {C.~{Fern\'{a}ndez--Prades} and J.~Arribas and P.~Closas and C.~Avil\'{e}s and L.~Esteve},
 TITLE = {{GNSS-SDR}: An Open Source Tool For Researchers and Developers},
 BOOKTITLE = {Proc. of the ION GNSS 2011 Conference},
 YEAR = {2011},
 address = {Portland, Oregon},
 month = {Sept.} }


@INPROCEEDINGS{GNSS-SDR10,
 AUTHOR = {C.~{Fern\'{a}ndez--Prades} and C.~Avil\'{e}s and L.~Esteve and J.~Arribas and P.~Closas},
 TITLE = {Design patterns for {GNSS} software receivers},
 BOOKTITLE = {Proc. of the 5th ESA Workshop on Satellite Navigation Technologies (NAVITEC'2010)},
 YEAR = {2010},
 address = {ESTEC, Noordwijk, The Netherlands},
 month = {Dec.},
 note = {DOI:10.1109/NAVITEC.2010.5707981} }
```

1.9 Ok, now what?

In order to start using GNSS-SDR, you may want to populate `gnss-sdr/data` folder (or anywhere else on your system) with raw data files. By "raw data" we mean the output of a Radio Frequency front-end's Analog-to-↩ Digital converter. GNSS-SDR needs signal samples already in baseband or in passband, at a suitable inteme-diate frequency (on the order of MHz). Prepare your configuration file, and then you are ready for going to the `gnss-sdr/install` folder, running `./gnss-sdr`, and see how the file is processed. Please ask the Devel-oper Team for a signal sample if you need one, and they will do their best ;-)

Another interesting option is working in real-time with a RF front-end. We provide drivers for UHD-compatible hardware (see Signal Source), for the GN3S v2 USB dongle and for some DVB-T USB dongles. Start with a low number of channels and then increase it in order to test how many channels your processor can handle in real-time.

You can find more information at the **GNSS-SDR Documentation page** or directly asking to the **GNSS-SDR Developers mailing list**.

You are also very welcome to contribute to the project, there are many ways to **participate in GNSS-SDR**. If you need some special feature not yet implemented, the Developer Team would love to be hired for developing it. Please do not hesitate to **contact them**.

Enjoy GNSS-SDR!

The Developer Team.

Chapter 2

Reference Documents

2.1 Interface Control Documents

2.1.1 GPS

All the current GPS Interface Control Documents can be downloaded from GPS.gov, the official U.S. Government webpage for GPS.

- GPS L1 and L2C: Global Positioning System Directorate, **Interface Specification IS-GPS-200 Revision H**. September, 2013.
- GPS L1C (available with first Block III launch): Global Positioning System Directorate, **Interface Specification IS-GPS-800 Revision D**. September, 2013.
- GPS L5 (first Block IIF satellite launched on May, 2010): Global Positioning System Directorate, **Interface Specification IS-GPS-705 Revision D**. September, 2013.

2.1.2 GLONASS

Official GLONASS webpage: Information-analytical centre official website.

- Standard Accuracy (ST) signals at L1 and L2: Russian Institute of Space Device Engineering, Global Navigation Satellite System GLONASS. **Interface Control Document. Navigational radiosignal in bands L1, L2. Edition 5.1**, Moscow, Russia, 2008

2.1.3 Galileo

Check the Galileo website of the European Commission and the Galileo website of the European Space Agency. There is a website with Galileo constellation status information from the International GNSS Service.

- Galileo E5, E6, and E1: European GNSS (Galileo) Open Service. **Signal In Space Interface Control Document. Ref: OS SIS ICD, Issue 1.2**, European Commission, Nov. 2015.
- European GNSS (Galileo) Open Service **Signal-In-Space Operational Status Definition**, European Commission, Sept. 2015.

The European Commission is granting free access to the technical information on the future Galileo open service signal, i.e. the specifications manufacturers and developers need to process data received from satellites. This document informs receiver manufacturers, application developers and service providers on how to use the future Galileo system and what they can expect in terms of performance.

2.1.4 BeiDou

Official webpage at beidou.gov.cn

- **BeiDou Navigation Satellite System Signal In Space Interface Control Document**. Open Service Signal (Version 2.0). China Satellite Navigation Office, December 2013.

- **BeiDou Navigation Satellite System Open Service Performance Standard**. (Version 1.0). China Satellite Navigation Office, December 2013.

2.1.5 Satellite Based Augmentation Systems (SBAS)

- **Minimum Operational Performance Standards for Global Positioning System/Wide Area Augmentation System Airborne Equipment, DO-229D**, RTCA, Washington, DC, Dec. 13, 2006. The 'RTCA MOPS D↩ O229D - appendix A' is the reference standard for WAAS/EGNOS application development. RTCA is an advisory committee of the US federal government, and issues standards for civil airborne equipment, among other duties. One such standard is MOPS 229D (Minimum Operational Performance Standards for Global Positioning System/Wide Area Augmentation System Airborne Equipment version D), which describes the implementation of satellite-based augmentation services (SBAS) for receivers designed for civil aviation use. An annex to DO229D contains the specifications for the SBAS signal and message. The RTCA provides regular updates to these standards. MOPS 229D is available for a fee from the RTCA website.

- **Global Positioning System Wide Area Augmentation System (WAAS) Performance Standard, 1st Edition**, Department of Transportation and Federal Aviation Administration, Oct. 31, 2008. This document defines the levels of performance the U.S. Government makes available to users of the GPS SPS augmented by the Wide Area Augmentation System.

- **EGNOS Open Service (OS) Service Definition Document. Ref: EGN-SDD OS, Revision 2.2**, European GNSS Agency (GSA), Feb. 12, 2015. This is a complementary document to the RTCA DO229D, mentioned above. It describes the scope of services provided by the EGNOS Open Service to be used by end-users or Application Specific Service Providers. It details the general conditions relating to the use of the EGNOS service, a technical description of the Signal-in-Space (SIS), the reference receiver, environmental conditions, the service performance achieved and aspects relating to service provision.

- **EGNOS Safety of Life Service Definition Document. Ref: EGN-SDD SoL, Revision 3.0**, European GNSS Agency (GSA), Sep. 22, 2015. The EGNOS Safety of Life (SoL) Service is provided openly and is freely accessible without any direct charge and is tailored to safety-critical transport applications in various domains, in particular for aviation applications. The service is thus compliant with the aviation APV-I (Approach with Vertical Guidance) requirements, as defined by ICAO in Annex 10, but may support also applications in other SoL domains.

- **EGNOS Data Access Service (EDAS) Service Definition Document, Rf: EG↩ N-SDD EDAS, V2.1**, European GNSS Agency (GSA), Dec. 19, 2014.

More information about EGNOS can be found through the EGNOS Portal.

2.2 Other Standards

2.2.1 RINEX

The final output of a navigation receiver is usually its position, speed or other related physical quantities. However, the calculation of those quantities are based on a series of measurements from one or more satellite constellations. Although receivers calculate positions in real time, in many cases it is interesting to store intermediate measures for later post-processing. RINEX is the standard format that allows the management and disposal of the measures generated by a receiver, as well as their off-line processing by a multitude of applications.

- The most common version at present is **RINEX: The Receiver Independent Exchange Format Version 2.11**, which enables storage of measurements from pseudorange, carrier-phase and Doppler systems for GPS or GLONASS, along with data from EGNOS and WAAS satellite based augmentation systems (SBAS).

- The most recent version is **RINEX: The Receiver Independent Exchange Format Version 3.01** published in June, 2009. It includes Galileo and improves the handling of multi-constellation data files.

- There is also available the **RINEX Extensions to Handle Clock Information**, published in September, 2010.

2.2.2 NMEA

The National Marine Electronics Association released the NMEA 0183 Interface Standard, which defines electrical signal requirements, data transmission protocol and time, and specific sentence formats for a 4800-baud serial data bus. The standard is available for purchase.

2.2.3 KML

KML is an XML language focused on geographic visualization, including annotation of maps and images. Geographic visualization includes not only the presentation of graphical data on the globe, but also the control of the user's navigation in the sense of where to go and where to look. Google submitted KML (formerly Keyhole Markup Language) to the Open Geospatial Consortium (OGC) to be evolved within the OGC consensus process with the following goal: KML Version 2.2 has been adopted as an OGC implementation standard.

- Open Geospatial Consortium, Inc., **OGC KML Version 2.2.0**, April 2008.

2.2.4 C++ Standards

In 1998, the C++ standards committee (the ISO/IEC JTC1/SC22/WG21 working group) standardized C++ and published the international standard ISO/IEC 14882:1998 (informally known as C++98). A technical corrigendum was approved in 2003, and the standard was published again as the ISO/IEC 14882:2003.

Published ISO and IEC standards can be purchased from a member body of ISO or IEC. Free copies of the C++ standard Committee Drafts were made public before the official standard was released.

In 2005, a technical report, called the Library Technical Report 1 (often known as TR1 for short), was released. While not an official part of the standard, it specified a number of extensions to the standard library, which were expected to be included in the next version of C++. The linked document is officially a draft, but that is due only to procedural issues; the content did not change. Scott Meyers provides more TR1 information.

Some ISO/IEC standards are publicly available, for instance: ISO/IEC TR 18015:2006 Technical Report on C++ Performance.

The standard for the next version of the language (previously known as C++0x) was finally published with the name of C++11 in September, 2011, as the ISO/IEC 14882:2011 Standard. GCC, the GNU Compiler Collection, provides partial C++11 support. Bjarne Stroustrup maintains a C++11 FAQ.

- The most recent public draft of the **Standard for Programming Language C++** was published in Feb. 2011.

2.2.5 Positioning protocols in wireless communication networks

Cellular industry location standards first appeared in the late 1990s, with the 3rd generation partnership project (3GPP) radio resource location services protocol (RRLP) technical specification 44.031 positioning protocol for GSM networks. Today, RRLP is the de facto standardized protocol to carry GNSS assistance data to GNSS-enabled mobile devices, and the term "3GPP specification" now covers all GSM (including GPRS and EDGE), W-CDMA and LTE (including LTE-A) specifications. Precisely, the label "LTE-A" is applied to networks compliant with LTE Release 10 and beyond, which fulfill the requirements issued by the International Telecommunication Union Radiocommunication Sector (ITU-R) in the global standard for international mobile telecommunications (IMT Advanced, also referred to as 4G) access technologies.

Control plane protocols:

- Radio Resource LCS Protocol (RRLP): **3GPP Technical Specification 44.031**.
- LTE Positioning Protocol (LPP): **3GPP Technical Specification 36.355**.

User plane protocols:

- Open Mobile Alliance (OMA), **Secure User Plane Location Architecture Version 1 (SUPL 1.0)**, June 2007.
- Open Mobile Alliance (OMA), **Secure User Plane Location Architecture Version 2 (SUPL 2.0)**, April 2012.

LTE Release 9 introduced extension hooks in LPP messages, so that the bodies external to 3GPP could extend the LPP feature set. OMA LPP extensions (LPPe), supported in SUPL 3.0, build on top of the 3GPP LPP reusing its procedures and data types. Check the OMA Location Working Group (WG) webpage for updated information about LPP Extensions (LPPe) Specification.

- The **OMA Mobile Location Protocol (MLP) V3.1** is an application-level protocol for getting the position of mobile stations (mobile phones, wireless personal digital assistants, etc.) independent of underlying network technology. The MLP serves as the interface between a Location Server and a Location Services (LCS) Client. This specification defines the core set of operations that a Location Server should be able to perform.

Chapter 3

Signal model

3.1 GNSS signal model

This page describes signals transmitted by GNSS space vehicles. Signal models are mathematical representations of the electromagnetic waves that are exciting the receiver's antenna. The current induced by those waves is then amplified, filtered and downconverted to a suitable frequency (can be at some intermediate frequency or directly to baseband), and then converted to 0s and 1s by the Analog-to-Digital Converter (ADC). That is the job of the Radio Frequency front-end, which at its output delivers a stream of digital samples. Those samples constitute the input of a software receiver, so for GNSS-SDR the signal models described below can be seen as *the rules of the game*.

GNSS' space vehicles are modern versions of lighthouses, but with better visibility. Each satellite is a reference point, and if we know our distance to several reference points, we can compute our location, just as mariners do when they see a couple of lighthouses. For each in-view satellite i of system s, we can write:

$$\rho_i = \sqrt{\left(x_i^{\mathsf{Tx}} - x\right)^2 + \left(y_i^{\mathsf{Tx}} - y\right)^2 + \left(z_i^{\mathsf{Tx}} - z\right)^2} + c\Delta t^{(s)} + \sigma_e, \tag{3.1}$$

where $\left(x_i^{\mathsf{Tx}}, y_i^{\mathsf{Tx}}, z_i^{\mathsf{Tx}}\right)$ is the satellite's position (known from the navigation message), (x, y, z) the receiver's position, and σ_e gathers other sources of error. Since the receiver needs to estimate its own 3D position (three spatial unknowns) and its clock deviation with respect to the satellites' time basis, at least $3 + N_s$ satellites must be seen by the receiver at the same time, where N_s is the number of different navigation systems available (in-view) at a given time. Each received satellite signal, once synchronized and demodulated at the receiver, defines one equation such as the one defined above, forming a set of nonlinear equations that can be solved algebraically by means of the Bancroft algorithm or numerically, resorting to multidimensional Newton-Raphson and weighted least square methods. When *a priori* information is added we resort to Bayesian estimation, a problem that can be solved recursively by a Kalman filter or any of its variants. The problem can be further expanded by adding other unknowns (for instance, parameters of ionospheric and tropospheric models), sources of information from other systems, mapping information, and even motion models of the receiver. In the design of multi-constellation GNSS receivers, the vector of unknowns can also include the receiver clock offset with respect to each system in order to take advantage of a higher number of in-view satellites and using them jointly in the navigation solution, therefore increasing accuracy.

The analytic representation of a signal received from a GNSS satellite can be generically expressed as

$$r(t) = \alpha(t) s_T\left(t - \tau(t)\right) e^{-j2\pi f_d(t)} e^{j2\pi f_c t} + n(t)\ , \tag{3.2}$$

where $\alpha(t)$ is the amplitude, $s_T(t)$ is the complex baseband transmitted signal, $\tau(t)$ is the time-varying delay, $f_d(t) = f_c \tau(t)$ is the Doppler shift, f_c is the carrier frequency, and $n(t)$ is a noise term. These signals arrive to the Earth's surface at extremely low power (e.g. -158.5 dBW for GPS L1 C/A-code, -157 dBW for Galileo E1), well below the noise floor. In order to estimate its distances to satellites, the receiver must correlate time-aligned replicas of the corresponding pseudorandom code with the incoming signal, in a process called *despreading* that

provides processing gain only to the signal of interest. After a coarse and fine estimation stages of the synchronization parameters (usually known as acquisition and tracking, respectively), signal processing output is in form of *observables*:

i) the pseudorange (code) measurement, equivalent to the difference of the time of reception (expressed in the time frame of the receiver) and the time of transmission (expressed in the time frame of the satellite) of a distinct satellite signal; and optionally

ii) the carrier-phase measurement, actually being a measurement on the beat frequency between the received carrier of the satellite signal and a receiver-generated reference frequency. Carrier phase measurements are ambiguous, in the sense that the integer number of carrier wavelengths between satellite and the receiver's antenna is unknown. Techniques such as Least-square AMBiguity Decorrelation Approach (LAMBDA) or Multi Carrier Ambiguity Resolution (MCAR) can be applied to resolve such ambiguity and provide an accurate estimation of the distance between the satellite and the receiver.

Then, depending on the required accuracy, the navigation solution can range from pseudorange-only, computationally low demanding, and limited accuracy least squares methods to sophisticated combinations of code and phase observables at different frequencies for high demanding applications such as surveying, geodesy, and geophysics.

Next sections provide brief descriptions of the space segment of different GNSSs and their broadcast signal structures accessible by civilians.

3.1.1 Global Positioning System (GPS) signal in space

The Global Positioning System (GPS) is a space-based radio-navigation system owned by the United States Government (USG) and operated by the United States Air Force (USAF). GPS provides positioning and timing services to military and civilian users on a continuous, worldwide basis. Two GPS services are provided: the Precise Positioning Service (PPS), available primarily to the military of the United States and its allies, and the Standard Positioning Service (SPS) open to civilian users.

- **GPS L1**. Defined at **Interface Specification IS-GPS-200 Revision F**, this band is centered at $f_{\text{GPS L1}} = 1575.42$ MHz. The complex baseband transmitted signal can be written as

$$s_T^{(\text{GPS L1})}(t) = e_{L1I}(t) + j e_{L1Q}(t) \, , \tag{3.3}$$

with

$$e_{L1I}(t) = \sum_{l=-\infty}^{\infty} D_{\text{NAV}}\Big[[l]_{204600}\Big] \oplus C_{\text{P(Y)}}\Big[|l|_{L_{\text{P(Y)}}}\Big] p(t - lT_{c,\text{P(Y)}}) \, , \tag{3.4}$$

$$e_{L1Q}(t) = \sum_{l=-\infty}^{\infty} D_{\text{NAV}}\Big[[l]_{20460}\Big] \oplus C_{\text{C/A}}\Big[|l|_{1023}\Big] p(t - lT_{c,\text{C/A}}) \, , \tag{3.5}$$

where $\oplus$ is the exclusive-or operation (modulo-2 addition), $|l|_L$ means l modulo L, $[l]_L$ means the integer part of $\frac{l}{L}$, D_{NAV} is the GPS navigation message bit sequence, transmitted at 50 bps, $T_{c,\text{P(Y)}} = \frac{1}{10.23}$ μs, $T_{c,\text{C/A}} = \frac{1}{1.023}$ μs, $L_{\text{P(Y)}} = 6.1871 \cdot 10^{12}$, and $p(t)$ is a rectangular pulse of a chip-period duration centered at $t = 0$ and filtered at the transmitter. According to the chip rate, the binary phase-shift keying modulations in the equations above are denoted as BPSK(10) and BPSK(1), respectively. The precision P codes (named Y codes whenever the anti-spoofing mode is activated, encrypting the code and thus denying non-↩ U.S. military users) are sequences of 7 days in length. Regarding the modernization plans for GPS, it is worthwhile to mention that there is a new civilian-use signal planned, called L1C and defined at **Interface Specification IS-GPS-800 Revision B**, to be broadcast on the same L1 frequency that currently contains the C/A signal. The L1C will be available with first Block III launch, currently scheduled for 2013. The implementation will provide C/A code to ensure backward compatibility.

- **GPS L2C**. Defined at **Interface Specification IS-GPS-200 Revision F**, is only available on Block IIR-M and subsequent satellite blocks. Centered at $f_{\text{GPS L2}} = 1227.60$ MHz, the signal structure is the same than in (eq:GPSL1}), with the precision code in the In-phase component, just as in (eq:L1CAI}) but with an optional presence of the navigation message D_{NAV}. For the Quadrature-phase component, three options are defined:

$$e_{L2CQ}(t) = \sum_{l=-\infty}^{\infty} D_{\text{CNAV}}\left[[l]_{10230}\right] \oplus \left(C_{\text{CL}}\left[|l|_{L_{\text{CL}}}\right] p_{1/2}\left(t - lT_{c,L2C}\right) + \right. \tag{3.6}$$

$$\left. + C_{\text{CM}}\left[|l|_{L_{\text{CM}}}\right] p_{1/2}\left(t - \left(l + \frac{3}{4}\right) T_{c,L2C}\right)\right), \tag{3.7}$$

$$e_{L2CQ}(t) = \sum_{l=-\infty}^{\infty} D_{\text{NAV}}\left[[l]_{20460}\right] \oplus C_{\text{C/A}}\left[|l|_{1023}\right] p\left(t - lT_{c,\text{C/A}}\right), \text{ or} \tag{3.8}$$

$$e_{L2CQ}(t) = \sum_{l=-\infty}^{\infty} C_{\text{C/A}}\left[|l|_{1023}\right] p(t - lT_{c,\text{C/A}}), \tag{3.9}$$

where $T_{c,L2C} = \frac{1}{511.5}$ ms and $p_{1/2}(t)$ is a rectangular pulse of half chip-period duration, thus time-multiplexing both codes. The civilian long code C_{CL} is $L_{\text{CL}} = 767250$ chips long, repeating every 1.5 s, while the civilian moderate code C_{CM} is $L_{\text{CL}} = 10230$ chips long and its repeats every 20 ms. The CNAV data is an upgraded version of the original NAV navigation message, containing higher precision representation and nominally more accurate data than the NAV data. It is transmitted at 25 bps with forward error correction (FEC) encoding, resulting in 50 sps.

- **GPS L5**. The GPS L5 link, defined at **Interface Specification IS-GPS-705 Revision B**, is only available in Block IIF (first satellite launched on May, 2010) and subsequent satellite blocks. Centered at $f_{\text{GPS L5}} = 1176.45$ MHz, this signal in space can be written as:

$$s_T^{(\text{GPS L5})}(t) = e_{L5I}(t) + je_{L5Q}(t), \tag{3.10}$$

$$e_{L5I}(t) = \sum_{m=-\infty}^{+\infty} C_{nh_{10}}\left[|m|_{10}\right] \oplus D_{\text{CNAV}}\left[[m]_{10}\right] \oplus \tag{3.11}$$

$$\oplus \sum_{l=1}^{102300} C_{L5I}\left[|l|_{10230}\right] p(t - mT_{c,nh} - lT_{c,L5}),$$

$$e_{L5Q}(t) = \sum_{m=-\infty}^{+\infty} C_{nh_{20}}\left[|m|_{20}\right] \oplus \sum_{l=1}^{102300} C_{L5Q}\left[|l|_{10230}\right] \cdot$$

$$\cdot\, p(t - mT_{c,nh} - lT_{c,L5}), \tag{3.12}$$

where $T_{c,nh} = 1$ ms and $T_{c,L5} = \frac{1}{10.23}$ μs, thus defining a BPSK(10) modulation. Both L5I and L5Q contain synchronization sequences. {itemize}

3.1.2 GLONASS signal in space

The nominal baseline constellation of the Russian Federation's Global Navigation Satellite System (GLONASS) comprises 24 GLONASS-M satellites that are uniformly deployed in three roughly circular orbital planes at an inclination of 64.8^o to the equator. The altitude of the orbit is $19,100$ km. The orbit period of each satellite is 11 hours, 15 minutes, and 45 seconds. The orbital planes are separated by 120° right ascension of the ascending node. Eight satellites are equally spaced in each plane with 45° argument of latitude. Moreover, the orbital planes have an argument of latitude displacement of 15° relative to each other.

GLONASS civil signal-in-space is defined at **Interface Control Document. Navigational radiosignal in bands L1, L2. Edition 5.1**. This system makes use of a frequency-division multiple access (FDMA) signal structure, transmitting in two bands: $f_{GLOL1}^{(k)} = 1602 + k \cdot 0.5625$ MHz and $f_{GLOL2}^{(k)} = 1246 + k \cdot 0.4375$ MHz, where $k \in \{-7, -6, \cdots, 5, 6\}$ is the channel number. Satellites in opposite points of an orbit plane transmit signals on equal frequencies, as these satellites will never be in view simultaneously by a ground-based user.

- **GLONASS L1**. Two kind of signals are transmitted: a standard precision (SP) and an obfuscated high precision (HP) signal. The complex baseband transmitted signal can be written as

$$s_T^{(\text{GLO L1})}(t) = e_{L1I}(t) + j e_{L1Q}(t) \ , \tag{3.13}$$

with BPSK(5) and BPSK(0.5) modulations:

$$e_{L1I}(t) = \sum_{l=-\infty}^{\infty} D_{\text{GNAV}}\Big[[l]_{102200}\Big] \oplus C_{\text{HP}}\Big[|l|_{L_{\text{HP}}}\Big] p(t - lT_{c,\text{HP}}) \ , \tag{3.14}$$

$$e_{L1Q}(t) = \sum_{l=-\infty}^{\infty} D_{\text{GNAV}}\Big[[l]_{10220}\Big] \oplus C_{\text{SP}}\Big[|l|_{511}\Big] p(t - lT_{c,\text{SP}}) \ , \tag{3.15}$$

where $T_{c,\text{HP}} = \frac{1}{5.11}$ μs, $T_{c,\text{SP}} = \frac{1}{0.511}$ μs, and $L_{\text{HP}} = 3.3554 \cdot 10^7$. The navigation message D_{GNAV} is transmitted at 50 bps. Details of its content and structure, as well as the generation of the C_{SP} code, can be found at the ICD. The usage of the HP signal should be agreed with the Russian Federation Defense Ministry, and no more details have been disclosed.

- **GLONASS L2**. Beginning with the second generation of satellites, called GLONASS-M and first launched in 2001, a second civil signal is available using the same SP code than the one in the L1 band.

The use of FDMA techniques, in which the same code is used to broadcast navigation signals on different frequencies, and the placement of civil GLONASS transmissions on frequencies close to 1600 MHz, well above the GPS L1 band, have complicated the design of combined GLONASS/GPS receivers, particularly low-cost equipment for mass-market applications. Future plans of modernization are intended to increase compatibility and interoperability with other GNSS, and include the addition of a code-division multiple access (CDMA) structure, and possibly binary offset carrier (BOC) modulation, beginning with the third civil signal in the L3 band ($1197.648 - 1212.255$ MHz). Russia is implementing the new signals on the next-generation GLONASS-K satellites, with a first prototype successfully launched into orbit on February 26, 2011.

3.1.3 Galileo signal in space

The nominal Galileo constellation comprises a total of 27 operational satellites (plus 3 active spares), that are evenly distributed among three orbital planes inclined at 56° relative to the equator. There are nine operational satellites per orbital plane, occupying evenly distributed orbital slots. Three additional spare satellites (one per orbital plane) complement the nominal constellation configuration. The Galileo satellites are placed in quasi-circular Earth orbits with a nominal semi-major axis of about $30,000$ km and an approximate revolution period of 14 hours. The Control segment full infrastructure will be composed of $30 - 40$ sensor stations, 3 control centers, 9 Mission Uplink stations, and 5 TT&C stations.

Galileo's Open Service is defined at **Signal In Space Interface Control Document. Ref: OS SIS ICD, Issue 1.1**, where the following signal structures are specified:

- **Galileo E1**. This band, centered at $f_{\text{Gal E1}} = 1575.420$ MHz and with a reference bandwidth of 24.5520 MHz, uses the so-called composite binary offset carrier CBOC(6,1, $\frac{1}{11}$) modulation, defined in baseband as:

$$s_T^{(\text{Gal E1})}(t) = \frac{1}{\sqrt{2}}\Big(e_{E1B}(t)\left(\alpha sc_A(t) + \beta sc_B(t)\right) + \tag{3.16}$$

$$- e_{E1C}(t)\left(\alpha sc_A(t) - \beta sc_B(t)\right)\Big) \ , \tag{3.17}$$

where the subcarriers $sc(t)$ are defined as

$$sc_A(t) = \text{sign}\Big(\sin(2\pi f_{s,E1A} t)\Big) \ , \tag{3.18}$$

$$sc_B(t) = \text{sign}\Big(\sin(2\pi f_{s,E1B} t)\Big) \ , \tag{3.19}$$

and $f_{s,E1A} = 1.023$ MHz, $f_{s,E1B} = 6.138$ MHz are the subcarrier rates, $\alpha = \sqrt{\frac{10}{11}}$, and $\beta = \sqrt{\frac{1}{11}}$. Channel B contains the I/NAV type of navigation message, $D_{\text{I/NAV}}$, intended for Safety-of-Life (SoL) services:

$$e_{E1B}(t) = \sum_{l=-\infty}^{+\infty} D_{\text{I/NAV}}\left[[l]_{4092}\right] \oplus C_{E1B}\left[|l|_{4092}\right] p(t - lT_{c,E1B}). \tag{3.20}$$

In case of channel C, it is a pilot (dataless) channel with a secondary code, forming a tiered code:

$$\begin{aligned} e_{E1C}(t) = &\sum_{m=-\infty}^{+\infty} C_{E1Cs}\left[|m|_{25}\right] \oplus \sum_{l=1}^{4092} C_{E1Cp}\left[l\right] \cdot \\ &\cdot p(t - mT_{c,E1Cs} - lT_{c,E1Cp}) \ , \end{aligned} \tag{3.21}$$

with $T_{c,E1B} = T_{c,E1Cp} = \frac{1}{1.023}$ μs and $T_{c,E1Cs} = 4$ ms. The C_{E1B} and C_{E1Cp} primary codes are pseudorandom memory code sequences defined at Annex C.7 and C.8 of OS SIS ICD. The binary sequence of the secondary code C_{E1Cs} is 0011100000001010110110010. This band also contains another component, Galileo E1A, intended for the Public Regulated Service (PRS). It uses a BOC(15,2.5) modulation with cosine-shaped subcarrier $f_{s,E1A} = 15.345$ MHz and $T_{c,E1A} = \frac{1}{2.5575}$ μs. The PRS spreading codes and the structure of the navigation message have not been made public.

- **Galileo E6**. Intended for the Commercial Service and centered at $f_{\text{Gal E6}} = 1278.750$ MHz, this band provides pilot and data components

$$s_T^{(\text{Gal E6})}(t) = \frac{1}{\sqrt{2}} \left(e_{E6B}(t) - e_{E6C}(t)\right) \ , \tag{3.22}$$

$$\begin{aligned} e_{E6B}(t) = &\sum_{m=-\infty}^{+\infty} D_{\text{C/NAV}}\left[[l]_{5115}\right] \oplus C_{E6B}\left[|l|_{L_{E6B}}\right] \cdot \\ &\cdot p(t - lT_{c,E6}), \end{aligned} \tag{3.23}$$

$$\begin{aligned} e_{E6C}(t) = &\sum_{m=-\infty}^{+\infty} C_{E6Cs}\left[|m|_{100}\right] \oplus \sum_{l=1}^{L_{E6C}} C_{E6Cp}\left[l\right] \cdot \\ &\cdot p(t - mT_{c,E6s} - lT_{c,E6p}), \end{aligned} \tag{3.24}$$

where $D_{\text{C/NAV}}$ is the C/NAV navigation data stream, which is modulated with the encrypted ranging code C_{E6B} with chip period $T_{c,E6} = \frac{1}{5.115}$ μs, thus being a BPSK(5) modulation. Codes C_{E6B} and primary codes C_{E6Cs} and their respective lengths, L_{E6B} and L_{E6C}, have not been published. The secondary codes for the pilot component, C_{E6Cs}, are available at the OS SIS ICD. The receiver reference bandwidth for this signal is 40.920 MHz. This band also contains another component, Galileo E6A, intended for PRS.

- **Galileo E5**. Centered at $f_{\text{Gal E5}} = 1191.795$ MHz and with a total bandwidth of 51.150 MHz, its signal structure deserves some analysis. The AltBOC modulation can be generically expressed as

$$s^{\text{AltBOC}}(t) = x_1(t)v^*(t) + x_2(t)v(t) \ , \tag{3.25}$$

where $v(t) = \frac{1}{\sqrt{2}} \left(\text{sign}\left(\cos(2\pi f_s t)\right) + j\text{sign}\left(\sin(2\pi f_s t)\right)\right)$ is the single side-band subcarrier, f_s is the subcarrier frequency, $(\cdot)^*$ stands for the conjugate operation, and $x_1(t)$ and $x_2(t)$ are QPSK signals. The resulting waveform does not exhibit constant envelope. In case of Galileo, the need for high efficiency of the satellites' onboard High Power Amplifier (HPA) has pushed a modification on the signal in order to make it envelope-constant and thus use the HPA at saturation. This can be done by adding some inter-modulation products to the expression above, coming up with the following definition:

$$\begin{aligned} s_T^{(\text{Gal E5})}(t) = &e_{E5a}(t)ssc_s^*(t) + e_{E5b}(t)ssc_s(t) + \\ &+ \bar{e}_{E5a}(t)ssc_p^*(t) + \bar{e}_{E5b}(t)ssc_p(t) \ , \end{aligned} \tag{3.26}$$

where the single and product side-band signal subcarriers are

$$ssc_s(t) = sc_s(t) + jsc_s\left(t - \frac{T_s}{4}\right), \tag{3.27}$$

$$ssc_p(t) = sc_p(t) + jsc_p\left(t - \frac{T_s}{4}\right), \tag{3.28}$$

and

$$e_{E5a}(t) = e_{E5aI}(t) + je_{E5aQ}(t), \tag{3.29}$$
$$e_{E5b}(t) = e_{E5bI}(t) + je_{E5bQ}(t), \tag{3.30}$$
$$\bar{e}_{E5a}(t) = \bar{e}_{E5aI}(t) + j\bar{e}_{E5aQ}(t), \tag{3.31}$$
$$\bar{e}_{E5b}(t) = \bar{e}_{E5bI}(t) + j\bar{e}_{E5bQ}(t), \tag{3.32}$$
$$\bar{e}_{E5aI}(t) = e_{E5aQ}(t)e_{E5bI}(t)e_{E5bQ}(t), \tag{3.33}$$
$$\bar{e}_{E5aQ}(t) = e_{E5aI}(t)e_{E5bI}(t)e_{E5bQ}(t), \tag{3.34}$$
$$\bar{e}_{E5bI}(t) = e_{E5bQ}(t)e_{E5aI}(t)e_{E5aQ}(t), \tag{3.35}$$
$$\bar{e}_{E5bQ}(t) = e_{E5bI}(t)e_{E5aI}(t)e_{E5aQ}(t). \tag{3.36}$$

The signal components are defined as

$$e_{E5aI}(t) = \sum_{m=-\infty}^{+\infty} C_{E5aIs}\Big[|m|_{20}\Big] \oplus \sum_{l=1}^{10230} C_{E5aIp}\Big[l\Big] \oplus \tag{3.37}$$
$$\oplus\, D_{\text{F/NAV}}\Big[[l]_{204600}\Big]p(t - mT_{c,E5s} - lT_{c,E5p}), \tag{3.38}$$
$$e_{E5aQ}(t) = \sum_{m=-\infty}^{+\infty} C_{E5aQs}\Big[|m|_{100}\Big] \oplus \sum_{l=1}^{10230} C_{E5aQp}\Big[l\Big] \cdot \tag{3.39}$$
$$\cdot\, p(t - mT_{c,E5s} - lT_{c,E5p}), \tag{3.40}$$
$$e_{E5bI}(t) = \sum_{m=-\infty}^{+\infty} C_{E5bIs}\Big[|m|_{4}\Big] \oplus \sum_{l=1}^{10230} C_{E5aIp}\Big[l\Big] \oplus \tag{3.41}$$
$$\oplus\, D_{\text{I/NAV}}\Big[[l]_{40920}\Big]p(t - mT_{c,E5s} - lT_{c,E5p}), \tag{3.42}$$
$$e_{E5bQ}(t) = \sum_{m=-\infty}^{+\infty} C_{E5bQs}\Big[|m|_{100}\Big] \oplus \sum_{l=1}^{10230} C_{E5bQp}\Big[l\Big] \cdot \tag{3.43}$$
$$\cdot\, p(t - mT_{c,E5s} - lT_{c,E5p}), \tag{3.44}$$

where $T_{c,E5s} = 1$ ms and $T_{c,E5p} = \frac{1}{10.23}$ μs. Channel A contains the F/NAV type of navigation message, $D_{\text{F/NAV}}$, intended for the Open Service. The I/NAV message structures for the E5bI and E1B signals use the same page layout. Only page sequencing is different, with page swapping between both components in order to allow a fast reception of data by a dual frequency receiver. The single subcarrier $sc_s(t)$ and the product subcarrier $sc_p(t)$ are defined as:

$$sc_s(t) = \frac{\sqrt{2}}{4}\text{sign}\left(\cos\left(2\pi f_s t - \frac{\pi}{4}\right)\right) + \tag{3.45}$$
$$+ \frac{1}{2}\text{sign}\Big(\cos\left(2\pi f_s t\right)\Big) +$$
$$+ \frac{\sqrt{2}}{4}\text{sign}\left(\cos\left(2\pi f_s t + \frac{\pi}{4}\right)\right), \tag{3.46}$$

$$sc_p(t) = -\frac{\sqrt{2}}{4}\text{sign}\left(\cos\left(2\pi f_s t - \frac{\pi}{4}\right)\right) + \tag{3.47}$$
$$+ \frac{1}{2}\text{sign}\Big(\cos\left(2\pi f_s t\right)\Big) +$$
$$- \frac{\sqrt{2}}{4}\text{sign}\left(\cos\left(2\pi f_s t + \frac{\pi}{4}\right)\right), \tag{3.48}$$

with a subcarrier frequency of $f_s = 15.345$ MHz, thus defining an AltBOC(15,10) modulation. The QPSK(10) signal $e_{E5a}(t)$ defined above is shifted to $f_{\text{Gal E5a}} \doteq f_{\text{Gal E5}} - f_s = 1176.450$ MHz, while $e_{E5b}(t)$ is shifted to $f_{\text{Gal E5b}} \doteq f_{\text{Gal E5}} + f_s = 1207.140$ MHz. Thus, we can bandpass filter around $f_{\text{Gal E5a}}$ and get a good approximation of a QPSK(10) signal, with very low energy components of $e_{E5b}(t)$, $\bar{e}_{E5a}(t)$, and $\bar{e}_{E5b}(t)$:

$$s_T^{(\text{Gal E5a})}(t) \simeq e_{E5aI}(t) + j e_{E5aQ}(t). \tag{3.49}$$

The same applies to $e_{E5b}(t)$, allowing an independent reception of two QPSK(10) signals and thus requiring considerably less bandwidth than the processing of the whole E5 band.

3.1.4 Reference

This text is an except of the following paper:

- C. Fernández-Prades, L. Lo Presti, E. Falleti, *Satellite Radiolocalization From GPS to GNSS and Beyond: Novel Technologies and Applications for Civil Mass-Market*. Proceedings of the IEEE. Vol 99, No. 11, pp. 1882-1904. November, 2011. DOI: 10.1109/JPROC.2011.2158032

Chapter 4

Todo List

Member ALPHA_0 ({{69, 8}})

read all pages of subframe 4

Member T_OA ({{69, 8}})

read all pages of subframe 5

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

block
- channel_msg_receiver_cc . . . 61
- direct_resampler_conditioner_cb . . . 75
- direct_resampler_conditioner_cc . . . 76
- direct_resampler_conditioner_cs . . . 77
- galileo_e1_dll_pll_veml_tracking_cc . . . 87
- galileo_e1_observables_cc . . . 89
- galileo_e1_pvt_cc . . . 90
- Galileo_E1_Tcp_Connector_Tracking_cc . . . 91
- galileo_e1b_telemetry_decoder_cc . . . 92
- Galileo_E5a_Dll_Pll_Tracking_cc . . . 93
- galileo_e5a_noncoherentIQ_acquisition_caf_cc . . . 94
- galileo_e5a_telemetry_decoder_cc . . . 98
- galileo_pcps_8ms_acquisition_cc . . . 127
- gps_l1_ca_dll_pll_c_aid_tracking_cc . . . 213
- gps_l1_ca_dll_pll_c_aid_tracking_sc . . . 213
- Gps_L1_Ca_Dll_Pll_Tracking_cc . . . 214
- Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc . . . 215
- gps_l1_ca_observables_cc . . . 217
- gps_l1_ca_pvt_cc . . . 218
- Gps_L1_Ca_Tcp_Connector_Tracking_cc . . . 219
- gps_l1_ca_telemetry_decoder_cc . . . 220
- gps_l2_m_dll_pll_tracking_cc . . . 221
- gps_l2c_telemetry_decoder_cc . . . 222
- hybrid_observables_cc . . . 285
- hybrid_pvt_cc . . . 285
- pcps_acquisition_cc . . . 308
- pcps_acquisition_fine_doppler_cc . . . 312
- pcps_acquisition_sc . . . 315
- pcps_assisted_acquisition_cc . . . 319
- pcps_cccwsr_acquisition_cc . . . 323
- pcps_multithread_acquisition_cc . . . 326
- pcps_opencl_acquisition_cc . . . 330
- pcps_quicksync_acquisition_cc . . . 334
- pcps_tong_acquisition_cc . . . 337

sbas_l1_telemetry_decoder_cc . . . 366
signal_generator_c . . . 372
cl_fft_plan . . . 64
clFFT_Complex . . . 64
clFFT_Dim3 . . . 65
clFFT_SplitComplex . . . 65
concurrent_map< Data > . . . 67
concurrent_queue< Data > . . . 67
ConfigurationInterface . . . 68
FileConfiguration . . . 79
InMemoryConfiguration . . . 294
ControlMessage . . . 69
ControlMessage_ . . . 69
ControlMessageFactory . . . 70
ControlThread . . . 71
cpu_multicorrelator . . . 73
cpu_multicorrelator_16sc . . . 73
cuda_multicorrelator . . . 74
Fast_Correction . . . 78
FrontEndCal . . . 84
Galileo_Almanac . . . 85
Galileo_Ephemeris . . . 99
Galileo_Fnav_Message . . . 107
Galileo_Iono . . . 112
Galileo_Navigation_Message . . . 114
Galileo_Utc_Model . . . 130
GeoJSON_Printer . . . 162
Gnss_Satellite . . . 163
gnss_sdr_supl_client . . . 165
Gnss_Signal . . . 168
Gnss_Synchro . . . 169
GNSSBlockFactory . . . 173
GNSSBlockInterface . . . 174
AcquisitionInterface . . . 53
GalileoE1Pcps8msAmbiguousAcquisition . . . 136
GalileoE1PcpsAmbiguousAcquisition . . . 139
GalileoE1PcpsCccwsrAmbiguousAcquisition . . . 142
GalileoE1PcpsQuickSyncAmbiguousAcquisition . . . 145
GalileoE1PcpsTongAmbiguousAcquisition . . . 148
GalileoE5aNoncoherentIQAcquisitionCaf . . . 155
GpsL1CaPcpsAcquisition . . . 247
GpsL1CaPcpsAcquisitionFineDoppler . . . 250
GpsL1CaPcpsAssistedAcquisition . . . 253
GpsL1CaPcpsMultithreadAcquisition . . . 256
GpsL1CaPcpsOpenClAcquisition . . . 259
GpsL1CaPcpsQuickSyncAcquisition . . . 262
GpsL1CaPcpsTongAcquisition . . . 265
GpsL2MPcpsAcquisition . . . 279
ArraySignalConditioner . . . 54
BeamformerFilter . . . 56
ByteToShort . . . 58
ChannelInterface . . . 63
Channel . . . 59
DirectResamplerConditioner . . . 77
FileSignalSource . . . 80
FirFilter . . . 81
FlexibandSignalSource . . . 82
FreqXlatingFirFilter . . . 83

GenSignalSource . . . 160
Gn3sSignalSource . . . 162
IbyteToCbyte . . . 289
IbyteToComplex . . . 290
IbyteToCshort . . . 291
IshortToComplex . . . 297
IshortToCshort . . . 298
NsrFileSignalSource . . . 304
ObservablesInterface . . . 305
GalileoE1Observables . . . 134
GalileoE5aObservables . . . 158
GpsL1CaObservables . . . 246
GpsL2CObservables . . . 275
HybridObservables . . . 287
OsmosdrSignalSource . . . 306
Pass_Through . . . 307
PvtInterface . . . 347
GalileoE1Pvt . . . 151
GpsL1CaPvt . . . 268
HybridPvt . . . 288
RawArraySignalSource . . . 347
RtlTcpSignalSource . . . 362
SignalConditioner . . . 373
SignalGenerator . . . 375
SpirFileSignalSource . . . 376
TelemetryDecoderInterface . . . 378
GalileoE1BTelemetryDecoder . . . 132
GalileoE5aTelemetryDecoder . . . 159
GpsL1CaTelemetryDecoder . . . 274
GpsL2CTelemetryDecoder . . . 276
SbasL1TelemetryDecoder . . . 370
TrackingInterface . . . 384
GalileoE1DllPllVemlTracking . . . 133
GalileoE1TcpConnectorTracking . . . 152
GalileoE5aDllPllTracking . . . 153
GpsL1CaDllPllCAidTracking . . . 241
GpsL1CaDllPllTracking . . . 243
GpsL1CaDllPllTrackingGPU . . . 244
GpsL1CaTcpConnectorTracking . . . 273
GpsL2MDllPllTracking . . . 277
TwoBitCpxFileSignalSource . . . 385
TwoBitPackedFileSignalSource . . . 386
UhdSignalSource . . . 387
GNSSFlowgraph . . . 175
Gps_Acq_Assist . . . 177
Gps_Almanac . . . 179
Gps_CNAV_Ephemeris . . . 182
Gps_CNAV_Iono . . . 192
Gps_CNAV_Navigation_Message . . . 194
Gps_CNAV_Utc_Model . . . 197
Gps_Ephemeris . . . 200
Gps_Iono . . . 210
Gps_Navigation_Message . . . 223
Gps_Ref_Location . . . 237
Gps_Ref_Time . . . 238
Gps_Utc_Model . . . 239
GPU_Complex . . . 282
GPU_Complex_Short . . . 283

Igp . . . 292
Igp_Band . . . 293
INIReader . . . 293
kernel_info_t . . . 299
Kml_Printer . . . 299
Kml_Printer_gpstk . . . 300
Long_Term_Correction . . . 301
Nmea_Printer . . . 303
Pvt_Solution . . . 341
  Ls_Pvt . . . 302
    galileo_e1_ls_pvt . . . 88
    gps_l1_ca_ls_pvt . . . 216
    hybrid_ls_pvt . . . 283
Rinex_Printer . . . 348
Rtcm . . . 354
Rtcm_Printer . . . 359
rtl_tcp_dongle_info . . . 360
Sbas_Ephemeris . . . 363
Sbas_Ionosphere_Correction . . . 365
Sbas_Raw_Msg . . . 367
Sbas_Satellite_Correction . . . 368
Sbas_Telemetry_Data . . . 369
Sbas_Time . . . 369
Sbas_Time_Relation . . . 370
state_machine
  ChannelFsm . . . 62
  GpsL1CaSubframeFsm . . . 270
StringConverter . . . 377
sync_block
  beamformer . . . 56
  byte_x2_to_complex_byte . . . 57
  complex_byte_to_float_x2 . . . 65
  complex_float_to_complex_byte . . . 66
  cshort_to_float_x2 . . . 74
  gnss_sdr_valve . . . 167
  rtl_tcp_signal_source_c . . . 361
  short_x2_to_cshort . . . 371
sync_decimator
  interleaved_byte_to_complex_byte . . . 295
  interleaved_byte_to_complex_short . . . 296
  interleaved_short_to_complex_short . . . 296
sync_interpolator
  unpack_2bit_samples . . . 388
  unpack_byte_2bit_cpx_samples . . . 389
  unpack_byte_2bit_samples . . . 390
  unpack_intspir_1bit_samples . . . 390
tcp_communication . . . 377
tcp_packet_data . . . 378
tlm_dump_reader . . . 379
Tracking_2nd_DLL_filter . . . 379
Tracking_2nd_PLL_filter . . . 381
tracking_dump_reader . . . 381
Tracking_FLL_PLL_filter . . . 382
Tracking_loop_filter . . . 383
tracking_true_obs_reader . . . 383
Viterbi_Decoder . . . 391

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AcquisitionInterface
: This abstract class represents an interface to an acquisition GNSS block . . . 53

ArraySignalConditioner
: This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal . . . 54

beamformer
: This class implements a real-time software-defined spatial filter using the CTTC GNSS experimental antenna array input and a set of dynamically reloadable weights . . . 56

BeamformerFilter
: Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface . 56

byte_x2_to_complex_byte
: This class adapts two signed char streams into a std::complex<signed char> stream . . . 57

ByteToShort
: Adapts an 8-bits sample stream (IF) to a short int stream (IF) . . . 58

Channel
: This class represents a GNSS channel. It wraps an AcquisitionInterface, a Tracking Interface and a TelemetryDecoderInterface, and handles their interaction through a Finite State Machine 59

channel_msg_receiver_cc
: GNU Radio block that receives asynchronous channel messages from acquisition and tracking blocks . . . 61

ChannelFsm
: This class implements a State Machine for channel using boost::statechart . . . 62

ChannelInterface
: This abstract class represents an interface to a channel GNSS block . . . 63

cl_fft_plan . . . 64

clFFT_Complex . . . 64

clFFT_Dim3 . . . 65

clFFT_SplitComplex . . . 65

complex_byte_to_float_x2
: This class adapts a std::complex<signed char> stream into two 16-bits (short) streams . . . 65

complex_float_to_complex_byte
: This class adapts a gr_complex stream into a std::complex<signed char> stream . . . 66

concurrent_map< Data >
: This class implements a thread-safe std::map . . . 67

concurrent_queue< Data >
: This class implements a thread-safe std::queue . . . 67

ConfigurationInterface
This abstract class represents an interface to configuration parameters . . . 68
ControlMessage
This class defines the different Control Messages . . . 69
ControlMessage_
Message described by who sent it and what it says . . . 69
ControlMessageFactory
This class implements a factory for Control Messages . . . 70
ControlThread
This class represents the main thread of the application, so the name is ControlThread. This is the GNSS Receiver Control Plane: it connects the flowgraph, starts running it, and while it does not stop, reads the control messages generated by the blocks, processes them, and applies the corresponding actions . . . 71
cpu_multicorrelator
Class that implements carrier wipe-off and correlators . . . 73
cpu_multicorrelator_16sc
Class that implements carrier wipe-off and correlators . . . 73
cshort_to_float_x2
This class adapts a std::complex<short> stream into two 32-bits (float) streams . . . 74
cuda_multicorrelator
Class that implements carrier wipe-off and correlators using NVIDIA CUDA GPU accelerators . 74
direct_resampler_conditioner_cb
This class implements a direct resampler conditioner for std::complex<signed char> . . . 75
direct_resampler_conditioner_cc
This class implements a direct resampler conditioner for complex data . . . 76
direct_resampler_conditioner_cs
This class implements a direct resampler conditioner for std::complex<short> . . . 77
DirectResamplerConditioner
Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface . 77
Fast_Correction . . . 78
FileConfiguration
This class is an implementation of the interface ConfigurationInterface . . . 79
FileSignalSource
Class that reads signals samples from a file and adapts it to a SignalSourceInterface . . . 80
FirFilter
This class adapts a GNU Radio gr_fir_filter designed with pm_remez . . . 81
FlexibandSignalSource
This class configures and reads samples from Teleorbit Flexiband front-end. This software requires a Flexiband GNU Radio driver installed (not included with GNSS-SDR) . . . 82
FreqXlatingFirFilter
This class adapts a gnuradio gr_freq_xlating_fir_filter designed with pm_remez . . . 83
FrontEndCal . . . 84
Galileo_Almanac
This class is a storage for the GALILEO ALMANAC data as described in GALILEO ICD . . . 85
galileo_e1_dll_pll_veml_tracking_cc
This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals . . . 87
galileo_e1_ls_pvt
This class implements a simple PVT Least Squares solution . . . 88
galileo_e1_observables_cc
This class implements a block that computes Galileo observables . . . 89
galileo_e1_pvt_cc
This class implements a block that computes the PVT solution with Galileo E1 signals . . . 90
Galileo_E1_Tcp_Connector_Tracking_cc
This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals . . . 91
galileo_e1b_telemetry_decoder_cc
This class implements a block that decodes the INAV data defined in Galileo ICD . . . 92

Galileo_E5a_Dll_Pll_Tracking_cc
This class implements a DLL + PLL tracking loop block . . . 93
galileo_e5a_noncoherentIQ_acquisition_caf_cc
This class implements a Parallel Code Phase Search Acquisition . . . 94
galileo_e5a_telemetry_decoder_cc
This class implements a block that decodes the FNAV data defined in Galileo ICD . . . 98
Galileo_Ephemeris
This class is a storage and orbital model functions for the Galileo SV ephemeris data as described in Galileo ICD paragraph 5.1.1 (See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf) . . . 99
Galileo_Fnav_Message
This class handles the Galileo F/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf . . . 107
Galileo_Iono
This class is a storage for the GALILEO IONOSPHERIC data as described in Galileo ICD paragraph 5.1.6 . . . 112
Galileo_Navigation_Message
This class handles the Galileo I/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf . . . 114
galileo_pcps_8ms_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition for Galileo E1 signals with coherent integration time = 8 ms (two codes) . . . 127
Galileo_Utc_Model
This class is a storage for the GALILEO UTC MODEL data as described in Galileo ICD https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf paragraph 5.1.7 . . . 130
GalileoE1BTelemetryDecoder
This class implements a NAV data decoder for Galileo INAV frames in E1B radio link . . . 132
GalileoE1DllPllVemlTracking
This class Adapts a DLL+PLL VEML (Very Early Minus Late) tracking loop block to a TrackingInterface for Galileo E1 signals . . . 133
GalileoE1Observables
This class implements an ObservablesInterface for Galileo E1B . . . 134
GalileoE1Pcps8msAmbiguousAcquisition
Adapts a PCPS 8ms acquisition block to an AcquisitionInterface for Galileo E1 Signals . . . 136
GalileoE1PcpsAmbiguousAcquisition
This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals . . 139
GalileoE1PcpsCccwsrAmbiguousAcquisition
Adapts a PCPS CCCWSR acquisition block to an AcquisitionInterface for Galileo E1 Signals . 142
GalileoE1PcpsQuickSyncAmbiguousAcquisition
This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals . . 145
GalileoE1PcpsTongAmbiguousAcquisition
Adapts a PCPS Tong acquisition block to an AcquisitionInterface for Galileo E1 Signals . . . 148
GalileoE1Pvt
This class implements a PvtInterface for Galileo E1 . . . 151
GalileoE1TcpConnectorTracking
This class implements a code DLL + carrier PLL tracking loop . . . 152
GalileoE5aDllPllTracking
This class implements a code DLL + carrier PLL tracking loop . . . 153
GalileoE5aNoncoherentIQAcquisitionCaf . . . 155
GalileoE5aObservables
This class implements an ObservablesInterface for Galileo E5A . . . 158
GalileoE5aTelemetryDecoder
This class implements a NAV data decoder for Galileo INAV frames in E1B radio link . . . 159

GenSignalSource
This class wraps blocks that generates synthesized GNSS signal and filters the signal 160
GeoJSON_Printer
Prints PVT solutions in GeoJSON format file 162
Gn3sSignalSource
This class reads samples from a GN3S USB dongle, a RF front-end signal sampler 162
Gnss_Satellite
This class represents a GNSS satellite 163
gnss_sdr_supl_client
Class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library. 165
gnss_sdr_valve
Implementation of a GNU Radio block that sends a STOP message to the control queue right after a specific number of samples have passed through it 167
Gnss_Signal
This class represents a GNSS signal 168
Gnss_Synchro
This is the class that contains the information that is shared by the processing blocks 169
GNSSBlockFactory
Class that produces all kinds of GNSS blocks 173
GNSSBlockInterface
This abstract class represents an interface to GNSS blocks 174
GNSSFlowgraph
This class represents a GNSS flowgraph 175
Gps_Acq_Assist
This class is a storage for the GPS GSM RRLL acquisition assistance data as described in Digital cellular telecommunications system (Phase 2+); Location Services (LCS); Mobile Station (MS) - Serving Mobile Location Centre (SMLC) Radio Resource LCS Protocol (RRLP) (3GPP TS 44.031 version 5.12.0 Release 5) 177
Gps_Almanac
This class is a storage for the GPS SV ALMANAC data as described in IS-GPS-200E 179
Gps_CNAV_Ephemeris
This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200H 182
Gps_CNAV_Iono
This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200H 192
Gps_CNAV_Navigation_Message
This class decodes a GPS CNAV Data message as described in IS-GPS-200H 194
Gps_CNAV_Utc_Model
This class is a storage for the GPS UTC MODEL data as described in in IS-GPS-200H 197
Gps_Ephemeris
This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200E 200
Gps_Iono
This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200E 210
gps_l1_ca_dll_pll_c_aid_tracking_cc
This class implements a DLL + PLL tracking loop block 213
gps_l1_ca_dll_pll_c_aid_tracking_sc
This class implements a DLL + PLL tracking loop block 213
Gps_L1_Ca_Dll_Pll_Tracking_cc
This class implements a DLL + PLL tracking loop block 214
Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc
This class implements a DLL + PLL tracking loop block 215
gps_l1_ca_ls_pvt
This class implements a simple PVT Least Squares solution for GPS L1 C/A signals 216
gps_l1_ca_observables_cc
This class implements a block that computes GPS L1 C/A observables 217

gps_l1_ca_pvt_cc
This class implements a block that computes the PVT solution . . . 218
Gps_L1_Ca_Tcp_Connector_Tracking_cc
This class implements a DLL + PLL tracking loop block . . . 219
gps_l1_ca_telemetry_decoder_cc
This class implements a block that decodes the NAV data defined in IS-GPS-200E . . . 220
gps_l2_m_dll_pll_tracking_cc
This class implements a DLL + PLL tracking loop block . . . 221
gps_l2c_telemetry_decoder_cc
This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229 . . . 222
Gps_Navigation_Message
This class decodes a GPS NAV Data message as described in IS-GPS-200E . . . 223
Gps_Ref_Location
Interface of a GPS REFERENCE LOCATION storage . . . 237
Gps_Ref_Time
Interface of a GPS REFERENCE TIME storage . . . 238
Gps_Utc_Model
This class is a storage for the GPS UTC MODEL data as described in IS-GPS-200E . . . 239
GpsL1CaDllPllCAidTracking
This class implements a code DLL + carrier PLL tracking loop . . . 241
GpsL1CaDllPllTracking
This class implements a code DLL + carrier PLL tracking loop . . . 243
GpsL1CaDllPllTrackingGPU
This class implements a code DLL + carrier PLL tracking loop using GPU accelerated functions 244
GpsL1CaObservables
This class implements an ObservablesInterface for GPS L1 C/A . . . 246
GpsL1CaPcpsAcquisition
This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals . 247
GpsL1CaPcpsAcquisitionFineDoppler
This class Adapts a PCPS acquisition block with fine Doppler estimation to an AcquisitionInterface for GPS L1 C/A signals . . . 250
GpsL1CaPcpsAssistedAcquisition
This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals . 253
GpsL1CaPcpsMultithreadAcquisition
This class adapts a multithread PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals . . . 256
GpsL1CaPcpsOpenClAcquisition
This class adapts an OpenCL PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals . . . 259
GpsL1CaPcpsQuickSyncAcquisition
This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals . 262
GpsL1CaPcpsTongAcquisition
This class adapts a PCPS Tong acquisition block to an AcquisitionInterface for GPS L1 C/A signals . . . 265
GpsL1CaPvt
This class implements a PvtInterface for GPS L1 C/A . . . 268
GpsL1CaSubframeFsm
This class implements a Finite State Machine that handles the decoding of the GPS L1 C/A NAV message . . . 270
GpsL1CaTcpConnectorTracking
This class implements a code DLL + carrier PLL tracking loop . . . 273
GpsL1CaTelemetryDecoder
This class implements a NAV data decoder for GPS L1 C/A . . . 274
GpsL2CObservables
This class implements an ObservablesInterface for GPS L2C . . . 275
GpsL2CTelemetryDecoder
This class implements a NAV data decoder for GPS L2 M . . . 276

GpsL2MDllPllTracking
This class implements a code DLL + carrier PLL tracking loop . . . 277
GpsL2MPcpsAcquisition
This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L2 M signals . . 279
GPU_Complex . . . 282
GPU_Complex_Short . . . 283
hybrid_ls_pvt
This class implements a simple PVT Least Squares solution . . . 283
hybrid_observables_cc
This class implements a block that computes Galileo observables . . . 285
hybrid_pvt_cc
This class implements a block that computes the PVT solution with Galileo E1 signals . . . 285
HybridObservables
This class implements an ObservablesInterface for observables of all kind of GNSS signals . . 287
HybridPvt
This class implements a PvtInterface for Galileo E1 . . . 288
IbyteToCbyte . . . 289
IbyteToComplex
Adapts an I/Q interleaved byte integer sample stream to a gr_complex (float) stream . . . 290
IbyteToCshort
Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream 291
Igp
Struct that represents a Ionospheric Grid Point (IGP) . . . 292
Igp_Band
Struct that represents the band of a Ionospheric Grid Point (IGP) . . . 293
INIReader
Read an INI file into easy-to-access name/value pairs. (Note that I've gone for simplicity here rather than speed, but it should be pretty decent.) . . . 293
InMemoryConfiguration
This class is an implementation of the interface ConfigurationInterface . . . 294
interleaved_byte_to_complex_byte
This class adapts an 8-bits interleaved sample stream into a 16-bits complex stream (std↩ ::complex<unsigned char>) . . . 295
interleaved_byte_to_complex_short
This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream 296
interleaved_short_to_complex_short
This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream 296
IshortToComplex
Adapts an I/Q interleaved short integer sample stream to a gr_complex (float) stream . . . 297
IshortToCshort
Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream 298
kernel_info_t . . . 299
Kml_Printer
Prints PVT information to OGC KML format file (can be viewed with Google Earth) . . . 299
Kml_Printer_gpstk
Prints PVT information to OGC KML format file (can be viewed with Google Earth) . . . 300
Long_Term_Correction . . . 301
Ls_Pvt
Base class for the Least Squares PVT solution . . . 302
Nmea_Printer
This class provides a implementation of a subset of the NMEA-0183 standard for interfacing marine electronic devices as defined by the National Marine Electronics Association (NMEA) . 303
NsrFileSignalSource
Class that reads signals samples from a file and adapts it to a SignalSourceInterface . . . 304
ObservablesInterface
This abstract class represents an interface to an observables block . . . 305

OsmosdrSignalSource
This class reads samples OsmoSDR-compatible front-ends, such as HackRF or Realtek's RTL2832U-based USB dongle DVB-T receivers (see http://sdr.osmocom.org/trac/wiki/rtl-sdr) . . . 306
Pass_Through
This class implements a block that connects input and output (does nothing) . . . 307
pcps_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition . . . 308
pcps_acquisition_fine_doppler_cc
This class implements a Parallel Code Phase Search Acquisition . . . 312
pcps_acquisition_sc
This class implements a Parallel Code Phase Search Acquisition . . . 315
pcps_assisted_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition . . . 319
pcps_cccwsr_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition with Coherent Channel Combining With Sign Recovery scheme . . . 323
pcps_multithread_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition . . . 326
pcps_opencl_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition . . . 330
pcps_quicksync_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition with the implementation of the Sparse QuickSync Algorithm . . . 334
pcps_tong_acquisition_cc
This class implements a Parallel Code Phase Search Acquisition with Tong algorithm . . . 337
Pvt_Solution
Base class for a PVT solution . . . 341
PvtInterface
This class represents an interface to a PVT block . . . 347
RawArraySignalSource
This class reads samples from a GN3S USB dongle, a RF front-end signal sampler . . . 347
Rinex_Printer
Class that handles the generation of Receiver INdependent EXchange format (RINEX) files . . . 348
Rtcm . . . 354
Rtcm_Printer
This class provides a implementation of a subset of the RTCM Standard 10403.2 messages . . . 359
rtl_tcp_dongle_info
This class represents the dongle information which is sent by rtl_tcp . . . 360
rtl_tcp_signal_source_c
This class reads interleaved I/Q samples from an rtl_tcp server and outputs complex types . . . 361
RtlTcpSignalSource
This class reads from rtl_tcp, which streams interleaved I/Q samples over TCP. (see http://sdr.osmocom.org/trac/wiki/rtl-sdr) . . . 362
Sbas_Ephemeris
This class stores SBAS SV ephemeris data . . . 363
Sbas_Ionosphere_Correction
Class that handles valid SBAS ionospheric correction for GPS . . . 365
sbas_l1_telemetry_decoder_cc
This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229 . . . 366
Sbas_Raw_Msg
Represents a raw SBAS message of 250cbits + 6 bits padding (8b preamble + 6b message type + 212b data + 24b CRC + 6b zero padding) . . . 367
Sbas_Satellite_Correction
Valid long and fast term SBAS corrections for one SV . . . 368
Sbas_Telemetry_Data . . . 369

Sbas_Time
Sbas_Time relates the relative sample stamp time scale with the absolute GPS time scale. There are three different states for a Sbas_Time object: . . . 369
Sbas_Time_Relation . . . 370
SbasL1TelemetryDecoder
This class implements a NAV data decoder for SBAS frames in L1 radio link . . . 370
short_x2_to_cshort
This class adapts two short streams into a std::complex<short> stream . . . 371
signal_generator_c
This class generates synthesized GNSS signal . . . 372
SignalConditioner
This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal . . . 373
SignalGenerator
This class generates synthesized GNSS signal . . . 375
SpirFileSignalSource
Class that reads signals samples from a file and adapts it to a SignalSourceInterface . . . 376
StringConverter
Class that interprets the contents of a string and converts it into different types . . . 377
tcp_communication
TCP communication class . . . 377
tcp_packet_data
Class that implements a TCP data packet . . . 378
TelemetryDecoderInterface
This abstract class represents an interface to a navigation GNSS block . . . 378
tlm_dump_reader . . . 379
Tracking_2nd_DLL_filter
This class implements a 2nd order DLL filter for code tracking loop . . . 379
Tracking_2nd_PLL_filter
This class implements a 2nd order PLL filter for carrier tracking loop . . . 381
tracking_dump_reader . . . 381
Tracking_FLL_PLL_filter
This class implements a hybrid FLL and PLL filter for tracking carrier loop . . . 382
Tracking_loop_filter
This class implements a generic 1st, 2nd or 3rd order loop filter . . . 383
tracking_true_obs_reader . . . 383
TrackingInterface
This abstract class represents an interface to a tracking block . . . 384
TwoBitCpxFileSignalSource
Class that reads signals samples from a file and adapts it to a SignalSourceInterface . . . 385
TwoBitPackedFileSignalSource
Class that reads signals samples from a file and adapts it to a SignalSourceInterface . . . 386
UhdSignalSource
This class reads samples from a UHD device (see http://code.ettus.com/redmine/ettus/projects/ul 387
unpack_2bit_samples
This class takes 2 bit samples that have been packed into bytes or shorts as input and generates a byte for each sample. It generates eight times as much data as is input (every two bits become 16 bits) . . . 388
unpack_byte_2bit_cpx_samples
This class implements conversion between byte packet samples to 2bit_cpx samples 1 byte = 2 x complex 2bit I, + 2bit Q samples . . . 389
unpack_byte_2bit_samples
This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples . . . 390
unpack_intspir_1bit_samples
This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples . . . 390

Viterbi_Decoder
Class that implements a Viterbi decoder 391

Chapter 7

File Index

7.1 File List

Here is a list of all documented files with brief descriptions:

src/algorithms/acquisition/adapters/galileo_e1_pcps_8ms_ambiguous_acquisition.h
Adapts a PCPS 8ms acquisition block to an AcquisitionInterface for Galileo E1 Signals 393
src/algorithms/acquisition/adapters/galileo_e1_pcps_ambiguous_acquisition.h
Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals 394
src/algorithms/acquisition/adapters/galileo_e1_pcps_cccwsr_ambiguous_acquisition.h
Adapts a PCPS CCCWSR acquisition block to an AcquisitionInterface for Galileo E1 Signals . 395
src/algorithms/acquisition/adapters/galileo_e1_pcps_quicksync_ambiguous_acquisition.h
Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals 395
src/algorithms/acquisition/adapters/galileo_e1_pcps_tong_ambiguous_acquisition.h
Adapts a PCPS Tong acquisition block to an AcquisitionInterface for Galileo E1 Signals 396
src/algorithms/acquisition/adapters/galileo_e5a_noncoherent_iq_acquisition_caf.h
Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E5a data and pilot Signals 397
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition.h
Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals 398
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition_fine_doppler.h
Adapts a PCPS acquisition block with fine Doppler estimation to an AcquisitionInterface for GPS L1 C/A signals 399
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_assisted_acquisition.h
Adapts a PCPS Assisted acquisition block to an AcquisitionInterface for GPS L1 C/A signals . 400
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_multithread_acquisition.h
Adapts a multithread PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals 401
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_opencl_acquisition.h
Adapts an OpenCL PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals . 402
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_quicksync_acquisition.h
Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals implementing the QuickSync Algorithm 403
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_tong_acquisition.h
Adapts a PCPS Tong acquisition block to an AcquisitionInterface for GPS L1 C/A signals . . . 404
src/algorithms/acquisition/adapters/gps_l2_m_pcps_acquisition.h
Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L2 M signals 405
src/algorithms/acquisition/gnuradio_blocks/galileo_e5a_noncoherent_iq_acquisition_caf_cc.h
Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E5a data and pilot Signals 406
src/algorithms/acquisition/gnuradio_blocks/galileo_pcps_8ms_acquisition_cc.h
This class implements a Parallel Code Phase Search Acquisition for Galileo E1 signals with coherent integration time = 8 ms (two codes) 407

src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_cc.h
This class implements a Parallel Code Phase Search Acquisition . . . 408
src/algorithms/acquisition/gnuradio_blocks/**pcps_acquisition_fine_doppler_cc.h** . . . **??**
src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_sc.h
This class implements a Parallel Code Phase Search Acquisition . . . 410
src/algorithms/acquisition/gnuradio_blocks/pcps_assisted_acquisition_cc.h
This class implements a Parallel Code Phase Search Acquisition with assistance and multi-dwells . . . 411
src/algorithms/acquisition/gnuradio_blocks/pcps_cccwsr_acquisition_cc.h
This class implements a Parallel Code Phase Search acquisition with Coherent Channel Combining With Sign Recovery scheme . . . 412
src/algorithms/acquisition/gnuradio_blocks/pcps_multithread_acquisition_cc.h
This class implements a Parallel Code Phase Search Acquisition . . . 414
src/algorithms/acquisition/gnuradio_blocks/pcps_opencl_acquisition_cc.h
This class implements a Parallel Code Phase Search Acquisition using OpenCL to offload some functions to the GPU . . . 415
src/algorithms/acquisition/gnuradio_blocks/pcps_quicksync_acquisition_cc.h
This class implements a Parallel Code Phase Search Acquisition with the QuickSync Algorithm 417
src/algorithms/acquisition/gnuradio_blocks/pcps_tong_acquisition_cc.h
This class implements a Parallel Code Phase Search Acquisition with Tong algorithm . . . 418
src/algorithms/channel/adapters/channel.h
Interface of a GNSS channel . . . 420
src/algorithms/channel/libs/channel_fsm.h
Interface of the State Machine for channel using boost::statechart . . . 421
src/algorithms/channel/libs/channel_msg_receiver_cc.h
GNU Radio block that receives asynchronous channel messages from acquisition and tracking blocks . . . 421
src/algorithms/conditioner/adapters/array_signal_conditioner.h
It wraps blocks to change data type, filter and resample input data, adapted to array receiver . 422
src/algorithms/conditioner/adapters/signal_conditioner.h
It wraps blocks to change data type, filter and resample input data . . . 423
src/algorithms/data_type_adapter/adapters/byte_to_short.h
Adapts an 8-bits sample stream (IF) to a short int stream (IF) . . . 424
src/algorithms/data_type_adapter/adapters/ibyte_to_cbyte.h
Adapts an I/Q interleaved byte (unsigned char) sample stream into a std::complex<unsigned char> stream . . . 425
src/algorithms/data_type_adapter/adapters/ibyte_to_complex.h
Adapts an I/Q interleaved byte integer sample stream to a gr_complex (float) stream . . . 425
src/algorithms/data_type_adapter/adapters/ibyte_to_cshort.h
Adapts a short interleaved sample stream into a std::complex<short> stream . . . 426
src/algorithms/data_type_adapter/adapters/ishort_to_complex.h
Adapts an I/Q interleaved short integer sample stream to a gr_complex (float) stream . . . 427
src/algorithms/data_type_adapter/adapters/ishort_to_cshort.h
Adapts a short interleaved sample stream into a std::complex<short> stream . . . 428
src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_byte.h
Adapts an 8-bits interleaved sample stream into a 16-bits complex stream . . . 428
src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_short.h
Adapts a byte (8-bits) interleaved sample stream into a std::complex<short> stream . . . 429
src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_short_to_complex_short.h
Adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream . . . 430
src/algorithms/input_filter/adapters/beamformer_filter.h
Interface of an adapter of a digital beamformer . . . 431
src/algorithms/input_filter/adapters/fir_filter.h
Adapts a gnuradio gr_fir_filter designed with pm_remez . . . 432
src/algorithms/input_filter/adapters/freq_xlating_fir_filter.h
Adapts a gnuradio gr_freq_xlating_fir_filter designed with gr_remez . . . 433
src/algorithms/input_filter/gnuradio_blocks/beamformer.h
Simple spatial filter using RAW array input and beamforming coefficients . . . 434

src/algorithms/libs/byte_x2_to_complex_byte.h
Adapts two signed char streams into a std::complex<signed char> stream . . . 435
src/algorithms/libs/**cIFFT.h** . . . **??**
src/algorithms/libs/complex_byte_to_float_x2.h
Adapts a std::complex<signed char> stream into two 16-bits (short) streams . . . 436
src/algorithms/libs/complex_float_to_complex_byte.h
Adapts a gr_complex stream into a std::complex<signed char> stream . . . 437
src/algorithms/libs/cshort_to_float_x2.h
Adapts a std::complex<short> stream into two float streams . . . 438
src/algorithms/libs/**fft_base_kernels.h** . . . **??**
src/algorithms/libs/**fft_internal.h** . . . **??**
src/algorithms/libs/galileo_e1_signal_processing.h
This library implements various functions for Galileo E1 signals . . . 439
src/algorithms/libs/**galileo_e5_signal_processing.h** . . . **??**
src/algorithms/libs/gnss_sdr_valve.h
Interface of a GNU Radio block that sends a STOP message to the control queue right after a specific number of samples have passed through it . . . 441
src/algorithms/libs/gnss_signal_processing.h
This library gathers a few functions used by the algorithms of gnss-sdr, regardless of system used 442
src/algorithms/libs/gps_l2c_signal.h
This class implements signal generators for the GPS L2C signals . . . 443
src/algorithms/libs/gps_sdr_signal_processing.h
This class implements various functions for GPS L1 CA signals . . . 444
src/algorithms/libs/pass_through.h
Interface of a block that just puts its input in its output . . . 445
src/algorithms/libs/short_x2_to_cshort.h
Adapts two short streams into a std::complex<short> stream . . . 446
src/algorithms/observables/adapters/galileo_e1_observables.h
Implementation of an adapter of a Galileo E1 observables block to a ObservablesInterface . . 447
src/algorithms/observables/adapters/galileo_e5a_observables.h
Implementation of an adapter of a Galileo E5a observables block to a ObservablesInterface . . 447
src/algorithms/observables/adapters/gps_l1_ca_observables.h
Interface of an adapter of a GPS L1 C/A observables block to a ObservablesInterface . . . 448
src/algorithms/observables/adapters/gps_l2c_observables.h
Implementation of an adapter of a GPS L2C(M) observables block to a ObservablesInterface . 449
src/algorithms/observables/adapters/hybrid_observables.h
Implementation of an adapter of an observables block accepting all kind of signals to a ObservablesInterface . . . 450
src/algorithms/observables/gnuradio_blocks/galileo_e1_observables_cc.h
Interface of the observables computation block for Galileo E1 . . . 450
src/algorithms/observables/gnuradio_blocks/gps_l1_ca_observables_cc.h
Interface of the pseudorange computation block for GPS L1 C/A . . . 451
src/algorithms/observables/gnuradio_blocks/hybrid_observables_cc.h
Interface of the observables computation block for Galileo E1 . . . 452
src/algorithms/PVT/adapters/galileo_e1_pvt.h
Interface of an adapter of a GALILEO E1 PVT solver block to a PvtInterface . . . 453
src/algorithms/PVT/adapters/gps_l1_ca_pvt.h
Interface of an adapter of a GPS L1 C/A PVT solver block to a PvtInterface Position Velocity and Time . . . 454
src/algorithms/PVT/adapters/hybrid_pvt.h
Interface of an adapter of a GALILEO E1 PVT solver block to a PvtInterface . . . 455
src/algorithms/PVT/gnuradio_blocks/galileo_e1_pvt_cc.h
Interface of a Position Velocity and Time computation block for Galileo E1 . . . 456
src/algorithms/PVT/gnuradio_blocks/gps_l1_ca_pvt_cc.h
Interface of a Position Velocity and Time computation block for GPS L1 C/A . . . 457
src/algorithms/PVT/gnuradio_blocks/hybrid_pvt_cc.h
Interface of a Position Velocity and Time computation block for Galileo E1 . . . 458

src/algorithms/PVT/libs/galileo_e1_ls_pvt.h
Interface of a Least Squares Position, Velocity, and Time (PVT) solver, based on K.Borre's Matlab receiver . . . 459
src/algorithms/PVT/libs/geojson_printer.h
Interface of a class that prints PVT solutions in GeoJSON format . . . 459
src/algorithms/PVT/libs/gps_l1_ca_ls_pvt.h
Interface of a Least Squares Position, Velocity, and Time (PVT) solver for GPS L1 C/A, based on K.Borre's Matlab receiver . . . 460
src/algorithms/PVT/libs/**hybrid_ls_pvt.h** . . . **??**
src/algorithms/PVT/libs/kml_printer.h
Interface of a class that prints PVT information to a kml file . . . 461
src/algorithms/PVT/libs/ls_pvt.h
Interface of a base class for Least Squares PVT solutions . . . 462
src/algorithms/PVT/libs/nmea_printer.h
Interface of a NMEA 2.1 printer for GNSS-SDR This class provides a implementation of a subset of the NMEA-0183 standard for interfacing marine electronic devices as defined by the National Marine Electronics Association (NMEA). See http://www.nmea.org/ for the NMEA 183 standard . . . 463
src/algorithms/PVT/libs/pvt_solution.h
Interface of a base class for a PVT solution . . . 464
src/algorithms/PVT/libs/rinex_printer.h
Interface of a RINEX 2.11 / 3.01 printer See http://igscb.jpl.nasa.gov/igscb/data/format/rinex30.pdf . . . 465
src/algorithms/PVT/libs/rtcm_printer.h
Interface of a RTCM 3.2 printer for GNSS-SDR This class provides a implementation of a subset of the RTCM Standard 10403.2 for Differential GNSS Services . . . 466
src/algorithms/resampler/adapters/direct_resampler_conditioner.h
Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface . 467
src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cb.h
Nearest neighborhood resampler with std::complex<signed char> input and std::complex<signed char> output . . . 468
src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cc.h
Nearest neighborhood resampler with gr_complex input and gr_complex output . . . 469
src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cs.h
Nearest neighborhood resampler with std::complex<short> input and std::complex<short> output . . . 470
src/algorithms/signal_generator/adapters/signal_generator.h
Adapter of a class that generates synthesized GNSS signal . . . 471
src/algorithms/signal_generator/gnuradio_blocks/signal_generator_c.h
GNU Radio source block that generates synthesized GNSS signal . . . 472
src/algorithms/signal_source/adapters/file_signal_source.h
Interface of a class that reads signals samples from a file and adapts it to a SignalSourceInterface 474
src/algorithms/signal_source/adapters/**flexiband_signal_source.h** . . . **??**
src/algorithms/signal_source/adapters/gen_signal_source.h
It wraps blocks that generates synthesized GNSS signal and filters it . . . 474
src/algorithms/signal_source/adapters/gn3s_signal_source.h
GN3S USB dongle GPS RF front-end signal sampler driver . . . 475
src/algorithms/signal_source/adapters/nsr_file_signal_source.h
Implementation of a class that reads signals samples from a NSR 2 bits sampler front-end file and adapts it to a SignalSourceInterface. More information about the front-end here http://www.ifen.com/products/sx-scientific-gnss-solutions/nsr-software-receiver.html . . . 476
src/algorithms/signal_source/adapters/osmosdr_signal_source.h
Signal source wrapper for OsmoSDR-compatible front-ends, such as HackRF or Realtek's RTL2832U-based USB dongle DVB-T receivers (see http://sdr.osmocom.org/trac/wiki/rtl-sdr for more information) . . . 477

src/algorithms/signal_source/adapters/raw_array_signal_source.h
Signal Source adapter for the Teleorbit Flexiband front-end device. This adapter requires a Flexiband GNURadio driver installed (not included with GNSS-SDR) . . . 478
src/algorithms/signal_source/adapters/rtl_tcp_signal_source.h
Signal source which reads from rtl_tcp. (see http://sdr.osmocom.org/trac/wiki/rtl-sdr for more information) . . . 479
src/algorithms/signal_source/adapters/spir_file_signal_source.h
Implementation of a class that reads signals samples from a SPIR file and adapts it to a SignalSourceInterface . . . 480
src/algorithms/signal_source/adapters/two_bit_cpx_file_signal_source.h
Interface of a class that reads signals samples from a 2 bit complex sampler front-end file and adapts it to a SignalSourceInterface . . . 481
src/algorithms/signal_source/adapters/two_bit_packed_file_signal_source.h
Interface of a class that reads signals samples from a file. Each sample is two bits, which are packed into bytes or shorts . . . 482
src/algorithms/signal_source/adapters/uhd_signal_source.h
Interface for the Universal Hardware Driver signal source . . . 483
src/algorithms/signal_source/gnuradio_blocks/rtl_tcp_signal_source_c.h
Interface of an rtl_tcp signal source reader . . . 484
src/algorithms/signal_source/gnuradio_blocks/unpack_2bit_samples.h
Unpacks 2 bit samples samples may be packed in any of the following ways: 1) Into bytes [item == byte] 1a) Big endian ordering within the byte 1b) Little endian ordering within the byte 2) Into shorts [item == short] 2a) Big endian ordering of bytes, big endian within the byte 2b) Big endian ordering of bytes, little endian within the byte 2c) Little endian ordering of bytes, big endian within the byte 2d) Little endian ordering of bytes, little endian within the byte . . . 485
src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_cpx_samples.h
Unpacks byte samples to 2 bits complex samples. Packing Order Most Significant Nibble - Sample n Least Significant Nibble - Sample n+1 Packing order in Nibble Q1 Q0 I1 I0 . . . 487
src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_samples.h
Unpacks byte samples to NSR 2 bits samples . . . 488
src/algorithms/signal_source/gnuradio_blocks/unpack_intspir_1bit_samples.h
Unpacks SPIR int samples to NSR 1 bit samples . . . 489
src/algorithms/signal_source/libs/rtl_tcp_commands.h
Defines structures and constants for communicating with rtl_tcp . . . 490
src/algorithms/signal_source/libs/rtl_tcp_dongle_info.h
Interface for a structure sent by rtl_tcp defining the hardware . . . 491
src/algorithms/telemetry_decoder/adapters/galileo_e1b_telemetry_decoder.h
Interface of an adapter of a GALILEO E1B NAV data decoder block to a TelemetryDecoderInterface . . . 492
src/algorithms/telemetry_decoder/adapters/galileo_e5a_telemetry_decoder.h
Interface of an adapter of a GALILEO E5a FNAV data decoder block to a TelemetryDecoderInterface . . . 492
src/algorithms/telemetry_decoder/adapters/gps_l1_ca_telemetry_decoder.h
Interface of an adapter of a GPS L1 C/A NAV data decoder block to a TelemetryDecoderInterface 493
src/algorithms/telemetry_decoder/adapters/gps_l2c_telemetry_decoder.h
Interface of an adapter of a GPS L2C (CNAV) data decoder block to a TelemetryDecoderInterface 494
src/algorithms/telemetry_decoder/adapters/sbas_l1_telemetry_decoder.h
Interface of an adapter of a SBAS telemetry data decoder block to a TelemetryDecoderInterface 495
src/algorithms/telemetry_decoder/gnuradio_blocks/galileo_e1b_telemetry_decoder_cc.h
Interface of a Galileo INAV message demodulator block . . . 495
src/algorithms/telemetry_decoder/gnuradio_blocks/**galileo_e5a_telemetry_decoder_cc.h** . . . **??**
src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l1_ca_telemetry_decoder_cc.h
Interface of a NAV message demodulator block based on Kay Borre book MATLAB-based GPS receiver . . . 496
src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l2c_telemetry_decoder_cc.h
Interface of a CNAV message demodulator block based on Kay Borre book MATLAB-based GPS receiver . . . 497

src/algorithms/telemetry_decoder/gnuradio_blocks/sbas_l1_telemetry_decoder_cc.h
Interface of a SBAS telemetry data decoder block . . . 498
src/algorithms/telemetry_decoder/libs/convolutional.h
General functions used to implement convolutional encoding . . . 499
src/algorithms/telemetry_decoder/libs/gps_l1_ca_subframe_fsm.h
Interface of a GPS NAV message word-to-subframe decoder state machine . . . 500
src/algorithms/telemetry_decoder/libs/viterbi_decoder.h
Interface of a Viterbi decoder class based on the Iterative Solutions Coded Modulation Library by Matthew C. Valenti . . . 501
src/algorithms/tracking/adapters/galileo_e1_dll_pll_veml_tracking.h
Adapts a DLL+PLL VEML (Very Early Minus Late) tracking loop block to a TrackingInterface for Galileo E1 signals . . . 502
src/algorithms/tracking/adapters/galileo_e1_tcp_connector_tracking.h
Interface of an adapter of a TCP connector block based on code DLL + carrier PLL for Galileo E1 to a TrackingInterface . . . 502
src/algorithms/tracking/adapters/galileo_e5a_dll_pll_tracking.h
Adapts a code DLL + carrier PLL tracking block to a TrackingInterface for Galileo E5a signals . 503
src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_c_aid_tracking.h
Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface 504
src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking.h
Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface 505
src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking_gpu.h
Implementation of an adapter of a DLL+PLL tracking loop block using GPU accelerated functions for GPS L1 C/A to a TrackingInterface . . . 506
src/algorithms/tracking/adapters/gps_l1_ca_tcp_connector_tracking.h
Interface of an adapter of a TCP connector block based on code DLL + carrier PLL for GPS L1 C/A to a TrackingInterface . . . 507
src/algorithms/tracking/adapters/gps_l2_m_dll_pll_tracking.h
Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface 508
src/algorithms/tracking/gnuradio_blocks/galileo_e1_dll_pll_veml_tracking_cc.h
Implementation of a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals . . . 509
src/algorithms/tracking/gnuradio_blocks/galileo_e1_tcp_connector_tracking_cc.h
Interface of a TCP connector block based on code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals . . . 510
src/algorithms/tracking/gnuradio_blocks/galileo_e5a_dll_pll_tracking_cc.h
Implementation of a code DLL + carrier PLL tracking block for Galileo E5a signals . . . 511
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_cc.h
Interface of a code DLL + carrier PLL tracking block . . . 512
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_sc.h
Interface of a code DLL + carrier PLL tracking block . . . 514
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_cc.h
Interface of a code DLL + carrier PLL tracking block . . . 515
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_gpu_cc.h
Implementation of a code DLL + carrier PLL tracking block, GPU ACCELERATED . . . 516
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_tcp_connector_tracking_cc.h
Interface of a TCP connector block based on code DLL + carrier PLL . . . 517
src/algorithms/tracking/gnuradio_blocks/gps_l2_m_dll_pll_tracking_cc.h
Interface of a code DLL + carrier PLL tracking block . . . 518
src/algorithms/tracking/libs/cpu_multicorrelator.h
High optimized CPU vector multiTAP correlator class . . . 519
src/algorithms/tracking/libs/cpu_multicorrelator_16sc.h
High optimized CPU vector multiTAP correlator class for lv_16sc_t (short int complex) . . . 519
src/algorithms/tracking/libs/cuda_multicorrelator.h
High optimized CUDA GPU vector multiTAP correlator class . . . 520
src/algorithms/tracking/libs/lock_detectors.h
Interface of a library with a set of code and carrier phase lock detectors . . . 521

src/algorithms/tracking/libs/tcp_communication.h
Interface of the TCP communication class . . . 523
src/algorithms/tracking/libs/tcp_packet_data.h
Interface of the TCP data packet class . . . 523
src/algorithms/tracking/libs/tracking_2nd_DLL_filter.h
Interface of a 2nd order DLL filter for code tracking loop . . . 524
src/algorithms/tracking/libs/tracking_2nd_PLL_filter.h
Interface of a 2nd order PLL filter for carrier tracking loop . . . 525
src/algorithms/tracking/libs/tracking_discriminators.h
Interface of a library with a set of code tracking and carrier tracking discriminators . . . 526
src/algorithms/tracking/libs/tracking_FLL_PLL_filter.h
Interface of a hybrid FLL and PLL filter for tracking carrier loop . . . 528
src/algorithms/tracking/libs/tracking_loop_filter.h
Generic 1st to 3rd order loop filter implementation . . . 528
src/core/interfaces/acquisition_interface.h
Header file of the interface to an acquisition GNSS block . . . 529
src/core/interfaces/channel_interface.h
This class represents an interface to a channel GNSS block . . . 530
src/core/interfaces/configuration_interface.h
This class represents an interface to configuration parameters . . . 531
src/core/interfaces/gnss_block_interface.h
This interface represents a GNSS block . . . 532
src/core/interfaces/observables_interface.h
This class represents an interface to an Observables block . . . 533
src/core/interfaces/pvt_interface.h
This class represents an interface to a PVT block . . . 534
src/core/interfaces/telemetry_decoder_interface.h
This class represents an interface to a telemetry decoder block . . . 535
src/core/interfaces/tracking_interface.h
This class represents an interface to a tracking block . . . 536
src/core/libs/gnss_sdr_supl_client.h
Class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library . . . 537
src/core/libs/ini.h
This function parses an INI file into easy-to-access name/value pairs . . . 538
src/core/libs/INIReader.h
This class reads an INI file into easy-to-access name/value pairs . . . 539
src/core/libs/string_converter.h
Interface of a class that interprets the contents of a string and converts it into different types . . . 540
src/core/receiver/concurrent_map.h
Interface of a thread-safe std::map . . . 541
src/core/receiver/concurrent_queue.h
Interface of a thread-safe std::queue . . . 542
src/core/receiver/control_message.h
Interface for the different control messages . . . 543
src/core/receiver/control_message_factory.h
Interface of a factory for control messages . . . 543
src/core/receiver/control_thread.h
Interface of the receiver control plane . . . 544
src/core/receiver/file_configuration.h
A ConfigurationInterface that reads the configuration from a file . . . 545
src/core/receiver/gnss_block_factory.h
Interface of a factory that returns smart pointers to GNSS blocks . . . 546
src/core/receiver/gnss_flowgraph.h
Interface of a GNSS receiver flowgraph . . . 547
src/core/receiver/in_memory_configuration.h
A ConfigurationInterface for testing purposes . . . 548

src/core/system_parameters/galileo_almanac.h
Interface of a Galileo ALMANAC storage . . . 549
src/core/system_parameters/Galileo_E1.h
Defines system parameters for Galileo E1 signal and NAV data . . . 550
src/core/system_parameters/**Galileo_E5a.h** . . . **??**
src/core/system_parameters/galileo_ephemeris.h
Interface of a Galileo EPHEMERIS storage . . . 557
src/core/system_parameters/galileo_fnav_message.h
Implementation of a Galileo F/NAV Data message as described in Galileo OS SIS ICD Issue 1.2 (Nov. 2015) . . . 558
src/core/system_parameters/galileo_iono.h
Interface of a Galileo Ionospheric Model storage . . . 559
src/core/system_parameters/galileo_navigation_message.h
Implementation of a Galileo I/NAV Data message as described in Galileo OS SIS ICD Issue 1.2 (Nov. 2015) . . . 560
src/core/system_parameters/galileo_utc_model.h
Interface of a Galileo UTC MODEL storage . . . 561
src/core/system_parameters/gnss_satellite.h
Interface of the Gnss_Satellite class . . . 562
src/core/system_parameters/gnss_signal.h
Implementation of the Gnss_Signal class . . . 562
src/core/system_parameters/gnss_synchro.h
Interface of the Gnss_Synchro class . . . 563
src/core/system_parameters/gps_acq_assist.h
Interface of a GPS RRLL ACQUISITION ASSISTACE storage . . . 564
src/core/system_parameters/gps_almanac.h
Interface of a GPS ALMANAC storage . . . 565
src/core/system_parameters/gps_cnav_ephemeris.h
Interface of a GPS CNAV EPHEMERIS storage . . . 565
src/core/system_parameters/gps_cnav_iono.h
Interface of a GPS CNAV IONOSPHERIC MODEL storage . . . 566
src/core/system_parameters/gps_cnav_navigation_message.h
Interface of a GPS CNAV Data message decoder . . . 567
src/core/system_parameters/**gps_cnav_utc_model.h** . . . **??**
src/core/system_parameters/gps_ephemeris.h
Interface of a GPS EPHEMERIS storage . . . 567
src/core/system_parameters/gps_iono.h
Interface of a GPS IONOSPHERIC MODEL storage . . . 568
src/core/system_parameters/GPS_L1_CA.h
Defines system parameters for GPS L1 C/A signal and NAV data . . . 569
src/core/system_parameters/GPS_L2C.h
Defines system parameters for GPS L2C signal . . . 576
src/core/system_parameters/gps_navigation_message.h
Interface of a GPS NAV Data message decoder . . . 582
src/core/system_parameters/gps_ref_location.h
Interface of a GPS REFERENCE LOCATION storage . . . 583
src/core/system_parameters/gps_ref_time.h
Interface of a GPS REFERENCE TIME storage . . . 584
src/core/system_parameters/gps_utc_model.h
Interface of a GPS UTC MODEL storage . . . 585
src/core/system_parameters/MATH_CONSTANTS.h
Defines useful mathematical constants and their scaled versions . . . 585
src/core/system_parameters/rtcm.h
Interface for the RTCM 3.2 Standard . . . 594
src/core/system_parameters/sbas_ephemeris.h
Interface of a SBAS REFERENCE LOCATION storage . . . 595
src/core/system_parameters/sbas_ionospheric_correction.h
Interface of the SBAS ionosphere correction set based on SBAS RTKLIB functions . . . 596

src/core/system_parameters/sbas_satellite_correction.h
Interface of the SBAS satellite correction set based on SBAS RTKLIB functions 597
src/core/system_parameters/sbas_telemetry_data.h
Interface of the SBAS telemetry parser based on SBAS RTKLIB functions 597
src/core/system_parameters/sbas_time.h
Interface and implementation of classes to handle and relate sample stamp and GPS time based time scales 599
src/tests/common-files/signal_generator_flags.h
Helper file for unit testing 600
src/tests/unit-tests/signal-processing-blocks/libs/tlm_dump_reader.h
Helper file for unit testing 601
src/tests/unit-tests/signal-processing-blocks/libs/tracking_dump_reader.h
Helper file for unit testing 602
src/tests/unit-tests/signal-processing-blocks/libs/tracking_true_obs_reader.h
Helper file for unit testing 602
src/utils/front-end-cal/front_end_cal.h
Interface of the Front-end calibration program 603
src/utils/gpstk/gnsspvt/src/**kml_printer_gpstk.h** . . . . . **??**

Chapter 8

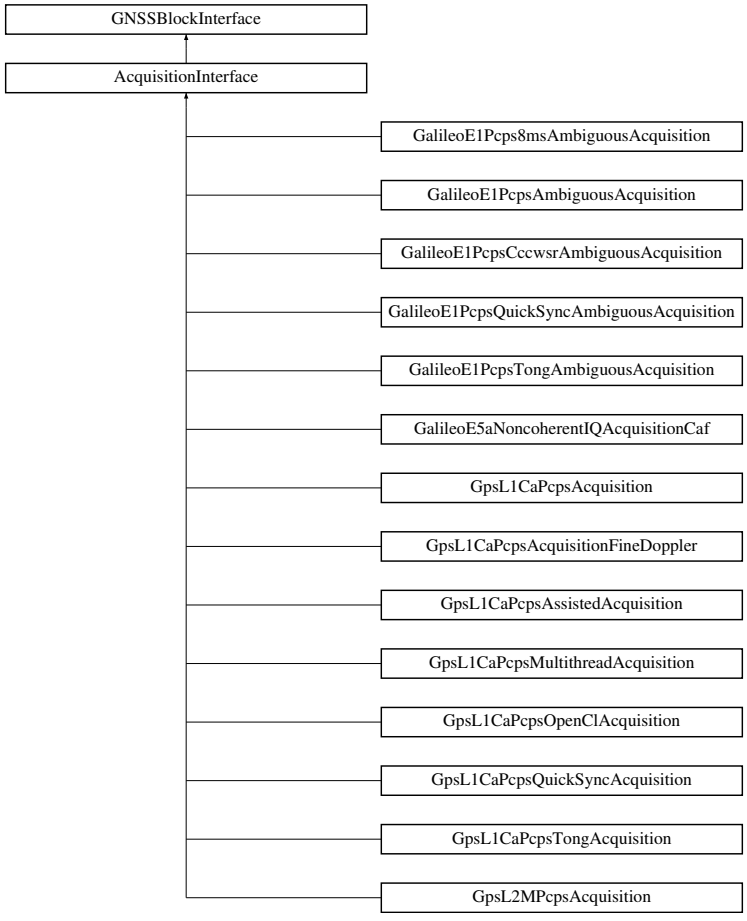
Class Documentation

8.1 AcquisitionInterface Class Reference

This abstract class represents an interface to an acquisition GNSS block.

`#include <acquisition_interface.h>`

Inheritance diagram for AcquisitionInterface:



Public Member Functions

- virtual void **set_gnss_synchro** (Gnss_Synchro *gnss_synchro)=0
- virtual void **set_channel** (unsigned int channel)=0
- virtual void **set_threshold** (float threshold)=0
- virtual void **set_doppler_max** (unsigned int doppler_max)=0
- virtual void **set_doppler_step** (unsigned int doppler_step)=0
- virtual void **init** ()=0
- virtual void **set_local_code** ()=0
- virtual signed int **mag** ()=0
- virtual void **reset** ()=0

8.1.1 Detailed Description

This abstract class represents an interface to an acquisition GNSS block.

Abstract class for acquisition algorithms. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 53 of file acquisition_interface.h.

The documentation for this class was generated from the following file:

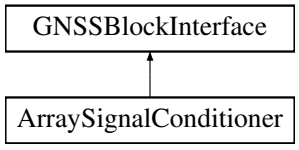
- src/core/interfaces/acquisition_interface.h

8.2 ArraySignalConditioner Class Reference

This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal.

`#include <array_signal_conditioner.h>`

Inheritance diagram for ArraySignalConditioner:



Public Member Functions

- ArraySignalConditioner (ConfigurationInterface *configuration, std::shared_ptr< GNSSBlockInterface > data_type_adapt, std::shared_ptr< GNSSBlockInterface > in_filt, std::shared_ptr< GNSSBlockInterface > res, std::string role, std::string implementation)

  Constructor.
- virtual ~ArraySignalConditioner ()

  Virtual destructor.
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **role** ()
- std::string implementation ()

  Returns "Signal_Conditioner".
- size_t **item_size** ()
- std::shared_ptr< GNSSBlockInterface > **data_type_adapter** ()
- std::shared_ptr< GNSSBlockInterface > **input_filter** ()
- std::shared_ptr< GNSSBlockInterface > **resampler** ()

8.2.1 Detailed Description

This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal.

Definition at line 50 of file array_signal_conditioner.h.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 ArraySignalConditioner::ArraySignalConditioner (ConfigurationInterface * *configuration*, std::shared_ptr< GNSSBlockInterface > *data_type_adapt*, std::shared_ptr< GNSSBlockInterface > *in_filt*, std::shared_ptr< GNSSBlockInterface > *res*, std::string *role*, std::string *implementation*)

Constructor.

8.2.2.2 virtual ArraySignalConditioner::~ArraySignalConditioner ( ) [virtual]

Virtual destructor.

8.2.3 Member Function Documentation

8.2.3.1 std::string ArraySignalConditioner::implementation ( ) [inline], [virtual]

Returns "Signal_Conditioner".

Implements GNSSBlockInterface.

Definition at line 68 of file array_signal_conditioner.h.

The documentation for this class was generated from the following file:

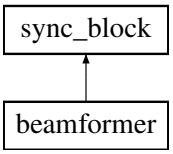
- src/algorithms/conditioner/adapters/array_signal_conditioner.h

8.3 beamformer Class Reference

This class implements a real-time software-defined spatial filter using the CTTC GNSS experimental antenna array input and a set of dynamically reloadable weights.

`#include <beamformer.h>`

Inheritance diagram for beamformer:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- beamformer_sptr **make_beamformer_sptr** ()

8.3.1 Detailed Description

This class implements a real-time software-defined spatial filter using the CTTC GNSS experimental antenna array input and a set of dynamically reloadable weights.

Definition at line 44 of file beamformer.h.

The documentation for this class was generated from the following file:

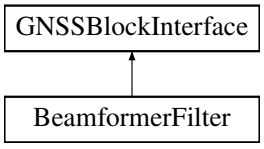
- src/algorithms/input_filter/gnuradio_blocks/beamformer.h

8.4 BeamformerFilter Class Reference

Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

`#include <beamformer_filter.h>`

Inheritance diagram for BeamformerFilter:



Public Member Functions

- **BeamformerFilter** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream)
- std::string **role** ()
- std::string implementation ()

  returns "Direct_Resampler"
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.4.1 Detailed Description

Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

Definition at line 45 of file beamformer_filter.h.

8.4.2 Member Function Documentation

8.4.2.1 std::string BeamformerFilter::implementation ( ) [inline], [virtual]

returns "Direct_Resampler"

Implements GNSSBlockInterface.

Definition at line 58 of file beamformer_filter.h.

The documentation for this class was generated from the following file:

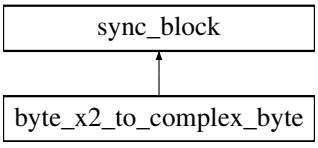
- src/algorithms/input_filter/adapters/beamformer_filter.h

8.5 byte_x2_to_complex_byte Class Reference

This class adapts two signed char streams into a std::complex<signed char> stream.

`#include <byte_x2_to_complex_byte.h>`

Inheritance diagram for byte_x2_to_complex_byte:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- byte_x2_to_complex_byte_sptr **make_byte_x2_to_complex_byte** ()

8.5.1 Detailed Description

This class adapts two signed char streams into a std::complex<signed char> stream.

Definition at line 48 of file byte_x2_to_complex_byte.h.

The documentation for this class was generated from the following file:

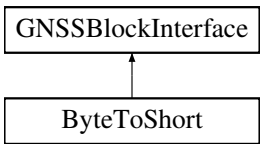
- src/algorithms/libs/byte_x2_to_complex_byte.h

8.6 ByteToShort Class Reference

Adapts an 8-bits sample stream (IF) to a short int stream (IF)

`#include <byte_to_short.h>`

Inheritance diagram for ByteToShort:



Public Member Functions

- **ByteToShort** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Byte_To_Short".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.6.1 Detailed Description

Adapts an 8-bits sample stream (IF) to a short int stream (IF)

Definition at line 46 of file byte_to_short.h.

8.6.2 Member Function Documentation

8.6.2.1 std::string ByteToShort::implementation ( ) [inline], [virtual]

Returns "Byte_To_Short".

Implements GNSSBlockInterface.

Definition at line 60 of file byte_to_short.h.

The documentation for this class was generated from the following file:

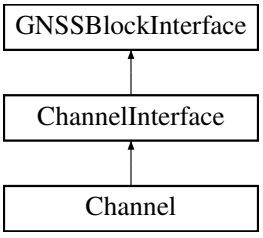
- src/algorithms/data_type_adapter/adapters/byte_to_short.h

8.7 Channel Class Reference

This class represents a GNSS channel. It wraps an AcquisitionInterface, a Tracking Interface and a Telemetry↩ DecoderInterface, and handles their interaction through a Finite State Machine.

`#include <channel.h>`

Inheritance diagram for Channel:



Public Member Functions

- Channel (ConfigurationInterface *configuration, unsigned int channel, std::shared_ptr< GNSSBlockInterface > pass_through, std::shared_ptr< AcquisitionInterface > acq, std::shared_ptr< TrackingInterface > trk, std::shared_ptr< TelemetryDecoderInterface > nav, std::string role, std::string implementation, boost::shared_ptr< gr::msg_queue > queue)

  Constructor.
- virtual ~Channel ()

  Virtual destructor.
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **role** ()
- std::string implementation ()

  Returns "Channel".
- size_t **item_size** ()
- Gnss_Signal **get_signal** () const
- std::shared_ptr< AcquisitionInterface > **acquisition** ()
- std::shared_ptr< TrackingInterface > **tracking** ()
- std::shared_ptr< TelemetryDecoderInterface > **telemetry** ()
- void start_acquisition ()

  Start the State Machine.
- void set_signal (const Gnss_Signal &gnss_signal_)

  Sets the channel GNSS signal.
- void **msg_handler_events** (pmt::pmt_t msg)

8.7.1 Detailed Description

This class represents a GNSS channel. It wraps an AcquisitionInterface, a Tracking Interface and a TelemetryDecoderInterface, and handles their interaction through a Finite State Machine.

Definition at line 58 of file channel.h.

8.7.2 Constructor & Destructor Documentation

8.7.2.1 Channel::Channel (ConfigurationInterface * *configuration,* unsigned int *channel,* std::shared_ptr< GNSSBlockInterface > *pass_through,* std::shared_ptr< AcquisitionInterface > *acq,* std::shared_ptr< TrackingInterface > *trk,* std::shared_ptr< TelemetryDecoderInterface > *nav,* std::string *role,* std::string *implementation,* boost::shared_ptr< gr::msg_queue > *queue*)

Constructor.

8.7.2.2 virtual Channel::~Channel ( ) [virtual]

Virtual destructor.

8.7.3 Member Function Documentation

8.7.3.1 std::string Channel::implementation ( ) [inline], [virtual]

Returns "Channel".

Implements GNSSBlockInterface.

Definition at line 77 of file channel.h.

References set_signal(), and start_acquisition().

8.7.3.2 void Channel::set_signal (const Gnss_Signal & *gnss_signal_*) [virtual]

Sets the channel GNSS signal.

Implements ChannelInterface.

Referenced by implementation().

8.7.3.3 void Channel::start_acquisition ( ) [virtual]

Start the State Machine.

Implements ChannelInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

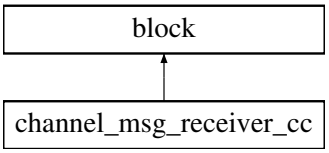
- src/algorithms/channel/adapters/channel.h

8.8 channel_msg_receiver_cc Class Reference

GNU Radio block that receives asynchronous channel messages from acquisition and tracking blocks.

`#include <channel_msg_receiver_cc.h>`

Inheritance diagram for channel_msg_receiver_cc:



Public Member Functions

- ~channel_msg_receiver_cc ()
  Default destructor.

Friends

- channel_msg_receiver_cc_sptr **channel_msg_receiver_make_cc** (ChannelFsm *channel_fsm, bool repeat)

8.8.1 Detailed Description

GNU Radio block that receives asynchronous channel messages from acquisition and tracking blocks.

Definition at line 46 of file channel_msg_receiver_cc.h.

8.8.2 Constructor & Destructor Documentation

8.8.2.1 channel_msg_receiver_cc::~channel_msg_receiver_cc ( )

Default destructor.

The documentation for this class was generated from the following file:

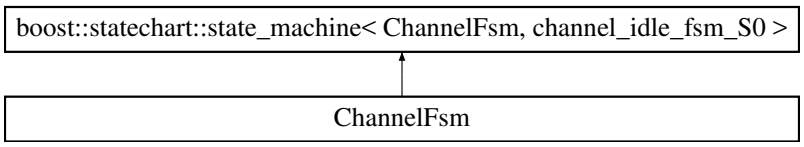
- src/algorithms/channel/libs/channel_msg_receiver_cc.h

8.9 ChannelFsm Class Reference

This class implements a State Machine for channel using boost::statechart.

`#include <channel_fsm.h>`

Inheritance diagram for ChannelFsm:



Public Member Functions

- **ChannelFsm** (std::shared_ptr< AcquisitionInterface > acquisition)
- void **set_acquisition** (std::shared_ptr< AcquisitionInterface > acquisition)
- void **set_tracking** (std::shared_ptr< TrackingInterface > tracking)
- void **set_queue** (boost::shared_ptr< gr::msg_queue > queue)
- void **set_channel** (unsigned int channel)
- void **start_acquisition** ()
- void **start_tracking** ()
- void **request_satellite** ()
- void **notify_stop_tracking** ()
- void **Event_start_acquisition** ()
- void **Event_valid_acquisition** ()
- void **Event_failed_acquisition_repeat** ()
- void **Event_failed_acquisition_no_repeat** ()
- void **Event_failed_tracking_standby** ()

8.9.1 Detailed Description

This class implements a State Machine for channel using boost::statechart.

Definition at line 54 of file channel_fsm.h.

The documentation for this class was generated from the following file:

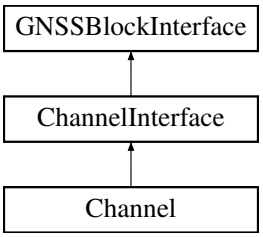
- src/algorithms/channel/libs/channel_fsm.h

8.10 ChannelInterface Class Reference

This abstract class represents an interface to a channel GNSS block.

`#include <channel_interface.h>`

Inheritance diagram for ChannelInterface:



Public Member Functions

- virtual Gnss_Signal **get_signal** () const =0
- virtual void **start_acquisition** ()=0
- virtual void **set_signal** (const Gnss_Signal &)=0

8.10.1 Detailed Description

This abstract class represents an interface to a channel GNSS block.

Abstract class for channel blocks. Since all its methods are pure virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 51 of file channel_interface.h.

The documentation for this class was generated from the following file:

- src/core/interfaces/channel_interface.h

8.11 cl_fft_plan Struct Reference

Public Attributes

- cl_context **context**
- clFFT_Dim3 **n**
- clFFT_Dimension **dim**
- clFFT_DataFormat **format**
- string * **kernel_string**
- cl_program **program**
- cl_fft_kernel_info * **kernel_info**
- int **num_kernels**
- cl_kernel **twist_kernel**
- cl_int **temp_buffer_needed**
- unsigned **last_batch_size**
- cl_mem **tempmemobj**
- cl_mem **tempmemobj_real**
- cl_mem **tempmemobj_imag**
- unsigned **max_localmem_fft_size**
- unsigned **max_work_item_per_workgroup**
- unsigned **max_radix**
- unsigned **min_mem_coalesce_width**
- unsigned **num_local_mem_banks**

8.11.1 Detailed Description

Definition at line 79 of file fft_internal.h.

The documentation for this struct was generated from the following file:

- src/algorithms/libs/fft_internal.h

8.12 clFFT_Complex Struct Reference

Public Attributes

- float **real**
- float **imag**

8.12.1 Detailed Description

Definition at line 101 of file clFFT.h.

The documentation for this struct was generated from the following file:

- src/algorithms/libs/clFFT.h

8.13 clFFT_Dim3 Struct Reference

Public Attributes

- unsigned int **x**
- unsigned int **y**
- unsigned int **z**

8.13.1 Detailed Description

Definition at line 88 of file clFFT.h.

The documentation for this struct was generated from the following file:

- src/algorithms/libs/clFFT.h

8.14 clFFT_SplitComplex Struct Reference

Public Attributes

- float * **real**
- float * **imag**

8.14.1 Detailed Description

Definition at line 95 of file clFFT.h.

The documentation for this struct was generated from the following file:

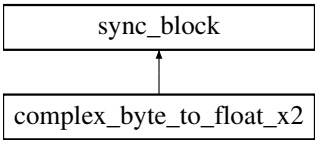
- src/algorithms/libs/clFFT.h

8.15 complex_byte_to_float_x2 Class Reference

This class adapts a std::complex<signed char> stream into two 16-bits (short) streams.

`#include <complex_byte_to_float_x2.h>`

Inheritance diagram for complex_byte_to_float_x2:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- complex_byte_to_float_x2_sptr **make_complex_byte_to_float_x2** ()

8.15.1 Detailed Description

This class adapts a std::complex<signed char> stream into two 16-bits (short) streams.

Definition at line 48 of file complex_byte_to_float_x2.h.

The documentation for this class was generated from the following file:

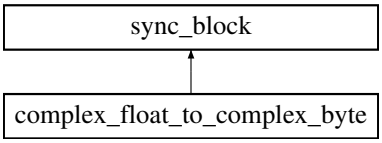
- src/algorithms/libs/complex_byte_to_float_x2.h

8.16 complex_float_to_complex_byte Class Reference

This class adapts a gr_complex stream into a std::complex<signed char> stream.

`#include <complex_float_to_complex_byte.h>`

Inheritance diagram for complex_float_to_complex_byte:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- complex_float_to_complex_byte_sptr **make_complex_float_to_complex_byte** ()

8.16.1 Detailed Description

This class adapts a gr_complex stream into a std::complex<signed char> stream.

Definition at line 46 of file complex_float_to_complex_byte.h.

The documentation for this class was generated from the following file:

- src/algorithms/libs/complex_float_to_complex_byte.h

8.17 concurrent_map< Data > Class Template Reference

This class implements a thread-safe std::map.

`#include <concurrent_map.h>`

Public Member Functions

- void **write** (int key, Data const &data)
- std::map< int, Data > **get_map_copy** ()
- size_t **size** ()
- bool **read** (int key, Data &p_data)

8.17.1 Detailed Description

template<typename Data>
class concurrent_map< Data >

This class implements a thread-safe std::map.

Definition at line 45 of file concurrent_map.h.

The documentation for this class was generated from the following file:

- src/core/receiver/concurrent_map.h

8.18 concurrent_queue< Data > Class Template Reference

This class implements a thread-safe std::queue.

`#include <concurrent_queue.h>`

Public Member Functions

- void **push** (Data const &data)
- bool **empty** () const
- bool **try_pop** (Data &popped_value)
- void **wait_and_pop** (Data &popped_value)

8.18.1 Detailed Description

template<typename Data>
class concurrent_queue< Data >

This class implements a thread-safe std::queue.

Thread-safe object queue which uses the library boost_thread to perform MUTEX based on the code available at http://www.justsoftwaresolutions.co.uk/threading/implementing-a-thread-safe-queue-using-
html

Definition at line 44 of file acquisition_interface.h.

The documentation for this class was generated from the following files:

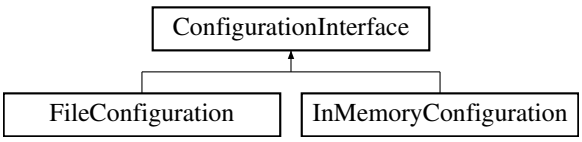
- src/core/interfaces/acquisition_interface.h
- src/core/receiver/concurrent_queue.h

8.19 ConfigurationInterface Class Reference

This abstract class represents an interface to configuration parameters.

`#include <configuration_interface.h>`

Inheritance diagram for ConfigurationInterface:



Public Member Functions

- virtual std::string **property** (std::string property_name, std::string default_value)=0
- virtual bool **property** (std::string property_name, bool default_value)=0
- virtual long **property** (std::string property_name, long default_value)=0
- virtual int **property** (std::string property_name, int default_value)=0
- virtual unsigned int **property** (std::string property_name, unsigned int default_value)=0
- virtual unsigned short **property** (std::string property_name, unsigned short default_value)=0
- virtual float **property** (std::string property_name, float default_value)=0
- virtual double **property** (std::string property_name, double default_value)=0
- virtual void **set_property** (std::string property_name, std::string value)=0

8.19.1 Detailed Description

This abstract class represents an interface to configuration parameters.

The interface defines an accessor method that gets a parameter name as input and returns the value of this parameter, a string, as output. Property names are defined here. This is an abstract class for interfaces. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 51 of file configuration_interface.h.

The documentation for this class was generated from the following file:

- src/core/interfaces/configuration_interface.h

8.20 ControlMessage Class Reference

This class defines the different Control Messages.

`#include <control_message.h>`

Static Public Attributes

- static unsigned int const **ack_success** = 0
- static unsigned int const **ack_failed** = 1
- static unsigned int const **trk_failed** = 2
- static unsigned int const **channel_init** = 3

8.20.1 Detailed Description

This class defines the different Control Messages.

Definition at line 39 of file control_message.h.

The documentation for this class was generated from the following file:

- src/core/receiver/control_message.h

8.21 ControlMessage_ Struct Reference

Message described by who sent it and what it says.

`#include <control_message_factory.h>`

Public Attributes

- unsigned int **who**
- unsigned int **what**

8.21.1 Detailed Description

Message described by who sent it and what it says.

Definition at line 39 of file control_message_factory.h.

The documentation for this struct was generated from the following file:

- src/core/receiver/control_message_factory.h

8.22 ControlMessageFactory Class Reference

This class implements a factory for Control Messages.

`#include <control_message_factory.h>`

Public Member Functions

- ControlMessageFactory ()
  Constructor.
- virtual ~ControlMessageFactory ()
  Virtual destructor.
- boost::shared_ptr< gr::message > **GetQueueMessage** (unsigned int who, unsigned int what)
- std::shared_ptr< std::vector< std::shared_ptr< ControlMessage > > > **GetControlMessages** (gr::message::sptr queue_message)

8.22.1 Detailed Description

This class implements a factory for Control Messages.

It encapsulates the complexity behind getting Queue Messages and associated Control Messages

Definition at line 51 of file control_message_factory.h.

8.22.2 Constructor & Destructor Documentation

8.22.2.1 ControlMessageFactory::ControlMessageFactory ( )

Constructor.

8.22.2.2 virtual ControlMessageFactory::~ControlMessageFactory ( ) [virtual]

Virtual destructor.

The documentation for this class was generated from the following file:

- src/core/receiver/control_message_factory.h

8.23 ControlThread Class Reference

This class represents the main thread of the application, so the name is ControlThread. This is the GNSS Receiver Control Plane: it connects the flowgraph, starts running it, and while it does not stop, reads the control messages generated by the blocks, processes them, and applies the corresponding actions.

`#include <control_thread.h>`

Public Member Functions

- ControlThread ()
  Default constructor.
- ControlThread (std::shared_ptr< ConfigurationInterface > configuration)
  Constructor that initializes the class with parameters.
- ~ControlThread ()
  Destructor.
- void run ()
  Runs the control thread.
- void set_control_queue (boost::shared_ptr< gr::msg_queue > control_queue)
  Sets the control_queue.
- unsigned int **processed_control_messages** ()
- unsigned int **applied_actions** ()
- std::shared_ptr< GNSSFlowgraph > flowgraph ()
  Instantiates a flowgraph.

8.23.1 Detailed Description

This class represents the main thread of the application, so the name is ControlThread. This is the GNSS Receiver Control Plane: it connects the flowgraph, starts running it, and while it does not stop, reads the control messages generated by the blocks, processes them, and applies the corresponding actions.

Definition at line 55 of file control_thread.h.

8.23.2 Constructor & Destructor Documentation

8.23.2.1 ControlThread::ControlThread ( )

Default constructor.

8.23.2.2 ControlThread::ControlThread (std::shared_ptr< ConfigurationInterface > *configuration*)

Constructor that initializes the class with parameters.

Parameters

in	*configuration*	Pointer to a ConfigurationInterface

8.23.2.3 ControlThread::~ControlThread ( )

Destructor.

8.23.3 Member Function Documentation

8.23.3.1 std::shared_ptr<GNSSFlowgraph> ControlThread::flowgraph ( ) [inline]

Instantiates a flowgraph.

Returns

Returns a smart pointer to a flowgraph object

Definition at line 110 of file control_thread.h.

8.23.3.2 void ControlThread::run ( )

Runs the control thread.

This is the main loop that reads and process the control messages:

- Connect the GNSS receiver flowgraph;
- Start the GNSS receiver flowgraph;

while (flowgraph_->running() && !stop_){

- Read control messages and process them; }

8.23.3.3 void ControlThread::set_control_queue (boost::shared_ptr< gr::msg_queue > *control_queue*)

Sets the control_queue.

Parameters

in	*boost::shared_ptr<gr::msg_queue>*	control_queue

The documentation for this class was generated from the following file:

- src/core/receiver/control_thread.h

8.24 cpu_multicorrelator Class Reference

Class that implements carrier wipe-off and correlators.

`#include <cpu_multicorrelator.h>`

Public Member Functions

- bool **init** (int max_signal_length_samples, int n_correlators)
- bool **set_local_code_and_taps** (int code_length_chips, const std::complex< float > *local_code_in, float *shifts_chips)
- bool **set_input_output_vectors** (std::complex< float > *corr_out, const std::complex< float > *sig_in)
- void **update_local_code** (int correlator_length_samples, float rem_code_phase_chips, float code_phase_step_chips)
- bool **Carrier_wipeoff_multicorrelator_resampler** (float rem_carrier_phase_in_rad, float phase_step_rad, float rem_code_phase_chips, float code_phase_step_chips, int signal_length_samples)
- bool **free** ()

8.24.1 Detailed Description

Class that implements carrier wipe-off and correlators.

Definition at line 44 of file cpu_multicorrelator.h.

The documentation for this class was generated from the following file:

- src/algorithms/tracking/libs/cpu_multicorrelator.h

8.25 cpu_multicorrelator_16sc Class Reference

Class that implements carrier wipe-off and correlators.

`#include <cpu_multicorrelator_16sc.h>`

Public Member Functions

- bool **init** (int max_signal_length_samples, int n_correlators)
- bool **set_local_code_and_taps** (int code_length_chips, const lv_16sc_t *local_code_in, float *shifts_chips)
- bool **set_input_output_vectors** (lv_16sc_t *corr_out, const lv_16sc_t *sig_in)
- void **update_local_code** (int correlator_length_samples, float rem_code_phase_chips, float code_phase_step_chips)
- bool **Carrier_wipeoff_multicorrelator_resampler** (float rem_carrier_phase_in_rad, float phase_step_rad, float rem_code_phase_chips, float code_phase_step_chips, int signal_length_samples)
- bool **free** ()

8.25.1 Detailed Description

Class that implements carrier wipe-off and correlators.

Definition at line 44 of file cpu_multicorrelator_16sc.h.

The documentation for this class was generated from the following file:

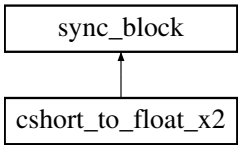
- src/algorithms/tracking/libs/cpu_multicorrelator_16sc.h

8.26 cshort_to_float_x2 Class Reference

This class adapts a std::complex<short> stream into two 32-bits (float) streams.

`#include <cshort_to_float_x2.h>`

Inheritance diagram for cshort_to_float_x2:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- cshort_to_float_x2_sptr **make_cshort_to_float_x2** ()

8.26.1 Detailed Description

This class adapts a std::complex<short> stream into two 32-bits (float) streams.

Definition at line 48 of file cshort_to_float_x2.h.

The documentation for this class was generated from the following file:

- src/algorithms/libs/cshort_to_float_x2.h

8.27 cuda_multicorrelator Class Reference

Class that implements carrier wipe-off and correlators using NVIDIA CUDA GPU accelerators.

`#include <cuda_multicorrelator.h>`

Public Member Functions

- bool **init_cuda_integrated_resampler** (int signal_length_samples, int code_length_chips, int n_correlators)
- bool **set_local_code_and_taps** (int code_length_chips, const std::complex< float > *local_codes_in, float *shifts_chips, int n_correlators)
- bool **set_input_output_vectors** (std::complex< float > *corr_out, std::complex< float > *sig_in)
- bool **free_cuda** ()
- bool **Carrier_wipeoff_multicorrelator_resampler_cuda** (float rem_carrier_phase_in_rad, float phase_↩ step_rad, float code_phase_step_chips, float rem_code_phase_chips, int signal_length_samples, int n_↩ correlators)

8.27.1 Detailed Description

Class that implements carrier wipe-off and correlators using NVIDIA CUDA GPU accelerators.

Definition at line 115 of file cuda_multicorrelator.h.

The documentation for this class was generated from the following file:

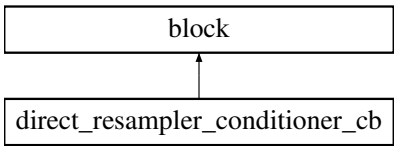
- src/algorithms/tracking/libs/cuda_multicorrelator.h

8.28 direct_resampler_conditioner_cb Class Reference

This class implements a direct resampler conditioner for std::complex<signed char>

`#include <direct_resampler_conditioner_cb.h>`

Inheritance diagram for direct_resampler_conditioner_cb:



Public Member Functions

- unsigned int **sample_freq_in** () const
- unsigned int **sample_freq_out** () const
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- direct_resampler_conditioner_cb_sptr **direct_resampler_make_conditioner_cb** (double sample_freq_in, double sample_freq_out)

8.28.1 Detailed Description

This class implements a direct resampler conditioner for std::complex<signed char>

Direct resampling without interpolation

Definition at line 50 of file direct_resampler_conditioner_cb.h.

The documentation for this class was generated from the following file:

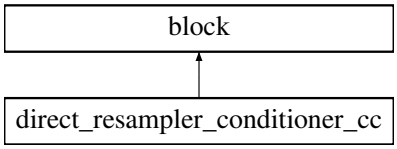
- src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cb.h

8.29 direct_resampler_conditioner_cc Class Reference

This class implements a direct resampler conditioner for complex data.

`#include <direct_resampler_conditioner_cc.h>`

Inheritance diagram for direct_resampler_conditioner_cc:



Public Member Functions

- unsigned int **sample_freq_in** () const
- unsigned int **sample_freq_out** () const
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- direct_resampler_conditioner_cc_sptr **direct_resampler_make_conditioner_cc** (double sample_freq_in, double sample_freq_out)

8.29.1 Detailed Description

This class implements a direct resampler conditioner for complex data.

Direct resampling without interpolation

Definition at line 56 of file direct_resampler_conditioner_cc.h.

The documentation for this class was generated from the following file:

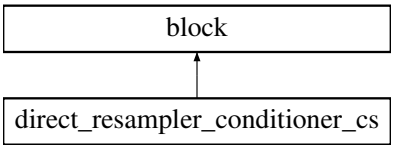
- src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cc.h

8.30 direct_resampler_conditioner_cs Class Reference

This class implements a direct resampler conditioner for std::complex<short>

`#include <direct_resampler_conditioner_cs.h>`

Inheritance diagram for direct_resampler_conditioner_cs:



Public Member Functions

- unsigned int **sample_freq_in** () const
- unsigned int **sample_freq_out** () const
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- direct_resampler_conditioner_cs_sptr **direct_resampler_make_conditioner_cs** (double sample_freq_in, double sample_freq_out)

8.30.1 Detailed Description

This class implements a direct resampler conditioner for std::complex<short>

Direct resampling without interpolation

Definition at line 50 of file direct_resampler_conditioner_cs.h.

The documentation for this class was generated from the following file:

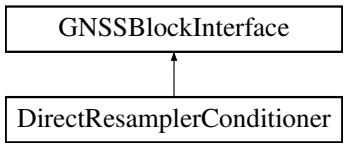
- src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cs.h

8.31 DirectResamplerConditioner Class Reference

Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

`#include <direct_resampler_conditioner.h>`

Inheritance diagram for DirectResamplerConditioner:



Public Member Functions

- **DirectResamplerConditioner** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream)
- std::string **role** ()
- std::string implementation ()

  returns "Direct_Resampler"
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.31.1 Detailed Description

Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

Definition at line 46 of file direct_resampler_conditioner.h.

8.31.2 Member Function Documentation

8.31.2.1 std::string DirectResamplerConditioner::implementation ( ) [inline], [virtual]

returns "Direct_Resampler"

Implements GNSSBlockInterface.

Definition at line 59 of file direct_resampler_conditioner.h.

The documentation for this class was generated from the following file:

- src/algorithms/resampler/adapters/direct_resampler_conditioner.h

8.32 Fast_Correction Struct Reference

Public Attributes

- Sbas_Time **d_tof**
- double **d_prc**
- double **d_rrc**
- double **d_dt**
- int **d_udre**
- int **d_ai**
- int **d_tlat**

8.32.1 Detailed Description

Definition at line 36 of file sbas_satellite_correction.h.

The documentation for this struct was generated from the following file:

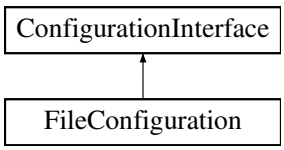
- src/core/system_parameters/sbas_satellite_correction.h

8.33 FileConfiguration Class Reference

This class is an implementation of the interface ConfigurationInterface.

`#include <file_configuration.h>`

Inheritance diagram for FileConfiguration:



Public Member Functions

- **FileConfiguration** (std::string filename)
- ~FileConfiguration ()

  Virtual destructor.
- std::string **property** (std::string property_name, std::string default_value)
- bool **property** (std::string property_name, bool default_value)
- long **property** (std::string property_name, long default_value)
- int **property** (std::string property_name, int default_value)
- unsigned int **property** (std::string property_name, unsigned int default_value)
- unsigned short **property** (std::string property_name, unsigned short default_value)
- float **property** (std::string property_name, float default_value)
- double **property** (std::string property_name, double default_value)
- void **set_property** (std::string property_name, std::string value)

8.33.1 Detailed Description

This class is an implementation of the interface ConfigurationInterface.

Derived from ConfigurationInterface, this class implements an interface to a configuration file. This implementation has a text file as the source for the values of the parameters. The file is in the INI format, containing sections and pairs of names and values. For more information about the INI format, see http://en.wikipedia.org/wiki/INI_file

Definition at line 57 of file file_configuration.h.

8.33.2 Constructor & Destructor Documentation

8.33.2.1 FileConfiguration::~FileConfiguration ( )

Virtual destructor.

The documentation for this class was generated from the following file:

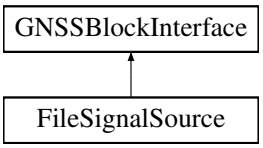
- src/core/receiver/file_configuration.h

8.34 FileSignalSource Class Reference

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

`#include <file_signal_source.h>`

Inheritance diagram for FileSignalSource:



Public Member Functions

- **FileSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "File_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **filename** ()
- std::string **item_type** ()
- bool **repeat** ()
- long **sampling_frequency** ()
- long **samples** ()

8.34.1 Detailed Description

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

Definition at line 53 of file file_signal_source.h.

8.34.2 Member Function Documentation

8.34.2.1 std::string FileSignalSource::implementation ( ) [inline], [virtual]

Returns "File_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 69 of file file_signal_source.h.

The documentation for this class was generated from the following file:

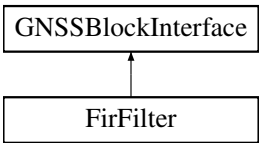
- src/algorithms/signal_source/adapters/file_signal_source.h

8.35 FirFilter Class Reference

This class adapts a GNU Radio gr_fir_filter designed with pm_remez.

`#include <fir_filter.h>`

Inheritance diagram for FirFilter:



Public Member Functions

- FirFilter (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_↩ streams)

  Constructor.
- virtual ~FirFilter ()

  Destructor.
- std::string **role** ()
- std::string implementation ()

  Returns "Fir_Filter".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.35.1 Detailed Description

This class adapts a GNU Radio gr_fir_filter designed with pm_remez.

See Parks-McClellan FIR filter design, http://en.wikipedia.org/wiki/Parks-McClellan_↩ filter_design_algorithm Calculates the optimal (in the Chebyshev/minimax sense) FIR filter impulse response given a set of band edges, the desired response on those bands, and the weight given to the error in those bands.

Definition at line 62 of file fir_filter.h.

8.35.2 Constructor & Destructor Documentation

8.35.2.1 FirFilter::FirFilter (ConfigurationInterface * *configuration,* std::string *role,* unsigned int *in_streams,* unsigned int *out_streams*)

Constructor.

8.35.2.2 virtual FirFilter::~FirFilter ( ) [virtual]

Destructor.

8.35.3 Member Function Documentation

8.35.3.1 std::string FirFilter::implementation ( ) [inline], [virtual]

Returns "Fir_Filter".

Implements GNSSBlockInterface.

Definition at line 79 of file fir_filter.h.

The documentation for this class was generated from the following file:

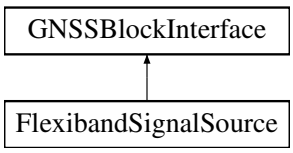
- src/algorithms/input_filter/adapters/fir_filter.h

8.36 FlexibandSignalSource Class Reference

This class configures and reads samples from Teleorbit Flexiband front-end. This software requires a Flexiband GNU Radio driver installed (not included with GNSS-SDR).

`#include <flexiband_signal_source.h>`

Inheritance diagram for FlexibandSignalSource:



Public Member Functions

- **FlexibandSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream, gr::msg_queue::sptr queue)
- std::string **role** ()
- std::string implementation ()

  Returns "Flexiband_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- gr::basic_block_sptr **get_right_block** (int RF_channel)

8.36.1 Detailed Description

This class configures and reads samples from Teleorbit Flexiband front-end. This software requires a Flexiband GNU Radio driver installed (not included with GNSS-SDR).

Definition at line 52 of file flexiband_signal_source.h.

8.36.2 Member Function Documentation

8.36.2.1 std::string FlexibandSignalSource::implementation ( ) [inline], [virtual]

Returns "Flexiband_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 68 of file flexiband_signal_source.h.

The documentation for this class was generated from the following file:

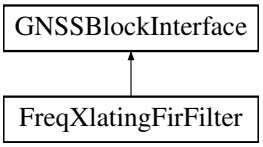
- src/algorithms/signal_source/adapters/flexiband_signal_source.h

8.37 FreqXlatingFirFilter Class Reference

This class adapts a gnuradio gr_freq_xlating_fir_filter designed with pm_remez.

`#include <freq_xlating_fir_filter.h>`

Inheritance diagram for FreqXlatingFirFilter:



Public Member Functions

- **FreqXlatingFirFilter** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Freq_Xlating_Fir_Filter".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.37.1 Detailed Description

This class adapts a gnuradio gr_freq_xlating_fir_filter designed with pm_remez.

Construct a FIR filter with the given taps and a composite frequency translation that shifts intermediate_freq_ down to zero Hz. The frequency translation logically comes before the filtering operation.

See Parks-McClellan FIR filter design, http://en.wikipedia.org/wiki/Parks-McClellan_↩filter_design_algorithm Calculates the optimal (in the Chebyshev/minimax sense) FIR filter impulse response given a set of band edges, the desired response on those bands, and the weight given to the error in those bands.

Definition at line 63 of file freq_xlating_fir_filter.h.

8.37.2 Member Function Documentation

8.37.2.1 std::string FreqXlatingFirFilter::implementation ( ) [inline], [virtual]

Returns "Freq_Xlating_Fir_Filter".

Implements GNSSBlockInterface.

Definition at line 77 of file freq_xlating_fir_filter.h.

The documentation for this class was generated from the following file:

- src/algorithms/input_filter/adapters/freq_xlating_fir_filter.h

8.38 FrontEndCal Class Reference

Public Member Functions

- void set_configuration (std::shared_ptr< ConfigurationInterface > configuration)

  Sets the configuration data required by get_ephemeris function.
- bool get_ephemeris ()

  This function connects to a Secure User Location Protocol (SUPL) server to obtain the current GPS ephemeris and GPS assistance data. It requires the configuration parameters set by set_configuration function.
- double estimate_doppler_from_eph (unsigned int PRN, double TOW, double lat, double lon, double height)

  This function estimates the GPS L1 satellite Doppler frequency [Hz] using the following data: 1- Orbital model from the ephemeris 2- Approximate GPS Time of Week (TOW) 3- Approximate receiver Latitude and Longitude (WGS-84)
- void GPS_L1_front_end_model_E4000 (double f_bb_true_Hz, double f_bb_meas_Hz, double fs_nominal↩_hz, double *estimated_fs_Hz, double *estimated_f_if_Hz, double *f_osc_err_ppm)

  This function models the Elonics E4000 + RTL2832 front-end Inputs: f_bb_true_Hz - Ideal output frequency in baseband [Hz] f_in_bb_meas_Hz - measured output frequency in baseband [Hz] Outputs: estimated_fs_Hz - Sampling frequency estimation based on the measurements and the front-end model estimated_f_if_bb_Hz - Equivalent bb if frequency estimation based on the measurements and the front-end model Front-end TUNER Elonics E4000 + RTL2832 sampler For GPS L1 1575.42 MHz.

8.38.1 Detailed Description

Definition at line 40 of file front_end_cal.h.

8.38.2 Member Function Documentation

8.38.2.1 double FrontEndCal::estimate_doppler_from_eph (unsigned int *PRN*, double *TOW*, double *lat*, double *lon*, double *height*)

This function estimates the GPS L1 satellite Doppler frequency [Hz] using the following data: 1- Orbital model from the ephemeris 2- Approximate GPS Time of Week (TOW) 3- Approximate receiver Latitude and Longitude (WGS-84)

8.38.2.2 bool FrontEndCal::get_ephemeris ( )

This function connects to a Secure User Location Protocol (SUPL) server to obtain the current GPS ephemeris and GPS assistance data. It requires the configuration parameters set by set_configuration function.

8.38.2.3 void FrontEndCal::GPS_L1_front_end_model_E4000 (double *f_bb_true_Hz*, double *f_bb_meas_Hz*, double *fs_nominal_hz*, double * *estimated_fs_Hz*, double * *estimated_f_if_Hz*, double * *f_osc_err_ppm*)

This function models the Elonics E4000 + RTL2832 front-end Inputs: f_bb_true_Hz - Ideal output frequency in baseband [Hz] f_in_bb_meas_Hz - measured output frequency in baseband [Hz] Outputs: estimated_fs_Hz - Sampling frequency estimation based on the measurements and the front-end model estimated_f_if_bb_Hz - Equivalent bb if frequency estimation based on the measurements and the front-end model Front-end TUNER Elonics E4000 + RTL2832 sampler For GPS L1 1575.42 MHz.

8.38.2.4 void FrontEndCal::set_configuration (std::shared_ptr< **ConfigurationInterface** > *configuration*)

Sets the configuration data required by get_ephemeris function.

The documentation for this class was generated from the following file:

- src/utils/front-end-cal/front_end_cal.h

8.39 Galileo_Almanac Class Reference

This class is a storage for the GALILEO ALMANAC data as described in GALILEO ICD.

`#include <galileo_almanac.h>`

Public Member Functions

- Galileo_Almanac ()

  Default constructor.

Public Attributes

- int **IOD_a_7**
- double **WN_a_7**
- double **t0a_7**
- int **SVID1_7**
- double **DELTA_A_7**
- double **e_7**
- double **omega_7**
- double **delta_i_7**
- double **Omega0_7**
- double **Omega_dot_7**
- double **M0_7**
- int **IOD_a_8**
- double **af0_8**
- double **af1_8**
- double **E5b_HS_8**
- double **E1B_HS_8**
- double **E5a_HS_8**
- int **SVID2_8**
- double **DELTA_A_8**
- double **e_8**
- double **omega_8**
- double **delta_i_8**
- double **Omega0_8**
- double **Omega_dot_8**
- int **IOD_a_9**
- double **WN_a_9**
- double **t0a_9**
- double **M0_9**
- double **af0_9**
- double **af1_9**
- double **E5b_HS_9**
- double **E1B_HS_9**
- double **E5a_HS_9**
- int **SVID3_9**
- double **DELTA_A_9**
- double **e_9**
- double **omega_9**
- double **delta_i_9**
- int **IOD_a_10**
- double **Omega0_10**
- double **Omega_dot_10**
- double **M0_10**
- double **af0_10**
- double **af1_10**
- double **E5b_HS_10**
- double **E1B_HS_10**
- double **E5a_HS_10**
- double **A_0G_10**
- double **A_1G_10**
- double **t_0G_10**
- double **WN_0G_10**

8.39.1 Detailed Description

This class is a storage for the GALILEO ALMANAC data as described in GALILEO ICD.

See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf paragraph 5.1.10

Definition at line 40 of file galileo_almanac.h.

8.39.2 Constructor & Destructor Documentation

8.39.2.1 Galileo_Almanac::Galileo_Almanac ( )

Default constructor.

The documentation for this class was generated from the following file:

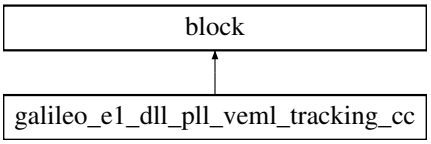
- src/core/system_parameters/galileo_almanac.h

8.40 galileo_e1_dll_pll_veml_tracking_cc Class Reference

This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

`#include <galileo_e1_dll_pll_veml_tracking_cc.h>`

Inheritance diagram for galileo_e1_dll_pll_veml_tracking_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007.
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- galileo_e1_dll_pll_veml_tracking_cc_sptr **galileo_e1_dll_pll_veml_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips, float very_early_late_space_chips)

8.40.1 Detailed Description

This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

Definition at line 63 of file galileo_e1_dll_pll_veml_tracking_cc.h.

8.40.2 Member Function Documentation

8.40.2.1 int galileo_e1_dll_pll_veml_tracking_cc::general_work (int *noutput_items*, gr_vector_int & *ninput_items*, gr_vector_const_void_star & *input_items*, gr_vector_void_star & *output_items*)

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007.

The documentation for this class was generated from the following file:

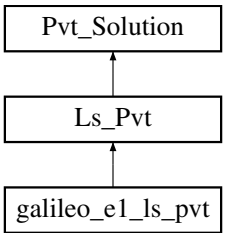
- src/algorithms/tracking/gnuradio_blocks/galileo_e1_dll_pll_veml_tracking_cc.h

8.41 galileo_e1_ls_pvt Class Reference

This class implements a simple PVT Least Squares solution.

`#include <galileo_e1_ls_pvt.h>`

Inheritance diagram for galileo_e1_ls_pvt:



Public Member Functions

- **galileo_e1_ls_pvt** (int nchannels, std::string dump_filename, bool flag_dump_to_file)
- bool **get_PVT** (std::map< int, Gnss_Synchro > gnss_pseudoranges_map, double galileo_current_time, bool flag_averaging)

Public Attributes

- int d_nchannels

  Number of available channels for positioning.
- Galileo_Navigation_Message * **d_ephemeris**
- std::map< int, Galileo_Ephemeris > galileo_ephemeris_map

  Map storing new Galileo_Ephemeris.
- Galileo_Utc_Model **galileo_utc_model**
- Galileo_Iono **galileo_iono**
- Galileo_Almanac **galileo_almanac**
- double **d_galileo_current_time**
- bool **d_flag_dump_enabled**
- bool **d_flag_averaging**
- std::string **d_dump_filename**
- std::ofstream **d_dump_file**

8.41.1 Detailed Description

This class implements a simple PVT Least Squares solution.

Definition at line 49 of file galileo_e1_ls_pvt.h.

8.41.2 Member Data Documentation

8.41.2.1 int galileo_e1_ls_pvt::d_nchannels

Number of available channels for positioning.

Definition at line 57 of file galileo_e1_ls_pvt.h.

8.41.2.2 std::map<int,Galileo_Ephemeris> galileo_e1_ls_pvt::galileo_ephemeris_map

Map storing new Galileo_Ephemeris.

Definition at line 60 of file galileo_e1_ls_pvt.h.

The documentation for this class was generated from the following file:

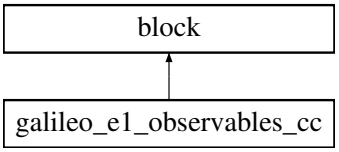
- src/algorithms/PVT/libs/galileo_e1_ls_pvt.h

8.42 galileo_e1_observables_cc Class Reference

This class implements a block that computes Galileo observables.

`#include <galileo_e1_observables_cc.h>`

Inheritance diagram for galileo_e1_observables_cc:



Public Member Functions

- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- galileo_e1_observables_cc_sptr **galileo_e1_make_observables_cc** (unsigned int nchannels, bool dump, std::string dump_filename, unsigned int deep_history)

8.42.1 Detailed Description

This class implements a block that computes Galileo observables.

Definition at line 52 of file galileo_e1_observables_cc.h.

The documentation for this class was generated from the following file:

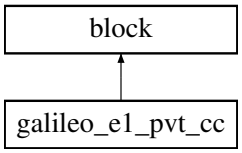
- src/algorithms/observables/gnuradio_blocks/galileo_e1_observables_cc.h

8.43 galileo_e1_pvt_cc Class Reference

This class implements a block that computes the PVT solution with Galileo E1 signals.

`#include <galileo_e1_pvt_cc.h>`

Inheritance diagram for galileo_e1_pvt_cc:



Public Member Functions

- ~galileo_e1_pvt_cc ()

  Default destructor.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  PVT Signal Processing.

Friends

- galileo_e1_pvt_cc_sptr **galileo_e1_make_pvt_cc** (unsigned int nchannels, bool dump, std::string dump↩ _filename, int averaging_depth, bool flag_averaging, int output_rate_ms, int display_rate_ms, bool flag_↩ nmea_tty_port, std::string nmea_dump_filename, std::string nmea_dump_devname, bool flag_rtcm_server, bool flag_rtcm_tty_port, unsigned short rtcm_tcp_port, unsigned short rtcm_station_id, std::map< int, int > rtcm_msg_rate_ms, std::string rtcm_dump_devname)

8.43.1 Detailed Description

This class implements a block that computes the PVT solution with Galileo E1 signals.

Definition at line 73 of file galileo_e1_pvt_cc.h.

8.43.2 Constructor & Destructor Documentation

8.43.2.1 galileo_e1_pvt_cc::~galileo_e1_pvt_cc ( )

Default destructor.

8.43.3 Member Function Documentation

8.43.3.1 int galileo_e1_pvt_cc::general_work (int *noutput_items*, gr_vector_int & *ninput_items*, gr_vector_const_void_star & *input_items*, gr_vector_void_star & *output_items*)

PVT Signal Processing.

The documentation for this class was generated from the following file:

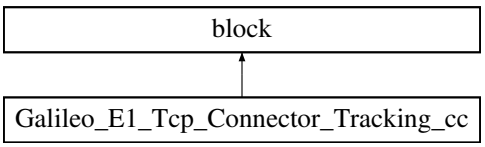
- src/algorithms/PVT/gnuradio_blocks/galileo_e1_pvt_cc.h

8.44 Galileo_E1_Tcp_Connector_Tracking_cc Class Reference

This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

`#include <galileo_e1_tcp_connector_tracking_cc.h>`

Inheritance diagram for Galileo_E1_Tcp_Connector_Tracking_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- galileo_e1_tcp_connector_tracking_cc_sptr **galileo_e1_tcp_connector_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips, float very_early_late_space_chips, size_t port_ch0)

8.44.1 Detailed Description

This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

Definition at line 72 of file galileo_e1_tcp_connector_tracking_cc.h.

The documentation for this class was generated from the following file:

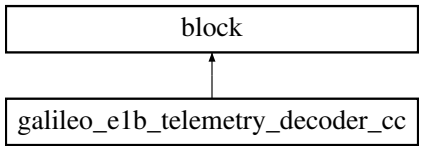
- src/algorithms/tracking/gnuradio_blocks/galileo_e1_tcp_connector_tracking_cc.h

8.45 galileo_e1b_telemetry_decoder_cc Class Reference

This class implements a block that decodes the INAV data defined in Galileo ICD.

`#include <galileo_e1b_telemetry_decoder_cc.h>`

Inheritance diagram for galileo_e1b_telemetry_decoder_cc:



Public Member Functions

- void set_satellite (Gnss_Satellite satellite)

  Set satellite PRN.
- void set_channel (int channel)

  Set receiver's channel.
- void set_decimation (int decimation)

  Set decimation factor to average the GPS synchronization estimation output from the tracking module.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  This is where all signal processing takes place.
- void forecast (int noutput_items, gr_vector_int &ninput_items_required)

  Function which tells the scheduler how many input items are required to produce noutput_items output items.

Public Attributes

- int **flag_even_word_arrived**

Friends

- galileo_e1b_telemetry_decoder_cc_sptr **galileo_e1b_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

8.45.1 Detailed Description

This class implements a block that decodes the INAV data defined in Galileo ICD.

Definition at line 59 of file galileo_e1b_telemetry_decoder_cc.h.

8.45.2 Member Function Documentation

8.45.2.1 void galileo_e1b_telemetry_decoder_cc::forecast (int *noutput_items*, gr_vector_int & *ninput_items_required*)

Function which tells the scheduler how many input items are required to produce noutput_items output items.

8.45.2.2 int galileo_e1b_telemetry_decoder_cc::general_work (int *noutput_items*, gr_vector_int & *ninput_items*, gr_vector_const_void_star & *input_items*, gr_vector_void_star & *output_items*)

This is where all signal processing takes place.

8.45.2.3 void galileo_e1b_telemetry_decoder_cc::set_channel (int *channel*)

Set receiver's channel.

8.45.2.4 void galileo_e1b_telemetry_decoder_cc::set_decimation (int *decimation*)

Set decimation factor to average the GPS synchronization estimation output from the tracking module.

8.45.2.5 void galileo_e1b_telemetry_decoder_cc::set_satellite (Gnss_Satellite *satellite*)

Set satellite PRN.

The documentation for this class was generated from the following file:

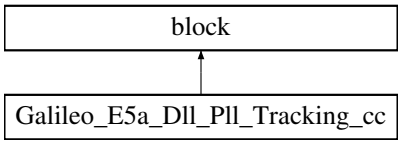
- src/algorithms/telemetry_decoder/gnuradio_blocks/galileo_e1b_telemetry_decoder_cc.h

8.46 Galileo_E5a_Dll_Pll_Tracking_cc Class Reference

This class implements a DLL + PLL tracking loop block.

`#include <galileo_e5a_dll_pll_tracking_cc.h>`

Inheritance diagram for Galileo_E5a_Dll_Pll_Tracking_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- galileo_e5a_dll_pll_tracking_cc_sptr **galileo_e5a_dll_pll_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float pll_↩ bw_init_hz, float dll_bw_init_hz, int ti_ms, float early_late_space_chips)

8.46.1 Detailed Description

This class implements a DLL + PLL tracking loop block.

Definition at line 72 of file galileo_e5a_dll_pll_tracking_cc.h.

The documentation for this class was generated from the following file:

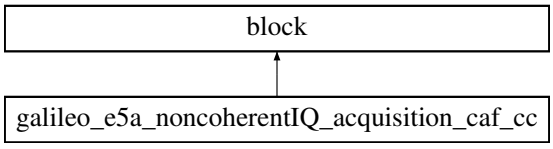
- src/algorithms/tracking/gnuradio_blocks/galileo_e5a_dll_pll_tracking_cc.h

8.47 galileo_e5a_noncoherentIQ_acquisition_caf_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition.

`#include <galileo_e5a_noncoherent_iq_acquisition_caf_cc.h>`

Inheritance diagram for galileo_e5a_noncoherentIQ_acquisition_caf_cc:



Public Member Functions

- ~galileo_e5a_noncoherentIQ_acquisition_caf_cc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code, std::complex< float > *codeQ)

  Sets local code for PCPS acquisition algorithm.
- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.

Friends

- galileo_e5a_noncoherentIQ_acquisition_caf_cc_sptr **galileo_e5a_noncoherentIQ_make_acquisition_caf_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_transition_flag, bool dump, std::string dump_filename, bool both_signal_components_, int CAF_window_hz_, int Zero_padding_)

8.47.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Check An Open Source Galileo E1 Software Receiver, Algorithm 1, for a pseudocode description of this implementation.

Definition at line 70 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

8.47.2 Constructor & Destructor Documentation

8.47.2.1 galileo_e5a_noncoherentIQ_acquisition_caf_cc::~galileo_e5a_noncoherentIQ_acquisition_caf_cc ( )

Default destructor.

8.47.3 Member Function Documentation

8.47.3.1 int galileo_e5a_noncoherentIQ_acquisition_caf_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.47.3.2 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.47.3.3 unsigned int galileo_e5a_noncoherentIQ_acquisition_caf_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 173 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

References init(), and set_local_code().

8.47.3.4 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 194 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

References set_state().

8.47.3.5 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 210 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

8.47.3.6 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 229 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

8.47.3.7 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 238 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

References general_work().

8.47.3.8 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 165 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

8.47.3.9 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_local_code (std::complex< float > * *code*, std::complex< float > * *codeQ*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.47.3.10 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.47.3.11 void galileo_e5a_noncoherentIQ_acquisition_caf_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 220 of file galileo_e5a_noncoherent_iq_acquisition_caf_cc.h.

The documentation for this class was generated from the following file:

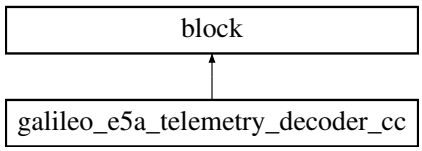
- src/algorithms/acquisition/gnuradio_blocks/galileo_e5a_noncoherent_iq_acquisition_caf_cc.h

8.48 galileo_e5a_telemetry_decoder_cc Class Reference

This class implements a block that decodes the FNAV data defined in Galileo ICD.

`#include <galileo_e5a_telemetry_decoder_cc.h>`

Inheritance diagram for galileo_e5a_telemetry_decoder_cc:



Public Member Functions

- void set_satellite (Gnss_Satellite satellite)

  Set satellite PRN.
- void set_channel (int channel)

  Set receiver's channel.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  This is where all signal processing takes place.
- void forecast (int noutput_items, gr_vector_int &ninput_items_required)

  Function which tells the scheduler how many input items are required to produce noutput_items output items.

Friends

- galileo_e5a_telemetry_decoder_cc_sptr **galileo_e5a_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

8.48.1 Detailed Description

This class implements a block that decodes the FNAV data defined in Galileo ICD.

Definition at line 64 of file galileo_e5a_telemetry_decoder_cc.h.

8.48.2 Member Function Documentation

8.48.2.1 void galileo_e5a_telemetry_decoder_cc::forecast (int *noutput_items,* gr_vector_int & *ninput_items_required*)

Function which tells the scheduler how many input items are required to produce noutput_items output items.

8.48.2.2 int galileo_e5a_telemetry_decoder_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

This is where all signal processing takes place.

8.48.2.3 void galileo_e5a_telemetry_decoder_cc::set_channel (int *channel*)

Set receiver's channel.

8.48.2.4 void galileo_e5a_telemetry_decoder_cc::set_satellite (Gnss_Satellite *satellite*)

Set satellite PRN.

The documentation for this class was generated from the following file:

- src/algorithms/telemetry_decoder/gnuradio_blocks/galileo_e5a_telemetry_decoder_cc.h

8.49 Galileo_Ephemeris Class Reference

This class is a storage and orbital model functions for the Galileo SV ephemeris data as described in Galileo ICD paragraph 5.1.1 (See https://www.gsc-europa.eu/system/files/galileo_documents/↩Galileo_OS_SIS_ICD.pdf)

`#include <galileo_ephemeris.h>`

Public Member Functions

- void satellitePosition (double transmitTime)

  Computes the ECEF SV coordinates and ECEF velocity.
- double Galileo_System_Time (double WN, double TOW)

  Galileo System Time (GST), ICD paragraph 5.1.2.
- double sv_clock_drift (double transmitTime)

  Satellite Time Correction Algorithm, ICD 5.1.4.
- double sv_clock_relativistic_term (double transmitTime)

  Satellite Time Correction Algorithm, ICD 5.1.4.
- template<class Archive >
  void serialize (Archive &archive, const unsigned int version)

  Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

Public Attributes

- bool **flag_all_ephemeris**
- int **IOD_ephemeris**
- int **IOD_nav_1**
- int **SV_ID_PRN_4**
- double M0_1

  Mean anomaly at reference time [semi-circles].
- double delta_n_3

  Mean motion difference from computed value [semi-circles/sec].
- double e_1

  Eccentricity.
- double A_1

  Square root of the semi-major axis [metres^ 1/2].
- double OMEGA_0_2

  Longitude of ascending node of orbital plane at weekly epoch [semi-circles].
- double i_0_2

  Inclination angle at reference time [semi-circles].
- double omega_2

  Argument of perigee [semi-circles].
- double OMEGA_dot_3

  Rate of right ascension [semi-circles/sec].
- double iDot_2

  Rate of inclination angle [semi-circles/sec].
- double C_uc_3

  Amplitude of the cosine harmonic correction term to the argument of latitude [radians].
- double C_us_3

  Amplitude of the sine harmonic correction term to the argument of latitude [radians].
- double C_rc_3

  Amplitude of the cosine harmonic correction term to the orbit radius [meters].
- double C_rs_3

  Amplitude of the sine harmonic correction term to the orbit radius [meters].
- double C_ic_4

  Amplitude of the cosine harmonic correction term to the angle of inclination [radians].
- double C_is_4

  Amplitude of the sine harmonic correction term to the angle of inclination [radians].

- double t0e_1

  Ephemeris reference time [s].
- double t0c_4

  Clock correction data reference Time of Week [sec].
- double af0_4

  SV clock bias correction coefficient [s].
- double af1_4

  SV clock drift correction coefficient [s/s].
- double af2_4

  SV clock drift rate correction coefficient [s/s^ 2].
- double WN_5

  Week number.
- double TOW_5

  Time of Week.
- double **Galileo_satClkDrift**
- double Galileo_dtr

  relativistic clock correction term
- double **SISA_3**
- unsigned int E5a_HS

  E5a Signal Health Status.
- double E5b_HS_5

  E5b Signal Health Status.
- double E1B_HS_5

  E1B Signal Health Status.
- bool E5a_DVS

  E5a Data Validity Status.
- double E5b_DVS_5

  E5b Data Validity Status.
- double E1B_DVS_5

  E1B Data Validity Status.
- double BGD_E1E5a_5

  E1-E5a Broadcast Group Delay [s].
- double BGD_E1E5b_5

  E1-E5b Broadcast Group Delay [s].
- double d_satpos_X

  Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.
- double d_satpos_Y

  Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.
- double d_satpos_Z

  Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).
- double d_satvel_X

  Earth-fixed velocity coordinate x of the satellite [m].
- double d_satvel_Y

  Earth-fixed velocity coordinate y of the satellite [m].
- double d_satvel_Z

  Earth-fixed velocity coordinate z of the satellite [m].
- unsigned int i_satellite_PRN

  SV PRN NUMBER.

8.49.1 Detailed Description

This class is a storage and orbital model functions for the Galileo SV ephemeris data as described in Galileo ICD paragraph 5.1.1 (See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf)

Definition at line 45 of file galileo_ephemeris.h.

8.49.2 Member Function Documentation

8.49.2.1 double Galileo_Ephemeris::Galileo_System_Time (double *WN*, double *TOW*)

Galileo System Time (GST), ICD paragraph 5.1.2.

8.49.2.2 void Galileo_Ephemeris::satellitePosition (double *transmitTime*)

Computes the ECEF SV coordinates and ECEF velocity.

8.49.2.3 template<class Archive > void Galileo_Ephemeris::serialize (Archive & *archive*, const unsigned int *version*) [inline]

Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

Definition at line 119 of file galileo_ephemeris.h.

8.49.2.4 double Galileo_Ephemeris::sv_clock_drift (double *transmitTime*)

Satellite Time Correction Algorithm, ICD 5.1.4.

8.49.2.5 double Galileo_Ephemeris::sv_clock_relativistic_term (double *transmitTime*)

Satellite Time Correction Algorithm, ICD 5.1.4.

8.49.3 Member Data Documentation

8.49.3.1 double Galileo_Ephemeris::A_1

Square root of the semi-major axis [metres^1/2].

Definition at line 57 of file galileo_ephemeris.h.

8.49.3.2 double Galileo_Ephemeris::af0_4

SV clock bias correction coefficient [s].

Definition at line 73 of file galileo_ephemeris.h.

8.49.3.3 double Galileo_Ephemeris::af1_4

SV clock drift correction coefficient [s/s].

Definition at line 74 of file galileo_ephemeris.h.

8.49.3.4 double Galileo_Ephemeris::af2_4

SV clock drift rate correction coefficient [s/s^2].

Definition at line 75 of file galileo_ephemeris.h.

8.49.3.5 double Galileo_Ephemeris::BGD_E1E5a_5

E1-E5a Broadcast Group Delay [s].

Definition at line 93 of file galileo_ephemeris.h.

8.49.3.6 double Galileo_Ephemeris::BGD_E1E5b_5

E1-E5b Broadcast Group Delay [s].

Definition at line 94 of file galileo_ephemeris.h.

8.49.3.7 double Galileo_Ephemeris::C_ic_4

Amplitude of the cosine harmonic correction term to the angle of inclination [radians].

Definition at line 67 of file galileo_ephemeris.h.

8.49.3.8 double Galileo_Ephemeris::C_is_4

Amplitude of the sine harmonic correction term to the angle of inclination [radians].

Definition at line 68 of file galileo_ephemeris.h.

8.49.3.9 double Galileo_Ephemeris::C_rc_3

Amplitude of the cosine harmonic correction term to the orbit radius [meters].

Definition at line 65 of file galileo_ephemeris.h.

8.49.3.10 double Galileo_Ephemeris::C_rs_3

Amplitude of the sine harmonic correction term to the orbit radius [meters].

Definition at line 66 of file galileo_ephemeris.h.

8.49.3.11 double Galileo_Ephemeris::C_uc_3

Amplitude of the cosine harmonic correction term to the argument of latitude [radians].

Definition at line 63 of file galileo_ephemeris.h.

8.49.3.12 double Galileo_Ephemeris::C_us_3

Amplitude of the sine harmonic correction term to the argument of latitude [radians].

Definition at line 64 of file galileo_ephemeris.h.

8.49.3.13 double Galileo_Ephemeris::d_satpos_X

Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

Definition at line 97 of file galileo_ephemeris.h.

8.49.3.14 double Galileo_Ephemeris::d_satpos_Y

Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

Definition at line 98 of file galileo_ephemeris.h.

8.49.3.15 double Galileo_Ephemeris::d_satpos_Z

Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

Definition at line 99 of file galileo_ephemeris.h.

8.49.3.16 double Galileo_Ephemeris::d_satvel_X

Earth-fixed velocity coordinate x of the satellite [m].

Definition at line 102 of file galileo_ephemeris.h.

8.49.3.17 double Galileo_Ephemeris::d_satvel_Y

Earth-fixed velocity coordinate y of the satellite [m].

Definition at line 103 of file galileo_ephemeris.h.

8.49.3.18 double Galileo_Ephemeris::d_satvel_Z

Earth-fixed velocity coordinate z of the satellite [m].

Definition at line 104 of file galileo_ephemeris.h.

8.49.3.19 double Galileo_Ephemeris::delta_n_3

Mean motion difference from computed value [semi-circles/sec].

Definition at line 55 of file galileo_ephemeris.h.

8.49.3.20 double Galileo_Ephemeris::E1B_DVS_5

E1B Data Validity Status.

Definition at line 91 of file galileo_ephemeris.h.

8.49.3.21 double Galileo_Ephemeris::E1B_HS_5

E1B Signal Health Status.

Definition at line 88 of file galileo_ephemeris.h.

8.49.3.22 bool Galileo_Ephemeris::E5a_DVS

E5a Data Validity Status.

Definition at line 89 of file galileo_ephemeris.h.

8.49.3.23 unsigned int Galileo_Ephemeris::E5a_HS

E5a Signal Health Status.

Definition at line 86 of file galileo_ephemeris.h.

8.49.3.24 double Galileo_Ephemeris::E5b_DVS_5

E5b Data Validity Status.

Definition at line 90 of file galileo_ephemeris.h.

8.49.3.25 double Galileo_Ephemeris::E5b_HS_5

E5b Signal Health Status.

Definition at line 87 of file galileo_ephemeris.h.

8.49.3.26 double Galileo_Ephemeris::e_1

Eccentricity.

Definition at line 56 of file galileo_ephemeris.h.

8.49.3.27 double Galileo_Ephemeris::Galileo_dtr

relativistic clock correction term

Definition at line 82 of file galileo_ephemeris.h.

8.49.3.28 double Galileo_Ephemeris::i_0_2

Inclination angle at reference time [semi-circles].

Definition at line 59 of file galileo_ephemeris.h.

8.49.3.29 unsigned int Galileo_Ephemeris::i_satellite_PRN

SV PRN NUMBER.

Definition at line 106 of file galileo_ephemeris.h.

8.49.3.30 double Galileo_Ephemeris::iDot_2

Rate of inclination angle [semi-circles/sec].

Definition at line 62 of file galileo_ephemeris.h.

8.49.3.31 double Galileo_Ephemeris::M0_1

Mean anomaly at reference time [semi-circles].

Definition at line 54 of file galileo_ephemeris.h.

8.49.3.32 double Galileo_Ephemeris::OMEGA_0_2

Longitude of ascending node of orbital plane at weekly epoch [semi-circles].

Definition at line 58 of file galileo_ephemeris.h.

8.49.3.33 double Galileo_Ephemeris::omega_2

Argument of perigee [semi-circles].

Definition at line 60 of file galileo_ephemeris.h.

8.49.3.34 double Galileo_Ephemeris::OMEGA_dot_3

Rate of right ascension [semi-circles/sec].

Definition at line 61 of file galileo_ephemeris.h.

8.49.3.35 double Galileo_Ephemeris::t0c_4

Clock correction data reference Time of Week [sec].

Definition at line 72 of file galileo_ephemeris.h.

8.49.3.36 double Galileo_Ephemeris::t0e_1

Ephemeris reference time [s].

Definition at line 69 of file galileo_ephemeris.h.

8.49.3.37 double Galileo_Ephemeris::TOW_5

Time of Week.

Definition at line 80 of file galileo_ephemeris.h.

8.49.3.38 double Galileo_Ephemeris::WN_5

Week number.

Definition at line 79 of file galileo_ephemeris.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/galileo_ephemeris.h

8.50 Galileo_Fnav_Message Class Reference

This class handles the Galileo F/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.↩eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf.

`#include <galileo_fnav_message.h>`

Public Member Functions

- void **split_page** (std::string page_string)
- void **reset** ()
- bool **have_new_ephemeris** ()
- bool **have_new_iono_and_GST** ()
- bool **have_new_utc_model** ()
- bool **have_new_almanac** ()
- Galileo_Ephemeris **get_ephemeris** ()
- Galileo_Iono **get_iono** ()
- Galileo_Utc_Model **get_utc_model** ()
- Galileo_Almanac **get_almanac** ()

Public Attributes

- bool **flag_CRC_test**
- bool flag_all_ephemeris

  Flag indicating that all words containing ephemeris have been received.
- bool flag_ephemeris_1

  Flag indicating that ephemeris 1/3 (word 2) have been received.
- bool flag_ephemeris_2

  Flag indicating that ephemeris 2/3 (word 3) have been received.
- bool flag_ephemeris_3

  Flag indicating that ephemeris 3/3 (word 4) have been received.
- bool flag_iono_and_GST

  Flag indicating that ionospheric and GST parameters (word 1) have been received.
- bool **flag_TOW_1**
- bool **flag_TOW_2**
- bool **flag_TOW_3**
- bool **flag_TOW_4**
- bool flag_TOW_set

  it is true when page 1,2,3 or 4 arrives
- bool flag_utc_model

  Flag indicating that utc model parameters (word 4) have been received.
- bool flag_all_almanac

  Flag indicating that all almanac have been received.
- bool flag_almanac_1

  Flag indicating that almanac 1/2 (word 5) have been received.
- bool flag_almanac_2

  Flag indicating that almanac 2/2 (word 6) have been received.
- int **IOD_ephemeris**
- int **page_type**
- int **FNAV_SV_ID_PRN_1**
- int **FNAV_IODnav_1**
- double **FNAV_t0c_1**
- double **FNAV_af0_1**
- double **FNAV_af1_1**
- double **FNAV_af2_1**
- double **FNAV_SISA_1**
- double **FNAV_ai0_1**
- double **FNAV_ai1_1**
- double **FNAV_ai2_1**

- bool **FNAV_region1_1**
- bool **FNAV_region2_1**
- bool **FNAV_region3_1**
- bool **FNAV_region4_1**
- bool **FNAV_region5_1**
- double **FNAV_BGD_1**
- double **FNAV_E5ahs_1**
- double **FNAV_WN_1**
- double **FNAV_TOW_1**
- bool **FNAV_E5advs_1**
- int **FNAV_IODnav_2**
- double **FNAV_M0_2**
- double **FNAV_omegadot_2**
- double **FNAV_e_2**
- double **FNAV_a12_2**
- double **FNAV_omega0_2**
- double **FNAV_idot_2**
- double **FNAV_WN_2**
- double **FNAV_TOW_2**
- int **FNAV_IODnav_3**
- double **FNAV_i0_3**
- double **FNAV_w_3**
- double **FNAV_deltan_3**
- double **FNAV_Cuc_3**
- double **FNAV_Cus_3**
- double **FNAV_Crc_3**
- double **FNAV_Crs_3**
- double **FNAV_t0e_3**
- double **FNAV_WN_3**
- double **FNAV_TOW_3**
- int **FNAV_IODnav_4**
- double **FNAV_Cic_4**
- double **FNAV_Cis_4**
- double **FNAV_A0_4**
- double **FNAV_A1_4**
- double **FNAV_deltatls_4**
- double **FNAV_t0t_4**
- double **FNAV_WNot_4**
- double **FNAV_WNlsf_4**
- double **FNAV_DN_4**
- double **FNAV_deltatlsf_4**
- double **FNAV_t0g_4**
- double **FNAV_A0g_4**
- double **FNAV_A1g_4**
- double **FNAV_WN0g_4**
- double **FNAV_TOW_4**
- int **FNAV_IODa_5**
- double **FNAV_WNa_5**
- double **FNAV_t0a_5**
- int **FNAV_SVID1_5**
- double **FNAV_Deltaa12_1_5**
- double **FNAV_e_1_5**
- double **FNAV_w_1_5**
- double **FNAV_deltai_1_5**
- double **FNAV_Omega0_1_5**

- double **FNAV_Omegadot_1_5**
- double **FNAV_M0_1_5**
- double **FNAV_af0_1_5**
- double **FNAV_af1_1_5**
- unsigned int **FNAV_E5ahs_1_5**
- int **FNAV_SVID2_5**
- double **FNAV_Deltaa12_2_5**
- double **FNAV_e_2_5**
- double **FNAV_w_2_5**
- double **FNAV_deltai_2_5**
- int **FNAV_IODa_6**
- double **FNAV_Omega0_2_6**
- double **FNAV_Omegadot_2_6**
- double **FNAV_M0_2_6**
- double **FNAV_af0_2_6**
- double **FNAV_af1_2_6**
- double **FNAV_E5ahs_2_6**
- int **FNAV_SVID3_6**
- double **FNAV_Deltaa12_3_6**
- double **FNAV_e_3_6**
- double **FNAV_w_3_6**
- double **FNAV_deltai_3_6**
- double **FNAV_Omega0_3_6**
- double **FNAV_Omegadot_3_6**
- double **FNAV_M0_3_6**
- double **FNAV_af0_3_6**
- double **FNAV_af1_3_6**
- double **FNAV_E5ahs_3_6**

8.50.1 Detailed Description

This class handles the Galileo F/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf.

Definition at line 57 of file galileo_fnav_message.h.

8.50.2 Member Data Documentation

8.50.2.1 bool Galileo_Fnav_Message::flag_all_almanac

Flag indicating that all almanac have been received.

Definition at line 98 of file galileo_fnav_message.h.

8.50.2.2 bool Galileo_Fnav_Message::flag_all_ephemeris

Flag indicating that all words containing ephemeris have been received.

Definition at line 85 of file galileo_fnav_message.h.

8.50.2.3 bool Galileo_Fnav_Message::flag_almanac_1

Flag indicating that almanac 1/2 (word 5) have been received.

Definition at line 99 of file galileo_fnav_message.h.

8.50.2.4 bool Galileo_Fnav_Message::flag_almanac_2

Flag indicating that almanac 2/2 (word 6) have been received.

Definition at line 100 of file galileo_fnav_message.h.

8.50.2.5 bool Galileo_Fnav_Message::flag_ephemeris_1

Flag indicating that ephemeris 1/3 (word 2) have been received.

Definition at line 86 of file galileo_fnav_message.h.

8.50.2.6 bool Galileo_Fnav_Message::flag_ephemeris_2

Flag indicating that ephemeris 2/3 (word 3) have been received.

Definition at line 87 of file galileo_fnav_message.h.

8.50.2.7 bool Galileo_Fnav_Message::flag_ephemeris_3

Flag indicating that ephemeris 3/3 (word 4) have been received.

Definition at line 88 of file galileo_fnav_message.h.

8.50.2.8 bool Galileo_Fnav_Message::flag_iono_and_GST

Flag indicating that ionospheric and GST parameters (word 1) have been received.

Definition at line 90 of file galileo_fnav_message.h.

8.50.2.9 bool Galileo_Fnav_Message::flag_TOW_set

it is true when page 1,2,3 or 4 arrives

Definition at line 95 of file galileo_fnav_message.h.

8.50.2.10 bool Galileo_Fnav_Message::flag_utc_model

Flag indicating that utc model parameters (word 4) have been received.

Definition at line 96 of file galileo_fnav_message.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/galileo_fnav_message.h

8.51 Galileo_Iono Class Reference

This class is a storage for the GALILEO IONOSPHERIC data as described in Galileo ICD paragraph 5.1.6.

`#include <galileo_iono.h>`

Public Member Functions

- Galileo_Iono ()

Public Attributes

- double ai0_5
  Effective Ionisation Level 1st order parameter [sfu].
- double ai1_5
  Effective Ionisation Level 2st order parameter [sfu/degree].
- double ai2_5
  Effective Ionisation Level 3st order parameter [sfu/degree].
- bool Region1_flag_5
  Ionospheric Disturbance Flag for region 1.
- bool Region2_flag_5
  Ionospheric Disturbance Flag for region 2.
- bool Region3_flag_5
  Ionospheric Disturbance Flag for region 3.
- bool Region4_flag_5
  Ionospheric Disturbance Flag for region 4.
- bool Region5_flag_5
  Ionospheric Disturbance Flag for region 5.
- double TOW_5
  UTC data reference Time of Week [s].
- double WN_5
  UTC data reference Week number [week].

8.51.1 Detailed Description

This class is a storage for the GALILEO IONOSPHERIC data as described in Galileo ICD paragraph 5.1.6.

See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS↩_ICD.pdf

Definition at line 41 of file galileo_iono.h.

8.51.2 Constructor & Destructor Documentation

8.51.2.1 Galileo_Iono::Galileo_Iono ( )

Default constructor

8.51.3 Member Data Documentation

8.51.3.1 double Galileo_Iono::ai0_5

Effective Ionisation Level 1st order parameter [sfu].

Definition at line 45 of file galileo_iono.h.

8.51.3.2 double Galileo_Iono::ai1_5

Effective Ionisation Level 2st order parameter [sfu/degree].

Definition at line 46 of file galileo_iono.h.

8.51.3.3 double Galileo_Iono::ai2_5

Effective Ionisation Level 3st order parameter [sfu/degree].

Definition at line 47 of file galileo_iono.h.

8.51.3.4 bool Galileo_Iono::Region1_flag_5

Ionospheric Disturbance Flag for region 1.

Definition at line 50 of file galileo_iono.h.

8.51.3.5 bool Galileo_Iono::Region2_flag_5

Ionospheric Disturbance Flag for region 2.

Definition at line 51 of file galileo_iono.h.

8.51.3.6 bool Galileo_Iono::Region3_flag_5

Ionospheric Disturbance Flag for region 3.

Definition at line 52 of file galileo_iono.h.

8.51.3.7 bool Galileo_Iono::Region4_flag_5

Ionospheric Disturbance Flag for region 4.

Definition at line 53 of file galileo_iono.h.

8.51.3.8 bool Galileo_Iono::Region5_flag_5

Ionospheric Disturbance Flag for region 5.

Definition at line 54 of file galileo_iono.h.

8.51.3.9 double Galileo_Iono::TOW_5

UTC data reference Time of Week [s].

Definition at line 57 of file galileo_iono.h.

8.51.3.10 double Galileo_Iono::WN_5

UTC data reference Week number [week].

Definition at line 58 of file galileo_iono.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/galileo_iono.h

8.52 Galileo_Navigation_Message Class Reference

This class handles the Galileo I/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf.

`#include <galileo_navigation_message.h>`

Public Member Functions

- void **split_page** (std::string page_string, int flag_even_word)
- int **page_jk_decoder** (const char *data_jk)
- void **reset** ()
- bool **have_new_ephemeris** ()
- bool **have_new_iono_and_GST** ()
- bool **have_new_utc_model** ()
- bool **have_new_almanac** ()
- Galileo_Ephemeris **get_ephemeris** ()
- Galileo_Iono **get_iono** ()
- Galileo_Utc_Model **get_utc_model** ()
- Galileo_Almanac **get_almanac** ()

Public Attributes

- int **Page_type_time_stamp**
- int **flag_even_word**
- std::string **page_Even**
- bool **flag_CRC_test**
- bool flag_all_ephemeris

  Flag indicating that all words containing ephemeris have been received.
- bool flag_ephemeris_1

  Flag indicating that ephemeris 1/4 (word 1) have been received.
- bool flag_ephemeris_2

  Flag indicating that ephemeris 2/4 (word 2) have been received.
- bool flag_ephemeris_3

  Flag indicating that ephemeris 3/4 (word 3) have been received.
- bool flag_ephemeris_4

  Flag indicating that ephemeris 4/4 (word 4) have been received.
- bool flag_iono_and_GST

  Flag indicating that ionospheric and GST parameters (word 5) have been received.
- bool **flag_TOW_5**
- bool **flag_TOW_6**
- bool flag_TOW_set

  it is true when page 5 or page 6 arrives
- bool flag_utc_model

  Flag indicating that utc model parameters (word 6) have been received.
- bool flag_all_almanac

  Flag indicating that all almanac have been received.
- bool flag_almanac_1

  Flag indicating that almanac 1/4 (word 7) have been received.
- bool flag_almanac_2

  Flag indicating that almanac 2/4 (word 8) have been received.
- bool flag_almanac_3

  Flag indicating that almanac 3/4 (word 9) have been received.
- bool flag_almanac_4

  Flag indicating that almanac 4/4 (word 10) have been received.
- int **IOD_ephemeris**
- bool **flag_GGTO**
- bool **flag_GGTO_1**
- bool **flag_GGTO_2**
- bool **flag_GGTO_3**
- bool **flag_GGTO_4**
- int IOD_nav_1

  IOD_nav page 1.
- double t0e_1

  Ephemeris reference time [s].
- double M0_1

  Mean anomaly at reference time [semi-circles].
- double e_1

  Eccentricity.
- double A_1

  Square root of the semi-major axis [metres^ 1/2].
- int IOD_nav_2

  IOD_nav page 2.

- double OMEGA_0_2

  Longitude of ascending node of orbital plane at weekly epoch [semi-circles].
- double i_0_2

  Inclination angle at reference time [semi-circles].
- double omega_2

  Argument of perigee [semi-circles].
- double iDot_2

  Rate of inclination angle [semi-circles/sec].
- int **IOD_nav_3**
- double OMEGA_dot_3

  Rate of right ascension [semi-circles/sec].
- double delta_n_3

  Mean motion difference from computed value [semi-circles/sec].
- double C_uc_3

  Amplitude of the cosine harmonic correction term to the argument of latitude [radians].
- double C_us_3

  Amplitude of the sine harmonic correction term to the argument of latitude [radians].
- double C_rc_3

  Amplitude of the cosine harmonic correction term to the orbit radius [meters].
- double C_rs_3

  Amplitude of the sine harmonic correction term to the orbit radius [meters].
- double **SISA_3**
- int **IOD_nav_4**
- int **SV_ID_PRN_4**
- double C_ic_4

  Amplitude of the cosine harmonic correction term to the angle of inclination [radians].
- double C_is_4

  Amplitude of the sine harmonic correction term to the angle of inclination [radians].
- double t0c_4

  Clock correction data reference Time of Week [sec].
- double af0_4

  SV clock bias correction coefficient [s].
- double af1_4

  SV clock drift correction coefficient [s/s].
- double af2_4

  clock drift rate correction coefficient [s/s^ 2]
- double **spare_4**
- double ai0_5

  Effective Ionisation Level 1st order parameter [sfu].
- double ai1_5

  Effective Ionisation Level 2st order parameter [sfu/degree].
- double ai2_5

  Effective Ionisation Level 3st order parameter [sfu/degree].
- bool Region1_flag_5

  Ionospheric Disturbance Flag for region 1.
- bool Region2_flag_5

  Ionospheric Disturbance Flag for region 2.
- bool Region3_flag_5

  Ionospheric Disturbance Flag for region 3.
- bool Region4_flag_5

  Ionospheric Disturbance Flag for region 4.

- bool Region5_flag_5

  Ionospheric Disturbance Flag for region 5.
- double BGD_E1E5a_5

  E1-E5a Broadcast Group Delay [s].
- double BGD_E1E5b_5

  E1-E5b Broadcast Group Delay [s].
- double E5b_HS_5

  E5b Signal Health Status.
- double E1B_HS_5

  E1B Signal Health Status.
- double E5b_DVS_5

  E5b Data Validity Status.
- double E1B_DVS_5

  E1B Data Validity Status.
- double **WN_5**
- double **TOW_5**
- double **spare_5**
- double **A0_6**
- double **A1_6**
- double **Delta_tLS_6**
- double **t0t_6**
- double **WNot_6**
- double **WN_LSF_6**
- double **DN_6**
- double **Delta_tLSF_6**
- double **TOW_6**
- int **IOD_a_7**
- double **WN_a_7**
- double **t0a_7**
- int **SVID1_7**
- double **DELTA_A_7**
- double **e_7**
- double **omega_7**
- double **delta_i_7**
- double **Omega0_7**
- double **Omega_dot_7**
- double **M0_7**
- int **IOD_a_8**
- double **af0_8**
- double **af1_8**
- double **E5b_HS_8**
- double **E1B_HS_8**
- int **SVID2_8**
- double **DELTA_A_8**
- double **e_8**
- double **omega_8**
- double **delta_i_8**
- double **Omega0_8**
- double **Omega_dot_8**
- int **IOD_a_9**
- double **WN_a_9**
- double **t0a_9**
- double **M0_9**

- double **af0_9**
- double **af1_9**
- double **E5b_HS_9**
- double **E1B_HS_9**
- int **SVID3_9**
- double **DELTA_A_9**
- double **e_9**
- double **omega_9**
- double **delta_i_9**
- int **IOD_a_10**
- double **Omega0_10**
- double **Omega_dot_10**
- double **M0_10**
- double **af0_10**
- double **af1_10**
- double **E5b_HS_10**
- double **E1B_HS_10**
- double A_0G_10

  Constant term of the offset Delta t systems.
- double A_1G_10

  Rate of change of the offset Delta t systems.
- double t_0G_10

  Reference time for Galileo/GPS Time Offset (GGTO) data.
- double WN_0G_10

  Week Number of Galileo/GPS Time Offset (GGTO) reference.
- double **Time_0**
- double **WN_0**
- double **TOW_0**
- double **Galileo_satClkDrift**
- double Galileo_dtr

  Relativistic clock correction term.
- double galileo_satpos_X

  Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.
- double galileo_satpos_Y

  Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.
- double galileo_satpos_Z

  Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).
- double galileo_satvel_X

  Earth-fixed velocity coordinate x of the satellite [m].
- double galileo_satvel_Y

  Earth-fixed velocity coordinate y of the satellite [m].
- double galileo_satvel_Z

  Earth-fixed velocity coordinate z of the satellite [m].

8.52.1 Detailed Description

This class handles the Galileo I/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf.

Definition at line 53 of file galileo_navigation_message.h.

8.52.2 Member Data Documentation

8.52.2.1 double Galileo_Navigation_Message::A_0G_10

Constant term of the offset Delta t systems.

Definition at line 216 of file galileo_navigation_message.h.

8.52.2.2 double Galileo_Navigation_Message::A_1

Square root of the semi-major axis [metres^1/2].

Definition at line 98 of file galileo_navigation_message.h.

8.52.2.3 double Galileo_Navigation_Message::A_1G_10

Rate of change of the offset Delta t systems.

Definition at line 217 of file galileo_navigation_message.h.

8.52.2.4 double Galileo_Navigation_Message::af0_4

SV clock bias correction coefficient [s].

Definition at line 124 of file galileo_navigation_message.h.

8.52.2.5 double Galileo_Navigation_Message::af1_4

SV clock drift correction coefficient [s/s].

Definition at line 125 of file galileo_navigation_message.h.

8.52.2.6 double Galileo_Navigation_Message::af2_4

clock drift rate correction coefficient [s/s^2]

Definition at line 126 of file galileo_navigation_message.h.

8.52.2.7 double Galileo_Navigation_Message::ai0_5

Effective Ionisation Level 1st order parameter [sfu].

Definition at line 131 of file galileo_navigation_message.h.

8.52.2.8 double Galileo_Navigation_Message::ai1_5

Effective Ionisation Level 2st order parameter [sfu/degree].

Definition at line 132 of file galileo_navigation_message.h.

8.52.2.9 double Galileo_Navigation_Message::ai2_5

Effective Ionisation Level 3st order parameter [sfu/degree].

Definition at line 133 of file galileo_navigation_message.h.

8.52.2.10 double Galileo_Navigation_Message::BGD_E1E5a_5

E1-E5a Broadcast Group Delay [s].

Definition at line 141 of file galileo_navigation_message.h.

8.52.2.11 double Galileo_Navigation_Message::BGD_E1E5b_5

E1-E5b Broadcast Group Delay [s].

Definition at line 142 of file galileo_navigation_message.h.

8.52.2.12 double Galileo_Navigation_Message::C_ic_4

Amplitude of the cosine harmonic correction term to the angle of inclination [radians].

Definition at line 120 of file galileo_navigation_message.h.

8.52.2.13 double Galileo_Navigation_Message::C_is_4

Amplitude of the sine harmonic correction term to the angle of inclination [radians].

Definition at line 121 of file galileo_navigation_message.h.

8.52.2.14 double Galileo_Navigation_Message::C_rc_3

Amplitude of the cosine harmonic correction term to the orbit radius [meters].

Definition at line 113 of file galileo_navigation_message.h.

8.52.2.15 double Galileo_Navigation_Message::C_rs_3

Amplitude of the sine harmonic correction term to the orbit radius [meters].

Definition at line 114 of file galileo_navigation_message.h.

8.52.2.16 double Galileo_Navigation_Message::C_uc_3

Amplitude of the cosine harmonic correction term to the argument of latitude [radians].

Definition at line 111 of file galileo_navigation_message.h.

8.52.2.17 double Galileo_Navigation_Message::C_us_3

Amplitude of the sine harmonic correction term to the argument of latitude [radians].

Definition at line 112 of file galileo_navigation_message.h.

8.52.2.18 double Galileo_Navigation_Message::delta_n_3

Mean motion difference from computed value [semi-circles/sec].

Definition at line 110 of file galileo_navigation_message.h.

8.52.2.19 double Galileo_Navigation_Message::E1B_DVS_5

E1B Data Validity Status.

Definition at line 147 of file galileo_navigation_message.h.

8.52.2.20 double Galileo_Navigation_Message::E1B_HS_5

E1B Signal Health Status.

Definition at line 145 of file galileo_navigation_message.h.

8.52.2.21 double Galileo_Navigation_Message::E5b_DVS_5

E5b Data Validity Status.

Definition at line 146 of file galileo_navigation_message.h.

8.52.2.22 double Galileo_Navigation_Message::E5b_HS_5

E5b Signal Health Status.

Definition at line 144 of file galileo_navigation_message.h.

8.52.2.23 double Galileo_Navigation_Message::e_1

Eccentricity.

Definition at line 97 of file galileo_navigation_message.h.

8.52.2.24 bool Galileo_Navigation_Message::flag_all_almanac

Flag indicating that all almanac have been received.

Definition at line 79 of file galileo_navigation_message.h.

8.52.2.25 bool Galileo_Navigation_Message::flag_all_ephemeris

Flag indicating that all words containing ephemeris have been received.

Definition at line 67 of file galileo_navigation_message.h.

8.52.2.26 bool Galileo_Navigation_Message::flag_almanac_1

Flag indicating that almanac 1/4 (word 7) have been received.

Definition at line 80 of file galileo_navigation_message.h.

8.52.2.27 bool Galileo_Navigation_Message::flag_almanac_2

Flag indicating that almanac 2/4 (word 8) have been received.

Definition at line 81 of file galileo_navigation_message.h.

8.52.2.28 bool Galileo_Navigation_Message::flag_almanac_3

Flag indicating that almanac 3/4 (word 9) have been received.

Definition at line 82 of file galileo_navigation_message.h.

8.52.2.29 bool Galileo_Navigation_Message::flag_almanac_4

Flag indicating that almanac 4/4 (word 10) have been received.

Definition at line 83 of file galileo_navigation_message.h.

8.52.2.30 bool Galileo_Navigation_Message::flag_ephemeris_1

Flag indicating that ephemeris 1/4 (word 1) have been received.

Definition at line 68 of file galileo_navigation_message.h.

8.52.2.31 bool Galileo_Navigation_Message::flag_ephemeris_2

Flag indicating that ephemeris 2/4 (word 2) have been received.

Definition at line 69 of file galileo_navigation_message.h.

8.52.2.32 bool Galileo_Navigation_Message::flag_ephemeris_3

Flag indicating that ephemeris 3/4 (word 3) have been received.

Definition at line 70 of file galileo_navigation_message.h.

8.52.2.33 bool Galileo_Navigation_Message::flag_ephemeris_4

Flag indicating that ephemeris 4/4 (word 4) have been received.

Definition at line 71 of file galileo_navigation_message.h.

8.52.2.34 bool Galileo_Navigation_Message::flag_iono_and_GST

Flag indicating that ionospheric and GST parameters (word 5) have been received.

Definition at line 73 of file galileo_navigation_message.h.

8.52.2.35 bool Galileo_Navigation_Message::flag_TOW_set

it is true when page 5 or page 6 arrives

Definition at line 76 of file galileo_navigation_message.h.

8.52.2.36 bool Galileo_Navigation_Message::flag_utc_model

Flag indicating that utc model parameters (word 6) have been received.

Definition at line 77 of file galileo_navigation_message.h.

8.52.2.37 double Galileo_Navigation_Message::Galileo_dtr

Relativistic clock correction term.

Definition at line 226 of file galileo_navigation_message.h.

8.52.2.38 double Galileo_Navigation_Message::galileo_satpos_X

Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

Definition at line 229 of file galileo_navigation_message.h.

8.52.2.39 double Galileo_Navigation_Message::galileo_satpos_Y

Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

Definition at line 230 of file galileo_navigation_message.h.

8.52.2.40 double Galileo_Navigation_Message::galileo_satpos_Z

Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

Definition at line 231 of file galileo_navigation_message.h.

8.52.2.41 double Galileo_Navigation_Message::galileo_satvel_X

Earth-fixed velocity coordinate x of the satellite [m].

Definition at line 233 of file galileo_navigation_message.h.

8.52.2.42 double Galileo_Navigation_Message::galileo_satvel_Y

Earth-fixed velocity coordinate y of the satellite [m].

Definition at line 234 of file galileo_navigation_message.h.

8.52.2.43 double Galileo_Navigation_Message::galileo_satvel_Z

Earth-fixed velocity coordinate z of the satellite [m].

Definition at line 235 of file galileo_navigation_message.h.

8.52.2.44 double Galileo_Navigation_Message::i_0_2

Inclination angle at reference time [semi-circles].

Definition at line 103 of file galileo_navigation_message.h.

8.52.2.45 double Galileo_Navigation_Message::iDot_2

Rate of inclination angle [semi-circles/sec].

Definition at line 105 of file galileo_navigation_message.h.

8.52.2.46 int Galileo_Navigation_Message::IOD_nav_1

IOD_nav page 1.

Definition at line 94 of file galileo_navigation_message.h.

8.52.2.47 int Galileo_Navigation_Message::IOD_nav_2

IOD_nav page 2.

Definition at line 101 of file galileo_navigation_message.h.

8.52.2.48 double Galileo_Navigation_Message::M0_1

Mean anomaly at reference time [semi-circles].

Definition at line 96 of file galileo_navigation_message.h.

8.52.2.49 double Galileo_Navigation_Message::OMEGA_0_2

Longitude of ascending node of orbital plane at weekly epoch [semi-circles].

Definition at line 102 of file galileo_navigation_message.h.

8.52.2.50 double Galileo_Navigation_Message::omega_2

Argument of perigee [semi-circles].

Definition at line 104 of file galileo_navigation_message.h.

8.52.2.51 double Galileo_Navigation_Message::OMEGA_dot_3

Rate of right ascension [semi-circles/sec].

Definition at line 109 of file galileo_navigation_message.h.

8.52.2.52 bool Galileo_Navigation_Message::Region1_flag_5

Ionospheric Disturbance Flag for region 1.

Definition at line 136 of file galileo_navigation_message.h.

8.52.2.53 bool Galileo_Navigation_Message::Region2_flag_5

Ionospheric Disturbance Flag for region 2.

Definition at line 137 of file galileo_navigation_message.h.

8.52.2.54 bool Galileo_Navigation_Message::Region3_flag_5

Ionospheric Disturbance Flag for region 3.

Definition at line 138 of file galileo_navigation_message.h.

8.52.2.55 bool Galileo_Navigation_Message::Region4_flag_5

Ionospheric Disturbance Flag for region 4.

Definition at line 139 of file galileo_navigation_message.h.

8.52.2.56 bool Galileo_Navigation_Message::Region5_flag_5

Ionospheric Disturbance Flag for region 5.

Definition at line 140 of file galileo_navigation_message.h.

8.52.2.57 double Galileo_Navigation_Message::t0c_4

Clock correction data reference Time of Week [sec].

Definition at line 123 of file galileo_navigation_message.h.

8.52.2.58 double Galileo_Navigation_Message::t0e_1

Ephemeris reference time [s].

Definition at line 95 of file galileo_navigation_message.h.

8.52.2.59 double Galileo_Navigation_Message::t_0G_10

Reference time for Galileo/GPS Time Offset (GGTO) data.

Definition at line 218 of file galileo_navigation_message.h.

8.52.2.60 double Galileo_Navigation_Message::WN_0G_10

Week Number of Galileo/GPS Time Offset (GGTO) reference.

Definition at line 219 of file galileo_navigation_message.h.

The documentation for this class was generated from the following file:

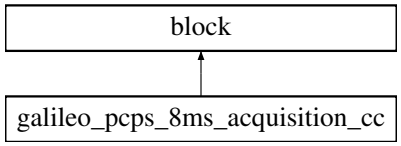
- src/core/system_parameters/galileo_navigation_message.h

8.53 galileo_pcps_8ms_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition for Galileo E1 signals with coherent integration time = 8 ms (two codes)

`#include <galileo_pcps_8ms_acquisition_cc.h>`

Inheritance diagram for galileo_pcps_8ms_acquisition_cc:



Public Member Functions

- ~galileo_pcps_8ms_acquisition_cc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)

  Sets local code for PCPS acquisition algorithm.
- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.

Friends

- galileo_pcps_8ms_acquisition_cc_sptr **galileo_pcps_8ms_make_acquisition_cc** (unsigned int sampled↩_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool dump, std::string dump_filename)

8.53.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition for Galileo E1 signals with coherent integration time = 8 ms (two codes)

Definition at line 56 of file galileo_pcps_8ms_acquisition_cc.h.

8.53.2 Constructor & Destructor Documentation

8.53.2.1 galileo_pcps_8ms_acquisition_cc::~galileo_pcps_8ms_acquisition_cc ( )

Default destructor.

8.53.3 Member Function Documentation

8.53.3.1 int galileo_pcps_8ms_acquisition_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.53.3.2 void galileo_pcps_8ms_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.53.3.3 unsigned int galileo_pcps_8ms_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 127 of file galileo_pcps_8ms_acquisition_cc.h.

References init(), and set_local_code().

8.53.3.4 void galileo_pcps_8ms_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 148 of file galileo_pcps_8ms_acquisition_cc.h.

References set_state().

8.53.3.5 void galileo_pcps_8ms_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 164 of file galileo_pcps_8ms_acquisition_cc.h.

8.53.3.6 void galileo_pcps_8ms_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 183 of file galileo_pcps_8ms_acquisition_cc.h.

8.53.3.7 void galileo_pcps_8ms_acquisition_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 192 of file galileo_pcps_8ms_acquisition_cc.h.

References general_work().

8.53.3.8 void galileo_pcps_8ms_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 119 of file galileo_pcps_8ms_acquisition_cc.h.

8.53.3.9 void galileo_pcps_8ms_acquisition_cc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.53.3.10 void galileo_pcps_8ms_acquisition_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.53.3.11 void galileo_pcps_8ms_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 174 of file galileo_pcps_8ms_acquisition_cc.h.

The documentation for this class was generated from the following file:

- src/algorithms/acquisition/gnuradio_blocks/galileo_pcps_8ms_acquisition_cc.h

8.54 Galileo_Utc_Model Class Reference

This class is a storage for the GALILEO UTC MODEL data as described in Galileo ICD https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf paragraph 5.1.7.

`#include <galileo_utc_model.h>`

Public Member Functions

- double GST_to_UTC_time (double t_e, int WN)

  GST-UTC Conversion Algorithm and Parameters.
- Galileo_Utc_Model ()

Public Attributes

- double **A0_6**
- double **A1_6**
- double **Delta_tLS_6**
- double t0t_6

  UTC data reference Time of Week [s].
- double WNot_6

  UTC data reference Week number [week].
- double **WN_LSF_6**
- double **DN_6**
- double **Delta_tLSF_6**
- bool **flag_utc_model**

8.54.1 Detailed Description

This class is a storage for the GALILEO UTC MODEL data as described in Galileo ICD https://www.↩gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf paragraph 5.1.7.

Definition at line 42 of file galileo_utc_model.h.

8.54.2 Constructor & Destructor Documentation

8.54.2.1 Galileo_Utc_Model::Galileo_Utc_Model ( )

Default constructor

8.54.3 Member Function Documentation

8.54.3.1 double Galileo_Utc_Model::GST_to_UTC_time (double *t_e*, int *WN*)

GST-UTC Conversion Algorithm and Parameters.

8.54.4 Member Data Documentation

8.54.4.1 double Galileo_Utc_Model::t0t_6

UTC data reference Time of Week [s].

Definition at line 49 of file galileo_utc_model.h.

8.54.4.2 double Galileo_Utc_Model::WNot_6

UTC data reference Week number [week].

Definition at line 50 of file galileo_utc_model.h.

The documentation for this class was generated from the following file:

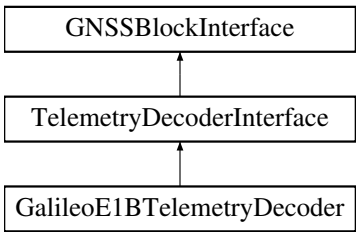
- src/core/system_parameters/galileo_utc_model.h

8.55 GalileoE1BTelemetryDecoder Class Reference

This class implements a NAV data decoder for Galileo INAV frames in E1B radio link.

`#include <galileo_e1b_telemetry_decoder.h>`

Inheritance diagram for GalileoE1BTelemetryDecoder:



Public Member Functions

- **GalileoE1BTelemetryDecoder** (ConfigurationInterface *configuration, std::string role, unsigned int in_↩ streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1B_Telemetry_Decoder".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **set_satellite** (Gnss_Satellite satellite)
- void **set_channel** (int channel)
- void **reset** ()
- size_t **item_size** ()

8.55.1 Detailed Description

This class implements a NAV data decoder for Galileo INAV frames in E1B radio link.

Definition at line 49 of file galileo_e1b_telemetry_decoder.h.

8.55.2 Member Function Documentation

8.55.2.1 std::string GalileoE1BTelemetryDecoder::implementation ( ) [inline], [virtual]

Returns "Galileo_E1B_Telemetry_Decoder".

Implements GNSSBlockInterface.

Definition at line 66 of file galileo_e1b_telemetry_decoder.h.

The documentation for this class was generated from the following file:

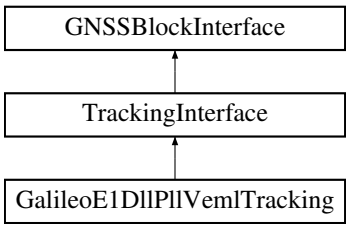
- src/algorithms/telemetry_decoder/adapters/galileo_e1b_telemetry_decoder.h

8.56 GalileoE1DllPllVemlTracking Class Reference

This class Adapts a DLL+PLL VEML (Very Early Minus Late) tracking loop block to a TrackingInterface for Galileo E1 signals.

`#include <galileo_e1_dll_pll_veml_tracking.h>`

Inheritance diagram for GalileoE1DllPllVemlTracking:



Public Member Functions

- **GalileoE1DllPllVemlTracking** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1_DLL_PLL_VEML_Tracking".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.56.1 Detailed Description

This class Adapts a DLL+PLL VEML (Very Early Minus Late) tracking loop block to a TrackingInterface for Galileo E1 signals.

Definition at line 52 of file galileo_e1_dll_pll_veml_tracking.h.

8.56.2 Member Function Documentation

8.56.2.1 std::string GalileoE1DllPllVemlTracking::implementation ( ) [inline], [virtual]

Returns "Galileo_E1_DLL_PLL_VEML_Tracking".

Implements GNSSBlockInterface.

Definition at line 68 of file galileo_e1_dll_pll_veml_tracking.h.

References set_channel(), and set_gnss_synchro().

8.56.2.2 void GalileoE1DllPllVemlTracking::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.56.2.3 void GalileoE1DllPllVemlTracking::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

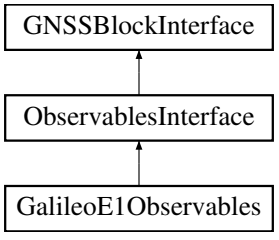
- src/algorithms/tracking/adapters/galileo_e1_dll_pll_veml_tracking.h

8.57 GalileoE1Observables Class Reference

This class implements an ObservablesInterface for Galileo E1B.

`#include <galileo_e1_observables.h>`

Inheritance diagram for GalileoE1Observables:



Public Member Functions

- **GalileoE1Observables** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1B_Observables".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation.

8.57.1 Detailed Description

This class implements an ObservablesInterface for Galileo E1B.

Definition at line 47 of file galileo_e1_observables.h.

8.57.2 Member Function Documentation

8.57.2.1 std::string GalileoE1Observables::implementation ( ) [inline], [virtual]

Returns "Galileo_E1B_Observables".

Implements GNSSBlockInterface.

Definition at line 61 of file galileo_e1_observables.h.

8.57.2.2 size_t GalileoE1Observables::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation.

Implements GNSSBlockInterface.

Definition at line 75 of file galileo_e1_observables.h.

The documentation for this class was generated from the following file:

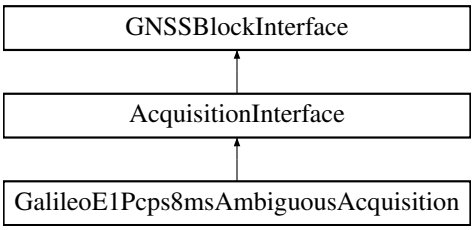
- src/algorithms/observables/adapters/galileo_e1_observables.h

8.58 GalileoE1Pcps8msAmbiguousAcquisition Class Reference

Adapts a PCPS 8ms acquisition block to an AcquisitionInterface for Galileo E1 Signals.

`#include <galileo_e1_pcps_8ms_ambiguous_acquisition.h>`

Inheritance diagram for GalileoE1Pcps8msAmbiguousAcquisition:



Public Member Functions

- **GalileoE1Pcps8msAmbiguousAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1_PCPS_8ms_Ambiguous_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for Galileo E1 PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.

8.58.1 Detailed Description

Adapts a PCPS 8ms acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Definition at line 48 of file galileo_e1_pcps_8ms_ambiguous_acquisition.h.

8.58.2 Member Function Documentation

8.58.2.1 std::string GalileoE1Pcps8msAmbiguousAcquisition::implementation ( ) [inline], [virtual]

Returns "Galileo_E1_PCPS_8ms_Ambiguous_Acquisition".

Implements GNSSBlockInterface.

Definition at line 65 of file galileo_e1_pcps_8ms_ambiguous_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_local_code(), and set_threshold().

8.58.2.2 void GalileoE1Pcps8msAmbiguousAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.3 signed int GalileoE1Pcps8msAmbiguousAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.4 void GalileoE1Pcps8msAmbiguousAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.5 void GalileoE1Pcps8msAmbiguousAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.6 void GalileoE1Pcps8msAmbiguousAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.7 void GalileoE1Pcps8msAmbiguousAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.8 void GalileoE1Pcps8msAmbiguousAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.9 void GalileoE1Pcps8msAmbiguousAcquisition::set_local_code ( ) [virtual]

Sets local code for Galileo E1 PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.58.2.10 void GalileoE1Pcps8msAmbiguousAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

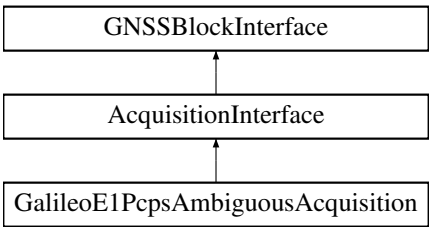
- src/algorithms/acquisition/adapters/galileo_e1_pcps_8ms_ambiguous_acquisition.h

8.59 GalileoE1PcpsAmbiguousAcquisition Class Reference

This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

`#include <galileo_e1_pcps_ambiguous_acquisition.h>`

Inheritance diagram for GalileoE1PcpsAmbiguousAcquisition:



Public Member Functions

- **GalileoE1PcpsAmbiguousAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1_PCPS_Ambiguous_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for Galileo E1 PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.59.1 Detailed Description

This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Definition at line 52 of file galileo_e1_pcps_ambiguous_acquisition.h.

8.59.2 Member Function Documentation

8.59.2.1 std::string GalileoE1PcpsAmbiguousAcquisition::implementation ( ) [inline], [virtual]

Returns "Galileo_E1_PCPS_Ambiguous_Acquisition".

Implements GNSSBlockInterface.

Definition at line 69 of file galileo_e1_pcps_ambiguous_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_local_code(), set_state(), and set_threshold().

8.59.2.2 void GalileoE1PcpsAmbiguousAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.3 signed int GalileoE1PcpsAmbiguousAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.4 void GalileoE1PcpsAmbiguousAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.5 void GalileoE1PcpsAmbiguousAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.6 void GalileoE1PcpsAmbiguousAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.7 void GalileoE1PcpsAmbiguousAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.8 void GalileoE1PcpsAmbiguousAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.9 void GalileoE1PcpsAmbiguousAcquisition::set_local_code ( ) [virtual]

Sets local code for Galileo E1 PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.59.2.10 void GalileoE1PcpsAmbiguousAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.59.2.11 void GalileoE1PcpsAmbiguousAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

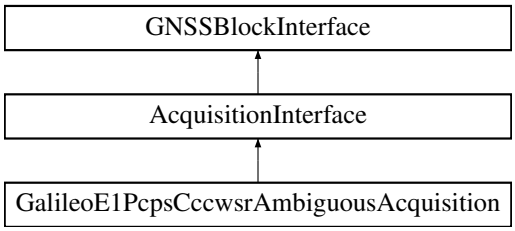
- src/algorithms/acquisition/adapters/galileo_e1_pcps_ambiguous_acquisition.h

8.60 GalileoE1PcpsCccwsrAmbiguousAcquisition Class Reference

Adapts a PCPS CCCWSR acquisition block to an AcquisitionInterface for Galileo E1 Signals.

`#include <galileo_e1_pcps_cccwsr_ambiguous_acquisition.h>`

Inheritance diagram for GalileoE1PcpsCccwsrAmbiguousAcquisition:



Public Member Functions

- **GalileoE1PcpsCccwsrAmbiguousAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1_PCPS_CCCWSR_Ambiguous_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of CCCWSR algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void **set_local_code** ()
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.60.1 Detailed Description

Adapts a PCPS CCCWSR acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Definition at line 48 of file galileo_e1_pcps_cccwsr_ambiguous_acquisition.h.

8.60.2 Member Function Documentation

8.60.2.1 std::string GalileoE1PcpsCccwsrAmbiguousAcquisition::implementation ( ) [inline], [virtual]

Returns "Galileo_E1_PCPS_CCCWSR_Ambiguous_Acquisition".

Implements GNSSBlockInterface.

Definition at line 65 of file galileo_e1_pcps_cccwsr_ambiguous_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_state(), and set_threshold().

8.60.2.2 void GalileoE1PcpsCccwsrAmbiguousAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.60.2.3 signed int GalileoE1PcpsCccwsrAmbiguousAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.60.2.4 void GalileoE1PcpsCccwsrAmbiguousAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.60.2.5 void GalileoE1PcpsCccwsrAmbiguousAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.60.2.6 void GalileoE1PcpsCccwsrAmbiguousAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.60.2.7 void GalileoE1PcpsCccwsrAmbiguousAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.60.2.8 void GalileoE1PcpsCccwsrAmbiguousAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.60.2.9 void GalileoE1PcpsCccwsrAmbiguousAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.60.2.10 void GalileoE1PcpsCccwsrAmbiguousAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of CCCWSR algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

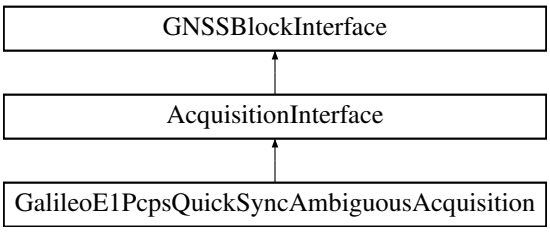
- src/algorithms/acquisition/adapters/galileo_e1_pcps_cccwsr_ambiguous_acquisition.h

8.61 GalileoE1PcpsQuickSyncAmbiguousAcquisition Class Reference

This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

`#include <galileo_e1_pcps_quicksync_ambiguous_acquisition.h>`

Inheritance diagram for GalileoE1PcpsQuickSyncAmbiguousAcquisition:



Public Member Functions

- **GalileoE1PcpsQuickSyncAmbiguousAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1_PCPS_Ambiguous_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for Galileo E1 PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.61.1 Detailed Description

This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Definition at line 48 of file galileo_e1_pcps_quicksync_ambiguous_acquisition.h.

8.61.2 Member Function Documentation

8.61.2.1 std::string GalileoE1PcpsQuickSyncAmbiguousAcquisition::implementation ( ) [inline], [virtual]

Returns "Galileo_E1_PCPS_Ambiguous_Acquisition".

Implements GNSSBlockInterface.

Definition at line 65 of file galileo_e1_pcps_quicksync_ambiguous_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_local_code(), set_state(), and set_threshold().

8.61.2.2 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.3 signed int GalileoE1PcpsQuickSyncAmbiguousAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.4 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.5 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.6 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.7 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.8 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.9 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::set_local_code ( ) [virtual]

Sets local code for Galileo E1 PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.61.2.10 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.61.2.11 void GalileoE1PcpsQuickSyncAmbiguousAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

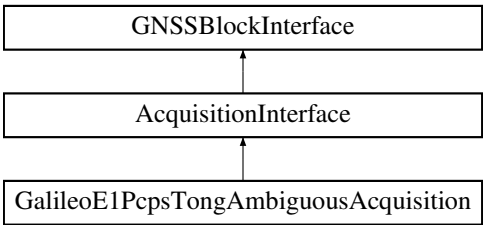
- src/algorithms/acquisition/adapters/galileo_e1_pcps_quicksync_ambiguous_acquisition.h

8.62 GalileoE1PcpsTongAmbiguousAcquisition Class Reference

Adapts a PCPS Tong acquisition block to an AcquisitionInterface for Galileo E1 Signals.

`#include <galileo_e1_pcps_tong_ambiguous_acquisition.h>`

Inheritance diagram for GalileoE1PcpsTongAmbiguousAcquisition:



Public Member Functions

- **GalileoE1PcpsTongAmbiguousAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1_PCPS_Tong_Ambiguous_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of TONG algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for Galileo E1 TONG acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.62.1 Detailed Description

Adapts a PCPS Tong acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Definition at line 48 of file galileo_e1_pcps_tong_ambiguous_acquisition.h.

8.62.2 Member Function Documentation

8.62.2.1 std::string GalileoE1PcpsTongAmbiguousAcquisition::implementation ( ) [inline], [virtual]

Returns "Galileo_E1_PCPS_Tong_Ambiguous_Acquisition".

Implements GNSSBlockInterface.

Definition at line 65 of file galileo_e1_pcps_tong_ambiguous_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_local_code(), set_state(), and set_threshold().

8.62.2.2 void GalileoE1PcpsTongAmbiguousAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.3 signed int GalileoE1PcpsTongAmbiguousAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.4 void GalileoE1PcpsTongAmbiguousAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.5 void GalileoE1PcpsTongAmbiguousAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.6 void GalileoE1PcpsTongAmbiguousAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.7 void GalileoE1PcpsTongAmbiguousAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.8 void GalileoE1PcpsTongAmbiguousAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.9 void GalileoE1PcpsTongAmbiguousAcquisition::set_local_code ( ) [virtual]

Sets local code for Galileo E1 TONG acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.62.2.10 void GalileoE1PcpsTongAmbiguousAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.62.2.11 void GalileoE1PcpsTongAmbiguousAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of TONG algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

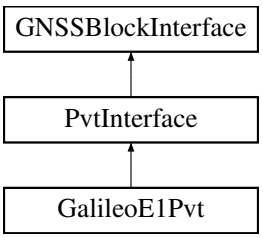
- src/algorithms/acquisition/adapters/galileo_e1_pcps_tong_ambiguous_acquisition.h

8.63 GalileoE1Pvt Class Reference

This class implements a PvtInterface for Galileo E1.

`#include <galileo_e1_pvt.h>`

Inheritance diagram for GalileoE1Pvt:



Public Member Functions

- **GalileoE1Pvt** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GALILEO_E1_PVT".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation. Returns sizeof(gr_complex)

8.63.1 Detailed Description

This class implements a PvtInterface for Galileo E1.

Definition at line 47 of file galileo_e1_pvt.h.

8.63.2 Member Function Documentation

8.63.2.1 std::string GalileoE1Pvt::implementation ( ) [inline], [virtual]

Returns "GALILEO_E1_PVT".

Implements GNSSBlockInterface.

Definition at line 63 of file galileo_e1_pvt.h.

8.63.2.2 size_t GalileoE1Pvt::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation. Returns sizeof(gr_complex)

Implements GNSSBlockInterface.

Definition at line 79 of file galileo_e1_pvt.h.

The documentation for this class was generated from the following file:

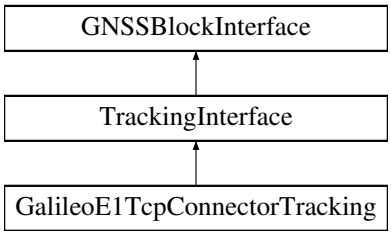
- src/algorithms/PVT/adapters/galileo_e1_pvt.h

8.64 GalileoE1TcpConnectorTracking Class Reference

This class implements a code DLL + carrier PLL tracking loop.

`#include <galileo_e1_tcp_connector_tracking.h>`

Inheritance diagram for GalileoE1TcpConnectorTracking:



Public Member Functions

- **GalileoE1TcpConnectorTracking** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E1_TCP_CONNECTOR_Tracking".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.64.1 Detailed Description

This class implements a code DLL + carrier PLL tracking loop.

Definition at line 52 of file galileo_e1_tcp_connector_tracking.h.

8.64.2 Member Function Documentation

8.64.2.1 std::string GalileoE1TcpConnectorTracking::implementation ( ) [inline], [virtual]

Returns "Galileo_E1_TCP_CONNECTOR_Tracking".

Implements GNSSBlockInterface.

Definition at line 68 of file galileo_e1_tcp_connector_tracking.h.

References set_channel(), and set_gnss_synchro().

8.64.2.2 void GalileoE1TcpConnectorTracking::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.64.2.3 void GalileoE1TcpConnectorTracking::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

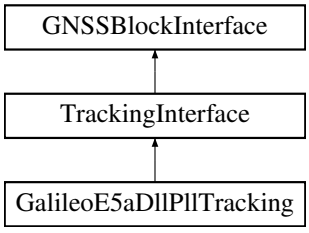
- src/algorithms/tracking/adapters/galileo_e1_tcp_connector_tracking.h

8.65 GalileoE5aDllPllTracking Class Reference

This class implements a code DLL + carrier PLL tracking loop.

`#include <galileo_e5a_dll_pll_tracking.h>`

Inheritance diagram for GalileoE5aDllPllTracking:



Public Member Functions

- **GalileoE5aDllPllTracking** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E5a_DLL_PLL_Tracking".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.65.1 Detailed Description

This class implements a code DLL + carrier PLL tracking loop.

Definition at line 52 of file galileo_e5a_dll_pll_tracking.h.

8.65.2 Member Function Documentation

8.65.2.1 std::string GalileoE5aDllPllTracking::implementation ( ) [inline], [virtual]

Returns "Galileo_E5a_DLL_PLL_Tracking".

Implements GNSSBlockInterface.

Definition at line 68 of file galileo_e5a_dll_pll_tracking.h.

References set_channel(), and set_gnss_synchro().

8.65.2.2 void GalileoE5aDllPllTracking::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.65.2.3 void GalileoE5aDllPllTracking::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

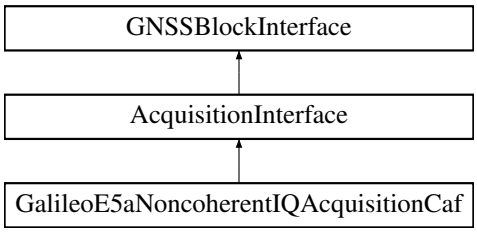
Referenced by implementation().

The documentation for this class was generated from the following file:

- src/algorithms/tracking/adapters/galileo_e5a_dll_pll_tracking.h

8.66 GalileoE5aNoncoherentIQAcquisitionCaf Class Reference

Inheritance diagram for GalileoE5aNoncoherentIQAcquisitionCaf:



Public Member Functions

- **GalileoE5aNoncoherentIQAcquisitionCaf** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E5a_Noncoherent_IQ_Acquisition_CAF".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.

- void set_local_code ()

  Sets local Galileo E5a code for PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.

8.66.1 Detailed Description

Definition at line 49 of file galileo_e5a_noncoherent_iq_acquisition_caf.h.

8.66.2 Member Function Documentation

8.66.2.1 std::string GalileoE5aNoncoherentIQAcquisitionCaf::implementation ( ) [inline], [virtual]

Returns "Galileo_E5a_Noncoherent_IQ_Acquisition_CAF".

Implements GNSSBlockInterface.

Definition at line 66 of file galileo_e5a_noncoherent_iq_acquisition_caf.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set↩_local_code(), set_state(), and set_threshold().

8.66.2.2 void GalileoE5aNoncoherentIQAcquisitionCaf::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.3 signed int GalileoE5aNoncoherentIQAcquisitionCaf::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.4 void GalileoE5aNoncoherentIQAcquisitionCaf::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.5 void GalileoE5aNoncoherentIQAcquisitionCaf::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.6 void GalileoE5aNoncoherentIQAcquisitionCaf::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.7 void GalileoE5aNoncoherentIQAcquisitionCaf::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.8 void GalileoE5aNoncoherentIQAcquisitionCaf::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.9 void GalileoE5aNoncoherentIQAcquisitionCaf::set_local_code ( ) [virtual]

Sets local Galileo E5a code for PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.66.2.10 void GalileoE5aNoncoherentIQAcquisitionCaf::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by implementation().

8.66.2.11 void GalileoE5aNoncoherentIQAcquisitionCaf::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

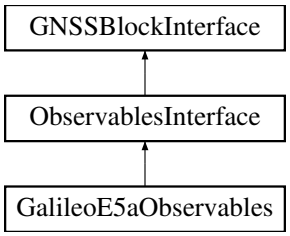
- src/algorithms/acquisition/adapters/galileo_e5a_noncoherent_iq_acquisition_caf.h

8.67 GalileoE5aObservables Class Reference

This class implements an ObservablesInterface for Galileo E5A.

`#include <galileo_e5a_observables.h>`

Inheritance diagram for GalileoE5aObservables:



Public Member Functions

- **GalileoE5aObservables** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E5A_Observables".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation.

8.67.1 Detailed Description

This class implements an ObservablesInterface for Galileo E5A.

Definition at line 46 of file galileo_e5a_observables.h.

8.67.2 Member Function Documentation

8.67.2.1 std::string GalileoE5aObservables::implementation ( ) [inline], [virtual]

Returns "Galileo_E5A_Observables".

Implements GNSSBlockInterface.

Definition at line 60 of file galileo_e5a_observables.h.

8.67.2.2 size_t GalileoE5aObservables::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation.

Implements GNSSBlockInterface.

Definition at line 74 of file galileo_e5a_observables.h.

The documentation for this class was generated from the following file:

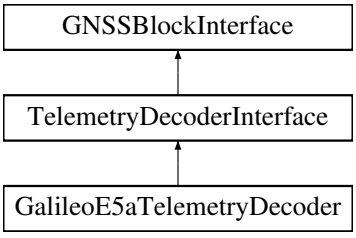
- src/algorithms/observables/adapters/galileo_e5a_observables.h

8.68 GalileoE5aTelemetryDecoder Class Reference

This class implements a NAV data decoder for Galileo INAV frames in E1B radio link.

```
#include <galileo_e5a_telemetry_decoder.h>
```

Inheritance diagram for GalileoE5aTelemetryDecoder:



Public Member Functions

- **GalileoE5aTelemetryDecoder** (ConfigurationInterface *configuration, std::string role, unsigned int in_↩ streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Galileo_E5a_Telemetry_Decoder".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **set_satellite** (Gnss_Satellite satellite)
- void **set_channel** (int channel)
- void **reset** ()
- size_t **item_size** ()

8.68.1 Detailed Description

This class implements a NAV data decoder for Galileo INAV frames in E1B radio link.

Definition at line 50 of file galileo_e5a_telemetry_decoder.h.

8.68.2 Member Function Documentation

8.68.2.1 std::string GalileoE5aTelemetryDecoder::implementation ( ) [inline], [virtual]

Returns "Galileo_E5a_Telemetry_Decoder".

Implements GNSSBlockInterface.

Definition at line 67 of file galileo_e5a_telemetry_decoder.h.

The documentation for this class was generated from the following file:

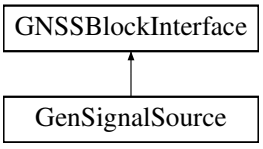
- src/algorithms/telemetry_decoder/adapters/galileo_e5a_telemetry_decoder.h

8.69 GenSignalSource Class Reference

This class wraps blocks that generates synthesized GNSS signal and filters the signal.

`#include <gen_signal_source.h>`

Inheritance diagram for GenSignalSource:



Public Member Functions

- GenSignalSource (GNSSBlockInterface *signal_generator, GNSSBlockInterface *filter, std::string role, boost::shared_ptr< gr::msg_queue > queue)

  Constructor.
- virtual ~GenSignalSource ()

  Virtual destructor.
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **role** ()
- std::string implementation ()

  Returns "Signal Source".
- size_t **item_size** ()
- GNSSBlockInterface * **signal_generator** ()

8.69.1 Detailed Description

This class wraps blocks that generates synthesized GNSS signal and filters the signal.

Definition at line 45 of file gen_signal_source.h.

8.69.2 Constructor & Destructor Documentation

8.69.2.1 GenSignalSource::GenSignalSource (GNSSBlockInterface * *signal_generator,* GNSSBlockInterface * *filter,* std::string *role,* boost::shared_ptr< gr::msg_queue > *queue*)

Constructor.

8.69.2.2 virtual GenSignalSource::~GenSignalSource ( ) [virtual]

Virtual destructor.

8.69.3 Member Function Documentation

8.69.3.1 std::string GenSignalSource::implementation ( ) [inline], [virtual]

Returns "Signal Source".

Implements GNSSBlockInterface.

Definition at line 63 of file gen_signal_source.h.

The documentation for this class was generated from the following file:

- src/algorithms/signal_source/adapters/gen_signal_source.h

8.70 GeoJSON_Printer Class Reference

Prints PVT solutions in GeoJSON format file.

`#include <geojson_printer.h>`

Public Member Functions

- bool **set_headers** (std::string filename, bool time_tag_name=true)
- bool **print_position** (const std::shared_ptr< Pvt_Solution > &position, bool print_average_values)
- bool **close_file** ()

8.70.1 Detailed Description

Prints PVT solutions in GeoJSON format file.

See http://geojson.org/geojson-spec.html

Definition at line 47 of file geojson_printer.h.

The documentation for this class was generated from the following file:

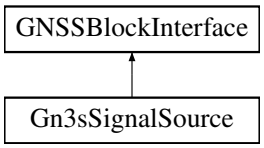
- src/algorithms/PVT/libs/geojson_printer.h

8.71 Gn3sSignalSource Class Reference

This class reads samples from a GN3S USB dongle, a RF front-end signal sampler.

`#include <gn3s_signal_source.h>`

Inheritance diagram for Gn3sSignalSource:



Public Member Functions

- **Gn3sSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream, gr::msg_queue::sptr queue)
- std::string **role** ()
- std::string implementation ()

  Returns "Gn3sSignalSource".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.71.1 Detailed Description

This class reads samples from a GN3S USB dongle, a RF front-end signal sampler.

Definition at line 47 of file gn3s_signal_source.h.

8.71.2 Member Function Documentation

8.71.2.1 std::string Gn3sSignalSource::implementation ( ) [inline], [virtual]

Returns "Gn3sSignalSource".

Implements GNSSBlockInterface.

Definition at line 63 of file gn3s_signal_source.h.

The documentation for this class was generated from the following file:

- src/algorithms/signal_source/adapters/gn3s_signal_source.h

8.72 Gnss_Satellite Class Reference

This class represents a GNSS satellite.

`#include <gnss_satellite.h>`

Public Member Functions

- Gnss_Satellite ()
  Default Constructor.
- Gnss_Satellite (const std::string &system_, unsigned int PRN_)
  Concrete GNSS satellite Constructor.
- ~Gnss_Satellite ()
  Default Destructor.
- unsigned int get_PRN () const
  Gets satellite's PRN.
- std::string get_system () const
  Gets the satellite system {"GPS", "GLONASS", "SBAS", "Galileo", "Beidou"}.
- std::string get_system_short () const
  Gets the satellite system {"G", "R", "SBAS", "E", "C"}.
- std::string get_block () const
  Gets the satellite block. If GPS, returns {"IIA", "IIR", "IIR-M", "IIF"}.
- std::string what_block (const std::string &system_, unsigned int PRN_)
  Gets the block of a given satellite.

Friends

- bool operator== (const Gnss_Satellite &, const Gnss_Satellite &)

  operator== for comparison
- std::ostream & operator<< (std::ostream &, const Gnss_Satellite &)

  operator<< for pretty printing

8.72.1 Detailed Description

This class represents a GNSS satellite.

It contains information about the space vehicles currently operational of GPS, Glonass, SBAS and Galileo constellations.

Definition at line 47 of file gnss_satellite.h.

8.72.2 Constructor & Destructor Documentation

8.72.2.1 Gnss_Satellite::Gnss_Satellite ( )

Default Constructor.

8.72.2.2 Gnss_Satellite::Gnss_Satellite (const std::string & *system_*, unsigned int *PRN_*)

Concrete GNSS satellite Constructor.

8.72.2.3 Gnss_Satellite::~Gnss_Satellite ( )

Default Destructor.

8.72.3 Member Function Documentation

8.72.3.1 std::string Gnss_Satellite::get_block ( ) const

Gets the satellite block. If GPS, returns {"IIA", "IIR", "IIR-M", "IIF"}.

8.72.3.2 unsigned int Gnss_Satellite::get_PRN ( ) const

Gets satellite's PRN.

8.72.3.3 std::string Gnss_Satellite::get_system ( ) const

Gets the satellite system {"GPS", "GLONASS", "SBAS", "Galileo", "Beidou"}.

8.72.3.4 std::string Gnss_Satellite::get_system_short ( ) const

Gets the satellite system {"G", "R", "SBAS", "E", "C"}.

8.72.3.5 std::string Gnss_Satellite::what_block (const std::string & *system_*, unsigned int *PRN_*)

Gets the block of a given satellite.

8.72.4 Friends And Related Function Documentation

8.72.4.1 std::ostream& operator<< (std::ostream & , const Gnss_Satellite &) [friend]

operator<< for pretty printing

8.72.4.2 bool operator== (const Gnss_Satellite & , const Gnss_Satellite &) [friend]

operator== for comparison

The documentation for this class was generated from the following file:

- src/core/system_parameters/gnss_satellite.h

8.73 gnss_sdr_supl_client Class Reference

class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library..

`#include <gnss_sdr_supl_client.h>`

Public Member Functions

- int **get_assistance** (int i_mcc, int i_mns, int i_lac, int i_ci)
- void **read_supl_data** ()
- bool load_ephemeris_xml (const std::string file_name)
  Read ephemeris map from XML file.
- bool save_ephemeris_map_xml (const std::string file_name, std::map< int, Gps_Ephemeris > eph_map)
  Save ephemeris map to XML file.
- bool load_utc_xml (const std::string file_name)
  Read utc model from XML file.
- bool save_utc_map_xml (const std::string file_name, std::map< int, Gps_Utc_Model > utc_map)
  Save utc model map to XML file To be called by ControlThread::gps_utc_model_data_write_to_XML()
- bool load_iono_xml (const std::string file_name)
  Read iono from XML file.
- bool save_iono_map_xml (const std::string file_name, std::map< int, Gps_Iono > iono_map)
  Save iono map to XML file.
- bool load_ref_time_xml (const std::string file_name)
  Read ref time from XML file.
- bool save_ref_time_map_xml (const std::string file_name, std::map< int, Gps_Ref_Time > ref_time_map)
  Save ref time map to XML file.
- bool load_ref_location_xml (const std::string file_name)
  Read ref location from XML file.
- bool save_ref_location_map_xml (std::string file_name, std::map< int, Gps_Ref_Location > ref_location_↩ map)
  Save ref location map to XML file.
- void **print_assistance** ()

Public Attributes

- std::string **server_name**
- int **server_port**
- int **request**
- std::map< int, Gps_Ephemeris > **gps_ephemeris_map**
- std::map< int, Gps_Almanac > **gps_almanac_map**
- Gps_Iono **gps_iono**
- Gps_Ref_Time **gps_time**
- Gps_Utc_Model **gps_utc**
- Gps_Ref_Location **gps_ref_loc**
- std::map< int, Gps_Acq_Assist > **gps_acq_map**

8.73.1 Detailed Description

class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library..

Definition at line 60 of file gnss_sdr_supl_client.h.

8.73.2 Member Function Documentation

8.73.2.1 bool gnss_sdr_supl_client::load_ephemeris_xml (const std::string *file_name*)

Read ephemeris map from XML file.

8.73.2.2 bool gnss_sdr_supl_client::load_iono_xml (const std::string *file_name*)

Read iono from XML file.

8.73.2.3 bool gnss_sdr_supl_client::load_ref_location_xml (const std::string *file_name*)

Read ref location from XML file.

8.73.2.4 bool gnss_sdr_supl_client::load_ref_time_xml (const std::string *file_name*)

Read ref time from XML file.

8.73.2.5 bool gnss_sdr_supl_client::load_utc_xml (const std::string *file_name*)

Read utc model from XML file.

8.73.2.6 bool gnss_sdr_supl_client::save_ephemeris_map_xml (const std::string *file_name,* std::map< int, Gps_Ephemeris > *eph_map*)

Save ephemeris map to XML file.

8.73.2.7 bool gnss_sdr_supl_client::save_iono_map_xml (const std::string *file_name,* std::map< int, Gps_Iono > *iono_map*)

Save iono map to XML file.

8.73.2.8 bool gnss_sdr_supl_client::save_ref_location_map_xml (std::string *file_name,* std::map< int, Gps_Ref_Location > *ref_location_map*)

Save ref location map to XML file.

8.73.2.9 bool gnss_sdr_supl_client::save_ref_time_map_xml (const std::string *file_name,* std::map< int, Gps_Ref_Time > *ref_time_map*)

Save ref time map to XML file.

8.73.2.10 bool gnss_sdr_supl_client::save_utc_map_xml (const std::string *file_name,* std::map< int, Gps_Utc_Model > *utc_map*)

Save utc model map to XML file To be called by ControlThread::gps_utc_model_data_write_to_XML()

The documentation for this class was generated from the following file:

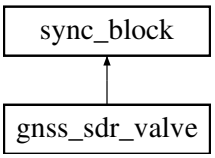
- src/core/libs/gnss_sdr_supl_client.h

8.74 gnss_sdr_valve Class Reference

Implementation of a GNU Radio block that sends a STOP message to the control queue right after a specific number of samples have passed through it.

`#include <gnss_sdr_valve.h>`

Inheritance diagram for gnss_sdr_valve:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- boost::shared_ptr< gr::block > **gnss_sdr_make_valve** (size_t sizeof_stream_item, unsigned long long nitems, gr::msg_queue::sptr queue)

8.74.1 Detailed Description

Implementation of a GNU Radio block that sends a STOP message to the control queue right after a specific number of samples have passed through it.

Definition at line 49 of file gnss_sdr_valve.h.

The documentation for this class was generated from the following file:

- src/algorithms/libs/gnss_sdr_valve.h

8.75 Gnss_Signal Class Reference

This class represents a GNSS signal.

`#include <gnss_signal.h>`

Public Member Functions

- **Gnss_Signal** (const std::string &signal_)
- **Gnss_Signal** (const Gnss_Satellite &satellite_, const std::string &signal_)
- std::string get_signal_str () const

  Get the satellite signal {"1C" for GPS L1 C/A, "2S" for GPS L2C (M), "1B" for Galileo E1B, "5X" for Galileo E5a}.
- Gnss_Satellite get_satellite () const

  Get the Gnss_Satellite associated to the signal.

Friends

- bool operator== (const Gnss_Signal &, const Gnss_Signal &)

  operator== for comparison
- std::ostream & operator<< (std::ostream &, const Gnss_Signal &)

  operator<< for pretty printing

8.75.1 Detailed Description

This class represents a GNSS signal.

It contains information about the space vehicle and the specific signal.

Definition at line 44 of file gnss_signal.h.

8.75.2 Member Function Documentation

8.75.2.1 Gnss_Satellite Gnss_Signal::get_satellite ( ) const

Get the Gnss_Satellite associated to the signal.

8.75.2.2 std::string Gnss_Signal::get_signal_str ( ) const

Get the satellite signal {"1C" for GPS L1 C/A, "2S" for GPS L2C (M), "1B" for Galileo E1B, "5X" for Galileo E5a}.

8.75.3 Friends And Related Function Documentation

8.75.3.1 std::ostream& operator<< (std::ostream & , const Gnss_Signal &) [friend]

operator<< for pretty printing

8.75.3.2 bool operator== (const Gnss_Signal & , const Gnss_Signal &) [friend]

operator== for comparison

The documentation for this class was generated from the following file:

- src/core/system_parameters/gnss_signal.h

8.76 Gnss_Synchro Class Reference

This is the class that contains the information that is shared by the processing blocks.

`#include <gnss_synchro.h>`

Public Attributes

- char System
  Set by Channel::set_signal(Gnss_Signal gnss_signal)
- char Signal [3]
  Set by Channel::set_signal(Gnss_Signal gnss_signal)
- unsigned int PRN
  Set by Channel::set_signal(Gnss_Signal gnss_signal)
- int Channel_ID
  Set by Channel constructor.
- double Acq_delay_samples
  Set by Acquisition processing block.
- double Acq_doppler_hz
  Set by Acquisition processing block.
- unsigned long int Acq_samplestamp_samples
  Set by Acquisition processing block.
- bool Flag_valid_acquisition
  Set by Acquisition processing block.
- double Prompt_I
  Set by Tracking processing block.
- double Prompt_Q
  Set by Tracking processing block.

- double CN0_dB_hz
  Set by Tracking processing block.
- double Carrier_Doppler_hz
  Set by Tracking processing block.
- double Carrier_phase_rads
  Set by Tracking processing block.
- double Tracking_timestamp_secs
  Set by Tracking processing block.
- double Rem_code_phase_secs
  Set by Tracking processing block.
- bool Flag_valid_symbol_output
  Set by Tracking processing block.
- int correlation_length_ms
  Set by Tracking processing block.
- double Prn_timestamp_ms
  Set by Telemetry Decoder processing block.
- double Prn_timestamp_at_preamble_ms
  Set by Telemetry Decoder processing block.
- bool Flag_valid_word
  Set by Telemetry Decoder processing block.
- bool Flag_preamble
  Set by Telemetry Decoder processing block.
- double d_TOW
  Set by Telemetry Decoder processing block.
- double **d_TOW_at_current_symbol**
- double **d_TOW_hybrid_at_current_symbol**
- double **Pseudorange_m**
- bool **Flag_valid_pseudorange**

8.76.1 Detailed Description

This is the class that contains the information that is shared by the processing blocks.

Definition at line 41 of file gnss_synchro.h.

8.76.2 Member Data Documentation

8.76.2.1 double Gnss_Synchro::Acq_delay_samples

Set by Acquisition processing block.

Definition at line 50 of file gnss_synchro.h.

8.76.2.2 double Gnss_Synchro::Acq_doppler_hz

Set by Acquisition processing block.

Definition at line 51 of file gnss_synchro.h.

8.76.2.3 unsigned long int Gnss_Synchro::Acq_samplestamp_samples

Set by Acquisition processing block.

Definition at line 52 of file gnss_synchro.h.

8.76.2.4 double Gnss_Synchro::Carrier_Doppler_hz

Set by Tracking processing block.

Definition at line 58 of file gnss_synchro.h.

8.76.2.5 double Gnss_Synchro::Carrier_phase_rads

Set by Tracking processing block.

Definition at line 59 of file gnss_synchro.h.

8.76.2.6 int Gnss_Synchro::Channel_ID

Set by Channel constructor.

Definition at line 48 of file gnss_synchro.h.

8.76.2.7 double Gnss_Synchro::CN0_dB_hz

Set by Tracking processing block.

Definition at line 57 of file gnss_synchro.h.

8.76.2.8 int Gnss_Synchro::correlation_length_ms

Set by Tracking processing block.

Definition at line 64 of file gnss_synchro.h.

8.76.2.9 double Gnss_Synchro::d_TOW

Set by Telemetry Decoder processing block.

Definition at line 72 of file gnss_synchro.h.

8.76.2.10 bool Gnss_Synchro::Flag_preamble

Set by Telemetry Decoder processing block.

Definition at line 71 of file gnss_synchro.h.

8.76.2.11 bool Gnss_Synchro::Flag_valid_acquisition

Set by Acquisition processing block.

Definition at line 53 of file gnss_synchro.h.

8.76.2.12 bool Gnss_Synchro::Flag_valid_symbol_output

Set by Tracking processing block.

Definition at line 63 of file gnss_synchro.h.

8.76.2.13 bool Gnss_Synchro::Flag_valid_word

Set by Telemetry Decoder processing block.

Definition at line 70 of file gnss_synchro.h.

8.76.2.14 unsigned int Gnss_Synchro::PRN

Set by Channel::set_signal(Gnss_Signal gnss_signal)

Definition at line 47 of file gnss_synchro.h.

8.76.2.15 double Gnss_Synchro::Prn_timestamp_at_preamble_ms

Set by Telemetry Decoder processing block.

Definition at line 68 of file gnss_synchro.h.

8.76.2.16 double Gnss_Synchro::Prn_timestamp_ms

Set by Telemetry Decoder processing block.

Definition at line 67 of file gnss_synchro.h.

8.76.2.17 double Gnss_Synchro::Prompt_I

Set by Tracking processing block.

Definition at line 55 of file gnss_synchro.h.

8.76.2.18 double Gnss_Synchro::Prompt_Q

Set by Tracking processing block.

Definition at line 56 of file gnss_synchro.h.

8.76.2.19 double Gnss_Synchro::Rem_code_phase_secs

Set by Tracking processing block.

Definition at line 61 of file gnss_synchro.h.

8.76.2.20 char Gnss_Synchro::Signal[3]

Set by Channel::set_signal(Gnss_Signal gnss_signal)

Definition at line 46 of file gnss_synchro.h.

8.76.2.21 char Gnss_Synchro::System

Set by Channel::set_signal(Gnss_Signal gnss_signal)

Definition at line 45 of file gnss_synchro.h.

8.76.2.22 double Gnss_Synchro::Tracking_timestamp_secs

Set by Tracking processing block.

Definition at line 60 of file gnss_synchro.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gnss_synchro.h

8.77 GNSSBlockFactory Class Reference

Class that produces all kinds of GNSS blocks.

`#include <gnss_block_factory.h>`

Public Member Functions

- std::unique_ptr< GNSSBlockInterface > **GetSignalSource** (std::shared_ptr< ConfigurationInterface > configuration, boost::shared_ptr< gr::msg_queue > queue, int ID=-1)
- std::unique_ptr< GNSSBlockInterface > **GetSignalConditioner** (std::shared_ptr< ConfigurationInterface > configuration, int ID=-1)
- std::unique_ptr< GNSSBlockInterface > **GetPVT** (std::shared_ptr< ConfigurationInterface > configuration)
- std::unique_ptr< GNSSBlockInterface > **GetObservables** (std::shared_ptr< ConfigurationInterface > configuration)
- std::unique_ptr< std::vector< std::unique_ptr< GNSSBlockInterface > > > **GetChannels** (std::shared_ptr< ConfigurationInterface > configuration, boost::shared_ptr< gr::msg_queue > queue)
- std::unique_ptr< GNSSBlockInterface > **GetBlock** (std::shared_ptr< ConfigurationInterface > configuration, std::string role, std::string implementation, unsigned int in_streams, unsigned int out_streams, boost::shared_ptr< gr::msg_queue > queue=nullptr)

8.77.1 Detailed Description

Class that produces all kinds of GNSS blocks.

Definition at line 54 of file gnss_block_factory.h.

The documentation for this class was generated from the following file:

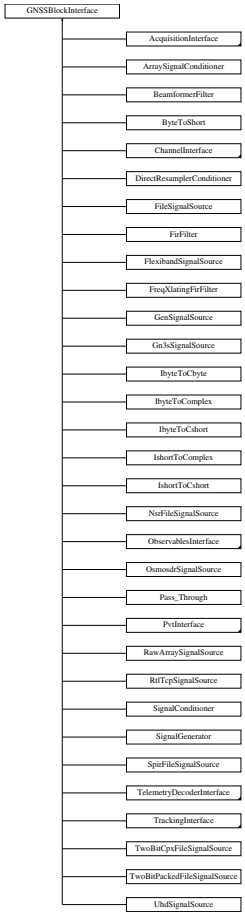
- src/core/receiver/gnss_block_factory.h

8.78 GNSSBlockInterface Class Reference

This abstract class represents an interface to GNSS blocks.

`#include <gnss_block_interface.h>`

Inheritance diagram for GNSSBlockInterface:



Public Member Functions

- virtual std::string **role** ()=0
- virtual std::string **implementation** ()=0
- virtual size_t **item_size** ()=0
- virtual void **connect** (gr::top_block_sptr top_block)=0
- virtual void **disconnect** (gr::top_block_sptr top_block)=0
- virtual gr::basic_block_sptr **get_left_block** ()=0
- virtual gr::basic_block_sptr **get_right_block** ()=0
- virtual gr::basic_block_sptr **get_left_block** (int RF_channel)
- virtual gr::basic_block_sptr **get_right_block** (int RF_channel)

8.78.1 Detailed Description

This abstract class represents an interface to GNSS blocks.

Abstract class for GNSS block interfaces. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 52 of file gnss_block_interface.h.

The documentation for this class was generated from the following file:

- src/core/interfaces/gnss_block_interface.h

8.79 GNSSFlowgraph Class Reference

This class represents a GNSS flowgraph.

`#include <gnss_flowgraph.h>`

Public Member Functions

- GNSSFlowgraph (std::shared_ptr< ConfigurationInterface > configuration, boost::shared_ptr< gr::msg_↩ queue > queue)

  Constructor that initializes the receiver flowgraph.
- virtual ~GNSSFlowgraph ()

  Virtual destructor.
- void start ()

  Start the flowgraph.
- void stop ()

  Stop the flowgraph.
- void connect ()

  Connects the defined blocks in the flowgraph.
- void **wait** ()
- void apply_action (unsigned int who, unsigned int what)

  Applies an action to the flowgraph.
- void **set_configuration** (std::shared_ptr< ConfigurationInterface > configuration)
- unsigned int **applied_actions** ()
- bool **connected** ()
- bool **running** ()
- bool send_telemetry_msg (pmt::pmt_t msg)

  Sends a GNURadio asyncronous message from telemetry to PVT.

8.79.1 Detailed Description

This class represents a GNSS flowgraph.

It contains a signal source, a signal conditioner, a set of channels, a PVT and an output filter.

Definition at line 59 of file gnss_flowgraph.h.

8.79.2 Constructor & Destructor Documentation

8.79.2.1 GNSSFlowgraph::GNSSFlowgraph (std::shared_ptr< ConfigurationInterface > *configuration,* boost::shared_ptr< gr::msg_queue > *queue*)

Constructor that initializes the receiver flowgraph.

8.79.2.2 virtual GNSSFlowgraph::~GNSSFlowgraph ( ) [virtual]

Virtual destructor.

8.79.3 Member Function Documentation

8.79.3.1 void GNSSFlowgraph::apply_action (unsigned int *who,* unsigned int *what*)

Applies an action to the flowgraph.

Parameters

in	*who*	Who generated the action
in	*what*	What is the action 0: acquisition failed

8.79.3.2 void GNSSFlowgraph::connect ( )

Connects the defined blocks in the flowgraph.

Signal Source > Signal conditioner > Channels >> Observables >> PVT > Output filter

8.79.3.3 bool GNSSFlowgraph::send_telemetry_msg (pmt::pmt_t *msg*)

Sends a GNURadio asyncronous message from telemetry to PVT.

It is used to assist the receiver with external ephemeris data

8.79.3.4 void GNSSFlowgraph::start ( )

Start the flowgraph.

8.79.3.5 void GNSSFlowgraph::stop ( )

Stop the flowgraph.

The documentation for this class was generated from the following file:

- src/core/receiver/gnss_flowgraph.h

8.80 Gps_Acq_Assist Class Reference

This class is a storage for the GPS GSM RRLL acquisition assistance data as described in Digital cellular telecommunications system (Phase 2+); Location Services (LCS); Mobile Station (MS) - Serving Mobile Location Centre (SMLC) Radio Resource LCS Protocol (RRLP) (3GPP TS 44.031 version 5.12.0 Release 5)

`#include <gps_acq_assist.h>`

Public Member Functions

- Gps_Acq_Assist ()

Public Attributes

- unsigned int i_satellite_PRN
  SV PRN NUMBER.
- double d_TOW
  Time Of Week assigned to the acquisition data.
- double d_Doppler0
  Doppler (0 order term) [Hz].
- double d_Doppler1
  Doppler (1 order term) [Hz].
- double dopplerUncertainty
  Doppler Uncertainty [Hz].
- double Code_Phase
  Code phase [chips].
- double Code_Phase_int
  Integer Code Phase [1 C/A code period].
- double GPS_Bit_Number
  GPS Bit Number.
- double Code_Phase_window
  Code Phase search window [chips].
- double Azimuth
  Satellite Azimuth [deg].
- double Elevation
  Satellite Elevation [deg].

8.80.1 Detailed Description

This class is a storage for the GPS GSM RRLL acquisition assistance data as described in Digital cellular telecommunications system (Phase 2+); Location Services (LCS); Mobile Station (MS) - Serving Mobile Location Centre (SMLC) Radio Resource LCS Protocol (RRLP) (3GPP TS 44.031 version 5.12.0 Release 5)

Definition at line 45 of file gps_acq_assist.h.

8.80.2 Constructor & Destructor Documentation

8.80.2.1 Gps_Acq_Assist::Gps_Acq_Assist ( )

Default constructor

8.80.3 Member Data Documentation

8.80.3.1 double Gps_Acq_Assist::Azimuth

Satellite Azimuth [deg].

Definition at line 57 of file gps_acq_assist.h.

8.80.3.2 double Gps_Acq_Assist::Code_Phase

Code phase [chips].

Definition at line 53 of file gps_acq_assist.h.

8.80.3.3 double Gps_Acq_Assist::Code_Phase_int

Integer Code Phase [1 C/A code period].

Definition at line 54 of file gps_acq_assist.h.

8.80.3.4 double Gps_Acq_Assist::Code_Phase_window

Code Phase search window [chips].

Definition at line 56 of file gps_acq_assist.h.

8.80.3.5 double Gps_Acq_Assist::d_Doppler0

Doppler (0 order term) [Hz].

Definition at line 50 of file gps_acq_assist.h.

8.80.3.6 double Gps_Acq_Assist::d_Doppler1

Doppler (1 order term) [Hz].

Definition at line 51 of file gps_acq_assist.h.

8.80.3.7 double Gps_Acq_Assist::d_TOW

Time Of Week assigned to the acquisition data.

Definition at line 49 of file gps_acq_assist.h.

8.80.3.8 double Gps_Acq_Assist::dopplerUncertainty

Doppler Uncertainty [Hz].

Definition at line 52 of file gps_acq_assist.h.

8.80.3.9 double Gps_Acq_Assist::Elevation

Satellite Elevation [deg].

Definition at line 58 of file gps_acq_assist.h.

8.80.3.10 double Gps_Acq_Assist::GPS_Bit_Number

GPS Bit Number.

Definition at line 55 of file gps_acq_assist.h.

8.80.3.11 unsigned int Gps_Acq_Assist::i_satellite_PRN

SV PRN NUMBER.

Definition at line 48 of file gps_acq_assist.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_acq_assist.h

8.81 Gps_Almanac Class Reference

This class is a storage for the GPS SV ALMANAC data as described in IS-GPS-200E.

`#include <gps_almanac.h>`

Public Member Functions

- Gps_Almanac ()

Public Attributes

- unsigned int i_satellite_PRN

  SV PRN NUMBER.
- double **d_Delta_i**
- double d_Toa

  Almanac data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].
- double d_M_0

  Mean Anomaly at Reference Time [semi-circles].
- double d_e_eccentricity

  Eccentricity [dimensionless].
- double d_sqrt_A

  Square Root of the Semi-Major Axis [sqrt(m)].
- double d_OMEGA0

  Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles].
- double d_OMEGA

  Argument of Perigee [semi-cicles].
- double d_OMEGA_DOT

  Rate of Right Ascension [semi-circles/s].
- int **i_SV_health**
- double d_A_f0

  Coefficient 0 of code phase offset model [s].
- double d_A_f1

  Coefficient 1 of code phase offset model [s/s].

8.81.1 Detailed Description

This class is a storage for the GPS SV ALMANAC data as described in IS-GPS-200E.

See http://www.gps.gov/technical/icwg/IS-GPS-200E.pdf Appendix II

Definition at line 41 of file gps_almanac.h.

8.81.2 Constructor & Destructor Documentation

8.81.2.1 Gps_Almanac::Gps_Almanac ( )

Default constructor

8.81.3 Member Data Documentation

8.81.3.1 double Gps_Almanac::d_A_f0

Coefficient 0 of code phase offset model [s].

Definition at line 54 of file gps_almanac.h.

8.81.3.2 double Gps_Almanac::d_A_f1

Coefficient 1 of code phase offset model [s/s].

Definition at line 55 of file gps_almanac.h.

8.81.3.3 double Gps_Almanac::d_e_eccentricity

Eccentricity [dimensionless].

Definition at line 48 of file gps_almanac.h.

8.81.3.4 double Gps_Almanac::d_M_0

Mean Anomaly at Reference Time [semi-circles].

Definition at line 47 of file gps_almanac.h.

8.81.3.5 double Gps_Almanac::d_OMEGA

Argument of Perigee [semi-cicles].

Definition at line 51 of file gps_almanac.h.

8.81.3.6 double Gps_Almanac::d_OMEGA0

Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles].

Definition at line 50 of file gps_almanac.h.

8.81.3.7 double Gps_Almanac::d_OMEGA_DOT

Rate of Right Ascension [semi-circles/s].

Definition at line 52 of file gps_almanac.h.

8.81.3.8 double Gps_Almanac::d_sqrt_A

Square Root of the Semi-Major Axis [sqrt(m)].

Definition at line 49 of file gps_almanac.h.

8.81.3.9 double Gps_Almanac::d_Toa

Almanac data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].

Definition at line 46 of file gps_almanac.h.

8.81.3.10 unsigned int Gps_Almanac::i_satellite_PRN

SV PRN NUMBER.

Definition at line 44 of file gps_almanac.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_almanac.h

8.82 Gps_CNAV_Ephemeris Class Reference

This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200H.

`#include <gps_cnav_ephemeris.h>`

Public Member Functions

- template<class Archive >
  void serialize (Archive &archive, const unsigned int version)

  Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.
- void satellitePosition (double transmitTime)

  Compute the ECEF SV coordinates and ECEF velocity Implementation of Table 20-IV (IS-GPS-200E)
- double sv_clock_drift (double transmitTime)

  Sets (d_satClkDrift)and returns the clock drift in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)
- double sv_clock_relativistic_term (double transmitTime)

  Sets (d_dtr) and returns the clock relativistic correction term in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)
- Gps_CNAV_Ephemeris ()

Public Attributes

- unsigned int **i_satellite_PRN**
- int i_GPS_week

  GPS week number, aka WN [week].
- int i_URA

  ED Accuracy Index.
- int i_signal_health

  Signal health (L1/L2/L5)
- double d_Top

  Data predict time of week.
- double d_DELTA_A

  Semi-major axis difference at reference time.
- double d_A_DOT

  Change rate in semi-major axis.
- double d_Delta_n

  Mean Motion Difference From Computed Value [semi-circles/s].

- double d_DELTA_DOT_N

  Rate of mean motion difference from computed value.
- double d_M_0

  Mean Anomaly at Reference Time [semi-circles].
- double d_e_eccentricity

  Eccentricity.
- double d_OMEGA

  Argument of Perigee [semi-cicles].
- double d_OMEGA0

  Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-cicles].
- double d_Toe1

  Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].
- double d_Toe2

  Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].
- double d_DELTA_OMEGA_DOT

  Rate of Right Ascension difference [semi-circles/s].
- double d_i_0

  Inclination Angle at Reference Time [semi-circles].
- double d_IDOT

  Rate of Inclination Angle [semi-circles/s].
- double d_Cis

  Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad].
- double d_Cic

  Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad].
- double d_Crs

  Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m].
- double d_Crc

  Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m].
- double d_Cus

  Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad].
- double d_Cuc

  Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad].
- double d_Toc

  clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]
- double d_A_f0

  Coefficient 0 of code phase offset model [s].
- double d_A_f1

  Coefficient 1 of code phase offset model [s/s].
- double d_A_f2

  Coefficient 2 of code phase offset model [s/s^2].
- double d_URA0

  NED Accuracy Index.
- double d_URA1

  NED Accuracy Change Index.
- double d_URA2

  NED Accuracy Change Rate Index.
- double d_TGD

  Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s].
- double **d_ISCL1**
- double **d_ISCL2**
- double **d_ISCL5I**

- double **d_ISCL5Q**
- double d_TOW

  Time of GPS Week of the ephemeris set (taken from subframes TOW) [s].
- bool b_integrity_status_flag

  If true, enhanced level of integrity assurance.
- bool **b_l2c_phasing_flag**
- bool b_alert_flag

  If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.
- bool b_antispoofing_flag

  If true, the AntiSpoofing mode is ON in that SV.
- double d_satClkDrift

  GPS clock error.
- double d_dtr

  relativistic clock correction term
- double d_satpos_X

  Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.
- double d_satpos_Y

  Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.
- double d_satpos_Z

  Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).
- double d_satvel_X

  Earth-fixed velocity coordinate x of the satellite [m].
- double d_satvel_Y

  Earth-fixed velocity coordinate y of the satellite [m].
- double d_satvel_Z

  Earth-fixed velocity coordinate z of the satellite [m].

8.82.1 Detailed Description

This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200H.

See http://www.gps.gov/technical/icwg/IS-GPS-200H.pdf Appendix III

Definition at line 44 of file gps_cnav_ephemeris.h.

8.82.2 Constructor & Destructor Documentation

8.82.2.1 Gps_CNAV_Ephemeris::Gps_CNAV_Ephemeris ( )

Default constructor

Referenced by serialize().

8.82.3 Member Function Documentation

8.82.3.1 void Gps_CNAV_Ephemeris::satellitePosition (double *transmitTime*)

Compute the ECEF SV coordinates and ECEF velocity Implementation of Table 20-IV (IS-GPS-200E)

Referenced by serialize().

8.82.3.2 template<class Archive > void Gps_CNAV_Ephemeris::serialize (Archive & *archive,* const unsigned int *version*) [inline]

Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

< Time of GPS Week of the ephemeris set (taken from subframes TOW) [s]

< Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m]

< Mean Anomaly at Reference Time [semi-circles]

< Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad]

< Eccentricity [dimensionless]

< Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad]

< Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s]

< Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s]

< clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]

< Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad]

< Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles]

< Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad]

< Inclination Angle at Reference Time [semi-circles]

< Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m]

< Argument of Perigee [semi-cicles]

< Rate of Inclination Angle [semi-circles/s]

< GPS week number, aka WN [week]

< Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s]

< Coefficient 0 of code phase offset model [s]

< Coefficient 1 of code phase offset model [s/s]

< Coefficient 2 of code phase offset model [s/s^2]

< If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.

< If true, the AntiSpoofing mode is ON in that SV

Definition at line 132 of file gps_cnav_ephemeris.h.

References Gps_CNAV_Ephemeris(), satellitePosition(), sv_clock_drift(), and sv_clock_relativistic_term().

8.82.3.3 double Gps_CNAV_Ephemeris::sv_clock_drift (double *transmitTime*)

Sets (*d_satClkDrift*)and returns the clock drift in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)

Referenced by serialize().

8.82.3.4 double Gps_CNAV_Ephemeris::sv_clock_relativistic_term (double *transmitTime*)

Sets (*d_dtr*) and returns the clock relativistic correction term in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)

Referenced by serialize().

8.82.4 Member Data Documentation

8.82.4.1 bool Gps_CNAV_Ephemeris::b_alert_flag

If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.

Definition at line 110 of file gps_cnav_ephemeris.h.

8.82.4.2 bool Gps_CNAV_Ephemeris::b_antispoofing_flag

If true, the AntiSpoofing mode is ON in that SV.

Definition at line 111 of file gps_cnav_ephemeris.h.

8.82.4.3 bool Gps_CNAV_Ephemeris::b_integrity_status_flag

If true, enhanced level of integrity assurance.

If false, indicates that the conveying signal is provided with the legacy level of integrity assurance. That is, the probability that the instantaneous URE of the conveying signal exceeds 4.42 times the upper bound value of the current broadcast URA index, for more than 5.2 seconds, without an accompanying alert, is less than 1E-5 per hour. If true, indicates that the conveying signal is provided with an enhanced level of integrity assurance. That is, the probability that the instantaneous URE of the conveying signal exceeds 5.73 times the upper bound value of the current broadcast URA index, for more than 5.2 seconds, without an accompanying alert, is less than 1E-8 per hour.

Definition at line 108 of file gps_cnav_ephemeris.h.

8.82.4.4 double Gps_CNAV_Ephemeris::d_A_DOT

Change rate in semi-major axis.

Definition at line 57 of file gps_cnav_ephemeris.h.

8.82.4.5 double Gps_CNAV_Ephemeris::d_A_f0

Coefficient 0 of code phase offset model [s].

Definition at line 78 of file gps_cnav_ephemeris.h.

8.82.4.6 double Gps_CNAV_Ephemeris::d_A_f1

Coefficient 1 of code phase offset model [s/s].

Definition at line 79 of file gps_cnav_ephemeris.h.

8.82.4.7 double Gps_CNAV_Ephemeris::d_A_f2

Coefficient 2 of code phase offset model [s/s^2].

Definition at line 80 of file gps_cnav_ephemeris.h.

8.82.4.8 double Gps_CNAV_Ephemeris::d_Cic

Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad].

Definition at line 70 of file gps_cnav_ephemeris.h.

8.82.4.9 double Gps_CNAV_Ephemeris::d_Cis

Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad].

Definition at line 69 of file gps_cnav_ephemeris.h.

8.82.4.10 double Gps_CNAV_Ephemeris::d_Crc

Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m].

Definition at line 72 of file gps_cnav_ephemeris.h.

8.82.4.11 double Gps_CNAV_Ephemeris::d_Crs

Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m].

Definition at line 71 of file gps_cnav_ephemeris.h.

8.82.4.12 double Gps_CNAV_Ephemeris::d_Cuc

Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad].

Definition at line 74 of file gps_cnav_ephemeris.h.

8.82.4.13 double Gps_CNAV_Ephemeris::d_Cus

Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad].

Definition at line 73 of file gps_cnav_ephemeris.h.

8.82.4.14 double Gps_CNAV_Ephemeris::d_DELTA_A

Semi-major axis difference at reference time.

Definition at line 56 of file gps_cnav_ephemeris.h.

8.82.4.15 double Gps_CNAV_Ephemeris::d_DELTA_DOT_N

Rate of mean motion difference from computed value.

Definition at line 59 of file gps_cnav_ephemeris.h.

8.82.4.16 double Gps_CNAV_Ephemeris::d_Delta_n

Mean Motion Difference From Computed Value [semi-circles/s].

Definition at line 58 of file gps_cnav_ephemeris.h.

8.82.4.17 double Gps_CNAV_Ephemeris::d_DELTA_OMEGA_DOT

Rate of Right Ascension difference [semi-circles/s].

Definition at line 66 of file gps_cnav_ephemeris.h.

8.82.4.18 double Gps_CNAV_Ephemeris::d_dtr

relativistic clock correction term

Definition at line 115 of file gps_cnav_ephemeris.h.

8.82.4.19 double Gps_CNAV_Ephemeris::d_e_eccentricity

Eccentricity.

Definition at line 61 of file gps_cnav_ephemeris.h.

8.82.4.20 double Gps_CNAV_Ephemeris::d_i_0

Inclination Angle at Reference Time [semi-circles].

Definition at line 67 of file gps_cnav_ephemeris.h.

8.82.4.21 double Gps_CNAV_Ephemeris::d_IDOT

Rate of Inclination Angle [semi-circles/s].

Definition at line 68 of file gps_cnav_ephemeris.h.

8.82.4.22 double Gps_CNAV_Ephemeris::d_M_0

Mean Anomaly at Reference Time [semi-circles].

Definition at line 60 of file gps_cnav_ephemeris.h.

8.82.4.23 double Gps_CNAV_Ephemeris::d_OMEGA

Argument of Perigee [semi-cicles].

Definition at line 62 of file gps_cnav_ephemeris.h.

8.82.4.24 double Gps_CNAV_Ephemeris::d_OMEGA0

Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-cicles].

Definition at line 63 of file gps_cnav_ephemeris.h.

8.82.4.25 double Gps_CNAV_Ephemeris::d_satClkDrift

GPS clock error.

Definition at line 114 of file gps_cnav_ephemeris.h.

8.82.4.26 double Gps_CNAV_Ephemeris::d_satpos_X

Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

Definition at line 118 of file gps_cnav_ephemeris.h.

8.82.4.27 double Gps_CNAV_Ephemeris::d_satpos_Y

Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

Definition at line 119 of file gps_cnav_ephemeris.h.

8.82.4.28 double Gps_CNAV_Ephemeris::d_satpos_Z

Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

Definition at line 120 of file gps_cnav_ephemeris.h.

8.82.4.29 double Gps_CNAV_Ephemeris::d_satvel_X

Earth-fixed velocity coordinate x of the satellite [m].

Definition at line 123 of file gps_cnav_ephemeris.h.

8.82.4.30 double Gps_CNAV_Ephemeris::d_satvel_Y

Earth-fixed velocity coordinate y of the satellite [m].

Definition at line 124 of file gps_cnav_ephemeris.h.

8.82.4.31 double Gps_CNAV_Ephemeris::d_satvel_Z

Earth-fixed velocity coordinate z of the satellite [m].

Definition at line 125 of file gps_cnav_ephemeris.h.

8.82.4.32 double Gps_CNAV_Ephemeris::d_TGD

Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s].

Definition at line 88 of file gps_cnav_ephemeris.h.

8.82.4.33 double Gps_CNAV_Ephemeris::d_Toc

clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]

Definition at line 77 of file gps_cnav_ephemeris.h.

8.82.4.34 double Gps_CNAV_Ephemeris::d_Toe1

Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].

Definition at line 64 of file gps_cnav_ephemeris.h.

8.82.4.35 double Gps_CNAV_Ephemeris::d_Toe2

Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].

Definition at line 65 of file gps_cnav_ephemeris.h.

8.82.4.36 double Gps_CNAV_Ephemeris::d_Top

Data predict time of week.

Definition at line 55 of file gps_cnav_ephemeris.h.

8.82.4.37 double Gps_CNAV_Ephemeris::d_TOW

Time of GPS Week of the ephemeris set (taken from subframes TOW) [s].

Definition at line 94 of file gps_cnav_ephemeris.h.

8.82.4.38 double Gps_CNAV_Ephemeris::d_URA0

NED Accuracy Index.

Definition at line 82 of file gps_cnav_ephemeris.h.

8.82.4.39 double Gps_CNAV_Ephemeris::d_URA1

NED Accuracy Change Index.

Definition at line 83 of file gps_cnav_ephemeris.h.

8.82.4.40 double Gps_CNAV_Ephemeris::d_URA2

NED Accuracy Change Rate Index.

Definition at line 84 of file gps_cnav_ephemeris.h.

8.82.4.41 int Gps_CNAV_Ephemeris::i_GPS_week

GPS week number, aka WN [week].

Definition at line 52 of file gps_cnav_ephemeris.h.

8.82.4.42 int Gps_CNAV_Ephemeris::i_signal_health

Signal health (L1/L2/L5)

Definition at line 54 of file gps_cnav_ephemeris.h.

8.82.4.43 int Gps_CNAV_Ephemeris::i_URA

ED Accuracy Index.

Definition at line 53 of file gps_cnav_ephemeris.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_cnav_ephemeris.h

8.83 Gps_CNAV_Iono Class Reference

This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200H.

`#include <gps_cnav_iono.h>`

Public Member Functions

- Gps_CNAV_Iono ()

  Default constructor.
- template<class Archive >
  void serialize (Archive &archive, const unsigned int version)

  Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

Public Attributes

- bool valid

  Valid flag.
- double d_alpha0

  Coefficient 0 of a cubic equation representing the amplitude of the vertical delay [s].
- double d_alpha1

  Coefficient 1 of a cubic equation representing the amplitude of the vertical delay [s/semi-circle].
- double d_alpha2

  Coefficient 2 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^2].
- double d_alpha3

  Coefficient 3 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^3].
- double d_beta0

  Coefficient 0 of a cubic equation representing the period of the model [s].
- double d_beta1

  Coefficient 1 of a cubic equation representing the period of the model [s/semi-circle].
- double d_beta2

  Coefficient 2 of a cubic equation representing the period of the model [s(semi-circle)^2].
- double d_beta3

  Coefficient 3 of a cubic equation representing the period of the model [s(semi-circle)^3].

8.83.1 Detailed Description

This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200H.

See http://www.gps.gov/technical/icwg/IS-GPS-200H.pdf Appendix III

Definition at line 45 of file gps_cnav_iono.h.

8.83.2 Constructor & Destructor Documentation

8.83.2.1 Gps_CNAV_Iono::Gps_CNAV_Iono ( )

Default constructor.

8.83.3 Member Function Documentation

8.83.3.1 template<class Archive > void Gps_CNAV_Iono::serialize (Archive & *archive,* const unsigned int *version*) [inline]

Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

Definition at line 66 of file gps_cnav_iono.h.

8.83.4 Member Data Documentation

8.83.4.1 double Gps_CNAV_Iono::d_alpha0

Coefficient 0 of a cubic equation representing the amplitude of the vertical delay [s].

Definition at line 50 of file gps_cnav_iono.h.

8.83.4.2 double Gps_CNAV_Iono::d_alpha1

Coefficient 1 of a cubic equation representing the amplitude of the vertical delay [s/semi-circle].

Definition at line 51 of file gps_cnav_iono.h.

8.83.4.3 double Gps_CNAV_Iono::d_alpha2

Coefficient 2 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^2].

Definition at line 52 of file gps_cnav_iono.h.

8.83.4.4 double Gps_CNAV_Iono::d_alpha3

Coefficient 3 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^3].

Definition at line 53 of file gps_cnav_iono.h.

8.83.4.5 double Gps_CNAV_Iono::d_beta0

Coefficient 0 of a cubic equation representing the period of the model [s].

Definition at line 54 of file gps_cnav_iono.h.

8.83.4.6 double Gps_CNAV_Iono::d_beta1

Coefficient 1 of a cubic equation representing the period of the model [s/semi-circle].

Definition at line 55 of file gps_cnav_iono.h.

8.83.4.7 double Gps_CNAV_Iono::d_beta2

Coefficient 2 of a cubic equation representing the period of the model [s(semi-circle)^2].

Definition at line 56 of file gps_cnav_iono.h.

8.83.4.8 double Gps_CNAV_Iono::d_beta3

Coefficient 3 of a cubic equation representing the period of the model [s(semi-circle)^3].

Definition at line 57 of file gps_cnav_iono.h.

8.83.4.9 bool Gps_CNAV_Iono::valid

Valid flag.

Definition at line 48 of file gps_cnav_iono.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_cnav_iono.h

8.84 Gps_CNAV_Navigation_Message Class Reference

This class decodes a GPS CNAV Data message as described in IS-GPS-200H.

`#include <gps_cnav_navigation_message.h>`

Public Member Functions

- void **reset** ()
- void **decode_page** (std::vector< int > data)
- Gps_CNAV_Ephemeris get_ephemeris ()

  Obtain a GPS SV Ephemeris class filled with current SV data.
- bool have_new_iono ()

  Check if we have a new iono record stored in the galileo navigation class.
- Gps_CNAV_Iono get_iono ()

  Obtain a GPS ionospheric correction parameters class filled with current SV data.
- Gps_CNAV_Utc_Model get_utc_model ()

  Obtain a GPS UTC model parameters class filled with current SV data.
- bool have_new_ephemeris ()

  Check if we have a new ephemeris stored in the galileo navigation class.
- Gps_CNAV_Navigation_Message ()

Public Attributes

- double **d_TOW**
- bool **b_flag_ephemeris_1**
- bool **b_flag_ephemeris_2**
- bool b_flag_iono_valid

  If set, it indicates that the ionospheric parameters are filled (page 18 has arrived and decoded)
- std::map< int, std::string > satelliteBlock

  Map that stores to which block the PRN belongs http://www.navcen.uscg.gov/?Do=constellation↩Status.
- double d_satpos_X

  Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.
- double d_satpos_Y

  Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.
- double d_satpos_Z

  Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).
- int **i_channel_ID**
- unsigned int **i_satellite_PRN**
- double d_satvel_X

  Earth-fixed velocity coordinate x of the satellite [m].
- double d_satvel_Y

  Earth-fixed velocity coordinate y of the satellite [m].
- double d_satvel_Z

  Earth-fixed velocity coordinate z of the satellite [m].

8.84.1 Detailed Description

This class decodes a GPS CNAV Data message as described in IS-GPS-200H.

See http://www.gps.gov/technical/icwg/IS-GPS-200H.pdf Appendix III

Definition at line 55 of file gps_cnav_navigation_message.h.

8.84.2 Constructor & Destructor Documentation

8.84.2.1 Gps_CNAV_Navigation_Message::Gps_CNAV_Navigation_Message ( )

Default constructor

8.84.3 Member Function Documentation

8.84.3.1 Gps_CNAV_Ephemeris Gps_CNAV_Navigation_Message::get_ephemeris ( )

Obtain a GPS SV Ephemeris class filled with current SV data.

8.84.3.2 Gps_CNAV_Iono Gps_CNAV_Navigation_Message::get_iono ( )

Obtain a GPS ionospheric correction parameters class filled with current SV data.

8.84.3.3 Gps_CNAV_Utc_Model Gps_CNAV_Navigation_Message::get_utc_model ( )

Obtain a GPS UTC model parameters class filled with current SV data.

8.84.3.4 bool Gps_CNAV_Navigation_Message::have_new_ephemeris ( )

Check if we have a new ephemeris stored in the galileo navigation class.

8.84.3.5 bool Gps_CNAV_Navigation_Message::have_new_iono ( )

Check if we have a new iono record stored in the galileo navigation class.

8.84.4 Member Data Documentation

8.84.4.1 bool Gps_CNAV_Navigation_Message::b_flag_iono_valid

If set, it indicates that the ionospheric parameters are filled (page 18 has arrived and decoded)

Definition at line 71 of file gps_cnav_navigation_message.h.

8.84.4.2 double Gps_CNAV_Navigation_Message::d_satpos_X

Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

Definition at line 76 of file gps_cnav_navigation_message.h.

8.84.4.3 double Gps_CNAV_Navigation_Message::d_satpos_Y

Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

Definition at line 77 of file gps_cnav_navigation_message.h.

8.84.4.4 double Gps_CNAV_Navigation_Message::d_satpos_Z

Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

Definition at line 78 of file gps_cnav_navigation_message.h.

8.84.4.5 double Gps_CNAV_Navigation_Message::d_satvel_X

Earth-fixed velocity coordinate x of the satellite [m].

Definition at line 85 of file gps_cnav_navigation_message.h.

8.84.4.6 double Gps_CNAV_Navigation_Message::d_satvel_Y

Earth-fixed velocity coordinate y of the satellite [m].

Definition at line 86 of file gps_cnav_navigation_message.h.

8.84.4.7 double Gps_CNAV_Navigation_Message::d_satvel_Z

Earth-fixed velocity coordinate z of the satellite [m].

Definition at line 87 of file gps_cnav_navigation_message.h.

8.84.4.8 std::map<int,std::string> Gps_CNAV_Navigation_Message::satelliteBlock

Map that stores to which block the PRN belongs http://www.navcen.uscg.gov/?Do=constellation↩Status.

Definition at line 73 of file gps_cnav_navigation_message.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_cnav_navigation_message.h

8.85 Gps_CNAV_Utc_Model Class Reference

This class is a storage for the GPS UTC MODEL data as described in in IS-GPS-200H.

`#include <gps_cnav_utc_model.h>`

Public Member Functions

- Gps_CNAV_Utc_Model ()
- template<class Archive >
  void **serialize** (Archive &archive, const unsigned int version)

Public Attributes

- bool **valid**
- double d_A1

  1st order term of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200H) [s/s]
- double d_A0

  Constant of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200H) [s].
- double d_t_OT

  Reference time for UTC data (reference 20.3.4.5 and 20.3.3.5.2.4 IS-GPS-200H) [s].
- int i_WN_T

  UTC reference week number [weeks].
- double d_DeltaT_LS

  delta time due to leap seconds [s]. Number of leap seconds since 6-Jan-1980 as transmitted by the GPS almanac.
- int i_WN_LSF

  Week number at the end of which the leap second becomes effective [weeks].
- int i_DN

  Day number (DN) at the end of which the leap second becomes effective [days].
- double d_DeltaT_LSF

  Scheduled future or recent past (relative to NAV message upload) value of the delta time due to leap seconds [s].

8.85.1 Detailed Description

This class is a storage for the GPS UTC MODEL data as described in in IS-GPS-200H.

See http://www.gps.gov/technical/icwg/IS-GPS-200H.pdf Appendix III

Definition at line 44 of file gps_cnav_utc_model.h.

8.85.2 Constructor & Destructor Documentation

8.85.2.1 Gps_CNAV_Utc_Model::Gps_CNAV_Utc_Model ( )

Default constructor

8.85.3 Member Data Documentation

8.85.3.1 double Gps_CNAV_Utc_Model::d_A0

Constant of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200H) [s].

Definition at line 50 of file gps_cnav_utc_model.h.

8.85.3.2 double Gps_CNAV_Utc_Model::d_A1

1st order term of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200H) [s/s]

Definition at line 49 of file gps_cnav_utc_model.h.

8.85.3.3 double Gps_CNAV_Utc_Model::d_DeltaT_LS

delta time due to leap seconds [s]. Number of leap seconds since 6-Jan-1980 as transmitted by the GPS almanac.

Definition at line 53 of file gps_cnav_utc_model.h.

8.85.3.4 double Gps_CNAV_Utc_Model::d_DeltaT_LSF

Scheduled future or recent past (relative to NAV message upload) value of the delta time due to leap seconds [s].

Definition at line 56 of file gps_cnav_utc_model.h.

8.85.3.5 double Gps_CNAV_Utc_Model::d_t_OT

Reference time for UTC data (reference 20.3.4.5 and 20.3.3.5.2.4 IS-GPS-200H) [s].

Definition at line 51 of file gps_cnav_utc_model.h.

8.85.3.6 int Gps_CNAV_Utc_Model::i_DN

Day number (DN) at the end of which the leap second becomes effective [days].

Definition at line 55 of file gps_cnav_utc_model.h.

8.85.3.7 int Gps_CNAV_Utc_Model::i_WN_LSF

Week number at the end of which the leap second becomes effective [weeks].

Definition at line 54 of file gps_cnav_utc_model.h.

8.85.3.8 int Gps_CNAV_Utc_Model::i_WN_T

UTC reference week number [weeks].

Definition at line 52 of file gps_cnav_utc_model.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_cnav_utc_model.h

8.86 Gps_Ephemeris Class Reference

This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200E.

`#include <gps_ephemeris.h>`

Public Member Functions

- template<class Archive >
  void serialize (Archive &archive, const unsigned int version)

  Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.
- double satellitePosition (double transmitTime)

  Compute the ECEF SV coordinates and ECEF velocity Implementation of Table 20-IV (IS-GPS-200E) and compute the clock bias term including relativistic effect (return value)
- double sv_clock_drift (double transmitTime)

  Sets (d_satClkDrift)and returns the clock drift in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)
- double sv_clock_relativistic_term (double transmitTime)

  Sets (d_dtr) and returns the clock relativistic correction term in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)
- Gps_Ephemeris ()

Public Attributes

- unsigned int **i_satellite_PRN**
- double d_TOW

  Time of GPS Week of the ephemeris set (taken from subframes TOW) [s].
- double d_Crs

  Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m].
- double d_Delta_n

  Mean Motion Difference From Computed Value [semi-circles/s].
- double d_M_0

  Mean Anomaly at Reference Time [semi-circles].
- double d_Cuc

  Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad].
- double d_e_eccentricity

  Eccentricity [dimensionless].
- double d_Cus

  Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad].
- double d_sqrt_A

  Square Root of the Semi-Major Axis [sqrt(m)].
- double d_Toe

  Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].
- double d_Toc

  clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]
- double d_Cic

  Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad].
- double d_OMEGA0

  Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles].

- double d_Cis

  Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad].
- double d_i_0

  Inclination Angle at Reference Time [semi-circles].
- double d_Crc

  Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m].
- double d_OMEGA

  Argument of Perigee [semi-cicles].
- double d_OMEGA_DOT

  Rate of Right Ascension [semi-circles/s].
- double d_IDOT

  Rate of Inclination Angle [semi-circles/s].
- int i_code_on_L2

  If 1, P code ON in L2; if 2, C/A code ON in L2;.
- int i_GPS_week

  GPS week number, aka WN [week].
- bool b_L2_P_data_flag

  When true, indicates that the NAV data stream was commanded OFF on the P-code of the L2 channel.
- int i_SV_accuracy

  User Range Accuracy (URA) index of the SV (reference paragraph 6.2.1) for the standard positioning service user (Ref 20.3.3.3.1.3 IS-GPS-200E)
- int **i_SV_health**
- double d_TGD

  Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s].
- double d_IODC

  Issue of Data, Clock.
- double d_IODE_SF2

  Issue of Data, Ephemeris (IODE), subframe 2.
- double d_IODE_SF3

  Issue of Data, Ephemeris(IODE), subframe 3.
- int i_AODO

  Age of Data Offset (AODO) term for the navigation message correction table (NMCT) contained in subframe 4 (reference paragraph 20.3.3.5.1.9) [s].
- bool b_fit_interval_flag

  indicates the curve-fit interval used by the CS (Block II/IIA/IIR/IIR-M/IIF) and SS (Block IIIA) in determining the ephemeris parameters, as follows: 0 = 4 hours, 1 = greater than 4 hours.
- double **d_spare1**
- double **d_spare2**
- double d_A_f0

  Coefficient 0 of code phase offset model [s].
- double d_A_f1

  Coefficient 1 of code phase offset model [s/s].
- double d_A_f2

  Coefficient 2 of code phase offset model [s/s^2].
- bool b_integrity_status_flag

  If true, enhanced level of integrity assurance.
- bool b_alert_flag

  If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.
- bool b_antispoofing_flag

  If true, the AntiSpoofing mode is ON in that SV.
- double d_satClkDrift

GPS clock error.

- double d_dtr

  relativistic clock correction term

- double d_satpos_X

  Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

- double d_satpos_Y

  Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

- double d_satpos_Z

  Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

- double d_satvel_X

  Earth-fixed velocity coordinate x of the satellite [m].

- double d_satvel_Y

  Earth-fixed velocity coordinate y of the satellite [m].

- double d_satvel_Z

  Earth-fixed velocity coordinate z of the satellite [m].

- std::map< int, std::string > satelliteBlock

  Map that stores to which block the PRN belongs http://www.navcen.uscg.gov/?Do=constellation↩Status.

8.86.1 Detailed Description

This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200E.

See http://www.gps.gov/technical/icwg/IS-GPS-200E.pdf Appendix II

Definition at line 48 of file gps_ephemeris.h.

8.86.2 Constructor & Destructor Documentation

8.86.2.1 Gps_Ephemeris::Gps_Ephemeris ( )

Default constructor

Referenced by serialize().

8.86.3 Member Function Documentation

8.86.3.1 double Gps_Ephemeris::satellitePosition (double *transmitTime*)

Compute the ECEF SV coordinates and ECEF velocity Implementation of Table 20-IV (IS-GPS-200E) and compute the clock bias term including relativistic effect (return value)

Referenced by serialize().

8.86.3.2 template<class Archive > void Gps_Ephemeris::serialize (Archive & *archive,* const unsigned int *version*) [inline]

Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

< Time of GPS Week of the ephemeris set (taken from subframes TOW) [s]

< Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m]

< Mean Motion Difference From Computed Value [semi-circles/s]

< Mean Anomaly at Reference Time [semi-circles]

< Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad]

< Eccentricity [dimensionless]

< Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad]

< Square Root of the Semi-Major Axis [sqrt(m)]

< Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s]

< clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]

< Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad]

< Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles]

< Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad]

< Inclination Angle at Reference Time [semi-circles]

< Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m]

< Argument of Perigee [semi-cicles]

< Rate of Right Ascension [semi-circles/s]

< Rate of Inclination Angle [semi-circles/s]

< If 1, P code ON in L2; if 2, C/A code ON in L2;

< GPS week number, aka WN [week]

< When true, indicates that the NAV data stream was commanded OFF on the P-code of the L2 channel

< User Range Accuracy (URA) index of the SV (reference paragraph 6.2.1) for the standard positioning service user (Ref 20.3.3.3.1.3 IS-GPS-200E)

< Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s]

< Issue of Data, Clock

< Age of Data Offset (AODO) term for the navigation message correction table (NMCT) contained in subframe 4 (reference paragraph 20.3.3.5.1.9) [s]

< indicates the curve-fit interval used by the CS (Block II/IIA/IIR/IIR-M/IIF) and SS (Block IIIA) in determining the ephemeris parameters, as follows: 0 = 4 hours, 1 = greater than 4 hours.

< Coefficient 0 of code phase offset model [s]

< Coefficient 1 of code phase offset model [s/s]

< Coefficient 2 of code phase offset model [s/s^2]

< If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.

< If true, the AntiSpoofing mode is ON in that SV

Definition at line 135 of file gps_ephemeris.h.

References Gps_Ephemeris(), satellitePosition(), sv_clock_drift(), and sv_clock_relativistic_term().

8.86.3.3 double Gps_Ephemeris::sv_clock_drift (double *transmitTime*)

Sets (*d_satClkDrift*)and returns the clock drift in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)

Referenced by serialize().

8.86.3.4 double Gps_Ephemeris::sv_clock_relativistic_term (double *transmitTime*)

Sets (*d_dtr*) and returns the clock relativistic correction term in seconds according to the User Algorithm for SV Clock Correction (IS-GPS-200E, 20.3.3.3.3.1)

Referenced by serialize().

8.86.4 Member Data Documentation

8.86.4.1 bool Gps_Ephemeris::b_alert_flag

If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.

Definition at line 111 of file gps_ephemeris.h.

8.86.4.2 bool Gps_Ephemeris::b_antispoofing_flag

If true, the AntiSpoofing mode is ON in that SV.

Definition at line 112 of file gps_ephemeris.h.

8.86.4.3 bool Gps_Ephemeris::b_fit_interval_flag

indicates the curve-fit interval used by the CS (Block II/IIA/IIR/IIR-M/IIF) and SS (Block IIIA) in determining the ephemeris parameters, as follows: 0 = 4 hours, 1 = greater than 4 hours.

Definition at line 90 of file gps_ephemeris.h.

8.86.4.4 bool Gps_Ephemeris::b_integrity_status_flag

If true, enhanced level of integrity assurance.

If false, indicates that the conveying signal is provided with the legacy level of integrity assurance. That is, the probability that the instantaneous URE of the conveying signal exceeds 4.42 times the upper bound value of the current broadcast URA index, for more than 5.2 seconds, without an accompanying alert, is less than 1E-5 per hour. If true, indicates that the conveying signal is provided with an enhanced level of integrity assurance. That is, the probability that the instantaneous URE of the conveying signal exceeds 5.73 times the upper bound value of the current broadcast URA index, for more than 5.2 seconds, without an accompanying alert, is less than 1E-8 per hour.

Definition at line 110 of file gps_ephemeris.h.

8.86.4.5 bool Gps_Ephemeris::b_L2_P_data_flag

When true, indicates that the NAV data stream was commanded OFF on the P-code of the L2 channel.

Definition at line 81 of file gps_ephemeris.h.

8.86.4.6 double Gps_Ephemeris::d_A_f0

Coefficient 0 of code phase offset model [s].

Definition at line 94 of file gps_ephemeris.h.

8.86.4.7 double Gps_Ephemeris::d_A_f1

Coefficient 1 of code phase offset model [s/s].

Definition at line 95 of file gps_ephemeris.h.

8.86.4.8 double Gps_Ephemeris::d_A_f2

Coefficient 2 of code phase offset model [s/s^2].

Definition at line 96 of file gps_ephemeris.h.

8.86.4.9 double Gps_Ephemeris::d_Cic

Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad].

Definition at line 71 of file gps_ephemeris.h.

8.86.4.10 double Gps_Ephemeris::d_Cis

Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad].

Definition at line 73 of file gps_ephemeris.h.

8.86.4.11 double Gps_Ephemeris::d_Crc

Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m].

Definition at line 75 of file gps_ephemeris.h.

8.86.4.12 double Gps_Ephemeris::d_Crs

Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m].

Definition at line 62 of file gps_ephemeris.h.

8.86.4.13 double Gps_Ephemeris::d_Cuc

Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad].

Definition at line 65 of file gps_ephemeris.h.

8.86.4.14 double Gps_Ephemeris::d_Cus

Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad].

Definition at line 67 of file gps_ephemeris.h.

8.86.4.15 double Gps_Ephemeris::d_Delta_n

Mean Motion Difference From Computed Value [semi-circles/s].

Definition at line 63 of file gps_ephemeris.h.

8.86.4.16 double Gps_Ephemeris::d_dtr

relativistic clock correction term

Definition at line 116 of file gps_ephemeris.h.

8.86.4.17 double Gps_Ephemeris::d_e_eccentricity

Eccentricity [dimensionless].

Definition at line 66 of file gps_ephemeris.h.

8.86.4.18 double Gps_Ephemeris::d_i_0

Inclination Angle at Reference Time [semi-circles].

Definition at line 74 of file gps_ephemeris.h.

8.86.4.19 double Gps_Ephemeris::d_IDOT

Rate of Inclination Angle [semi-circles/s].

Definition at line 78 of file gps_ephemeris.h.

8.86.4.20 double Gps_Ephemeris::d_IODC

Issue of Data, Clock.

Definition at line 85 of file gps_ephemeris.h.

8.86.4.21 double Gps_Ephemeris::d_IODE_SF2

Issue of Data, Ephemeris (IODE), subframe 2.

Definition at line 86 of file gps_ephemeris.h.

8.86.4.22 double Gps_Ephemeris::d_IODE_SF3

Issue of Data, Ephemeris(IODE), subframe 3.

Definition at line 87 of file gps_ephemeris.h.

8.86.4.23 double Gps_Ephemeris::d_M_0

Mean Anomaly at Reference Time [semi-circles].

Definition at line 64 of file gps_ephemeris.h.

8.86.4.24 double Gps_Ephemeris::d_OMEGA

Argument of Perigee [semi-cicles].

Definition at line 76 of file gps_ephemeris.h.

8.86.4.25 double Gps_Ephemeris::d_OMEGA0

Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles].

Definition at line 72 of file gps_ephemeris.h.

8.86.4.26 double Gps_Ephemeris::d_OMEGA_DOT

Rate of Right Ascension [semi-circles/s].

Definition at line 77 of file gps_ephemeris.h.

8.86.4.27 double Gps_Ephemeris::d_satClkDrift

GPS clock error.

Definition at line 115 of file gps_ephemeris.h.

8.86.4.28 double Gps_Ephemeris::d_satpos_X

Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

Definition at line 119 of file gps_ephemeris.h.

8.86.4.29 double Gps_Ephemeris::d_satpos_Y

Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

Definition at line 120 of file gps_ephemeris.h.

8.86.4.30 double Gps_Ephemeris::d_satpos_Z

Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

Definition at line 121 of file gps_ephemeris.h.

8.86.4.31 double Gps_Ephemeris::d_satvel_X

Earth-fixed velocity coordinate x of the satellite [m].

Definition at line 124 of file gps_ephemeris.h.

8.86.4.32 double Gps_Ephemeris::d_satvel_Y

Earth-fixed velocity coordinate y of the satellite [m].

Definition at line 125 of file gps_ephemeris.h.

8.86.4.33 double Gps_Ephemeris::d_satvel_Z

Earth-fixed velocity coordinate z of the satellite [m].

Definition at line 126 of file gps_ephemeris.h.

8.86.4.34 double Gps_Ephemeris::d_sqrt_A

Square Root of the Semi-Major Axis [sqrt(m)].

Definition at line 68 of file gps_ephemeris.h.

8.86.4.35 double Gps_Ephemeris::d_TGD

Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s].

Definition at line 84 of file gps_ephemeris.h.

8.86.4.36 double Gps_Ephemeris::d_Toc

clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]

Definition at line 70 of file gps_ephemeris.h.

8.86.4.37 double Gps_Ephemeris::d_Toe

Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].

Definition at line 69 of file gps_ephemeris.h.

8.86.4.38 double Gps_Ephemeris::d_TOW

Time of GPS Week of the ephemeris set (taken from subframes TOW) [s].

Definition at line 61 of file gps_ephemeris.h.

8.86.4.39 int Gps_Ephemeris::i_AODO

Age of Data Offset (AODO) term for the navigation message correction table (NMCT) contained in subframe 4 (reference paragraph 20.3.3.5.1.9) [s].

Definition at line 88 of file gps_ephemeris.h.

8.86.4.40 int Gps_Ephemeris::i_code_on_L2

If 1, P code ON in L2; if 2, C/A code ON in L2;.

Definition at line 79 of file gps_ephemeris.h.

8.86.4.41 int Gps_Ephemeris::i_GPS_week

GPS week number, aka WN [week].

Definition at line 80 of file gps_ephemeris.h.

8.86.4.42 int Gps_Ephemeris::i_SV_accuracy

User Range Accuracy (URA) index of the SV (reference paragraph 6.2.1) for the standard positioning service user (Ref 20.3.3.3.1.3 IS-GPS-200E)

Definition at line 82 of file gps_ephemeris.h.

8.86.4.43 std::map<int,std::string> Gps_Ephemeris::satelliteBlock

Map that stores to which block the PRN belongs http://www.navcen.uscg.gov/?Do=constellation↩Status.

Definition at line 128 of file gps_ephemeris.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_ephemeris.h

8.87 Gps_Iono Class Reference

This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200E.

`#include <gps_iono.h>`

Public Member Functions

- Gps_Iono ()

  Default constructor.
- template<class Archive >
  void serialize (Archive &archive, const unsigned int version)

  Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

Public Attributes

- bool valid

  Valid flag.
- double d_alpha0

  Coefficient 0 of a cubic equation representing the amplitude of the vertical delay [s].
- double d_alpha1

  Coefficient 1 of a cubic equation representing the amplitude of the vertical delay [s/semi-circle].
- double d_alpha2

  Coefficient 2 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^2].
- double d_alpha3

  Coefficient 3 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^3].
- double d_beta0

  Coefficient 0 of a cubic equation representing the period of the model [s].
- double d_beta1

  Coefficient 1 of a cubic equation representing the period of the model [s/semi-circle].
- double d_beta2

  Coefficient 2 of a cubic equation representing the period of the model [s(semi-circle)^2].
- double d_beta3

  Coefficient 3 of a cubic equation representing the period of the model [s(semi-circle)^3].

8.87.1 Detailed Description

This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200E.

See http://www.gps.gov/technical/icwg/IS-GPS-200E.pdf Appendix II

Definition at line 45 of file gps_iono.h.

8.87.2 Constructor & Destructor Documentation

8.87.2.1 Gps_Iono::Gps_Iono ( )

Default constructor.

8.87.3 Member Function Documentation

8.87.3.1 template<class Archive > void Gps_Iono::serialize (Archive & *archive,* const unsigned int *version*) [inline]

Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ephemeris data on disk file.

Definition at line 66 of file gps_iono.h.

8.87.4 Member Data Documentation

8.87.4.1 double Gps_Iono::d_alpha0

Coefficient 0 of a cubic equation representing the amplitude of the vertical delay [s].

Definition at line 50 of file gps_iono.h.

8.87.4.2 double Gps_Iono::d_alpha1

Coefficient 1 of a cubic equation representing the amplitude of the vertical delay [s/semi-circle].

Definition at line 51 of file gps_iono.h.

8.87.4.3 double Gps_Iono::d_alpha2

Coefficient 2 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^2].

Definition at line 52 of file gps_iono.h.

8.87.4.4 double Gps_Iono::d_alpha3

Coefficient 3 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^3].

Definition at line 53 of file gps_iono.h.

8.87.4.5 double Gps_Iono::d_beta0

Coefficient 0 of a cubic equation representing the period of the model [s].

Definition at line 54 of file gps_iono.h.

8.87.4.6 double Gps_Iono::d_beta1

Coefficient 1 of a cubic equation representing the period of the model [s/semi-circle].

Definition at line 55 of file gps_iono.h.

8.87.4.7 double Gps_Iono::d_beta2

Coefficient 2 of a cubic equation representing the period of the model [s(semi-circle)^2].

Definition at line 56 of file gps_iono.h.

8.87.4.8 double Gps_Iono::d_beta3

Coefficient 3 of a cubic equation representing the period of the model [s(semi-circle)^3].

Definition at line 57 of file gps_iono.h.

8.87.4.9 bool Gps_Iono::valid

Valid flag.

Definition at line 48 of file gps_iono.h.

The documentation for this class was generated from the following file:

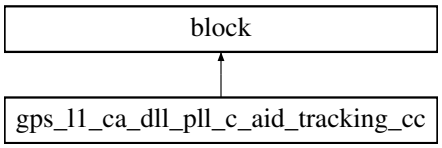
- src/core/system_parameters/gps_iono.h

8.88 gps_l1_ca_dll_pll_c_aid_tracking_cc Class Reference

This class implements a DLL + PLL tracking loop block.

`#include <gps_l1_ca_dll_pll_c_aid_tracking_cc.h>`

Inheritance diagram for gps_l1_ca_dll_pll_c_aid_tracking_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- gps_l1_ca_dll_pll_c_aid_tracking_cc_sptr **gps_l1_ca_dll_pll_c_aid_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float pll_bw_narrow_hz, float dll_bw_narrow_hz, int extend_correlation_ms, float early_late_space_chips)

8.88.1 Detailed Description

This class implements a DLL + PLL tracking loop block.

Definition at line 75 of file gps_l1_ca_dll_pll_c_aid_tracking_cc.h.

The documentation for this class was generated from the following file:

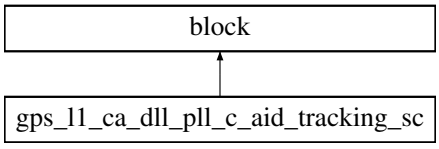
- src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_cc.h

8.89 gps_l1_ca_dll_pll_c_aid_tracking_sc Class Reference

This class implements a DLL + PLL tracking loop block.

`#include <gps_l1_ca_dll_pll_c_aid_tracking_sc.h>`

Inheritance diagram for gps_l1_ca_dll_pll_c_aid_tracking_sc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- gps_l1_ca_dll_pll_c_aid_tracking_sc_sptr **gps_l1_ca_dll_pll_c_aid_make_tracking_sc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float pll_bw_narrow_hz, float dll_bw_narrow_hz, int extend_correlation_ms, float early_late_space_chips)

8.89.1 Detailed Description

This class implements a DLL + PLL tracking loop block.

Definition at line 76 of file gps_l1_ca_dll_pll_c_aid_tracking_sc.h.

The documentation for this class was generated from the following file:

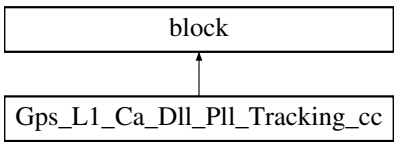
- src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_sc.h

8.90 Gps_L1_Ca_Dll_Pll_Tracking_cc Class Reference

This class implements a DLL + PLL tracking loop block.

`#include <gps_l1_ca_dll_pll_tracking_cc.h>`

Inheritance diagram for Gps_L1_Ca_Dll_Pll_Tracking_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- gps_l1_ca_dll_pll_tracking_cc_sptr **gps_l1_ca_dll_pll_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips)

8.90.1 Detailed Description

This class implements a DLL + PLL tracking loop block.

Definition at line 69 of file gps_l1_ca_dll_pll_tracking_cc.h.

The documentation for this class was generated from the following file:

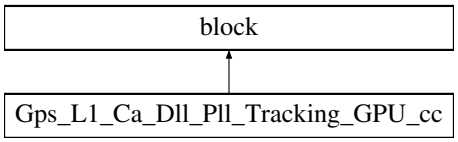
- src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_cc.h

8.91 Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc Class Reference

This class implements a DLL + PLL tracking loop block.

`#include <gps_l1_ca_dll_pll_tracking_gpu_cc.h>`

Inheritance diagram for Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- gps_l1_ca_dll_pll_tracking_gpu_cc_sptr **gps_l1_ca_dll_pll_make_tracking_gpu_cc** (long if_freq, long fs↩_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips)

8.91.1 Detailed Description

This class implements a DLL + PLL tracking loop block.

Definition at line 68 of file gps_l1_ca_dll_pll_tracking_gpu_cc.h.

The documentation for this class was generated from the following file:

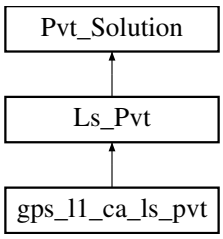
- src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_gpu_cc.h

8.92 gps_l1_ca_ls_pvt Class Reference

This class implements a simple PVT Least Squares solution for GPS L1 C/A signals.

`#include <gps_l1_ca_ls_pvt.h>`

Inheritance diagram for gps_l1_ca_ls_pvt:



Public Member Functions

- **gps_l1_ca_ls_pvt** (int nchannels, std::string dump_filename, bool flag_dump_to_file)
- bool **get_PVT** (std::map< int, Gnss_Synchro > gnss_pseudoranges_map, double GPS_current_time, bool flag_averaging)

Public Attributes

- int d_nchannels

  Number of available channels for positioning.
- Gps_Navigation_Message * **d_ephemeris**
- std::map< int, Gps_Ephemeris > gps_ephemeris_map

  Map storing new Gps_Ephemeris.
- Gps_Utc_Model **gps_utc_model**
- Gps_Iono **gps_iono**
- Sbas_Ionosphere_Correction **sbas_iono**
- std::map< int, Sbas_Satellite_Correction > **sbas_sat_corr_map**
- std::map< int, Sbas_Ephemeris > **sbas_ephemeris_map**
- double **d_GPS_current_time**
- bool **d_flag_dump_enabled**
- bool **d_flag_averaging**
- std::string **d_dump_filename**
- std::ofstream **d_dump_file**

8.92.1 Detailed Description

This class implements a simple PVT Least Squares solution for GPS L1 C/A signals.

Definition at line 52 of file gps_l1_ca_ls_pvt.h.

8.92.2 Member Data Documentation

8.92.2.1 int gps_l1_ca_ls_pvt::d_nchannels

Number of available channels for positioning.

Definition at line 59 of file gps_l1_ca_ls_pvt.h.

8.92.2.2 std::map<int,Gps_Ephemeris> gps_l1_ca_ls_pvt::gps_ephemeris_map

Map storing new Gps_Ephemeris.

Definition at line 64 of file gps_l1_ca_ls_pvt.h.

The documentation for this class was generated from the following file:

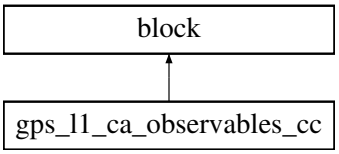
- src/algorithms/PVT/libs/gps_l1_ca_ls_pvt.h

8.93 gps_l1_ca_observables_cc Class Reference

This class implements a block that computes GPS L1 C/A observables.

`#include <gps_l1_ca_observables_cc.h>`

Inheritance diagram for gps_l1_ca_observables_cc:



Public Member Functions

- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- gps_l1_ca_observables_cc_sptr **gps_l1_ca_make_observables_cc** (unsigned int nchannels, bool dump, std::string dump_filename, unsigned int deep_history)

8.93.1 Detailed Description

This class implements a block that computes GPS L1 C/A observables.

Definition at line 52 of file gps_l1_ca_observables_cc.h.

The documentation for this class was generated from the following file:

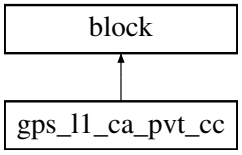
- src/algorithms/observables/gnuradio_blocks/gps_l1_ca_observables_cc.h

8.94 gps_l1_ca_pvt_cc Class Reference

This class implements a block that computes the PVT solution.

`#include <gps_l1_ca_pvt_cc.h>`

Inheritance diagram for gps_l1_ca_pvt_cc:



Public Member Functions

- std::map< int, Gps_Ephemeris > get_GPS_L1_ephemeris_map ()

  Get latest set of GPS L1 ephemeris from PVT block.
- ~gps_l1_ca_pvt_cc ()

  Default destructor.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  PVT Signal Processing.

Friends

- gps_l1_ca_pvt_cc_sptr **gps_l1_ca_make_pvt_cc** (unsigned int nchannels, bool dump, std::string dump↩ _filename, int averaging_depth, bool flag_averaging, int output_rate_ms, int display_rate_ms, bool flag_↩ nmea_tty_port, std::string nmea_dump_filename, std::string nmea_dump_devname, bool flag_rtcm_server, bool flag_rtcm_tty_port, unsigned short rtcm_tcp_port, unsigned short rtcm_station_id, std::map< int, int > rtcm_msg_rate_ms, std::string rtcm_dump_devname, int rinex_version)

8.94.1 Detailed Description

This class implements a block that computes the PVT solution.

Definition at line 73 of file gps_l1_ca_pvt_cc.h.

8.94.2 Constructor & Destructor Documentation

8.94.2.1 gps_l1_ca_pvt_cc::~gps_l1_ca_pvt_cc ( )

Default destructor.

8.94.3 Member Function Documentation

8.94.3.1 int gps_l1_ca_pvt_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

PVT Signal Processing.

8.94.3.2 std::map<int,Gps_Ephemeris> gps_l1_ca_pvt_cc::get_GPS_L1_ephemeris_map ( )

Get latest set of GPS L1 ephemeris from PVT block.

It is used to save the assistance data at the receiver shutdown

The documentation for this class was generated from the following file:

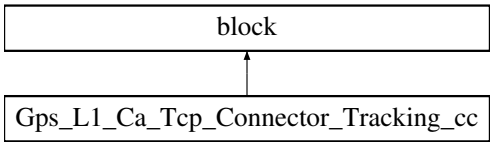
- src/algorithms/PVT/gnuradio_blocks/gps_l1_ca_pvt_cc.h

8.95 Gps_L1_Ca_Tcp_Connector_Tracking_cc Class Reference

This class implements a DLL + PLL tracking loop block.

`#include <gps_l1_ca_tcp_connector_tracking_cc.h>`

Inheritance diagram for Gps_L1_Ca_Tcp_Connector_Tracking_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- gps_l1_ca_tcp_connector_tracking_cc_sptr **gps_l1_ca_tcp_connector_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float early_late_space_chips, size_t port_ch0)

8.95.1 Detailed Description

This class implements a DLL + PLL tracking loop block.

Definition at line 67 of file gps_l1_ca_tcp_connector_tracking_cc.h.

The documentation for this class was generated from the following file:

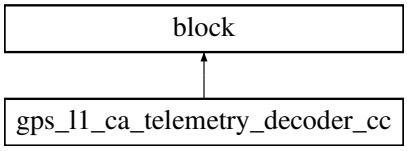
- src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_tcp_connector_tracking_cc.h

8.96 gps_l1_ca_telemetry_decoder_cc Class Reference

This class implements a block that decodes the NAV data defined in IS-GPS-200E.

`#include <gps_l1_ca_telemetry_decoder_cc.h>`

Inheritance diagram for gps_l1_ca_telemetry_decoder_cc:



Public Member Functions

- void set_satellite (Gnss_Satellite satellite)

  Set satellite PRN.
- void set_channel (int channel)

  Set receiver's channel.
- void set_decimation (int decimation)

  Set decimation factor to average the GPS synchronization estimation output from the tracking module.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  This is where all signal processing takes place.
- void forecast (int noutput_items, gr_vector_int &ninput_items_required)

  Function which tells the scheduler how many input items are required to produce noutput_items output items.

Friends

- gps_l1_ca_telemetry_decoder_cc_sptr **gps_l1_ca_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

8.96.1 Detailed Description

This class implements a block that decodes the NAV data defined in IS-GPS-200E.

Definition at line 56 of file gps_l1_ca_telemetry_decoder_cc.h.

8.96.2 Member Function Documentation

8.96.2.1 void gps_l1_ca_telemetry_decoder_cc::forecast (int *noutput_items,* gr_vector_int & *ninput_items_required*)

Function which tells the scheduler how many input items are required to produce noutput_items output items.

8.96.2.2 int gps_l1_ca_telemetry_decoder_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

This is where all signal processing takes place.

8.96.2.3 void gps_l1_ca_telemetry_decoder_cc::set_channel (int *channel*)

Set receiver's channel.

8.96.2.4 void gps_l1_ca_telemetry_decoder_cc::set_decimation (int *decimation*)

Set decimation factor to average the GPS synchronization estimation output from the tracking module.

8.96.2.5 void gps_l1_ca_telemetry_decoder_cc::set_satellite (Gnss_Satellite *satellite*)

Set satellite PRN.

The documentation for this class was generated from the following file:

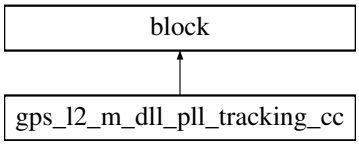
- src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l1_ca_telemetry_decoder_cc.h

8.97 gps_l2_m_dll_pll_tracking_cc Class Reference

This class implements a DLL + PLL tracking loop block.

```
#include <gps_l2_m_dll_pll_tracking_cc.h>
```

Inheritance diagram for gps_l2_m_dll_pll_tracking_cc:



Public Member Functions

- void **set_channel** (unsigned int channel)
- void **set_gnss_synchro** (Gnss_Synchro *p_gnss_synchro)
- void **start_tracking** ()
- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- gps_l2_m_dll_pll_tracking_cc_sptr **gps_l2_m_dll_pll_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_↩ space_chips)

8.97.1 Detailed Description

This class implements a DLL + PLL tracking loop block.

Definition at line 68 of file gps_l2_m_dll_pll_tracking_cc.h.

The documentation for this class was generated from the following file:

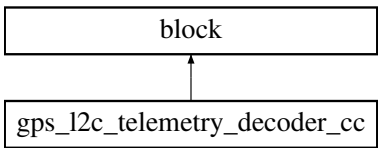
- src/algorithms/tracking/gnuradio_blocks/gps_l2_m_dll_pll_tracking_cc.h

8.98 gps_l2c_telemetry_decoder_cc Class Reference

This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229.

`#include <gps_l2c_telemetry_decoder_cc.h>`

Inheritance diagram for gps_l2c_telemetry_decoder_cc:



Public Member Functions

- void set_satellite (Gnss_Satellite satellite)

  Set satellite PRN.
- void set_channel (int channel)

  Set receiver's channel.
- void **set_decimation** (int decimation)
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  This is where all signal processing takes place.
- void forecast (int noutput_items, gr_vector_int &ninput_items_required)

  Function which tells the scheduler how many input items are required to produce noutput_items output items.

Friends

- gps_l2c_telemetry_decoder_cc_sptr **gps_l2c_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

8.98.1 Detailed Description

This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229.

Definition at line 61 of file gps_l2c_telemetry_decoder_cc.h.

8.98.2 Member Function Documentation

8.98.2.1 void gps_l2c_telemetry_decoder_cc::forecast (int *noutput_items,* gr_vector_int & *ninput_items_required*)

Function which tells the scheduler how many input items are required to produce noutput_items output items.

8.98.2.2 int gps_l2c_telemetry_decoder_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

This is where all signal processing takes place.

8.98.2.3 void gps_l2c_telemetry_decoder_cc::set_channel (int *channel*)

Set receiver's channel.

8.98.2.4 void gps_l2c_telemetry_decoder_cc::set_satellite (Gnss_Satellite *satellite*)

Set satellite PRN.

The documentation for this class was generated from the following file:

- src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l2c_telemetry_decoder_cc.h

8.99 Gps_Navigation_Message Class Reference

This class decodes a GPS NAV Data message as described in IS-GPS-200E.

```
#include <gps_navigation_message.h>
```

Public Member Functions

- void **reset** ()
- Gps_Ephemeris get_ephemeris ()

  Obtain a GPS SV Ephemeris class filled with current SV data.
- Gps_Iono get_iono ()

  Obtain a GPS ionospheric correction parameters class filled with current SV data.
- Gps_Utc_Model get_utc_model ()

  Obtain a GPS UTC model parameters class filled with current SV data.
- int subframe_decoder (char *subframe)

  Decodes the GPS NAV message.
- void satellitePosition (double transmitTime)

  Computes the position of the satellite.
- double sv_clock_correction (double transmitTime)

  Sets (d_satClkCorr) according to the User Algorithm for SV Clock Correction and returns the corrected clock (IS-GPS-200E, 20.3.3.3.3.1)
- double utc_time (const double gpstime_corrected) const

  Computes the Coordinated Universal Time (UTC) and returns it in s
- bool **satellite_validation** ()
- Gps_Navigation_Message ()

Public Attributes

- bool **b_valid_ephemeris_set_flag**
- double d_TOW

  Time of GPS Week of the ephemeris set (taken from subframes TOW) [s].
- double d_TOW_SF1

  Time of GPS Week from HOW word of Subframe 1 [s].
- double d_TOW_SF2

  Time of GPS Week from HOW word of Subframe 2 [s].
- double d_TOW_SF3

  Time of GPS Week from HOW word of Subframe 3 [s].
- double d_TOW_SF4

  Time of GPS Week from HOW word of Subframe 4 [s].
- double d_TOW_SF5

  Time of GPS Week from HOW word of Subframe 5 [s].
- double **d_IODE_SF2**
- double **d_IODE_SF3**
- double d_Crs

  Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m].
- double d_Delta_n

  Mean Motion Difference From Computed Value [semi-circles/s].
- double d_M_0

  Mean Anomaly at Reference Time [semi-circles].
- double d_Cuc

  Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad].
- double d_e_eccentricity

  Eccentricity [dimensionless].
- double d_Cus

  Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad].
- double d_sqrt_A

Square Root of the Semi-Major Axis [sqrt(m)].

- double d_Toe

  Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].
- double d_Toc

  clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]
- double d_Cic

  Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad].
- double d_OMEGA0

  Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles].
- double d_Cis

  Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad].
- double d_i_0

  Inclination Angle at Reference Time [semi-circles].
- double d_Crc

  Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m].
- double d_OMEGA

  Argument of Perigee [semi-cicles].
- double d_OMEGA_DOT

  Rate of Right Ascension [semi-circles/s].
- double d_IDOT

  Rate of Inclination Angle [semi-circles/s].
- int i_code_on_L2

  If 1, P code ON in L2; if 2, C/A code ON in L2;.
- int i_GPS_week

  GPS week number, aka WN [week].
- bool b_L2_P_data_flag

  When true, indicates that the NAV data stream was commanded OFF on the P-code of the L2 channel.
- int i_SV_accuracy

  User Range Accuracy (URA) index of the SV (reference paragraph 6.2.1) for the standard positioning service user (Ref 20.3.3.3.1.3 IS-GPS-200E)
- int **i_SV_health**
- double d_TGD

  Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s].
- double d_IODC

  Issue of Data, Clock.
- int i_AODO

  Age of Data Offset (AODO) term for the navigation message correction table (NMCT) contained in subframe 4 (reference paragraph 20.3.3.5.1.9) [s].
- bool b_fit_interval_flag

  indicates the curve-fit interval used by the CS (Block II/IIA/IIR/IIR-M/IIF) and SS (Block IIIA) in determining the ephemeris parameters, as follows: 0 = 4 hours, 1 = greater than 4 hours.
- double **d_spare1**
- double **d_spare2**
- double d_A_f0

  Coefficient 0 of code phase offset model [s].
- double d_A_f1

  Coefficient 1 of code phase offset model [s/s].
- double d_A_f2

  Coefficient 2 of code phase offset model [s/s^2].
- double d_Toa

  Almanac reference time [s].
- int i_WN_A

Modulo 256 of the GPS week number to which the almanac reference time (d_Toa) is referenced.

- std::map< int, int > almanacHealth

  Map that stores the health information stored in the almanac.

- std::map< int, std::string > satelliteBlock

  Map that stores to which block the PRN belongs http://www.navcen.uscg.gov/?Do=constellation↩Status.

- bool b_integrity_status_flag

  If true, enhanced level of integrity assurance.

- bool b_alert_flag

  If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.

- bool b_antispoofing_flag

  If true, the AntiSpoofing mode is ON in that SV.

- double **d_satClkCorr**
- double **d_dtr**
- double **d_satClkDrift**
- double d_satpos_X

  Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

- double d_satpos_Y

  Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

- double d_satpos_Z

  Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

- int **i_channel_ID**
- unsigned int **i_satellite_PRN**
- double **d_subframe_timestamp_ms**
- bool flag_iono_valid

  If set, it indicates that the ionospheric parameters are filled (page 18 has arrived and decoded)

- double d_alpha0

  Coefficient 0 of a cubic equation representing the amplitude of the vertical delay [s].

- double d_alpha1

  Coefficient 1 of a cubic equation representing the amplitude of the vertical delay [s/semi-circle].

- double d_alpha2

  Coefficient 2 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^2].

- double d_alpha3

  Coefficient 3 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^3].

- double d_beta0

  Coefficient 0 of a cubic equation representing the period of the model [s].

- double d_beta1

  Coefficient 1 of a cubic equation representing the period of the model [s/semi-circle].

- double d_beta2

  Coefficient 2 of a cubic equation representing the period of the model [s(semi-circle)^2].

- double d_beta3

  Coefficient 3 of a cubic equation representing the period of the model [s(semi-circle)^3].

- bool flag_utc_model_valid

  If set, it indicates that the UTC model parameters are filled.

- double d_A1

  1st order term of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s/s]

- double d_A0

  Constant of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s].

- double d_t_OT

Reference time for UTC data (reference 20.3.4.5 and 20.3.3.5.2.4 IS-GPS-200E) [s].

- int i_WN_T

  UTC reference week number [weeks].
- double d_DeltaT_LS

  delta time due to leap seconds [s]. Number of leap seconds since 6-Jan-1980 as transmitted by the GPS almanac.
- int i_WN_LSF

  Week number at the end of which the leap second becomes effective [weeks].
- int i_DN

  Day number (DN) at the end of which the leap second becomes effective [days].
- double d_DeltaT_LSF

  Scheduled future or recent past (relative to NAV message upload) value of the delta time due to leap seconds [s].
- double d_satvel_X

  Earth-fixed velocity coordinate x of the satellite [m].
- double d_satvel_Y

  Earth-fixed velocity coordinate y of the satellite [m].
- double d_satvel_Z

  Earth-fixed velocity coordinate z of the satellite [m].

8.99.1 Detailed Description

This class decodes a GPS NAV Data message as described in IS-GPS-200E.

See http://www.gps.gov/technical/icwg/IS-GPS-200E.pdf Appendix II

Definition at line 54 of file gps_navigation_message.h.

8.99.2 Constructor & Destructor Documentation

8.99.2.1 Gps_Navigation_Message::Gps_Navigation_Message ( )

Default constructor

8.99.3 Member Function Documentation

8.99.3.1 Gps_Ephemeris Gps_Navigation_Message::get_ephemeris ( )

Obtain a GPS SV Ephemeris class filled with current SV data.

8.99.3.2 Gps_Iono Gps_Navigation_Message::get_iono ( )

Obtain a GPS ionospheric correction parameters class filled with current SV data.

8.99.3.3 Gps_Utc_Model Gps_Navigation_Message::get_utc_model ( )

Obtain a GPS UTC model parameters class filled with current SV data.

8.99.3.4 void Gps_Navigation_Message::satellitePosition (double *transmitTime*)

Computes the position of the satellite.

Implementation of Table 20-IV (IS-GPS-200E)

8.99.3.5 int Gps_Navigation_Message::subframe_decoder (char * *subframe*)

Decodes the GPS NAV message.

8.99.3.6 double Gps_Navigation_Message::sv_clock_correction (double *transmitTime*)

Sets (*d_satClkCorr*) according to the User Algorithm for SV Clock Correction and returns the corrected clock (IS-GPS-200E, 20.3.3.3.3.1)

8.99.3.7 double Gps_Navigation_Message::utc_time (const double *gpstime_corrected*) const

Computes the Coordinated Universal Time (UTC) and returns it in s

8.99.4 Member Data Documentation

8.99.4.1 std::map<int,int> Gps_Navigation_Message::almanacHealth

Map that stores the health information stored in the almanac.

Definition at line 126 of file gps_navigation_message.h.

8.99.4.2 bool Gps_Navigation_Message::b_alert_flag

If true, indicates that the SV URA may be worse than indicated in d_SV_accuracy, use that SV at our own risk.

Definition at line 143 of file gps_navigation_message.h.

8.99.4.3 bool Gps_Navigation_Message::b_antispoofing_flag

If true, the AntiSpoofing mode is ON in that SV.

Definition at line 144 of file gps_navigation_message.h.

8.99.4.4 bool Gps_Navigation_Message::b_fit_interval_flag

indicates the curve-fit interval used by the CS (Block II/IIA/IIR/IIR-M/IIF) and SS (Block IIIA) in determining the ephemeris parameters, as follows: 0 = 4 hours, 1 = greater than 4 hours.

Definition at line 114 of file gps_navigation_message.h.

8.99.4.5 bool Gps_Navigation_Message::b_integrity_status_flag

If true, enhanced level of integrity assurance.

If false, indicates that the conveying signal is provided with the legacy level of integrity assurance. That is, the probability that the instantaneous URE of the conveying signal exceeds 4.42 times the upper bound value of the current broadcast URA index, for more than 5.2 seconds, without an accompanying alert, is less than 1E-5 per hour. If true, indicates that the conveying signal is provided with an enhanced level of integrity assurance. That is, the probability that the instantaneous URE of the conveying signal exceeds 5.73 times the upper bound value of the current broadcast URA index, for more than 5.2 seconds, without an accompanying alert, is less than 1E-8 per hour.

Definition at line 142 of file gps_navigation_message.h.

8.99.4.6 bool Gps_Navigation_Message::b_L2_P_data_flag

When true, indicates that the NAV data stream was commanded OFF on the P-code of the L2 channel.

Definition at line 105 of file gps_navigation_message.h.

8.99.4.7 double Gps_Navigation_Message::d_A0

Constant of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s].

Definition at line 178 of file gps_navigation_message.h.

8.99.4.8 double Gps_Navigation_Message::d_A1

1st order term of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s/s]

Definition at line 177 of file gps_navigation_message.h.

8.99.4.9 double Gps_Navigation_Message::d_A_f0

Coefficient 0 of code phase offset model [s].

Definition at line 118 of file gps_navigation_message.h.

8.99.4.10 double Gps_Navigation_Message::d_A_f1

Coefficient 1 of code phase offset model [s/s].

Definition at line 119 of file gps_navigation_message.h.

8.99.4.11 double Gps_Navigation_Message::d_A_f2

Coefficient 2 of code phase offset model [s/s^2].

Definition at line 120 of file gps_navigation_message.h.

8.99.4.12 double Gps_Navigation_Message::d_alpha0

Coefficient 0 of a cubic equation representing the amplitude of the vertical delay [s].

Definition at line 166 of file gps_navigation_message.h.

8.99.4.13 double Gps_Navigation_Message::d_alpha1

Coefficient 1 of a cubic equation representing the amplitude of the vertical delay [s/semi-circle].

Definition at line 167 of file gps_navigation_message.h.

8.99.4.14 double Gps_Navigation_Message::d_alpha2

Coefficient 2 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^2].

Definition at line 168 of file gps_navigation_message.h.

8.99.4.15 double Gps_Navigation_Message::d_alpha3

Coefficient 3 of a cubic equation representing the amplitude of the vertical delay [s(semi-circle)^3].

Definition at line 169 of file gps_navigation_message.h.

8.99.4.16 double Gps_Navigation_Message::d_beta0

Coefficient 0 of a cubic equation representing the period of the model [s].

Definition at line 170 of file gps_navigation_message.h.

8.99.4.17 double Gps_Navigation_Message::d_beta1

Coefficient 1 of a cubic equation representing the period of the model [s/semi-circle].

Definition at line 171 of file gps_navigation_message.h.

8.99.4.18 double Gps_Navigation_Message::d_beta2

Coefficient 2 of a cubic equation representing the period of the model [s(semi-circle)^2].

Definition at line 172 of file gps_navigation_message.h.

8.99.4.19 double Gps_Navigation_Message::d_beta3

Coefficient 3 of a cubic equation representing the period of the model [s(semi-circle)^3].

Definition at line 173 of file gps_navigation_message.h.

8.99.4.20 double Gps_Navigation_Message::d_Cic

Amplitude of the Cosine Harmonic Correction Term to the Angle of Inclination [rad].

Definition at line 93 of file gps_navigation_message.h.

8.99.4.21 double Gps_Navigation_Message::d_Cis

Amplitude of the Sine Harmonic Correction Term to the Angle of Inclination [rad].

Definition at line 95 of file gps_navigation_message.h.

8.99.4.22 double Gps_Navigation_Message::d_Crc

Amplitude of the Cosine Harmonic Correction Term to the Orbit Radius [m].

Definition at line 98 of file gps_navigation_message.h.

8.99.4.23 double Gps_Navigation_Message::d_Crs

Amplitude of the Sine Harmonic Correction Term to the Orbit Radius [m].

Definition at line 82 of file gps_navigation_message.h.

8.99.4.24 double Gps_Navigation_Message::d_Cuc

Amplitude of the Cosine Harmonic Correction Term to the Argument of Latitude [rad].

Definition at line 86 of file gps_navigation_message.h.

8.99.4.25 double Gps_Navigation_Message::d_Cus

Amplitude of the Sine Harmonic Correction Term to the Argument of Latitude [rad].

Definition at line 88 of file gps_navigation_message.h.

8.99.4.26 double Gps_Navigation_Message::d_Delta_n

Mean Motion Difference From Computed Value [semi-circles/s].

Definition at line 83 of file gps_navigation_message.h.

8.99.4.27 double Gps_Navigation_Message::d_DeltaT_LS

delta time due to leap seconds [s]. Number of leap seconds since 6-Jan-1980 as transmitted by the GPS almanac.

Definition at line 181 of file gps_navigation_message.h.

8.99.4.28 double Gps_Navigation_Message::d_DeltaT_LSF

Scheduled future or recent past (relative to NAV message upload) value of the delta time due to leap seconds [s].

Definition at line 184 of file gps_navigation_message.h.

8.99.4.29 double Gps_Navigation_Message::d_e_eccentricity

Eccentricity [dimensionless].

Definition at line 87 of file gps_navigation_message.h.

8.99.4.30 double Gps_Navigation_Message::d_i_0

Inclination Angle at Reference Time [semi-circles].

Definition at line 97 of file gps_navigation_message.h.

8.99.4.31 double Gps_Navigation_Message::d_IDOT

Rate of Inclination Angle [semi-circles/s].

Definition at line 102 of file gps_navigation_message.h.

8.99.4.32 double Gps_Navigation_Message::d_IODC

Issue of Data, Clock.

Definition at line 110 of file gps_navigation_message.h.

8.99.4.33 double Gps_Navigation_Message::d_M_0

Mean Anomaly at Reference Time [semi-circles].

Definition at line 84 of file gps_navigation_message.h.

8.99.4.34 double Gps_Navigation_Message::d_OMEGA

Argument of Perigee [semi-cicles].

Definition at line 99 of file gps_navigation_message.h.

8.99.4.35 double Gps_Navigation_Message::d_OMEGA0

Longitude of Ascending Node of Orbit Plane at Weekly Epoch [semi-circles].

Definition at line 94 of file gps_navigation_message.h.

8.99.4.36 double Gps_Navigation_Message::d_OMEGA_DOT

Rate of Right Ascension [semi-circles/s].

Definition at line 100 of file gps_navigation_message.h.

8.99.4.37 double Gps_Navigation_Message::d_satpos_X

Earth-fixed coordinate x of the satellite [m]. Intersection of the IERS Reference Meridian (IRM) and the plane passing through the origin and normal to the Z-axis.

Definition at line 153 of file gps_navigation_message.h.

8.99.4.38 double Gps_Navigation_Message::d_satpos_Y

Earth-fixed coordinate y of the satellite [m]. Completes a right-handed, Earth-Centered, Earth-Fixed orthogonal coordinate system.

Definition at line 154 of file gps_navigation_message.h.

8.99.4.39 double Gps_Navigation_Message::d_satpos_Z

Earth-fixed coordinate z of the satellite [m]. The direction of the IERS (International Earth Rotation and Reference Systems Service) Reference Pole (IRP).

Definition at line 155 of file gps_navigation_message.h.

8.99.4.40 double Gps_Navigation_Message::d_satvel_X

Earth-fixed velocity coordinate x of the satellite [m].

Definition at line 187 of file gps_navigation_message.h.

8.99.4.41 double Gps_Navigation_Message::d_satvel_Y

Earth-fixed velocity coordinate y of the satellite [m].

Definition at line 188 of file gps_navigation_message.h.

8.99.4.42 double Gps_Navigation_Message::d_satvel_Z

Earth-fixed velocity coordinate z of the satellite [m].

Definition at line 189 of file gps_navigation_message.h.

8.99.4.43 double Gps_Navigation_Message::d_sqrt_A

Square Root of the Semi-Major Axis [sqrt(m)].

Definition at line 89 of file gps_navigation_message.h.

8.99.4.44 double Gps_Navigation_Message::d_t_OT

Reference time for UTC data (reference 20.3.4.5 and 20.3.3.5.2.4 IS-GPS-200E) [s].

Definition at line 179 of file gps_navigation_message.h.

8.99.4.45 double Gps_Navigation_Message::d_TGD

Estimated Group Delay Differential: L1-L2 correction term only for the benefit of "L1 P(Y)" or "L2 P(Y)" s users [s].

Definition at line 109 of file gps_navigation_message.h.

8.99.4.46 double Gps_Navigation_Message::d_Toa

Almanac reference time [s].

Definition at line 124 of file gps_navigation_message.h.

8.99.4.47 double Gps_Navigation_Message::d_Toc

clock data reference time (Ref. 20.3.3.3.3.1 IS-GPS-200E) [s]

Definition at line 92 of file gps_navigation_message.h.

8.99.4.48 double Gps_Navigation_Message::d_Toe

Ephemeris data reference time of week (Ref. 20.3.3.4.3 IS-GPS-200E) [s].

Definition at line 91 of file gps_navigation_message.h.

8.99.4.49 double Gps_Navigation_Message::d_TOW

Time of GPS Week of the ephemeris set (taken from subframes TOW) [s].

Definition at line 73 of file gps_navigation_message.h.

8.99.4.50 double Gps_Navigation_Message::d_TOW_SF1

Time of GPS Week from HOW word of Subframe 1 [s].

Definition at line 74 of file gps_navigation_message.h.

8.99.4.51 double Gps_Navigation_Message::d_TOW_SF2

Time of GPS Week from HOW word of Subframe 2 [s].

Definition at line 75 of file gps_navigation_message.h.

8.99.4.52 double Gps_Navigation_Message::d_TOW_SF3

Time of GPS Week from HOW word of Subframe 3 [s].

Definition at line 76 of file gps_navigation_message.h.

8.99.4.53 double Gps_Navigation_Message::d_TOW_SF4

Time of GPS Week from HOW word of Subframe 4 [s].

Definition at line 77 of file gps_navigation_message.h.

8.99.4.54 double Gps_Navigation_Message::d_TOW_SF5

Time of GPS Week from HOW word of Subframe 5 [s].

Definition at line 78 of file gps_navigation_message.h.

8.99.4.55 bool Gps_Navigation_Message::flag_iono_valid

If set, it indicates that the ionospheric parameters are filled (page 18 has arrived and decoded)

Definition at line 165 of file gps_navigation_message.h.

8.99.4.56 bool Gps_Navigation_Message::flag_utc_model_valid

If set, it indicates that the UTC model parameters are filled.

Definition at line 176 of file gps_navigation_message.h.

8.99.4.57 int Gps_Navigation_Message::i_AODO

Age of Data Offset (AODO) term for the navigation message correction table (NMCT) contained in subframe 4 (reference paragraph 20.3.3.5.1.9) [s].

Definition at line 112 of file gps_navigation_message.h.

8.99.4.58 int Gps_Navigation_Message::i_code_on_L2

If 1, P code ON in L2; if 2, C/A code ON in L2;.

Definition at line 103 of file gps_navigation_message.h.

8.99.4.59 int Gps_Navigation_Message::i_DN

Day number (DN) at the end of which the leap second becomes effective [days].

Definition at line 183 of file gps_navigation_message.h.

8.99.4.60 int Gps_Navigation_Message::i_GPS_week

GPS week number, aka WN [week].

Definition at line 104 of file gps_navigation_message.h.

8.99.4.61 int Gps_Navigation_Message::i_SV_accuracy

User Range Accuracy (URA) index of the SV (reference paragraph 6.2.1) for the standard positioning service user (Ref 20.3.3.3.1.3 IS-GPS-200E)

Definition at line 107 of file gps_navigation_message.h.

8.99.4.62 int Gps_Navigation_Message::i_WN_A

Modulo 256 of the GPS week number to which the almanac reference time (d_Toa) is referenced.

Definition at line 125 of file gps_navigation_message.h.

8.99.4.63 int Gps_Navigation_Message::i_WN_LSF

Week number at the end of which the leap second becomes effective [weeks].

Definition at line 182 of file gps_navigation_message.h.

8.99.4.64 int Gps_Navigation_Message::i_WN_T

UTC reference week number [weeks].

Definition at line 180 of file gps_navigation_message.h.

8.99.4.65 std::map<int,std::string> Gps_Navigation_Message::satelliteBlock

Map that stores to which block the PRN belongs http://www.navcen.uscg.gov/?Do=constellation↩Status.

Definition at line 128 of file gps_navigation_message.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_navigation_message.h

8.100 Gps_Ref_Location Class Reference

Interface of a GPS REFERENCE LOCATION storage.

`#include <gps_ref_location.h>`

Public Member Functions

- Gps_Ref_Location ()
- template<class Archive >
  void serialize (Archive &archive, const unsigned int version)

  Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the Ref location on disk file.

Public Attributes

- bool **valid**
- double **lat**
- double **lon**
- double **uncertainty**

8.100.1 Detailed Description

Interface of a GPS REFERENCE LOCATION storage.

Definition at line 43 of file gps_ref_location.h.

8.100.2 Constructor & Destructor Documentation

8.100.2.1 Gps_Ref_Location::Gps_Ref_Location ( )

Default constructor

8.100.3 Member Function Documentation

8.100.3.1 template<class Archive > void Gps_Ref_Location::serialize (Archive & *archive,* const unsigned int *version*) [inline]

Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the Ref location on disk file.

Definition at line 60 of file gps_ref_location.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_ref_location.h

8.101 Gps_Ref_Time Class Reference

Interface of a GPS REFERENCE TIME storage.

`#include <gps_ref_time.h>`

Public Member Functions

- Gps_Ref_Time ()
- template<class Archive >
  void serialize (Archive &archive, const unsigned int version)

  Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ref time data on disk file.

Public Attributes

- bool **valid**
- double **d_TOW**
- double **d_Week**
- double **d_tv_sec**
- double **d_tv_usec**

8.101.1 Detailed Description

Interface of a GPS REFERENCE TIME storage.

Definition at line 43 of file gps_ref_time.h.

8.101.2 Constructor & Destructor Documentation

8.101.2.1 Gps_Ref_Time::Gps_Ref_Time ( )

Default constructor

8.101.3 Member Function Documentation

8.101.3.1 template<class Archive > void Gps_Ref_Time::serialize (Archive & *archive,* const unsigned int *version*) [inline]

Serialize is a boost standard method to be called by the boost XML serialization. Here is used to save the ref time data on disk file.

Definition at line 61 of file gps_ref_time.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/gps_ref_time.h

8.102 Gps_Utc_Model Class Reference

This class is a storage for the GPS UTC MODEL data as described in IS-GPS-200E.

`#include <gps_utc_model.h>`

Public Member Functions

- Gps_Utc_Model ()
- template<class Archive >
  void **serialize** (Archive &archive, const unsigned int version)
- double utc_time (double gpstime_corrected, int i_GPS_week)

  Computes the Coordinated Universal Time (UTC) and returns it in s

Public Attributes

- bool **valid**
- double d_A1

  1st order term of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s/s]
- double d_A0

  Constant of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s].
- double d_t_OT

  Reference time for UTC data (reference 20.3.4.5 and 20.3.3.5.2.4 IS-GPS-200E) [s].
- int i_WN_T

  UTC reference week number [weeks].
- double d_DeltaT_LS

  delta time due to leap seconds [s]. Number of leap seconds since 6-Jan-1980 as transmitted by the GPS almanac.
- int i_WN_LSF

  Week number at the end of which the leap second becomes effective [weeks].
- int i_DN

  Day number (DN) at the end of which the leap second becomes effective [days].
- double d_DeltaT_LSF

  Scheduled future or recent past (relative to NAV message upload) value of the delta time due to leap seconds [s].

8.102.1 Detailed Description

This class is a storage for the GPS UTC MODEL data as described in IS-GPS-200E.

See http://www.gps.gov/technical/icwg/IS-GPS-200E.pdf Appendix II

Definition at line 44 of file gps_utc_model.h.

8.102.2 Constructor & Destructor Documentation

8.102.2.1 Gps_Utc_Model::Gps_Utc_Model ( )

Default constructor

8.102.3 Member Function Documentation

8.102.3.1 double Gps_Utc_Model::utc_time (double *gpstime_corrected,* int *i_GPS_week*)

Computes the Coordinated Universal Time (UTC) and returns it in s

8.102.4 Member Data Documentation

8.102.4.1 double Gps_Utc_Model::d_A0

Constant of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s].

Definition at line 50 of file gps_utc_model.h.

8.102.4.2 double Gps_Utc_Model::d_A1

1st order term of a model that relates GPS and UTC time (ref. 20.3.3.5.2.4 IS-GPS-200E) [s/s]

Definition at line 49 of file gps_utc_model.h.

8.102.4.3 double Gps_Utc_Model::d_DeltaT_LS

delta time due to leap seconds [s]. Number of leap seconds since 6-Jan-1980 as transmitted by the GPS almanac.

Definition at line 53 of file gps_utc_model.h.

8.102.4.4 double Gps_Utc_Model::d_DeltaT_LSF

Scheduled future or recent past (relative to NAV message upload) value of the delta time due to leap seconds [s].

Definition at line 56 of file gps_utc_model.h.

8.102.4.5 double Gps_Utc_Model::d_t_OT

Reference time for UTC data (reference 20.3.4.5 and 20.3.3.5.2.4 IS-GPS-200E) [s].

Definition at line 51 of file gps_utc_model.h.

8.102.4.6 int Gps_Utc_Model::i_DN

Day number (DN) at the end of which the leap second becomes effective [days].

Definition at line 55 of file gps_utc_model.h.

8.102.4.7 int Gps_Utc_Model::i_WN_LSF

Week number at the end of which the leap second becomes effective [weeks].

Definition at line 54 of file gps_utc_model.h.

8.102.4.8 int Gps_Utc_Model::i_WN_T

UTC reference week number [weeks].

Definition at line 52 of file gps_utc_model.h.

The documentation for this class was generated from the following file:

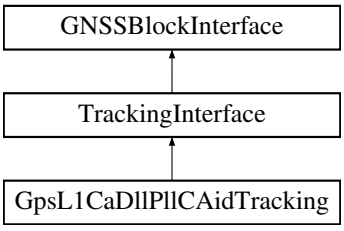
- src/core/system_parameters/gps_utc_model.h

8.103 GpsL1CaDllPllCAidTracking Class Reference

This class implements a code DLL + carrier PLL tracking loop.

`#include <gps_l1_ca_dll_pll_c_aid_tracking.h>`

Inheritance diagram for GpsL1CaDllPllCAidTracking:



Public Member Functions

- **GpsL1CaDllPllCAidTracking** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_DLL_PLL_C_Aid_Tracking".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.103.1 Detailed Description

This class implements a code DLL + carrier PLL tracking loop.

Definition at line 52 of file gps_l1_ca_dll_pll_c_aid_tracking.h.

8.103.2 Member Function Documentation

8.103.2.1 std::string GpsL1CaDllPllCAidTracking::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_DLL_PLL_C_Aid_Tracking".

Implements GNSSBlockInterface.

Definition at line 69 of file gps_l1_ca_dll_pll_c_aid_tracking.h.

References set_channel(), and set_gnss_synchro().

8.103.2.2 void GpsL1CaDllPllCAidTracking::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.103.2.3 void GpsL1CaDllPllCAidTracking::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

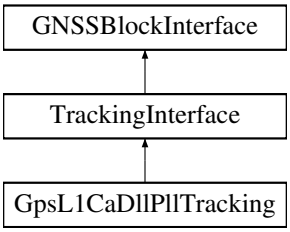
- src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_c_aid_tracking.h

8.104 GpsL1CaDllPllTracking Class Reference

This class implements a code DLL + carrier PLL tracking loop.

`#include <gps_l1_ca_dll_pll_tracking.h>`

Inheritance diagram for GpsL1CaDllPllTracking:



Public Member Functions

- **GpsL1CaDllPllTracking** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_DLL_PLL_Tracking".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.104.1 Detailed Description

This class implements a code DLL + carrier PLL tracking loop.

Definition at line 51 of file gps_l1_ca_dll_pll_tracking.h.

8.104.2 Member Function Documentation

8.104.2.1 std::string GpsL1CaDllPllTracking::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_DLL_PLL_Tracking".

Implements GNSSBlockInterface.

Definition at line 67 of file gps_l1_ca_dll_pll_tracking.h.

References set_channel(), and set_gnss_synchro().

8.104.2.2 void GpsL1CaDllPllTracking::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.104.2.3 void GpsL1CaDllPllTracking::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

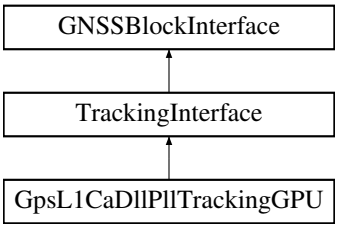
- src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking.h

8.105 GpsL1CaDllPllTrackingGPU Class Reference

This class implements a code DLL + carrier PLL tracking loop using GPU accelerated functions.

`#include <gps_l1_ca_dll_pll_tracking_gpu.h>`

Inheritance diagram for GpsL1CaDllPllTrackingGPU:



Public Member Functions

- **GpsL1CaDllPllTrackingGPU** (ConfigurationInterface *configuration, std::string role, unsigned int in_↩ streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_DLL_PLL_Tracking_GPU".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.105.1 Detailed Description

This class implements a code DLL + carrier PLL tracking loop using GPU accelerated functions.

Definition at line 50 of file gps_l1_ca_dll_pll_tracking_gpu.h.

8.105.2 Member Function Documentation

8.105.2.1 std::string GpsL1CaDllPllTrackingGPU::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_DLL_PLL_Tracking_GPU".

Implements GNSSBlockInterface.

Definition at line 66 of file gps_l1_ca_dll_pll_tracking_gpu.h.

References set_channel(), and set_gnss_synchro().

8.105.2.2 void GpsL1CaDllPllTrackingGPU::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.105.2.3 void GpsL1CaDllPllTrackingGPU::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

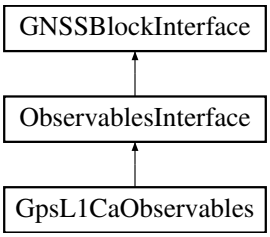
- src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking_gpu.h

8.106 GpsL1CaObservables Class Reference

This class implements an ObservablesInterface for GPS L1 C/A.

`#include <gps_l1_ca_observables.h>`

Inheritance diagram for GpsL1CaObservables:



Public Member Functions

- **GpsL1CaObservables** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_Observables".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation.

8.106.1 Detailed Description

This class implements an ObservablesInterface for GPS L1 C/A.

Definition at line 46 of file gps_l1_ca_observables.h.

8.106.2 Member Function Documentation

8.106.2.1 std::string GpsL1CaObservables::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_Observables".

Implements GNSSBlockInterface.

Definition at line 60 of file gps_l1_ca_observables.h.

8.106.2.2 size_t GpsL1CaObservables::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation.

Implements GNSSBlockInterface.

Definition at line 74 of file gps_l1_ca_observables.h.

The documentation for this class was generated from the following file:

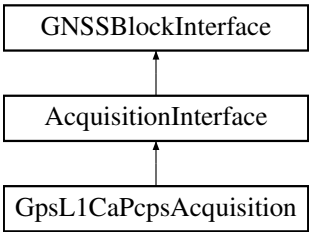
- src/algorithms/observables/adapters/gps_l1_ca_observables.h

8.107 GpsL1CaPcpsAcquisition Class Reference

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

`#include <gps_l1_ca_pcps_acquisition.h>`

Inheritance diagram for GpsL1CaPcpsAcquisition:



Public Member Functions

- **GpsL1CaPcpsAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PCPS_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for GPS L1/CA PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.107.1 Detailed Description

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Definition at line 57 of file gps_l1_ca_pcps_acquisition.h.

8.107.2 Member Function Documentation

8.107.2.1 std::string GpsL1CaPcpsAcquisition::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PCPS_Acquisition".

Implements GNSSBlockInterface.

Definition at line 74 of file gps_l1_ca_pcps_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_local_code(), set_state(), and set_threshold().

8.107.2.2 void GpsL1CaPcpsAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.3 signed int GpsL1CaPcpsAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.4 void GpsL1CaPcpsAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.5 void GpsL1CaPcpsAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.6 void GpsL1CaPcpsAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.7 void GpsL1CaPcpsAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.8 void GpsL1CaPcpsAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.9 void GpsL1CaPcpsAcquisition::set_local_code ( ) [virtual]

Sets local code for GPS L1/CA PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.107.2.10 void GpsL1CaPcpsAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.107.2.11 void GpsL1CaPcpsAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

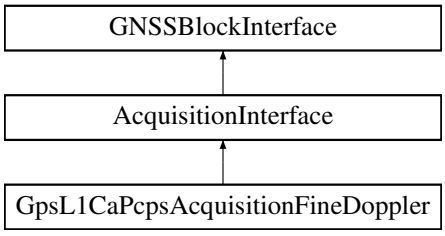
- src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition.h

8.108 GpsL1CaPcpsAcquisitionFineDoppler Class Reference

This class Adapts a PCPS acquisition block with fine Doppler estimation to an AcquisitionInterface for GPS L1 C/A signals.

`#include <gps_l1_ca_pcps_acquisition_fine_doppler.h>`

Inheritance diagram for GpsL1CaPcpsAcquisitionFineDoppler:



Public Member Functions

- **GpsL1CaPcpsAcquisitionFineDoppler** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PCPS_Acquisition_Fine_Doppler".
- size_t **item_size** ()
- void **connect** (boost::shared_ptr< gr::top_block > top_block)
- void **disconnect** (boost::shared_ptr< gr::top_block > top_block)
- boost::shared_ptr< gr::basic_block > **get_left_block** ()
- boost::shared_ptr< gr::basic_block > **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void **set_local_code** ()
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.

8.108.1 Detailed Description

This class Adapts a PCPS acquisition block with fine Doppler estimation to an AcquisitionInterface for GPS L1 C/A signals.

Definition at line 50 of file gps_l1_ca_pcps_acquisition_fine_doppler.h.

8.108.2 Member Function Documentation

8.108.2.1 std::string GpsL1CaPcpsAcquisitionFineDoppler::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PCPS_Acquisition_Fine_Doppler".

Implements GNSSBlockInterface.

Definition at line 67 of file gps_l1_ca_pcps_acquisition_fine_doppler.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), and set_threshold().

8.108.2.2 void GpsL1CaPcpsAcquisitionFineDoppler::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.108.2.3 signed int GpsL1CaPcpsAcquisitionFineDoppler::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.108.2.4 void GpsL1CaPcpsAcquisitionFineDoppler::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.108.2.5 void GpsL1CaPcpsAcquisitionFineDoppler::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.108.2.6 void GpsL1CaPcpsAcquisitionFineDoppler::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.108.2.7 void GpsL1CaPcpsAcquisitionFineDoppler::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.108.2.8 void GpsL1CaPcpsAcquisitionFineDoppler::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.108.2.9 void GpsL1CaPcpsAcquisitionFineDoppler::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

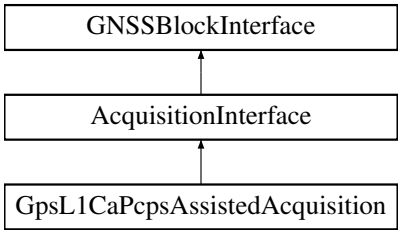
- src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition_fine_doppler.h

8.109 GpsL1CaPcpsAssistedAcquisition Class Reference

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

```
#include <gps_l1_ca_pcps_assisted_acquisition.h>
```

Inheritance diagram for GpsL1CaPcpsAssistedAcquisition:



Public Member Functions

- **GpsL1CaPcpsAssistedAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PCPS_Assisted_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void **set_local_code** ()
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.

8.109.1 Detailed Description

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Definition at line 50 of file gps_l1_ca_pcps_assisted_acquisition.h.

8.109.2 Member Function Documentation

8.109.2.1 std::string GpsL1CaPcpsAssistedAcquisition::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PCPS_Assisted_Acquisition".

Implements GNSSBlockInterface.

Definition at line 67 of file gps_l1_ca_pcps_assisted_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), and set_threshold().

8.109.2.2 void GpsL1CaPcpsAssistedAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.109.2.3 signed int GpsL1CaPcpsAssistedAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.109.2.4 void GpsL1CaPcpsAssistedAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.109.2.5 void GpsL1CaPcpsAssistedAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.109.2.6 void GpsL1CaPcpsAssistedAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.109.2.7 void GpsL1CaPcpsAssistedAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.109.2.8 void GpsL1CaPcpsAssistedAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.109.2.9 void GpsL1CaPcpsAssistedAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

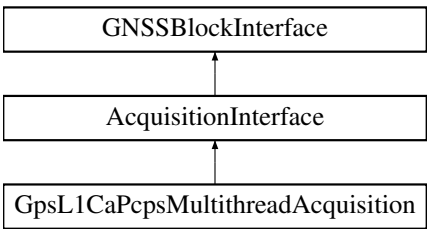
- src/algorithms/acquisition/adapters/gps_l1_ca_pcps_assisted_acquisition.h

8.110 GpsL1CaPcpsMultithreadAcquisition Class Reference

This class adapts a multithread PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

`#include <gps_l1_ca_pcps_multithread_acquisition.h>`

Inheritance diagram for GpsL1CaPcpsMultithreadAcquisition:



Public Member Functions

- **GpsL1CaPcpsMultithreadAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PCPS_Multithread_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for GPS L1/CA PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.

8.110.1 Detailed Description

This class adapts a multithread PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Definition at line 49 of file gps_l1_ca_pcps_multithread_acquisition.h.

8.110.2 Member Function Documentation

8.110.2.1 std::string GpsL1CaPcpsMultithreadAcquisition::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PCPS_Multithread_Acquisition".

Implements GNSSBlockInterface.

Definition at line 66 of file gps_l1_ca_pcps_multithread_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_local_code(), and set_threshold().

8.110.2.2 void GpsL1CaPcpsMultithreadAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.3 signed int GpsL1CaPcpsMultithreadAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.4 void GpsL1CaPcpsMultithreadAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.5 void GpsL1CaPcpsMultithreadAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.6 void GpsL1CaPcpsMultithreadAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.7 void GpsL1CaPcpsMultithreadAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.8 void GpsL1CaPcpsMultithreadAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.9 void GpsL1CaPcpsMultithreadAcquisition::set_local_code ( ) [virtual]

Sets local code for GPS L1/CA PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.110.2.10 void GpsL1CaPcpsMultithreadAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

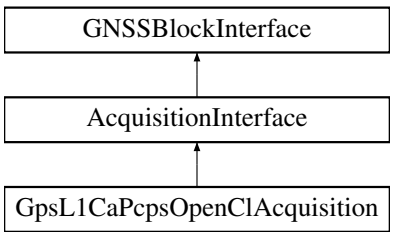
- src/algorithms/acquisition/adapters/gps_l1_ca_pcps_multithread_acquisition.h

8.111 GpsL1CaPcpsOpenClAcquisition Class Reference

This class adapts an OpenCL PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

`#include <gps_l1_ca_pcps_opencl_acquisition.h>`

Inheritance diagram for GpsL1CaPcpsOpenClAcquisition:



Public Member Functions

- **GpsL1CaPcpsOpenClAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PCPS_OpenCl_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for GPS L1/CA PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.

8.111.1 Detailed Description

This class adapts an OpenCL PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Definition at line 49 of file gps_l1_ca_pcps_opencl_acquisition.h.

8.111.2 Member Function Documentation

8.111.2.1 std::string GpsL1CaPcpsOpenClAcquisition::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PCPS_OpenCl_Acquisition".

Implements GNSSBlockInterface.

Definition at line 66 of file gps_l1_ca_pcps_opencl_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set_local_code(), and set_threshold().

8.111.2.2 void GpsL1CaPcpsOpenClAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.3 signed int GpsL1CaPcpsOpenClAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.4 void GpsL1CaPcpsOpenClAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.5 void GpsL1CaPcpsOpenClAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.6 void GpsL1CaPcpsOpenClAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.7 void GpsL1CaPcpsOpenClAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.8 void GpsL1CaPcpsOpenClAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.9 void GpsL1CaPcpsOpenClAcquisition::set_local_code ( ) [virtual]

Sets local code for GPS L1/CA PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.111.2.10 void GpsL1CaPcpsOpenClAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

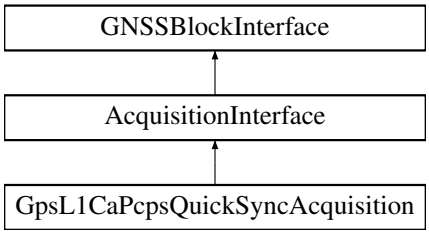
- src/algorithms/acquisition/adapters/gps_l1_ca_pcps_opencl_acquisition.h

8.112 GpsL1CaPcpsQuickSyncAcquisition Class Reference

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

`#include <gps_l1_ca_pcps_quicksync_acquisition.h>`

Inheritance diagram for GpsL1CaPcpsQuickSyncAcquisition:



Public Member Functions

- **GpsL1CaPcpsQuickSyncAcquisition** (ConfigurationInterface ∗configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PCPS_QuickSync_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro ∗p_gnss_synchro)

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for GPS L1/CA PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.112.1 Detailed Description

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Definition at line 51 of file gps_l1_ca_pcps_quicksync_acquisition.h.

8.112.2 Member Function Documentation

8.112.2.1 std::string GpsL1CaPcpsQuickSyncAcquisition::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PCPS_QuickSync_Acquisition".

Implements GNSSBlockInterface.

Definition at line 68 of file gps_l1_ca_pcps_quicksync_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set↩_local_code(), set_state(), and set_threshold().

8.112.2.2 void GpsL1CaPcpsQuickSyncAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.3 signed int GpsL1CaPcpsQuickSyncAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.4 void GpsL1CaPcpsQuickSyncAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.5 void GpsL1CaPcpsQuickSyncAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.6 void GpsL1CaPcpsQuickSyncAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.7 void GpsL1CaPcpsQuickSyncAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.8 void GpsL1CaPcpsQuickSyncAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.9 void GpsL1CaPcpsQuickSyncAcquisition::set_local_code ( ) [virtual]

Sets local code for GPS L1/CA PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.112.2.10 void GpsL1CaPcpsQuickSyncAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.112.2.11 void GpsL1CaPcpsQuickSyncAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

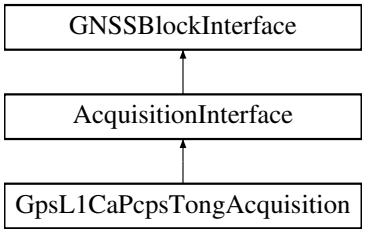
- src/algorithms/acquisition/adapters/gps_l1_ca_pcps_quicksync_acquisition.h

8.113 GpsL1CaPcpsTongAcquisition Class Reference

This class adapts a PCPS Tong acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

`#include <gps_l1_ca_pcps_tong_acquisition.h>`

Inheritance diagram for GpsL1CaPcpsTongAcquisition:



Public Member Functions

- **GpsL1CaPcpsTongAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_↩ streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PCPS_Tong_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of TONG algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for GPS L1/CA TONG acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.113.1 Detailed Description

This class adapts a PCPS Tong acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Definition at line 48 of file gps_l1_ca_pcps_tong_acquisition.h.

8.113.2 Member Function Documentation

8.113.2.1 std::string GpsL1CaPcpsTongAcquisition::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PCPS_Tong_Acquisition".

Implements GNSSBlockInterface.

Definition at line 65 of file gps_l1_ca_pcps_tong_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set↩ _local_code(), set_state(), and set_threshold().

8.113.2.2 void GpsL1CaPcpsTongAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.3 signed int GpsL1CaPcpsTongAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.4 void GpsL1CaPcpsTongAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.5 void GpsL1CaPcpsTongAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.6 void GpsL1CaPcpsTongAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.7 void GpsL1CaPcpsTongAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.8 void GpsL1CaPcpsTongAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.9 void GpsL1CaPcpsTongAcquisition::set_local_code ( ) [virtual]

Sets local code for GPS L1/CA TONG acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.113.2.10 void GpsL1CaPcpsTongAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.113.2.11 void GpsL1CaPcpsTongAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of TONG algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

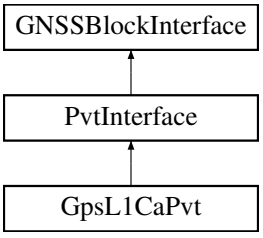
- src/algorithms/acquisition/adapters/gps_l1_ca_pcps_tong_acquisition.h

8.114 GpsL1CaPvt Class Reference

This class implements a PvtInterface for GPS L1 C/A.

`#include <gps_l1_ca_pvt.h>`

Inheritance diagram for GpsL1CaPvt:



Public Member Functions

- **GpsL1CaPvt** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_PVT".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation. Returns sizeof(gr_complex)

8.114.1 Detailed Description

This class implements a PvtInterface for GPS L1 C/A.

Definition at line 49 of file gps_l1_ca_pvt.h.

8.114.2 Member Function Documentation

8.114.2.1 std::string GpsL1CaPvt::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_PVT".

Implements GNSSBlockInterface.

Definition at line 65 of file gps_l1_ca_pvt.h.

8.114.2.2 size_t GpsL1CaPvt::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation. Returns sizeof(gr_complex)

Implements GNSSBlockInterface.

Definition at line 81 of file gps_l1_ca_pvt.h.

The documentation for this class was generated from the following file:

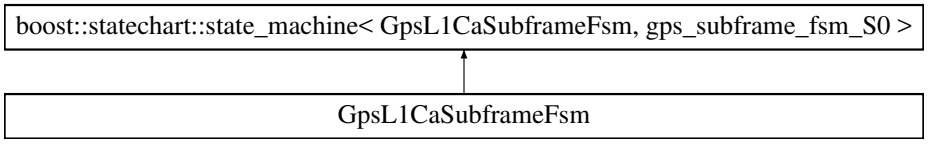
- src/algorithms/PVT/adapters/gps_l1_ca_pvt.h

8.115 GpsL1CaSubframeFsm Class Reference

This class implements a Finite State Machine that handles the decoding of the GPS L1 C/A NAV message.

`#include <gps_l1_ca_subframe_fsm.h>`

Inheritance diagram for GpsL1CaSubframeFsm:



Public Member Functions

- GpsL1CaSubframeFsm ()
  The constructor starts the Finite State Machine.
- void **clear_flag_new_subframe** ()
- void gps_word_to_subframe (int position)
  inserts the word in the correct position of the subframe
- void gps_subframe_to_nav_msg ()
  This function decodes a NAv message subframe and pushes the information to the right queues.
- void Event_gps_word_valid ()
  FSM event: the received word is valid.
- void Event_gps_word_invalid ()
  FSM event: the received word is not valid.
- void Event_gps_word_preamble ()
  FSM event: word preamble detected.

Public Attributes

- int i_channel_ID
  Channel id.
- unsigned int i_satellite_PRN
  Satellite PRN number.
- Gps_Navigation_Message d_nav
  GPS L1 C/A navigation message object.
- Gps_Ephemeris ephemeris
  Object that handles GPS ephemeris parameters.
- Gps_Almanac almanac
  Object that handles GPS almanac data.
- Gps_Utc_Model utc_model
  Object that handles UTM model parameters.
- Gps_Iono iono
  Object that handles ionospheric parameters.
- char **d_subframe** [GPS_SUBFRAME_LENGTH]
- int **d_subframe_ID**
- bool **d_flag_new_subframe**
- char **d_GPS_frame_4bytes** [GPS_WORD_LENGTH]
- double **d_preamble_time_ms**

8.115.1 Detailed Description

This class implements a Finite State Machine that handles the decoding of the GPS L1 C/A NAV message.

Definition at line 65 of file gps_l1_ca_subframe_fsm.h.

8.115.2 Constructor & Destructor Documentation

8.115.2.1 GpsL1CaSubframeFsm::GpsL1CaSubframeFsm ( )

The constructor starts the Finite State Machine.

8.115.3 Member Function Documentation

8.115.3.1 void GpsL1CaSubframeFsm::Event_gps_word_invalid ( )

FSM event: the received word is not valid.

8.115.3.2 void GpsL1CaSubframeFsm::Event_gps_word_preamble ( )

FSM event: word preamble detected.

8.115.3.3 void GpsL1CaSubframeFsm::Event_gps_word_valid ( )

FSM event: the received word is valid.

8.115.3.4 void GpsL1CaSubframeFsm::gps_subframe_to_nav_msg ( )

This function decodes a NAv message subframe and pushes the information to the right queues.

8.115.3.5 void GpsL1CaSubframeFsm::gps_word_to_subframe (int *position*)

inserts the word in the correct position of the subframe

8.115.4 Member Data Documentation

8.115.4.1 Gps_Almanac GpsL1CaSubframeFsm::almanac

Object that handles GPS almanac data.

Definition at line 78 of file gps_l1_ca_subframe_fsm.h.

8.115.4.2 Gps_Navigation_Message GpsL1CaSubframeFsm::d_nav

GPS L1 C/A navigation message object.

Definition at line 74 of file gps_l1_ca_subframe_fsm.h.

8.115.4.3 Gps_Ephemeris GpsL1CaSubframeFsm::ephemeris

Object that handles GPS ephemeris parameters.

Definition at line 77 of file gps_l1_ca_subframe_fsm.h.

8.115.4.4 int GpsL1CaSubframeFsm::i_channel_ID

Channel id.

Definition at line 71 of file gps_l1_ca_subframe_fsm.h.

8.115.4.5 unsigned int GpsL1CaSubframeFsm::i_satellite_PRN

Satellite PRN number.

Definition at line 72 of file gps_l1_ca_subframe_fsm.h.

8.115.4.6 Gps_Iono GpsL1CaSubframeFsm::iono

Object that handles ionospheric parameters.

Definition at line 80 of file gps_l1_ca_subframe_fsm.h.

8.115.4.7 Gps_Utc_Model GpsL1CaSubframeFsm::utc_model

Object that handles UTM model parameters.

Definition at line 79 of file gps_l1_ca_subframe_fsm.h.

The documentation for this class was generated from the following file:

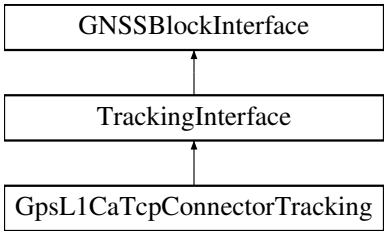
- src/algorithms/telemetry_decoder/libs/gps_l1_ca_subframe_fsm.h

8.116 GpsL1CaTcpConnectorTracking Class Reference

This class implements a code DLL + carrier PLL tracking loop.

`#include <gps_l1_ca_tcp_connector_tracking.h>`

Inheritance diagram for GpsL1CaTcpConnectorTracking:



Public Member Functions

- **GpsL1CaTcpConnectorTracking** (ConfigurationInterface *configuration, std::string role, unsigned int in_↩ streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_TCP_CONNECTOR_Tracking".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.116.1 Detailed Description

This class implements a code DLL + carrier PLL tracking loop.

Definition at line 51 of file gps_l1_ca_tcp_connector_tracking.h.

8.116.2 Member Function Documentation

8.116.2.1 std::string GpsL1CaTcpConnectorTracking::implementation ( ) [inline], [virtual]

Returns "GPS_L1_CA_TCP_CONNECTOR_Tracking".

Implements GNSSBlockInterface.

Definition at line 69 of file gps_l1_ca_tcp_connector_tracking.h.

References set_channel(), and set_gnss_synchro().

8.116.2.2 void GpsL1CaTcpConnectorTracking::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.116.2.3 void GpsL1CaTcpConnectorTracking::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

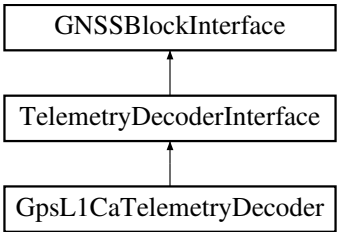
- src/algorithms/tracking/adapters/gps_l1_ca_tcp_connector_tracking.h

8.117 GpsL1CaTelemetryDecoder Class Reference

This class implements a NAV data decoder for GPS L1 C/A.

`#include <gps_l1_ca_telemetry_decoder.h>`

Inheritance diagram for GpsL1CaTelemetryDecoder:



Public Member Functions

- **GpsL1CaTelemetryDecoder** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L1_CA_Telemetry_Decoder".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **set_satellite** (Gnss_Satellite satellite)
- void **set_channel** (int channel)
- void **reset** ()
- size_t **item_size** ()

8.117.1 Detailed Description

This class implements a NAV data decoder for GPS L1 C/A.

Definition at line 46 of file gps_l1_ca_telemetry_decoder.h.

8.117.2 Member Function Documentation

8.117.2.1 std::string GpsL1CaTelemetryDecoder::implementation ( ) [inline],[virtual]

Returns "GPS_L1_CA_Telemetry_Decoder".

Implements GNSSBlockInterface.

Definition at line 61 of file gps_l1_ca_telemetry_decoder.h.

The documentation for this class was generated from the following file:

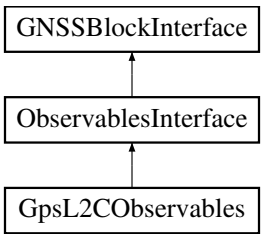
- src/algorithms/telemetry_decoder/adapters/gps_l1_ca_telemetry_decoder.h

8.118 GpsL2CObservables Class Reference

This class implements an ObservablesInterface for GPS L2C.

`#include <gps_l2c_observables.h>`

Inheritance diagram for GpsL2CObservables:



Public Member Functions

- **GpsL2CObservables** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L2C_Observables".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation.

8.118.1 Detailed Description

This class implements an ObservablesInterface for GPS L2C.

Definition at line 46 of file gps_l2c_observables.h.

8.118.2 Member Function Documentation

8.118.2.1 std::string GpsL2CObservables::implementation ( ) [inline], [virtual]

Returns "GPS_L2C_Observables".

Implements GNSSBlockInterface.

Definition at line 60 of file gps_l2c_observables.h.

8.118.2.2 size_t GpsL2CObservables::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation.

Implements GNSSBlockInterface.

Definition at line 74 of file gps_l2c_observables.h.

The documentation for this class was generated from the following file:

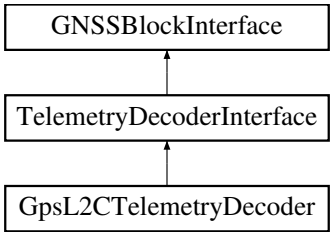
- src/algorithms/observables/adapters/gps_l2c_observables.h

8.119 GpsL2CTelemetryDecoder Class Reference

This class implements a NAV data decoder for GPS L2 M.

`#include <gps_l2c_telemetry_decoder.h>`

Inheritance diagram for GpsL2CTelemetryDecoder:



Public Member Functions

- **GpsL2CTelemetryDecoder** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L2C_Telemetry_Decoder".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **set_satellite** (Gnss_Satellite satellite)
- void **set_channel** (int channel)
- void **reset** ()
- size_t **item_size** ()

8.119.1 Detailed Description

This class implements a NAV data decoder for GPS L2 M.

Definition at line 46 of file gps_l2c_telemetry_decoder.h.

8.119.2 Member Function Documentation

8.119.2.1 std::string GpsL2CTelemetryDecoder::implementation ( ) [inline], [virtual]

Returns "GPS_L2C_Telemetry_Decoder".

Implements GNSSBlockInterface.

Definition at line 61 of file gps_l2c_telemetry_decoder.h.

The documentation for this class was generated from the following file:

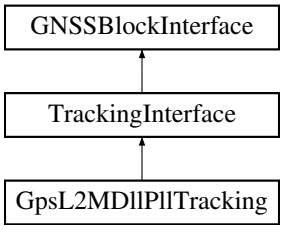
- src/algorithms/telemetry_decoder/adapters/gps_l2c_telemetry_decoder.h

8.120 GpsL2MDllPllTracking Class Reference

This class implements a code DLL + carrier PLL tracking loop.

`#include <gps_l2_m_dll_pll_tracking.h>`

Inheritance diagram for GpsL2MDllPllTracking:



Public Member Functions

- **GpsL2MDllPllTracking** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L2_M_DLL_PLL_Tracking".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_channel (unsigned int channel)

  Set tracking channel unique ID.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void **start_tracking** ()

8.120.1 Detailed Description

This class implements a code DLL + carrier PLL tracking loop.

Definition at line 51 of file gps_l2_m_dll_pll_tracking.h.

8.120.2 Member Function Documentation

8.120.2.1 std::string GpsL2MDllPllTracking::implementation ( ) [inline], [virtual]

Returns "GPS_L2_M_DLL_PLL_Tracking".

Implements GNSSBlockInterface.

Definition at line 67 of file gps_l2_m_dll_pll_tracking.h.

References set_channel(), and set_gnss_synchro().

8.120.2.2 void GpsL2MDllPllTracking::set_channel (unsigned int *channel*) [virtual]

Set tracking channel unique ID.

Implements TrackingInterface.

Referenced by implementation().

8.120.2.3 void GpsL2MDllPllTracking::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements TrackingInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

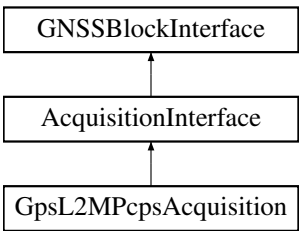
- src/algorithms/tracking/adapters/gps_l2_m_dll_pll_tracking.h

8.121 GpsL2MPcpsAcquisition Class Reference

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L2 M signals.

`#include <gps_l2_m_pcps_acquisition.h>`

Inheritance diagram for GpsL2MPcpsAcquisition:



Public Member Functions

- **GpsL2MPcpsAcquisition** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "GPS_L2_M_PCPS_Acquisition".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler off grid search.

- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code ()

  Sets local code for GPS L2/M PCPS acquisition algorithm.
- signed int mag ()

  Returns the maximum peak of grid search.
- void reset ()

  Restart acquisition algorithm.
- void set_state (int state)

  If state = 1, it forces the block to start acquiring from the first sample.

8.121.1 Detailed Description

This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L2 M signals.

Definition at line 55 of file gps_l2_m_pcps_acquisition.h.

8.121.2 Member Function Documentation

8.121.2.1 std::string GpsL2MPcpsAcquisition::implementation ( ) [inline], [virtual]

Returns "GPS_L2_M_PCPS_Acquisition".

Implements GNSSBlockInterface.

Definition at line 72 of file gps_l2_m_pcps_acquisition.h.

References init(), mag(), reset(), set_channel(), set_doppler_max(), set_doppler_step(), set_gnss_synchro(), set↩_local_code(), set_state(), and set_threshold().

8.121.2.2 void GpsL2MPcpsAcquisition::init ( ) [virtual]

Initializes acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.3 signed int GpsL2MPcpsAcquisition::mag ( ) [virtual]

Returns the maximum peak of grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.4 void GpsL2MPcpsAcquisition::reset ( ) [virtual]

Restart acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.5 void GpsL2MPcpsAcquisition::set_channel (unsigned int *channel*) [virtual]

Set acquisition channel unique ID.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.6 void GpsL2MPcpsAcquisition::set_doppler_max (unsigned int *doppler_max*) [virtual]

Set maximum Doppler off grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.7 void GpsL2MPcpsAcquisition::set_doppler_step (unsigned int *doppler_step*) [virtual]

Set Doppler steps for the grid search.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.8 void GpsL2MPcpsAcquisition::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [virtual]

Set acquisition/tracking common Gnss_Synchro object pointer to efficiently exchange synchronization data between acquisition and tracking blocks.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.9 void GpsL2MPcpsAcquisition::set_local_code ( ) [virtual]

Sets local code for GPS L2/M PCPS acquisition algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

8.121.2.10 void GpsL2MPcpsAcquisition::set_state (int *state*)

If state = 1, it forces the block to start acquiring from the first sample.

Referenced by implementation().

8.121.2.11 void GpsL2MPcpsAcquisition::set_threshold (float *threshold*) [virtual]

Set statistics threshold of PCPS algorithm.

Implements AcquisitionInterface.

Referenced by implementation().

The documentation for this class was generated from the following file:

- src/algorithms/acquisition/adapters/gps_l2_m_pcps_acquisition.h

8.122 GPU_Complex Struct Reference

Public Member Functions

- CUDA_CALLABLE_MEMBER_DEVICE **GPU_Complex** (float a, float b)
- CUDA_CALLABLE_MEMBER_DEVICE float **magnitude2** (void)
- CUDA_CALLABLE_MEMBER_DEVICE GPU_Complex **operator*** (const GPU_Complex &a)
- CUDA_CALLABLE_MEMBER_DEVICE GPU_Complex **operator+** (const GPU_Complex &a)
- CUDA_CALLABLE_MEMBER_DEVICE void **operator+=** (const GPU_Complex &a)
- CUDA_CALLABLE_MEMBER_DEVICE void **multiply_acc** (const GPU_Complex &a, const GPU_Complex &b)

Public Attributes

- float **r**
- float **i**

8.122.1 Detailed Description

Definition at line 53 of file cuda_multicorrelator.h.

The documentation for this struct was generated from the following file:

- src/algorithms/tracking/libs/cuda_multicorrelator.h

8.123 GPU_Complex_Short Struct Reference

Public Member Functions

- CUDA_CALLABLE_MEMBER_DEVICE **GPU_Complex_Short** (short int a, short int b)
- CUDA_CALLABLE_MEMBER_DEVICE float **magnitude2** (void)
- CUDA_CALLABLE_MEMBER_DEVICE GPU_Complex_Short **operator*** (const GPU_Complex_Short &a)
- CUDA_CALLABLE_MEMBER_DEVICE GPU_Complex_Short **operator+** (const GPU_Complex_Short &a)

Public Attributes

- float **r**
- float **i**

8.123.1 Detailed Description

Definition at line 92 of file cuda_multicorrelator.h.

The documentation for this struct was generated from the following file:

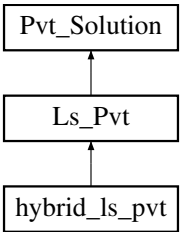
- src/algorithms/tracking/libs/cuda_multicorrelator.h

8.124 hybrid_ls_pvt Class Reference

This class implements a simple PVT Least Squares solution.

`#include <hybrid_ls_pvt.h>`

Inheritance diagram for hybrid_ls_pvt:



Public Member Functions

- **hybrid_ls_pvt** (int nchannels, std::string dump_filename, bool flag_dump_to_file)
- bool **get_PVT** (std::map< int, Gnss_Synchro > gnss_observables_map, double hybrid_current_time, bool flag_averaging)

Public Attributes

- int d_nchannels

  Number of available channels for positioning.
- std::map< int, Galileo_Ephemeris > galileo_ephemeris_map

  Map storing new Galileo_Ephemeris.
- std::map< int, Gps_Ephemeris > gps_ephemeris_map

  Map storing new GPS_Ephemeris.
- std::map< int, Gps_CNAV_Ephemeris > **gps_cnav_ephemeris_map**
- Galileo_Utc_Model **galileo_utc_model**
- Galileo_Iono **galileo_iono**
- Galileo_Almanac **galileo_almanac**
- Gps_Utc_Model **gps_utc_model**
- Gps_Iono **gps_iono**
- Gps_CNAV_Iono **gps_cnav_iono**
- Gps_CNAV_Utc_Model **gps_cnav_utc_model**
- double **d_galileo_current_time**
- int **count_valid_position**
- bool **d_flag_dump_enabled**
- bool **d_flag_averaging**
- std::string **d_dump_filename**
- std::ofstream **d_dump_file**

8.124.1 Detailed Description

This class implements a simple PVT Least Squares solution.

Definition at line 49 of file hybrid_ls_pvt.h.

8.124.2 Member Data Documentation

8.124.2.1 int hybrid_ls_pvt::d_nchannels

Number of available channels for positioning.

Definition at line 56 of file hybrid_ls_pvt.h.

8.124.2.2 std::map<int,Galileo_Ephemeris> hybrid_ls_pvt::galileo_ephemeris_map

Map storing new Galileo_Ephemeris.

Definition at line 58 of file hybrid_ls_pvt.h.

8.124.2.3 std::map<int,Gps_Ephemeris> hybrid_ls_pvt::gps_ephemeris_map

Map storing new GPS_Ephemeris.

Definition at line 59 of file hybrid_ls_pvt.h.

The documentation for this class was generated from the following file:

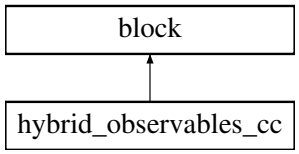
- src/algorithms/PVT/libs/hybrid_ls_pvt.h

8.125 hybrid_observables_cc Class Reference

This class implements a block that computes Galileo observables.

`#include <hybrid_observables_cc.h>`

Inheritance diagram for hybrid_observables_cc:



Public Member Functions

- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- hybrid_observables_cc_sptr **hybrid_make_observables_cc** (unsigned int nchannels, bool dump, std::string dump_filename, unsigned int deep_history)

8.125.1 Detailed Description

This class implements a block that computes Galileo observables.

Definition at line 51 of file hybrid_observables_cc.h.

The documentation for this class was generated from the following file:

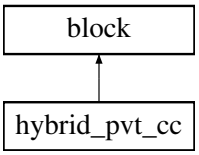
- src/algorithms/observables/gnuradio_blocks/hybrid_observables_cc.h

8.126 hybrid_pvt_cc Class Reference

This class implements a block that computes the PVT solution with Galileo E1 signals.

`#include <hybrid_pvt_cc.h>`

Inheritance diagram for hybrid_pvt_cc:



Public Member Functions

- std::map< int, Gps_Ephemeris > get_GPS_L1_ephemeris_map ()

  Get latest set of GPS L1 ephemeris from PVT block.
- ~hybrid_pvt_cc ()

  Default destructor.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  PVT Signal Processing.

Friends

- hybrid_pvt_cc_sptr **hybrid_make_pvt_cc** (unsigned int nchannels, bool dump, std::string dump_filename, int averaging_depth, bool flag_averaging, int output_rate_ms, int display_rate_ms, bool flag_nmea_tty_port, std::string nmea_dump_filename, std::string nmea_dump_devname, bool flag_rtcm_server, bool flag_rtcm↩_tty_port, unsigned short rtcm_tcp_port, unsigned short rtcm_station_id, std::map< int, int > rtcm_msg_↩rate_ms, std::string rtcm_dump_devname, const unsigned int type_of_receiver)

8.126.1 Detailed Description

This class implements a block that computes the PVT solution with Galileo E1 signals.

Definition at line 74 of file hybrid_pvt_cc.h.

8.126.2 Constructor & Destructor Documentation

8.126.2.1 hybrid_pvt_cc::~hybrid_pvt_cc ( )

Default destructor.

8.126.3 Member Function Documentation

8.126.3.1 int hybrid_pvt_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

PVT Signal Processing.

8.126.3.2 std::map<int,Gps_Ephemeris> hybrid_pvt_cc::get_GPS_L1_ephemeris_map ( )

Get latest set of GPS L1 ephemeris from PVT block.

It is used to save the assistance data at the receiver shutdown

The documentation for this class was generated from the following file:

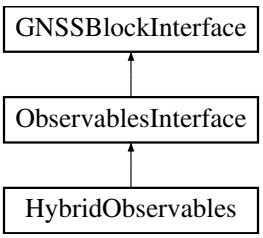
- src/algorithms/PVT/gnuradio_blocks/hybrid_pvt_cc.h

8.127 HybridObservables Class Reference

This class implements an ObservablesInterface for observables of all kind of GNSS signals.

`#include <hybrid_observables.h>`

Inheritance diagram for HybridObservables:



Public Member Functions

- **HybridObservables** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Hybrid_Observables".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation.

8.127.1 Detailed Description

This class implements an ObservablesInterface for observables of all kind of GNSS signals.

Definition at line 47 of file hybrid_observables.h.

8.127.2 Member Function Documentation

8.127.2.1 std::string HybridObservables::implementation ( ) [inline], [virtual]

Returns "Hybrid_Observables".

Implements GNSSBlockInterface.

Definition at line 61 of file hybrid_observables.h.

8.127.2.2 size_t HybridObservables::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation.

Implements GNSSBlockInterface.

Definition at line 75 of file hybrid_observables.h.

The documentation for this class was generated from the following file:

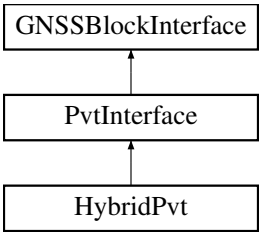
- src/algorithms/observables/adapters/hybrid_observables.h

8.128 HybridPvt Class Reference

This class implements a PvtInterface for Galileo E1.

`#include <hybrid_pvt.h>`

Inheritance diagram for HybridPvt:



Public Member Functions

- **HybridPvt** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Hybrid_Pvt".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **reset** ()
- size_t item_size ()

  All blocks must have an item_size() function implementation. Returns sizeof(gr_complex)

8.128.1 Detailed Description

This class implements a PvtInterface for Galileo E1.

Definition at line 47 of file hybrid_pvt.h.

8.128.2 Member Function Documentation

8.128.2.1 std::string HybridPvt::implementation ( ) [inline], [virtual]

Returns "Hybrid_Pvt".

Implements GNSSBlockInterface.

Definition at line 63 of file hybrid_pvt.h.

8.128.2.2 size_t HybridPvt::item_size ( ) [inline], [virtual]

All blocks must have an item_size() function implementation. Returns sizeof(gr_complex)

Implements GNSSBlockInterface.

Definition at line 79 of file hybrid_pvt.h.

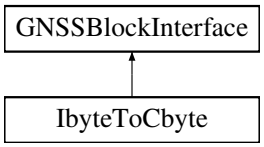
The documentation for this class was generated from the following file:

- src/algorithms/PVT/adapters/hybrid_pvt.h

8.129 IbyteToCbyte Class Reference

`#include <ibyte_to_cbyte.h>`

Inheritance diagram for IbyteToCbyte:



Public Member Functions

- **IbyteToCbyte** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Ibyte_To_Cbyte".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.129.1 Detailed Description

an I/Q interleaved byte (unsigned char) sample stream into a std::complex<unsigned char> stream

Definition at line 47 of file ibyte_to_cbyte.h.

8.129.2 Member Function Documentation

8.129.2.1 std::string IbyteToCbyte::implementation ( ) [inline], [virtual]

Returns "Ibyte_To_Cbyte".

Implements GNSSBlockInterface.

Definition at line 61 of file ibyte_to_cbyte.h.

The documentation for this class was generated from the following file:

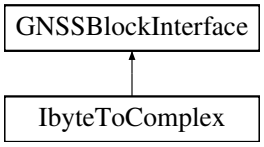
- src/algorithms/data_type_adapter/adapters/ibyte_to_cbyte.h

8.130 IbyteToComplex Class Reference

Adapts an I/Q interleaved byte integer sample stream to a gr_complex (float) stream.

`#include <ibyte_to_complex.h>`

Inheritance diagram for IbyteToComplex:



Public Member Functions

- **IbyteToComplex** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Ibyte_To_Complex".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.130.1 Detailed Description

Adapts an I/Q interleaved byte integer sample stream to a gr_complex (float) stream.

Definition at line 47 of file ibyte_to_complex.h.

8.130.2 Member Function Documentation

8.130.2.1 std::string IbyteToComplex::implementation ( ) [inline], [virtual]

Returns "Ibyte_To_Complex".

Implements GNSSBlockInterface.

Definition at line 61 of file ibyte_to_complex.h.

The documentation for this class was generated from the following file:

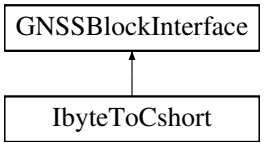
- src/algorithms/data_type_adapter/adapters/ibyte_to_complex.h

8.131 IbyteToCshort Class Reference

Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream.

`#include <ibyte_to_cshort.h>`

Inheritance diagram for IbyteToCshort:



Public Member Functions

- **IbyteToCshort** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Ibyte_To_Cshort".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.131.1 Detailed Description

Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream.

Definition at line 47 of file ibyte_to_cshort.h.

8.131.2 Member Function Documentation

8.131.2.1 std::string IbyteToCshort::implementation ( ) [inline], [virtual]

Returns "Ibyte_To_Cshort".

Implements GNSSBlockInterface.

Definition at line 61 of file ibyte_to_cshort.h.

The documentation for this class was generated from the following file:

- src/algorithms/data_type_adapter/adapters/ibyte_to_cshort.h

8.132 Igp Struct Reference

Struct that represents a Ionospheric Grid Point (IGP)

`#include <sbas_ionospheric_correction.h>`

Public Attributes

- double **t0**
- int **d_latitude**
- int **d_longitude**
- int **d_give**
- double **d_delay**

Friends

- class **boost::serialization::access**

8.132.1 Detailed Description

Struct that represents a Ionospheric Grid Point (IGP)

Definition at line 47 of file sbas_ionospheric_correction.h.

The documentation for this struct was generated from the following file:

- src/core/system_parameters/sbas_ionospheric_correction.h

8.133 Igp_Band Struct Reference

Struct that represents the band of a Ionospheric Grid Point (IGP)

`#include <sbas_ionospheric_correction.h>`

Public Attributes

- std::vector< Igp > **d_igps**

Friends

- class **boost::serialization::access**

8.133.1 Detailed Description

Struct that represents the band of a Ionospheric Grid Point (IGP)

Definition at line 73 of file sbas_ionospheric_correction.h.

The documentation for this struct was generated from the following file:

- src/core/system_parameters/sbas_ionospheric_correction.h

8.134 INIReader Class Reference

Read an INI file into easy-to-access name/value pairs. (Note that I've gone for simplicity here rather than speed, but it should be pretty decent.)

`#include <INIReader.h>`

Public Member Functions

- INIReader (std::string filename)

  Construct INIReader and parse given filename. See ini.h for more info about the parsing.
- int ParseError ()

  Return the result of ini_parse(), i.e., 0 on success, line number of first error on parse error, or -1 on file open error.
- std::string Get (std::string section, std::string name, std::string default_value)

  Get a string value from INI file, returning default_value if not found.
- long GetInteger (std::string section, std::string name, long default_value)

  Get an integer (long) value from INI file, returning default_value if not found.

8.134.1 Detailed Description

Read an INI file into easy-to-access name/value pairs. (Note that I've gone for simplicity here rather than speed, but it should be pretty decent.)

Definition at line 58 of file INIReader.h.

8.134.2 Constructor & Destructor Documentation

8.134.2.1 INIReader::INIReader (std::string *filename*)

Construct INIReader and parse given filename. See ini.h for more info about the parsing.

8.134.3 Member Function Documentation

8.134.3.1 std::string INIReader::Get (std::string *section,* std::string *name,* std::string *default_value*)

Get a string value from INI file, returning default_value if not found.

8.134.3.2 long INIReader::GetInteger (std::string *section,* std::string *name,* long *default_value*)

Get an integer (long) value from INI file, returning default_value if not found.

8.134.3.3 int INIReader::ParseError ( )

Return the result of ini_parse(), i.e., 0 on success, line number of first error on parse error, or -1 on file open error.

The documentation for this class was generated from the following file:

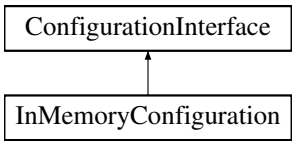
- src/core/libs/INIReader.h

8.135 InMemoryConfiguration Class Reference

This class is an implementation of the interface ConfigurationInterface.

`#include <in_memory_configuration.h>`

Inheritance diagram for InMemoryConfiguration:



Public Member Functions

- std::string **property** (std::string property_name, std::string default_value)
- bool **property** (std::string property_name, bool default_value)
- long **property** (std::string property_name, long default_value)
- int **property** (std::string property_name, int default_value)
- unsigned int **property** (std::string property_name, unsigned int default_value)
- unsigned short **property** (std::string property_name, unsigned short default_value)
- float **property** (std::string property_name, float default_value)
- double **property** (std::string property_name, double default_value)
- void **set_property** (std::string property_name, std::string value)
- bool **is_present** (std::string property_name)

8.135.1 Detailed Description

This class is an implementation of the interface ConfigurationInterface.

This implementation accepts configuration parameters upon instantiation and it is intended to be used in unit testing.

Definition at line 52 of file in_memory_configuration.h.

The documentation for this class was generated from the following file:

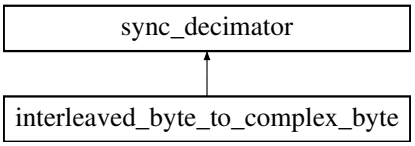
- src/core/receiver/in_memory_configuration.h

8.136 interleaved_byte_to_complex_byte Class Reference

This class adapts an 8-bits interleaved sample stream into a 16-bits complex stream (std::complex<unsigned char>)

`#include <interleaved_byte_to_complex_byte.h>`

Inheritance diagram for interleaved_byte_to_complex_byte:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- interleaved_byte_to_complex_byte_sptr **make_interleaved_byte_to_complex_byte** ()

8.136.1 Detailed Description

This class adapts an 8-bits interleaved sample stream into a 16-bits complex stream (std::complex<unsigned char>)

Definition at line 48 of file interleaved_byte_to_complex_byte.h.

The documentation for this class was generated from the following file:

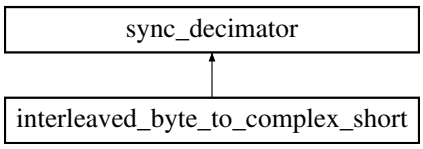
- src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_byte.h

8.137 interleaved_byte_to_complex_short Class Reference

This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

`#include <interleaved_byte_to_complex_short.h>`

Inheritance diagram for interleaved_byte_to_complex_short:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- interleaved_byte_to_complex_short_sptr **make_interleaved_byte_to_complex_short** ()

8.137.1 Detailed Description

This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

Definition at line 47 of file interleaved_byte_to_complex_short.h.

The documentation for this class was generated from the following file:

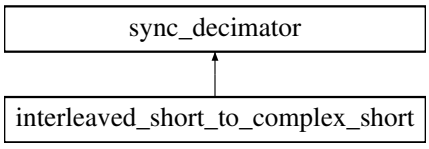
- src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_short.h

8.138 interleaved_short_to_complex_short Class Reference

This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

`#include <interleaved_short_to_complex_short.h>`

Inheritance diagram for interleaved_short_to_complex_short:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- interleaved_short_to_complex_short_sptr **make_interleaved_short_to_complex_short** ()

8.138.1 Detailed Description

This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

Definition at line 47 of file interleaved_short_to_complex_short.h.

The documentation for this class was generated from the following file:

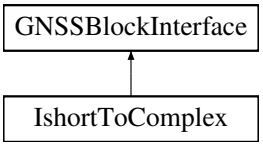
- src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_short_to_complex_short.h

8.139 IshortToComplex Class Reference

Adapts an I/Q interleaved short integer sample stream to a gr_complex (float) stream.

`#include <ishort_to_complex.h>`

Inheritance diagram for IshortToComplex:



Public Member Functions

- **IshortToComplex** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Ishort_To_Complex".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.139.1 Detailed Description

Adapts an I/Q interleaved short integer sample stream to a gr_complex (float) stream.

Definition at line 46 of file ishort_to_complex.h.

8.139.2 Member Function Documentation

8.139.2.1 std::string IshortToComplex::implementation ( ) [inline], [virtual]

Returns "Ishort_To_Complex".

Implements GNSSBlockInterface.

Definition at line 60 of file ishort_to_complex.h.

The documentation for this class was generated from the following file:

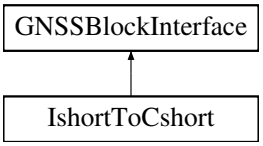
- src/algorithms/data_type_adapter/adapters/ishort_to_complex.h

8.140 IshortToCshort Class Reference

Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream.

`#include <ishort_to_cshort.h>`

Inheritance diagram for IshortToCshort:



Public Member Functions

- **IshortToCshort** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "Ishort_To_Cshort".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.140.1 Detailed Description

Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream.

Definition at line 47 of file ishort_to_cshort.h.

8.140.2 Member Function Documentation

8.140.2.1 std::string IshortToCshort::implementation ( ) [inline], [virtual]

Returns "Ishort_To_Cshort".

Implements GNSSBlockInterface.

Definition at line 61 of file ishort_to_cshort.h.

The documentation for this class was generated from the following file:

- src/algorithms/data_type_adapter/adapters/ishort_to_cshort.h

8.141 kernel_info_t Struct Reference

Public Attributes

- cl_kernel **kernel**
- char * **kernel_name**
- unsigned **lmem_size**
- unsigned **num_workgroups**
- unsigned **num_xforms_per_workgroup**
- unsigned **num_workitems_per_workgroup**
- cl_fft_kernel_dir **dir**
- int **in_place_possible**
- kernel_info_t * **next**

8.141.1 Detailed Description

Definition at line 66 of file fft_internal.h.

The documentation for this struct was generated from the following file:

- src/algorithms/libs/fft_internal.h

8.142 Kml_Printer Class Reference

Prints PVT information to OGC KML format file (can be viewed with Google Earth)

```
#include <kml_printer.h>
```

Public Member Functions

- bool **set_headers** (std::string filename, bool time_tag_name=true)
- bool **print_position** (const std::shared_ptr< Pvt_Solution > &position, bool print_average_values)
- bool **close_file** ()

8.142.1 Detailed Description

Prints PVT information to OGC KML format file (can be viewed with Google Earth)

See http://www.opengeospatial.org/standards/kml

Definition at line 47 of file kml_printer.h.

The documentation for this class was generated from the following file:

- src/algorithms/PVT/libs/kml_printer.h

8.143 Kml_Printer_gpstk Class Reference

Prints PVT information to OGC KML format file (can be viewed with Google Earth)

```
#include <kml_printer_gpstk.h>
```

Public Member Functions

- bool **set_headers** (std::string filename)
- bool **print_position** (gpstk::Position position)
- bool **close_file** ()

8.143.1 Detailed Description

Prints PVT information to OGC KML format file (can be viewed with Google Earth)

See http://www.opengeospatial.org/standards/kml

Definition at line 47 of file kml_printer_gpstk.h.

The documentation for this class was generated from the following file:

- src/utils/gpstk/gnsspvt/src/kml_printer_gpstk.h

8.144 Long_Term_Correction Struct Reference

Public Attributes

- double d_trx
  Time when message was received.
- int i_tapp
  Time of applicability (only valid if vel=1, equals the sent t0)
- int i_vel
  Use velocity corrections if vel=1.
- int **d_iode**
- double d_dpos [3]
  position correction
- double d_dvel [3]
  velocity correction
- double d_daf0
  clock offset correction
- double d_daf1
  clock drift correction

8.144.1 Detailed Description

Definition at line 48 of file sbas_satellite_correction.h.

8.144.2 Member Data Documentation

8.144.2.1 double Long_Term_Correction::d_daf0

clock offset correction

Definition at line 56 of file sbas_satellite_correction.h.

8.144.2.2 double Long_Term_Correction::d_daf1

clock drift correction

Definition at line 57 of file sbas_satellite_correction.h.

8.144.2.3 double Long_Term_Correction::d_dpos[3]

position correction

Definition at line 54 of file sbas_satellite_correction.h.

8.144.2.4 double Long_Term_Correction::d_dvel[3]

velocity correction

Definition at line 55 of file sbas_satellite_correction.h.

8.144.2.5 double Long_Term_Correction::d_trx

Time when message was received.

Definition at line 50 of file sbas_satellite_correction.h.

8.144.2.6 int Long_Term_Correction::i_tapp

Time of applicability (only valid if vel=1, equals the sent t0)

Definition at line 51 of file sbas_satellite_correction.h.

8.144.2.7 int Long_Term_Correction::i_vel

Use velocity corrections if vel=1.

Definition at line 52 of file sbas_satellite_correction.h.

The documentation for this struct was generated from the following file:

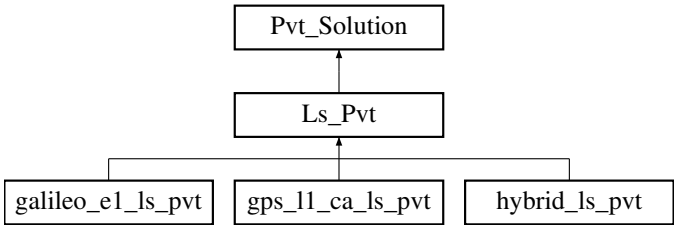
- src/core/system_parameters/sbas_satellite_correction.h

8.145 Ls_Pvt Class Reference

Base class for the Least Squares PVT solution.

`#include <ls_pvt.h>`

Inheritance diagram for Ls_Pvt:



Public Member Functions

- arma::vec bancroftPos (const arma::mat &satpos, const arma::vec &obs)
  Computes the initial position solution based on the Bancroft algorithm.
- arma::vec leastSquarePos (const arma::mat &satpos, const arma::vec &obs, const arma::vec &w_vec)
  Computes the Weighted Least Squares position solution.

Additional Inherited Members

8.145.1 Detailed Description

Base class for the Least Squares PVT solution.

Definition at line 42 of file ls_pvt.h.

8.145.2 Member Function Documentation

8.145.2.1 arma::vec Ls_Pvt::bancroftPos (const arma::mat & *satpos,* const arma::vec & *obs*)

Computes the initial position solution based on the Bancroft algorithm.

8.145.2.2 arma::vec Ls_Pvt::leastSquarePos (const arma::mat & *satpos,* const arma::vec & *obs,* const arma::vec & *w_vec*)

Computes the Weighted Least Squares position solution.

The documentation for this class was generated from the following file:

- src/algorithms/PVT/libs/ls_pvt.h

8.146 Nmea_Printer Class Reference

This class provides a implementation of a subset of the NMEA-0183 standard for interfacing marine electronic devices as defined by the National Marine Electronics Association (NMEA).

`#include <nmea_printer.h>`

Public Member Functions

- Nmea_Printer (std::string filename, bool flag_nmea_tty_port, std::string nmea_dump_filename)
  Default constructor.
- bool Print_Nmea_Line (const std::shared_ptr< Pvt_Solution > &position, bool print_average_values)
  Print NMEA PVT and satellite info to the initialized device.
- ~Nmea_Printer ()
  Default destructor.

8.146.1 Detailed Description

This class provides a implementation of a subset of the NMEA-0183 standard for interfacing marine electronic devices as defined by the National Marine Electronics Association (NMEA).

See http://en.wikipedia.org/wiki/NMEA_0183

Definition at line 52 of file nmea_printer.h.

8.146.2 Constructor & Destructor Documentation

8.146.2.1 Nmea_Printer::Nmea_Printer (std::string *filename,* bool *flag_nmea_tty_port,* std::string *nmea_dump_filename*)

Default constructor.

8.146.2.2 Nmea_Printer::~Nmea_Printer ( )

Default destructor.

8.146.3 Member Function Documentation

8.146.3.1 bool Nmea_Printer::Print_Nmea_Line (const std::shared_ptr< Pvt_Solution > & *position,* bool *print_average_values*)

Print NMEA PVT and satellite info to the initialized device.

The documentation for this class was generated from the following file:

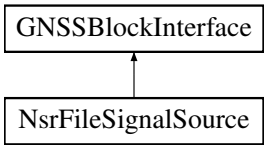
- src/algorithms/PVT/libs/nmea_printer.h

8.147 NsrFileSignalSource Class Reference

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

`#include <nsr_file_signal_source.h>`

Inheritance diagram for NsrFileSignalSource:



Public Member Functions

- **NsrFileSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "Nsr_File_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **filename** ()
- std::string **item_type** ()
- bool **repeat** ()
- long **sampling_frequency** ()
- long **samples** ()

8.147.1 Detailed Description

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

Definition at line 54 of file nsr_file_signal_source.h.

8.147.2 Member Function Documentation

8.147.2.1 std::string NsrFileSignalSource::implementation ( ) [inline], [virtual]

Returns "Nsr_File_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 70 of file nsr_file_signal_source.h.

The documentation for this class was generated from the following file:

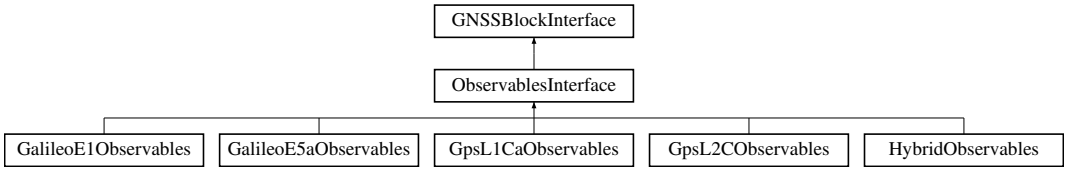
- src/algorithms/signal_source/adapters/nsr_file_signal_source.h

8.148 ObservablesInterface Class Reference

This abstract class represents an interface to an observables block.

`#include <observables_interface.h>`

Inheritance diagram for ObservablesInterface:



Public Member Functions

- virtual void **reset** ()=0

8.148.1 Detailed Description

This abstract class represents an interface to an observables block.

Abstract class for pseudorange_intefaces, derived from GNSSBlockInterface. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 51 of file observables_interface.h.

The documentation for this class was generated from the following file:

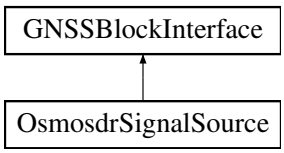
- src/core/interfaces/observables_interface.h

8.149 OsmosdrSignalSource Class Reference

This class reads samples OsmoSDR-compatible front-ends, such as HackRF or Realtek's RTL2832U-based USB dongle DVB-T receivers (see http://sdr.osmocom.org/trac/wiki/rtl-sdr)

`#include <osmosdr_signal_source.h>`

Inheritance diagram for OsmosdrSignalSource:



Public Member Functions

- **OsmosdrSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "Osmosdr_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.149.1 Detailed Description

This class reads samples OsmoSDR-compatible front-ends, such as HackRF or Realtek's RTL2832U-based USB dongle DVB-T receivers (see http://sdr.osmocom.org/trac/wiki/rtl-sdr)

Definition at line 50 of file osmosdr_signal_source.h.

8.149.2 Member Function Documentation

8.149.2.1 std::string OsmosdrSignalSource::implementation ( ) [inline], [virtual]

Returns "Osmosdr_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 67 of file osmosdr_signal_source.h.

The documentation for this class was generated from the following file:

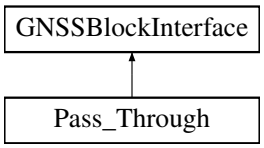
- src/algorithms/signal_source/adapters/osmosdr_signal_source.h

8.150 Pass_Through Class Reference

This class implements a block that connects input and output (does nothing)

`#include <pass_through.h>`

Inheritance diagram for Pass_Through:



Public Member Functions

- **Pass_Through** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream)
- std::string **role** ()
- std::string implementation ()

  returns "Pass_Through"
- std::string **item_type** ()
- size_t **vector_size** ()
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.150.1 Detailed Description

This class implements a block that connects input and output (does nothing)

Definition at line 46 of file pass_through.h.

8.150.2 Member Function Documentation

8.150.2.1 std::string Pass_Through::implementation ( ) [inline], [virtual]

returns "Pass_Through"

Implements GNSSBlockInterface.

Definition at line 60 of file pass_through.h.

The documentation for this class was generated from the following file:

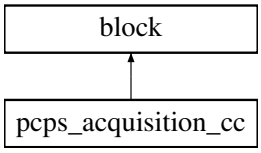
- src/algorithms/libs/pass_through.h

8.151 pcps_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition.

`#include <pcps_acquisition_cc.h>`

Inheritance diagram for pcps_acquisition_cc:



Public Member Functions

- ~pcps_acquisition_cc ()
  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)
  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()
  Returns the maximum peak of grid search.
- void init ()
  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)
  Sets local code for PCPS acquisition algorithm.
- void set_active (bool active)

Starts acquisition algorithm, turning from standby mode to active mode.

- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.

Friends

- pcps_acquisition_cc_sptr **pcps_make_acquisition_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_↩ transition_flag, bool use_CFAR_algorithm_flag, bool dump, std::string dump_filename)

8.151.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Check An Open Source Galileo E1 Software Receiver, Algorithm 1, for a pseudocode description of this implementation.

Definition at line 78 of file pcps_acquisition_cc.h.

8.151.2 Constructor & Destructor Documentation

8.151.2.1 pcps_acquisition_cc::~pcps_acquisition_cc ( )

Default destructor.

8.151.3 Member Function Documentation

8.151.3.1 int pcps_acquisition_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.151.3.2 void pcps_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.151.3.3 unsigned int pcps_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 152 of file pcps_acquisition_cc.h.

References init(), and set_local_code().

8.151.3.4 void pcps_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 173 of file pcps_acquisition_cc.h.

References set_state().

8.151.3.5 void pcps_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 189 of file pcps_acquisition_cc.h.

8.151.3.6 void pcps_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 208 of file pcps_acquisition_cc.h.

8.151.3.7 void pcps_acquisition_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 217 of file pcps_acquisition_cc.h.

References general_work().

8.151.3.8 void pcps_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 144 of file pcps_acquisition_cc.h.

8.151.3.9 void pcps_acquisition_cc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.151.3.10 void pcps_acquisition_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.151.3.11 void pcps_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 199 of file pcps_acquisition_cc.h.

The documentation for this class was generated from the following file:

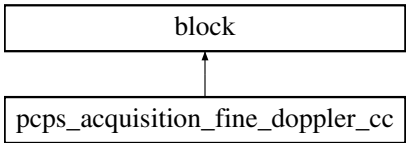
- src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_cc.h

8.152 pcps_acquisition_fine_doppler_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition.

`#include <pcps_acquisition_fine_doppler_cc.h>`

Inheritance diagram for pcps_acquisition_fine_doppler_cc:



Public Member Functions

- ~pcps_acquisition_fine_doppler_cc ()
  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)
  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()
  Returns the maximum peak of grid search.
- void init ()
  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)
  Sets local code for PCPS acquisition algorithm.
- void set_active (bool active)
  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_channel (unsigned int channel)
  Set acquisition channel unique ID.
- void set_threshold (float threshold)
  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)
  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)
  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
  Parallel Code Phase Search Acquisition signal processing.
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- pcps_acquisition_fine_doppler_cc_sptr **pcps_make_acquisition_fine_doppler_cc** (int max_dwells, unsigned int sampled_ms, int doppler_max, int doppler_min, long freq, long fs_in, int samples_per_ms, bool dump, std::string dump_filename)

8.152.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Check An Open Source Galileo E1 Software Receiver, Algorithm 1, for a pseudocode description of this implementation.

Definition at line 75 of file pcps_acquisition_fine_doppler_cc.h.

8.152.2 Constructor & Destructor Documentation

8.152.2.1 pcps_acquisition_fine_doppler_cc::~pcps_acquisition_fine_doppler_cc ( )

Default destructor.

8.152.3 Member Function Documentation

8.152.3.1 int pcps_acquisition_fine_doppler_cc::general_work (int *noutput_items*, gr_vector_int & *ninput_items*, gr_vector_const_void_star & *input_items*, gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_max().

8.152.3.2 void pcps_acquisition_fine_doppler_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.152.3.3 unsigned int pcps_acquisition_fine_doppler_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 158 of file pcps_acquisition_fine_doppler_cc.h.

References init(), and set_local_code().

8.152.3.4 void pcps_acquisition_fine_doppler_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 179 of file pcps_acquisition_fine_doppler_cc.h.

8.152.3.5 void pcps_acquisition_fine_doppler_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 188 of file pcps_acquisition_fine_doppler_cc.h.

8.152.3.6 void pcps_acquisition_fine_doppler_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 207 of file pcps_acquisition_fine_doppler_cc.h.

References general_work(), and set_doppler_step().

8.152.3.7 void pcps_acquisition_fine_doppler_cc::set_doppler_step (unsigned int *doppler_step*)

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Referenced by set_doppler_max().

8.152.3.8 void pcps_acquisition_fine_doppler_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 150 of file pcps_acquisition_fine_doppler_cc.h.

8.152.3.9 void pcps_acquisition_fine_doppler_cc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.152.3.10 void pcps_acquisition_fine_doppler_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 198 of file pcps_acquisition_fine_doppler_cc.h.

The documentation for this class was generated from the following file:

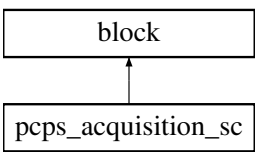
- src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_fine_doppler_cc.h

8.153 pcps_acquisition_sc Class Reference

This class implements a Parallel Code Phase Search Acquisition.

`#include <pcps_acquisition_sc.h>`

Inheritance diagram for pcps_acquisition_sc:



Public Member Functions

- ~pcps_acquisition_sc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)

  Sets local code for PCPS acquisition algorithm.
- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.

Friends

- pcps_acquisition_sc_sptr **pcps_make_acquisition_sc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_↩ transition_flag, bool use_CFAR_algorithm_flag, bool dump, std::string dump_filename)

8.153.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Check An Open Source Galileo E1 Software Receiver, Algorithm 1, for a pseudocode description of this implementation.

Definition at line 78 of file pcps_acquisition_sc.h.

8.153.2 Constructor & Destructor Documentation

8.153.2.1 pcps_acquisition_sc::~pcps_acquisition_sc ( )

Default destructor.

8.153.3 Member Function Documentation

8.153.3.1 int pcps_acquisition_sc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.153.3.2 void pcps_acquisition_sc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.153.3.3 unsigned int pcps_acquisition_sc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 155 of file pcps_acquisition_sc.h.

References init(), and set_local_code().

8.153.3.4 void pcps_acquisition_sc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 176 of file pcps_acquisition_sc.h.

References set_state().

8.153.3.5 void pcps_acquisition_sc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 192 of file pcps_acquisition_sc.h.

8.153.3.6 void pcps_acquisition_sc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 211 of file pcps_acquisition_sc.h.

8.153.3.7 void pcps_acquisition_sc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 220 of file pcps_acquisition_sc.h.

References general_work().

8.153.3.8 void pcps_acquisition_sc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 147 of file pcps_acquisition_sc.h.

8.153.3.9 void pcps_acquisition_sc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.153.3.10 void pcps_acquisition_sc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.153.3.11 void pcps_acquisition_sc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 202 of file pcps_acquisition_sc.h.

The documentation for this class was generated from the following file:

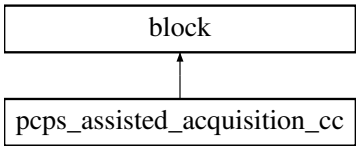
- src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_sc.h

8.154 pcps_assisted_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition.

`#include <pcps_assisted_acquisition_cc.h>`

Inheritance diagram for pcps_assisted_acquisition_cc:



Public Member Functions

- ~pcps_assisted_acquisition_cc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)

  Sets local code for PCPS acquisition algorithm.

- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.
- void **forecast** (int noutput_items, gr_vector_int &ninput_items_required)

Friends

- pcps_assisted_acquisition_cc_sptr **pcps_make_assisted_acquisition_cc** (int max_dwells, unsigned int sampled_ms, int doppler_max, int doppler_min, long freq, long fs_in, int samples_per_ms, bool dump, std↩ ::string dump_filename)

8.154.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Check An Open Source Galileo E1 Software Receiver, Algorithm 1, for a pseudocode description of this implementation.

Definition at line 74 of file pcps_assisted_acquisition_cc.h.

8.154.2 Constructor & Destructor Documentation

8.154.2.1 pcps_assisted_acquisition_cc::∼pcps_assisted_acquisition_cc ( )

Default destructor.

8.154.3 Member Function Documentation

8.154.3.1 int pcps_assisted_acquisition_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_max().

8.154.3.2 void pcps_assisted_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.154.3.3 unsigned int pcps_assisted_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 159 of file pcps_assisted_acquisition_cc.h.

References init(), and set_local_code().

8.154.3.4 void pcps_assisted_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 180 of file pcps_assisted_acquisition_cc.h.

8.154.3.5 void pcps_assisted_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 189 of file pcps_assisted_acquisition_cc.h.

8.154.3.6 void pcps_assisted_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 208 of file pcps_assisted_acquisition_cc.h.

References general_work(), and set_doppler_step().

8.154.3.7 void pcps_assisted_acquisition_cc::set_doppler_step (unsigned int *doppler_step*)

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Referenced by set_doppler_max().

8.154.3.8 void pcps_assisted_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 151 of file pcps_assisted_acquisition_cc.h.

8.154.3.9 void pcps_assisted_acquisition_cc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.154.3.10 void pcps_assisted_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 199 of file pcps_assisted_acquisition_cc.h.

The documentation for this class was generated from the following file:

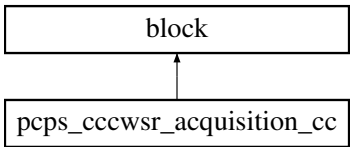
- src/algorithms/acquisition/gnuradio_blocks/pcps_assisted_acquisition_cc.h

8.155 pcps_cccwsr_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition with Coherent Channel Combining With Sign Recovery scheme.

`#include <pcps_cccwsr_acquisition_cc.h>`

Inheritance diagram for pcps_cccwsr_acquisition_cc:



Public Member Functions

- ~pcps_cccwsr_acquisition_cc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code_data, std::complex< float > *code_pilot)

  Sets local code for CCCWSR acquisition algorithm.
- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of CCCWSR algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Coherent Channel Combining With Sign Recovery Acquisition signal processing.

Friends

- pcps_cccwsr_acquisition_cc_sptr **pcps_cccwsr_make_acquisition_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_↩ per_code, bool dump, std::string dump_filename)

8.155.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition with Coherent Channel Combining With Sign Recovery scheme.

Definition at line 62 of file pcps_cccwsr_acquisition_cc.h.

8.155.2 Constructor & Destructor Documentation

8.155.2.1 pcps_cccwsr_acquisition_cc::~pcps_cccwsr_acquisition_cc ( )

Default destructor.

8.155.3 Member Function Documentation

8.155.3.1 int pcps_cccwsr_acquisition_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Coherent Channel Combining With Sign Recovery Acquisition signal processing.

Referenced by set_doppler_step().

8.155.3.2 void pcps_cccwsr_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.155.3.3 unsigned int pcps_cccwsr_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 136 of file pcps_cccwsr_acquisition_cc.h.

References init(), and set_local_code().

8.155.3.4 void pcps_cccwsr_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 158 of file pcps_cccwsr_acquisition_cc.h.

References set_state().

8.155.3.5 void pcps_cccwsr_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 174 of file pcps_cccwsr_acquisition_cc.h.

8.155.3.6 void pcps_cccwsr_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 193 of file pcps_cccwsr_acquisition_cc.h.

8.155.3.7 void pcps_cccwsr_acquisition_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 202 of file pcps_cccwsr_acquisition_cc.h.

References general_work().

8.155.3.8 void pcps_cccwsr_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 128 of file pcps_cccwsr_acquisition_cc.h.

8.155.3.9 void pcps_cccwsr_acquisition_cc::set_local_code (std::complex< float > * *code_data,* std::complex< float > * *code_pilot*)

Sets local code for CCCWSR acquisition algorithm.

Parameters

data_code	- Pointer to the data PRN code.
pilot_code	- Pointer to the pilot PRN code.

Referenced by mag().

8.155.3.10 void pcps_cccwsr_acquisition_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.155.3.11 void pcps_cccwsr_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of CCCWSR algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 184 of file pcps_cccwsr_acquisition_cc.h.

The documentation for this class was generated from the following file:

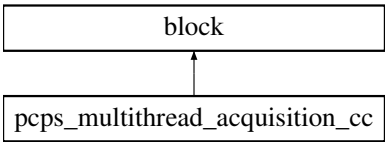
- src/algorithms/acquisition/gnuradio_blocks/pcps_cccwsr_acquisition_cc.h

8.156 pcps_multithread_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition.

`#include <pcps_multithread_acquisition_cc.h>`

Inheritance diagram for pcps_multithread_acquisition_cc:



Public Member Functions

- ~pcps_multithread_acquisition_cc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)

  Sets local code for PCPS acquisition algorithm.
- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.
- void **acquisition_core** ()

Friends

- pcps_multithread_acquisition_cc_sptr **pcps_make_multithread_acquisition_cc** (unsigned int sampled_↩ ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_transition_flag, bool dump, std::string dump_filename)

8.156.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Check An Open Source Galileo E1 Software Receiver, Algorithm 1, for a pseudocode description of this implementation.

Definition at line 80 of file pcps_multithread_acquisition_cc.h.

8.156.2 Constructor & Destructor Documentation

8.156.2.1 pcps_multithread_acquisition_cc::~pcps_multithread_acquisition_cc ( )

Default destructor.

8.156.3 Member Function Documentation

8.156.3.1 int pcps_multithread_acquisition_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.156.3.2 void pcps_multithread_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.156.3.3 unsigned int pcps_multithread_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 160 of file pcps_multithread_acquisition_cc.h.

References init(), and set_local_code().

8.156.3.4 void pcps_multithread_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 181 of file pcps_multithread_acquisition_cc.h.

References set_state().

8.156.3.5 void pcps_multithread_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 197 of file pcps_multithread_acquisition_cc.h.

8.156.3.6 void pcps_multithread_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 216 of file pcps_multithread_acquisition_cc.h.

8.156.3.7 void pcps_multithread_acquisition_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 225 of file pcps_multithread_acquisition_cc.h.

References general_work().

8.156.3.8 void pcps_multithread_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 152 of file pcps_multithread_acquisition_cc.h.

8.156.3.9 void pcps_multithread_acquisition_cc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.156.3.10 void pcps_multithread_acquisition_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.156.3.11 void pcps_multithread_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 207 of file pcps_multithread_acquisition_cc.h.

The documentation for this class was generated from the following file:

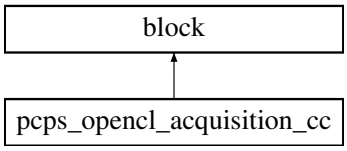
- src/algorithms/acquisition/gnuradio_blocks/pcps_multithread_acquisition_cc.h

8.157 pcps_opencl_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition.

`#include <pcps_opencl_acquisition_cc.h>`

Inheritance diagram for pcps_opencl_acquisition_cc:



Public Member Functions

- ~pcps_opencl_acquisition_cc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)

  Sets local code for PCPS acquisition algorithm.

- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.
- void **acquisition_core_volk** ()
- void **acquisition_core_opencl** ()

Friends

- pcps_opencl_acquisition_cc_sptr **pcps_make_opencl_acquisition_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_↩ per_code, bool bit_transition_flag, bool dump, std::string dump_filename)

8.157.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Check An Open Source Galileo E1 Software Receiver, Algorithm 1, for a pseudocode description of this implementation.

Definition at line 87 of file pcps_opencl_acquisition_cc.h.

8.157.2 Constructor & Destructor Documentation

8.157.2.1 pcps_opencl_acquisition_cc::~pcps_opencl_acquisition_cc ( )

Default destructor.

8.157.3 Member Function Documentation

8.157.3.1 int pcps_opencl_acquisition_cc::general_work (int *noutput_items*, gr_vector_int & *ninput_items*, gr_vector_const_void_star & *input_items*, gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.157.3.2 void pcps_opencl_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.157.3.3 unsigned int pcps_opencl_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 186 of file pcps_opencl_acquisition_cc.h.

References init(), and set_local_code().

8.157.3.4 void pcps_opencl_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 207 of file pcps_opencl_acquisition_cc.h.

References set_state().

8.157.3.5 void pcps_opencl_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 223 of file pcps_opencl_acquisition_cc.h.

8.157.3.6 void pcps_opencl_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 242 of file pcps_opencl_acquisition_cc.h.

8.157.3.7 void pcps_opencl_acquisition_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 251 of file pcps_opencl_acquisition_cc.h.

References general_work().

8.157.3.8 void pcps_opencl_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 178 of file pcps_opencl_acquisition_cc.h.

8.157.3.9 void pcps_opencl_acquisition_cc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.157.3.10 void pcps_opencl_acquisition_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.157.3.11 void pcps_opencl_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 233 of file pcps_opencl_acquisition_cc.h.

The documentation for this class was generated from the following file:

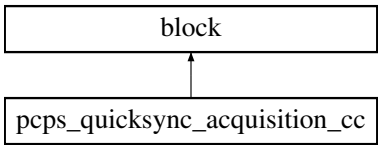
- src/algorithms/acquisition/gnuradio_blocks/pcps_opencl_acquisition_cc.h

8.158 pcps_quicksync_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition with the implementation of the Sparse QuickSync Algorithm.

`#include <pcps_quicksync_acquisition_cc.h>`

Inheritance diagram for pcps_quicksync_acquisition_cc:



Public Member Functions

- ~pcps_quicksync_acquisition_cc ()
  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)
  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()
  Returns the maximum peak of grid search.
- void init ()
  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)
  Sets local code for PCPS acquisition algorithm.
- void set_active (bool active)
  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)
  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)
  Set acquisition channel unique ID.
- void set_threshold (float threshold)
  Set statistics threshold of PCPS algorithm.
- void set_doppler_max (unsigned int doppler_max)
  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)
  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
  Parallel Code Phase Search Acquisition signal processing.

Friends

- pcps_quicksync_acquisition_cc_sptr **pcps_quicksync_make_acquisition_cc** (unsigned int folding_factor, unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_transition_flag, bool dump, std::string dump_filename)

8.158.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition with the implementation of the Sparse QuickSync Algorithm.

Check Faster GPS via the Sparse Fourier Transform, for details of its implementation and functionality.

Definition at line 85 of file pcps_quicksync_acquisition_cc.h.

8.158.2 Constructor & Destructor Documentation

8.158.2.1 pcps_quicksync_acquisition_cc::~pcps_quicksync_acquisition_cc ( )

Default destructor.

8.158.3 Member Function Documentation

8.158.3.1 int pcps_quicksync_acquisition_cc::general_work (int *noutput_items,* gr_vector_int & *ninput_items,* gr_vector_const_void_star & *input_items,* gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.158.3.2 void pcps_quicksync_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.158.3.3 unsigned int pcps_quicksync_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 172 of file pcps_quicksync_acquisition_cc.h.

References init(), and set_local_code().

8.158.3.4 void pcps_quicksync_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 193 of file pcps_quicksync_acquisition_cc.h.

References set_state().

8.158.3.5 void pcps_quicksync_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 209 of file pcps_quicksync_acquisition_cc.h.

8.158.3.6 void pcps_quicksync_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 228 of file pcps_quicksync_acquisition_cc.h.

8.158.3.7 void pcps_quicksync_acquisition_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 237 of file pcps_quicksync_acquisition_cc.h.

References general_work().

8.158.3.8 void pcps_quicksync_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 164 of file pcps_quicksync_acquisition_cc.h.

8.158.3.9 void pcps_quicksync_acquisition_cc::set_local_code (std::complex< float > * *code*)

Sets local code for PCPS acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.158.3.10 void pcps_quicksync_acquisition_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.158.3.11 void pcps_quicksync_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of PCPS algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 219 of file pcps_quicksync_acquisition_cc.h.

The documentation for this class was generated from the following file:

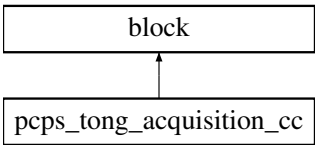
- src/algorithms/acquisition/gnuradio_blocks/pcps_quicksync_acquisition_cc.h

8.159 pcps_tong_acquisition_cc Class Reference

This class implements a Parallel Code Phase Search Acquisition with Tong algorithm.

`#include <pcps_tong_acquisition_cc.h>`

Inheritance diagram for pcps_tong_acquisition_cc:



Public Member Functions

- ~pcps_tong_acquisition_cc ()

  Default destructor.
- void set_gnss_synchro (Gnss_Synchro *p_gnss_synchro)

  Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.
- unsigned int mag ()

  Returns the maximum peak of grid search.
- void init ()

  Initializes acquisition algorithm.
- void set_local_code (std::complex< float > *code)

  Sets local code for TONG acquisition algorithm.
- void set_active (bool active)

  Starts acquisition algorithm, turning from standby mode to active mode.
- void set_state (int state)

  If set to 1, ensures that acquisition starts at the first available sample.
- void set_channel (unsigned int channel)

  Set acquisition channel unique ID.
- void set_threshold (float threshold)

  Set statistics threshold of TONG algorithm.
- void set_doppler_max (unsigned int doppler_max)

  Set maximum Doppler grid search.
- void set_doppler_step (unsigned int doppler_step)

  Set Doppler steps for the grid search.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  Parallel Code Phase Search Acquisition signal processing.

Friends

- pcps_tong_acquisition_cc_sptr **pcps_tong_make_acquisition_cc** (unsigned int sampled_ms, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, unsigned int tong_init_val, unsigned int tong_max_val, unsigned int tong_max_dwells, bool dump, std::string dump_filename)

8.159.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition with Tong algorithm.

Definition at line 76 of file pcps_tong_acquisition_cc.h.

8.159.2 Constructor & Destructor Documentation

8.159.2.1 pcps_tong_acquisition_cc::~pcps_tong_acquisition_cc ( )

Default destructor.

8.159.3 Member Function Documentation

8.159.3.1 int pcps_tong_acquisition_cc::general_work (int *noutput_items*, gr_vector_int & *ninput_items*, gr_vector_const_void_star & *input_items*, gr_vector_void_star & *output_items*)

Parallel Code Phase Search Acquisition signal processing.

Referenced by set_doppler_step().

8.159.3.2 void pcps_tong_acquisition_cc::init ( )

Initializes acquisition algorithm.

Referenced by mag().

8.159.3.3 unsigned int pcps_tong_acquisition_cc::mag ( ) [inline]

Returns the maximum peak of grid search.

Definition at line 151 of file pcps_tong_acquisition_cc.h.

References init(), and set_local_code().

8.159.3.4 void pcps_tong_acquisition_cc::set_active (bool *active*) [inline]

Starts acquisition algorithm, turning from standby mode to active mode.

Parameters

active	- bool that activates/deactivates the block.

Definition at line 172 of file pcps_tong_acquisition_cc.h.

References set_state().

8.159.3.5 void pcps_tong_acquisition_cc::set_channel (unsigned int *channel*) [inline]

Set acquisition channel unique ID.

Parameters

channel	- receiver channel.

Definition at line 188 of file pcps_tong_acquisition_cc.h.

8.159.3.6 void pcps_tong_acquisition_cc::set_doppler_max (unsigned int *doppler_max*) [inline]

Set maximum Doppler grid search.

Parameters

doppler_max	- Maximum Doppler shift considered in the grid search [Hz].

Definition at line 207 of file pcps_tong_acquisition_cc.h.

8.159.3.7 void pcps_tong_acquisition_cc::set_doppler_step (unsigned int *doppler_step*) [inline]

Set Doppler steps for the grid search.

Parameters

doppler_step	- Frequency bin of the search grid [Hz].

Definition at line 216 of file pcps_tong_acquisition_cc.h.

References general_work().

8.159.3.8 void pcps_tong_acquisition_cc::set_gnss_synchro (Gnss_Synchro * *p_gnss_synchro*) [inline]

Set acquisition/tracking common Gnss_Synchro object pointer to exchange synchronization data between acquisition and tracking blocks.

Parameters

p_gnss_synchro	Satellite information shared by the processing blocks.

Definition at line 143 of file pcps_tong_acquisition_cc.h.

8.159.3.9 void pcps_tong_acquisition_cc::set_local_code (std::complex< float > * *code*)

Sets local code for TONG acquisition algorithm.

Parameters

code	- Pointer to the PRN code.

Referenced by mag().

8.159.3.10 void pcps_tong_acquisition_cc::set_state (int *state*)

If set to 1, ensures that acquisition starts at the first available sample.

Parameters

state	- int=1 forces start of acquisition

Referenced by set_active().

8.159.3.11 void pcps_tong_acquisition_cc::set_threshold (float *threshold*) [inline]

Set statistics threshold of TONG algorithm.

Parameters

threshold	- Threshold for signal detection (check Navitec2012, Algorithm 1, for a definition of this threshold).

Definition at line 198 of file pcps_tong_acquisition_cc.h.

The documentation for this class was generated from the following file:

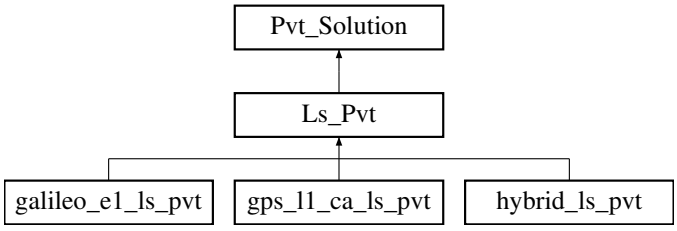
- src/algorithms/acquisition/gnuradio_blocks/pcps_tong_acquisition_cc.h

8.160 Pvt_Solution Class Reference

Base class for a PVT solution.

`#include <pvt_solution.h>`

Inheritance diagram for Pvt_Solution:



Public Member Functions

- int **pos_averaging** (bool flag_averaging)
- int compute_DOP ()

  Compute Dilution Of Precision parameters.
- int **set_averaging_depth** (int depth)
- arma::vec **rotateSatellite** (double traveltime, const arma::vec &X_sat)
- int cart2geo (double X, double Y, double Z, int elipsoid_selection)

  Conversion of Cartesian coordinates (X,Y,Z) to geographical coordinates (d_latitude_d, d_longitude_d, d_height_m) on a selected reference ellipsoid.
- int topocent (double *Az, double *El, double *D, const arma::vec &x, const arma::vec &dx)

  Transformation of vector dx into topocentric coordinate system with origin at x.
- int togeod (double *dphi, double *dlambda, double *h, double a, double finv, double X, double Y, double Z)

  Subroutine to calculate geodetic coordinates latitude, longitude, height given Cartesian coordinates X,Y,Z, and reference ellipsoid values semi-major axis (a) and the inverse of flattening (finv).
- int tropo (double *ddr_m, double sinel, double hsta_km, double p_mb, double t_kel, double hum, double hp_km, double htkel_km, double hhum_km)

  Tropospheric correction.

Public Attributes

- double d_latitude_d

  RX position Latitude WGS84 [deg].
- double d_longitude_d

  RX position Longitude WGS84 [deg].
- double d_height_m

  RX position height WGS84 [m].
- arma::vec **d_rx_pos**
- double d_rx_dt_s

  RX time offset [s].
- boost::posix_time::ptime **d_position_UTC_time**
- bool **b_valid_position**
- int d_valid_observations

  Number of valid pseudorange observations (valid satellites)
- int d_visible_satellites_IDs [PVT_MAX_CHANNELS] = {}

  Array with the IDs of the valid satellites.
- double d_visible_satellites_El [PVT_MAX_CHANNELS] = {}

  Array with the LOS Elevation of the valid satellites.
- double d_visible_satellites_Az [PVT_MAX_CHANNELS] = {}

  Array with the LOS Azimuth of the valid satellites.
- double d_visible_satellites_Distance [PVT_MAX_CHANNELS] = {}

  Array with the LOS Distance of the valid satellites.
- double d_visible_satellites_CN0_dB [PVT_MAX_CHANNELS] = {}

  Array with the IDs of the valid satellites.
- int d_averaging_depth

  Length of averaging window.
- std::deque< double > **d_hist_latitude_d**
- std::deque< double > **d_hist_longitude_d**
- std::deque< double > **d_hist_height_m**
- double d_avg_latitude_d

  Averaged latitude in degrees.

- double d_avg_longitude_d

  Averaged longitude in degrees.
- double d_avg_height_m

  Averaged height [m].
- arma::mat **d_Q**
- double **d_GDOP**
- double **d_PDOP**
- double **d_HDOP**
- double **d_VDOP**
- double **d_TDOP**
- bool **d_flag_averaging**

8.160.1 Detailed Description

Base class for a PVT solution.

Definition at line 46 of file pvt_solution.h.

8.160.2 Member Function Documentation

8.160.2.1 int Pvt_Solution::cart2geo (double *X*, double *Y*, double *Z*, int *elipsoid_selection*)

Conversion of Cartesian coordinates (X,Y,Z) to geographical coordinates (d_latitude_d, d_longitude_d, d_height_m) on a selected reference ellipsoid.

Parameters

in	*X*	[m] Cartesian coordinate
in	*Y*	[m] Cartesian coordinate
in	*Z*	[m] Cartesian coordinate
in	*elipsoid_selection.*	Choices of Reference Ellipsoid for Geographical Coordinates: 0 - International Ellipsoid 1924. 1 - International Ellipsoid 1967. 2 - World Geodetic System 1972. 3 - Geodetic Reference System 1980. 4 - World Geodetic System 1984.

8.160.2.2 int Pvt_Solution::compute_DOP ( )

Compute Dilution Of Precision parameters.

8.160.2.3 int Pvt_Solution::togeod (double * *dphi*, double * *dlambda*, double * *h*, double *a*, double *finv*, double *X*, double *Y*, double *Z*)

Subroutine to calculate geodetic coordinates latitude, longitude, height given Cartesian coordinates X,Y,Z, and reference ellipsoid values semi-major axis (a) and the inverse of flattening (finv).

The output units of angular quantities will be in decimal degrees (15.5 degrees not 15 deg 30 min). The output units of h will be the same as the units of X,Y,Z,a.

Parameters

in	*a*	- semi-major axis of the reference ellipsoid
in	*finv*	- inverse of flattening of the reference ellipsoid
in	*X,Y,Z*	- Cartesian coordinates
out	*dphi*	- latitude
out	*dlambda*	- longitude
out	*h*	- height above reference ellipsoid

Based in a Matlab function by Kai Borre

8.160.2.4 int Pvt_Solution::topocent (double * *Az,* double * *El,* double * *D,* const arma::vec & *x,* const arma::vec & *dx*)

Transformation of vector dx into topocentric coordinate system with origin at x.

Parameters

in	*x*	Vector origin coordinates (in ECEF system [X; Y; Z;])
in	*dx*	Vector ([dX; dY; dZ;]).
out	*D*	Vector length. Units like the input
out	*Az*	Azimuth from north positive clockwise, degrees
out	*El*	Elevation angle, degrees

Based on a Matlab function by Kai Borre

8.160.2.5 int Pvt_Solution::tropo (double * *ddr_m,* double *sinel,* double *hsta_km,* double *p_mb,* double *t_kel,* double *hum,* double *hp_km,* double *htkel_km,* double *hhum_km*)

Tropospheric correction.

Parameters

in	*sinel*	- sin of elevation angle of satellite
in	*hsta_km*	- height of station in km
in	*p_mb*	- atmospheric pressure in mb at height hp_km
in	*t_kel*	- surface temperature in degrees Kelvin at height htkel_km
in	*hum*	- humidity in % at height hhum_km
in	*hp_km*	- height of pressure measurement in km
in	*htkel_km*	- height of temperature measurement in km
in	*hhum_km*	- height of humidity measurement in km
out	*ddr_m*	- range correction (meters)

Reference: Goad, C.C. & Goodman, L. (1974) A Modified Hopfield Tropospheric Refraction Correction Model. Paper presented at the American Geophysical Union Annual Fall Meeting, San Francisco, December 12-17

Translated to C++ by Carles Fernandez from a Matlab implementation by Kai Borre

8.160.3 Member Data Documentation

8.160.3.1 int Pvt_Solution::d_averaging_depth

Length of averaging window.

Definition at line 70 of file pvt_solution.h.

8.160.3.2 double Pvt_Solution::d_avg_height_m

Averaged height [m].

Definition at line 77 of file pvt_solution.h.

8.160.3.3 double Pvt_Solution::d_avg_latitude_d

Averaged latitude in degrees.

Definition at line 75 of file pvt_solution.h.

8.160.3.4 double Pvt_Solution::d_avg_longitude_d

Averaged longitude in degrees.

Definition at line 76 of file pvt_solution.h.

8.160.3.5 double Pvt_Solution::d_height_m

RX position height WGS84 [m].

Definition at line 53 of file pvt_solution.h.

8.160.3.6 double Pvt_Solution::d_latitude_d

RX position Latitude WGS84 [deg].

Definition at line 51 of file pvt_solution.h.

8.160.3.7 double Pvt_Solution::d_longitude_d

RX position Longitude WGS84 [deg].

Definition at line 52 of file pvt_solution.h.

8.160.3.8 double Pvt_Solution::d_rx_dt_s

RX time offset [s].

Definition at line 56 of file pvt_solution.h.

8.160.3.9 int Pvt_Solution::d_valid_observations

Number of valid pseudorange observations (valid satellites)

Definition at line 62 of file pvt_solution.h.

8.160.3.10 double Pvt_Solution::d_visible_satellites_Az[PVT_MAX_CHANNELS] = {}

Array with the LOS Azimuth of the valid satellites.

Definition at line 65 of file pvt_solution.h.

8.160.3.11 double Pvt_Solution::d_visible_satellites_CN0_dB[PVT_MAX_CHANNELS] = {}

Array with the IDs of the valid satellites.

Definition at line 67 of file pvt_solution.h.

8.160.3.12 double Pvt_Solution::d_visible_satellites_Distance[PVT_MAX_CHANNELS] = {}

Array with the LOS Distance of the valid satellites.

Definition at line 66 of file pvt_solution.h.

8.160.3.13 double Pvt_Solution::d_visible_satellites_El[PVT_MAX_CHANNELS] = {}

Array with the LOS Elevation of the valid satellites.

Definition at line 64 of file pvt_solution.h.

8.160.3.14 int Pvt_Solution::d_visible_satellites_IDs[PVT_MAX_CHANNELS] = {}

Array with the IDs of the valid satellites.

Definition at line 63 of file pvt_solution.h.

The documentation for this class was generated from the following file:

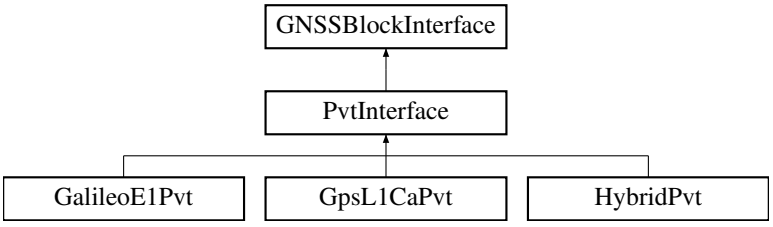
- src/algorithms/PVT/libs/pvt_solution.h

8.161 PvtInterface Class Reference

This class represents an interface to a PVT block.

`#include <pvt_interface.h>`

Inheritance diagram for PvtInterface:



Public Member Functions

- virtual void **reset** ()=0

8.161.1 Detailed Description

This class represents an interface to a PVT block.

Abstract class for PVT interfaces, derived from GNSSBlockInterface. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 51 of file pvt_interface.h.

The documentation for this class was generated from the following file:

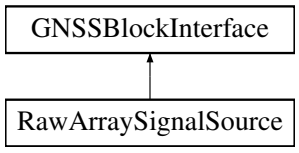
- src/core/interfaces/pvt_interface.h

8.162 RawArraySignalSource Class Reference

This class reads samples from a GN3S USB dongle, a RF front-end signal sampler.

`#include <raw_array_signal_source.h>`

Inheritance diagram for RawArraySignalSource:



Public Member Functions

- **RawArraySignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream, gr::msg_queue::sptr queue)
- std::string **role** ()
- std::string implementation ()

  Returns "RawArraySignalSource".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.162.1 Detailed Description

This class reads samples from a GN3S USB dongle, a RF front-end signal sampler.

Definition at line 47 of file raw_array_signal_source.h.

8.162.2 Member Function Documentation

8.162.2.1 std::string RawArraySignalSource::implementation ( ) [inline], [virtual]

Returns "RawArraySignalSource".

Implements GNSSBlockInterface.

Definition at line 63 of file raw_array_signal_source.h.

The documentation for this class was generated from the following file:

- src/algorithms/signal_source/adapters/raw_array_signal_source.h

8.163 Rinex_Printer Class Reference

Class that handles the generation of Receiver INdependent EXchange format (RINEX) files.

```
#include <rinex_printer.h>
```

Public Member Functions

- Rinex_Printer (int version=0)

  Default constructor. Creates GPS Navigation and Observables RINEX files and their headers.
- ~Rinex_Printer ()

  Default destructor. Closes GPS Navigation and Observables RINEX files.
- void rinex_nav_header (std::fstream &out, const Gps_Iono &iono, const Gps_Utc_Model &utc_model)

  Generates the GPS L1 C/A Navigation Data header.
- void rinex_nav_header (std::fstream &out, const Gps_CNAV_Iono &iono, const Gps_CNAV_Utc_Model &utc_model)

  Generates the GPS L2C(M) Navigation Data header.
- void rinex_nav_header (std::fstream &out, const Galileo_Iono &iono, const Galileo_Utc_Model &utc_model, const Galileo_Almanac &galileo_almanac)

  Generates the Galileo Navigation Data header.
- void rinex_nav_header (std::fstream &out, const Gps_Iono &gps_iono, const Gps_Utc_Model &gps_utc↩_model, const Galileo_Iono &galileo_iono, const Galileo_Utc_Model &galileo_utc_model, const Galileo_↩Almanac &galileo_almanac)

  Generates the Mixed (GPS/Galileo) Navigation Data header.
- void rinex_obs_header (std::fstream &out, const Gps_Ephemeris &eph, const double d_TOW_first_↩observation)

  Generates the GPS Observation data header.
- void rinex_obs_header (std::fstream &out, const Gps_CNAV_Ephemeris &eph, const double d_TOW_first↩_observation)

  Generates the GPS L2 Observation data header.
- void rinex_obs_header (std::fstream &out, const Gps_Ephemeris &eph, const Gps_CNAV_Ephemeris &eph_cnav, const double d_TOW_first_observation)

  Generates the dual frequency GPS L1 & L2 Observation data header.
- void rinex_obs_header (std::fstream &out, const Galileo_Ephemeris &eph, const double d_TOW_first_↩observation, const std::string bands="1B")

  Generates the Galileo Observation data header. Example: bands("1B"), bands("1B 5X"), bands("5X"), ... Default: "1B".
- void rinex_obs_header (std::fstream &out, const Gps_Ephemeris &gps_eph, const Galileo_Ephemeris &galileo_eph, const double d_TOW_first_observation, const std::string galileo_bands="1B")

  Generates the Mixed (GPS/Galileo) Observation data header. Example: galileo_bands("1B"), galileo_bands("1B 5↩X"), galileo_bands("5X"), ... Default: "1B".
- void rinex_sbs_header (std::fstream &out)

  Generates the SBAS raw data header.
- boost::posix_time::ptime compute_UTC_time (const Gps_Navigation_Message &nav_msg)

  Computes the UTC time and returns a boost::posix_time::ptime object.
- boost::posix_time::ptime compute_GPS_time (const Gps_Ephemeris &eph, const double obs_time)

  Computes the GPS time and returns a boost::posix_time::ptime object.
- boost::posix_time::ptime compute_GPS_time (const Gps_CNAV_Ephemeris &eph, const double obs_time)

  Computes the GPS time and returns a boost::posix_time::ptime object.
- boost::posix_time::ptime compute_Galileo_time (const Galileo_Ephemeris &eph, const double obs_time)

  Computes the Galileo time and returns a boost::posix_time::ptime object.
- void log_rinex_nav (std::fstream &out, const std::map< int, Gps_Ephemeris > &eph_map)

  Writes data from the GPS L1 C/A navigation message into the RINEX file.
- void log_rinex_nav (std::fstream &out, const std::map< int, Gps_CNAV_Ephemeris > &eph_map)

  Writes data from the GPS L2 navigation message into the RINEX file.
- void log_rinex_nav (std::fstream &out, const std::map< int, Galileo_Ephemeris > &eph_map)

  Writes data from the Galileo navigation message into the RINEX file.
- void log_rinex_nav (std::fstream &out, const std::map< int, Gps_Ephemeris > &gps_eph_map, const std↩::map< int, Galileo_Ephemeris > &galileo_eph_map)

Writes data from the Mixed (GPS/Galileo) navigation message into the RINEX file.

- void log_rinex_obs (std::fstream &out, const Gps_Ephemeris &eph, double obs_time, const std::map< int, Gnss_Synchro > &observables)

  Writes GPS L1 observables into the RINEX file.

- void log_rinex_obs (std::fstream &out, const Gps_CNAV_Ephemeris &eph, double obs_time, const std::map< int, Gnss_Synchro > &observables)

  Writes GPS L2 observables into the RINEX file.

- void log_rinex_obs (std::fstream &out, const Gps_Ephemeris &eph, const Gps_CNAV_Ephemeris &eph_cnav, double obs_time, const std::map< int, Gnss_Synchro > &observables)

  Writes dual frequency GPS L1 and L2 observables into the RINEX file.

- void log_rinex_obs (std::fstream &out, const Galileo_Ephemeris &eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, const std::string galileo_bands="1B")

  Writes Galileo observables into the RINEX file. Example: galileo_bands("1B"), galileo_bands("1B 5X"), galileo_bands("5X"), ... Default: "1B".

- void log_rinex_obs (std::fstream &out, const Gps_Ephemeris &gps_eph, const Galileo_Ephemeris &galileo_eph, const double gps_obs_time, const std::map< int, Gnss_Synchro > &observables)

  Writes Mixed GPS / Galileo observables into the RINEX file.

- void to_date_time (int gps_week, int gps_tow, int &year, int &month, int &day, int &hour, int &minute, int &second)

  Represents GPS time in the date time format. Leap years are considered, but leap seconds are not.

- void log_rinex_sbs (std::fstream &out, const Sbas_Raw_Msg &sbs_message)

  Writes raw SBAS messages into the RINEX file.

- void **update_nav_header** (std::fstream &out, const Gps_Utc_Model &gps_utc, const Gps_Iono &gps_iono)
- void **update_nav_header** (std::fstream &out, const Gps_CNAV_Utc_Model &utc_model, const Gps_CNAV_Iono &iono)
- void **update_nav_header** (std::fstream &out, const Gps_Iono &gps_iono, const Gps_Utc_Model &gps_utc_model, const Galileo_Iono &galileo_iono, const Galileo_Utc_Model &galileo_utc_model, const Galileo_Almanac &galileo_almanac)
- void **update_nav_header** (std::fstream &out, const Galileo_Iono &galileo_iono, const Galileo_Utc_Model &utc_model, const Galileo_Almanac &galileo_almanac)
- void **update_obs_header** (std::fstream &out, const Gps_Utc_Model &utc_model)
- void **update_obs_header** (std::fstream &out, const Gps_CNAV_Utc_Model &utc_model)
- void **update_obs_header** (std::fstream &out, const Galileo_Utc_Model &galileo_utc_model)

Public Attributes

- std::fstream **obsFile**
- std::fstream **navFile**
- std::fstream **sbsFile**
- std::fstream **navGalFile**
- std::fstream **navMixFile**
- std::map< std::string, std::string > **satelliteSystem**
- std::map< std::string, std::string > **observationType**
- std::map< std::string, std::string > **observationCode**
- std::string **stringVersion**
- std::string **navfilename**
- std::string **obsfilename**
- std::string **sbsfilename**
- std::string **navGalfilename**
- std::string **navMixfilename**

8.163.1 Detailed Description

Class that handles the generation of Receiver INdependent EXchange format (RINEX) files.

Definition at line 75 of file rinex_printer.h.

8.163.2 Constructor & Destructor Documentation

8.163.2.1 Rinex_Printer::Rinex_Printer (int *version* = 0)

Default constructor. Creates GPS Navigation and Observables RINEX files and their headers.

8.163.2.2 Rinex_Printer::~Rinex_Printer ( )

Default destructor. Closes GPS Navigation and Observables RINEX files.

8.163.3 Member Function Documentation

8.163.3.1 boost::posix_time::ptime Rinex_Printer::compute_Galileo_time (const Galileo_Ephemeris & *eph,* const double *obs_time*)

Computes the Galileo time and returns a boost::posix_time::ptime object.

8.163.3.2 boost::posix_time::ptime Rinex_Printer::compute_GPS_time (const Gps_Ephemeris & *eph,* const double *obs_time*)

Computes the GPS time and returns a boost::posix_time::ptime object.

8.163.3.3 boost::posix_time::ptime Rinex_Printer::compute_GPS_time (const Gps_CNAV_Ephemeris & *eph,* const double *obs_time*)

Computes the GPS time and returns a boost::posix_time::ptime object.

8.163.3.4 boost::posix_time::ptime Rinex_Printer::compute_UTC_time (const Gps_Navigation_Message & *nav_msg*)

Computes the UTC time and returns a boost::posix_time::ptime object.

8.163.3.5 void Rinex_Printer::log_rinex_nav (std::fstream & *out,* const std::map< int, Gps_Ephemeris > & *eph_map*)

Writes data from the GPS L1 C/A navigation message into the RINEX file.

8.163.3.6 void Rinex_Printer::log_rinex_nav (std::fstream & *out,* const std::map< int, **Gps_CNAV_Ephemeris** > & *eph_map*)

Writes data from the GPS L2 navigation message into the RINEX file.

8.163.3.7 void Rinex_Printer::log_rinex_nav (std::fstream & *out,* const std::map< int, **Galileo_Ephemeris** > & *eph_map*)

Writes data from the Galileo navigation message into the RINEX file.

8.163.3.8 void Rinex_Printer::log_rinex_nav (std::fstream & *out,* const std::map< int, **Gps_Ephemeris** > & *gps_eph_map,* const std::map< int, **Galileo_Ephemeris** > & *galileo_eph_map*)

Writes data from the Mixed (GPS/Galileo) navigation message into the RINEX file.

8.163.3.9 void Rinex_Printer::log_rinex_obs (std::fstream & *out,* const **Gps_Ephemeris** & *eph,* double *obs_time,* const std::map< int, **Gnss_Synchro** > & *observables*)

Writes GPS L1 observables into the RINEX file.

8.163.3.10 void Rinex_Printer::log_rinex_obs (std::fstream & *out,* const **Gps_CNAV_Ephemeris** & *eph,* double *obs_time,* const std::map< int, **Gnss_Synchro** > & *observables*)

Writes GPS L2 observables into the RINEX file.

8.163.3.11 void Rinex_Printer::log_rinex_obs (std::fstream & *out,* const **Gps_Ephemeris** & *eph,* const **Gps_CNAV_Ephemeris** & *eph_cnav,* double *obs_time,* const std::map< int, **Gnss_Synchro** > & *observables*)

Writes dual frequency GPS L1 and L2 observables into the RINEX file.

8.163.3.12 void Rinex_Printer::log_rinex_obs (std::fstream & *out,* const **Galileo_Ephemeris** & *eph,* double *obs_time,* const std::map< int, **Gnss_Synchro** > & *observables,* const std::string *galileo_bands* = "1B")

Writes Galileo observables into the RINEX file. Example: galileo_bands("1B"), galileo_bands("1B 5X"), galileo_↩ bands("5X"), ... Default: "1B".

8.163.3.13 void Rinex_Printer::log_rinex_obs (std::fstream & *out,* const **Gps_Ephemeris** & *gps_eph,* const **Galileo_Ephemeris** & *galileo_eph,* const double *gps_obs_time,* const std::map< int, **Gnss_Synchro** > & *observables*)

Writes Mixed GPS / Galileo observables into the RINEX file.

8.163.3.14 void Rinex_Printer::log_rinex_sbs (std::fstream & *out,* const **Sbas_Raw_Msg** & *sbs_message*)

Writes raw SBAS messages into the RINEX file.

8.163.3.15 void Rinex_Printer::rinex_nav_header (std::fstream & *out,* const **Gps_Iono** & *iono,* const **Gps_Utc_Model** & *utc_model*)

Generates the GPS L1 C/A Navigation Data header.

8.163.3.16 void Rinex_Printer::rinex_nav_header (std::fstream & *out,* const **Gps_CNAV_Iono** & *iono,* const **Gps_CNAV_Utc_Model** & *utc_model*)

Generates the GPS L2C(M) Navigation Data header.

8.163.3.17 void Rinex_Printer::rinex_nav_header (std::fstream & *out,* const **Galileo_Iono** & *iono,* const **Galileo_Utc_Model** & *utc_model,* const **Galileo_Almanac** & *galileo_almanac*)

Generates the Galileo Navigation Data header.

8.163.3.18 void Rinex_Printer::rinex_nav_header (std::fstream & *out,* const **Gps_Iono** & *gps_iono,* const **Gps_Utc_Model** & *gps_utc_model,* const **Galileo_Iono** & *galileo_iono,* const **Galileo_Utc_Model** & *galileo_utc_model,* const **Galileo_Almanac** & *galileo_almanac*)

Generates the Mixed (GPS/Galileo) Navigation Data header.

8.163.3.19 void Rinex_Printer::rinex_obs_header (std::fstream & *out,* const **Gps_Ephemeris** & *eph,* const double *d_TOW_first_observation*)

Generates the GPS Observation data header.

8.163.3.20 void Rinex_Printer::rinex_obs_header (std::fstream & *out,* const **Gps_CNAV_Ephemeris** & *eph,* const double *d_TOW_first_observation*)

Generates the GPS L2 Observation data header.

8.163.3.21 void Rinex_Printer::rinex_obs_header (std::fstream & *out,* const **Gps_Ephemeris** & *eph,* const **Gps_CNAV_Ephemeris** & *eph_cnav,* const double *d_TOW_first_observation*)

Generates the dual frequency GPS L1 & L2 Observation data header.

8.163.3.22 void Rinex_Printer::rinex_obs_header (std::fstream & *out,* const **Galileo_Ephemeris** & *eph,* const double *d_TOW_first_observation,* const std::string *bands* = "1B")

Generates the Galileo Observation data header. Example: bands("1B"), bands("1B 5X"), bands("5X"), ... Default: "1B".

8.163.3.23 void Rinex_Printer::rinex_obs_header (std::fstream & *out,* const Gps_Ephemeris & *gps_eph,* const Galileo_Ephemeris & *galileo_eph,* const double *d_TOW_first_observation,* const std::string *galileo_bands* = "1B")

Generates the Mixed (GPS/Galileo) Observation data header. Example: galileo_bands("1B"), galileo_bands("1B 5X"), galileo_bands("5X"), ... Default: "1B".

8.163.3.24 void Rinex_Printer::rinex_sbs_header (std::fstream & *out*)

Generates the SBAS raw data header.

8.163.3.25 void Rinex_Printer::to_date_time (int *gps_week,* int *gps_tow,* int & *year,* int & *month,* int & *day,* int & *hour,* int & *minute,* int & *second*)

Represents GPS time in the date time format. Leap years are considered, but leap seconds are not.

The documentation for this class was generated from the following file:

- src/algorithms/PVT/libs/rinex_printer.h

8.164 Rtcm Class Reference

`#include <rtcm.h>`

Public Member Functions

- **Rtcm** (unsigned short port=2101)
- std::string print_MT1001 (const Gps_Ephemeris &gps_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id)

  Prints message type 1001 (L1-Only GPS RTK Observables)
- std::string print_MT1002 (const Gps_Ephemeris &gps_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id)

  Prints message type 1002 (Extended L1-Only GPS RTK Observables)
- std::string print_MT1003 (const Gps_Ephemeris &ephL1, const Gps_CNAV_Ephemeris &ephL2, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id)

  Prints message type 1003 (L1 & L2 GPS RTK Observables)
- std::string print_MT1004 (const Gps_Ephemeris &ephL1, const Gps_CNAV_Ephemeris &ephL2, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned short station_id)

  Prints message type 1004 (Extended L1 & L2 GPS RTK Observables)
- std::string print_MT1005 (unsigned int ref_id, double ecef_x, double ecef_y, double ecef_z, bool gps, bool glonass, bool galileo, bool non_physical, bool single_oscillator, unsigned int quarter_cycle_indicator)

  Prints message type 1005 (Stationary Antenna Reference Point)
- int read_MT1005 (const std::string &message, unsigned int &ref_id, double &ecef_x, double &ecef_y, double &ecef_z, bool &gps, bool &glonass, bool &galileo)

  Verifies and reads messages of type 1005 (Stationary Antenna Reference Point). Returns 1 if anything goes wrong, 0 otherwise.

- std::string print_MT1006 (unsigned int ref_id, double ecef_x, double ecef_y, double ecef_z, bool gps, bool glonass, bool galileo, bool non_physical, bool single_oscillator, unsigned int quarter_cycle_indicator, double height)

  Prints message type 1006 (Stationary Antenna Reference Point, with Height Information)
- std::string **print_MT1005_test** ()
- std::string print_MT1008 (unsigned int ref_id, const std::string &antenna_descriptor, unsigned int antenna↩ _setup_id, const std::string &antenna_serial_number)

  Prints message type 1008 (Antenna Descriptor & Serial Number)
- std::string print_MT1019 (const Gps_Ephemeris &gps_eph)

  Prints message type 1019 (GPS Ephemeris), should be broadcast in the event that the IODC does not match the IODE, and every 2 minutes.
- int read_MT1019 (const std::string &message, Gps_Ephemeris &gps_eph)

  Verifies and reads messages of type 1019 (GPS Ephemeris). Returns 1 if anything goes wrong, 0 otherwise.
- std::string print_MT1029 (unsigned int ref_id, const Gps_Ephemeris &gps_eph, double obs_time, const std↩ ::string &message)

  Prints message type 1029 (Unicode Text String)
- std::string print_MT1045 (const Galileo_Ephemeris &gal_eph)

  Prints message type 1045 (Galileo Ephemeris), should be broadcast every 2 minutes.
- int read_MT1045 (const std::string &message, Galileo_Ephemeris &gal_eph)

  Verifies and reads messages of type 1045 (Galileo Ephemeris). Returns 1 if anything goes wrong, 0 otherwise.
- std::string print_MSM_1 (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth↩ _int, bool divergence_free, bool more_messages)

  Prints messages of type MSM1 (Compact GNSS observables)
- std::string print_MSM_2 (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth↩ _int, bool divergence_free, bool more_messages)

  Prints messages of type MSM2 (Compact GNSS phaseranges)
- std::string print_MSM_3 (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth↩ _int, bool divergence_free, bool more_messages)

  Prints messages of type MSM3 (Compact GNSS pseudoranges and phaseranges)
- std::string print_MSM_4 (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth↩ _int, bool divergence_free, bool more_messages)

  Prints messages of type MSM4 (Full GNSS pseudoranges and phaseranges plus CNR)
- std::string print_MSM_5 (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth↩ _int, bool divergence_free, bool more_messages)

  Prints messages of type MSM5 (Full GNSS pseudoranges, phaseranges, phaserange rate and CNR)
- std::string print_MSM_6 (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth↩ _int, bool divergence_free, bool more_messages)

  Prints messages of type MSM6 (Full GNSS pseudoranges and phaseranges plus CNR, high resolution)
- std::string print_MSM_7 (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int ref_id, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth↩ _int, bool divergence_free, bool more_messages)

Prints messages of type MSM7 (Full GNSS pseudoranges, phaseranges, phaserange rate and CNR, high resolution)

- unsigned int **lock_time** (const Gps_Ephemeris &eph, double obs_time, const Gnss_Synchro &gnss_synchro)
- unsigned int **lock_time** (const Gps_CNAV_Ephemeris &eph, double obs_time, const Gnss_Synchro &gnss↩_synchro)
- unsigned int **lock_time** (const Galileo_Ephemeris &eph, double obs_time, const Gnss_Synchro &gnss_↩synchro)
- std::string **bin_to_hex** (const std::string &s) const
- std::string **hex_to_bin** (const std::string &s) const
- std::string **bin_to_binary_data** (const std::string &s) const
- std::string **binary_data_to_bin** (const std::string &s) const
- unsigned long int **bin_to_uint** (const std::string &s) const
- long int **bin_to_int** (const std::string &s) const
- double **bin_to_double** (const std::string &s) const
- unsigned long int **hex_to_uint** (const std::string &s) const
- long int **hex_to_int** (const std::string &s) const
- bool **check_CRC** (const std::string &message) const
- void **run_server** ()
- void **stop_server** ()
- void **send_message** (const std::string &message)
- bool **is_server_running** () const

8.164.1 Detailed Description

This class implements the generation and reading of some Message Types defined in the RTCM 3.2 Standard, plus some utilities to handle messages.

Generation of the following Message Types: 1001, 1002, 1003, 1004, 1005, 1006, 1008, 1019, 1029, 1045

Decoding of the following Message Types: 1019, 1045

Generation of the following Multiple Signal Messages: MSM1 (message types 1071, 1091) MSM2 (message types 1072, 1092) MSM3 (message types 1073, 1093) MSM4 (message types 1074, 1094) MSM5 (message types 1075, 1095) MSM6 (message types 1076, 1096) MSM7 (message types 1077, 1097)

RTCM 3 message format (size in bits): +---------—+-------—+----------—+------------------—+---------—+ | preamble | 000000 | length | data message | parity | +---------—+-------—+----------—+------------------—+---------—+ |<– 8 —>|<- 6 –>|<– 10 —>|<— length x 8 —>|<– 24 –>| +---------—+-------—+----------—+------------------—+---------—+

(C) Carles Fernandez-Prades, 2015. cfernandez(at)cttc.es

Definition at line 83 of file rtcm.h.

8.164.2 Member Function Documentation

8.164.2.1 std::string Rtcm::print_MSM_1 (const Gps_Ephemeris & *gps_eph,* const Gps_CNAV_Ephemeris & *gps_cnav_eph,* const Galileo_Ephemeris & *gal_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned int *ref_id,* unsigned int *clock_steering_indicator,* unsigned int *external_clock_indicator,* int *smooth_int,* bool *divergence_free,* bool *more_messages*)

Prints messages of type MSM1 (Compact GNSS observables)

8.164.2.2 std::string Rtcm::print_MSM_2 (const Gps_Ephemeris & *gps_eph,* const Gps_CNAV_Ephemeris & *gps_cnav_eph,* const Galileo_Ephemeris & *gal_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned int *ref_id,* unsigned int *clock_steering_indicator,* unsigned int *external_clock_indicator,* int *smooth_int,* bool *divergence_free,* bool *more_messages*)

Prints messages of type MSM2 (Compact GNSS phaseranges)

8.164.2.3 std::string Rtcm::print_MSM_3 (const Gps_Ephemeris & *gps_eph,* const Gps_CNAV_Ephemeris & *gps_cnav_eph,* const Galileo_Ephemeris & *gal_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned int *ref_id,* unsigned int *clock_steering_indicator,* unsigned int *external_clock_indicator,* int *smooth_int,* bool *divergence_free,* bool *more_messages*)

Prints messages of type MSM3 (Compact GNSS pseudoranges and phaseranges)

8.164.2.4 std::string Rtcm::print_MSM_4 (const Gps_Ephemeris & *gps_eph,* const Gps_CNAV_Ephemeris & *gps_cnav_eph,* const Galileo_Ephemeris & *gal_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned int *ref_id,* unsigned int *clock_steering_indicator,* unsigned int *external_clock_indicator,* int *smooth_int,* bool *divergence_free,* bool *more_messages*)

Prints messages of type MSM4 (Full GNSS pseudoranges and phaseranges plus CNR)

8.164.2.5 std::string Rtcm::print_MSM_5 (const Gps_Ephemeris & *gps_eph,* const Gps_CNAV_Ephemeris & *gps_cnav_eph,* const Galileo_Ephemeris & *gal_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned int *ref_id,* unsigned int *clock_steering_indicator,* unsigned int *external_clock_indicator,* int *smooth_int,* bool *divergence_free,* bool *more_messages*)

Prints messages of type MSM5 (Full GNSS pseudoranges, phaseranges, phaserange rate and CNR)

8.164.2.6 std::string Rtcm::print_MSM_6 (const Gps_Ephemeris & *gps_eph,* const Gps_CNAV_Ephemeris & *gps_cnav_eph,* const Galileo_Ephemeris & *gal_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned int *ref_id,* unsigned int *clock_steering_indicator,* unsigned int *external_clock_indicator,* int *smooth_int,* bool *divergence_free,* bool *more_messages*)

Prints messages of type MSM6 (Full GNSS pseudoranges and phaseranges plus CNR, high resolution)

8.164.2.7 std::string Rtcm::print_MSM_7 (const Gps_Ephemeris & *gps_eph,* const Gps_CNAV_Ephemeris & *gps_cnav_eph,* const Galileo_Ephemeris & *gal_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned int *ref_id,* unsigned int *clock_steering_indicator,* unsigned int *external_clock_indicator,* int *smooth_int,* bool *divergence_free,* bool *more_messages*)

Prints messages of type MSM7 (Full GNSS pseudoranges, phaseranges, phaserange rate and CNR, high resolution)

8.164.2.8 std::string Rtcm::print_MT1001 (const Gps_Ephemeris & *gps_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned short *station_id*)

Prints message type 1001 (L1-Only GPS RTK Observables)

8.164.2.9 std::string Rtcm::print_MT1002 (const Gps_Ephemeris & *gps_eph,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned short *station_id*)

Prints message type 1002 (Extended L1-Only GPS RTK Observables)

8.164.2.10 std::string Rtcm::print_MT1003 (const Gps_Ephemeris & *ephL1,* const Gps_CNAV_Ephemeris & *ephL2,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned short *station_id*)

Prints message type 1003 (L1 & L2 GPS RTK Observables)

8.164.2.11 std::string Rtcm::print_MT1004 (const Gps_Ephemeris & *ephL1,* const Gps_CNAV_Ephemeris & *ephL2,* double *obs_time,* const std::map< int, Gnss_Synchro > & *observables,* unsigned short *station_id*)

Prints message type 1004 (Extended L1 & L2 GPS RTK Observables)

8.164.2.12 std::string Rtcm::print_MT1005 (unsigned int *ref_id,* double *ecef_x,* double *ecef_y,* double *ecef_z,* bool *gps,* bool *glonass,* bool *galileo,* bool *non_physical,* bool *single_oscillator,* unsigned int *quarter_cycle_indicator*)

Prints message type 1005 (Stationary Antenna Reference Point)

8.164.2.13 std::string Rtcm::print_MT1006 (unsigned int *ref_id,* double *ecef_x,* double *ecef_y,* double *ecef_z,* bool *gps,* bool *glonass,* bool *galileo,* bool *non_physical,* bool *single_oscillator,* unsigned int *quarter_cycle_indicator,* double *height*)

Prints message type 1006 (Stationary Antenna Reference Point, with Height Information)

8.164.2.14 std::string Rtcm::print_MT1008 (unsigned int *ref_id,* const std::string & *antenna_descriptor,* unsigned int *antenna_setup_id,* const std::string & *antenna_serial_number*)

Prints message type 1008 (Antenna Descriptor & Serial Number)

8.164.2.15 std::string Rtcm::print_MT1019 (const Gps_Ephemeris & *gps_eph*)

Prints message type 1019 (GPS Ephemeris), should be broadcast in the event that the IODC does not match the IODE, and every 2 minutes.

8.164.2.16 std::string Rtcm::print_MT1029 (unsigned int *ref_id,* const Gps_Ephemeris & *gps_eph,* double *obs_time,* const std::string & *message*)

Prints message type 1029 (Unicode Text String)

8.164.2.17 std::string Rtcm::print_MT1045 (const Galileo_Ephemeris & *gal_eph*)

Prints message type 1045 (Galileo Ephemeris), should be broadcast every 2 minutes.

8.164.2.18 int Rtcm::read_MT1005 (const std::string & *message,* unsigned int & *ref_id,* double & *ecef_x,* double & *ecef_y,* double & *ecef_z,* bool & *gps,* bool & *glonass,* bool & *galileo*)

Verifies and reads messages of type 1005 (Stationary Antenna Reference Point). Returns 1 if anything goes wrong, 0 otherwise.

8.164.2.19 int Rtcm::read_MT1019 (const std::string & *message,* Gps_Ephemeris & *gps_eph*)

Verifies and reads messages of type 1019 (GPS Ephemeris). Returns 1 if anything goes wrong, 0 otherwise.

8.164.2.20 int Rtcm::read_MT1045 (const std::string & *message,* Galileo_Ephemeris & *gal_eph*)

Verifies and reads messages of type 1045 (Galileo Ephemeris). Returns 1 if anything goes wrong, 0 otherwise.

The documentation for this class was generated from the following file:

- src/core/system_parameters/rtcm.h

8.165 Rtcm_Printer Class Reference

This class provides a implementation of a subset of the RTCM Standard 10403.2 messages.

`#include <rtcm_printer.h>`

Public Member Functions

- Rtcm_Printer (std::string filename, bool flag_rtcm_server, bool flag_rtcm_tty_port, unsigned short rtcm_tcp↩ _port, unsigned short rtcm_station_id, std::string rtcm_dump_filename, bool time_tag_name=true)

  Default constructor.
- ~Rtcm_Printer ()

  Default destructor.
- bool **Print_Rtcm_MT1001** (const Gps_Ephemeris &gps_eph, double obs_time, const std::map< int, Gnss↩ _Synchro > &observables)
- bool **Print_Rtcm_MT1002** (const Gps_Ephemeris &gps_eph, double obs_time, const std::map< int, Gnss↩ _Synchro > &observables)
- bool **Print_Rtcm_MT1003** (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &cnav_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables)
- bool **Print_Rtcm_MT1004** (const Gps_Ephemeris &gps_eph, const Gps_CNAV_Ephemeris &cnav_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables)
- bool **Print_Rtcm_MT1019** (const Gps_Ephemeris &gps_eph)
- bool **Print_Rtcm_MT1045** (const Galileo_Ephemeris &gal_eph)
- bool **Print_Rtcm_MSM** (unsigned int msm_number, const Gps_Ephemeris &gps_eph, const Gps_CNA↩ V_Ephemeris &gps_cnav_eph, const Galileo_Ephemeris &gal_eph, double obs_time, const std::map< int, Gnss_Synchro > &observables, unsigned int clock_steering_indicator, unsigned int external_clock_indicator, int smooth_int, bool divergence_free, bool more_messages)
- std::string **print_MT1005_test** ()
- unsigned int **lock_time** (const Gps_Ephemeris &eph, double obs_time, const Gnss_Synchro &gnss_synchro)
- unsigned int **lock_time** (const Gps_CNAV_Ephemeris &eph, double obs_time, const Gnss_Synchro &gnss↩ _synchro)
- unsigned int **lock_time** (const Galileo_Ephemeris &eph, double obs_time, const Gnss_Synchro &gnss_↩ synchro)

8.165.1 Detailed Description

This class provides a implementation of a subset of the RTCM Standard 10403.2 messages.

Definition at line 45 of file rtcm_printer.h.

8.165.2 Constructor & Destructor Documentation

8.165.2.1 Rtcm_Printer::Rtcm_Printer (std::string *filename,* bool *flag_rtcm_server,* bool *flag_rtcm_tty_port,* unsigned short *rtcm_tcp_port,* unsigned short *rtcm_station_id,* std::string *rtcm_dump_filename,* bool *time_tag_name* = `true`)

Default constructor.

8.165.2.2 Rtcm_Printer::~Rtcm_Printer ( )

Default destructor.

The documentation for this class was generated from the following file:

- src/algorithms/PVT/libs/rtcm_printer.h

8.166 rtl_tcp_dongle_info Class Reference

This class represents the dongle information which is sent by rtl_tcp.

`#include <rtl_tcp_dongle_info.h>`

Public Types

- enum {
  TUNER_UNKNOWN = 0, **TUNER_E4000**, **TUNER_FC0012**, **TUNER_FC0013**,
  TUNER_FC2580, **TUNER_R820T**, **TUNER_R828D** }

Public Member Functions

- boost::system::error_code **read** (boost::asio::ip::tcp::socket &socket)
- bool **is_valid** () const
- const char * **get_type_name** () const
- double **clip_gain** (int gain) const
- uint32_t **get_tuner_type** () const
- uint32_t **get_tuner_gain_count** () const

8.166.1 Detailed Description

This class represents the dongle information which is sent by rtl_tcp.

Definition at line 41 of file rtl_tcp_dongle_info.h.

The documentation for this class was generated from the following file:

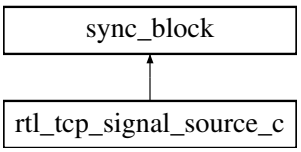
- src/algorithms/signal_source/libs/rtl_tcp_dongle_info.h

8.167 rtl_tcp_signal_source_c Class Reference

This class reads interleaved I/Q samples from an rtl_tcp server and outputs complex types.

`#include <rtl_tcp_signal_source_c.h>`

Inheritance diagram for rtl_tcp_signal_source_c:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)
- void **set_frequency** (int frequency)
- void **set_sample_rate** (int sample_rate)
- void **set_agc_mode** (bool agc)
- void **set_gain** (int gain)
- void **set_if_gain** (int gain)

Friends

- rtl_tcp_signal_source_c_sptr **rtl_tcp_make_signal_source_c** (const std::string &address, short port, bool flip_iq)

8.167.1 Detailed Description

This class reads interleaved I/Q samples from an rtl_tcp server and outputs complex types.

Definition at line 64 of file rtl_tcp_signal_source_c.h.

The documentation for this class was generated from the following file:

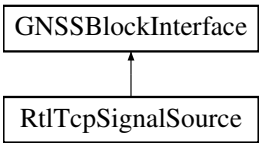
- src/algorithms/signal_source/gnuradio_blocks/rtl_tcp_signal_source_c.h

8.168 RtlTcpSignalSource Class Reference

This class reads from rtl_tcp, which streams interleaved I/Q samples over TCP. (see http://sdr.osmocom.↩ org/trac/wiki/rtl-sdr)

`#include <rtl_tcp_signal_source.h>`

Inheritance diagram for RtlTcpSignalSource:



Public Member Functions

- **RtlTcpSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "RtlTcp_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.168.1 Detailed Description

This class reads from rtl_tcp, which streams interleaved I/Q samples over TCP. (see http://sdr.osmocom.↩ org/trac/wiki/rtl-sdr)

Definition at line 51 of file rtl_tcp_signal_source.h.

8.168.2 Member Function Documentation

8.168.2.1 std::string RtlTcpSignalSource::implementation ( ) [inline], [virtual]

Returns "RtlTcp_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 68 of file rtl_tcp_signal_source.h.

The documentation for this class was generated from the following file:

- src/algorithms/signal_source/adapters/rtl_tcp_signal_source.h

8.169 Sbas_Ephemeris Class Reference

This class stores SBAS SV ephemeris data.

`#include <sbas_ephemeris.h>`

Public Member Functions

- void **print** (std::ostream &out)

Public Attributes

- int i_prn
  PRN number.
- int **i_t0**
- double **d_tof**
- int i_sv_ura
  SV accuracy (URA index), not standardized.
- bool b_sv_do_not_use
  Health status (false:do not use / true:usable)
- double d_pos [3]
  Satellite position (m) (ECEF)
- double d_vel [3]
  Satellite velocity (m/s) (ECEF)
- double d_acc [3]
  Satellite acceleration (m/s^2) (ECEF)
- double d_af0
  Satellite clock-offset (s)
- double d_af1
  Satellite drift (s/s)

8.169.1 Detailed Description

This class stores SBAS SV ephemeris data.

Definition at line 39 of file sbas_ephemeris.h.

8.169.2 Member Data Documentation

8.169.2.1 bool Sbas_Ephemeris::b_sv_do_not_use

Health status (false:do not use / true:usable)

Definition at line 49 of file sbas_ephemeris.h.

8.169.2.2 double Sbas_Ephemeris::d_acc[3]

Satellite acceleration (m/s^2) (ECEF)

Definition at line 52 of file sbas_ephemeris.h.

8.169.2.3 double Sbas_Ephemeris::d_af0

Satellite clock-offset (s)

Definition at line 53 of file sbas_ephemeris.h.

8.169.2.4 double Sbas_Ephemeris::d_af1

Satellite drift (s/s)

Definition at line 54 of file sbas_ephemeris.h.

8.169.2.5 double Sbas_Ephemeris::d_pos[3]

Satellite position (m) (ECEF)

Definition at line 50 of file sbas_ephemeris.h.

8.169.2.6 double Sbas_Ephemeris::d_vel[3]

Satellite velocity (m/s) (ECEF)

Definition at line 51 of file sbas_ephemeris.h.

8.169.2.7 int Sbas_Ephemeris::i_prn

PRN number.

Definition at line 43 of file sbas_ephemeris.h.

8.169.2.8 int Sbas_Ephemeris::i_sv_ura

SV accuracy (URA index), not standardized.

Definition at line 48 of file sbas_ephemeris.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/sbas_ephemeris.h

8.170 Sbas_Ionosphere_Correction Class Reference

Class that handles valid SBAS ionospheric correction for GPS.

`#include <sbas_ionospheric_correction.h>`

Public Member Functions

- void **print** (std::ostream &out)
- bool apply (double sample_stamp, double latitude_d, double longitude_d, double azimut_d, double evaluation_d, double &delay, double &var)

  Computes SBAS ionospheric delay correction.

Public Attributes

- std::vector< Igp_Band > **d_bands**

Friends

- class **boost::serialization::access**

8.170.1 Detailed Description

Class that handles valid SBAS ionospheric correction for GPS.

Definition at line 92 of file sbas_ionospheric_correction.h.

8.170.2 Member Function Documentation

8.170.2.1 bool Sbas_Ionosphere_Correction::apply (double *sample_stamp,* double *latitude_d,* double *longitude_d,* double *azimut_d,* double *evaluation_d,* double & *delay,* double & *var*)

Computes SBAS ionospheric delay correction.

Parameters

out	*delay*	Slant ionospheric delay (L1) (m)
out	*var*	Variance of ionospheric delay (m^2)
in	*sample_stamp*	Sample stamp of observable on which the correction will be applied
in	*longitude_d*	Receiver's longitude in terms of WGS84 (degree)
in	*latitude_d*	Receiver's latitude in terms of WGS84 (degree)
in	*azimuth_d*	Satellite azimuth/elavation angle (rad). Azimuth is the angle of the satellite from the user's location measured clockwise from north
in	*elevation_d*	Elevation is the angle of the satellite from the user's location measured with respect to the local-tangent-plane

The documentation for this class was generated from the following file:

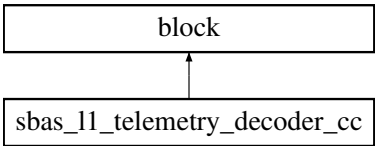
- src/core/system_parameters/sbas_ionospheric_correction.h

8.171 sbas_l1_telemetry_decoder_cc Class Reference

This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229.

`#include <sbas_l1_telemetry_decoder_cc.h>`

Inheritance diagram for sbas_l1_telemetry_decoder_cc:



Public Member Functions

- void set_satellite (Gnss_Satellite satellite)

  Set satellite PRN.
- void set_channel (int channel)

  Set receiver's channel.
- int general_work (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

  This is where all signal processing takes place.
- void forecast (int noutput_items, gr_vector_int &ninput_items_required)

  Function which tells the scheduler how many input items are required to produce noutput_items output items.

Friends

- sbas_l1_telemetry_decoder_cc_sptr **sbas_l1_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

8.171.1 Detailed Description

This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229.

Definition at line 57 of file sbas_l1_telemetry_decoder_cc.h.

8.171.2 Member Function Documentation

8.171.2.1 void sbas_l1_telemetry_decoder_cc::forecast (int *noutput_items,* gr_vector_int & *ninput_items_required*)

Function which tells the scheduler how many input items are required to produce noutput_items output items.

8.171.2.2 int sbas_l1_telemetry_decoder_cc::general_work (int *noutput_items*, gr_vector_int & *ninput_items*, gr_vector_const_void_star & *input_items*, gr_vector_void_star & *output_items*)

This is where all signal processing takes place.

8.171.2.3 void sbas_l1_telemetry_decoder_cc::set_channel (int *channel*)

Set receiver's channel.

8.171.2.4 void sbas_l1_telemetry_decoder_cc::set_satellite (Gnss_Satellite *satellite*)

Set satellite PRN.

The documentation for this class was generated from the following file:

- src/algorithms/telemetry_decoder/gnuradio_blocks/sbas_l1_telemetry_decoder_cc.h

8.172 Sbas_Raw_Msg Class Reference

Represents a raw SBAS message of 250cbits + 6 bits padding (8b preamble + 6b message type + 212b data + 24b CRC + 6b zero padding)

`#include <sbas_telemetry_data.h>`

Public Member Functions

- **Sbas_Raw_Msg** (double sample_stamp, int prn, const std::vector< unsigned char > msg)
- double get_sample_stamp ()

  Time of reception sample stamp (first sample of preample)
- void **relate** (Sbas_Time_Relation time_relation)
- Sbas_Time **get_rx_time_obj** () const
- int **get_prn** () const
- std::vector< unsigned char > **get_msg** () const
- int **get_preamble** ()
- int **get_msg_type** () const
- int **get_crc** ()

8.172.1 Detailed Description

Represents a raw SBAS message of 250cbits + 6 bits padding (8b preamble + 6b message type + 212b data + 24b CRC + 6b zero padding)

Definition at line 57 of file sbas_telemetry_data.h.

8.172.2 Member Function Documentation

8.172.2.1 double Sbas_Raw_Msg::get_sample_stamp ( ) [inline]

Time of reception sample stamp (first sample of preample)

Definition at line 62 of file sbas_telemetry_data.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/sbas_telemetry_data.h

8.173 Sbas_Satellite_Correction Class Reference

Valid long and fast term SBAS corrections for one SV.

`#include <sbas_satellite_correction.h>`

Public Member Functions

- void **print** (std::ostream &out)
- void **print_fast_correction** (std::ostream &out)
- void **print_long_term_correction** (std::ostream &out)
- int **apply_fast** (double sample_stamp, double &pr, double &var)
- int **apply_long_term_sv_pos** (double sample_stamp, double sv_pos[ ], double &var)
- int **apply_long_term_sv_clk** (double sample_stamp, double &dts, double &var)
- bool **alarm** ()

Public Attributes

- int **d_prn**
- Fast_Correction **d_fast_correction**
- Long_Term_Correction **d_long_term_correction**

8.173.1 Detailed Description

Valid long and fast term SBAS corrections for one SV.

Definition at line 64 of file sbas_satellite_correction.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/sbas_satellite_correction.h

8.174 Sbas_Telemetry_Data Class Reference

Public Member Functions

- int **update** (Sbas_Raw_Msg sbas_raw_msg)
- Sbas_Telemetry_Data ()
- ~Sbas_Telemetry_Data ()

8.174.1 Detailed Description

Definition at line 97 of file sbas_telemetry_data.h.

8.174.2 Constructor & Destructor Documentation

8.174.2.1 Sbas_Telemetry_Data::Sbas_Telemetry_Data ( )

Default constructor

8.174.2.2 Sbas_Telemetry_Data::~Sbas_Telemetry_Data ( )

Default deconstructor

The documentation for this class was generated from the following file:

- src/core/system_parameters/sbas_telemetry_data.h

8.175 Sbas_Time Class Reference

Sbas_Time relates the relative sample stamp time scale with the absolute GPS time scale. There are three different states for a Sbas_Time object:

`#include <sbas_time.h>`

Public Types

- enum **Sbas_Time_State** { **RELATIVE**, **RELATED**, **UNDEFINED** }

Public Member Functions

- **Sbas_Time** (double time_stamp_sec)
- **Sbas_Time** (double time_stamp_sec, Sbas_Time_Relation relation)
- void **relate** (Sbas_Time_Relation sbas_time_realtion)
- double **get_time_stamp** ()
- bool **get_gps_time** (int &gps_week, double &gps_sec)
- bool **is_only_relativ** ()
- bool **is_related** ()
- Sbas_Time_State **get_state** ()

8.175.1 Detailed Description

Sbas_Time relates the relative sample stamp time scale with the absolute GPS time scale. There are three different states for a Sbas_Time object:

- only relative time (sample stamp) is known
- only absolute time (gps time) is known
- absolute and relative time and their relation is known

Definition at line 95 of file sbas_time.h.

The documentation for this class was generated from the following file:

- src/core/system_parameters/sbas_time.h

8.176 Sbas_Time_Relation Class Reference

Public Member Functions

- **Sbas_Time_Relation** (double time_stamp_sec, int gps_week, double gps_sec)
- bool **to_gps_time** (double time_stamp_sec, int &gps_week, double &gps_sec)
- bool **to_sample_stamp** (int gps_week, double gps_sec, double &time_stamp_sec)
- bool **is_valid** ()

8.176.1 Detailed Description

Definition at line 41 of file sbas_time.h.

The documentation for this class was generated from the following file:

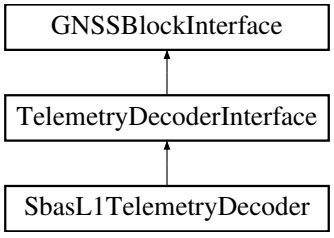
- src/core/system_parameters/sbas_time.h

8.177 SbasL1TelemetryDecoder Class Reference

This class implements a NAV data decoder for SBAS frames in L1 radio link.

`#include <sbas_l1_telemetry_decoder.h>`

Inheritance diagram for SbasL1TelemetryDecoder:



Public Member Functions

- **SbasL1TelemetryDecoder** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams)
- std::string **role** ()
- std::string implementation ()

  Returns "SBAS_L1_Telemetry_Decoder".
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- void **set_satellite** (Gnss_Satellite satellite)
- void **set_channel** (int channel)
- void **reset** ()
- size_t **item_size** ()

8.177.1 Detailed Description

This class implements a NAV data decoder for SBAS frames in L1 radio link.

Definition at line 47 of file sbas_l1_telemetry_decoder.h.

8.177.2 Member Function Documentation

8.177.2.1 std::string SbasL1TelemetryDecoder::implementation ( ) [inline], [virtual]

Returns "SBAS_L1_Telemetry_Decoder".

Implements GNSSBlockInterface.

Definition at line 64 of file sbas_l1_telemetry_decoder.h.

The documentation for this class was generated from the following file:

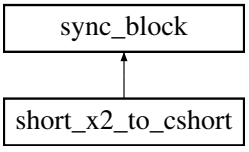
- src/algorithms/telemetry_decoder/adapters/sbas_l1_telemetry_decoder.h

8.178 short_x2_to_cshort Class Reference

This class adapts two short streams into a std::complex<short> stream.

`#include <short_x2_to_cshort.h>`

Inheritance diagram for short_x2_to_cshort:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- short_x2_to_cshort_sptr **make_short_x2_to_cshort** ()

8.178.1 Detailed Description

This class adapts two short streams into a std::complex<short> stream.

Definition at line 47 of file short_x2_to_cshort.h.

The documentation for this class was generated from the following file:

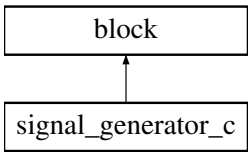
- src/algorithms/libs/short_x2_to_cshort.h

8.179 signal_generator_c Class Reference

This class generates synthesized GNSS signal.

`#include <signal_generator_c.h>`

Inheritance diagram for signal_generator_c:



Public Member Functions

- int **general_work** (int noutput_items, gr_vector_int &ninput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- signal_generator_c_sptr signal_make_generator_c (std::vector< std::string > signal1, std::vector< std::string > system, const std::vector< unsigned int > &PRN, const std::vector< float > &CN0_dB, const std::vector< float > &doppler_Hz, const std::vector< unsigned int > &delay_chips, const std::vector< unsigned int > &delay_sec, bool data_flag, bool noise_flag, unsigned int fs_in, unsigned int vector_length, float BW_BB)

  Return a shared_ptr to a new instance of gen_source.

8.179.1 Detailed Description

This class generates synthesized GNSS signal.

See also

gen_source for a version that subclasses gr_block.

Definition at line 74 of file signal_generator_c.h.

8.179.2 Friends And Related Function Documentation

8.179.2.1 signal_generator_c_sptr signal_make_generator_c (std::vector< std::string > *signal1*, std::vector< std::string > *system*, const std::vector< unsigned int > & *PRN*, const std::vector< float > & *CN0_dB*, const std::vector< float > & *doppler_Hz*, const std::vector< unsigned int > & *delay_chips*, const std::vector< unsigned int > & *delay_sec*, bool *data_flag*, bool *noise_flag*, unsigned int *fs_in*, unsigned int *vector_length*, float *BW_BB*) [friend]

Return a shared_ptr to a new instance of gen_source.

To avoid accidental use of raw pointers, gen_source's constructor is private. signal_make_generator_c is the public interface for creating new instances.

The documentation for this class was generated from the following file:

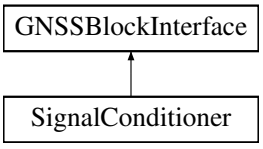
- src/algorithms/signal_generator/gnuradio_blocks/signal_generator_c.h

8.180 SignalConditioner Class Reference

This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal.

`#include <signal_conditioner.h>`

Inheritance diagram for SignalConditioner:



Public Member Functions

- SignalConditioner (ConfigurationInterface *configuration, std::shared_ptr< GNSSBlockInterface > data_type_adapt, std::shared_ptr< GNSSBlockInterface > in_filt, std::shared_ptr< GNSSBlockInterface > res, std::string role, std::string implementation)

  Constructor.
- virtual ~SignalConditioner ()

  Virtual destructor.
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **role** ()
- std::string implementation ()

  Returns "Signal_Conditioner".
- size_t **item_size** ()
- std::shared_ptr< GNSSBlockInterface > **data_type_adapter** ()
- std::shared_ptr< GNSSBlockInterface > **input_filter** ()
- std::shared_ptr< GNSSBlockInterface > **resampler** ()

8.180.1 Detailed Description

This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal.

Definition at line 48 of file signal_conditioner.h.

8.180.2 Constructor & Destructor Documentation

8.180.2.1 SignalConditioner::SignalConditioner (ConfigurationInterface * *configuration*, std::shared_ptr< GNSSBlockInterface > *data_type_adapt*, std::shared_ptr< GNSSBlockInterface > *in_filt*, std::shared_ptr< GNSSBlockInterface > *res*, std::string *role*, std::string *implementation*)

Constructor.

8.180.2.2 virtual SignalConditioner::~SignalConditioner ( ) [virtual]

Virtual destructor.

8.180.3 Member Function Documentation

8.180.3.1 std::string SignalConditioner::implementation ( ) [inline], [virtual]

Returns "Signal_Conditioner".

Implements GNSSBlockInterface.

Definition at line 66 of file signal_conditioner.h.

The documentation for this class was generated from the following file:

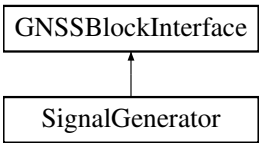
- src/algorithms/conditioner/adapters/signal_conditioner.h

8.181 SignalGenerator Class Reference

This class generates synthesized GNSS signal.

`#include <signal_generator.h>`

Inheritance diagram for SignalGenerator:



Public Member Functions

- **SignalGenerator** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "GNSSSignalGenerator".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()

8.181.1 Detailed Description

This class generates synthesized GNSS signal.

Definition at line 52 of file signal_generator.h.

8.181.2 Member Function Documentation

8.181.2.1 std::string SignalGenerator::implementation ( ) [inline], [virtual]

Returns "GNSSSignalGenerator".

Implements GNSSBlockInterface.

Definition at line 68 of file signal_generator.h.

The documentation for this class was generated from the following file:

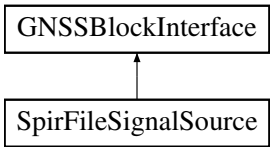
- src/algorithms/signal_generator/adapters/signal_generator.h

8.182 SpirFileSignalSource Class Reference

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

`#include <spir_file_signal_source.h>`

Inheritance diagram for SpirFileSignalSource:



Public Member Functions

- **SpirFileSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "Spir_File_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **filename** ()
- std::string **item_type** ()
- bool **repeat** ()
- long **sampling_frequency** ()
- long **samples** ()

8.182.1 Detailed Description

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

Definition at line 51 of file spir_file_signal_source.h.

8.182.2 Member Function Documentation

8.182.2.1 std::string SpirFileSignalSource::implementation ( ) [inline], [virtual]

Returns "Spir_File_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 67 of file spir_file_signal_source.h.

The documentation for this class was generated from the following file:

- src/algorithms/signal_source/adapters/spir_file_signal_source.h

8.183 StringConverter Class Reference

Class that interprets the contents of a string and converts it into different types.

`#include <string_converter.h>`

Public Member Functions

- bool **convert** (const std::string &value, bool default_value)
- long **convert** (const std::string &value, long default_value)
- int **convert** (const std::string &value, int default_value)
- unsigned int **convert** (const std::string &value, unsigned int default_value)
- unsigned short **convert** (const std::string &value, unsigned short default_value)
- float **convert** (const std::string &value, float default_value)
- double **convert** (const std::string &value, double default_value)

8.183.1 Detailed Description

Class that interprets the contents of a string and converts it into different types.

Definition at line 42 of file string_converter.h.

The documentation for this class was generated from the following file:

- src/core/libs/string_converter.h

8.184 tcp_communication Class Reference

TCP communication class.

`#include <tcp_communication.h>`

Public Member Functions

- int **listen_tcp_connection** (size_t d_port_, size_t d_port_ch0_)
- void **send_receive_tcp_packet_galileo_e1** (boost::array< float, NUM_TX_VARIABLES_GALILEO_E1 > buf, tcp_packet_data *tcp_data_)
- void **send_receive_tcp_packet_gps_l1_ca** (boost::array< float, NUM_TX_VARIABLES_GPS_L1_CA > buf, tcp_packet_data *tcp_data_)
- void **close_tcp_connection** (size_t d_port_)

8.184.1 Detailed Description

TCP communication class.

Definition at line 46 of file tcp_communication.h.

The documentation for this class was generated from the following file:

- src/algorithms/tracking/libs/tcp_communication.h

8.185 tcp_packet_data Class Reference

Class that implements a TCP data packet.

`#include <tcp_packet_data.h>`

Public Attributes

- float **proc_pack_code_error**
- float **proc_pack_carr_error**
- float **proc_pack_carrier_doppler_hz**

8.185.1 Detailed Description

Class that implements a TCP data packet.

Definition at line 38 of file tcp_packet_data.h.

The documentation for this class was generated from the following file:

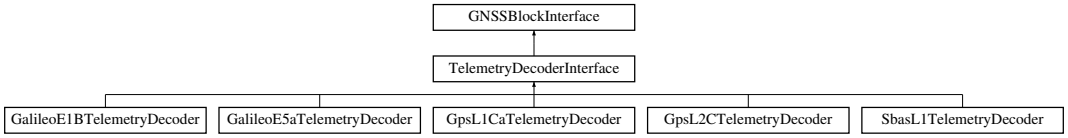
- src/algorithms/tracking/libs/tcp_packet_data.h

8.186 TelemetryDecoderInterface Class Reference

This abstract class represents an interface to a navigation GNSS block.

`#include <telemetry_decoder_interface.h>`

Inheritance diagram for TelemetryDecoderInterface:



Public Member Functions

- virtual void **reset** ()=0
- virtual void **set_satellite** (Gnss_Satellite sat)=0
- virtual void **set_channel** (int channel)=0

8.186.1 Detailed Description

This abstract class represents an interface to a navigation GNSS block.

Abstract class for navigation interfaces. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 51 of file telemetry_decoder_interface.h.

The documentation for this class was generated from the following file:

- src/core/interfaces/telemetry_decoder_interface.h

8.187 tlm_dump_reader Class Reference

Public Member Functions

- bool **read_binary_obs** ()
- bool **restart** ()
- long int **num_epochs** ()
- bool **open_obs_file** (std::string out_file)

Public Attributes

- double **TOW_at_current_symbol**
- double **Prn_timestamp_ms**
- double **d_TOW_at_Preamble**

8.187.1 Detailed Description

Definition at line 39 of file tlm_dump_reader.h.

The documentation for this class was generated from the following file:

- src/tests/unit-tests/signal-processing-blocks/libs/tlm_dump_reader.h

8.188 Tracking_2nd_DLL_filter Class Reference

This class implements a 2nd order DLL filter for code tracking loop.

```
#include <tracking_2nd_DLL_filter.h>
```

Public Member Functions

- void **set_DLL_BW** (float dll_bw_hz)
- void set_pdi (float pdi_code)

  Set DLL filter bandwidth [Hz].
- void initialize ()

  Set Summation interval for code [s].
- float get_code_nco (float DLL_discriminator)

  Start tracking with acquisition information.
- Tracking_2nd_DLL_filter (float pdi_code)

  Numerically controlled oscillator.

8.188.1 Detailed Description

This class implements a 2nd order DLL filter for code tracking loop.

The algorithm is described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S. H. Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007, Applied and Numerical Harmonic Analysis.

Definition at line 48 of file tracking_2nd_DLL_filter.h.

8.188.2 Constructor & Destructor Documentation

8.188.2.1 Tracking_2nd_DLL_filter::Tracking_2nd_DLL_filter (float *pdi_code*)

Numerically controlled oscillator.

8.188.3 Member Function Documentation

8.188.3.1 float Tracking_2nd_DLL_filter::get_code_nco (float *DLL_discriminator*)

Start tracking with acquisition information.

8.188.3.2 void Tracking_2nd_DLL_filter::initialize ( )

Set Summation interval for code [s].

8.188.3.3 void Tracking_2nd_DLL_filter::set_pdi (float *pdi_code*)

Set DLL filter bandwidth [Hz].

The documentation for this class was generated from the following file:

- src/algorithms/tracking/libs/tracking_2nd_DLL_filter.h

8.189 Tracking_2nd_PLL_filter Class Reference

This class implements a 2nd order PLL filter for carrier tracking loop.

`#include <tracking_2nd_PLL_filter.h>`

Public Member Functions

- void **set_PLL_BW** (float pll_bw_hz)
- void set_pdi (float pdi_carr)

  Set PLL loop bandwidth [Hz].
- void initialize ()

  Set Summation interval for code [s].
- float **get_carrier_nco** (float PLL_discriminator)
- **Tracking_2nd_PLL_filter** (float pdi_carr)

8.189.1 Detailed Description

This class implements a 2nd order PLL filter for carrier tracking loop.

The algorithm is described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S. H. Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007, Applied and Numerical Harmonic Analysis.

Definition at line 47 of file tracking_2nd_PLL_filter.h.

8.189.2 Member Function Documentation

8.189.2.1 void Tracking_2nd_PLL_filter::initialize ( )

Set Summation interval for code [s].

8.189.2.2 void Tracking_2nd_PLL_filter::set_pdi (float *pdi_carr*)

Set PLL loop bandwidth [Hz].

The documentation for this class was generated from the following file:

- src/algorithms/tracking/libs/tracking_2nd_PLL_filter.h

8.190 tracking_dump_reader Class Reference

Public Member Functions

- bool **read_binary_obs** ()
- bool **restart** ()
- long int **num_epochs** ()
- bool **open_obs_file** (std::string out_file)

Public Attributes

- float **abs_E**
- float **abs_P**
- float **abs_L**
- float **prompt_I**
- float **prompt_Q**
- unsigned long int **PRN_start_sample_count**
- double **acc_carrier_phase_rad**
- double **carrier_doppler_hz**
- double **code_freq_chips**
- double **carr_error_hz**
- double **carr_error_filt_hz**
- double **code_error_chips**
- double **code_error_filt_chips**
- double **CN0_SNV_dB_Hz**
- double **carrier_lock_test**
- double **aux1**
- double **aux2**

8.190.1 Detailed Description

Definition at line 39 of file tracking_dump_reader.h.

The documentation for this class was generated from the following file:

- src/tests/unit-tests/signal-processing-blocks/libs/tracking_dump_reader.h

8.191 Tracking_FLL_PLL_filter Class Reference

This class implements a hybrid FLL and PLL filter for tracking carrier loop.

`#include <tracking_FLL_PLL_filter.h>`

Public Member Functions

- void **set_params** (float fll_bw_hz, float pll_bw_hz, int order)
- void **initialize** (float d_acq_carrier_doppler_hz)
- float **get_carrier_error** (float FLL_discriminator, float PLL_discriminator, float correlation_time_s)

8.191.1 Detailed Description

This class implements a hybrid FLL and PLL filter for tracking carrier loop.

Definition at line 37 of file tracking_FLL_PLL_filter.h.

The documentation for this class was generated from the following file:

- src/algorithms/tracking/libs/tracking_FLL_PLL_filter.h

8.192 Tracking_loop_filter Class Reference

This class implements a generic 1st, 2nd or 3rd order loop filter.

`#include <tracking_loop_filter.h>`

Public Member Functions

- float **get_noise_bandwidth** (void) const
- float **get_update_interval** (void) const
- bool **get_include_last_integrator** (void) const
- int **get_order** (void) const
- void **set_noise_bandwidth** (float noise_bandwidth)
- void **set_update_interval** (float update_interval)
- void **set_include_last_integrator** (bool include_last_integrator)
- void **set_order** (int loop_order)
- void **initialize** (float initial_output=0.0)
- float **apply** (float current_input)
- **Tracking_loop_filter** (float update_interval, float noise_bandwidth, int loop_order=2, bool include_last_↩ integrator=false)

8.192.1 Detailed Description

This class implements a generic 1st, 2nd or 3rd order loop filter.

Definition at line 44 of file tracking_loop_filter.h.

The documentation for this class was generated from the following file:

- src/algorithms/tracking/libs/tracking_loop_filter.h

8.193 tracking_true_obs_reader Class Reference

Public Member Functions

- bool **read_binary_obs** ()
- bool **restart** ()
- long int **num_epochs** ()
- bool **open_obs_file** (std::string out_file)

Public Attributes

- bool **d_dump**
- double **signal_timestamp_s**
- double **acc_carrier_phase_cycles**
- double **doppler_l1_hz**
- double **prn_delay_chips**
- double **tow**

8.193.1 Detailed Description

Definition at line 39 of file tracking_true_obs_reader.h.

The documentation for this class was generated from the following file:

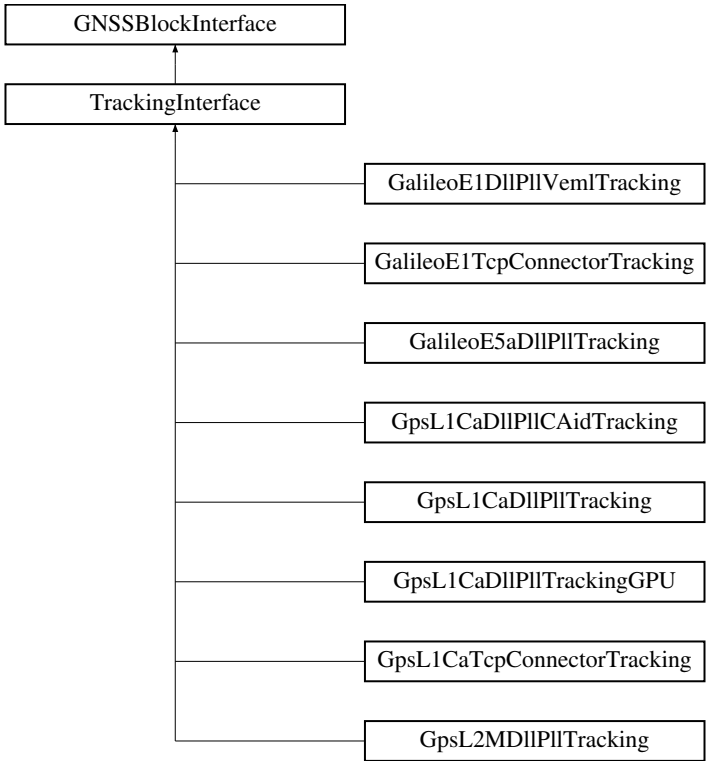
- src/tests/unit-tests/signal-processing-blocks/libs/tracking_true_obs_reader.h

8.194 TrackingInterface Class Reference

This abstract class represents an interface to a tracking block.

`#include <tracking_interface.h>`

Inheritance diagram for TrackingInterface:



Public Member Functions

- virtual void **start_tracking** ()=0
- virtual void **set_gnss_synchro** (Gnss_Synchro *gnss_synchro)=0
- virtual void **set_channel** (unsigned int channel)=0

8.194.1 Detailed Description

This abstract class represents an interface to a tracking block.

Abstract class for tracking interfaces. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Definition at line 54 of file tracking_interface.h.

The documentation for this class was generated from the following file:

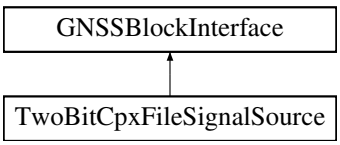
- src/core/interfaces/tracking_interface.h

8.195 TwoBitCpxFileSignalSource Class Reference

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

`#include <two_bit_cpx_file_signal_source.h>`

Inheritance diagram for TwoBitCpxFileSignalSource:



Public Member Functions

- **TwoBitCpxFileSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_streams, unsigned int out_streams, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "Two_Bit_Cpx_File_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **filename** ()
- std::string **item_type** ()
- bool **repeat** ()
- long **sampling_frequency** ()
- long **samples** ()

8.195.1 Detailed Description

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

Definition at line 55 of file two_bit_cpx_file_signal_source.h.

8.195.2 Member Function Documentation

8.195.2.1 std::string TwoBitCpxFileSignalSource::implementation ( ) [inline], [virtual]

Returns "Two_Bit_Cpx_File_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 71 of file two_bit_cpx_file_signal_source.h.

The documentation for this class was generated from the following file:

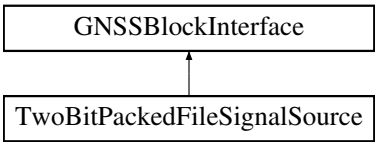
- src/algorithms/signal_source/adapters/two_bit_cpx_file_signal_source.h

8.196 TwoBitPackedFileSignalSource Class Reference

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

`#include <two_bit_packed_file_signal_source.h>`

Inheritance diagram for TwoBitPackedFileSignalSource:



Public Member Functions

- **TwoBitPackedFileSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_↩ streams, unsigned int out_streams, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "Two_Bit_Packed_File_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- std::string **filename** ()
- std::string **item_type** ()
- bool **repeat** ()
- long **sampling_frequency** ()
- long **samples** ()
- bool **big_endian_items** ()
- bool **big_endian_bytes** ()
- bool **is_complex** ()
- bool **reverse_interleaving** ()

8.196.1 Detailed Description

Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

Definition at line 56 of file two_bit_packed_file_signal_source.h.

8.196.2 Member Function Documentation

8.196.2.1 std::string TwoBitPackedFileSignalSource::implementation ( ) [inline], [virtual]

Returns "Two_Bit_Packed_File_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 72 of file two_bit_packed_file_signal_source.h.

The documentation for this class was generated from the following file:

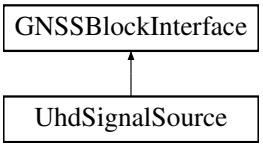
- src/algorithms/signal_source/adapters/two_bit_packed_file_signal_source.h

8.197 UhdSignalSource Class Reference

This class reads samples from a UHD device (see http://code.ettus.com/redmine/ettus/projects/uhd/wiki)

`#include <uhd_signal_source.h>`

Inheritance diagram for UhdSignalSource:



Public Member Functions

- **UhdSignalSource** (ConfigurationInterface *configuration, std::string role, unsigned int in_stream, unsigned int out_stream, boost::shared_ptr< gr::msg_queue > queue)
- std::string **role** ()
- std::string implementation ()

  Returns "UHD_Signal_Source".
- size_t **item_size** ()
- void **connect** (gr::top_block_sptr top_block)
- void **disconnect** (gr::top_block_sptr top_block)
- gr::basic_block_sptr **get_left_block** ()
- gr::basic_block_sptr **get_right_block** ()
- gr::basic_block_sptr **get_right_block** (int RF_channel)

8.197.1 Detailed Description

This class reads samples from a UHD device (see http://code.ettus.com/redmine/ettus/projects/uhd/wiki)

Definition at line 48 of file uhd_signal_source.h.

8.197.2 Member Function Documentation

8.197.2.1 std::string UhdSignalSource::implementation ( ) [inline], [virtual]

Returns "UHD_Signal_Source".

Implements GNSSBlockInterface.

Definition at line 66 of file uhd_signal_source.h.

The documentation for this class was generated from the following file:

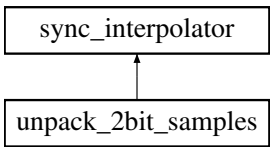
- src/algorithms/signal_source/adapters/uhd_signal_source.h

8.198 unpack_2bit_samples Class Reference

This class takes 2 bit samples that have been packed into bytes or shorts as input and generates a byte for each sample. It generates eight times as much data as is input (every two bits become 16 bits)

`#include <unpack_2bit_samples.h>`

Inheritance diagram for unpack_2bit_samples:



Public Member Functions

- **unpack_2bit_samples** (bool big_endianBytes, size_t item_size, bool big_endian_items, bool reverse_↩ interleaving)
- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- unpack_2bit_samples_sptr **make_unpack_2bit_samples_sptr** (bool big_endian_bytes, size_t item_size, bool big_endian_items, bool reverse_interleaving)

8.198.1 Detailed Description

This class takes 2 bit samples that have been packed into bytes or shorts as input and generates a byte for each sample. It generates eight times as much data as is input (every two bits become 16 bits)

Definition at line 89 of file unpack_2bit_samples.h.

The documentation for this class was generated from the following file:

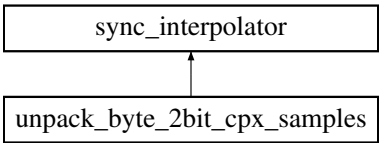
- src/algorithms/signal_source/gnuradio_blocks/unpack_2bit_samples.h

8.199 unpack_byte_2bit_cpx_samples Class Reference

This class implements conversion between byte packet samples to 2bit_cpx samples 1 byte = 2 x complex 2bit I, + 2bit Q samples.

`#include <unpack_byte_2bit_cpx_samples.h>`

Inheritance diagram for unpack_byte_2bit_cpx_samples:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- unpack_byte_2bit_cpx_samples_sptr **make_unpack_byte_2bit_cpx_samples_sptr** ()

8.199.1 Detailed Description

This class implements conversion between byte packet samples to 2bit_cpx samples 1 byte = 2 x complex 2bit I, + 2bit Q samples.

Definition at line 50 of file unpack_byte_2bit_cpx_samples.h.

The documentation for this class was generated from the following file:

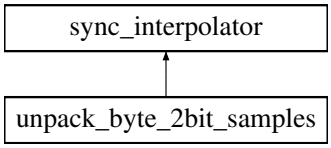
- src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_cpx_samples.h

8.200 unpack_byte_2bit_samples Class Reference

This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples.

`#include <unpack_byte_2bit_samples.h>`

Inheritance diagram for unpack_byte_2bit_samples:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- unpack_byte_2bit_samples_sptr **make_unpack_byte_2bit_samples_sptr** ()

8.200.1 Detailed Description

This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples.

Definition at line 46 of file unpack_byte_2bit_samples.h.

The documentation for this class was generated from the following file:

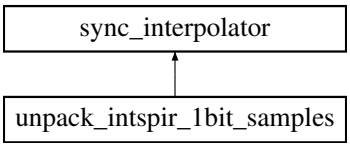
- src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_samples.h

8.201 unpack_intspir_1bit_samples Class Reference

This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples.

`#include <unpack_intspir_1bit_samples.h>`

Inheritance diagram for unpack_intspir_1bit_samples:



Public Member Functions

- int **work** (int noutput_items, gr_vector_const_void_star &input_items, gr_vector_void_star &output_items)

Friends

- unpack_intspir_1bit_samples_sptr **make_unpack_intspir_1bit_samples_sptr** ()

8.201.1 Detailed Description

This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples.

Definition at line 46 of file unpack_intspir_1bit_samples.h.

The documentation for this class was generated from the following file:

- src/algorithms/signal_source/gnuradio_blocks/unpack_intspir_1bit_samples.h

8.202 Viterbi_Decoder Class Reference

Class that implements a Viterbi decoder.

`#include <viterbi_decoder.h>`

Public Member Functions

- **Viterbi_Decoder** (const int g_encoder[ ], const int KK, const int nn)
- void **reset** ()
- float decode_block (const double input_c[ ], int *output_u_int, const int LL)

  Uses the Viterbi algorithm to perform hard-decision decoding of a convolutional code.
- float **decode_continuous** (const double sym[ ], const int traceback_depth, int output_u_int[ ], const int nbits_requested, int &nbits_decoded)

8.202.1 Detailed Description

Class that implements a Viterbi decoder.

Definition at line 41 of file viterbi_decoder.h.

8.202.2 Member Function Documentation

8.202.2.1 float Viterbi_Decoder::decode_block (const double *input_c[ ],* int * *output_u_int,* const int *LL*)

Uses the Viterbi algorithm to perform hard-decision decoding of a convolutional code.

Parameters

in	*input_c[]*	The received signal in LLR-form. For BPSK, must be in form r = 2∗a∗y/(sigma^2).
in	*LL*	The number of data bits to be decoded (does not include the mm zero-tail-bits)

Returns

output_u_int[] Hard decisions on the data bits (without the mm zero-tail-bits)

The documentation for this class was generated from the following file:

- src/algorithms/telemetry_decoder/libs/viterbi_decoder.h

Chapter 9

File Documentation

9.1 src/algorithms/acquisition/adapters/galileo_e1_pcps_8ms_ambiguous_acquisition.h File Reference

Adapts a PCPS 8ms acquisition block to an AcquisitionInterface for Galileo E1 Signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "galileo_pcps_8ms_acquisition_cc.h"
```

Classes

- class GalileoE1Pcps8msAmbiguousAcquisition

  Adapts a PCPS 8ms acquisition block to an AcquisitionInterface for Galileo E1 Signals.

9.1.1 Detailed Description

Adapts a PCPS 8ms acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.2 src/algorithms/acquisition/adapters/galileo_e1_pcps_ambiguous_acquisition.h File Reference

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include <gnuradio/blocks/float_to_complex.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_acquisition_cc.h"
#include "pcps_acquisition_sc.h"
#include "complex_byte_to_float_x2.h"
#include <volk_gnsssdr/volk_gnsssdr.h>
```

Classes

- class GalileoE1PcpsAmbiguousAcquisition

  This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

9.2.1 Detailed Description

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.3 src/algorithms/acquisition/adapters/galileo_e1_pcps_cccwsr_ambiguous_acquisition.h File Reference

Adapts a PCPS CCCWSR acquisition block to an AcquisitionInterface for Galileo E1 Signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_cccwsr_acquisition_cc.h"
```

Classes

- class GalileoE1PcpsCccwsrAmbiguousAcquisition

  Adapts a PCPS CCCWSR acquisition block to an AcquisitionInterface for Galileo E1 Signals.

9.3.1 Detailed Description

Adapts a PCPS CCCWSR acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.4 src/algorithms/acquisition/adapters/galileo_e1_pcps_quicksync_ambiguous_acquisition.h File Reference

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_quicksync_acquisition_cc.h"
```

Classes

- class GalileoE1PcpsQuickSyncAmbiguousAcquisition

  This class adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

9.4.1 Detailed Description

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Date

June, 2014

Author

Damian Miralles Sanchez. dmiralles2009@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.5 src/algorithms/acquisition/adapters/galileo_e1_pcps_tong_ambiguous_acquisition.h File Reference

Adapts a PCPS Tong acquisition block to an AcquisitionInterface for Galileo E1 Signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_tong_acquisition_cc.h"
```

Classes

- class GalileoE1PcpsTongAmbiguousAcquisition

  Adapts a PCPS Tong acquisition block to an AcquisitionInterface for Galileo E1 Signals.

9.5.1 Detailed Description

Adapts a PCPS Tong acquisition block to an AcquisitionInterface for Galileo E1 Signals.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.6 src/algorithms/acquisition/adapters/galileo_e5a_noncoherent_iq_acquisition_caf.h File Reference

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E5a data and pilot Signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "galileo_e5a_noncoherent_iq_acquisition_caf_cc.h"
```

Classes

- class GalileoE5aNoncoherentIQAcquisitionCaf

9.6.1 Detailed Description

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E5a data and pilot Signals.

Author

Marc Sales, 2014. marcsales92(at)gmail.com on work from:

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com
- Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.7 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition.h File Reference

Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include <gnuradio/blocks/float_to_complex.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_acquisition_cc.h"
#include "pcps_acquisition_sc.h"
#include "complex_byte_to_float_x2.h"
#include <volk_gnsssdr/volk_gnsssdr.h>
```

Classes

- class GpsL1CaPcpsAcquisition

  This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

9.7.1 Detailed Description

Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com
- Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.8 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition_fine_doppler.h File Reference

Adapts a PCPS acquisition block with fine Doppler estimation to an AcquisitionInterface for GPS L1 C/A signals.

```
#include <string>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_acquisition_fine_doppler_cc.h"
```

Classes

- class GpsL1CaPcpsAcquisitionFineDoppler

  This class Adapts a PCPS acquisition block with fine Doppler estimation to an AcquisitionInterface for GPS L1 C/A signals.

9.8.1 Detailed Description

Adapts a PCPS acquisition block with fine Doppler estimation to an AcquisitionInterface for GPS L1 C/A signals.

Authors

- Javier Arribas, 2013. jarribas(at)cttc.es

*

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.9 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_assisted_acquisition.h File Reference

Adapts a PCPS Assisted acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

```
#include <string>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_assisted_acquisition_cc.h"
```

Classes

- class GpsL1CaPcpsAssistedAcquisition

  This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

9.9.1 Detailed Description

Adapts a PCPS Assisted acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es

*

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.10 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_multithread_acquisition.h File Reference

Adapts a multithread PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_multithread_acquisition_cc.h"
```

Classes

- class GpsL1CaPcpsMultithreadAcquisition

  This class adapts a multithread PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

9.10.1 Detailed Description

Adapts a multithread PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.11 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_opencl_acquisition.h File Reference

Adapts an OpenCL PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_opencl_acquisition_cc.h"
```

Classes

- class GpsL1CaPcpsOpenClAcquisition

  This class adapts an OpenCL PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

9.11.1 Detailed Description

Adapts an OpenCL PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.12 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_quicksync_acquisition.h File Reference

Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals implementing the QuickSync Algorithm.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_quicksync_acquisition_cc.h"
#include "configuration_interface.h"
```

Classes

- class GpsL1CaPcpsQuickSyncAcquisition

  This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

9.12.1 Detailed Description

Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L1 C/A signals implementing the QuickSync Algorithm.

Date

June, 2014

Author

Damian Miralles Sanchez. dmiralles2009@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.13 src/algorithms/acquisition/adapters/gps_l1_ca_pcps_tong_acquisition.h File Reference

Adapts a PCPS Tong acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_tong_acquisition_cc.h"
#include "configuration_interface.h"
```

Classes

- class GpsL1CaPcpsTongAcquisition

  This class adapts a PCPS Tong acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

9.13.1 Detailed Description

Adapts a PCPS Tong acquisition block to an AcquisitionInterface for GPS L1 C/A signals.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.14 src/algorithms/acquisition/adapters/gps_l2_m_pcps_acquisition.h File Reference

Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L2 M signals.

```
#include <string>
#include <gnuradio/blocks/stream_to_vector.h>
#include <gnuradio/blocks/float_to_complex.h>
#include "gnss_synchro.h"
#include "acquisition_interface.h"
#include "pcps_acquisition_cc.h"
#include "pcps_acquisition_sc.h"
#include "complex_byte_to_float_x2.h"
#include <volk_gnsssdr/volk_gnsssdr.h>
```

Classes

- class GpsL2MPcpsAcquisition

  This class adapts a PCPS acquisition block to an AcquisitionInterface for GPS L2 M signals.

9.14.1 Detailed Description

Adapts a PCPS acquisition block to an AcquisitionInterface for GPS L2 M signals.

Authors

- Javier Arribas, 2015. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.15 src/algorithms/acquisition/gnuradio_blocks/galileo_e5a_noncoherent_iq_acquisition_caf_cc.h File Reference

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E5a data and pilot Signals.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class galileo_e5a_noncoherentIQ_acquisition_caf_cc

  This class implements a Parallel Code Phase Search Acquisition.

Typedefs

- typedef boost::shared_ptr< galileo_e5a_noncoherentIQ_acquisition_caf_cc > **galileo_e5a_noncoherentIQ_acquisition_caf_cc_sptr**

Functions

- galileo_e5a_noncoherentIQ_acquisition_caf_cc_sptr **galileo_e5a_noncoherentIQ_make_acquisition_↩ caf_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_transition_flag, bool dump, std::string dump_filename, bool both_signal_components_, int CAF_window_hz_, int Zero_padding_)

9.15.1 Detailed Description

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E5a data and pilot Signals.

Author

Marc Sales, 2014. marcsales92(at)gmail.com on work from:

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com
- Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.16 src/algorithms/acquisition/gnuradio_blocks/galileo_pcps_8ms_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search Acquisition for Galileo E1 signals with coherent integration time = 8 ms (two codes)

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class galileo_pcps_8ms_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition for Galileo E1 signals with coherent integration time = 8 ms (two codes)

Typedefs

- typedef boost::shared_ptr< galileo_pcps_8ms_acquisition_cc > **galileo_pcps_8ms_acquisition_cc_sptr**

Functions

- galileo_pcps_8ms_acquisition_cc_sptr **galileo_pcps_8ms_make_acquisition_cc** (unsigned int sampled↩ _ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool dump, std::string dump_filename)

9.16.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition for Galileo E1 signals with coherent integration time = 8 ms (two codes)

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.17 src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search Acquisition.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class pcps_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition.

Typedefs

- typedef boost::shared_ptr< pcps_acquisition_cc > **pcps_acquisition_cc_sptr**

Functions

- pcps_acquisition_cc_sptr **pcps_make_acquisition_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_↩ transition_flag, bool use_CFAR_algorithm_flag, bool dump, std::string dump_filename)

9.17.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Acquisition strategy (Kay Borre book + CFAR threshold).

1. Compute the input signal power estimation
2. Doppler serial search loop
3. Perform the FFT-based circular convolution (parallel time search)
4. Record the maximum peak and the associated synchronization parameters
5. Compute the test statistics and compare to the threshold
6. Declare positive or negative acquisition using a message queue

Kay Borre book: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, "A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach", Birkha user, 2007. pp 81-84

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com
- Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.18 src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_sc.h File Reference

This class implements a Parallel Code Phase Search Acquisition.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class pcps_acquisition_sc

  This class implements a Parallel Code Phase Search Acquisition.

Typedefs

- typedef boost::shared_ptr< pcps_acquisition_sc > **pcps_acquisition_sc_sptr**

Functions

- pcps_acquisition_sc_sptr **pcps_make_acquisition_sc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_↩ transition_flag, bool use_CFAR_algorithm_flag, bool dump, std::string dump_filename)

9.18.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Acquisition strategy (Kay Borre book + CFAR threshold).

1. Compute the input signal power estimation
2. Doppler serial search loop
3. Perform the FFT-based circular convolution (parallel time search)
4. Record the maximum peak and the associated synchronization parameters
5. Compute the test statistics and compare to the threshold
6. Declare positive or negative acquisition using a message port

Kay Borre book: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, "A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach", Birkha user, 2007. pp 81-84

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com
- Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.19 src/algorithms/acquisition/gnuradio_blocks/pcps_assisted_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search Acquisition with assistance and multi-dwells.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class pcps_assisted_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition.

Typedefs

- typedef boost::shared_ptr< pcps_assisted_acquisition_cc > **pcps_assisted_acquisition_cc_sptr**

Functions

- pcps_assisted_acquisition_cc_sptr **pcps_make_assisted_acquisition_cc** (int max_dwells, unsigned int sampled_ms, int doppler_max, int doppler_min, long freq, long fs_in, int samples_per_ms, bool dump, std::string dump_filename)

9.19.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition with assistance and multi-dwells.

Acquisition strategy (Kay Borre book + CFAR threshold).

1. Compute the input signal power estimation
2. Doppler serial search loop
3. Perform the FFT-based circular convolution (parallel time search)
4. Record the maximum peak and the associated synchronization parameters
5. Compute the test statistics and compare to the threshold
6. Declare positive or negative acquisition using a message queue

Kay Borre book: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, "A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach", Birkha user, 2007. pp 81-84

Authors

- Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.20 src/algorithms/acquisition/gnuradio_blocks/pcps_cccwsr_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search acquisition with Coherent Channel Combining With Sign Recovery scheme.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class pcps_cccwsr_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition with Coherent Channel Combining With Sign Recovery scheme.

Typedefs

- typedef boost::shared_ptr< pcps_cccwsr_acquisition_cc > **pcps_cccwsr_acquisition_cc_sptr**

Functions

- pcps_cccwsr_acquisition_cc_sptr **pcps_cccwsr_make_acquisition_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool dump, std::string dump_filename)

9.20.1 Detailed Description

This class implements a Parallel Code Phase Search acquisition with Coherent Channel Combining With Sign Recovery scheme.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

D.Borio, C.O'Driscoll, G.Lachapelle, "Coherent, Noncoherent and Differentially Coherent Combining Techniques for Acquisition of New Composite GNSS Signals", IEEE Transactions On Aerospace and Electronic Systems vol. 45 no. 3, July 2009, section IV

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.21 src/algorithms/acquisition/gnuradio_blocks/pcps_multithread_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search Acquisition.

```
#include <algorithm>
#include <fstream>
#include <string>
#include <vector>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class pcps_multithread_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition.

Typedefs

- typedef boost::shared_ptr< pcps_multithread_acquisition_cc > **pcps_multithread_acquisition_cc_sptr**

Functions

- pcps_multithread_acquisition_cc_sptr **pcps_make_multithread_acquisition_cc** (unsigned int sampled_↩ ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_transition_flag, bool dump, std::string dump_filename)

9.21.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition.

Acquisition strategy (Kay Borre book + CFAR threshold).

1. Compute the input signal power estimation
2. Doppler serial search loop
3. Perform the FFT-based circular convolution (parallel time search)
4. Record the maximum peak and the associated synchronization parameters
5. Compute the test statistics and compare to the threshold
6. Declare positive or negative acquisition using a message port

Kay Borre book: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, "A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach", Birkha user, 2007. pp 81-84

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com
- Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.22 src/algorithms/acquisition/gnuradio_blocks/pcps_opencl_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search Acquisition using OpenCL to offload some functions to the GPU.

```
#include <fstream>
#include <string>
#include <vector>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "fft_internal.h"
#include "gnss_synchro.h"
#include <CL/cl.hpp>
```

Classes

- class pcps_opencl_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition.

Typedefs

- typedef boost::shared_ptr< pcps_opencl_acquisition_cc > **pcps_opencl_acquisition_cc_sptr**

Functions

- pcps_opencl_acquisition_cc_sptr **pcps_make_opencl_acquisition_cc** (unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_↩ per_code, bool bit_transition_flag, bool dump, std::string dump_filename)

9.22.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition using OpenCL to offload some functions to the GPU.

Acquisition strategy (Kay Borre book + CFAR threshold).

1. Compute the input signal power estimation
2. Doppler serial search loop
3. Perform the FFT-based circular convolution (parallel time search)
4. Record the maximum peak and the associated synchronization parameters
5. Compute the test statistics and compare to the threshold
6. Declare positive or negative acquisition using a message port

Kay Borre book: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, "A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach", Birkha user, 2007. pp 81-84

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com
- Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.23 src/algorithms/acquisition/gnuradio_blocks/pcps_quicksync_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search Acquisition with the QuickSync Algorithm.

```
#include <fstream>
#include <string>
#include <algorithm>
#include <functional>
#include <assert.h>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class pcps_quicksync_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition with the implementation of the Sparse QuickSync Algorithm.

Typedefs

- typedef boost::shared_ptr< pcps_quicksync_acquisition_cc > **pcps_quicksync_acquisition_cc_sptr**

Functions

- pcps_quicksync_acquisition_cc_sptr **pcps_quicksync_make_acquisition_cc** (unsigned int folding_factor, unsigned int sampled_ms, unsigned int max_dwells, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, bool bit_transition_flag, bool dump, std::string dump_filename)

9.23.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition with the QuickSync Algorithm.

Acquisition strategy (Kay Borre book CFAR + threshold).

1. Compute the input signal power estimation
2. Doppler serial search loop
3. Perform folding of the incoming signal and local generated code
4. Perform the FFT-based circular convolution (parallel time search)
5. Record the maximum peak and the associated synchronization parameters
6. Compute the test statistics and compare to the threshold
7. Declare positive or negative acquisition using a message port
8. Obtain the adequate acquisition parameters by correlating the incoming signal shifted by the possible folded delays

Kay Borre book: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, "A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach", Birkha user, 2007. pp 81-84

Date

Jun2 2014

Author

Damian Miralles Sanchez, dmiralles2009@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.24 src/algorithms/acquisition/gnuradio_blocks/pcps_tong_acquisition_cc.h File Reference

This class implements a Parallel Code Phase Search Acquisition with Tong algorithm.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <gnuradio/gr_complex.h>
#include <gnuradio/fft/fft.h>
#include "gnss_synchro.h"
```

Classes

- class pcps_tong_acquisition_cc

  This class implements a Parallel Code Phase Search Acquisition with Tong algorithm.

Typedefs

- typedef boost::shared_ptr< pcps_tong_acquisition_cc > **pcps_tong_acquisition_cc_sptr**

Functions

- pcps_tong_acquisition_cc_sptr **pcps_tong_make_acquisition_cc** (unsigned int sampled_ms, unsigned int doppler_max, long freq, long fs_in, int samples_per_ms, int samples_per_code, unsigned int tong_init_val, unsigned int tong_max_val, unsigned int tong_max_dwells, bool dump, std::string dump_filename)

9.24.1 Detailed Description

This class implements a Parallel Code Phase Search Acquisition with Tong algorithm.

Author

Marc Molina, 2013. marc.molina.pena(at)gmail.com

Acquisition strategy (Kaplan book + CFAR threshold).

1. Compute the input signal power estimation.
2. Doppler serial search loop.
3. Perform the FFT-based circular convolution (parallel time search).
4. Compute the tests statistics for all the cells.
5. Accumulate the grid of tests statistics with the previous grids.
6. Record the maximum peak and the associated synchronization parameters.
7. Compare the maximum averaged test statistics with a threshold.
8. If the test statistics exceeds the threshold, increment the Tong counter.
9. Otherwise, decrement the Tong counter.
10. If the Tong counter is equal to a given maximum value, declare positive
11. acquisition. If the Tong counter is equa to zero, declare negative
12. acquisition. Otherwise, process the next block.

Kaplan book: D.Kaplan, J.Hegarty, "Understanding GPS. Principles and Applications", Artech House, 2006, pp 223-227

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.25 src/algorithms/channel/adapters/channel.h File Reference

Interface of a GNSS channel.

```
#include <memory>
#include <string>
#include <gnuradio/msg_queue.h>
#include <gnuradio/block.h>
#include "channel_interface.h"
#include "channel_fsm.h"
#include "gnss_synchro.h"
#include "channel_msg_receiver_cc.h"
```

Classes

- class Channel

  This class represents a GNSS channel. It wraps an AcquisitionInterface, a Tracking Interface and a TelemetryDecoderInterface, and handles their interaction through a Finite State Machine.

9.25.1 Detailed Description

Interface of a GNSS channel.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Luis Esteve, 2011. luis(at)epsilon-formacion.com

It holds blocks for acquisition, tracking, navigation data extraction and pseudorange calculation.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.26 src/algorithms/channel/libs/channel_fsm.h File Reference

Interface of the State Machine for channel using boost::statechart.

```
#include <boost/statechart/state_machine.hpp>
#include <gnuradio/msg_queue.h>
#include "acquisition_interface.h"
#include "tracking_interface.h"
#include "telemetry_decoder_interface.h"
```

Classes

- class ChannelFsm

  This class implements a State Machine for channel using boost::statechart.

9.26.1 Detailed Description

Interface of the State Machine for channel using boost::statechart.

Author

Luis Esteve, 2011. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.27 src/algorithms/channel/libs/channel_msg_receiver_cc.h File Reference

GNU Radio block that receives asynchronous channel messages from acquisition and tracking blocks.

```
#include <gnuradio/block.h>
#include "channel_fsm.h"
```

Classes

- class channel_msg_receiver_cc

  GNU Radio block that receives asynchronous channel messages from acquisition and tracking blocks.

Typedefs

- typedef boost::shared_ptr< channel_msg_receiver_cc > **channel_msg_receiver_cc_sptr**

Functions

- channel_msg_receiver_cc_sptr **channel_msg_receiver_make_cc** (ChannelFsm *channel_fsm, bool repeat)

9.27.1 Detailed Description

GNU Radio block that receives asynchronous channel messages from acquisition and tracking blocks.

Author

Javier Arribas, 2016. jarribas(at)cttc.es

Copyright (C) 2010-2016 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.28 src/algorithms/conditioner/adapters/array_signal_conditioner.h File Reference

It wraps blocks to change data type, filter and resample input data, adapted to array receiver.

```
#include <string>
#include <gnuradio/msg_queue.h>
#include "gnss_block_interface.h"
```

Classes

- class ArraySignalConditioner

  This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal.

9.28.1 Detailed Description

It wraps blocks to change data type, filter and resample input data, adapted to array receiver.

Author

Javier Arribas jarribas (at) cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.29 src/algorithms/conditioner/adapters/signal_conditioner.h File Reference

It wraps blocks to change data type, filter and resample input data.

```
#include <string>
#include "gnss_block_interface.h"
```

Classes

- class SignalConditioner

  This class wraps blocks to change data_type_adapter, input_filter and resampler to be applied to the input flow of sampled signal.

9.29.1 Detailed Description

It wraps blocks to change data type, filter and resample input data.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.30 src/algorithms/data_type_adapter/adapters/byte_to_short.h File Reference

Adapts an 8-bits sample stream (IF) to a short int stream (IF)

```
#include <string>
#include <gnuradio/blocks/char_to_short.h>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_block_interface.h"
```

Classes

- class ByteToShort

  Adapts an 8-bits sample stream (IF) to a short int stream (IF)

9.30.1 Detailed Description

Adapts an 8-bits sample stream (IF) to a short int stream (IF)

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.31 src/algorithms/data_type_adapter/adapters/ibyte_to_cbyte.h File Reference

Adapts an I/Q interleaved byte (unsigned char) sample stream into a std::complex<unsigned char> stream.

```
#include <string>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_block_interface.h"
#include "interleaved_byte_to_complex_byte.h"
```

Classes

- class IbyteToCbyte

9.31.1 Detailed Description

Adapts an I/Q interleaved byte (unsigned char) sample stream into a std::complex<unsigned char> stream.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.32 src/algorithms/data_type_adapter/adapters/ibyte_to_complex.h File Reference

Adapts an I/Q interleaved byte integer sample stream to a gr_complex (float) stream.

```
#include <string>
#include <gnuradio/blocks/interleaved_char_to_complex.h>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_synchro.h"
#include "gnss_block_interface.h"
```

Classes

- class IbyteToComplex

  Adapts an I/Q interleaved byte integer sample stream to a gr_complex (float) stream.

9.32.1 Detailed Description

Adapts an I/Q interleaved byte integer sample stream to a gr_complex (float) stream.

Author

Javier Arribas, jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.33 src/algorithms/data_type_adapter/adapters/ibyte_to_cshort.h File Reference

Adapts a short interleaved sample stream into a std::complex<short> stream.

```
#include <string>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_block_interface.h"
#include "interleaved_byte_to_complex_short.h"
```

Classes

- class IbyteToCshort

  Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream.

9.33.1 Detailed Description

Adapts a short interleaved sample stream into a std::complex<short> stream.

Author

Carles Fernandez-Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.34 src/algorithms/data_type_adapter/adapters/ishort_to_complex.h File Reference

Adapts an I/Q interleaved short integer sample stream to a gr_complex (float) stream.

```
#include <string>
#include <gnuradio/blocks/interleaved_short_to_complex.h>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_block_interface.h"
```

Classes

- class IshortToComplex

  Adapts an I/Q interleaved short integer sample stream to a gr_complex (float) stream.

9.34.1 Detailed Description

Adapts an I/Q interleaved short integer sample stream to a gr_complex (float) stream.

Author

Javier Arribas, jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.35 src/algorithms/data_type_adapter/adapters/ishort_to_cshort.h File Reference

Adapts a short interleaved sample stream into a std::complex<short> stream.

```
#include <string>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_block_interface.h"
#include "interleaved_short_to_complex_short.h"
```

Classes

- class IshortToCshort

  Adapts a short integer (16 bits) interleaved sample stream into a std::complex<short> stream.

9.35.1 Detailed Description

Adapts a short interleaved sample stream into a std::complex<short> stream.

Author

Carles Fernandez-Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.36 src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_byte.h File Reference

Adapts an 8-bits interleaved sample stream into a 16-bits complex stream.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_decimator.h>
```

Classes

- class interleaved_byte_to_complex_byte

  This class adapts an 8-bits interleaved sample stream into a 16-bits complex stream (std::complex<unsigned char>)

Typedefs

- typedef boost::shared_ptr< interleaved_byte_to_complex_byte > **interleaved_byte_to_complex_byte_sptr**

Functions

- interleaved_byte_to_complex_byte_sptr **make_interleaved_byte_to_complex_byte** ()

9.36.1 Detailed Description

Adapts an 8-bits interleaved sample stream into a 16-bits complex stream.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.37 src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_short.h File Reference

Adapts a byte (8-bits) interleaved sample stream into a std::complex<short> stream.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_decimator.h>
```

Classes

- class interleaved_byte_to_complex_short

  This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

Typedefs

- typedef boost::shared_ptr< interleaved_byte_to_complex_short > **interleaved_byte_to_complex_short_sptr**

Functions

- interleaved_byte_to_complex_short_sptr **make_interleaved_byte_to_complex_short** ()

9.37.1 Detailed Description

Adapts a byte (8-bits) interleaved sample stream into a std::complex<short> stream.

Author

Javier Arribas (jarribas(at)cttc.es)

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.38 src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_short_to_complex_short.h File Reference

Adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_decimator.h>
```

Classes

- class interleaved_short_to_complex_short

  This class adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

Typedefs

- typedef boost::shared_ptr< interleaved_short_to_complex_short > **interleaved_short_to_complex_short_sptr**

Functions

- interleaved_short_to_complex_short_sptr **make_interleaved_short_to_complex_short** ()

9.38.1 Detailed Description

Adapts a short (16-bits) interleaved sample stream into a std::complex<short> stream.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.39 src/algorithms/input_filter/adapters/beamformer_filter.h File Reference

Interface of an adapter of a digital beamformer.

```
#include <string>
#include <gnuradio/hier_block2.h>
#include "gnss_block_interface.h"
```

Classes

- class BeamformerFilter

  Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

9.39.1 Detailed Description

Interface of an adapter of a digital beamformer.

Author

Javier Arribas jarribas (at) cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.40 src/algorithms/input_filter/adapters/fir_filter.h File Reference

Adapts a gnuradio gr_fir_filter designed with pm_remez.

```
#include <cmath>
#include <string>
#include <vector>
#include <gnuradio/gr_complex.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/float_to_char.h>
#include <gnuradio/blocks/float_to_complex.h>
#include <gnuradio/blocks/float_to_short.h>
#include <gnuradio/filter/fir_filter_ccf.h>
#include <gnuradio/filter/fir_filter_fff.h>
#include "gnss_block_interface.h"
#include "complex_byte_to_float_x2.h"
#include "byte_x2_to_complex_byte.h"
#include "short_x2_to_cshort.h"
#include "cshort_to_float_x2.h"
```

Classes

- class FirFilter

  This class adapts a GNU Radio gr_fir_filter designed with pm_remez.

9.40.1 Detailed Description

Adapts a gnuradio gr_fir_filter designed with pm_remez.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Detailed description of the file here if needed.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.41 src/algorithms/input_filter/adapters/freq_xlating_fir_filter.h File Reference

Adapts a gnuradio gr_freq_xlating_fir_filter designed with gr_remez.

```
#include <string>
#include <vector>
#include <gnuradio/filter/freq_xlating_fir_filter_ccf.h>
#include <gnuradio/filter/freq_xlating_fir_filter_fcf.h>
#include <gnuradio/filter/freq_xlating_fir_filter_scf.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/complex_to_float.h>
#include <gnuradio/blocks/char_to_short.h>
#include <gnuradio/blocks/float_to_short.h>
#include "gnss_block_interface.h"
#include "short_x2_to_cshort.h"
#include "complex_float_to_complex_byte.h"
```

Classes

- class FreqXlatingFirFilter

  This class adapts a gnuradio gr_freq_xlating_fir_filter designed with pm_remez.

9.41.1 Detailed Description

Adapts a gnuradio gr_freq_xlating_fir_filter designed with gr_remez.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Detailed description of the file here if needed.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.42 src/algorithms/input_filter/gnuradio_blocks/beamformer.h File Reference

Simple spatial filter using RAW array input and beamforming coefficients.

```
#include <gnuradio/sync_block.h>
```

Classes

- class beamformer

  This class implements a real-time software-defined spatial filter using the CTTC GNSS experimental antenna array input and a set of dynamically reloadable weights.

Typedefs

- typedef boost::shared_ptr< beamformer > **beamformer_sptr**

Functions

- beamformer_sptr **make_beamformer** ()

9.42.1 Detailed Description

Simple spatial filter using RAW array input and beamforming coefficients.

Javier Arribas jarribas (at) cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.43 src/algorithms/libs/byte_x2_to_complex_byte.h File Reference

Adapts two signed char streams into a std::complex<signed char> stream.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_block.h>
```

Classes

- class byte_x2_to_complex_byte

  This class adapts two signed char streams into a std::complex<signed char> stream.

Typedefs

- typedef boost::shared_ptr< byte_x2_to_complex_byte > **byte_x2_to_complex_byte_sptr**

Functions

- byte_x2_to_complex_byte_sptr **make_byte_x2_to_complex_byte** ()

9.43.1 Detailed Description

Adapts two signed char streams into a std::complex<signed char> stream.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.44 src/algorithms/libs/complex_byte_to_float_x2.h File Reference

Adapts a std::complex<signed char> stream into two 16-bits (short) streams.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_block.h>
```

Classes

- class complex_byte_to_float_x2

  This class adapts a std::complex<signed char> stream into two 16-bits (short) streams.

Typedefs

- typedef boost::shared_ptr< complex_byte_to_float_x2 > **complex_byte_to_float_x2_sptr**

Functions

- complex_byte_to_float_x2_sptr **make_complex_byte_to_float_x2** ()

9.44.1 Detailed Description

Adapts a std::complex<signed char> stream into two 16-bits (short) streams.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.45 src/algorithms/libs/complex_float_to_complex_byte.h File Reference

Adapts a gr_complex stream into a std::complex<signed char> stream.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_block.h>
```

Classes

- class complex_float_to_complex_byte

  This class adapts a gr_complex stream into a std::complex<signed char> stream.

Typedefs

- typedef boost::shared_ptr< complex_float_to_complex_byte > **complex_float_to_complex_byte_sptr**

Functions

- complex_float_to_complex_byte_sptr **make_complex_float_to_complex_byte** ()

9.45.1 Detailed Description

Adapts a gr_complex stream into a std::complex<signed char> stream.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.46 src/algorithms/libs/cshort_to_float_x2.h File Reference

Adapts a std::complex<short> stream into two float streams.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_block.h>
```

Classes

- class cshort_to_float_x2

  This class adapts a std::complex<short> stream into two 32-bits (float) streams.

Typedefs

- typedef boost::shared_ptr< cshort_to_float_x2 > **cshort_to_float_x2_sptr**

Functions

- cshort_to_float_x2_sptr **make_cshort_to_float_x2** ()

9.46.1 Detailed Description

Adapts a std::complex<short> stream into two float streams.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.47 src/algorithms/libs/galileo_e1_signal_processing.h File Reference

This library implements various functions for Galileo E1 signals.

```
#include <complex>
```

Functions

- void galileo_e1_code_gen_int (int *_dest, char _Signal[3], signed int _prn)

  This function generates Galileo E1 code (one sample per chip).
- void galileo_e1_sinboc_11_gen (std::complex< float > *_dest, int *_prn, unsigned int _codeLength)

  *This function generates Galileo E1 sinboc(1,1) code (minimum 2 samples per chip), the _codeLength variable must be a multiple of 2*4092.*
- void galileo_e1_sinboc_61_gen (std::complex< float > *_dest, int *_prn, unsigned int _codeLength)

  *This function generates Galileo E1 sinboc(6,1) code (minimum 12 samples per chip), the _codeLength variable must be a multiple of 12*4092.*
- void galileo_e1_cboc_gen (std::complex< float > *_dest, int *_prn, char _Signal[3])

  This function generates Galileo E1 cboc code (12 samples per chip).
- void galileo_e1_code_gen_complex_sampled (std::complex< float > *_dest, char _Signal[3], bool _cboc, unsigned int _prn, signed int _fs, unsigned int _chip_shift, bool _secondary_flag)

  This function generates Galileo E1 code (can select E1B or E1C, cboc or sinboc and the sample frequency _fs).
- void galileo_e1_code_gen_complex_sampled (std::complex< float > *_dest, char _Signal[3], bool _cboc, unsigned int _prn, signed int _fs, unsigned int _chip_shift)

  galileo_e1_code_gen_complex_sampled without _secondary_flag for backward compatibility.

9.47.1 Detailed Description

This library implements various functions for Galileo E1 signals.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.47.2 Function Documentation

9.47.2.1 void galileo_e1_cboc_gen (std::complex< float > * _dest, int * _prn, char _Signal[3])

This function generates Galileo E1 cboc code (12 samples per chip).

9.47.2.2 void galileo_e1_code_gen_complex_sampled (std::complex< float > * _dest, char _Signal[3], bool _cboc, unsigned int _prn, signed int _fs, unsigned int _chip_shift, bool _secondary_flag)

This function generates Galileo E1 code (can select E1B or E1C, cboc or sinboc and the sample frequency _fs).

9.47.2.3 void galileo_e1_code_gen_complex_sampled (std::complex< float > * _dest, char _Signal[3], bool _cboc, unsigned int _prn, signed int _fs, unsigned int _chip_shift)

galileo_e1_code_gen_complex_sampled without _secondary_flag for backward compatibility.

9.47.2.4 void galileo_e1_code_gen_int (int * _dest, char _Signal[3], signed int _prn)

This function generates Galileo E1 code (one sample per chip).

9.47.2.5 void galileo_e1_sinboc_11_gen (std::complex< float > * _dest, int * _prn, unsigned int _codeLength)

This function generates Galileo E1 sinboc(1,1) code (minimum 2 samples per chip), the _codeLength variable must be a multiple of 2*4092.

9.47.2.6 void galileo_e1_sinboc_61_gen (std::complex< float > * *_dest,* int * *_prn,* unsigned int *_codeLength*)

This function generates Galileo E1 sinboc(6,1) code (minimum 12 samples per chip), the _codeLength variable must be a multiple of 12*4092.

9.48 src/algorithms/libs/gnss_sdr_valve.h File Reference

Interface of a GNU Radio block that sends a STOP message to the control queue right after a specific number of samples have passed through it.

```
#include <cstring>
#include <gnuradio/sync_block.h>
#include <gnuradio/msg_queue.h>
#include <boost/shared_ptr.hpp>
```

Classes

- class gnss_sdr_valve

  Implementation of a GNU Radio block that sends a STOP message to the control queue right after a specific number of samples have passed through it.

Functions

- boost::shared_ptr< gr::block > **gnss_sdr_make_valve** (size_t sizeof_stream_item, unsigned long long nitems, gr::msg_queue::sptr queue)

9.48.1 Detailed Description

Interface of a GNU Radio block that sends a STOP message to the control queue right after a specific number of samples have passed through it.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.49 src/algorithms/libs/gnss_signal_processing.h File Reference

This library gathers a few functions used by the algorithms of gnss-sdr, regardless of system used.

```
#include <complex>
```

Functions

- void complex_exp_gen (std::complex< float > *_dest, double _f, double _fs, unsigned int _samps)

  This function generates a complex exponential in _dest.
- void complex_exp_gen_conj (std::complex< float > *_dest, double _f, double _fs, unsigned int _samps)

  This function generates a conjugate complex exponential in _dest.
- void hex_to_binary_converter (int *_dest, char _from)

  This function makes a conversion from hex (the input is a char) to binary (the output are 4 ints with +1 or -1 values).
- void resampler (std::complex< float > *_from, std::complex< float > *_dest, float _fs_in, float _fs_out, unsigned int _length_in, unsigned int _length_out)

  This function resamples a sequence of complex values.

9.49.1 Detailed Description

This library gathers a few functions used by the algorithms of gnss-sdr, regardless of system used.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Detailed description of the file here if needed.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.49.2 Function Documentation

9.49.2.1 void complex_exp_gen (std::complex< float > * *_dest*, double *_f*, double *_fs*, unsigned int *_samps*)

This function generates a complex exponential in _dest.

9.49.2.2 void complex_exp_gen_conj (std::complex< float > * _dest, double _f, double _fs, unsigned int _samps)

This function generates a conjugate complex exponential in _dest.

9.49.2.3 void hex_to_binary_converter (int * _dest, char _from)

This function makes a conversion from hex (the input is a char) to binary (the output are 4 ints with +1 or -1 values).

9.49.2.4 void resampler (std::complex< float > * _from, std::complex< float > * _dest, float _fs_in, float _fs_out, unsigned int _length_in, unsigned int _length_out)

This function resamples a sequence of complex values.

9.50 src/algorithms/libs/gps_l2c_signal.h File Reference

This class implements signal generators for the GPS L2C signals.

`#include <complex>`

Functions

- void gps_l2c_m_code_gen_complex (std::complex< float > *_dest, unsigned int _prn)

  Generates complex GPS L2C M code for the desired SV ID.
- void gps_l2c_m_code_gen_complex_sampled (std::complex< float > *_dest, unsigned int _prn, signed int _fs)

  Generates complex GPS L2C M code for the desired SV ID, and sampled to specific sampling frequency.

9.50.1 Detailed Description

This class implements signal generators for the GPS L2C signals.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Detailed description of the file here if needed.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.50.2 Function Documentation

9.50.2.1 void gps_l2c_m_code_gen_complex (std::complex< float > * _dest, unsigned int _prn)

Generates complex GPS L2C M code for the desired SV ID.

9.50.2.2 void gps_l2c_m_code_gen_complex_sampled (std::complex< float > * _dest, unsigned int _prn, signed int _fs)

Generates complex GPS L2C M code for the desired SV ID, and sampled to specific sampling frequency.

9.51 src/algorithms/libs/gps_sdr_signal_processing.h File Reference

This class implements various functions for GPS L1 CA signals.

```
#include <complex>
```

Functions

- void gps_l1_ca_code_gen_complex (std::complex< float > *_dest, signed int _prn, unsigned int _chip_shift)

  Generates complex GPS L1 C/A code for the desired SV ID and code shift, and sampled to specific sampling frequency.
- void gps_l1_ca_code_gen_complex_sampled (std::complex< float > *_dest, unsigned int _prn, signed int _fs, unsigned int _chip_shift, unsigned int _ncodes)

  Generates N complex GPS L1 C/A codes for the desired SV ID and code shift.
- void gps_l1_ca_code_gen_complex_sampled (std::complex< float > *_dest, unsigned int _prn, signed int _fs, unsigned int _chip_shift)

  Generates complex GPS L1 C/A code for the desired SV ID and code shift.

9.51.1 Detailed Description

This class implements various functions for GPS L1 CA signals.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Detailed description of the file here if needed.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.51.2 Function Documentation

9.51.2.1 void gps_l1_ca_code_gen_complex (std::complex< float > * _dest, signed int _prn, unsigned int _chip_shift)

Generates complex GPS L1 C/A code for the desired SV ID and code shift, and sampled to specific sampling frequency.

9.51.2.2 void gps_l1_ca_code_gen_complex_sampled (std::complex< float > * _dest, unsigned int _prn, signed int _fs, unsigned int _chip_shift, unsigned int _ncodes)

Generates N complex GPS L1 C/A codes for the desired SV ID and code shift.

9.51.2.3 void gps_l1_ca_code_gen_complex_sampled (std::complex< float > * _dest, unsigned int _prn, signed int _fs, unsigned int _chip_shift)

Generates complex GPS L1 C/A code for the desired SV ID and code shift.

9.52 src/algorithms/libs/pass_through.h File Reference

Interface of a block that just puts its input in its output.

```
#include <string>
#include <gnuradio/hier_block2.h>
#include <gnuradio/blocks/copy.h>
#include "gnss_block_interface.h"
```

Classes

- class Pass_Through

  This class implements a block that connects input and output (does nothing)

9.52.1 Detailed Description

Interface of a block that just puts its input in its output.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.53 src/algorithms/libs/short_x2_to_cshort.h File Reference

Adapts two short streams into a std::complex<short> stream.

```
#include <boost/shared_ptr.hpp>
#include <gnuradio/sync_block.h>
```

Classes

- class short_x2_to_cshort

  This class adapts two short streams into a std::complex<short> stream.

Typedefs

- typedef boost::shared_ptr< short_x2_to_cshort > **short_x2_to_cshort_sptr**

Functions

- short_x2_to_cshort_sptr **make_short_x2_to_cshort** ()

9.53.1 Detailed Description

Adapts two short streams into a std::complex<short> stream.

Author

Carles Fernandez Prades, cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.54 src/algorithms/observables/adapters/galileo_e1_observables.h File Reference

Implementation of an adapter of a Galileo E1 observables block to a ObservablesInterface.

```
#include <string>
#include "observables_interface.h"
#include "galileo_e1_observables_cc.h"
```

Classes

- class GalileoE1Observables

  This class implements an ObservablesInterface for Galileo E1B.

9.54.1 Detailed Description

Implementation of an adapter of a Galileo E1 observables block to a ObservablesInterface.

Author

Mara Branzanti 2013. mara.branzanti(at)gmail.com
Javier Arribas 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.55 src/algorithms/observables/adapters/galileo_e5a_observables.h File Reference

Implementation of an adapter of a Galileo E5a observables block to a ObservablesInterface.

```
#include <string>
#include "observables_interface.h"
#include "hybrid_observables_cc.h"
```

Classes

- class GalileoE5aObservables

  This class implements an ObservablesInterface for Galileo E5A.

9.55.1 Detailed Description

Implementation of an adapter of a Galileo E5a observables block to a ObservablesInterface.

Author

Carles Fernandez 2016. carles.fernandez(at)cttc.es

Copyright (C) 2010-2016 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.56 src/algorithms/observables/adapters/gps_l1_ca_observables.h File Reference

Interface of an adapter of a GPS L1 C/A observables block to a ObservablesInterface.

```
#include <string>
#include "observables_interface.h"
#include "gps_l1_ca_observables_cc.h"
```

Classes

- class GpsL1CaObservables

  This class implements an ObservablesInterface for GPS L1 C/A.

9.56.1 Detailed Description

Interface of an adapter of a GPS L1 C/A observables block to a ObservablesInterface.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.57 src/algorithms/observables/adapters/gps_l2c_observables.h File Reference

Implementation of an adapter of a GPS L2C(M) observables block to a ObservablesInterface.

```
#include <string>
#include "observables_interface.h"
#include "hybrid_observables_cc.h"
```

Classes

- class GpsL2CObservables

  This class implements an ObservablesInterface for GPS L2C.

9.57.1 Detailed Description

Implementation of an adapter of a GPS L2C(M) observables block to a ObservablesInterface.

Author

Carles Fernandez 2016. carles.fernandez(at)cttc.es

Copyright (C) 2010-2016 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.58 src/algorithms/observables/adapters/hybrid_observables.h File Reference

Implementation of an adapter of an observables block accepting all kind of signals to a ObservablesInterface.

```
#include <string>
#include "observables_interface.h"
#include "hybrid_observables_cc.h"
```

Classes

- class HybridObservables

  This class implements an ObservablesInterface for observables of all kind of GNSS signals.

9.58.1 Detailed Description

Implementation of an adapter of an observables block accepting all kind of signals to a ObservablesInterface.

Author

Mara Branzanti 2013. mara.branzanti(at)gmail.com
Javier Arribas 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.59 src/algorithms/observables/gnuradio_blocks/galileo_e1_observables_cc.h File Reference

Interface of the observables computation block for Galileo E1.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
```

Classes

- class galileo_e1_observables_cc

  This class implements a block that computes Galileo observables.

Typedefs

- typedef boost::shared_ptr< galileo_e1_observables_cc > **galileo_e1_observables_cc_sptr**

Functions

- galileo_e1_observables_cc_sptr **galileo_e1_make_observables_cc** (unsigned int n_channels, bool dump, std::string dump_filename, unsigned int deep_history)

9.59.1 Detailed Description

Interface of the observables computation block for Galileo E1.

Author

Mara Branzanti 2013. mara.branzanti(at)gmail.com
Javier Arribas 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.60 src/algorithms/observables/gnuradio_blocks/gps_l1_ca_observables_cc.h File Reference

Interface of the pseudorange computation block for GPS L1 C/A.

```
#include <deque>
#include <fstream>
#include <string>
#include <vector>
#include <boost/shared_ptr.hpp>
#include <gnuradio/block.h>
```

Classes

- class gps_l1_ca_observables_cc

  This class implements a block that computes GPS L1 C/A observables.

Typedefs

- typedef boost::shared_ptr< gps_l1_ca_observables_cc > **gps_l1_ca_observables_cc_sptr**

Functions

- gps_l1_ca_observables_cc_sptr **gps_l1_ca_make_observables_cc** (unsigned int n_channels, bool dump, std::string dump_filename, unsigned int deep_history)

9.60.1 Detailed Description

Interface of the pseudorange computation block for GPS L1 C/A.

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.61 src/algorithms/observables/gnuradio_blocks/hybrid_observables_cc.h File Reference

Interface of the observables computation block for Galileo E1.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
```

Classes

- class hybrid_observables_cc

  This class implements a block that computes Galileo observables.

Typedefs

- typedef boost::shared_ptr< hybrid_observables_cc > **hybrid_observables_cc_sptr**

Functions

- hybrid_observables_cc_sptr **hybrid_make_observables_cc** (unsigned int n_channels, bool dump, std::string dump_filename, unsigned int deep_history)

9.61.1 Detailed Description

Interface of the observables computation block for Galileo E1.

Author

Mara Branzanti 2013. mara.branzanti(at)gmail.com
Javier Arribas 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.62 src/algorithms/PVT/adapters/galileo_e1_pvt.h File Reference

Interface of an adapter of a GALILEO E1 PVT solver block to a PvtInterface.

```
#include <string>
#include "pvt_interface.h"
#include "galileo_e1_pvt_cc.h"
```

Classes

- class GalileoE1Pvt

  This class implements a PvtInterface for Galileo E1.

9.62.1 Detailed Description

Interface of an adapter of a GALILEO E1 PVT solver block to a PvtInterface.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.63 src/algorithms/PVT/adapters/gps_l1_ca_pvt.h File Reference

Interface of an adapter of a GPS L1 C/A PVT solver block to a PvtInterface Position Velocity and Time.

```
#include <string>
#include "pvt_interface.h"
#include "gps_l1_ca_pvt_cc.h"
```

Classes

- class GpsL1CaPvt

  This class implements a PvtInterface for GPS L1 C/A.

9.63.1 Detailed Description

Interface of an adapter of a GPS L1 C/A PVT solver block to a PvtInterface Position Velocity and Time.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.64 src/algorithms/PVT/adapters/hybrid_pvt.h File Reference

Interface of an adapter of a GALILEO E1 PVT solver block to a PvtInterface.

```
#include <string>
#include "pvt_interface.h"
#include "hybrid_pvt_cc.h"
```

Classes

- class HybridPvt

  This class implements a PvtInterface for Galileo E1.

9.64.1 Detailed Description

Interface of an adapter of a GALILEO E1 PVT solver block to a PvtInterface.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.65 src/algorithms/PVT/gnuradio_blocks/galileo_e1_pvt_cc.h File Reference

Interface of a Position Velocity and Time computation block for Galileo E1.

```
#include <fstream>
#include <string>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <utility>
#include <gnuradio/block.h>
#include "nmea_printer.h"
#include "kml_printer.h"
#include "rinex_printer.h"
#include "geojson_printer.h"
#include "rtcm_printer.h"
#include "galileo_e1_ls_pvt.h"
```

Classes

- class galileo_e1_pvt_cc

  This class implements a block that computes the PVT solution with Galileo E1 signals.

Typedefs

- typedef boost::shared_ptr< galileo_e1_pvt_cc > **galileo_e1_pvt_cc_sptr**

Functions

- galileo_e1_pvt_cc_sptr **galileo_e1_make_pvt_cc** (unsigned int n_channels, bool dump, std::string dump↩ _filename, int averaging_depth, bool flag_averaging, int output_rate_ms, int display_rate_ms, bool flag_↩ nmea_tty_port, std::string nmea_dump_filename, std::string nmea_dump_devname, bool flag_rtcm_server, bool flag_rtcm_tty_port, unsigned short rtcm_tcp_port, unsigned short rtcm_station_id, std::map< int, int > rtcm_msg_rate_ms, std::string rtcm_dump_devname)

9.65.1 Detailed Description

Interface of a Position Velocity and Time computation block for Galileo E1.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.66 src/algorithms/PVT/gnuradio_blocks/gps_l1_ca_pvt_cc.h File Reference

Interface of a Position Velocity and Time computation block for GPS L1 C/A.

```
#include <fstream>
#include <string>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <gnuradio/block.h>
#include "nmea_printer.h"
#include "kml_printer.h"
#include "rinex_printer.h"
#include "geojson_printer.h"
#include "rtcm_printer.h"
#include "gps_l1_ca_ls_pvt.h"
```

Classes

- class gps_l1_ca_pvt_cc

  This class implements a block that computes the PVT solution.

Typedefs

- typedef boost::shared_ptr< gps_l1_ca_pvt_cc > **gps_l1_ca_pvt_cc_sptr**

Functions

- gps_l1_ca_pvt_cc_sptr **gps_l1_ca_make_pvt_cc** (unsigned int n_channels, bool dump, std::string dump↩ _filename, int averaging_depth, bool flag_averaging, int output_rate_ms, int display_rate_ms, bool flag_↩ nmea_tty_port, std::string nmea_dump_filename, std::string nmea_dump_devname, bool flag_rtcm_server, bool flag_rtcm_tty_port, unsigned short rtcm_tcp_port, unsigned short rtcm_station_id, std::map< int, int > rtcm_msg_rate_ms, std::string rtcm_dump_devname, int rinex_version)

9.66.1 Detailed Description

Interface of a Position Velocity and Time computation block for GPS L1 C/A.

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.67 src/algorithms/PVT/gnuradio_blocks/hybrid_pvt_cc.h File Reference

Interface of a Position Velocity and Time computation block for Galileo E1.

```
#include <fstream>
#include <utility>
#include <string>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <gnuradio/block.h>
#include "nmea_printer.h"
#include "kml_printer.h"
#include "geojson_printer.h"
#include "rinex_printer.h"
#include "rtcm_printer.h"
#include "hybrid_ls_pvt.h"
```

Classes

- class hybrid_pvt_cc

  This class implements a block that computes the PVT solution with Galileo E1 signals.

Typedefs

- typedef boost::shared_ptr< hybrid_pvt_cc > **hybrid_pvt_cc_sptr**

Functions

- hybrid_pvt_cc_sptr **hybrid_make_pvt_cc** (unsigned int n_channels, bool dump, std::string dump_filename, int averaging_depth, bool flag_averaging, int output_rate_ms, int display_rate_ms, bool flag_nmea_tty_port, std::string nmea_dump_filename, std::string nmea_dump_devname, bool flag_rtcm_server, bool flag_rtcm↩_tty_port, unsigned short rtcm_tcp_port, unsigned short rtcm_station_id, std::map< int, int > rtcm_msg_↩rate_ms, std::string rtcm_dump_devname, const unsigned int type_of_receiver)

9.67.1 Detailed Description

Interface of a Position Velocity and Time computation block for Galileo E1.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩://www.gnu.org/licenses/.

9.68 src/algorithms/PVT/libs/galileo_e1_ls_pvt.h File Reference

Interface of a Least Squares Position, Velocity, and Time (PVT) solver, based on K.Borre's Matlab receiver.

```
#include <fstream>
#include <iostream>
#include <map>
#include <string>
#include "ls_pvt.h"
#include "galileo_navigation_message.h"
#include "gnss_synchro.h"
#include "galileo_ephemeris.h"
#include "galileo_utc_model.h"
```

Classes

- class galileo_e1_ls_pvt

  This class implements a simple PVT Least Squares solution.

9.68.1 Detailed Description

Interface of a Least Squares Position, Velocity, and Time (PVT) solver, based on K.Borre's Matlab receiver.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.69 src/algorithms/PVT/libs/geojson_printer.h File Reference

Interface of a class that prints PVT solutions in GeoJSON format.

```
#include <fstream>
#include <memory>
#include <string>
#include "pvt_solution.h"
```

Classes

- class GeoJSON_Printer

  Prints PVT solutions in GeoJSON format file.

9.69.1 Detailed Description

Interface of a class that prints PVT solutions in GeoJSON format.

Author

Carles Fernandez-Prades, 2015. cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.70 src/algorithms/PVT/libs/gps_l1_ca_ls_pvt.h File Reference

Interface of a Least Squares Position, Velocity, and Time (PVT) solver for GPS L1 C/A, based on K.Borre's Matlab receiver.

```
#include <fstream>
#include <map>
#include <string>
#include "ls_pvt.h"
#include "GPS_L1_CA.h"
#include "gnss_synchro.h"
#include "gps_ephemeris.h"
#include "gps_navigation_message.h"
#include "gps_utc_model.h"
#include "sbas_telemetry_data.h"
#include "sbas_ionospheric_correction.h"
#include "sbas_satellite_correction.h"
#include "sbas_ephemeris.h"
```

Classes

- class gps_l1_ca_ls_pvt

  This class implements a simple PVT Least Squares solution for GPS L1 C/A signals.

9.70.1 Detailed Description

Interface of a Least Squares Position, Velocity, and Time (PVT) solver for GPS L1 C/A, based on K.Borre's Matlab receiver.

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.71 src/algorithms/PVT/libs/kml_printer.h File Reference

Interface of a class that prints PVT information to a kml file.

```
#include <iostream>
#include <fstream>
#include <memory>
#include <string>
#include "pvt_solution.h"
```

Classes

- class Kml_Printer

  Prints PVT information to OGC KML format file (can be viewed with Google Earth)

9.71.1 Detailed Description

Interface of a class that prints PVT information to a kml file.

Interface of a class that prints PVT information to a kml file for GPSTK data structures.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

Author

Javier Arribas, 2012. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.72 src/algorithms/PVT/libs/ls_pvt.h File Reference

Interface of a base class for Least Squares PVT solutions.

```
#include "pvt_solution.h"
```

Classes

- class Ls_Pvt

  Base class for the Least Squares PVT solution.

9.72.1 Detailed Description

Interface of a base class for Least Squares PVT solutions.

Author

Carles Fernandez-Prades, 2015. cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.73 src/algorithms/PVT/libs/nmea_printer.h File Reference

Interface of a NMEA 2.1 printer for GNSS-SDR This class provides a implementation of a subset of the NMEA-0183 standard for interfacing marine electronic devices as defined by the National Marine Electronics Association (NMEA). See http://www.nmea.org/ for the NMEA 183 standard.

```
#include <iostream>
#include <fstream>
#include <string>
#include "pvt_solution.h"
```

Classes

- class Nmea_Printer

  This class provides a implementation of a subset of the NMEA-0183 standard for interfacing marine electronic devices as defined by the National Marine Electronics Association (NMEA).

9.73.1 Detailed Description

Interface of a NMEA 2.1 printer for GNSS-SDR This class provides a implementation of a subset of the NMEA-0183 standard for interfacing marine electronic devices as defined by the National Marine Electronics Association (NMEA). See http://www.nmea.org/ for the NMEA 183 standard.

Author

Javier Arribas, 2012. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.74 src/algorithms/PVT/libs/pvt_solution.h File Reference

Interface of a base class for a PVT solution.

```
#include <deque>
#include <armadillo>
#include <boost/date_time/posix_time/posix_time.hpp>
```

Classes

- class Pvt_Solution

  Base class for a PVT solution.

Macros

- #define **PVT_MAX_CHANNELS** 24

9.74.1 Detailed Description

Interface of a base class for a PVT solution.

Author

Carles Fernandez-Prades, 2015. cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.75 src/algorithms/PVT/libs/rinex_printer.h File Reference

Interface of a RINEX 2.11 / 3.01 printer See http://igscb.jpl.nasa.gov/igscb/data/format/rinex301.pdf.

```
#include <string>
#include <fstream>
#include <iostream>
#include <sstream>
#include <iomanip>
#include <map>
#include <boost/date_time/posix_time/posix_time.hpp>
#include "gps_navigation_message.h"
#include "gps_cnav_navigation_message.h"
#include "galileo_navigation_message.h"
#include "sbas_telemetry_data.h"
#include "GPS_L1_CA.h"
#include "Galileo_E1.h"
#include "gnss_synchro.h"
```

Classes

- class Rinex_Printer

  Class that handles the generation of Receiver INdependent EXchange format (RINEX) files.

Functions

- std::string **asString** (const long double x, const std::string::size_type precision)
- long **asInt** (const std::string &s)

9.75.1 Detailed Description

Interface of a RINEX 2.11 / 3.01 printer See http://igscb.jpl.nasa.gov/igscb/data/format/rinex301.pdf.

Receiver Independent EXchange Format (RINEX): The first proposal for the Receiver Independent Exchange Format RINEX was developed by the Astronomical Institute of the University of Berne for the easy exchange of the GPS data to be collected during the large European GPS campaign EUREF 89, which involved more than 60 GPS receivers of 4 different manufacturers. The governing aspect during the development was the fact that most geodetic processing software for GPS data use a well-defined set of observables: 1) The carrier-phase measurement at one or both carriers (actually being a measurement on the beat frequency between the received carrier of the satellite signal and a receiver-generated reference frequency). 2) The pseudorange (code) measurement , equivalent to the difference of the time of reception (expressed in the time frame of the receiver) and the time of transmission (expressed in the time frame of the satellite) of a distinct satellite signal. 3) The observation time being the reading of the receiver clock at the instant of validity of the carrier-phase and/or the code measurements. Note: A collection of the formats currently used by the IGS can be found here: http://igscb.jpl.nasa.gov/components/formats.html

Carles Fernandez Prades, 2011. cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.76 src/algorithms/PVT/libs/rtcm_printer.h File Reference

Interface of a RTCM 3.2 printer for GNSS-SDR This class provides a implementation of a subset of the RTCM Standard 10403.2 for Differential GNSS Services.

```
#include <fstream>
#include <iostream>
#include <memory>
#include "rtcm.h"
```

Classes

- class Rtcm_Printer

  This class provides a implementation of a subset of the RTCM Standard 10403.2 messages.

9.76.1 Detailed Description

Interface of a RTCM 3.2 printer for GNSS-SDR This class provides a implementation of a subset of the RTCM Standard 10403.2 for Differential GNSS Services.

Author

Carles Fernandez-Prades, 2014. cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.77 src/algorithms/resampler/adapters/direct_resampler_conditioner.h File Reference

Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

```
#include <string>
#include <gnuradio/hier_block2.h>
#include "gnss_block_interface.h"
```

Classes

- class DirectResamplerConditioner

  Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

9.77.1 Detailed Description

Interface of an adapter of a direct resampler conditioner block to a SignalConditionerInterface.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.78 src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cb.h File Reference

Nearest neighborhood resampler with std::complex<signed char> input and std::complex<signed char> output.

```
#include <gnuradio/block.h>
#include <volk/volk.h>
```

Classes

- class direct_resampler_conditioner_cb

  This class implements a direct resampler conditioner for std::complex<signed char>

Typedefs

- typedef boost::shared_ptr< direct_resampler_conditioner_cb > **direct_resampler_conditioner_cb_sptr**

Functions

- direct_resampler_conditioner_cb_sptr **direct_resampler_make_conditioner_cb** (double sample_freq_in, double sample_freq_out)

9.78.1 Detailed Description

Nearest neighborhood resampler with std::complex<signed char> input and std::complex<signed char> output.

Author

Luis Esteve, 2011. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.79 src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cc.h File Reference

Nearest neighborhood resampler with gr_complex input and gr_complex output.

```
#include <gnuradio/block.h>
#include <volk/volk.h>
```

Classes

- class direct_resampler_conditioner_cc

  This class implements a direct resampler conditioner for complex data.

Typedefs

- typedef boost::shared_ptr< direct_resampler_conditioner_cc > **direct_resampler_conditioner_cc_sptr**

Functions

- direct_resampler_conditioner_cc_sptr **direct_resampler_make_conditioner_cc** (double sample_freq_in, double sample_freq_out)

9.79.1 Detailed Description

Nearest neighborhood resampler with gr_complex input and gr_complex output.

Author

Luis Esteve, 2011. luis(at)epsilon-formacion.com

This block takes in a signal stream and performs direct resampling. The theory behind this block can be found in Chapter 7.5 of the following book.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.80 src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cs.h File Reference

Nearest neighborhood resampler with std::complex<short> input and std::complex<short> output.

```
#include <gnuradio/block.h>
#include <volk/volk.h>
```

Classes

- class direct_resampler_conditioner_cs

  This class implements a direct resampler conditioner for std::complex<short>

Typedefs

- typedef boost::shared_ptr< direct_resampler_conditioner_cs > **direct_resampler_conditioner_cs_sptr**

Functions

- direct_resampler_conditioner_cs_sptr **direct_resampler_make_conditioner_cs** (double sample_freq_in, double sample_freq_out)

9.80.1 Detailed Description

Nearest neighborhood resampler with std::complex<short> input and std::complex<short> output.

Author

Luis Esteve, 2011. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.81 src/algorithms/signal_generator/adapters/signal_generator.h File Reference

Adapter of a class that generates synthesized GNSS signal.

```
#include <string>
#include <vector>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include <gnuradio/blocks/vector_to_stream.h>
#include "gnss_block_interface.h"
#include "signal_generator_c.h"
```

Classes

- class SignalGenerator

  This class generates synthesized GNSS signal.

9.81.1 Detailed Description

Adapter of a class that generates synthesized GNSS signal.

Author

Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.82 src/algorithms/signal_generator/gnuradio_blocks/signal_generator_c.h File Reference

GNU Radio source block that generates synthesized GNSS signal.

```
#include <string>
#include <vector>
#include <boost/scoped_array.hpp>
#include <gnuradio/random.h>
#include <gnuradio/block.h>
#include "gnss_signal.h"
```

Classes

- class signal_generator_c

  This class generates synthesized GNSS signal.

Typedefs

- typedef boost::shared_ptr< signal_generator_c > **signal_generator_c_sptr**

Functions

- signal_generator_c_sptr signal_make_generator_c (std::vector< std::string > signal1, std::vector< std::string > system, const std::vector< unsigned int > &PRN, const std::vector< float > &CN0_dB, const std::vector< float > &doppler_Hz, const std::vector< unsigned int > &delay_chips, const std::vector< unsigned int > &delay_sec, bool data_flag, bool noise_flag, unsigned int fs_in, unsigned int vector_length, float BW_BB)

  Return a shared_ptr to a new instance of gen_source.

9.82.1 Detailed Description

GNU Radio source block that generates synthesized GNSS signal.

Author

Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.82.2 Function Documentation

9.82.2.1 signal_generator_c_sptr signal_make_generator_c (std::vector< std::string > *signal1*, std::vector< std::string > *system*, const std::vector< unsigned int > & *PRN*, const std::vector< float > & *CN0_dB*, const std::vector< float > & *doppler_Hz*, const std::vector< unsigned int > & *delay_chips*, const std::vector< unsigned int > & *delay_sec*, bool *data_flag*, bool *noise_flag*, unsigned int *fs_in*, unsigned int *vector_length*, float *BW_BB*)

Return a shared_ptr to a new instance of gen_source.

To avoid accidental use of raw pointers, gen_source's constructor is private. signal_make_generator_c is the public interface for creating new instances.

9.83 src/algorithms/signal_source/adapters/file_signal_source.h File Reference

Interface of a class that reads signals samples from a file and adapts it to a SignalSourceInterface.

```
#include <string>
#include <gnuradio/blocks/file_source.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/throttle.h>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include "gnss_block_interface.h"
```

Classes

- class FileSignalSource

  Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

9.83.1 Detailed Description

Interface of a class that reads signals samples from a file and adapts it to a SignalSourceInterface.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

This class represents a file signal source. Internally it uses a GNU Radio's gr_file_source as a connector to the data.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.84 src/algorithms/signal_source/adapters/gen_signal_source.h File Reference

It wraps blocks that generates synthesized GNSS signal and filters it.

```
#include <string>
#include <gnuradio/msg_queue.h>
#include "gnss_block_interface.h"
```

Classes

- class GenSignalSource

  This class wraps blocks that generates synthesized GNSS signal and filters the signal.

9.84.1 Detailed Description

It wraps blocks that generates synthesized GNSS signal and filters it.

Author

Marc Molina, 2013. marc.molina.pena@gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.85 src/algorithms/signal_source/adapters/gn3s_signal_source.h File Reference

GN3S USB dongle GPS RF front-end signal sampler driver.

```
#include <string>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_block_interface.h"
```

Classes

- class Gn3sSignalSource

  This class reads samples from a GN3S USB dongle, a RF front-end signal sampler.

9.85.1 Detailed Description

GN3S USB dongle GPS RF front-end signal sampler driver.

Author

Javier Arribas, jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.86 src/algorithms/signal_source/adapters/nsr_file_signal_source.h File Reference

Implementation of a class that reads signals samples from a NSR 2 bits sampler front-end file and adapts it to a SignalSourceInterface. More information about the front-end here http://www.ifen.com/products/sx-scientific-gnss-solutions/nsr-software-receiver.html.

```
#include <string>
#include <gnuradio/blocks/file_source.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/throttle.h>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include "gnss_block_interface.h"
#include "unpack_byte_2bit_samples.h"
```

Classes

- class NsrFileSignalSource

  Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

9.86.1 Detailed Description

Implementation of a class that reads signals samples from a NSR 2 bits sampler front-end file and adapts it to a SignalSourceInterface. More information about the front-end here http://www.ifen.↩com/products/sx-scientific-gnss-solutions/nsr-software-receiver.html.

Author

Javier Arribas, 2013 jarribas(at)cttc.es

This class represents a file signal source.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩://www.gnu.org/licenses/.

9.87 src/algorithms/signal_source/adapters/osmosdr_signal_source.h File Reference

Signal source wrapper for OsmoSDR-compatible front-ends, such as HackRF or Realtek's RTL2832U-based USB dongle DVB-T receivers (see http://sdr.osmocom.org/trac/wiki/rtl-sdr for more information)

```
#include <string>
#include <boost/shared_ptr.hpp>
#include <gnuradio/msg_queue.h>
#include <gnuradio/blocks/file_sink.h>
#include <osmosdr/source.h>
#include "gnss_block_interface.h"
```

Classes

- class OsmosdrSignalSource

  This class reads samples OsmoSDR-compatible front-ends, such as HackRF or Realtek's RTL2832U-based USB dongle DVB-T receivers (see http://sdr.osmocom.org/trac/wiki/rtl-sdr)

9.87.1 Detailed Description

Signal source wrapper for OsmoSDR-compatible front-ends, such as HackRF or Realtek's RTL2832U-based USB dongle DVB-T receivers (see http://sdr.osmocom.org/trac/wiki/rtl-sdr for more information)

Author

Javier Arribas, 2012. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.88 src/algorithms/signal_source/adapters/raw_array_signal_source.h File Reference

Signal Source adapter for the Teleorbit Flexiband front-end device. This adapter requires a Flexiband GNURadio driver installed (not included with GNSS-SDR)

```
#include <string>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include <gnuradio/blocks/file_sink.h>
#include "gnss_block_interface.h"
```

Classes

- class RawArraySignalSource

  This class reads samples from a GN3S USB dongle, a RF front-end signal sampler.

9.88.1 Detailed Description

Signal Source adapter for the Teleorbit Flexiband front-end device. This adapter requires a Flexiband GNURadio driver installed (not included with GNSS-SDR)

CTTC Experimental GNSS 8 channels array signal source.

Author

Javier Arribas, jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.89 src/algorithms/signal_source/adapters/rtl_tcp_signal_source.h File Reference

Signal source which reads from rtl_tcp. (see http://sdr.osmocom.org/trac/wiki/rtl-sdr for more information)

```
#include <string>
#include <boost/shared_ptr.hpp>
#include <gnuradio/msg_queue.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/deinterleave.h>
#include <gnuradio/blocks/float_to_complex.h>
#include "rtl_tcp_signal_source_c.h"
#include "gnss_block_interface.h"
```

Classes

- class RtlTcpSignalSource

  This class reads from rtl_tcp, which streams interleaved I/Q samples over TCP. (see http://sdr.osmocom.org/trac/wiki/rtl-sdr)

9.89.1 Detailed Description

Signal source which reads from rtl_tcp. (see http://sdr.osmocom.org/trac/wiki/rtl-sdr for more information)

Author

Anthony Arnold, 2015. anthony.arnold(at)uqconnect.edu.au

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.90 src/algorithms/signal_source/adapters/spir_file_signal_source.h File Reference

Implementation of a class that reads signals samples from a SPIR file and adapts it to a SignalSourceInterface.

```
#include <string>
#include <gnuradio/blocks/file_source.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/throttle.h>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include "gnss_block_interface.h"
#include "unpack_intspir_1bit_samples.h"
```

Classes

- class SpirFileSignalSource

  Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

9.90.1 Detailed Description

Implementation of a class that reads signals samples from a SPIR file and adapts it to a SignalSourceInterface.

Author

Fran Fabra, 2014 fabra(at)ice.csic.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is not part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.91 src/algorithms/signal_source/adapters/two_bit_cpx_file_signal_source.h File Reference

Interface of a class that reads signals samples from a 2 bit complex sampler front-end file and adapts it to a SignalSourceInterface.

```
#include <string>
#include <gnuradio/blocks/file_source.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/throttle.h>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include <gnuradio/blocks/interleaved_short_to_complex.h>
#include "gnss_block_interface.h"
#include "unpack_byte_2bit_cpx_samples.h"
```

Classes

- class TwoBitCpxFileSignalSource

  Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

9.91.1 Detailed Description

Interface of a class that reads signals samples from a 2 bit complex sampler front-end file and adapts it to a SignalSourceInterface.

Author

Javier Arribas, 2015 jarribas(at)cttc.es

This class represents a file signal source.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.92 src/algorithms/signal_source/adapters/two_bit_packed_file_signal_source.h File Reference

Interface of a class that reads signals samples from a file. Each sample is two bits, which are packed into bytes or shorts.

```
#include <string>
#include <gnuradio/blocks/file_source.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/blocks/throttle.h>
#include <gnuradio/hier_block2.h>
#include <gnuradio/msg_queue.h>
#include <gnuradio/blocks/interleaved_char_to_complex.h>
#include "gnss_block_interface.h"
#include "unpack_2bit_samples.h"
```

Classes

- class TwoBitPackedFileSignalSource

  Class that reads signals samples from a file and adapts it to a SignalSourceInterface.

9.92.1 Detailed Description

Interface of a class that reads signals samples from a file. Each sample is two bits, which are packed into bytes or shorts.

Author

Cillian O'Driscoll, 2015 cillian.odriscoll (at) gmail.com

This class represents a file signal source.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.93 src/algorithms/signal_source/adapters/uhd_signal_source.h File Reference

Interface for the Universal Hardware Driver signal source.

```
#include <string>
#include <vector>
#include <boost/shared_ptr.hpp>
#include <gnuradio/hier_block2.h>
#include <gnuradio/uhd/usrp_source.h>
#include <gnuradio/blocks/file_sink.h>
#include <gnuradio/msg_queue.h>
#include "gnss_block_interface.h"
```

Classes

- class UhdSignalSource

  This class reads samples from a UHD device (see http://code.ettus.com/redmine/ettus/projects/uhd/wiki)

9.93.1 Detailed Description

Interface for the Universal Hardware Driver signal source.

Author

Javier Arribas, 2012. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.94 src/algorithms/signal_source/gnuradio_blocks/rtl_tcp_signal_source_c.h File Reference

Interface of an rtl_tcp signal source reader.

```
#include "rtl_tcp_dongle_info.h"
#include <boost/asio.hpp>
#include <gnuradio/sync_block.h>
#include <string>
#include <boost/thread/mutex.hpp>
#include <boost/thread/condition.hpp>
#include <boost/array.hpp>
#include <boost/circular_buffer.hpp>
```

Classes

- class rtl_tcp_signal_source_c

  This class reads interleaved I/Q samples from an rtl_tcp server and outputs complex types.

Typedefs

- typedef boost::shared_ptr< rtl_tcp_signal_source_c > **rtl_tcp_signal_source_c_sptr**

Functions

- rtl_tcp_signal_source_c_sptr **rtl_tcp_make_signal_source_c** (const std::string &address, short port, bool flip_iq=false)

9.94.1 Detailed Description

Interface of an rtl_tcp signal source reader.

Author

Anthony Arnold, 2015. anthony.arnold(at)uqconnect.edu.au

The implementation of this block is a combination of various helpful sources. The data format and command structure is taken from the original Osmocom rtl_tcp_source_f (http://git.osmocom.org/gr-osmosdr). The aynchronous reading code comes from the examples provides by Boost.Asio and the bounded buffer producer-consumer solution is taken from the Boost.CircularBuffer examples (http://boost.org/).

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.95 src/algorithms/signal_source/gnuradio_blocks/unpack_2bit_samples.h File Reference

Unpacks 2 bit samples samples may be packed in any of the following ways: 1) Into bytes [item == byte] 1a) Big endian ordering within the byte 1b) Little endian ordering within the byte 2) Into shorts [item == short] 2a) Big endian ordering of bytes, big endian within the byte 2b) Big endian ordering of bytes, little endian within the byte 2c) Little endian ordering of bytes, big endian within the byte 2d) Little endian ordering of bytes, little endian within the byte.

```
#include <cstdint>
#include <gnuradio/sync_interpolator.h>
```

Classes

- class unpack_2bit_samples

  This class takes 2 bit samples that have been packed into bytes or shorts as input and generates a byte for each sample. It generates eight times as much data as is input (every two bits become 16 bits)

Typedefs

- typedef boost::shared_ptr< unpack_2bit_samples > **unpack_2bit_samples_sptr**

Functions

- unpack_2bit_samples_sptr **make_unpack_2bit_samples** (bool big_endian_bytes, size_t item_size, bool big_endian_items, bool reverse_interleaving=false)

9.95.1 Detailed Description

Unpacks 2 bit samples samples may be packed in any of the following ways: 1) Into bytes [item == byte] 1a) Big endian ordering within the byte 1b) Little endian ordering within the byte 2) Into shorts [item == short] 2a) Big endian ordering of bytes, big endian within the byte 2b) Big endian ordering of bytes, little endian within the byte 2c) Little endian ordering of bytes, big endian within the byte 2d) Little endian ordering of bytes, little endian within the byte.

Within a byte the two possibilities look like this: 7 6 5 4 3 2 1 0 : Bit number x_n,1 x_n,0 x_n+1,1 x_n+1,0 x_n+2,1 x_n+2,0 x_n+3,1 x_n+3,0 : Little endian x_n+3,1 x_n+3,0 x_n+2,1 x_n+2,0 x_n+1,0 x_n+1,0 x_n,1 x_n, 0 : Big Endian

For a short (uint16_t) the bytes are either transmitted as follows:

1 0 : Byte number Byte_n Byte_n+1 : Little endian Byte_n+1 Byte_n : Bit endian

The two bit values are assumed to have the following mapping:

x_1 x_0 Value 0 0 +1 0 1 +3 1 0 -3 1 1 -1

Letting x denote the two's complement interpretation of x_1 x_0, then:

Value = 2∗x + 1

We want to output the data in the order:

Value_0, Value_1, Value_2, ..., Value_n, Value_n+1, Value_n+2, ...

Cillian O'Driscoll cillian.odriscoll (at) gmail . com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.96 src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_cpx_samples.h File Reference

Unpacks byte samples to 2 bits complex samples. Packing Order Most Significant Nibble - Sample n Least Significant Nibble - Sample n+1 Packing order in Nibble Q1 Q0 I1 I0.

```
#include <gnuradio/sync_interpolator.h>
```

Classes

- class unpack_byte_2bit_cpx_samples

  This class implements conversion between byte packet samples to 2bit_cpx samples 1 byte = 2 x complex 2bit I, + 2bit Q samples.

Typedefs

- typedef boost::shared_ptr< unpack_byte_2bit_cpx_samples > **unpack_byte_2bit_cpx_samples_sptr**

Functions

- unpack_byte_2bit_cpx_samples_sptr **make_unpack_byte_2bit_cpx_samples** ()

9.96.1 Detailed Description

Unpacks byte samples to 2 bits complex samples. Packing Order Most Significant Nibble - Sample n Least Significant Nibble - Sample n+1 Packing order in Nibble Q1 Q0 I1 I0.

Javier Arribas jarribas (at) cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.97 src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_samples.h File Reference

Unpacks byte samples to NSR 2 bits samples.

```
#include <gnuradio/sync_interpolator.h>
```

Classes

- class unpack_byte_2bit_samples

  This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples.

Typedefs

- typedef boost::shared_ptr< unpack_byte_2bit_samples > **unpack_byte_2bit_samples_sptr**

Functions

- unpack_byte_2bit_samples_sptr **make_unpack_byte_2bit_samples** ()

9.97.1 Detailed Description

Unpacks byte samples to NSR 2 bits samples.

Javier Arribas jarribas (at) cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.98 src/algorithms/signal_source/gnuradio_blocks/unpack_intspir_1bit_samples.h File Reference

Unpacks SPIR int samples to NSR 1 bit samples.

```
#include <gnuradio/sync_interpolator.h>
```

Classes

- class unpack_intspir_1bit_samples

  This class implements conversion between byte packet samples to 2bit samples 1 byte = 4 2bit samples.

Typedefs

- typedef boost::shared_ptr< unpack_intspir_1bit_samples > **unpack_intspir_1bit_samples_sptr**

Functions

- unpack_intspir_1bit_samples_sptr **make_unpack_intspir_1bit_samples** ()

9.98.1 Detailed Description

Unpacks SPIR int samples to NSR 1 bit samples.

Fran Fabra fabra (at) ice.csic.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is not part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.99 src/algorithms/signal_source/libs/rtl_tcp_commands.h File Reference

Defines structures and constants for communicating with rtl_tcp.

```
#include <boost/asio/ip/tcp.hpp>
```

Enumerations

- enum RTL_TCP_COMMAND {
  RTL_TCP_SET_FREQUENCY = 1, **RTL_TCP_SET_SAMPLE_RATE** = 2, **RTL_TCP_SET_GAIN_MODE** = 3, **RTL_TCP_SET_GAIN** = 4,
  RTL_TCP_SET_IF_GAIN = 6, **RTL_TCP_SET_AGC_MODE** = 8 }

  Command IDs for configuration rtl_tcp.

Functions

- boost::system::error_code rtl_tcp_command (RTL_TCP_COMMAND id, unsigned param, boost::asio::ip↩ ::tcp::socket &socket)

  Send a command to rtl_tcp over the given socket.

9.99.1 Detailed Description

Defines structures and constants for communicating with rtl_tcp.

Author

Anthony Arnold, 2015. anthony.arnold(at)uqconnect.edu.au

This file contains information taken from librtlsdr:

http://git.osmocom.org/rtl-sdr/

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.99.2 Enumeration Type Documentation

9.99.2.1 enum RTL_TCP_COMMAND

Command IDs for configuration rtl_tcp.

Definition at line 38 of file rtl_tcp_commands.h.

9.99.3 Function Documentation

9.99.3.1 boost::system::error_code rtl_tcp_command (RTL_TCP_COMMAND *id*, unsigned *param*, boost::asio::ip::tcp::socket & *socket*)

Send a command to rtl_tcp over the given socket.

9.100 src/algorithms/signal_source/libs/rtl_tcp_dongle_info.h File Reference

Interface for a structure sent by rtl_tcp defining the hardware.

```
#include <boost/asio/ip/tcp.hpp>
```

Classes

- class rtl_tcp_dongle_info
  This class represents the dongle information which is sent by rtl_tcp.

9.100.1 Detailed Description

Interface for a structure sent by rtl_tcp defining the hardware.

Author

Anthony Arnold, 2015. anthony.arnold(at)uqconnect.edu.au

This file contains information taken from librtlsdr:

http://git.osmocom.org/rtl-sdr/

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.101 src/algorithms/telemetry_decoder/adapters/galileo_e1b_telemetry_decoder.h File Reference

Interface of an adapter of a GALILEO E1B NAV data decoder block to a TelemetryDecoderInterface.

```
#include <string>
#include "telemetry_decoder_interface.h"
#include "galileo_e1b_telemetry_decoder_cc.h"
#include "gnss_satellite.h"
```

Classes

- class GalileoE1BTelemetryDecoder

  This class implements a NAV data decoder for Galileo INAV frames in E1B radio link.

9.101.1 Detailed Description

Interface of an adapter of a GALILEO E1B NAV data decoder block to a TelemetryDecoderInterface.

Author

Javier Arribas 2013 jarribas(at)cttc.es, Mara Branzanti 2013. mara.branzanti(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.102 src/algorithms/telemetry_decoder/adapters/galileo_e5a_telemetry_decoder.h File Reference

Interface of an adapter of a GALILEO E5a FNAV data decoder block to a TelemetryDecoderInterface.

```
#include <string>
#include "telemetry_decoder_interface.h"
#include "galileo_e5a_telemetry_decoder_cc.h"
```

Classes

- class GalileoE5aTelemetryDecoder

  This class implements a NAV data decoder for Galileo INAV frames in E1B radio link.

9.102.1 Detailed Description

Interface of an adapter of a GALILEO E5a FNAV data decoder block to a TelemetryDecoderInterface.

Author

Marc Sales, 2014. marcsales92(at)gmail.com on work from:

- Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.103 src/algorithms/telemetry_decoder/adapters/gps_l1_ca_telemetry_decoder.h File Reference

Interface of an adapter of a GPS L1 C/A NAV data decoder block to a TelemetryDecoderInterface.

```
#include <string>
#include "telemetry_decoder_interface.h"
#include "gps_l1_ca_telemetry_decoder_cc.h"
```

Classes

- class GpsL1CaTelemetryDecoder

  This class implements a NAV data decoder for GPS L1 C/A.

9.103.1 Detailed Description

Interface of an adapter of a GPS L1 C/A NAV data decoder block to a TelemetryDecoderInterface.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.104 src/algorithms/telemetry_decoder/adapters/gps_l2c_telemetry_decoder.h File Reference

Interface of an adapter of a GPS L2C (CNAV) data decoder block to a TelemetryDecoderInterface.

```
#include <string>
#include "telemetry_decoder_interface.h"
#include "gps_l2c_telemetry_decoder_cc.h"
```

Classes

- class GpsL2CTelemetryDecoder

  This class implements a NAV data decoder for GPS L2 M.

9.104.1 Detailed Description

Interface of an adapter of a GPS L2C (CNAV) data decoder block to a TelemetryDecoderInterface.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.105 src/algorithms/telemetry_decoder/adapters/sbas_l1_telemetry_decoder.h File Reference

Interface of an adapter of a SBAS telemetry data decoder block to a TelemetryDecoderInterface.

```
#include <string>
#include "telemetry_decoder_interface.h"
#include "sbas_l1_telemetry_decoder_cc.h"
```

Classes

- class SbasL1TelemetryDecoder

  This class implements a NAV data decoder for SBAS frames in L1 radio link.

9.105.1 Detailed Description

Interface of an adapter of a SBAS telemetry data decoder block to a TelemetryDecoderInterface.

Author

Daniel Fehr 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.106 src/algorithms/telemetry_decoder/gnuradio_blocks/galileo_e1b_telemetry_decoder_cc.h File Reference

Interface of a Galileo INAV message demodulator block.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include "Galileo_E1.h"
#include "concurrent_queue.h"
#include "gnss_satellite.h"
#include "galileo_navigation_message.h"
#include "galileo_ephemeris.h"
#include "galileo_almanac.h"
#include "galileo_iono.h"
#include "galileo_utc_model.h"
```

Classes

- class galileo_e1b_telemetry_decoder_cc

  This class implements a block that decodes the INAV data defined in Galileo ICD.

Typedefs

- typedef boost::shared_ptr< galileo_e1b_telemetry_decoder_cc > **galileo_e1b_telemetry_decoder_cc_sptr**

Functions

- galileo_e1b_telemetry_decoder_cc_sptr **galileo_e1b_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

9.106.1 Detailed Description

Interface of a Galileo INAV message demodulator block.

Author

Javier Arribas 2013 jarribas(at)cttc.es, Mara Branzanti 2013 mara.branzanti(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.107 src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l1_ca_telemetry_decoder_cc.h File Reference

Interface of a NAV message demodulator block based on Kay Borre book MATLAB-based GPS receiver.

```
#include <fstream>
#include <string>
#include <gnuradio/block.h>
#include <deque>
#include "GPS_L1_CA.h"
#include "gps_l1_ca_subframe_fsm.h"
#include "concurrent_queue.h"
#include "gnss_satellite.h"
```

Classes

- class gps_l1_ca_telemetry_decoder_cc

  This class implements a block that decodes the NAV data defined in IS-GPS-200E.

Typedefs

- typedef boost::shared_ptr< gps_l1_ca_telemetry_decoder_cc > **gps_l1_ca_telemetry_decoder_cc_sptr**

Functions

- gps_l1_ca_telemetry_decoder_cc_sptr **gps_l1_ca_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

9.107.1 Detailed Description

Interface of a NAV message demodulator block based on Kay Borre book MATLAB-based GPS receiver.

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.108 src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l2c_telemetry_decoder_cc.h File Reference

Interface of a CNAV message demodulator block based on Kay Borre book MATLAB-based GPS receiver.

```
#include <algorithm>
#include <deque>
#include <fstream>
#include <string>
#include <utility>
#include <vector>
#include <boost/crc.hpp>
#include <gnuradio/block.h>
#include "gnss_satellite.h"
#include "viterbi_decoder.h"
#include "gps_cnav_navigation_message.h"
#include "gps_cnav_ephemeris.h"
#include "gps_cnav_iono.h"
#include "concurrent_queue.h"
#include "GPS_L2C.h"
```

Classes

- class gps_l2c_telemetry_decoder_cc

  This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229.

Typedefs

- typedef boost::shared_ptr< gps_l2c_telemetry_decoder_cc > **gps_l2c_telemetry_decoder_cc_sptr**

Functions

- gps_l2c_telemetry_decoder_cc_sptr **gps_l2c_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

9.108.1 Detailed Description

Interface of a CNAV message demodulator block based on Kay Borre book MATLAB-based GPS receiver.

Javier Arribas, 2015. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.109 src/algorithms/telemetry_decoder/gnuradio_blocks/sbas_l1_telemetry_decoder_cc.h File Reference

Interface of a SBAS telemetry data decoder block.

```
#include <algorithm>
#include <deque>
#include <fstream>
#include <string>
#include <utility>
#include <vector>
#include <boost/crc.hpp>
#include <gnuradio/block.h>
#include "gnss_satellite.h"
#include "viterbi_decoder.h"
#include "sbas_telemetry_data.h"
```

Classes

- class sbas_l1_telemetry_decoder_cc

  This class implements a block that decodes the SBAS integrity and corrections data defined in RTCA MOPS DO-229.

Typedefs

- typedef boost::shared_ptr< sbas_l1_telemetry_decoder_cc > **sbas_l1_telemetry_decoder_cc_sptr**

Functions

- sbas_l1_telemetry_decoder_cc_sptr **sbas_l1_make_telemetry_decoder_cc** (Gnss_Satellite satellite, bool dump)

9.109.1 Detailed Description

Interface of a SBAS telemetry data decoder block.

Author

Daniel Fehr 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.110 src/algorithms/telemetry_decoder/libs/convolutional.h File Reference

General functions used to implement convolutional encoding.

```
#include <cstdlib>
```

Variables

- const float **MAXLOG** = 1e7

9.110.1 Detailed Description

General functions used to implement convolutional encoding.

Author

Matthew C. Valenti

Copyright (C) 2006-2008 Matthew C. Valenti

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This file is a derived work of the original file, which had this note:

Last updated on May 22, 2008

The functions in this file are part of the Iterative Solutions Coded Modulation Library. The Iterative Solutions Coded Modulation Library is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation; either version 2.1 of the License, or (at your option) any later version.

This library is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public License along with this library; if not, write to the Free Software Foundation, Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA

9.111 src/algorithms/telemetry_decoder/libs/gps_l1_ca_subframe_fsm.h File Reference

Interface of a GPS NAV message word-to-subframe decoder state machine.

```
#include <boost/statechart/state_machine.hpp>
#include "concurrent_queue.h"
#include "GPS_L1_CA.h"
#include "gps_navigation_message.h"
#include "gps_ephemeris.h"
#include "gps_iono.h"
#include "gps_almanac.h"
#include "gps_utc_model.h"
```

Classes

- class GpsL1CaSubframeFsm

  This class implements a Finite State Machine that handles the decoding of the GPS L1 C/A NAV message.

9.111.1 Detailed Description

Interface of a GPS NAV message word-to-subframe decoder state machine.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.112 src/algorithms/telemetry_decoder/libs/viterbi_decoder.h File Reference

Interface of a Viterbi decoder class based on the Iterative Solutions Coded Modulation Library by Matthew C. Valenti.

```
#include <deque>
#include <cstdio>
```

Classes

- class Viterbi_Decoder

  Class that implements a Viterbi decoder.

9.112.1 Detailed Description

Interface of a Viterbi decoder class based on the Iterative Solutions Coded Modulation Library by Matthew C. Valenti.

Author

Daniel Fehr 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.113 src/algorithms/tracking/adapters/galileo_e1_dll_pll_veml_tracking.h File Reference

Adapts a DLL+PLL VEML (Very Early Minus Late) tracking loop block to a TrackingInterface for Galileo E1 signals.

```
#include <string>
#include "tracking_interface.h"
#include "galileo_e1_dll_pll_veml_tracking_cc.h"
```

Classes

- class GalileoE1DllPllVemlTracking

  This class Adapts a DLL+PLL VEML (Very Early Minus Late) tracking loop block to a TrackingInterface for Galileo E1 signals.

9.113.1 Detailed Description

Adapts a DLL+PLL VEML (Very Early Minus Late) tracking loop block to a TrackingInterface for Galileo E1 signals.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkha user, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.114 src/algorithms/tracking/adapters/galileo_e1_tcp_connector_tracking.h File Reference

Interface of an adapter of a TCP connector block based on code DLL + carrier PLL for Galileo E1 to a TrackingInterface.

```
#include <string>
#include "tracking_interface.h"
#include "galileo_e1_tcp_connector_tracking_cc.h"
```

Classes

- class GalileoE1TcpConnectorTracking

  This class implements a code DLL + carrier PLL tracking loop.

9.114.1 Detailed Description

Interface of an adapter of a TCP connector block based on code DLL + carrier PLL for Galileo E1 to a TrackingInterface.

Author

David Pubill, 2012. dpubill(at)cttc.es Luis Esteve, 2012. luis(at)epsilon-formacion.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2012-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.115 src/algorithms/tracking/adapters/galileo_e5a_dll_pll_tracking.h File Reference

Adapts a code DLL + carrier PLL tracking block to a TrackingInterface for Galileo E5a signals.

```
#include <string>
#include "tracking_interface.h"
#include "galileo_e5a_dll_pll_tracking_cc.h"
```

Classes

- class GalileoE5aDllPllTracking

  This class implements a code DLL + carrier PLL tracking loop.

9.115.1 Detailed Description

Adapts a code DLL + carrier PLL tracking block to a TrackingInterface for Galileo E5a signals.

Adapts a PCPS acquisition block to an AcquisitionInterface for Galileo E5a data and pilot Signals

Author

Marc Sales, 2014. marcsales92(at)gmail.com on work from:

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.116 src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_c_aid_tracking.h File Reference

Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface.

```
#include <string>
#include "tracking_interface.h"
#include "gps_l1_ca_dll_pll_c_aid_tracking_cc.h"
#include "gps_l1_ca_dll_pll_c_aid_tracking_sc.h"
```

Classes

- class GpsL1CaDllPllCAidTracking

  This class implements a code DLL + carrier PLL tracking loop.

9.116.1 Detailed Description

Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkha user, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.117 src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking.h File Reference

Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface.

```
#include <string>
#include "tracking_interface.h"
#include "gps_l1_ca_dll_pll_tracking_cc.h"
```

Classes

- class GpsL1CaDllPllTracking

  This class implements a code DLL + carrier PLL tracking loop.

9.117.1 Detailed Description

Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkha user, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.118 src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_tracking_gpu.h File Reference

Implementation of an adapter of a DLL+PLL tracking loop block using GPU accelerated functions for GPS L1 C/A to a TrackingInterface.

```
#include <string>
#include "tracking_interface.h"
#include "gps_l1_ca_dll_pll_tracking_gpu_cc.h"
```

Classes

- class GpsL1CaDllPllTrackingGPU

  This class implements a code DLL + carrier PLL tracking loop using GPU accelerated functions.

9.118.1 Detailed Description

Implementation of an adapter of a DLL+PLL tracking loop block using GPU accelerated functions for GPS L1 C/A to a TrackingInterface.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkha user, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.119 src/algorithms/tracking/adapters/gps_l1_ca_tcp_connector_tracking.h File Reference

Interface of an adapter of a TCP connector block based on code DLL + carrier PLL for GPS L1 C/A to a TrackingInterface.

```
#include <string>
#include "tracking_interface.h"
#include "gps_l1_ca_tcp_connector_tracking_cc.h"
```

Classes

- class GpsL1CaTcpConnectorTracking

  This class implements a code DLL + carrier PLL tracking loop.

9.119.1 Detailed Description

Interface of an adapter of a TCP connector block based on code DLL + carrier PLL for GPS L1 C/A to a TrackingInterface.

Author

David Pubill, 2012. dpubill(at)cttc.es Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkha user, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.120 src/algorithms/tracking/adapters/gps_l2_m_dll_pll_tracking.h File Reference

Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface.

```
#include <string>
#include "tracking_interface.h"
#include "gps_l2_m_dll_pll_tracking_cc.h"
```

Classes

- class GpsL2MDllPllTracking

  This class implements a code DLL + carrier PLL tracking loop.

9.120.1 Detailed Description

Interface of an adapter of a DLL+PLL tracking loop block for GPS L1 C/A to a TrackingInterface.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkha user, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.121 src/algorithms/tracking/gnuradio_blocks/galileo_e1_dll_pll_veml_tracking_cc.h File Reference

Implementation of a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

```
#include <fstream>
#include <string>
#include <map>
#include <gnuradio/block.h>
#include "gnss_synchro.h"
#include "tracking_2nd_DLL_filter.h"
#include "tracking_2nd_PLL_filter.h"
#include "cpu_multicorrelator.h"
```

Classes

- class galileo_e1_dll_pll_veml_tracking_cc

  This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

Typedefs

- typedef boost::shared_ptr< galileo_e1_dll_pll_veml_tracking_cc > **galileo_e1_dll_pll_veml_tracking_cc_sptr**

Functions

- galileo_e1_dll_pll_veml_tracking_cc_sptr **galileo_e1_dll_pll_veml_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips, float very_early_late_space_chips)

9.121.1 Detailed Description

Implementation of a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.122 src/algorithms/tracking/gnuradio_blocks/galileo_e1_tcp_connector_tracking_cc.h File Reference

Interface of a TCP connector block based on code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

```
#include <fstream>
#include <map>
#include <string>
#include <gnuradio/block.h>
#include <volk/volk.h>
#include "gnss_synchro.h"
#include "cpu_multicorrelator.h"
#include "tcp_communication.h"
```

Classes

- class Galileo_E1_Tcp_Connector_Tracking_cc

  This class implements a code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

Typedefs

- typedef boost::shared_ptr< Galileo_E1_Tcp_Connector_Tracking_cc > **galileo_e1_tcp_connector_tracking_cc_sptr**

Functions

- galileo_e1_tcp_connector_tracking_cc_sptr **galileo_e1_tcp_connector_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips, float very_early_late_space_chips, size_t port_ch0)

9.122.1 Detailed Description

Interface of a TCP connector block based on code DLL + carrier PLL VEML (Very Early Minus Late) tracking block for Galileo E1 signals.

Author

David Pubill, 2012. dpubill(at)cttc.es Luis Esteve, 2012. luis(at)epsilon-formacion.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.123 src/algorithms/tracking/gnuradio_blocks/galileo_e5a_dll_pll_tracking_cc.h File Reference

Implementation of a code DLL + carrier PLL tracking block for Galileo E5a signals.

```
#include <fstream>
#include <map>
#include <string>
#include <gnuradio/block.h>
#include "gnss_synchro.h"
#include "tracking_2nd_DLL_filter.h"
#include "tracking_2nd_PLL_filter.h"
#include "cpu_multicorrelator.h"
```

Classes

- class Galileo_E5a_Dll_Pll_Tracking_cc

  This class implements a DLL + PLL tracking loop block.

Typedefs

- typedef boost::shared_ptr< Galileo_E5a_Dll_Pll_Tracking_cc > **galileo_e5a_dll_pll_tracking_cc_sptr**

Functions

- galileo_e5a_dll_pll_tracking_cc_sptr **galileo_e5a_dll_pll_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float pll_↩ bw_init_hz, float dll_bw_init_hz, int ti_ms, float early_late_space_chips)

9.123.1 Detailed Description

Implementation of a code DLL + carrier PLL tracking block for Galileo E5a signals.

Author

Marc Sales, 2014. marcsales92(at)gmail.com on work from:

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.124 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_cc.h File Reference

Interface of a code DLL + carrier PLL tracking block.

```
#include <fstream>
#include <map>
#include <deque>
#include <string>
#include <gnuradio/block.h>
#include <pmt/pmt.h>
#include "gnss_synchro.h"
#include "tracking_2nd_DLL_filter.h"
#include "tracking_FLL_PLL_filter.h"
#include "cpu_multicorrelator.h"
```

Classes

- class gps_l1_ca_dll_pll_c_aid_tracking_cc

  This class implements a DLL + PLL tracking loop block.

Typedefs

- typedef boost::shared_ptr< gps_l1_ca_dll_pll_c_aid_tracking_cc > **gps_l1_ca_dll_pll_c_aid_tracking_cc_sptr**

Functions

- gps_l1_ca_dll_pll_c_aid_tracking_cc_sptr **gps_l1_ca_dll_pll_c_aid_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float pll_bw_narrow_hz, float dll_bw_narrow_hz, int extend_correlation_ms, float early_late_space_chips)

9.124.1 Detailed Description

Interface of a code DLL + carrier PLL tracking block.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.125 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_c_aid_tracking_sc.h File Reference

Interface of a code DLL + carrier PLL tracking block.

```
#include <fstream>
#include <map>
#include <string>
#include <boost/thread/mutex.hpp>
#include <boost/thread/thread.hpp>
#include <gnuradio/block.h>
#include <volk/volk.h>
#include "gps_sdr_signal_processing.h"
#include "gnss_synchro.h"
#include "tracking_2nd_DLL_filter.h"
#include "tracking_FLL_PLL_filter.h"
#include "cpu_multicorrelator_16sc.h"
```

Classes

- class gps_l1_ca_dll_pll_c_aid_tracking_sc

  This class implements a DLL + PLL tracking loop block.

Typedefs

- typedef boost::shared_ptr< gps_l1_ca_dll_pll_c_aid_tracking_sc > **gps_l1_ca_dll_pll_c_aid_tracking_sc_sptr**

Functions

- gps_l1_ca_dll_pll_c_aid_tracking_sc_sptr **gps_l1_ca_dll_pll_c_aid_make_tracking_sc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float pll_bw_narrow_hz, float dll_bw_narrow_hz, int extend_correlation_ms, float early_late_space_chips)

9.125.1 Detailed Description

Interface of a code DLL + carrier PLL tracking block.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.126 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_cc.h File Reference

Interface of a code DLL + carrier PLL tracking block.

```
#include <fstream>
#include <map>
#include <string>
#include <gnuradio/block.h>
#include "gnss_synchro.h"
#include "tracking_2nd_DLL_filter.h"
#include "tracking_2nd_PLL_filter.h"
#include "cpu_multicorrelator.h"
```

Classes

- class Gps_L1_Ca_Dll_Pll_Tracking_cc

  This class implements a DLL + PLL tracking loop block.

Typedefs

- typedef boost::shared_ptr< Gps_L1_Ca_Dll_Pll_Tracking_cc > **gps_l1_ca_dll_pll_tracking_cc_sptr**

Functions

- gps_l1_ca_dll_pll_tracking_cc_sptr **gps_l1_ca_dll_pll_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips)

9.126.1 Detailed Description

Interface of a code DLL + carrier PLL tracking block.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.127 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_dll_pll_tracking_gpu_cc.h File Reference

Implementation of a code DLL + carrier PLL tracking block, GPU ACCELERATED.

```
#include <fstream>
#include <map>
#include <string>
#include <gnuradio/block.h>
#include "gnss_synchro.h"
#include "tracking_2nd_DLL_filter.h"
#include "tracking_FLL_PLL_filter.h"
#include "cuda_multicorrelator.h"
```

Classes

- class Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc

  This class implements a DLL + PLL tracking loop block.

Typedefs

- typedef boost::shared_ptr< Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc > **gps_l1_ca_dll_pll_tracking_gpu_cc_sptr**

Functions

- gps_l1_ca_dll_pll_tracking_gpu_cc_sptr **gps_l1_ca_dll_pll_make_tracking_gpu_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_space_chips)

9.127.1 Detailed Description

Implementation of a code DLL + carrier PLL tracking block, GPU ACCELERATED.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.128 src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_tcp_connector_tracking_cc.h File Reference

Interface of a TCP connector block based on code DLL + carrier PLL.

```
#include <fstream>
#include <map>
#include <string>
#include <gnuradio/block.h>
#include "gnss_synchro.h"
#include "cpu_multicorrelator.h"
#include "tcp_communication.h"
```

Classes

- class Gps_L1_Ca_Tcp_Connector_Tracking_cc

  This class implements a DLL + PLL tracking loop block.

Typedefs

- typedef boost::shared_ptr< Gps_L1_Ca_Tcp_Connector_Tracking_cc > **gps_l1_ca_tcp_connector_tracking_cc_sptr**

Functions

- gps_l1_ca_tcp_connector_tracking_cc_sptr **gps_l1_ca_tcp_connector_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float early_late_space_chips, size_t port_ch0)

9.128.1 Detailed Description

Interface of a TCP connector block based on code DLL + carrier PLL.

Author

David Pubill, 2012. dpubill(at)cttc.es Javier Arribas, 2011. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.129 src/algorithms/tracking/gnuradio_blocks/gps_l2_m_dll_pll_tracking_cc.h File Reference

Interface of a code DLL + carrier PLL tracking block.

```
#include <fstream>
#include <map>
#include <string>
#include <gnuradio/block.h>
#include "gnss_synchro.h"
#include "tracking_2nd_DLL_filter.h"
#include "tracking_2nd_PLL_filter.h"
#include "cpu_multicorrelator.h"
```

Classes

- class gps_l2_m_dll_pll_tracking_cc

  This class implements a DLL + PLL tracking loop block.

Typedefs

- typedef boost::shared_ptr< gps_l2_m_dll_pll_tracking_cc > **gps_l2_m_dll_pll_tracking_cc_sptr**

Functions

- gps_l2_m_dll_pll_tracking_cc_sptr **gps_l2_m_dll_pll_make_tracking_cc** (long if_freq, long fs_in, unsigned int vector_length, bool dump, std::string dump_filename, float pll_bw_hz, float dll_bw_hz, float early_late_↩ space_chips)

9.129.1 Detailed Description

Interface of a code DLL + carrier PLL tracking block.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Code DLL + carrier PLL according to the algorithms described in: K.Borre, D.M.Akos, N.Bertelsen, P.Rinder, and S.H.Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.130 src/algorithms/tracking/libs/cpu_multicorrelator.h File Reference

High optimized CPU vector multiTAP correlator class.

```
#include <complex>
```

Classes

- class cpu_multicorrelator

  Class that implements carrier wipe-off and correlators.

9.130.1 Detailed Description

High optimized CPU vector multiTAP correlator class.

Authors

- Javier Arribas, 2015. jarribas(at)cttc.es

Class that implements a high optimized vector multiTAP correlator class for CPUs

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.131 src/algorithms/tracking/libs/cpu_multicorrelator_16sc.h File Reference

High optimized CPU vector multiTAP correlator class for lv_16sc_t (short int complex)

```
#include <volk_gnsssdr/volk_gnsssdr.h>
```

Classes

- class cpu_multicorrelator_16sc

  Class that implements carrier wipe-off and correlators.

9.131.1 Detailed Description

High optimized CPU vector multiTAP correlator class for lv_16sc_t (short int complex)

Authors

- Javier Arribas, 2016. jarribas(at)cttc.es

Class that implements a high optimized vector multiTAP correlator class for CPUs

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.132 src/algorithms/tracking/libs/cuda_multicorrelator.h File Reference

High optimized CUDA GPU vector multiTAP correlator class.

```
#include <complex>
#include <cuda.h>
#include <cuda_runtime.h>
```

Classes

- struct GPU_Complex
- struct GPU_Complex_Short
- class cuda_multicorrelator

  Class that implements carrier wipe-off and correlators using NVIDIA CUDA GPU accelerators.

9.132.1 Detailed Description

High optimized CUDA GPU vector multiTAP correlator class.

Authors

- Javier Arribas, 2015. jarribas(at)cttc.es

Class that implements a high optimized vector multiTAP correlator class for NVIDIA CUDA GPUs

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.133 src/algorithms/tracking/libs/lock_detectors.h File Reference

Interface of a library with a set of code and carrier phase lock detectors.

```
#include <gnuradio/gr_complex.h>
```

Functions

- float cn0_svn_estimator (gr_complex *Prompt_buffer, int length, long fs_in, double code_length)
  CN0_SNV is a Carrier-to-Noise (CN0) estimator based on the Signal-to-Noise Variance (SNV) estimator.
- float carrier_lock_detector (gr_complex *Prompt_buffer, int length)
  A carrier lock detector.

9.133.1 Detailed Description

Interface of a library with a set of code and carrier phase lock detectors.

SNV_CN0 is a Carrier-to-Noise (CN0) estimator based on the Signal-to-Noise Variance (SNV) estimator [1]. Carrier lock detector using normalised estimate of the cosine of twice the carrier phase error [2].

[1] Marco Pini, Emanuela Falletti and Maurizio Fantino, "Performance Evaluation of C/N0 Estimators using a Real Time GNSS Software Receiver," IEEE 10th International Symposium on Spread Spectrum Techniques and Applications, pp.28-30, August 2008.

[2] Van Dierendonck, A.J. (1996), Global Positioning System: Theory and Applications, Volume I, Chapter 8: GPS Receivers, AJ Systems, Los Altos, CA 94024. Inc.: 329-407.

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.133.2 Function Documentation

9.133.2.1 float carrier_lock_detector (gr_complex * *Prompt_buffer,* int *length*)

A carrier lock detector.

The Carrier Phase Lock Detector block uses the estimate of the cosine of twice the carrier phase error is given by

$$C2\phi = \frac{NBD}{NBP}, \tag{9.1}$$

where $NBD = (\sum_{i=0}^{N-1} |Im(Pc(i))|)^2 + (\sum_{i=0}^{N-1} |Re(Pc(i))|)^2$, $NBP = \sum_{i=0}^{N-1} Im(Pc(i))^2 - \sum_{i=0}^{N-1} Re(Pc(i))^2$, and $Pc(i)$ is the prompt correlator output for the sample index i. Ref: Van Dierendonck, A.J. (1996), Global Positioning System: Theory and Applications, Volume I, Chapter 8: GPS Receivers, AJ Systems, Los Altos, CA 94024. Inc.: 329-407.

9.133.2.2 float cn0_svn_estimator (gr_complex * *Prompt_buffer,* int *length,* long *fs_in,* double *code_length*)

CN0_SNV is a Carrier-to-Noise (CN0) estimator based on the Signal-to-Noise Variance (SNV) estimator.

Signal-to-Noise (SNR) (ρ) estimator using the Signal-to-Noise Variance (SNV) estimator:

$$\hat{\rho} = \frac{\hat{P}_s}{\hat{P}_n} = \frac{\hat{P}_s}{\hat{P}_{tot} - \hat{P}_s}, \tag{9.2}$$

where $\hat{P}_s = \left(\frac{1}{N}\sum_{i=0}^{N-1} |Re(Pc(i))|\right)^2$ is the estimation of the signal power, $\hat{P}_{tot} = \frac{1}{N}\sum_{i=0}^{N-1} |Pc(i)|^2$ is the estimator of the total power, $|\cdot|$ is the absolute value, $Re(\cdot)$ stands for the real part of the value, and $Pc(i)$ is the prompt correlator output for the sample index i.

The SNR value is converted to CN0 [dB-Hz], taking to account the receiver bandwidth and the PRN code gain, using the following formula:

$$CN0_{dB} = 10 * log(\hat{\rho}) + 10 * log(\frac{f_s}{2}) - 10 * log(L_{PRN}), \tag{9.3}$$

where f_s is the sampling frequency and L_{PRN} is the PRN sequence length. Ref: Marco Pini, Emanuela Falletti and Maurizio Fantino, "Performance Evaluation of C/N0 Estimators using a Real Time GNSS Software Receiver," IEEE 10th International Symposium on Spread Spectrum Techniques and Applications, pp.28-30, August 2008.

9.134 src/algorithms/tracking/libs/tcp_communication.h File Reference

Interface of the TCP communication class.

```
#include <boost/asio.hpp>
#include <boost/array.hpp>
#include "tcp_packet_data.h"
```

Classes

- class tcp_communication

  TCP communication class.

Macros

- #define **NUM_TX_VARIABLES_GALILEO_E1** 13
- #define **NUM_TX_VARIABLES_GPS_L1_CA** 9
- #define **NUM_RX_VARIABLES** 4

9.134.1 Detailed Description

Interface of the TCP communication class.

Author

David Pubill, 2011. dpubill(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.135 src/algorithms/tracking/libs/tcp_packet_data.h File Reference

Interface of the TCP data packet class.

Classes

- class tcp_packet_data

  Class that implements a TCP data packet.

9.135.1 Detailed Description

Interface of the TCP data packet class.

Author

David Pubill, 2011. dpubill(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.136 src/algorithms/tracking/libs/tracking_2nd_DLL_filter.h File Reference

Interface of a 2nd order DLL filter for code tracking loop.

Classes

- class Tracking_2nd_DLL_filter

  This class implements a 2nd order DLL filter for code tracking loop.

9.136.1 Detailed Description

Interface of a 2nd order DLL filter for code tracking loop.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Class that implements a 2nd order PLL filter for code tracking loop. The algorithm is described in: K.Borre, D.M.↩ Akos, N.Bertelsen, P.Rinder, and S. H. Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007, Applied and Numerical Harmonic Analysis.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.137 src/algorithms/tracking/libs/tracking_2nd_PLL_filter.h File Reference

Interface of a 2nd order PLL filter for carrier tracking loop.

Classes

- class Tracking_2nd_PLL_filter

  This class implements a 2nd order PLL filter for carrier tracking loop.

9.137.1 Detailed Description

Interface of a 2nd order PLL filter for carrier tracking loop.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Class that implements 2 order PLL filter for tracking carrier loop. The algorithm is described in K.Borre, D.M.↩ Akos, N.Bertelsen, P.Rinder, and S.H. Jensen, A Software-Defined GPS and Galileo Receiver. A Single-Frequency Approach, Birkhauser, 2007, Applied and Numerical Harmonic Analysis.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http↩ ://www.gnu.org/licenses/.

9.138 src/algorithms/tracking/libs/tracking_discriminators.h File Reference

Interface of a library with a set of code tracking and carrier tracking discriminators.

```
#include <gnuradio/gr_complex.h>
```

Functions

- double fll_four_quadrant_atan (gr_complex prompt_s1, gr_complex prompt_s2, double t1, double t2)
- double pll_four_quadrant_atan (gr_complex prompt_s1)

  PLL four quadrant arctan discriminator.
- double pll_cloop_two_quadrant_atan (gr_complex prompt_s1)

  PLL Costas loop two quadrant arctan discriminator.
- double dll_nc_e_minus_l_normalized (gr_complex early_s1, gr_complex late_s1)

  DLL Noncoherent Early minus Late envelope normalized discriminator.
- double dll_nc_vemlp_normalized (gr_complex very_early_s1, gr_complex early_s1, gr_complex late_s1, gr_complex very_late_s1)

  DLL Noncoherent Very Early Minus Late Power (VEMLP) normalized discriminator.

9.138.1 Detailed Description

Interface of a library with a set of code tracking and carrier tracking discriminators.

Authors

- Javier Arribas, 2011. jarribas(at)cttc.es
- Luis Esteve, 2012. luis(at)epsilon-formacion.com

Library with a set of code tracking and carrier tracking discriminators that is used by the tracking algorithms.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.138.2 Function Documentation

9.138.2.1 double dll_nc_e_minus_l_normalized (gr_complex *early_s1*, gr_complex *late_s1*)

DLL Noncoherent Early minus Late envelope normalized discriminator.

DLL Noncoherent Early minus Late envelope normalized discriminator:

$$error = \frac{E - L}{E + L}, \tag{9.4}$$

where $E = \sqrt{I_{ES}^2 + Q_{ES}^2}$ is the Early correlator output absolute value and $L = \sqrt{I_{LS}^2 + Q_{LS}^2}$ is the Late correlator output absolute value. The output is in [chips].

9.138.2.2 double dll_nc_vemlp_normalized (gr_complex *very_early_s1*, gr_complex *early_s1*, gr_complex *late_s1*, gr_complex *very_late_s1*)

DLL Noncoherent Very Early Minus Late Power (VEMLP) normalized discriminator.

DLL Noncoherent Very Early Minus Late Power (VEMLP) normalized discriminator, using the outputs of four correlators, Very Early (VE), Early (E), Late (L) and Very Late (VL):

$$error = \frac{E - L}{E + L}, \tag{9.5}$$

where $E = \sqrt{I_{VE}^2 + Q_{VE}^2 + I_E^2 + Q_E^2}$ and $L = \sqrt{I_{VL}^2 + Q_{VL}^2 + I_L^2 + Q_L^2}$. The output is in [chips].

9.138.2.3 double fll_four_quadrant_atan (gr_complex *prompt_s1*, gr_complex *prompt_s2*, double *t1*, double *t2*)

brief FLL four quadrant arctan discriminator

FLL four quadrant arctan discriminator:

$$\frac{\phi_2 - \phi_1}{t_2 - t1} = \frac{ATAN2(cross, dot)}{t_1 - t_2}, \tag{9.6}$$

where $cross = I_{PS1}Q_{PS2} - I_{PS2}Q_{PS1}$ and $dot = I_{PS1}I_{PS2} + Q_{PS1}Q_{PS2}$, I_{PS1}, Q_{PS1} are the inphase and quadrature prompt correlator outputs respectively at sample time t_1, and I_{PS2}, Q_{PS2} are the inphase and quadrature prompt correlator outputs respectively at sample time t_2. The output is in [radians/second].

9.138.2.4 double pll_cloop_two_quadrant_atan (gr_complex *prompt_s1*)

PLL Costas loop two quadrant arctan discriminator.

PLL Costas loop two quadrant arctan discriminator:

$$\phi = ATAN\left(\frac{Q_{PS}}{I_{PS}}\right), \tag{9.7}$$

where I_{PS1}, Q_{PS1} are the inphase and quadrature prompt correlator outputs respectively. The output is in [radians].

9.138.2.5 double pll_four_quadrant_atan (gr_complex *prompt_s1*)

PLL four quadrant arctan discriminator.

PLL four quadrant arctan discriminator:

$$\phi = ATAN2(Q_{PS}, I_{PS}), \tag{9.8}$$

where I_{PS1}, Q_{PS1} are the inphase and quadrature prompt correlator outputs respectively. The output is in [radians].

9.139 src/algorithms/tracking/libs/tracking_FLL_PLL_filter.h File Reference

Interface of a hybrid FLL and PLL filter for tracking carrier loop.

Classes

- class Tracking_FLL_PLL_filter

  This class implements a hybrid FLL and PLL filter for tracking carrier loop.

9.139.1 Detailed Description

Interface of a hybrid FLL and PLL filter for tracking carrier loop.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.140 src/algorithms/tracking/libs/tracking_loop_filter.h File Reference

Generic 1st to 3rd order loop filter implementation.

```
#include <vector>
```

Classes

- class Tracking_loop_filter

  This class implements a generic 1st, 2nd or 3rd order loop filter.

9.140.1 Detailed Description

Generic 1st to 3rd order loop filter implementation.

Author

Cillian O'Driscoll, 2015. cillian.odriscoll(at)gmail.com

Class implementing a generic 1st, 2nd or 3rd order loop filter. Based on the bilinear transform of the standard Weiner filter.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.141 src/core/interfaces/acquisition_interface.h File Reference

Header file of the interface to an acquisition GNSS block.

```
#include "gnss_block_interface.h"
#include "gnss_synchro.h"
```

Classes

- class concurrent_queue< Data >

  This class implements a thread-safe std::queue.
- class AcquisitionInterface

  This abstract class represents an interface to an acquisition GNSS block.

9.141.1 Detailed Description

Header file of the interface to an acquisition GNSS block.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Luis Esteve, 2011. luis(at)epsilon-formacion.com

This header file contains the interface to an abstract class for acquisition algorithms. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.142 src/core/interfaces/channel_interface.h File Reference

This class represents an interface to a channel GNSS block.

```
#include "gnss_block_interface.h"
#include "gnss_signal.h"
```

Classes

- class ChannelInterface

  This abstract class represents an interface to a channel GNSS block.

9.142.1 Detailed Description

This class represents an interface to a channel GNSS block.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Luis Esteve, 2011. luis(at)epsilon-formacion.com

Abstract class for channel blocks. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.143 src/core/interfaces/configuration_interface.h File Reference

This class represents an interface to configuration parameters.

```
#include <string>
```

Classes

- class ConfigurationInterface

  This abstract class represents an interface to configuration parameters.

9.143.1 Detailed Description

This class represents an interface to configuration parameters.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

The interface defines an accessor method that gets a parameter name as input and returns the value of this parameter, a string, as output. Property names are defined here. This is an abstract class for interfaces.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.144 src/core/interfaces/gnss_block_interface.h File Reference

This interface represents a GNSS block.

```
#include <cassert>
#include <string>
#include <gnuradio/top_block.h>
```

Classes

- class GNSSBlockInterface

  This abstract class represents an interface to GNSS blocks.

9.144.1 Detailed Description

This interface represents a GNSS block.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Abstract class for GNSS block interfaces. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.145 src/core/interfaces/observables_interface.h File Reference

This class represents an interface to an Observables block.

```
#include "gnss_block_interface.h"
```

Classes

- class ObservablesInterface

  This abstract class represents an interface to an observables block.

9.145.1 Detailed Description

This class represents an interface to an Observables block.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Abstract class for Observables modules. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.146 src/core/interfaces/pvt_interface.h File Reference

This class represents an interface to a PVT block.

```
#include "gnss_block_interface.h"
```

Classes

- class PvtInterface

  This class represents an interface to a PVT block.

9.146.1 Detailed Description

This class represents an interface to a PVT block.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Abstract class for PVT solvers. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.147 src/core/interfaces/telemetry_decoder_interface.h File Reference

This class represents an interface to a telemetry decoder block.

```
#include "gnss_block_interface.h"
#include "gnss_satellite.h"
```

Classes

- class TelemetryDecoderInterface

  This abstract class represents an interface to a navigation GNSS block.

9.147.1 Detailed Description

This class represents an interface to a telemetry decoder block.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Abstract class for telemetry decoders. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.148 src/core/interfaces/tracking_interface.h File Reference

This class represents an interface to a tracking block.

```
#include "gnss_block_interface.h"
#include "gnss_synchro.h"
```

Classes

- class concurrent_queue< Data >

  This class implements a thread-safe std::queue.

- class TrackingInterface

  This abstract class represents an interface to a tracking block.

9.148.1 Detailed Description

This class represents an interface to a tracking block.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Abstract class for tracking interfaces. Since all its methods are virtual, this class cannot be instantiated directly, and a subclass can only be instantiated directly if all inherited pure virtual methods have been implemented by that class or a parent class.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.149 src/core/libs/gnss_sdr_supl_client.h File Reference

class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library.

```
#include <iostream>
#include <fstream>
#include <map>
#include <string>
#include <boost/archive/xml_oarchive.hpp>
#include <boost/archive/xml_iarchive.hpp>
#include <boost/serialization/map.hpp>
#include <glog/logging.h>
#include "supl.h"
#include "GPS_L1_CA.h"
#include "gps_ephemeris.h"
#include "gps_iono.h"
#include "gps_almanac.h"
#include "gps_utc_model.h"
#include "gps_acq_assist.h"
#include "gps_ref_time.h"
#include "gps_ref_location.h"
```

Classes

- class gnss_sdr_supl_client

  class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library..

9.149.1 Detailed Description

class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

TODO: put here supl.c author info class that implements a C++ interface to external Secure User Location Protocol (SUPL) client library.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.150 src/core/libs/ini.h File Reference

This function parses an INI file into easy-to-access name/value pairs.

Macros

- #define **INI_ALLOW_MULTILINE** 1

Functions

- int **ini_parse** (const char *filename, int(*handler)(void *user, const char *section, const char *name, const char *value), void *user)

9.150.1 Detailed Description

This function parses an INI file into easy-to-access name/value pairs.

Author

Brush Technologies, 2009.

inih (INI Not Invented Here) is a simple .INI file parser written in C++. It's only a couple of pages of code, and it was designed to be small and simple, so it's good for embedded systems. To use it, just give ini_parse() an INI file, and it will call a callback for every name=value pair parsed, giving you strings for the section, name, and value. It's done this way because it works well on low-memory embedded systems, but also because it makes for a K↩ ISS implementation. Parse given INI-style file. May have [section]s, name=value pairs (whitespace stripped), and comments starting with ';' (semicolon). Section is "" if name=value pair parsed before any section heading. For each name=value pair parsed, call handler function with given user pointer as well as section, name, and value (data only valid for duration of handler call). Handler should return nonzero on success, zero on error. Returns 0 on success, line number of first error on parse error, or -1 on file open error

inih and INIReader are released under the New BSD license:

Copyright (c) 2009, Brush Technology All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Brush Technology nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY BRUSH TECHNOLOGY "AS IS" AND ANY EXPRESS OR IMPLIED W↩ ARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL BRUSH TECHNOLOGY BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMA↩ GES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEO↩ RY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Go to the project home page for more info:

http://code.google.com/p/inih/

9.151 src/core/libs/INIReader.h File Reference

This class reads an INI file into easy-to-access name/value pairs.

```
#include <map>
#include <string>
```

Classes

- class INIReader

  Read an INI file into easy-to-access name/value pairs. (Note that I've gone for simplicity here rather than speed, but it should be pretty decent.)

9.151.1 Detailed Description

This class reads an INI file into easy-to-access name/value pairs.

Author

Brush Technologies, 2009.

inih (INI Not Invented Here) is a simple .INI file parser written in C++. It's only a couple of pages of code, and it was designed to be small and simple, so it's good for embedded systems. To use it, just give ini_parse() an INI file, and it will call a callback for every name=value pair parsed, giving you strings for the section, name, and value. It's done this way because it works well on low-memory embedded systems, but also because it makes for a KISS implementation.

inih and INIReader are released under the New BSD license:

Copyright (c) 2009, Brush Technology All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Brush Technology nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY BRUSH TECHNOLOGY "AS IS" AND ANY EXPRESS OR IMPLIED W↩ ARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL BRUSH TECHNOLOGY BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMA↩ GES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEO↩ RY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Go to the project home page for more info:

http://code.google.com/p/inih/

9.152 src/core/libs/string_converter.h File Reference

Interface of a class that interprets the contents of a string and converts it into different types.

```
#include <string>
```

Classes

- class StringConverter

  Class that interprets the contents of a string and converts it into different types.

9.152.1 Detailed Description

Interface of a class that interprets the contents of a string and converts it into different types.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.153 src/core/receiver/concurrent_map.h File Reference

Interface of a thread-safe std::map.

```
#include <map>
#include <utility>
#include <boost/thread/mutex.hpp>
```

Classes

- class concurrent_map< Data >

  This class implements a thread-safe std::map.

9.153.1 Detailed Description

Interface of a thread-safe std::map.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.154 src/core/receiver/concurrent_queue.h File Reference

Interface of a thread-safe std::queue.

```
#include <queue>
#include <boost/thread.hpp>
```

Classes

- class concurrent_queue< Data >

  This class implements a thread-safe std::queue.

9.154.1 Detailed Description

Interface of a thread-safe std::queue.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.155 src/core/receiver/control_message.h File Reference

Interface for the different control messages.

Classes

- class ControlMessage

  This class defines the different Control Messages.

9.155.1 Detailed Description

Interface for the different control messages.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.156 src/core/receiver/control_message_factory.h File Reference

Interface of a factory for control messages.

```
#include <memory>
#include <vector>
#include <gnuradio/message.h>
```

Classes

- struct ControlMessage_

  Message described by who sent it and what it says.
- class ControlMessageFactory

  This class implements a factory for Control Messages.

Typedefs

- typedef struct ControlMessage_ ControlMessage

  Message described by who sent it and what it says.

9.156.1 Detailed Description

Interface of a factory for control messages.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.156.2 Typedef Documentation

9.156.2.1 typedef struct ControlMessage_ ControlMessage

Message described by who sent it and what it says.

9.157 src/core/receiver/control_thread.h File Reference

Interface of the receiver control plane.

```
#include <memory>
#include <vector>
#include <boost/thread.hpp>
#include <gnuradio/msg_queue.h>
#include "control_message_factory.h"
#include "gnss_sdr_supl_client.h"
```

Classes

- class ControlThread

  This class represents the main thread of the application, so the name is ControlThread. This is the GNSS Receiver Control Plane: it connects the flowgraph, starts running it, and while it does not stop, reads the control messages generated by the blocks, processes them, and applies the corresponding actions.

9.157.1 Detailed Description

Interface of the receiver control plane.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

GNSS Receiver Control Plane: connects the flowgraph, starts running it, and while it does not stop, reads the control messages generated by the blocks, processes them, and applies the corresponding actions.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.158 src/core/receiver/file_configuration.h File Reference

A ConfigurationInterface that reads the configuration from a file.

```
#include "configuration_interface.h"
#include <memory>
#include <string>
```

Classes

- class FileConfiguration

  This class is an implementation of the interface ConfigurationInterface.

9.158.1 Detailed Description

A ConfigurationInterface that reads the configuration from a file.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

This implementation has a text file as the source for the values of the parameters. The file is in the INI format, containing sections and pairs of names and values. For more information about the INI format, see http://en.wikipedia.org/wiki/INI_file

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.159 src/core/receiver/gnss_block_factory.h File Reference

Interface of a factory that returns smart pointers to GNSS blocks.

```
#include <memory>
#include <string>
#include <vector>
#include <gnuradio/msg_queue.h>
```

Classes

- class GNSSBlockFactory

  Class that produces all kinds of GNSS blocks.

9.159.1 Detailed Description

Interface of a factory that returns smart pointers to GNSS blocks.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Luis Esteve, 2011. luis(at)epsilon-formacion.com Javier Arribas, 2011. jarribas(at)cttc.es Carles Fernandez-Prades, 2014. cfernandez(at)cttc.es

This class encapsulates the complexity behind the instantiation of GNSS blocks.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.160 src/core/receiver/gnss_flowgraph.h File Reference

Interface of a GNSS receiver flowgraph.

```
#include <list>
#include <memory>
#include <queue>
#include <string>
#include <vector>
#include <gnuradio/top_block.h>
#include <gnuradio/msg_queue.h>
#include "GPS_L1_CA.h"
#include "gnss_signal.h"
```

Classes

- class GNSSFlowgraph

  This class represents a GNSS flowgraph.

9.160.1 Detailed Description

Interface of a GNSS receiver flowgraph.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com Luis Esteve, 2011. luis(at)epsilon-formacion.com Carles Fernandez-Prades, 2014. cfernandez(at)cttc.es

It contains a signal source, a signal conditioner, a set of channels, a pvt and an output filter.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.161 src/core/receiver/in_memory_configuration.h File Reference

A ConfigurationInterface for testing purposes.

```
#include <map>
#include <memory>
#include <string>
#include "configuration_interface.h"
```

Classes

- class InMemoryConfiguration

  This class is an implementation of the interface ConfigurationInterface.

9.161.1 Detailed Description

A ConfigurationInterface for testing purposes.

Author

Carlos Aviles, 2010. carlos.avilesr(at)googlemail.com

This implementation accepts configuration parameters upon instantiation and it is intended to be used in unit testing.

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.162 src/core/system_parameters/galileo_almanac.h File Reference

Interface of a Galileo ALMANAC storage.

Classes

- class Galileo_Almanac

  This class is a storage for the GALILEO ALMANAC data as described in GALILEO ICD.

9.162.1 Detailed Description

Interface of a Galileo ALMANAC storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Mara Branzanti 2013. mara.branzanti(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.163 src/core/system_parameters/Galileo_E1.h File Reference

Defines system parameters for Galileo E1 signal and NAV data.

```
#include <string>
#include <vector>
#include <utility>
#include "MATH_CONSTANTS.h"
```

Macros

- #define **GALILEO_INAV_PREAMBLE** {0, 1, 0, 1, 1, 0, 0, 0, 0, 0}

Functions

- const std::vector< std::pair< int, int > > **type** ({{1, 6}})
- const std::vector< std::pair< int, int > > **PAGE_TYPE_bit** ({{1, 6}})
- const std::vector< std::pair< int, int > > **IOD_nav_1_bit** ({{7, 10}})
- const std::vector< std::pair< int, int > > **T0E_1_bit** ({{17, 14}})
- const std::vector< std::pair< int, int > > **M0_1_bit** ({{31, 32}})
- const std::vector< std::pair< int, int > > **e_1_bit** ({{63, 32}})
- const std::vector< std::pair< int, int > > **A_1_bit** ({{95, 32}})
- const std::vector< std::pair< int, int > > **IOD_nav_2_bit** ({{7, 10}})
- const std::vector< std::pair< int, int > > **OMEGA_0_2_bit** ({{17, 32}})
- const std::vector< std::pair< int, int > > **i_0_2_bit** ({{49, 32}})
- const std::vector< std::pair< int, int > > **omega_2_bit** ({{81, 32}})
- const std::vector< std::pair< int, int > > **iDot_2_bit** ({{113, 14}})
- const std::vector< std::pair< int, int > > **IOD_nav_3_bit** ({{7, 10}})
- const std::vector< std::pair< int, int > > **OMEGA_dot_3_bit** ({{17, 24}})
- const std::vector< std::pair< int, int > > **delta_n_3_bit** ({{41, 16}})
- const std::vector< std::pair< int, int > > **C_uc_3_bit** ({{57, 16}})
- const std::vector< std::pair< int, int > > **C_us_3_bit** ({{73, 16}})
- const std::vector< std::pair< int, int > > **C_rc_3_bit** ({{89, 16}})

- const std::vector< std::pair< int, int > > **C_rs_3_bit** ({{105, 16}})
- const std::vector< std::pair< int, int > > **SISA_3_bit** ({{121, 8}})
- const std::vector< std::pair< int, int > > **IOD_nav_4_bit** ({{7, 10}})
- const std::vector< std::pair< int, int > > **SV_ID_PRN_4_bit** ({{17, 6}})
- const std::vector< std::pair< int, int > > **C_ic_4_bit** ({{23, 16}})
- const std::vector< std::pair< int, int > > **C_is_4_bit** ({{39, 16}})
- const std::vector< std::pair< int, int > > **t0c_4_bit** ({{55, 14}})
- const std::vector< std::pair< int, int > > **af0_4_bit** ({{69, 31}})
- const std::vector< std::pair< int, int > > **af1_4_bit** ({{100, 21}})
- const std::vector< std::pair< int, int > > **af2_4_bit** ({{121, 6}})
- const std::vector< std::pair< int, int > > **spare_4_bit** ({{127, 2}})
- const std::vector< std::pair< int, int > > **ai0_5_bit** ({{7, 11}})
- const std::vector< std::pair< int, int > > **ai1_5_bit** ({{18, 11}})
- const std::vector< std::pair< int, int > > **ai2_5_bit** ({{29, 14}})
- const std::vector< std::pair< int, int > > **Region1_5_bit** ({{43, 1}})
- const std::vector< std::pair< int, int > > **Region2_5_bit** ({{44, 1}})
- const std::vector< std::pair< int, int > > **Region3_5_bit** ({{45, 1}})
- const std::vector< std::pair< int, int > > **Region4_5_bit** ({{46, 1}})
- const std::vector< std::pair< int, int > > **Region5_5_bit** ({{47, 1}})
- const std::vector< std::pair< int, int > > **BGD_E1E5a_5_bit** ({{48, 10}})
- const std::vector< std::pair< int, int > > **BGD_E1E5b_5_bit** ({{58, 10}})
- const std::vector< std::pair< int, int > > **E5b_HS_5_bit** ({{68, 2}})
- const std::vector< std::pair< int, int > > **E1B_HS_5_bit** ({{70, 2}})
- const std::vector< std::pair< int, int > > **E5b_DVS_5_bit** ({{72, 1}})
- const std::vector< std::pair< int, int > > **E1B_DVS_5_bit** ({{73, 1}})
- const std::vector< std::pair< int, int > > **WN_5_bit** ({{74, 12}})
- const std::vector< std::pair< int, int > > **TOW_5_bit** ({{86, 20}})
- const std::vector< std::pair< int, int > > **spare_5_bit** ({{106, 23}})
- const std::vector< std::pair< int, int > > **A0_6_bit** ({{7, 32}})
- const std::vector< std::pair< int, int > > **A1_6_bit** ({{39, 24}})
- const std::vector< std::pair< int, int > > **Delta_tLS_6_bit** ({{63, 8}})
- const std::vector< std::pair< int, int > > **t0t_6_bit** ({{71, 8}})
- const std::vector< std::pair< int, int > > **WNot_6_bit** ({{79, 8}})
- const std::vector< std::pair< int, int > > **WN_LSF_6_bit** ({{87, 8}})
- const std::vector< std::pair< int, int > > **DN_6_bit** ({{95, 3}})
- const std::vector< std::pair< int, int > > **Delta_tLSF_6_bit** ({{98, 8}})
- const std::vector< std::pair< int, int > > **TOW_6_bit** ({{106, 20}})
- const std::vector< std::pair< int, int > > **IOD_a_7_bit** ({{7, 4}})
- const std::vector< std::pair< int, int > > **WN_a_7_bit** ({{11, 2}})
- const std::vector< std::pair< int, int > > **t0a_7_bit** ({{13, 10}})
- const std::vector< std::pair< int, int > > **SVID1_7_bit** ({{23, 6}})
- const std::vector< std::pair< int, int > > **DELTA_A_7_bit** ({{29, 13}})
- const std::vector< std::pair< int, int > > **e_7_bit** ({{42, 11}})
- const std::vector< std::pair< int, int > > **omega_7_bit** ({{53, 16}})
- const std::vector< std::pair< int, int > > **delta_i_7_bit** ({{69, 11}})
- const std::vector< std::pair< int, int > > **Omega0_7_bit** ({{80, 16}})
- const std::vector< std::pair< int, int > > **Omega_dot_7_bit** ({{96, 11}})
- const std::vector< std::pair< int, int > > **M0_7_bit** ({{107, 16}})
- const std::vector< std::pair< int, int > > **IOD_a_8_bit** ({{7, 4}})
- const std::vector< std::pair< int, int > > **af0_8_bit** ({{11, 16}})
- const std::vector< std::pair< int, int > > **af1_8_bit** ({{27, 13}})
- const std::vector< std::pair< int, int > > **E5b_HS_8_bit** ({{40, 2}})
- const std::vector< std::pair< int, int > > **E1B_HS_8_bit** ({{42, 2}})
- const std::vector< std::pair< int, int > > **SVID2_8_bit** ({{44, 6}})
- const std::vector< std::pair< int, int > > **DELTA_A_8_bit** ({{50, 13}})

- const std::vector< std::pair< int, int > > **e_8_bit** ({{63, 11}})
- const std::vector< std::pair< int, int > > **omega_8_bit** ({{74, 16}})
- const std::vector< std::pair< int, int > > **delta_i_8_bit** ({{90, 11}})
- const std::vector< std::pair< int, int > > **Omega0_8_bit** ({{101, 16}})
- const std::vector< std::pair< int, int > > **Omega_dot_8_bit** ({{117, 11}})
- const std::vector< std::pair< int, int > > **IOD_a_9_bit** ({{7, 4}})
- const std::vector< std::pair< int, int > > **WN_a_9_bit** ({{11, 2}})
- const std::vector< std::pair< int, int > > **t0a_9_bit** ({{13, 10}})
- const std::vector< std::pair< int, int > > **M0_9_bit** ({{23, 16}})
- const std::vector< std::pair< int, int > > **af0_9_bit** ({{39, 16}})
- const std::vector< std::pair< int, int > > **af1_9_bit** ({{55, 13}})
- const std::vector< std::pair< int, int > > **E5b_HS_9_bit** ({{68, 2}})
- const std::vector< std::pair< int, int > > **E1B_HS_9_bit** ({{70, 2}})
- const std::vector< std::pair< int, int > > **SVID3_9_bit** ({{72, 6}})
- const std::vector< std::pair< int, int > > **DELTA_A_9_bit** ({{78, 13}})
- const std::vector< std::pair< int, int > > **e_9_bit** ({{91, 11}})
- const std::vector< std::pair< int, int > > **omega_9_bit** ({{102, 16}})
- const std::vector< std::pair< int, int > > **delta_i_9_bit** ({{118, 11}})
- const std::vector< std::pair< int, int > > **IOD_a_10_bit** ({{7, 4}})
- const std::vector< std::pair< int, int > > **Omega0_10_bit** ({{11, 16}})
- const std::vector< std::pair< int, int > > **Omega_dot_10_bit** ({{27, 11}})
- const std::vector< std::pair< int, int > > **M0_10_bit** ({{38, 16}})
- const std::vector< std::pair< int, int > > **af0_10_bit** ({{54, 16}})
- const std::vector< std::pair< int, int > > **af1_10_bit** ({{70, 13}})
- const std::vector< std::pair< int, int > > **E5b_HS_10_bit** ({{83, 2}})
- const std::vector< std::pair< int, int > > **E1B_HS_10_bit** ({{85, 2}})
- const std::vector< std::pair< int, int > > **A_0G_10_bit** ({{87, 16}})
- const std::vector< std::pair< int, int > > **A_1G_10_bit** ({{103, 12}})
- const std::vector< std::pair< int, int > > **t_0G_10_bit** ({{115, 8}})
- const std::vector< std::pair< int, int > > **WN_0G_10_bit** ({{123, 6}})
- const std::vector< std::pair< int, int > > **Time_0_bit** ({{7, 2}})
- const std::vector< std::pair< int, int > > **WN_0_bit** ({{97, 12}})
- const std::vector< std::pair< int, int > > **TOW_0_bit** ({{109, 20}})

Variables

- const double GALILEO_PI = 3.1415926535898

  Pi as defined in GALILEO ICD.
- const double GALILEO_TWO_PI = 6.283185307179600

  2∗Pi as defined in GALILEO ICD
- const double GALILEO_GM = 3.986004418e14

  Geocentric gravitational constant[m^3/s^2].
- const double GALILEO_OMEGA_EARTH_DOT = 7.2921151467e-5

  Mean angular velocity of the Earth [rad/s].
- const double GALILEO_C_m_s = 299792458.0

  The speed of light, [m/s].
- const double GALILEO_C_m_ms = 299792.4580

  The speed of light, [m/ms].
- const double GALILEO_F = -4.442807309e-10

  Constant, [s/(m)^(1/2)].
- const double Galileo_E1_FREQ_HZ = 1.57542e9

  Galileo E1 carrier frequency [Hz].

- const double Galileo_E1_CODE_CHIP_RATE_HZ = 1.023e6

  Galileo E1 code rate [chips/s].
- const double Galileo_E1_CODE_PERIOD = 0.004

  Galileo E1 code period [s].
- const double Galileo_E1_SUB_CARRIER_A_RATE_HZ = 1.023e6

  Galileo E1 sub-carrier 'a' rate [Hz].
- const double Galileo_E1_SUB_CARRIER_B_RATE_HZ = 6.138e6

  Galileo E1 sub-carrier 'b' rate [Hz].
- const double Galileo_E1_B_CODE_LENGTH_CHIPS = 4092.0

  Galileo E1-B code length [chips].
- const double Galileo_E1_B_SYMBOL_RATE_BPS = 250.0

  Galileo E1-B symbol rate [bits/second].
- const double Galileo_E1_C_SECONDARY_CODE_LENGTH = 25.0

  Galileo E1-C secondary code length [chips].
- const int **Galileo_E1_NUMBER_OF_CODES** = 50
- const double **GALILEO_STARTOFFSET_ms** = 68.802
- const int **GALILEO_E1_HISTORY_DEEP** =100
- const int **GALILEO_INAV_PREAMBLE_LENGTH_BITS** = 10
- const int **GALILEO_INAV_PREAMBLE_PERIOD_SYMBOLS** = 250
- const int GALILEO_INAV_PAGE_PART_SYMBOLS = 250

  Each Galileo INAV pages are composed of two parts (even and odd) each of 250 symbols, including preamble. See Galileo ICD 4.3.2.
- const int GALILEO_INAV_PAGE_SYMBOLS = 500

  The complete Galileo INAV page length.
- const int **GALILEO_INAV_PAGE_PART_SECONDS** = 1
- const int **GALILEO_INAV_PAGE_SECONDS** = 2
- const int **GALILEO_INAV_INTERLEAVER_ROWS** = 8
- const int **GALILEO_INAV_INTERLEAVER_COLS** = 30
- const int **GALILEO_TELEMETRY_RATE_BITS_SECOND** = 250
- const int **GALILEO_PAGE_TYPE_BITS** = 6
- const int **GALILEO_DATA_JK_BITS** = 128
- const int **GALILEO_DATA_FRAME_BITS** = 196
- const int **GALILEO_DATA_FRAME_BYTES** = 25
- const double **GALILEO_E1_CODE_PERIOD** = 0.004
- const double **t0e_1_LSB** = 60
- const double **M0_1_LSB** = PI_TWO_N31
- const double **e_1_LSB** = TWO_N33
- const double **A_1_LSB_gal** = TWO_N19
- const double **OMEGA_0_2_LSB** = PI_TWO_N31
- const double **i_0_2_LSB** = PI_TWO_N31
- const double **omega_2_LSB** = PI_TWO_N31
- const double **iDot_2_LSB** = PI_TWO_N43
- const double **OMEGA_dot_3_LSB** = PI_TWO_N43
- const double **delta_n_3_LSB** = PI_TWO_N43
- const double **C_uc_3_LSB** = TWO_N29
- const double **C_us_3_LSB** = TWO_N29
- const double **C_rc_3_LSB** = TWO_N5
- const double **C_rs_3_LSB** = TWO_N5
- const double **C_ic_4_LSB** = TWO_N29
- const double **C_is_4_LSB** = TWO_N29
- const double **t0c_4_LSB** = 60
- const double **af0_4_LSB** = TWO_N34
- const double **af1_4_LSB** = TWO_N46

- const double **af2_4_LSB** = TWO_N59
- const double **ai0_5_LSB** = TWO_N2
- const double **ai1_5_LSB** = TWO_N8
- const double **ai2_5_LSB** = TWO_N15
- const double **BGD_E1E5a_5_LSB** = TWO_N32
- const double **BGD_E1E5b_5_LSB** = TWO_N32
- const double **A0_6_LSB** = TWO_N30
- const double **A1_6_LSB** = TWO_N50
- const double **t0t_6_LSB** = 3600
- const double **t0a_7_LSB** = 600
- const double **DELTA_A_7_LSB** = TWO_N9
- const double **e_7_LSB** = TWO_N16
- const double **omega_7_LSB** = TWO_N15
- const double **delta_i_7_LSB** = TWO_N14
- const double **Omega0_7_LSB** = TWO_N15
- const double **Omega_dot_7_LSB** = TWO_N33
- const double **M0_7_LSB** = TWO_N15
- const double **af0_8_LSB** = TWO_N19
- const double **af1_8_LSB** = TWO_N38
- const double **DELTA_A_8_LSB** = TWO_N9
- const double **e_8_LSB** = TWO_N16
- const double **omega_8_LSB** = TWO_N15
- const double **delta_i_8_LSB** = TWO_N14
- const double **Omega0_8_LSB** = TWO_N15
- const double **Omega_dot_8_LSB** = TWO_N33
- const double **t0a_9_LSB** = 600
- const double **M0_9_LSB** = TWO_N15
- const double **af0_9_LSB** = TWO_N19
- const double **af1_9_LSB** = TWO_N38
- const double **DELTA_A_9_LSB** = TWO_N9
- const double **e_9_LSB** = TWO_N16
- const double **omega_9_LSB** = TWO_N15
- const double **delta_i_9_LSB** = TWO_N14
- const double **Omega0_10_LSB** = TWO_N15
- const double **Omega_dot_10_LSB** = TWO_N33
- const double **M0_10_LSB** = TWO_N15
- const double **af0_10_LSB** = TWO_N19
- const double **af1_10_LSB** = TWO_N38
- const double **A_0G_10_LSB** = TWO_N35
- const double **A_1G_10_LSB** = TWO_N51
- const double **t_0G_10_LSB** = 3600
- const std::string **Galileo_E1_B_PRIMARY_CODE** [Galileo_E1_NUMBER_OF_CODES]
- const std::string **Galileo_E1_C_PRIMARY_CODE** [Galileo_E1_NUMBER_OF_CODES]
- const std::string **Galileo_E1_C_SECONDARY_CODE** = "00111000000010101101100 10"

9.163.1 Detailed Description

Defines system parameters for Galileo E1 signal and NAV data.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com
Mara Branzanti 2013. mara.branzanti(at)gmail.com
Javier Arribas 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.163.2 Variable Documentation

9.163.2.1 const double GALILEO_C_m_ms = 299792.4580

The speed of light, [m/ms].

Definition at line 47 of file Galileo_E1.h.

9.163.2.2 const double GALILEO_C_m_s = 299792458.0

The speed of light, [m/s].

Definition at line 46 of file Galileo_E1.h.

9.163.2.3 const double Galileo_E1_B_CODE_LENGTH_CHIPS = 4092.0

Galileo E1-B code length [chips].

Definition at line 56 of file Galileo_E1.h.

9.163.2.4 const double Galileo_E1_B_SYMBOL_RATE_BPS = 250.0

Galileo E1-B symbol rate [bits/second].

Definition at line 57 of file Galileo_E1.h.

9.163.2.5 const double Galileo_E1_C_SECONDARY_CODE_LENGTH = 25.0

Galileo E1-C secondary code length [chips].

Definition at line 58 of file Galileo_E1.h.

9.163.2.6 const double Galileo_E1_CODE_CHIP_RATE_HZ = 1.023e6

Galileo E1 code rate [chips/s].

Definition at line 52 of file Galileo_E1.h.

9.163.2.7 const double Galileo_E1_CODE_PERIOD = 0.004

Galileo E1 code period [s].

Definition at line 53 of file Galileo_E1.h.

9.163.2.8 const double Galileo_E1_FREQ_HZ = 1.57542e9

Galileo E1 carrier frequency [Hz].

Definition at line 51 of file Galileo_E1.h.

9.163.2.9 const double Galileo_E1_SUB_CARRIER_A_RATE_HZ = 1.023e6

Galileo E1 sub-carrier 'a' rate [Hz].

Definition at line 54 of file Galileo_E1.h.

9.163.2.10 const double Galileo_E1_SUB_CARRIER_B_RATE_HZ = 6.138e6

Galileo E1 sub-carrier 'b' rate [Hz].

Definition at line 55 of file Galileo_E1.h.

9.163.2.11 const double GALILEO_F = -4.442807309e-10

Constant, [s/(m)^(1/2)].

Definition at line 48 of file Galileo_E1.h.

9.163.2.12 const double GALILEO_GM = 3.986004418e14

Geocentric gravitational constant[m^3/s^2].

Definition at line 44 of file Galileo_E1.h.

9.163.2.13 const int GALILEO_INAV_PAGE_PART_SYMBOLS = 250

Each Galileo INAV pages are composed of two parts (even and odd) each of 250 symbols, including preamble. See Galileo ICD 4.3.2.

Definition at line 73 of file Galileo_E1.h.

9.163.2.14 const int GALILEO_INAV_PAGE_SYMBOLS = 500

The complete Galileo INAV page length.

Definition at line 74 of file Galileo_E1.h.

9.163.2.15 const double GALILEO_OMEGA_EARTH_DOT = 7.2921151467e-5

Mean angular velocity of the Earth [rad/s].

Definition at line 45 of file Galileo_E1.h.

9.163.2.16 const double GALILEO_PI = 3.1415926535898

Pi as defined in GALILEO ICD.

Definition at line 42 of file Galileo_E1.h.

9.163.2.17 const double GALILEO_TWO_PI = 6.283185307179600

2∗Pi as defined in GALILEO ICD

Definition at line 43 of file Galileo_E1.h.

9.164 src/core/system_parameters/galileo_ephemeris.h File Reference

Interface of a Galileo EPHEMERIS storage.

```
#include <boost/assign.hpp>
#include <boost/serialization/nvp.hpp>
```

Classes

- class Galileo_Ephemeris

  This class is a storage and orbital model functions for the Galileo SV ephemeris data as described in Galileo ICD paragraph 5.1.1 (See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo↩_OS_SIS_ICD.pdf *)*

9.164.1 Detailed Description

Interface of a Galileo EPHEMERIS storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es,
Mara Branzanti 2013. mara.branzanti(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.165 src/core/system_parameters/galileo_fnav_message.h File Reference

Implementation of a Galileo F/NAV Data message as described in Galileo OS SIS ICD Issue 1.2 (Nov. 2015)

```
#include <bitset>
#include <string>
#include <vector>
#include <utility>
#include <boost/cstdint.hpp>
#include "galileo_ephemeris.h"
#include "galileo_iono.h"
#include "galileo_almanac.h"
#include "galileo_utc_model.h"
#include "Galileo_E5a.h"
```

Classes

- class Galileo_Fnav_Message

  This class handles the Galileo F/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf.

9.165.1 Detailed Description

Implementation of a Galileo F/NAV Data message as described in Galileo OS SIS ICD Issue 1.2 (Nov. 2015)

Author

Marc Sales, 2014. marcsales92(at)gmail.com on work from:

- Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.166 src/core/system_parameters/galileo_iono.h File Reference

Interface of a Galileo Ionospheric Model storage.

Classes

- class Galileo_Iono

  This class is a storage for the GALILEO IONOSPHERIC data as described in Galileo ICD paragraph 5.1.6.

9.166.1 Detailed Description

Interface of a Galileo Ionospheric Model storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Mara Branzanti 2013. mara.branzanti(at)gmail.com

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.167 src/core/system_parameters/galileo_navigation_message.h File Reference

Implementation of a Galileo I/NAV Data message as described in Galileo OS SIS ICD Issue 1.2 (Nov. 2015)

```
#include <bitset>
#include <map>
#include <string>
#include <utility>
#include <vector>
#include <boost/cstdint.hpp>
#include "galileo_ephemeris.h"
#include "galileo_iono.h"
#include "galileo_almanac.h"
#include "galileo_utc_model.h"
#include "Galileo_E1.h"
```

Classes

- class Galileo_Navigation_Message

  This class handles the Galileo I/NAV Data message, as described in the Galileo Open Service Signal in Space Interface Control Document (OS SIS ICD), Issue 1.2 (Nov 2015). See https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf.

9.167.1 Detailed Description

Implementation of a Galileo I/NAV Data message as described in Galileo OS SIS ICD Issue 1.2 (Nov. 2015)

Author

Mara Branzanti 2013. mara.branzanti(at)gmail.com
Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.168 src/core/system_parameters/galileo_utc_model.h File Reference

Interface of a Galileo UTC MODEL storage.

Classes

- class Galileo_Utc_Model

  This class is a storage for the GALILEO UTC MODEL data as described in Galileo ICD https://www.gsc-europa.eu/system/files/galileo_documents/Galileo_OS_SIS_ICD.pdf paragraph 5.1.7.

9.168.1 Detailed Description

Interface of a Galileo UTC MODEL storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.169 src/core/system_parameters/gnss_satellite.h File Reference

Interface of the Gnss_Satellite class.

```
#include <string>
#include <set>
#include <iostream>
#include <map>
```

Classes

- class Gnss_Satellite

  This class represents a GNSS satellite.

9.169.1 Detailed Description

Interface of the Gnss_Satellite class.

Author

Carles Fernandez-Prades, 2012. cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.170 src/core/system_parameters/gnss_signal.h File Reference

Implementation of the Gnss_Signal class.

```
#include "gnss_satellite.h"
#include <ostream>
#include <string>
```

Classes

- class Gnss_Signal

  This class represents a GNSS signal.

9.170.1 Detailed Description

Implementation of the Gnss_Signal class.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Javier Arribas, 2012. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.171 src/core/system_parameters/gnss_synchro.h File Reference

Interface of the Gnss_Synchro class.

```
#include "gnss_signal.h"
```

Classes

- class Gnss_Synchro

  This is the class that contains the information that is shared by the processing blocks.

9.171.1 Detailed Description

Interface of the Gnss_Synchro class.

Author

Luis Esteve, 2012. luis(at)epsilon-formacion.com

Javier Arribas, 2012. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.172 src/core/system_parameters/gps_acq_assist.h File Reference

Interface of a GPS RRLL ACQUISITION ASSISTACE storage.

Classes

- class Gps_Acq_Assist

  This class is a storage for the GPS GSM RRLL acquisition assistance data as described in Digital cellular telecommunications system (Phase 2+); Location Services (LCS); Mobile Station (MS) - Serving Mobile Location Centre (SMLC) Radio Resource LCS Protocol (RRLP) (3GPP TS 44.031 version 5.12.0 Release 5)

9.172.1 Detailed Description

Interface of a GPS RRLL ACQUISITION ASSISTACE storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.173 src/core/system_parameters/gps_almanac.h File Reference

Interface of a GPS ALMANAC storage.

Classes

- class Gps_Almanac

  This class is a storage for the GPS SV ALMANAC data as described in IS-GPS-200E.

9.173.1 Detailed Description

Interface of a GPS ALMANAC storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.174 src/core/system_parameters/gps_cnav_ephemeris.h File Reference

Interface of a GPS CNAV EPHEMERIS storage.

```
#include "boost/assign.hpp"
#include <boost/serialization/nvp.hpp>
```

Classes

- class Gps_CNAV_Ephemeris

  This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200H.

9.174.1 Detailed Description

Interface of a GPS CNAV EPHEMERIS storage.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.175 src/core/system_parameters/gps_cnav_iono.h File Reference

Interface of a GPS CNAV IONOSPHERIC MODEL storage.

```
#include "boost/assign.hpp"
#include <boost/serialization/nvp.hpp>
```

Classes

- class Gps_CNAV_Iono

  This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200H.

9.175.1 Detailed Description

Interface of a GPS CNAV IONOSPHERIC MODEL storage.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.176 src/core/system_parameters/gps_cnav_navigation_message.h File Reference

Interface of a GPS CNAV Data message decoder.

```
#include <bitset>
#include <map>
#include <string>
#include <vector>
#include <utility>
#include "GPS_L2C.h"
#include "gps_cnav_ephemeris.h"
#include "gps_cnav_iono.h"
#include "gps_cnav_utc_model.h"
```

Classes

- class Gps_CNAV_Navigation_Message

  This class decodes a GPS CNAV Data message as described in IS-GPS-200H.

9.176.1 Detailed Description

Interface of a GPS CNAV Data message decoder.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.177 src/core/system_parameters/gps_ephemeris.h File Reference

Interface of a GPS EPHEMERIS storage.

```
#include <map>
#include <string>
#include "boost/assign.hpp"
#include <boost/serialization/nvp.hpp>
```

Classes

- class Gps_Ephemeris

  This class is a storage and orbital model functions for the GPS SV ephemeris data as described in IS-GPS-200E.

9.177.1 Detailed Description

Interface of a GPS EPHEMERIS storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.178 src/core/system_parameters/gps_iono.h File Reference

Interface of a GPS IONOSPHERIC MODEL storage.

```
#include "boost/assign.hpp"
#include <boost/serialization/nvp.hpp>
```

Classes

- class Gps_Iono

  This class is a storage for the GPS IONOSPHERIC data as described in IS-GPS-200E.

9.178.1 Detailed Description

Interface of a GPS IONOSPHERIC MODEL storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.179 src/core/system_parameters/GPS_L1_CA.h File Reference

Defines system parameters for GPS L1 C/A signal and NAV data.

```
#include <vector>
#include <utility>
#include "MATH_CONSTANTS.h"
```

Macros

- #define **GPS_PREAMBLE** {1, 0, 0, 0, 1, 0, 1, 1}

Functions

- const std::vector< std::pair< int, int > > **TOW** ({{31, 17}})
- const std::vector< std::pair< int, int > > **INTEGRITY_STATUS_FLAG** ({{23, 1}})
- const std::vector< std::pair< int, int > > **ALERT_FLAG** ({{48, 1}})
- const std::vector< std::pair< int, int > > **ANTI_SPOOFING_FLAG** ({{49, 1}})
- const std::vector< std::pair< int, int > > **SUBFRAME_ID** ({{50, 3}})
- const std::vector< std::pair< int, int > > **GPS_WEEK** ({{61, 10}})
- const std::vector< std::pair< int, int > > **CA_OR_P_ON_L2** ({{71, 2}})
- const std::vector< std::pair< int, int > > **SV_ACCURACY** ({{73, 4}})
- const std::vector< std::pair< int, int > > **SV_HEALTH** ({{77, 6}})
- const std::vector< std::pair< int, int > > **L2_P_DATA_FLAG** ({{91, 1}})
- const std::vector< std::pair< int, int > > **T_GD** ({{197, 8}})

- const std::vector< std::pair< int, int > > **IODC** ({{83, 2},{211, 8}})
- const std::vector< std::pair< int, int > > **T_OC** ({{219, 16}})
- const std::vector< std::pair< int, int > > **A_F2** ({{241, 8}})
- const std::vector< std::pair< int, int > > **A_F1** ({{249, 16}})
- const std::vector< std::pair< int, int > > **A_F0** ({{271, 22}})
- const std::vector< std::pair< int, int > > **IODE_SF2** ({{61, 8}})
- const std::vector< std::pair< int, int > > **C_RS** ({{69, 16}})
- const std::vector< std::pair< int, int > > **DELTA_N** ({{91, 16}})
- const std::vector< std::pair< int, int > > **M_0** ({{107, 8},{121, 24}})
- const std::vector< std::pair< int, int > > **C_UC** ({{151, 16}})
- const std::vector< std::pair< int, int > > **E** ({{167, 8},{181, 24}})
- const std::vector< std::pair< int, int > > **C_US** ({{211, 16}})
- const std::vector< std::pair< int, int > > **SQRT_A** ({{227, 8},{241, 24}})
- const std::vector< std::pair< int, int > > **T_OE** ({{271, 16}})
- const std::vector< std::pair< int, int > > **FIT_INTERVAL_FLAG** ({{271, 1}})
- const std::vector< std::pair< int, int > > **AODO** ({{272, 5}})
- const std::vector< std::pair< int, int > > **C_IC** ({{61, 16}})
- const std::vector< std::pair< int, int > > **OMEGA_0** ({{77, 8},{91, 24}})
- const std::vector< std::pair< int, int > > **C_IS** ({{121, 16}})
- const std::vector< std::pair< int, int > > **I_0** ({{137, 8},{151, 24}})
- const std::vector< std::pair< int, int > > **C_RC** ({{181, 16}})
- const std::vector< std::pair< int, int > > **OMEGA** ({{197, 8},{211, 24}})
- const std::vector< std::pair< int, int > > **OMEGA_DOT** ({{241, 24}})
- const std::vector< std::pair< int, int > > **IODE_SF3** ({{271, 8}})
- const std::vector< std::pair< int, int > > **I_DOT** ({{279, 14}})
- const std::vector< std::pair< int, int > > **SV_DATA_ID** ({{61, 2}})
- const std::vector< std::pair< int, int > > **SV_PAGE** ({{63, 6}})
- const std::vector< std::pair< int, int > > ALPHA_0 ({{69, 8}})
- const std::vector< std::pair< int, int > > **ALPHA_1** ({{77, 8}})
- const std::vector< std::pair< int, int > > **ALPHA_2** ({{91, 8}})
- const std::vector< std::pair< int, int > > **ALPHA_3** ({{99, 8}})
- const std::vector< std::pair< int, int > > **BETA_0** ({{107, 8}})
- const std::vector< std::pair< int, int > > **BETA_1** ({{121, 8}})
- const std::vector< std::pair< int, int > > **BETA_2** ({{129, 8}})
- const std::vector< std::pair< int, int > > **BETA_3** ({{137, 8}})
- const std::vector< std::pair< int, int > > **A_1** ({{151, 24}})
- const std::vector< std::pair< int, int > > **A_0** ({{181, 24},{211, 8}})
- const std::vector< std::pair< int, int > > **T_OT** ({{219, 8}})
- const std::vector< std::pair< int, int > > **WN_T** ({{227, 8}})
- const std::vector< std::pair< int, int > > **DELTAT_LS** ({{241, 8}})
- const std::vector< std::pair< int, int > > **WN_LSF** ({{249, 8}})
- const std::vector< std::pair< int, int > > **DN** ({{257, 8}})
- const std::vector< std::pair< int, int > > **DELTAT_LSF** ({{271, 8}})
- const std::vector< std::pair< int, int > > **HEALTH_SV25** ({{229, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV26** ({{241, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV27** ({{247, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV28** ({{253, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV29** ({{259, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV30** ({{271, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV31** ({{277, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV32** ({{283, 6}})
- const std::vector< std::pair< int, int > > T_OA ({{69, 8}})
- const std::vector< std::pair< int, int > > **WN_A** ({{77, 8}})
- const std::vector< std::pair< int, int > > **HEALTH_SV1** ({{91, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV2** ({{97, 6}})

- const std::vector< std::pair< int, int > > **HEALTH_SV3** ({{103, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV4** ({{109, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV5** ({{121, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV6** ({{127, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV7** ({{133, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV8** ({{139, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV9** ({{151, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV10** ({{157, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV11** ({{163, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV12** ({{169, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV13** ({{181, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV14** ({{187, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV15** ({{193, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV16** ({{199, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV17** ({{211, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV18** ({{217, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV19** ({{223, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV20** ({{229, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV21** ({{241, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV22** ({{247, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV23** ({{253, 6}})
- const std::vector< std::pair< int, int > > **HEALTH_SV24** ({{259, 6}})

Variables

- const double GPS_C_m_s = 299792458.0

  The speed of light, [m/s].
- const double GPS_C_m_ms = 299792.4580

  The speed of light, [m/ms].
- const double GPS_PI = 3.1415926535898

  Pi as defined in IS-GPS-200E.
- const double GPS_TWO_PI = 6.283185307179586

  2Pi as defined in IS-GPS-200E
- const double OMEGA_EARTH_DOT = 7.2921151467e-5

  Earth rotation rate, [rad/s].
- const double GM = 3.986005e14

  Universal gravitational constant times the mass of the Earth, [m^3/s^2].
- const double F = -4.442807633e-10

  Constant, [s/(m)^(1/2)].
- const double GPS_L1_FREQ_HZ = 1.57542e9

  L1 [Hz].
- const double GPS_L1_CA_CODE_RATE_HZ = 1.023e6

  GPS L1 C/A code rate [chips/s].
- const double GPS_L1_CA_CODE_LENGTH_CHIPS = 1023.0

  GPS L1 C/A code length [chips].
- const double GPS_L1_CA_CODE_PERIOD = 0.001

  GPS L1 C/A code period [seconds].
- const double GPS_L1_CA_CHIP_PERIOD = 9.7752e-07

  GPS L1 C/A chip period [seconds].
- const double MAX_TOA_DELAY_MS = 20

  Maximum Time-Of-Arrival (TOA) difference between satellites for a receiver operated on Earth surface is 20 ms.

- const double **GPS_STARTOFFSET_ms** = 68.802
- const int **GPS_L1_CA_HISTORY_DEEP** = 100
- const int **GPS_CA_PREAMBLE_LENGTH_BITS** = 8
- const int **GPS_CA_PREAMBLE_LENGTH_SYMBOLS** = 160
- const int GPS_CA_TELEMETRY_RATE_BITS_SECOND = 50

  NAV message bit rate [bits/s].
- const int **GPS_CA_TELEMETRY_SYMBOLS_PER_BIT** = 20
- const int GPS_CA_TELEMETRY_RATE_SYMBOLS_SECOND = GPS_CA_TELEMETRY_RATE_BITS_↩ SECOND∗GPS_CA_TELEMETRY_SYMBOLS_PER_BIT

  NAV message bit rate [symbols/s].
- const int GPS_WORD_LENGTH = 4

  CRC + GPS WORD (-2 -1 0 ... 29) Bits = 4 bytes.
- const int GPS_SUBFRAME_LENGTH = 40

  GPS_WORD_LENGTH x 10 = 40 bytes.
- const int GPS_SUBFRAME_BITS = 300

  Number of bits per subframe in the NAV message [bits].
- const int GPS_SUBFRAME_SECONDS = 6

  Subframe duration [seconds].
- const int GPS_SUBFRAME_MS = 6000

  Subframe duration [seconds].
- const int GPS_WORD_BITS = 30

  Number of bits per word in the NAV message [bits].
- const double **T_GD_LSB** = TWO_N31
- const double **T_OC_LSB** = TWO_P4
- const double **A_F2_LSB** = TWO_N55
- const double **A_F1_LSB** = TWO_N43
- const double **A_F0_LSB** = TWO_N31
- const double **C_RS_LSB** = TWO_N5
- const double **DELTA_N_LSB** = PI_TWO_N43
- const double **M_0_LSB** = PI_TWO_N31
- const double **C_UC_LSB** = TWO_N29
- const double **E_LSB** = TWO_N33
- const double **C_US_LSB** = TWO_N29
- const double **SQRT_A_LSB** = TWO_N19
- const double **T_OE_LSB** = TWO_P4
- const int **AODO_LSB** = 900
- const double **C_IC_LSB** = TWO_N29
- const double **OMEGA_0_LSB** = PI_TWO_N31
- const double **C_IS_LSB** = TWO_N29
- const double **I_0_LSB** = PI_TWO_N31
- const double **C_RC_LSB** = TWO_N5
- const double **OMEGA_LSB** = PI_TWO_N31
- const double **OMEGA_DOT_LSB** = PI_TWO_N43
- const double **I_DOT_LSB** = PI_TWO_N43
- const double **ALPHA_0_LSB** = TWO_N30
- const double **ALPHA_1_LSB** = TWO_N27
- const double **ALPHA_2_LSB** = TWO_N24
- const double **ALPHA_3_LSB** = TWO_N24
- const double **BETA_0_LSB** = TWO_P11
- const double **BETA_1_LSB** = TWO_P14
- const double **BETA_2_LSB** = TWO_P16
- const double **BETA_3_LSB** = TWO_P16
- const double **A_1_LSB** = TWO_N50

- const double **A_0_LSB** = TWO_N30
- const double **T_OT_LSB** = TWO_P12
- const double **WN_T_LSB** = 1
- const double **DELTAT_LS_LSB** = 1
- const double **WN_LSF_LSB** = 1
- const double **DN_LSB** = 1
- const double **DELTAT_LSF_LSB** = 1
- const double **T_OA_LSB** = TWO_P12

9.179.1 Detailed Description

Defines system parameters for GPS L1 C/A signal and NAV data.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.179.2 Function Documentation

9.179.2.1 const std::vector<std::pair<int,int> > ALPHA_0 ({{69, 8}})

Todo read all pages of subframe 4

9.179.2.2 const std::vector<std::pair<int,int> > T_OA ({{69, 8}})

Todo read all pages of subframe 5

9.179.3 Variable Documentation

9.179.3.1 const double F = -4.442807633e-10

Constant, [s/(m)^(1/2)].

Definition at line 46 of file GPS_L1_CA.h.

9.179.3.2 const double GM = 3.986005e14

Universal gravitational constant times the mass of the Earth, [m^3/s^2].

Definition at line 45 of file GPS_L1_CA.h.

9.179.3.3 const double GPS_C_m_ms = 299792.4580

The speed of light, [m/ms].

Definition at line 41 of file GPS_L1_CA.h.

9.179.3.4 const double GPS_C_m_s = 299792458.0

The speed of light, [m/s].

Definition at line 40 of file GPS_L1_CA.h.

9.179.3.5 const int GPS_CA_TELEMETRY_RATE_BITS_SECOND = 50

NAV message bit rate [bits/s].

Definition at line 78 of file GPS_L1_CA.h.

9.179.3.6 const int GPS_CA_TELEMETRY_RATE_SYMBOLS_SECOND = GPS_CA_TELEMETRY_RATE_BITS_SECOND*GPS_CA_TELEMETRY_SYMBOLS_PER_BIT

NAV message bit rate [symbols/s].

Definition at line 80 of file GPS_L1_CA.h.

9.179.3.7 const double GPS_L1_CA_CHIP_PERIOD = 9.7752e-07

GPS L1 C/A chip period [seconds].

Definition at line 54 of file GPS_L1_CA.h.

9.179.3.8 const double GPS_L1_CA_CODE_LENGTH_CHIPS = 1023.0

GPS L1 C/A code length [chips].

Definition at line 52 of file GPS_L1_CA.h.

9.179.3.9 const double GPS_L1_CA_CODE_PERIOD = 0.001

GPS L1 C/A code period [seconds].

Definition at line 53 of file GPS_L1_CA.h.

9.179.3.10 const double GPS_L1_CA_CODE_RATE_HZ = 1.023e6

GPS L1 C/A code rate [chips/s].

Definition at line 51 of file GPS_L1_CA.h.

9.179.3.11 const double GPS_L1_FREQ_HZ = 1.57542e9

L1 [Hz].

Definition at line 50 of file GPS_L1_CA.h.

9.179.3.12 const double GPS_PI = 3.1415926535898

Pi as defined in IS-GPS-200E.

Definition at line 42 of file GPS_L1_CA.h.

9.179.3.13 const int GPS_SUBFRAME_BITS = 300

Number of bits per subframe in the NAV message [bits].

Definition at line 83 of file GPS_L1_CA.h.

9.179.3.14 const int GPS_SUBFRAME_LENGTH = 40

GPS_WORD_LENGTH x 10 = 40 bytes.

Definition at line 82 of file GPS_L1_CA.h.

9.179.3.15 const int GPS_SUBFRAME_MS = 6000

Subframe duration [seconds].

Definition at line 85 of file GPS_L1_CA.h.

9.179.3.16 const int GPS_SUBFRAME_SECONDS = 6

Subframe duration [seconds].

Definition at line 84 of file GPS_L1_CA.h.

9.179.3.17 const double GPS_TWO_PI = 6.283185307179586

2Pi as defined in IS-GPS-200E

Definition at line 43 of file GPS_L1_CA.h.

9.179.3.18 const int GPS_WORD_BITS = 30

Number of bits per word in the NAV message [bits].

Definition at line 86 of file GPS_L1_CA.h.

9.179.3.19 const int GPS_WORD_LENGTH = 4

CRC + GPS WORD (-2 -1 0 ... 29) Bits = 4 bytes.

Definition at line 81 of file GPS_L1_CA.h.

9.179.3.20 const double MAX_TOA_DELAY_MS = 20

Maximum Time-Of-Arrival (TOA) difference between satellites for a receiver operated on Earth surface is 20 ms.

According to the GPS orbit model described in [1] Pag. 32. It should be taken into account to set the buffer size for the PRN start timestamp in the pseudoranges block. [1] J. Bao-Yen Tsui, Fundamentals of Global Positioning System Receivers. A Software Approach, John Wiley & Sons, Inc., Hoboken, NJ, 2nd edition, 2005.

Definition at line 64 of file GPS_L1_CA.h.

9.179.3.21 const double OMEGA_EARTH_DOT = 7.2921151467e-5

Earth rotation rate, [rad/s].

Definition at line 44 of file GPS_L1_CA.h.

9.180 src/core/system_parameters/GPS_L2C.h File Reference

Defines system parameters for GPS L2C signal.

```
#include <cstdint>
#include <vector>
#include <utility>
#include "MATH_CONSTANTS.h"
```

Macros

- #define **GPS_CNAV_PREAMBLE** {1, 0, 0, 0, 1, 0, 1, 1}

Functions

- const std::vector< std::pair< int, int > > **CNAV_PRN** ({{9, 6}})
- const std::vector< std::pair< int, int > > **CNAV_MSG_TYPE** ({{15, 6}})
- const std::vector< std::pair< int, int > > **CNAV_TOW** ({{21, 17}})
- const std::vector< std::pair< int, int > > **CNAV_ALERT_FLAG** ({{38, 1}})
- const std::vector< std::pair< int, int > > **CNAV_WN** ({{39, 13}})
- const std::vector< std::pair< int, int > > **CNAV_HEALTH** ({{52, 3}})
- const std::vector< std::pair< int, int > > **CNAV_TOP1** ({{55, 11}})
- const std::vector< std::pair< int, int > > **CNAV_URA** ({{66, 5}})
- const std::vector< std::pair< int, int > > **CNAV_TOE1** ({{71, 11}})
- const std::vector< std::pair< int, int > > **CNAV_DELTA_A** ({{82, 26}})
- const std::vector< std::pair< int, int > > **CNAV_A_DOT** ({{108, 25}})
- const std::vector< std::pair< int, int > > **CNAV_DELTA_N0** ({{133, 17}})
- const std::vector< std::pair< int, int > > **CNAV_DELTA_N0_DOT** ({{150, 23}})
- const std::vector< std::pair< int, int > > **CNAV_M0** ({{173, 33}})
- const std::vector< std::pair< int, int > > **CNAV_E_ECCENTRICITY** ({{206, 33}})
- const std::vector< std::pair< int, int > > **CNAV_OMEGA** ({{239, 33}})
- const std::vector< std::pair< int, int > > **CNAV_INTEGRITY_FLAG** ({{272, 1}})
- const std::vector< std::pair< int, int > > **CNAV_L2_PHASING_FLAG** ({{273, 1}})
- const std::vector< std::pair< int, int > > **CNAV_TOE2** ({{39, 11}})
- const std::vector< std::pair< int, int > > **CNAV_OMEGA0** ({{50, 33}})
- const std::vector< std::pair< int, int > > **CNAV_I0** ({{83, 33}})
- const std::vector< std::pair< int, int > > **CNAV_DELTA_OMEGA_DOT** ({{116, 17}})
- const std::vector< std::pair< int, int > > **CNAV_I0_DOT** ({{133, 15}})
- const std::vector< std::pair< int, int > > **CNAV_CIS** ({{148, 16}})
- const std::vector< std::pair< int, int > > **CNAV_CIC** ({{164, 16}})
- const std::vector< std::pair< int, int > > **CNAV_CRS** ({{180, 24}})
- const std::vector< std::pair< int, int > > **CNAV_CRC** ({{204, 24}})
- const std::vector< std::pair< int, int > > **CNAV_CUS** ({{228, 21}})
- const std::vector< std::pair< int, int > > **CNAV_CUC** ({{249, 21}})
- const std::vector< std::pair< int, int > > **CNAV_TOP2** ({{39, 11}})
- const std::vector< std::pair< int, int > > **CNAV_URA_NED0** ({{50, 5}})
- const std::vector< std::pair< int, int > > **CNAV_URA_NED1** ({{55, 3}})
- const std::vector< std::pair< int, int > > **CNAV_URA_NED2** ({{58, 3}})
- const std::vector< std::pair< int, int > > **CNAV_TOC** ({{61, 11}})
- const std::vector< std::pair< int, int > > **CNAV_AF0** ({{72, 26}})
- const std::vector< std::pair< int, int > > **CNAV_AF1** ({{98, 20}})
- const std::vector< std::pair< int, int > > **CNAV_AF2** ({{118, 10}})
- const std::vector< std::pair< int, int > > **CNAV_TGD** ({{128, 13}})
- const std::vector< std::pair< int, int > > **CNAV_ISCL1** ({{141, 13}})
- const std::vector< std::pair< int, int > > **CNAV_ISCL2** ({{154, 13}})
- const std::vector< std::pair< int, int > > **CNAV_ISCL5I** ({{167, 13}})
- const std::vector< std::pair< int, int > > **CNAV_ISCL5Q** ({{180, 13}})
- const std::vector< std::pair< int, int > > **CNAV_ALPHA0** ({{193, 8}})
- const std::vector< std::pair< int, int > > **CNAV_ALPHA1** ({{201, 8}})
- const std::vector< std::pair< int, int > > **CNAV_ALPHA2** ({{209, 8}})
- const std::vector< std::pair< int, int > > **CNAV_ALPHA3** ({{217, 8}})
- const std::vector< std::pair< int, int > > **CNAV_BETA0** ({{225, 8}})
- const std::vector< std::pair< int, int > > **CNAV_BETA1** ({{233, 8}})
- const std::vector< std::pair< int, int > > **CNAV_BETA2** ({{241, 8}})
- const std::vector< std::pair< int, int > > **CNAV_BETA3** ({{249, 8}})
- const std::vector< std::pair< int, int > > **CNAV_WNOP** ({{257, 8}})

Variables

- const double GPS_L2_C_m_s = 299792458.0

  The speed of light, [m/s].
- const double GPS_L2_C_m_ms = 299792.4580

  The speed of light, [m/ms].
- const double GPS_L2_PI = 3.1415926535898

  Pi as defined in IS-GPS-200E.
- const double GPS_L2_TWO_PI = 6.283185307179586

  2Pi as defined in IS-GPS-200E
- const double GPS_L2_OMEGA_EARTH_DOT = 7.2921151467e-5

  Earth rotation rate, [rad/s].
- const double GPS_L2_GM = 3.986005e14

  Universal gravitational constant times the mass of the Earth, [m^3/s^2].
- const double GPS_L2_F = -4.442807633e-10

  Constant, [s/(m)^(1/2)].
- const double GPS_L2_FREQ_HZ = 1.2276e9

  L2 [Hz].
- const double GPS_L2_M_CODE_RATE_HZ = 0.5115e6

  GPS L2 M code rate [chips/s].
- const int GPS_L2_M_CODE_LENGTH_CHIPS = 10230

  GPS L2 M code length [chips].
- const double GPS_L2_M_PERIOD = 0.02

  GPS L2 M code period [seconds].
- const double GPS_L2_L_CODE_RATE_HZ = 0.5115e6

  GPS L2 L code rate [chips/s].
- const int GPS_L2_L_CODE_LENGTH_CHIPS = 767250

  GPS L2 L code length [chips].
- const double GPS_L2_L_PERIOD = 1.5

  GPS L2 L code period [seconds].
- const int **GPS_L2C_HISTORY_DEEP** = 5
- const int32_t **GPS_L2C_M_INIT_REG** [115]
- const int GPS_L2_CNAV_DATA_PAGE_BITS = 300

  GPS L2 CNAV page length, including preamble and CRC [bits].
- const int **GPS_L2_SYMBOLS_PER_BIT** = 2
- const int **GPS_L2_SAMPLES_PER_SYMBOL** = 1
- const int **GPS_L2_CNAV_DATA_PAGE_DURATION_S** = 12
- const int GPS_L2_CNAV_DATA_PAGE_BITS_EXTENDED_BYTES = 304

  GPS L2 CNAV page length, including preamble and CRC [bits].
- const double **CNAV_TOW_LSB** = 6.0
- const double **CNAV_TOP1_LSB** = 300.0
- const double **CNAV_TOE1_LSB** = 300.0
- const double **CNAV_DELTA_A_LSB** = TWO_N9
- const double **CNAV_A_DOT_LSB** = TWO_N21
- const double **CNAV_DELTA_N0_LSB** = TWO_N44
- const double **CNAV_DELTA_N0_DOT_LSB** = TWO_N57
- const double **CNAV_M0_LSB** = TWO_N32
- const double **CNAV_E_ECCENTRICITY_LSB** = TWO_N34
- const double **CNAV_OMEGA_LSB** = TWO_N32
- const double **CNAV_TOE2_LSB** = 300.0
- const double **CNAV_OMEGA0_LSB** = TWO_N32
- const double **CNAV_I0_LSB** = TWO_N32

- const double **CNAV_DELTA_OMEGA_DOT_LSB** = TWO_N44
- const double **CNAV_I0_DOT_LSB** = TWO_N44
- const double **CNAV_CIS_LSB** = TWO_N30
- const double **CNAV_CIC_LSB** = TWO_N30
- const double **CNAV_CRS_LSB** = TWO_N8
- const double **CNAV_CRC_LSB** = TWO_N8
- const double **CNAV_CUS_LSB** = TWO_N30
- const double **CNAV_CUC_LSB** = TWO_N30
- const double **CNAV_TOP2_LSB** = 300.0
- const double **CNAV_TOC_LSB** = 300.0
- const double **CNAV_AF0_LSB** = TWO_N60
- const double **CNAV_AF1_LSB** = TWO_N48
- const double **CNAV_AF2_LSB** = TWO_N35
- const double **CNAV_TGD_LSB** = TWO_N35
- const double **CNAV_ISCL1_LSB** = TWO_N35
- const double **CNAV_ISCL2_LSB** = TWO_N35
- const double **CNAV_ISCL5I_LSB** = TWO_N35
- const double **CNAV_ISCL5Q_LSB** = TWO_N35
- const double **CNAV_ALPHA0_LSB** = TWO_N30
- const double **CNAV_ALPHA1_LSB** = TWO_N27
- const double **CNAV_ALPHA2_LSB** = TWO_N24
- const double **CNAV_ALPHA3_LSB** = TWO_N24
- const double **CNAV_BETA0_LSB** = TWO_P11
- const double **CNAV_BETA1_LSB** = TWO_P14
- const double **CNAV_BETA2_LSB** = TWO_P16
- const double **CNAV_BETA3_LSB** = TWO_P16

9.180.1 Detailed Description

Defines system parameters for GPS L2C signal.

Author

Javier Arribas, 2015. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.180.2 Variable Documentation

9.180.2.1 const double GPS_L2_C_m_ms = 299792.4580

The speed of light, [m/ms].

Definition at line 42 of file GPS_L2C.h.

9.180.2.2 const double GPS_L2_C_m_s = 299792458.0

The speed of light, [m/s].

Definition at line 41 of file GPS_L2C.h.

9.180.2.3 const int GPS_L2_CNAV_DATA_PAGE_BITS = 300

GPS L2 CNAV page length, including preamble and CRC [bits].

Definition at line 99 of file GPS_L2C.h.

9.180.2.4 const int GPS_L2_CNAV_DATA_PAGE_BITS_EXTENDED_BYTES = 304

GPS L2 CNAV page length, including preamble and CRC [bits].

Definition at line 103 of file GPS_L2C.h.

9.180.2.5 const double GPS_L2_F = -4.442807633e-10

Constant, [s/(m)^(1/2)].

Definition at line 47 of file GPS_L2C.h.

9.180.2.6 const double GPS_L2_FREQ_HZ = 1.2276e9

L2 [Hz].

Definition at line 51 of file GPS_L2C.h.

9.180.2.7 const double GPS_L2_GM = 3.986005e14

Universal gravitational constant times the mass of the Earth, [m^3/s^2].

Definition at line 46 of file GPS_L2C.h.

9.180.2.8 const int GPS_L2_L_CODE_LENGTH_CHIPS = 767250

GPS L2 L code length [chips].

Definition at line 58 of file GPS_L2C.h.

9.180.2.9 const double GPS_L2_L_CODE_RATE_HZ = 0.5115e6

GPS L2 L code rate [chips/s].

Definition at line 57 of file GPS_L2C.h.

9.180.2.10 const double GPS_L2_L_PERIOD = 1.5

GPS L2 L code period [seconds].

Definition at line 59 of file GPS_L2C.h.

9.180.2.11 const int GPS_L2_M_CODE_LENGTH_CHIPS = 10230

GPS L2 M code length [chips].

Definition at line 54 of file GPS_L2C.h.

9.180.2.12 const double GPS_L2_M_CODE_RATE_HZ = 0.5115e6

GPS L2 M code rate [chips/s].

Definition at line 53 of file GPS_L2C.h.

9.180.2.13 const double GPS_L2_M_PERIOD = 0.02

GPS L2 M code period [seconds].

Definition at line 55 of file GPS_L2C.h.

9.180.2.14 const double GPS_L2_OMEGA_EARTH_DOT = 7.2921151467e-5

Earth rotation rate, [rad/s].

Definition at line 45 of file GPS_L2C.h.

9.180.2.15 const double GPS_L2_PI = 3.1415926535898

Pi as defined in IS-GPS-200E.

Definition at line 43 of file GPS_L2C.h.

9.180.2.16 const double GPS_L2_TWO_PI = 6.283185307179586

2Pi as defined in IS-GPS-200E

Definition at line 44 of file GPS_L2C.h.

9.180.2.17 const int32_t GPS_L2C_M_INIT_REG[115]

Initial value:

```
=
        {0742417664, 0756014035,0002747144,0066265724,
                0601403471, 0703232733, 0124510070, 0617316361,
                0047541621, 0733031046, 0713512145, 0024437606,
                0021264003, 0230655351, 0001314400, 0222021506,
                0540264026, 0205521705, 0064022144, 0120161274,
                0044023533, 0724744327, 0045743577, 0741201660,
                0700274134, 0010247261, 0713433445, 0737324162,
                0311627434, 0710452007, 0722462133, 0050172213,
                0500653703, 0755077436, 0136717361, 0756675453,
                0435506112, 0771353753, 0226107701, 0022025110,
                0402466344, 0752566114, 0702011164, 0041216771,
                0047457275, 0266333164, 0713167356, 0060546335,
                0355173035, 0617201036, 0157465571, 0767360553,
                0023127030, 0431343777, 0747317317, 0045706125,
                0002744276, 0060036467, 0217744147, 0603340174,
                0326616775, 0063240065, 0111460621,
                0604055104, 0157065232, 0013305707, 0603552017,
                0230461355, 0603653437, 0652346475, 0743107103,
                0401521277, 0167335110, 0014013575, 0362051132,
                0617753265, 0216363634, 0755561123, 0365304033,
                0625025543, 0054420334,  0415473671,  0662364360,
                0373446602,  0417564100,  0000526452,  0226631300,
                0113752074,  0706134401,  0041352546,  0664630154,
                0276524255,  0714720530,  0714051771,  0044526647,
                0207164322,  0262120161,  0204244652,  0202133131,
                0714351204,  0657127260,  0130567507,  0670517677,
                0607275514,  0045413633,  0212645405,  0613700455,
                0706202440,  0705056276,  0020373522,  0746013617,
                0132720621,  0434015513,  0566721727,  0140633660}
```

Definition at line 63 of file GPS_L2C.h.

9.181 src/core/system_parameters/gps_navigation_message.h File Reference

Interface of a GPS NAV Data message decoder.

```
#include <bitset>
#include <map>
#include <string>
#include <utility>
#include <vector>
#include "GPS_L1_CA.h"
#include "gps_ephemeris.h"
#include "gps_iono.h"
#include "gps_almanac.h"
#include "gps_utc_model.h"
```

Classes

- class Gps_Navigation_Message

  This class decodes a GPS NAV Data message as described in IS-GPS-200E.

9.181.1 Detailed Description

Interface of a GPS NAV Data message decoder.

Author

Javier Arribas, 2011. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.182 src/core/system_parameters/gps_ref_location.h File Reference

Interface of a GPS REFERENCE LOCATION storage.

```
#include "boost/assign.hpp"
#include <boost/serialization/nvp.hpp>
```

Classes

- class Gps_Ref_Location

  Interface of a GPS REFERENCE LOCATION storage.

9.182.1 Detailed Description

Interface of a GPS REFERENCE LOCATION storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.183 src/core/system_parameters/gps_ref_time.h File Reference

Interface of a GPS REFERENCE TIME storage.

```
#include "boost/assign.hpp"
#include <boost/serialization/nvp.hpp>
```

Classes

- class Gps_Ref_Time

  Interface of a GPS REFERENCE TIME storage.

9.183.1 Detailed Description

Interface of a GPS REFERENCE TIME storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.184 src/core/system_parameters/gps_utc_model.h File Reference

Interface of a GPS UTC MODEL storage.

```
#include "boost/assign.hpp"
#include <boost/serialization/nvp.hpp>
```

Classes

- class Gps_Utc_Model

  This class is a storage for the GPS UTC MODEL data as described in IS-GPS-200E.

9.184.1 Detailed Description

Interface of a GPS UTC MODEL storage.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.185 src/core/system_parameters/MATH_CONSTANTS.h File Reference

Defines useful mathematical constants and their scaled versions.

Variables

- const double TWO_P4 = (16)
  2^4
- const double TWO_P11 = (2048)
  2^11
- const double TWO_P12 = (4096)
  2^12
- const double TWO_P14 = (16384)
  2^14
- const double TWO_P16 = (65536)
  2^16
- const double TWO_P19 = (524288)
  2^19
- const double TWO_P31 = (2147483648.0)
  2^31
- const double TWO_P32 = (4294967296.0)
  2^32 this is too big for an int so add the x.0
- const double TWO_P56 = (7.205759403792794e+016)
  2^56
- const double TWO_P57 = (1.441151880758559e+017)
  2^57
- const double TWO_N2 = (0.25)
  2^-2
- const double TWO_N5 = (0.03125)
  2^-5
- const double TWO_N8 = (0.00390625)
  2^-8
- const double TWO_N9 = (0.001953125)
  2^-9
- const double TWO_N10 = (0.0009765625)
  2^-10
- const double TWO_N11 = (4.882812500000000e-004)
  2^-11
- const double TWO_N14 = (0.00006103515625)
  2^-14
- const double TWO_N15 = (0.00003051757813)
  2^-15
- const double TWO_N16 = (0.0000152587890625)
  2^-16
- const double TWO_N19 = (1.907348632812500e-006)
  2^-19
- const double TWO_N20 = (9.536743164062500e-007)
  2^-20
- const double TWO_N21 = (4.768371582031250e-007)
  2^-21
- const double TWO_N24 = (5.960464477539063e-008)
  2^-24
- const double TWO_N25 = (2.980232238769531e-008)
  2^-25
- const double TWO_N27 = (7.450580596923828e-009)

2^-27

- const double TWO_N29 = (1.862645149230957e-009)

  2^-29
- const double TWO_N30 = (9.313225746154785e-010)

  2^-30
- const double TWO_N31 = (4.656612873077393e-010)

  2^-31
- const double TWO_N32 = (2.328306436538696e-010)

  2^-32
- const double TWO_N33 = (1.164153218269348e-010)

  2^-33
- const double TWO_N34 = (5.82076609134674e-011)

  2^-34
- const double TWO_N35 = (2.91038304567337e-011)

  2^-35
- const double TWO_N38 = (3.637978807091713e-012)

  2^-38
- const double TWO_N43 = (1.136868377216160e-013)

  2^-43
- const double TWO_N44 = (5.684341886080802e-14)

  2^-44
- const double TWO_N46 = (1.4210854715202e-014)

  2^-46
- const double TWO_N48 = (3.552713678800501e-15)

  2^-46
- const double TWO_N50 = (8.881784197001252e-016)

  2^-50
- const double TWO_N51 = (4.44089209850063e-016)

  2^-51
- const double TWO_N55 = (2.775557561562891e-017)

  2^-55
- const double TWO_N57 = (6.938893903907228e-18)

  2^-57
- const double TWO_N59 = (1.73472347597681e-018)

  2^-59
- const double TWO_N60 = (8.673617379884036e-19)

  2^-60
- const double PI_TWO_N19 = (5.992112452678286e-006)

  *Pi*2^-19.*
- const double PI_TWO_N43 = (3.571577341960839e-013)

  *Pi*2^-43.*
- const double PI_TWO_N31 = (1.462918079267160e-009)

  *Pi*2^-31.*
- const double PI_TWO_N38 = (1.142904749427469e-011)

  *Pi*2^-38.*
- const double PI_TWO_N23 = (3.745070282923929e-007)

  *Pi*2^-23.*

9.185.1 Detailed Description

Defines useful mathematical constants and their scaled versions.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.185.2 Variable Documentation

9.185.2.1 const double PI_TWO_N19 = (5.992112452678286e-006)

Pi∗2^-19.

Definition at line 91 of file MATH_CONSTANTS.h.

9.185.2.2 const double PI_TWO_N23 = (3.745070282923929e-007)

Pi∗2^-23.

Definition at line 95 of file MATH_CONSTANTS.h.

9.185.2.3 const double PI_TWO_N31 = (1.462918079267160e-009)

Pi∗2^-31.

Definition at line 93 of file MATH_CONSTANTS.h.

9.185.2.4 const double PI_TWO_N38 = (1.142904749427469e-011)

Pi∗2^-38.

Definition at line 94 of file MATH_CONSTANTS.h.

9.185.2.5 const double PI_TWO_N43 = (3.571577341960839e-013)

Pi∗2^-43.

Definition at line 92 of file MATH_CONSTANTS.h.

9.185.2.6 const double TWO_N10 = (0.0009765625)

2^-10

Definition at line 58 of file MATH_CONSTANTS.h.

9.185.2.7 const double TWO_N11 = (4.882812500000000e-004)

2^-11

Definition at line 59 of file MATH_CONSTANTS.h.

9.185.2.8 const double TWO_N14 = (0.00006103515625)

2^-14

Definition at line 60 of file MATH_CONSTANTS.h.

9.185.2.9 const double TWO_N15 = (0.00003051757813)

2^-15

Definition at line 61 of file MATH_CONSTANTS.h.

9.185.2.10 const double TWO_N16 = (0.0000152587890625)

2^-16

Definition at line 62 of file MATH_CONSTANTS.h.

9.185.2.11 const double TWO_N19 = (1.907348632812500e-006)

2^-19

Definition at line 63 of file MATH_CONSTANTS.h.

9.185.2.12 const double TWO_N2 = (0.25)

2^-2

Definition at line 54 of file MATH_CONSTANTS.h.

9.185.2.13 const double TWO_N20 = (9.536743164062500e-007)

2^-20

Definition at line 64 of file MATH_CONSTANTS.h.

9.185.2.14 const double TWO_N21 = (4.768371582031250e-007)

2^-21

Definition at line 65 of file MATH_CONSTANTS.h.

9.185.2.15 const double TWO_N24 = (5.960464477539063e-008)

2^-24

Definition at line 66 of file MATH_CONSTANTS.h.

9.185.2.16 const double TWO_N25 = (2.980232238769531e-008)

2^-25

Definition at line 67 of file MATH_CONSTANTS.h.

9.185.2.17 const double TWO_N27 = (7.450580596923828e-009)

2^-27

Definition at line 68 of file MATH_CONSTANTS.h.

9.185.2.18 const double TWO_N29 = (1.862645149230957e-009)

2^-29

Definition at line 69 of file MATH_CONSTANTS.h.

9.185.2.19 const double TWO_N30 = (9.313225746154785e-010)

2^-30

Definition at line 70 of file MATH_CONSTANTS.h.

9.185.2.20 const double TWO_N31 = (4.656612873077393e-010)

2^-31

Definition at line 71 of file MATH_CONSTANTS.h.

9.185.2.21 const double TWO_N32 = (2.328306436538696e-010)

2^-32

Definition at line 72 of file MATH_CONSTANTS.h.

9.185.2.22 const double TWO_N33 = (1.164153218269348e-010)

2^-33

Definition at line 73 of file MATH_CONSTANTS.h.

9.185.2.23 const double TWO_N34 = (5.82076609134674e-011)

2^-34

Definition at line 74 of file MATH_CONSTANTS.h.

9.185.2.24 const double TWO_N35 = (2.91038304567337e-011)

2^-35

Definition at line 75 of file MATH_CONSTANTS.h.

9.185.2.25 const double TWO_N38 = (3.637978807091713e-012)

2^-38

Definition at line 77 of file MATH_CONSTANTS.h.

9.185.2.26 const double TWO_N43 = (1.136868377216160e-013)

2^-43

Definition at line 78 of file MATH_CONSTANTS.h.

9.185.2.27 const double TWO_N44 = (5.684341886080802e-14)

2^-44

Definition at line 79 of file MATH_CONSTANTS.h.

9.185.2.28 const double TWO_N46 = (1.4210854715202e-014)

2^-46

Definition at line 80 of file MATH_CONSTANTS.h.

9.185.2.29 const double TWO_N48 = (3.552713678800501e-15)

2^-46

Definition at line 81 of file MATH_CONSTANTS.h.

9.185.2.30 const double TWO_N5 = (0.03125)

2^-5

Definition at line 55 of file MATH_CONSTANTS.h.

9.185.2.31 const double TWO_N50 = (8.881784197001252e-016)

2^-50

Definition at line 83 of file MATH_CONSTANTS.h.

9.185.2.32 const double TWO_N51 = (4.44089209850063e-016)

2^-51

Definition at line 84 of file MATH_CONSTANTS.h.

9.185.2.33 const double TWO_N55 = (2.775557561562891e-017)

2^-55

Definition at line 85 of file MATH_CONSTANTS.h.

9.185.2.34 const double TWO_N57 = (6.938893903907228e-18)

2^-57

Definition at line 86 of file MATH_CONSTANTS.h.

9.185.2.35 const double TWO_N59 = (1.73472347597681e-018)

2^-59

Definition at line 87 of file MATH_CONSTANTS.h.

9.185.2.36 const double TWO_N60 = (8.673617379884036e-19)

2^-60

Definition at line 88 of file MATH_CONSTANTS.h.

9.185.2.37 const double TWO_N8 = (0.00390625)

2^-8

Definition at line 56 of file MATH_CONSTANTS.h.

9.185.2.38 const double TWO_N9 = (0.001953125)

2^-9

Definition at line 57 of file MATH_CONSTANTS.h.

9.185.2.39 const double TWO_P11 = (2048)

2^11

Definition at line 44 of file MATH_CONSTANTS.h.

9.185.2.40 const double TWO_P12 = (4096)

2^12

Definition at line 45 of file MATH_CONSTANTS.h.

9.185.2.41 const double TWO_P14 = (16384)

2^14

Definition at line 46 of file MATH_CONSTANTS.h.

9.185.2.42 const double TWO_P16 = (65536)

2^16

Definition at line 47 of file MATH_CONSTANTS.h.

9.185.2.43 const double TWO_P19 = (524288)

2^19

Definition at line 48 of file MATH_CONSTANTS.h.

9.185.2.44 const double TWO_P31 = (2147483648.0)

2^31

Definition at line 49 of file MATH_CONSTANTS.h.

9.185.2.45 const double TWO_P32 = (4294967296.0)

2^32 this is too big for an int so add the x.0

Definition at line 50 of file MATH_CONSTANTS.h.

9.185.2.46 const double TWO_P4 = (16)

2^4

Definition at line 43 of file MATH_CONSTANTS.h.

9.185.2.47 const double TWO_P56 = (7.205759403792794e+016)

2^56

Definition at line 51 of file MATH_CONSTANTS.h.

9.185.2.48 const double TWO_P57 = (1.441151880758559e+017)

2^57

Definition at line 52 of file MATH_CONSTANTS.h.

9.186 src/core/system_parameters/rtcm.h File Reference

Interface for the RTCM 3.2 Standard.

```
#include <bitset>
#include <deque>
#include <map>
#include <memory>
#include <set>
#include <string>
#include <thread>
#include <utility>
#include <vector>
#include <boost/asio.hpp>
#include <boost/date_time/posix_time/posix_time.hpp>
#include "concurrent_queue.h"
#include "gnss_synchro.h"
#include "galileo_fnav_message.h"
#include "gps_navigation_message.h"
#include "gps_cnav_navigation_message.h"
```

Classes

- class Rtcm

9.186.1 Detailed Description

Interface for the RTCM 3.2 Standard.

Author

Carles Fernandez-Prades, 2015. cfernandez(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.187 src/core/system_parameters/sbas_ephemeris.h File Reference

Interface of a SBAS REFERENCE LOCATION storage.

Classes

- class Sbas_Ephemeris

  This class stores SBAS SV ephemeris data.

9.187.1 Detailed Description

Interface of a SBAS REFERENCE LOCATION storage.

Author

Daniel Fehr, 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.188 src/core/system_parameters/sbas_ionospheric_correction.h File Reference

Interface of the SBAS ionosphere correction set based on SBAS RTKLIB functions.

```
#include <fstream>
#include <iostream>
#include <string>
#include <vector>
#include <boost/serialization/serialization.hpp>
#include <boost/archive/text_oarchive.hpp>
#include <boost/archive/text_iarchive.hpp>
#include <boost/serialization/vector.hpp>
```

Classes

- struct Igp

  Struct that represents a Ionospheric Grid Point (IGP)
- struct Igp_Band

  Struct that represents the band of a Ionospheric Grid Point (IGP)
- class Sbas_Ionosphere_Correction

  Class that handles valid SBAS ionospheric correction for GPS.

9.188.1 Detailed Description

Interface of the SBAS ionosphere correction set based on SBAS RTKLIB functions.

Author

Daniel Fehr 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.189 src/core/system_parameters/sbas_satellite_correction.h File Reference

Interface of the SBAS satellite correction set based on SBAS RTKLIB functions.

```
#include "sbas_time.h"
```

Classes

- struct Fast_Correction
- struct Long_Term_Correction
- class Sbas_Satellite_Correction

  Valid long and fast term SBAS corrections for one SV.

9.189.1 Detailed Description

Interface of the SBAS satellite correction set based on SBAS RTKLIB functions.

Author

Daniel Fehr 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.190 src/core/system_parameters/sbas_telemetry_data.h File Reference

Interface of the SBAS telemetry parser based on SBAS RTKLIB functions.

```
#include <algorithm>
#include <bitset>
#include <cmath>
#include <iostream>
#include <map>
#include <string>
#include <vector>
#include "boost/assign.hpp"
#include "concurrent_queue.h"
#include "sbas_time.h"
```

Classes

- class Sbas_Raw_Msg

  Represents a raw SBAS message of 250cbits + 6 bits padding (8b preamble + 6b message type + 212b data + 24b CRC + 6b zero padding)
- class Sbas_Telemetry_Data

Macros

- #define **SYS_NONE** 0x00 /∗ navigation system: none ∗/
- #define **SYS_GPS** 0x01 /∗ navigation system: GPS ∗/
- #define **SYS_SBS** 0x02 /∗ navigation system: SBAS ∗/
- #define **SYS_GLO** 0x04 /∗ navigation system: GLONASS ∗/
- #define **SYS_GAL** 0x08 /∗ navigation system: Galileo ∗/
- #define **SYS_QZS** 0x10 /∗ navigation system: QZSS ∗/
- #define **SYS_CMP** 0x20 /∗ navigation system: BeiDou ∗/
- #define **SYS_ALL** 0xFF /∗ navigation system: all ∗/
- #define **TSYS_GPS** 0 /∗ time system: GPS time ∗/
- #define **TSYS_UTC** 1 /∗ time system: UTC ∗/
- #define **TSYS_GLO** 2 /∗ time system: GLONASS time ∗/
- #define **TSYS_GAL** 3 /∗ time system: Galileo time ∗/
- #define **TSYS_QZS** 4 /∗ time system: QZSS time ∗/
- #define **TSYS_CMP** 5 /∗ time system: BeiDou time ∗/
- #define **NFREQ** 3 /∗ number of carrier frequencies ∗/
- #define **NFREQGLO** 2 /∗ number of carrier frequencies of GLONASS ∗/
- #define **NEXOBS** 0 /∗ number of extended obs codes ∗/
- #define **MINPRNGPS** 1 /∗ min satellite PRN number of GPS ∗/
- #define **MAXPRNGPS** 32 /∗ max satellite PRN number of GPS ∗/
- #define **NSATGPS** (MAXPRNGPS-MINPRNGPS+1) /∗ number of GPS satellites ∗/
- #define **NSYSGPS** 1
- #define **MINPRNGLO** 0
- #define **MAXPRNGLO** 0
- #define **NSATGLO** 0
- #define **NSYSGLO** 0
- #define **MINPRNGAL** 0
- #define **MAXPRNGAL** 0
- #define **NSATGAL** 0
- #define **NSYSGAL** 0
- #define **MINPRNQZS** 0
- #define **MAXPRNQZS** 0
- #define **NSATQZS** 0
- #define **NSYSQZS** 0
- #define **MINPRNCMP** 0
- #define **MAXPRNCMP** 0
- #define **NSATCMP** 0
- #define **NSYSCMP** 0
- #define **NSYS** (NSYSGPS+NSYSGLO+NSYSGAL+NSYSQZS+NSYSCMP) /∗ number of systems ∗/
- #define **MINPRNSBS** 120 /∗ min satellite PRN number of SBAS ∗/
- #define **MAXPRNSBS** 142 /∗ max satellite PRN number of SBAS ∗/
- #define **NSATSBS** (MAXPRNSBS-MINPRNSBS+1) /∗ number of SBAS satellites ∗/
- #define **MAXSAT** (NSATGPS+NSATGLO+NSATGAL+NSATQZS+NSATCMP+NSATSBS)
- #define **MAXOBS** 64 /∗ max number of obs in an epoch ∗/
- #define **MAXRCV** 64 /∗ max receiver number (1 to MAXRCV) ∗/
- #define **MAXOBSTYPE** 64 /∗ max number of obs type in RINEX ∗/

- #define **DTTOL** 0.005 /* tolerance of time difference (s) */
- #define **MAXDTOE** 7200.0 /* max time difference to ephem Toe (s) for GPS */
- #define **MAXDTOE_GLO** 1800.0 /* max time difference to GLONASS Toe (s) */
- #define **MAXDTOE_SBS** 360.0 /* max time difference to SBAS Toe (s) */
- #define **MAXDTOE_S** 86400.0 /* max time difference to ephem toe (s) for other */
- #define **MAXGDOP** 300.0 /* max GDOP */
- #define **MAXBAND** 10 /* max SBAS band of IGP */
- #define **MAXNIGP** 201 /* max number of IGP in SBAS band */
- #define **P2_11** 4.882812500000000E-04 /* 2^-11 */
- #define **P2_31** 4.656612873077393E-10 /* 2^-31 */
- #define **P2_39** 1.818989403545856E-12 /* 2^-39 */

9.190.1 Detailed Description

Interface of the SBAS telemetry parser based on SBAS RTKLIB functions.

Author

Daniel Fehr 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.191 src/core/system_parameters/sbas_time.h File Reference

Interface and implementation of classes to handle and relate sample stamp and GPS time based time scales.

```
#include <cmath>
#include <iostream>
#include <glog/logging.h>
```

Classes

- class Sbas_Time_Relation
- class Sbas_Time

  Sbas_Time relates the relative sample stamp time scale with the absolute GPS time scale. There are three different states for a Sbas_Time object:

Macros

- #define **EVENT** 2
- #define **FLOW** 3

9.191.1 Detailed Description

Interface and implementation of classes to handle and relate sample stamp and GPS time based time scales.

Author

Daniel Fehr 2013. daniel.co(at)bluewin.ch

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.192 src/tests/common-files/signal_generator_flags.h File Reference

Helper file for unit testing.

```
#include <gflags/gflags.h>
```

Functions

- **DEFINE_string** (generator_binary, std::string(SW_GENERATOR_BIN),"Path of software-defined signal generator binary")
- **DEFINE_string** (rinex_nav_file, std::string(DEFAULT_RINEX_NAV),"Input RINEX navigation file")
- **DEFINE_int32** (duration, 100,"Duration of the experiment [in seconds, max = 300]")
- **DEFINE_string** (static_position,"30.286502,120.032669,100","Static receiver position [log,lat,height]")
- **DEFINE_string** (dynamic_position,"","Observer positions file, in .csv or .nmea format")
- **DEFINE_string** (filename_rinex_obs,"sim.16o","Filename of output RINEX navigation file")
- **DEFINE_string** (filename_raw_data,"signal_out.bin","Filename of output raw data file")
- **DEFINE_int32** (fs_gen_hz, 2600000,"Samppling frequency [Hz]")
- **DEFINE_int32** (test_satellite_PRN, 1,"PRN of the satellite under test (must be visible during the observation time)")

9.192.1 Detailed Description

Helper file for unit testing.

Author

Carles Fernandez-Prades, 2017. cfernandez(at)cttc.es

Copyright (C) 2010-2017 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.193 src/tests/unit-tests/signal-processing-blocks/libs/tlm_dump_reader.h File Reference

Helper file for unit testing.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
```

Classes

- class tlm_dump_reader

9.193.1 Detailed Description

Helper file for unit testing.

Author

Javier Arribas, 2017. jarribas(at)cttc.es

Copyright (C) 2010-2017 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.194 src/tests/unit-tests/signal-processing-blocks/libs/tracking_dump_reader.h File Reference

Helper file for unit testing.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
```

Classes

- class tracking_dump_reader

9.194.1 Detailed Description

Helper file for unit testing.

Author

Javier Arribas, 2017. jarribas(at)cttc.es

Copyright (C) 2010-2017 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.195 src/tests/unit-tests/signal-processing-blocks/libs/tracking_true_obs_reader.h File Reference

Helper file for unit testing.

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
```

Classes

- class tracking_true_obs_reader

9.195.1 Detailed Description

Helper file for unit testing.

Author

Javier Arribas, 2017. jarribas(at)cttc.es

Copyright (C) 2010-2017 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

9.196 src/utils/front-end-cal/front_end_cal.h File Reference

Interface of the Front-end calibration program.

```
#include <armadillo>
#include "file_configuration.h"
#include "concurrent_map.h"
```

Classes

- class FrontEndCal

9.196.1 Detailed Description

Interface of the Front-end calibration program.

Author

Javier Arribas, 2013. jarribas(at)cttc.es

Copyright (C) 2010-2015 (see AUTHORS file for a list of contributors)

GNSS-SDR is a software defined Global Navigation Satellite Systems receiver

This file is part of GNSS-SDR.

GNSS-SDR is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNSS-SDR is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNSS-SDR. If not, see http://www.gnu.org/licenses/.

Index

~ArraySignalConditioner
ArraySignalConditioner, 55
~Channel
Channel, 60
~ControlMessageFactory
ControlMessageFactory, 70
~ControlThread
ControlThread, 72
~FileConfiguration
FileConfiguration, 80
~FirFilter
FirFilter, 82
~GNSSFlowgraph
GNSSFlowgraph, 176
~GenSignalSource
GenSignalSource, 161
~Gnss_Satellite
Gnss_Satellite, 164
~Nmea_Printer
Nmea_Printer, 304
~Rinex_Printer
Rinex_Printer, 351
~Rtcm_Printer
Rtcm_Printer, 360
~Sbas_Telemetry_Data
Sbas_Telemetry_Data, 369
~SignalConditioner
SignalConditioner, 374
~channel_msg_receiver_cc
channel_msg_receiver_cc, 62
~galileo_e1_pvt_cc
galileo_e1_pvt_cc, 91
~galileo_e5a_noncoherentIQ_acquisition_caf_cc
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 95
~galileo_pcps_8ms_acquisition_cc
galileo_pcps_8ms_acquisition_cc, 128
~gps_l1_ca_pvt_cc
gps_l1_ca_pvt_cc, 219
~hybrid_pvt_cc
hybrid_pvt_cc, 286
~pcps_acquisition_cc
pcps_acquisition_cc, 309
~pcps_acquisition_fine_doppler_cc
pcps_acquisition_fine_doppler_cc, 313
~pcps_acquisition_sc
pcps_acquisition_sc, 316
~pcps_assisted_acquisition_cc
pcps_assisted_acquisition_cc, 320
~pcps_cccwsr_acquisition_cc
pcps_cccwsr_acquisition_cc, 324
~pcps_multithread_acquisition_cc
pcps_multithread_acquisition_cc, 327
~pcps_opencl_acquisition_cc
pcps_opencl_acquisition_cc, 331
~pcps_quicksync_acquisition_cc
pcps_quicksync_acquisition_cc, 335
~pcps_tong_acquisition_cc
pcps_tong_acquisition_cc, 339

A_0G_10
Galileo_Navigation_Message, 119
A_1
Galileo_Ephemeris, 102
Galileo_Navigation_Message, 119
A_1G_10
Galileo_Navigation_Message, 119
ALPHA_0
GPS_L1_CA.h, 573
Acq_delay_samples
Gnss_Synchro, 170
Acq_doppler_hz
Gnss_Synchro, 170
Acq_samplestamp_samples
Gnss_Synchro, 170
AcquisitionInterface, 53
af0_4
Galileo_Ephemeris, 102
Galileo_Navigation_Message, 119
af1_4
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 119
af2_4
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 119
ai0_5
Galileo_Iono, 113
Galileo_Navigation_Message, 119
ai1_5
Galileo_Iono, 113
Galileo_Navigation_Message, 119
ai2_5
Galileo_Iono, 113
Galileo_Navigation_Message, 120
almanac
GpsL1CaSubframeFsm, 271
almanacHealth
Gps_Navigation_Message, 228
apply

Sbas_Ionosphere_Correction, 365
apply_action
GNSSFlowgraph, 176
ArraySignalConditioner, 54
~ArraySignalConditioner, 55
ArraySignalConditioner, 55
implementation, 55
Azimuth
Gps_Acq_Assist, 178

b_L2_P_data_flag
Gps_Ephemeris, 204
Gps_Navigation_Message, 229
b_alert_flag
Gps_CNAV_Ephemeris, 186
Gps_Ephemeris, 204
Gps_Navigation_Message, 228
b_antispoofing_flag
Gps_CNAV_Ephemeris, 186
Gps_Ephemeris, 204
Gps_Navigation_Message, 228
b_fit_interval_flag
Gps_Ephemeris, 204
Gps_Navigation_Message, 228
b_flag_iono_valid
Gps_CNAV_Navigation_Message, 196
b_integrity_status_flag
Gps_CNAV_Ephemeris, 186
Gps_Ephemeris, 204
Gps_Navigation_Message, 228
b_sv_do_not_use
Sbas_Ephemeris, 363
BGD_E1E5a_5
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 120
BGD_E1E5b_5
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 120
bancroftPos
Ls_Pvt, 303
beamformer, 56
BeamformerFilter, 56
implementation, 57
byte_x2_to_complex_byte, 57
ByteToShort, 58
implementation, 59

C_ic_4
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 120
C_is_4
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 120
C_rc_3
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 120
C_rs_3
Galileo_Ephemeris, 103
Galileo_Navigation_Message, 120
C_uc_3
Galileo_Ephemeris, 104
Galileo_Navigation_Message, 120
C_us_3
Galileo_Ephemeris, 104
Galileo_Navigation_Message, 121
CN0_dB_hz
Gnss_Synchro, 171
Carrier_Doppler_hz
Gnss_Synchro, 171
carrier_lock_detector
lock_detectors.h, 522
Carrier_phase_rads
Gnss_Synchro, 171
cart2geo
Pvt_Solution, 343
Channel, 59
~Channel, 60
Channel, 60
implementation, 61
set_signal, 61
start_acquisition, 61
Channel_ID
Gnss_Synchro, 171
channel_msg_receiver_cc, 61
~channel_msg_receiver_cc, 62
ChannelFsm, 62
ChannelInterface, 63
cl_fft_plan, 64
clFFT_Complex, 64
clFFT_Dim3, 65
clFFT_SplitComplex, 65
cn0_svn_estimator
lock_detectors.h, 522
Code_Phase
Gps_Acq_Assist, 178
Code_Phase_int
Gps_Acq_Assist, 178
Code_Phase_window
Gps_Acq_Assist, 178
complex_byte_to_float_x2, 65
complex_exp_gen
gnss_signal_processing.h, 442
complex_exp_gen_conj
gnss_signal_processing.h, 442
complex_float_to_complex_byte, 66
compute_DOP
Pvt_Solution, 343
compute_GPS_time
Rinex_Printer, 351
compute_Galileo_time
Rinex_Printer, 351
compute_UTC_time
Rinex_Printer, 351
concurrent_map< Data >, 67
concurrent_queue< Data >, 67
ConfigurationInterface, 68
connect

GNSSFlowgraph, 176
control_message_factory.h
ControlMessage, 544
ControlMessage, 69
control_message_factory.h, 544
ControlMessage_, 69
ControlMessageFactory, 70
~ControlMessageFactory, 70
ControlMessageFactory, 70
ControlThread, 71
~ControlThread, 72
ControlThread, 71
flowgraph, 72
run, 72
set_control_queue, 72
correlation_length_ms
Gnss_Synchro, 171
cpu_multicorrelator, 73
cpu_multicorrelator_16sc, 73
cshort_to_float_x2, 74
cuda_multicorrelator, 74

d_A0
Gps_CNAV_Utc_Model, 198
Gps_Navigation_Message, 229
Gps_Utc_Model, 240
d_A1
Gps_CNAV_Utc_Model, 198
Gps_Navigation_Message, 229
Gps_Utc_Model, 240
d_A_DOT
Gps_CNAV_Ephemeris, 186
d_A_f0
Gps_Almanac, 180
Gps_CNAV_Ephemeris, 186
Gps_Ephemeris, 205
Gps_Navigation_Message, 229
d_A_f1
Gps_Almanac, 180
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 205
Gps_Navigation_Message, 229
d_A_f2
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 205
Gps_Navigation_Message, 229
d_Cic
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 205
Gps_Navigation_Message, 230
d_Cis
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 205
Gps_Navigation_Message, 231
d_Crc
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 205
Gps_Navigation_Message, 231
d_Crs
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 205
Gps_Navigation_Message, 231
d_Cuc
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 205
Gps_Navigation_Message, 231
d_Cus
Gps_CNAV_Ephemeris, 187
Gps_Ephemeris, 206
Gps_Navigation_Message, 231
d_DELTA_DOT_N
Gps_CNAV_Ephemeris, 188
d_DELTA_OMEGA_DOT
Gps_CNAV_Ephemeris, 188
d_DELTA_A
Gps_CNAV_Ephemeris, 188
d_Delta_n
Gps_CNAV_Ephemeris, 188
Gps_Ephemeris, 206
Gps_Navigation_Message, 231
d_DeltaT_LSF
Gps_CNAV_Utc_Model, 199
Gps_Navigation_Message, 231
Gps_Utc_Model, 240
d_DeltaT_LS
Gps_CNAV_Utc_Model, 199
Gps_Navigation_Message, 231
Gps_Utc_Model, 240
d_Doppler0
Gps_Acq_Assist, 178
d_Doppler1
Gps_Acq_Assist, 178
d_IDOT
Gps_CNAV_Ephemeris, 188
Gps_Ephemeris, 206
Gps_Navigation_Message, 232
d_IODE_SF2
Gps_Ephemeris, 206
d_IODE_SF3
Gps_Ephemeris, 207
d_IODC
Gps_Ephemeris, 206
Gps_Navigation_Message, 232
d_M_0
Gps_Almanac, 181
Gps_CNAV_Ephemeris, 189
Gps_Ephemeris, 207
Gps_Navigation_Message, 232
d_OMEGA0
Gps_Almanac, 181
Gps_CNAV_Ephemeris, 189
Gps_Ephemeris, 207
Gps_Navigation_Message, 232
d_OMEGA_DOT
Gps_Almanac, 181
Gps_Ephemeris, 207
Gps_Navigation_Message, 232

d_OMEGA
Gps_Almanac, 181
Gps_CNAV_Ephemeris, 189
Gps_Ephemeris, 207
Gps_Navigation_Message, 232
d_TGD
Gps_CNAV_Ephemeris, 190
Gps_Ephemeris, 208
Gps_Navigation_Message, 234
d_TOW_SF1
Gps_Navigation_Message, 234
d_TOW_SF2
Gps_Navigation_Message, 234
d_TOW_SF3
Gps_Navigation_Message, 235
d_TOW_SF4
Gps_Navigation_Message, 235
d_TOW_SF5
Gps_Navigation_Message, 235
d_TOW
Gnss_Synchro, 171
Gps_Acq_Assist, 178
Gps_CNAV_Ephemeris, 191
Gps_Ephemeris, 209
Gps_Navigation_Message, 234
d_Toa
Gps_Almanac, 181
Gps_Navigation_Message, 234
d_Toc
Gps_CNAV_Ephemeris, 190
Gps_Ephemeris, 208
Gps_Navigation_Message, 234
d_Toe
Gps_Ephemeris, 209
Gps_Navigation_Message, 234
d_Toe1
Gps_CNAV_Ephemeris, 190
d_Toe2
Gps_CNAV_Ephemeris, 190
d_Top
Gps_CNAV_Ephemeris, 191
d_URA0
Gps_CNAV_Ephemeris, 191
d_URA1
Gps_CNAV_Ephemeris, 191
d_URA2
Gps_CNAV_Ephemeris, 191
d_acc
Sbas_Ephemeris, 363
d_af0
Sbas_Ephemeris, 364
d_af1
Sbas_Ephemeris, 364
d_alpha0
Gps_CNAV_Iono, 193
Gps_Iono, 211
Gps_Navigation_Message, 229
d_alpha1
Gps_CNAV_Iono, 193
Gps_Iono, 211
Gps_Navigation_Message, 230
d_alpha2
Gps_CNAV_Iono, 193
Gps_Iono, 211
Gps_Navigation_Message, 230
d_alpha3
Gps_CNAV_Iono, 193
Gps_Iono, 211
Gps_Navigation_Message, 230
d_averaging_depth
Pvt_Solution, 345
d_avg_height_m
Pvt_Solution, 345
d_avg_latitude_d
Pvt_Solution, 345
d_avg_longitude_d
Pvt_Solution, 345
d_beta0
Gps_CNAV_Iono, 194
Gps_Iono, 212
Gps_Navigation_Message, 230
d_beta1
Gps_CNAV_Iono, 194
Gps_Iono, 212
Gps_Navigation_Message, 230
d_beta2
Gps_CNAV_Iono, 194
Gps_Iono, 212
Gps_Navigation_Message, 230
d_beta3
Gps_CNAV_Iono, 194
Gps_Iono, 212
Gps_Navigation_Message, 230
d_daf0
Long_Term_Correction, 301
d_daf1
Long_Term_Correction, 301
d_dpos
Long_Term_Correction, 301
d_dtr
Gps_CNAV_Ephemeris, 188
Gps_Ephemeris, 206
d_dvel
Long_Term_Correction, 301
d_e_eccentricity
Gps_Almanac, 181
Gps_CNAV_Ephemeris, 188
Gps_Ephemeris, 206
Gps_Navigation_Message, 232
d_height_m
Pvt_Solution, 345
d_i_0
Gps_CNAV_Ephemeris, 188
Gps_Ephemeris, 206
Gps_Navigation_Message, 232
d_latitude_d

Pvt_Solution, 345
d_longitude_d
Pvt_Solution, 345
d_nav
GpsL1CaSubframeFsm, 271
d_nchannels
galileo_e1_ls_pvt, 89
gps_l1_ca_ls_pvt, 217
hybrid_ls_pvt, 284
d_pos
Sbas_Ephemeris, 364
d_rx_dt_s
Pvt_Solution, 345
d_satClkDrift
Gps_CNAV_Ephemeris, 189
Gps_Ephemeris, 207
d_satpos_X
Galileo_Ephemeris, 104
Gps_CNAV_Ephemeris, 189
Gps_CNAV_Navigation_Message, 196
Gps_Ephemeris, 207
Gps_Navigation_Message, 233
d_satpos_Y
Galileo_Ephemeris, 104
Gps_CNAV_Ephemeris, 189
Gps_CNAV_Navigation_Message, 196
Gps_Ephemeris, 207
Gps_Navigation_Message, 233
d_satpos_Z
Galileo_Ephemeris, 104
Gps_CNAV_Ephemeris, 189
Gps_CNAV_Navigation_Message, 197
Gps_Ephemeris, 208
Gps_Navigation_Message, 233
d_satvel_X
Galileo_Ephemeris, 104
Gps_CNAV_Ephemeris, 190
Gps_CNAV_Navigation_Message, 197
Gps_Ephemeris, 208
Gps_Navigation_Message, 233
d_satvel_Y
Galileo_Ephemeris, 104
Gps_CNAV_Ephemeris, 190
Gps_CNAV_Navigation_Message, 197
Gps_Ephemeris, 208
Gps_Navigation_Message, 233
d_satvel_Z
Galileo_Ephemeris, 105
Gps_CNAV_Ephemeris, 190
Gps_CNAV_Navigation_Message, 197
Gps_Ephemeris, 208
Gps_Navigation_Message, 233
d_sqrt_A
Gps_Almanac, 181
Gps_Ephemeris, 208
Gps_Navigation_Message, 233
d_t_OT
Gps_CNAV_Utc_Model, 199
Gps_Navigation_Message, 234
Gps_Utc_Model, 240
d_trx
Long_Term_Correction, 302
d_valid_observations
Pvt_Solution, 346
d_vel
Sbas_Ephemeris, 364
d_visible_satellites_Az
Pvt_Solution, 346
d_visible_satellites_CN0_dB
Pvt_Solution, 346
d_visible_satellites_Distance
Pvt_Solution, 346
d_visible_satellites_El
Pvt_Solution, 346
d_visible_satellites_IDs
Pvt_Solution, 346
decode_block
Viterbi_Decoder, 391
delta_n_3
Galileo_Ephemeris, 105
Galileo_Navigation_Message, 121
direct_resampler_conditioner_cb, 75
direct_resampler_conditioner_cc, 76
direct_resampler_conditioner_cs, 77
DirectResamplerConditioner, 77
implementation, 78
dll_nc_e_minus_l_normalized
tracking_discriminators.h, 527
dll_nc_vemlp_normalized
tracking_discriminators.h, 527
dopplerUncertainty
Gps_Acq_Assist, 178

E1B_DVS_5
Galileo_Ephemeris, 105
Galileo_Navigation_Message, 121
E1B_HS_5
Galileo_Ephemeris, 105
Galileo_Navigation_Message, 121
E5a_DVS
Galileo_Ephemeris, 105
E5a_HS
Galileo_Ephemeris, 105
E5b_DVS_5
Galileo_Ephemeris, 105
Galileo_Navigation_Message, 121
E5b_HS_5
Galileo_Ephemeris, 105
Galileo_Navigation_Message, 121
e_1
Galileo_Ephemeris, 106
Galileo_Navigation_Message, 121
Elevation
Gps_Acq_Assist, 179
ephemeris
GpsL1CaSubframeFsm, 272
estimate_doppler_from_eph

FrontEndCal, 85
Event_gps_word_invalid
GpsL1CaSubframeFsm, 271
Event_gps_word_preamble
GpsL1CaSubframeFsm, 271
Event_gps_word_valid
GpsL1CaSubframeFsm, 271

F
GPS_L1_CA.h, 573
Fast_Correction, 78
FileConfiguration, 79
~FileConfiguration, 80
FileSignalSource, 80
implementation, 81
FirFilter, 81
~FirFilter, 82
FirFilter, 82
implementation, 82
flag_TOW_set
Galileo_Fnav_Message, 111
Galileo_Navigation_Message, 123
flag_all_almanac
Galileo_Fnav_Message, 110
Galileo_Navigation_Message, 121
flag_all_ephemeris
Galileo_Fnav_Message, 110
Galileo_Navigation_Message, 122
flag_almanac_1
Galileo_Fnav_Message, 110
Galileo_Navigation_Message, 122
flag_almanac_2
Galileo_Fnav_Message, 111
Galileo_Navigation_Message, 122
flag_almanac_3
Galileo_Navigation_Message, 122
flag_almanac_4
Galileo_Navigation_Message, 122
flag_ephemeris_1
Galileo_Fnav_Message, 111
Galileo_Navigation_Message, 122
flag_ephemeris_2
Galileo_Fnav_Message, 111
Galileo_Navigation_Message, 122
flag_ephemeris_3
Galileo_Fnav_Message, 111
Galileo_Navigation_Message, 122
flag_ephemeris_4
Galileo_Navigation_Message, 123
flag_iono_and_GST
Galileo_Fnav_Message, 111
Galileo_Navigation_Message, 123
flag_iono_valid
Gps_Navigation_Message, 235
Flag_preamble
Gnss_Synchro, 171
flag_utc_model
Galileo_Fnav_Message, 111
Galileo_Navigation_Message, 123
flag_utc_model_valid
Gps_Navigation_Message, 235
Flag_valid_acquisition
Gnss_Synchro, 171
Flag_valid_symbol_output
Gnss_Synchro, 172
Flag_valid_word
Gnss_Synchro, 172
FlexibandSignalSource, 82
implementation, 83
fll_four_quadrant_atan
tracking_discriminators.h, 527
flowgraph
ControlThread, 72
forecast
galileo_e1b_telemetry_decoder_cc, 93
galileo_e5a_telemetry_decoder_cc, 99
gps_l1_ca_telemetry_decoder_cc, 221
gps_l2c_telemetry_decoder_cc, 223
sbas_l1_telemetry_decoder_cc, 366
FreqXlatingFirFilter, 83
implementation, 84
FrontEndCal, 84
estimate_doppler_from_eph, 85
GPS_L1_front_end_model_E4000, 85
get_ephemeris, 85
set_configuration, 85

GALILEO_C_m_ms
Galileo_E1.h, 555
GALILEO_C_m_s
Galileo_E1.h, 555
GALILEO_GM
Galileo_E1.h, 556
GALILEO_INAV_PAGE_PART_SYMBOLS
Galileo_E1.h, 556
GALILEO_INAV_PAGE_SYMBOLS
Galileo_E1.h, 557
GALILEO_OMEGA_EARTH_DOT
Galileo_E1.h, 557
GALILEO_PI
Galileo_E1.h, 557
GALILEO_TWO_PI
Galileo_E1.h, 557
GALILEO_F
Galileo_E1.h, 556
GNSSBlockFactory, 173
GNSSBlockInterface, 174
GNSSFlowgraph, 175
~GNSSFlowgraph, 176
apply_action, 176
connect, 176
GNSSFlowgraph, 176
send_telemetry_msg, 176
start, 176
stop, 176
GPS_Bit_Number
Gps_Acq_Assist, 179
GPS_C_m_ms

GPS_L1_CA.h, 574
GPS_C_m_s
GPS_L1_CA.h, 574
GPS_CA_TELEMETRY_RATE_BITS_SECOND
GPS_L1_CA.h, 574
GPS_CA_TELEMETRY_RATE_SYMBOLS_SECOND
GPS_L1_CA.h, 574
GPS_L1_CA.h
ALPHA_0, 573
F, 573
GPS_C_m_ms, 574
GPS_C_m_s, 574
GPS_CA_TELEMETRY_RATE_BITS_SECOND, 574
GPS_CA_TELEMETRY_RATE_SYMBOLS_SE↩ COND, 574
GPS_L1_CA_CHIP_PERIOD, 574
GPS_L1_CA_CODE_LENGTH_CHIPS, 574
GPS_L1_CA_CODE_PERIOD, 574
GPS_L1_CA_CODE_RATE_HZ, 575
GPS_L1_FREQ_HZ, 575
GPS_PI, 575
GPS_SUBFRAME_BITS, 575
GPS_SUBFRAME_LENGTH, 575
GPS_SUBFRAME_MS, 575
GPS_SUBFRAME_SECONDS, 575
GPS_TWO_PI, 575
GPS_WORD_BITS, 576
GPS_WORD_LENGTH, 576
GM, 573
MAX_TOA_DELAY_MS, 576
OMEGA_EARTH_DOT, 576
T_OA, 573
GPS_L1_CA_CHIP_PERIOD
GPS_L1_CA.h, 574
GPS_L1_CA_CODE_LENGTH_CHIPS
GPS_L1_CA.h, 574
GPS_L1_CA_CODE_PERIOD
GPS_L1_CA.h, 574
GPS_L1_CA_CODE_RATE_HZ
GPS_L1_CA.h, 575
GPS_L1_FREQ_HZ
GPS_L1_CA.h, 575
GPS_L1_front_end_model_E4000
FrontEndCal, 85
GPS_L2_C_m_ms
GPS_L2C.h, 580
GPS_L2_C_m_s
GPS_L2C.h, 580
GPS_L2_CNAV_DATA_PAGE_BITS_EXTENDED_B↩ YTES
GPS_L2C.h, 580
GPS_L2_CNAV_DATA_PAGE_BITS
GPS_L2C.h, 580
GPS_L2_FREQ_HZ
GPS_L2C.h, 580
GPS_L2_GM
GPS_L2C.h, 580
GPS_L2_L_CODE_LENGTH_CHIPS
GPS_L2C.h, 580
GPS_L2_L_CODE_RATE_HZ
GPS_L2C.h, 581
GPS_L2_L_PERIOD
GPS_L2C.h, 581
GPS_L2_M_CODE_LENGTH_CHIPS
GPS_L2C.h, 581
GPS_L2_M_CODE_RATE_HZ
GPS_L2C.h, 581
GPS_L2_M_PERIOD
GPS_L2C.h, 581
GPS_L2_OMEGA_EARTH_DOT
GPS_L2C.h, 581
GPS_L2_PI
GPS_L2C.h, 581
GPS_L2_TWO_PI
GPS_L2C.h, 581
GPS_L2_F
GPS_L2C.h, 580
GPS_L2C.h
GPS_L2_C_m_ms, 580
GPS_L2_C_m_s, 580
GPS_L2_CNAV_DATA_PAGE_BITS_EXTEND↩ ED_BYTES, 580
GPS_L2_CNAV_DATA_PAGE_BITS, 580
GPS_L2_FREQ_HZ, 580
GPS_L2_GM, 580
GPS_L2_L_CODE_LENGTH_CHIPS, 580
GPS_L2_L_CODE_RATE_HZ, 581
GPS_L2_L_PERIOD, 581
GPS_L2_M_CODE_LENGTH_CHIPS, 581
GPS_L2_M_CODE_RATE_HZ, 581
GPS_L2_M_PERIOD, 581
GPS_L2_OMEGA_EARTH_DOT, 581
GPS_L2_PI, 581
GPS_L2_TWO_PI, 581
GPS_L2_F, 580
GPS_L2C_M_INIT_REG, 582
GPS_L2C_M_INIT_REG
GPS_L2C.h, 582
GPS_PI
GPS_L1_CA.h, 575
GPS_SUBFRAME_BITS
GPS_L1_CA.h, 575
GPS_SUBFRAME_LENGTH
GPS_L1_CA.h, 575
GPS_SUBFRAME_MS
GPS_L1_CA.h, 575
GPS_SUBFRAME_SECONDS
GPS_L1_CA.h, 575
GPS_TWO_PI
GPS_L1_CA.h, 575
GPS_WORD_BITS
GPS_L1_CA.h, 576
GPS_WORD_LENGTH
GPS_L1_CA.h, 576
GPU_Complex, 282

GPU_Complex_Short, 283
GST_to_UTC_time
    Galileo_Utc_Model, 131
Galileo_Almanac, 85
    Galileo_Almanac, 87
Galileo_E1.h
    GALILEO_C_m_ms, 555
    GALILEO_C_m_s, 555
    GALILEO_GM, 556
    GALILEO_INAV_PAGE_PART_SYMBOLS, 556
    GALILEO_INAV_PAGE_SYMBOLS, 557
    GALILEO_OMEGA_EARTH_DOT, 557
    GALILEO_PI, 557
    GALILEO_TWO_PI, 557
    GALILEO_F, 556
    Galileo_E1_B_CODE_LENGTH_CHIPS, 555
    Galileo_E1_B_SYMBOL_RATE_BPS, 555
    Galileo_E1_C_SECONDARY_CODE_LENGTH, 555
    Galileo_E1_CODE_CHIP_RATE_HZ, 556
    Galileo_E1_CODE_PERIOD, 556
    Galileo_E1_FREQ_HZ, 556
    Galileo_E1_SUB_CARRIER_A_RATE_HZ, 556
    Galileo_E1_SUB_CARRIER_B_RATE_HZ, 556
Galileo_E1_B_CODE_LENGTH_CHIPS
    Galileo_E1.h, 555
Galileo_E1_B_SYMBOL_RATE_BPS
    Galileo_E1.h, 555
Galileo_E1_C_SECONDARY_CODE_LENGTH
    Galileo_E1.h, 555
Galileo_E1_CODE_CHIP_RATE_HZ
    Galileo_E1.h, 556
Galileo_E1_CODE_PERIOD
    Galileo_E1.h, 556
Galileo_E1_FREQ_HZ
    Galileo_E1.h, 556
Galileo_E1_SUB_CARRIER_A_RATE_HZ
    Galileo_E1.h, 556
Galileo_E1_SUB_CARRIER_B_RATE_HZ
    Galileo_E1.h, 556
Galileo_E1_Tcp_Connector_Tracking_cc, 91
Galileo_E5a_Dll_Pll_Tracking_cc, 93
Galileo_Ephemeris, 99
    A_1, 102
    af0_4, 102
    af1_4, 103
    af2_4, 103
    BGD_E1E5a_5, 103
    BGD_E1E5b_5, 103
    C_ic_4, 103
    C_is_4, 103
    C_rc_3, 103
    C_rs_3, 103
    C_uc_3, 104
    C_us_3, 104
    d_satpos_X, 104
    d_satpos_Y, 104
    d_satpos_Z, 104
    d_satvel_X, 104
    d_satvel_Y, 104
    d_satvel_Z, 105
    delta_n_3, 105
    E1B_DVS_5, 105
    E1B_HS_5, 105
    E5a_DVS, 105
    E5a_HS, 105
    E5b_DVS_5, 105
    E5b_HS_5, 105
    e_1, 106
    Galileo_System_Time, 102
    Galileo_dtr, 106
    i_0_2, 106
    i_satellite_PRN, 106
    iDot_2, 106
    M0_1, 106
    OMEGA_0_2, 106
    OMEGA_dot_3, 107
    omega_2, 106
    satellitePosition, 102
    serialize, 102
    sv_clock_drift, 102
    sv_clock_relativistic_term, 102
    t0c_4, 107
    t0e_1, 107
    TOW_5, 107
    WN_5, 107
Galileo_Fnav_Message, 107
    flag_TOW_set, 111
    flag_all_almanac, 110
    flag_all_ephemeris, 110
    flag_almanac_1, 110
    flag_almanac_2, 111
    flag_ephemeris_1, 111
    flag_ephemeris_2, 111
    flag_ephemeris_3, 111
    flag_iono_and_GST, 111
    flag_utc_model, 111
Galileo_Iono, 112
    ai0_5, 113
    ai1_5, 113
    ai2_5, 113
    Galileo_Iono, 113
    Region1_flag_5, 113
    Region2_flag_5, 113
    Region3_flag_5, 113
    Region4_flag_5, 113
    Region5_flag_5, 114
    TOW_5, 114
    WN_5, 114
Galileo_Navigation_Message, 114
    A_0G_10, 119
    A_1, 119
    A_1G_10, 119
    af0_4, 119
    af1_4, 119
    af2_4, 119

ai0_5, 119
ai1_5, 119
ai2_5, 120
BGD_E1E5a_5, 120
BGD_E1E5b_5, 120
C_ic_4, 120
C_is_4, 120
C_rc_3, 120
C_rs_3, 120
C_uc_3, 120
C_us_3, 121
delta_n_3, 121
E1B_DVS_5, 121
E1B_HS_5, 121
E5b_DVS_5, 121
E5b_HS_5, 121
e_1, 121
flag_TOW_set, 123
flag_all_almanac, 121
flag_all_ephemeris, 122
flag_almanac_1, 122
flag_almanac_2, 122
flag_almanac_3, 122
flag_almanac_4, 122
flag_ephemeris_1, 122
flag_ephemeris_2, 122
flag_ephemeris_3, 122
flag_ephemeris_4, 123
flag_iono_and_GST, 123
flag_utc_model, 123
Galileo_dtr, 123
galileo_satpos_X, 123
galileo_satpos_Y, 123
galileo_satpos_Z, 124
galileo_satvel_X, 124
galileo_satvel_Y, 124
galileo_satvel_Z, 124
i_0_2, 124
iDot_2, 124
IOD_nav_1, 124
IOD_nav_2, 125
M0_1, 125
OMEGA_0_2, 125
OMEGA_dot_3, 125
omega_2, 125
Region1_flag_5, 125
Region2_flag_5, 125
Region3_flag_5, 125
Region4_flag_5, 126
Region5_flag_5, 126
t0c_4, 126
t0e_1, 126
t_0G_10, 126
WN_0G_10, 126
Galileo_System_Time
Galileo_Ephemeris, 102
Galileo_Utc_Model, 130
GST_to_UTC_time, 131
Galileo_Utc_Model, 131
t0t_6, 131
WNot_6, 131
Galileo_dtr
Galileo_Ephemeris, 106
Galileo_Navigation_Message, 123
galileo_e1_cboc_gen
galileo_e1_signal_processing.h, 440
galileo_e1_code_gen_complex_sampled
galileo_e1_signal_processing.h, 440
galileo_e1_code_gen_int
galileo_e1_signal_processing.h, 440
galileo_e1_dll_pll_veml_tracking_cc, 87
general_work, 88
galileo_e1_ls_pvt, 88
d_nchannels, 89
galileo_ephemeris_map, 89
galileo_e1_observables_cc, 89
galileo_e1_pvt_cc, 90
∼galileo_e1_pvt_cc, 91
general_work, 91
galileo_e1_signal_processing.h
galileo_e1_cboc_gen, 440
galileo_e1_code_gen_complex_sampled, 440
galileo_e1_code_gen_int, 440
galileo_e1_sinboc_11_gen, 440
galileo_e1_sinboc_61_gen, 440
galileo_e1_sinboc_11_gen
galileo_e1_signal_processing.h, 440
galileo_e1_sinboc_61_gen
galileo_e1_signal_processing.h, 440
galileo_e1b_telemetry_decoder_cc, 92
forecast, 93
general_work, 93
set_channel, 93
set_decimation, 93
set_satellite, 93
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 94
∼galileo_e5a_noncoherentIQ_acquisition_caf_cc, 95
general_work, 96
init, 96
mag, 96
set_active, 96
set_channel, 96
set_doppler_max, 96
set_doppler_step, 97
set_gnss_synchro, 97
set_local_code, 97
set_state, 97
set_threshold, 98
galileo_e5a_telemetry_decoder_cc, 98
forecast, 99
general_work, 99
set_channel, 99
set_satellite, 99
galileo_ephemeris_map
galileo_e1_ls_pvt, 89

hybrid_ls_pvt, 284
galileo_pcps_8ms_acquisition_cc, 127
~galileo_pcps_8ms_acquisition_cc, 128
general_work, 128
init, 128
mag, 128
set_active, 128
set_channel, 129
set_doppler_max, 129
set_doppler_step, 129
set_gnss_synchro, 129
set_local_code, 129
set_state, 130
set_threshold, 130
galileo_satpos_X
Galileo_Navigation_Message, 123
galileo_satpos_Y
Galileo_Navigation_Message, 123
galileo_satpos_Z
Galileo_Navigation_Message, 124
galileo_satvel_X
Galileo_Navigation_Message, 124
galileo_satvel_Y
Galileo_Navigation_Message, 124
galileo_satvel_Z
Galileo_Navigation_Message, 124
GalileoE1BTelemetryDecoder, 132
implementation, 133
GalileoE1DllPllVemlTracking, 133
implementation, 134
set_channel, 134
set_gnss_synchro, 134
GalileoE1Observables, 134
implementation, 135
item_size, 135
GalileoE1Pcps8msAmbiguousAcquisition, 136
implementation, 137
init, 137
mag, 137
reset, 137
set_channel, 137
set_doppler_max, 137
set_doppler_step, 138
set_gnss_synchro, 138
set_local_code, 138
set_threshold, 138
GalileoE1PcpsAmbiguousAcquisition, 139
implementation, 140
init, 140
mag, 140
reset, 140
set_channel, 140
set_doppler_max, 140
set_doppler_step, 141
set_gnss_synchro, 141
set_local_code, 141
set_state, 141
set_threshold, 141
GalileoE1PcpsCccwsrAmbiguousAcquisition, 142
implementation, 143
init, 143
mag, 143
reset, 143
set_channel, 143
set_doppler_max, 143
set_doppler_step, 144
set_gnss_synchro, 144
set_state, 144
set_threshold, 144
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 145
implementation, 146
init, 146
mag, 146
reset, 146
set_channel, 146
set_doppler_max, 146
set_doppler_step, 147
set_gnss_synchro, 147
set_local_code, 147
set_state, 147
set_threshold, 147
GalileoE1PcpsTongAmbiguousAcquisition, 148
implementation, 149
init, 149
mag, 149
reset, 149
set_channel, 149
set_doppler_max, 149
set_doppler_step, 150
set_gnss_synchro, 150
set_local_code, 150
set_state, 150
set_threshold, 150
GalileoE1Pvt, 151
implementation, 151
item_size, 151
GalileoE1TcpConnectorTracking, 152
implementation, 153
set_channel, 153
set_gnss_synchro, 153
GalileoE5aDllPllTracking, 153
implementation, 154
set_channel, 154
set_gnss_synchro, 154
GalileoE5aNoncoherentIQAcquisitionCaf, 155
implementation, 156
init, 156
mag, 156
reset, 156
set_channel, 156
set_doppler_max, 157
set_doppler_step, 157
set_gnss_synchro, 157
set_local_code, 157
set_state, 157
set_threshold, 158

GalileoE5aObservables, 158
implementation, 159
item_size, 159
GalileoE5aTelemetryDecoder, 159
implementation, 160
GenSignalSource, 160
~GenSignalSource, 161
GenSignalSource, 161
implementation, 161
general_work
galileo_e1_dll_pll_veml_tracking_cc, 88
galileo_e1_pvt_cc, 91
galileo_e1b_telemetry_decoder_cc, 93
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 96
galileo_e5a_telemetry_decoder_cc, 99
galileo_pcps_8ms_acquisition_cc, 128
gps_l1_ca_pvt_cc, 219
gps_l1_ca_telemetry_decoder_cc, 221
gps_l2c_telemetry_decoder_cc, 223
hybrid_pvt_cc, 286
pcps_acquisition_cc, 309
pcps_acquisition_fine_doppler_cc, 313
pcps_acquisition_sc, 317
pcps_assisted_acquisition_cc, 320
pcps_cccwsr_acquisition_cc, 324
pcps_multithread_acquisition_cc, 328
pcps_opencl_acquisition_cc, 331
pcps_quicksync_acquisition_cc, 335
pcps_tong_acquisition_cc, 339
sbas_l1_telemetry_decoder_cc, 366
GeoJSON_Printer, 162
Get
INIReader, 294
get_GPS_L1_ephemeris_map
gps_l1_ca_pvt_cc, 219
hybrid_pvt_cc, 286
get_PRN
Gnss_Satellite, 164
get_block
Gnss_Satellite, 164
get_code_nco
Tracking_2nd_DLL_filter, 380
get_ephemeris
FrontEndCal, 85
Gps_CNAV_Navigation_Message, 196
Gps_Navigation_Message, 227
get_iono
Gps_CNAV_Navigation_Message, 196
Gps_Navigation_Message, 227
get_sample_stamp
Sbas_Raw_Msg, 368
get_satellite
Gnss_Signal, 168
get_signal_str
Gnss_Signal, 168
get_system
Gnss_Satellite, 164
get_system_short
Gnss_Satellite, 164
get_utc_model
Gps_CNAV_Navigation_Message, 196
Gps_Navigation_Message, 227
GetInteger
INIReader, 294
GM
GPS_L1_CA.h, 573
Gn3sSignalSource, 162
implementation, 163
Gnss_Satellite, 163
~Gnss_Satellite, 164
get_PRN, 164
get_block, 164
get_system, 164
get_system_short, 164
Gnss_Satellite, 164
operator<<, 165
operator==, 165
what_block, 165
Gnss_Signal, 168
get_satellite, 168
get_signal_str, 168
operator<<, 169
operator==, 169
Gnss_Synchro, 169
Acq_delay_samples, 170
Acq_doppler_hz, 170
Acq_samplestamp_samples, 170
CN0_dB_hz, 171
Carrier_Doppler_hz, 171
Carrier_phase_rads, 171
Channel_ID, 171
correlation_length_ms, 171
d_TOW, 171
Flag_preamble, 171
Flag_valid_acquisition, 171
Flag_valid_symbol_output, 172
Flag_valid_word, 172
PRN, 172
Prn_timestamp_at_preamble_ms, 172
Prn_timestamp_ms, 172
Prompt_I, 172
Prompt_Q, 172
Rem_code_phase_secs, 172
Signal, 173
System, 173
Tracking_timestamp_secs, 173
gnss_sdr_supl_client, 165
load_ephemeris_xml, 166
load_iono_xml, 166
load_ref_location_xml, 166
load_ref_time_xml, 166
load_utc_xml, 166
save_ephemeris_map_xml, 166
save_iono_map_xml, 166
save_ref_location_map_xml, 167

save_ref_time_map_xml, 167
save_utc_map_xml, 167
gnss_sdr_valve, 167
gnss_signal_processing.h
complex_exp_gen, 442
complex_exp_gen_conj, 442
hex_to_binary_converter, 443
resampler, 443
Gps_Acq_Assist, 177
Azimuth, 178
Code_Phase, 178
Code_Phase_int, 178
Code_Phase_window, 178
d_Doppler0, 178
d_Doppler1, 178
d_TOW, 178
dopplerUncertainty, 178
Elevation, 179
GPS_Bit_Number, 179
Gps_Acq_Assist, 177
i_satellite_PRN, 179
Gps_Almanac, 179
d_A_f0, 180
d_A_f1, 180
d_M_0, 181
d_OMEGA0, 181
d_OMEGA_DOT, 181
d_OMEGA, 181
d_Toa, 181
d_e_eccentricity, 181
d_sqrt_A, 181
Gps_Almanac, 180
i_satellite_PRN, 181
Gps_CNAV_Ephemeris, 182
b_alert_flag, 186
b_antispoofing_flag, 186
b_integrity_status_flag, 186
d_A_DOT, 186
d_A_f0, 186
d_A_f1, 187
d_A_f2, 187
d_Cic, 187
d_Cis, 187
d_Crc, 187
d_Crs, 187
d_Cuc, 187
d_Cus, 187
d_DELTA_DOT_N, 188
d_DELTA_OMEGA_DOT, 188
d_DELTA_A, 188
d_Delta_n, 188
d_IDOT, 188
d_M_0, 189
d_OMEGA0, 189
d_OMEGA, 189
d_TGD, 190
d_TOW, 191
d_Toc, 190
d_Toe1, 190
d_Toe2, 190
d_Top, 191
d_URA0, 191
d_URA1, 191
d_URA2, 191
d_dtr, 188
d_e_eccentricity, 188
d_i_0, 188
d_satClkDrift, 189
d_satpos_X, 189
d_satpos_Y, 189
d_satpos_Z, 189
d_satvel_X, 190
d_satvel_Y, 190
d_satvel_Z, 190
Gps_CNAV_Ephemeris, 184
i_GPS_week, 191
i_URA, 191
i_signal_health, 191
satellitePosition, 185
serialize, 185
sv_clock_drift, 185
sv_clock_relativistic_term, 186
Gps_CNAV_Iono, 192
d_alpha0, 193
d_alpha1, 193
d_alpha2, 193
d_alpha3, 193
d_beta0, 194
d_beta1, 194
d_beta2, 194
d_beta3, 194
Gps_CNAV_Iono, 193
serialize, 193
valid, 194
Gps_CNAV_Navigation_Message, 194
b_flag_iono_valid, 196
d_satpos_X, 196
d_satpos_Y, 196
d_satpos_Z, 197
d_satvel_X, 197
d_satvel_Y, 197
d_satvel_Z, 197
get_ephemeris, 196
get_iono, 196
get_utc_model, 196
Gps_CNAV_Navigation_Message, 196
have_new_ephemeris, 196
have_new_iono, 196
satelliteBlock, 197
Gps_CNAV_Utc_Model, 197
d_A0, 198
d_A1, 198
d_DeltaT_LSF, 199
d_DeltaT_LS, 199
d_t_OT, 199
Gps_CNAV_Utc_Model, 198

i_DN, 199
i_WN_LSF, 199
i_WN_T, 199
Gps_Ephemeris, 200
b_L2_P_data_flag, 204
b_alert_flag, 204
b_antispoofing_flag, 204
b_fit_interval_flag, 204
b_integrity_status_flag, 204
d_A_f0, 205
d_A_f1, 205
d_A_f2, 205
d_Cic, 205
d_Cis, 205
d_Crc, 205
d_Crs, 205
d_Cuc, 205
d_Cus, 206
d_Delta_n, 206
d_IDOT, 206
d_IODE_SF2, 206
d_IODE_SF3, 207
d_IODC, 206
d_M_0, 207
d_OMEGA0, 207
d_OMEGA_DOT, 207
d_OMEGA, 207
d_TGD, 208
d_TOW, 209
d_Toc, 208
d_Toe, 209
d_dtr, 206
d_e_eccentricity, 206
d_i_0, 206
d_satClkDrift, 207
d_satpos_X, 207
d_satpos_Y, 207
d_satpos_Z, 208
d_satvel_X, 208
d_satvel_Y, 208
d_satvel_Z, 208
d_sqrt_A, 208
Gps_Ephemeris, 202
i_AODO, 209
i_GPS_week, 209
i_SV_accuracy, 209
i_code_on_L2, 209
satelliteBlock, 209
satellitePosition, 202
serialize, 202
sv_clock_drift, 203
sv_clock_relativistic_term, 204
Gps_Iono, 210
d_alpha0, 211
d_alpha1, 211
d_alpha2, 211
d_alpha3, 211
d_beta0, 212
d_beta1, 212
d_beta2, 212
d_beta3, 212
Gps_Iono, 211
serialize, 211
valid, 212
Gps_L1_Ca_Dll_Pll_Tracking_GPU_cc, 215
Gps_L1_Ca_Dll_Pll_Tracking_cc, 214
Gps_L1_Ca_Tcp_Connector_Tracking_cc, 219
Gps_Navigation_Message, 223
almanacHealth, 228
b_L2_P_data_flag, 229
b_alert_flag, 228
b_antispoofing_flag, 228
b_fit_interval_flag, 228
b_integrity_status_flag, 228
d_A0, 229
d_A1, 229
d_A_f0, 229
d_A_f1, 229
d_A_f2, 229
d_Cic, 230
d_Cis, 231
d_Crc, 231
d_Crs, 231
d_Cuc, 231
d_Cus, 231
d_Delta_n, 231
d_DeltaT_LSF, 231
d_DeltaT_LS, 231
d_IDOT, 232
d_IODC, 232
d_M_0, 232
d_OMEGA0, 232
d_OMEGA_DOT, 232
d_OMEGA, 232
d_TGD, 234
d_TOW_SF1, 234
d_TOW_SF2, 234
d_TOW_SF3, 235
d_TOW_SF4, 235
d_TOW_SF5, 235
d_TOW, 234
d_Toa, 234
d_Toc, 234
d_Toe, 234
d_alpha0, 229
d_alpha1, 230
d_alpha2, 230
d_alpha3, 230
d_beta0, 230
d_beta1, 230
d_beta2, 230
d_beta3, 230
d_e_eccentricity, 232
d_i_0, 232
d_satpos_X, 233
d_satpos_Y, 233

d_satpos_Z, 233
d_satvel_X, 233
d_satvel_Y, 233
d_satvel_Z, 233
d_sqrt_A, 233
d_t_OT, 234
flag_iono_valid, 235
flag_utc_model_valid, 235
get_ephemeris, 227
get_iono, 227
get_utc_model, 227
Gps_Navigation_Message, 227
i_AODO, 235
i_DN, 236
i_GPS_week, 236
i_SV_accuracy, 236
i_WN_LSF, 236
i_WN_A, 236
i_WN_T, 236
i_code_on_L2, 235
satelliteBlock, 236
satellitePosition, 227
subframe_decoder, 228
sv_clock_correction, 228
utc_time, 228
Gps_Ref_Location, 237
Gps_Ref_Location, 237
serialize, 238
Gps_Ref_Time, 238
Gps_Ref_Time, 238
serialize, 239
Gps_Utc_Model, 239
d_A0, 240
d_A1, 240
d_DeltaT_LSF, 240
d_DeltaT_LS, 240
d_t_OT, 240
Gps_Utc_Model, 240
i_DN, 241
i_WN_LSF, 241
i_WN_T, 241
utc_time, 240
gps_ephemeris_map
gps_l1_ca_ls_pvt, 217
hybrid_ls_pvt, 284
gps_l1_ca_code_gen_complex
gps_sdr_signal_processing.h, 445
gps_l1_ca_code_gen_complex_sampled
gps_sdr_signal_processing.h, 445
gps_l1_ca_dll_pll_c_aid_tracking_cc, 213
gps_l1_ca_dll_pll_c_aid_tracking_sc, 213
gps_l1_ca_ls_pvt, 216
d_nchannels, 217
gps_ephemeris_map, 217
gps_l1_ca_observables_cc, 217
gps_l1_ca_pvt_cc, 218
~gps_l1_ca_pvt_cc, 219
general_work, 219
get_GPS_L1_ephemeris_map, 219
gps_l1_ca_telemetry_decoder_cc, 220
forecast, 221
general_work, 221
set_channel, 221
set_decimation, 221
set_satellite, 221
gps_l2_m_dll_pll_tracking_cc, 221
gps_l2c_m_code_gen_complex
gps_l2c_signal.h, 444
gps_l2c_m_code_gen_complex_sampled
gps_l2c_signal.h, 444
gps_l2c_signal.h
gps_l2c_m_code_gen_complex, 444
gps_l2c_m_code_gen_complex_sampled, 444
gps_l2c_telemetry_decoder_cc, 222
forecast, 223
general_work, 223
set_channel, 223
set_satellite, 223
gps_sdr_signal_processing.h
gps_l1_ca_code_gen_complex, 445
gps_l1_ca_code_gen_complex_sampled, 445
gps_subframe_to_nav_msg
GpsL1CaSubframeFsm, 271
gps_word_to_subframe
GpsL1CaSubframeFsm, 271
GpsL1CaDllPllCAidTracking, 241
implementation, 242
set_channel, 242
set_gnss_synchro, 242
GpsL1CaDllPllTracking, 243
implementation, 244
set_channel, 244
set_gnss_synchro, 244
GpsL1CaDllPllTrackingGPU, 244
implementation, 245
set_channel, 245
set_gnss_synchro, 245
GpsL1CaObservables, 246
implementation, 247
item_size, 247
GpsL1CaPcpsAcquisition, 247
implementation, 248
init, 248
mag, 249
reset, 249
set_channel, 249
set_doppler_max, 249
set_doppler_step, 249
set_gnss_synchro, 249
set_local_code, 250
set_state, 250
set_threshold, 250
GpsL1CaPcpsAcquisitionFineDoppler, 250
implementation, 251
init, 251
mag, 252

reset, 252
set_channel, 252
set_doppler_max, 252
set_doppler_step, 252
set_gnss_synchro, 252
set_threshold, 253
GpsL1CaPcpsAssistedAcquisition, 253
implementation, 254
init, 254
mag, 255
reset, 255
set_channel, 255
set_doppler_max, 255
set_doppler_step, 255
set_gnss_synchro, 255
set_threshold, 256
GpsL1CaPcpsMultithreadAcquisition, 256
implementation, 257
init, 257
mag, 257
reset, 258
set_channel, 258
set_doppler_max, 258
set_doppler_step, 258
set_gnss_synchro, 258
set_local_code, 258
set_threshold, 259
GpsL1CaPcpsOpenClAcquisition, 259
implementation, 260
init, 260
mag, 260
reset, 261
set_channel, 261
set_doppler_max, 261
set_doppler_step, 261
set_gnss_synchro, 261
set_local_code, 261
set_threshold, 262
GpsL1CaPcpsQuickSyncAcquisition, 262
implementation, 263
init, 263
mag, 263
reset, 264
set_channel, 264
set_doppler_max, 264
set_doppler_step, 264
set_gnss_synchro, 264
set_local_code, 264
set_state, 265
set_threshold, 265
GpsL1CaPcpsTongAcquisition, 265
implementation, 266
init, 266
mag, 267
reset, 267
set_channel, 267
set_doppler_max, 267
set_doppler_step, 267
set_gnss_synchro, 267
set_local_code, 268
set_state, 268
set_threshold, 268
GpsL1CaPvt, 268
implementation, 269
item_size, 269
GpsL1CaSubframeFsm, 270
almanac, 271
d_nav, 271
ephemeris, 272
Event_gps_word_invalid, 271
Event_gps_word_preamble, 271
Event_gps_word_valid, 271
gps_subframe_to_nav_msg, 271
gps_word_to_subframe, 271
GpsL1CaSubframeFsm, 271
i_channel_ID, 272
i_satellite_PRN, 272
iono, 272
utc_model, 272
GpsL1CaTcpConnectorTracking, 273
implementation, 273
set_channel, 273
set_gnss_synchro, 274
GpsL1CaTelemetryDecoder, 274
implementation, 275
GpsL2CObservables, 275
implementation, 276
item_size, 276
GpsL2CTelemetryDecoder, 276
implementation, 277
GpsL2MDllPllTracking, 277
implementation, 278
set_channel, 278
set_gnss_synchro, 278
GpsL2MPcpsAcquisition, 279
implementation, 280
init, 280
mag, 280
reset, 280
set_channel, 281
set_doppler_max, 281
set_doppler_step, 281
set_gnss_synchro, 281
set_local_code, 281
set_state, 281
set_threshold, 282

have_new_ephemeris
Gps_CNAV_Navigation_Message, 196
have_new_iono
Gps_CNAV_Navigation_Message, 196
hex_to_binary_converter
gnss_signal_processing.h, 443
hybrid_ls_pvt, 283
d_nchannels, 284
galileo_ephemeris_map, 284
gps_ephemeris_map, 284

hybrid_observables_cc, 285
hybrid_pvt_cc, 285
    ~hybrid_pvt_cc, 286
    general_work, 286
    get_GPS_L1_ephemeris_map, 286
HybridObservables, 287
    implementation, 287
    item_size, 287
HybridPvt, 288
    implementation, 289
    item_size, 289

i_0_2
    Galileo_Ephemeris, 106
    Galileo_Navigation_Message, 124
i_AODO
    Gps_Ephemeris, 209
    Gps_Navigation_Message, 235
i_DN
    Gps_CNAV_Utc_Model, 199
    Gps_Navigation_Message, 236
    Gps_Utc_Model, 241
i_GPS_week
    Gps_CNAV_Ephemeris, 191
    Gps_Ephemeris, 209
    Gps_Navigation_Message, 236
i_SV_accuracy
    Gps_Ephemeris, 209
    Gps_Navigation_Message, 236
i_URA
    Gps_CNAV_Ephemeris, 191
i_WN_LSF
    Gps_CNAV_Utc_Model, 199
    Gps_Navigation_Message, 236
    Gps_Utc_Model, 241
i_WN_A
    Gps_Navigation_Message, 236
i_WN_T
    Gps_CNAV_Utc_Model, 199
    Gps_Navigation_Message, 236
    Gps_Utc_Model, 241
i_channel_ID
    GpsL1CaSubframeFsm, 272
i_code_on_L2
    Gps_Ephemeris, 209
    Gps_Navigation_Message, 235
i_prn
    Sbas_Ephemeris, 364
i_satellite_PRN
    Galileo_Ephemeris, 106
    Gps_Acq_Assist, 179
    Gps_Almanac, 181
    GpsL1CaSubframeFsm, 272
i_signal_health
    Gps_CNAV_Ephemeris, 191
i_sv_ura
    Sbas_Ephemeris, 364
i_tapp
    Long_Term_Correction, 302
i_vel
    Long_Term_Correction, 302
iDot_2
    Galileo_Ephemeris, 106
    Galileo_Navigation_Message, 124
INIReader, 293
    Get, 294
    GetInteger, 294
    INIReader, 294
    ParseError, 294
IOD_nav_1
    Galileo_Navigation_Message, 124
IOD_nav_2
    Galileo_Navigation_Message, 125
IbyteToCbyte, 289
    implementation, 290
IbyteToComplex, 290
    implementation, 291
IbyteToCshort, 291
    implementation, 292
Igp, 292
Igp_Band, 293
implementation
    ArraySignalConditioner, 55
    BeamformerFilter, 57
    ByteToShort, 59
    Channel, 61
    DirectResamplerConditioner, 78
    FileSignalSource, 81
    FirFilter, 82
    FlexibandSignalSource, 83
    FreqXlatingFirFilter, 84
    GalileoE1BTelemetryDecoder, 133
    GalileoE1DllPllVemlTracking, 134
    GalileoE1Observables, 135
    GalileoE1Pcps8msAmbiguousAcquisition, 137
    GalileoE1PcpsAmbiguousAcquisition, 140
    GalileoE1PcpsCccwsrAmbiguousAcquisition, 143
    GalileoE1PcpsQuickSyncAmbiguousAcquisition, 146
    GalileoE1PcpsTongAmbiguousAcquisition, 149
    GalileoE1Pvt, 151
    GalileoE1TcpConnectorTracking, 153
    GalileoE5aDllPllTracking, 154
    GalileoE5aNoncoherentIQAcquisitionCaf, 156
    GalileoE5aObservables, 159
    GalileoE5aTelemetryDecoder, 160
    GenSignalSource, 161
    Gn3sSignalSource, 163
    GpsL1CaDllPllCAidTracking, 242
    GpsL1CaDllPllTracking, 244
    GpsL1CaDllPllTrackingGPU, 245
    GpsL1CaObservables, 247
    GpsL1CaPcpsAcquisition, 248
    GpsL1CaPcpsAcquisitionFineDoppler, 251
    GpsL1CaPcpsAssistedAcquisition, 254
    GpsL1CaPcpsMultithreadAcquisition, 257
    GpsL1CaPcpsOpenClAcquisition, 260

GpsL1CaPcpsQuickSyncAcquisition, 263
GpsL1CaPcpsTongAcquisition, 266
GpsL1CaPvt, 269
GpsL1CaTcpConnectorTracking, 273
GpsL1CaTelemetryDecoder, 275
GpsL2CObservables, 276
GpsL2CTelemetryDecoder, 277
GpsL2MDllPllTracking, 278
GpsL2MPcpsAcquisition, 280
HybridObservables, 287
HybridPvt, 289
IbyteToCbyte, 290
IbyteToComplex, 291
IbyteToCshort, 292
IshortToComplex, 298
IshortToCshort, 299
NsrFileSignalSource, 305
OsmosdrSignalSource, 307
Pass_Through, 308
RawArraySignalSource, 348
RtlTcpSignalSource, 362
SbasL1TelemetryDecoder, 371
SignalConditioner, 374
SignalGenerator, 375
SpirFileSignalSource, 376
TwoBitCpxFileSignalSource, 386
TwoBitPackedFileSignalSource, 387
UhdSignalSource, 388
InMemoryConfiguration, 294
init
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 96
galileo_pcps_8ms_acquisition_cc, 128
GalileoE1Pcps8msAmbiguousAcquisition, 137
GalileoE1PcpsAmbiguousAcquisition, 140
GalileoE1PcpsCccwsrAmbiguousAcquisition, 143
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 146
GalileoE1PcpsTongAmbiguousAcquisition, 149
GalileoE5aNoncoherentIQAcquisitionCaf, 156
GpsL1CaPcpsAcquisition, 248
GpsL1CaPcpsAcquisitionFineDoppler, 251
GpsL1CaPcpsAssistedAcquisition, 254
GpsL1CaPcpsMultithreadAcquisition, 257
GpsL1CaPcpsOpenClAcquisition, 260
GpsL1CaPcpsQuickSyncAcquisition, 263
GpsL1CaPcpsTongAcquisition, 266
GpsL2MPcpsAcquisition, 280
pcps_acquisition_cc, 309
pcps_acquisition_fine_doppler_cc, 313
pcps_acquisition_sc, 317
pcps_assisted_acquisition_cc, 320
pcps_cccwsr_acquisition_cc, 324
pcps_multithread_acquisition_cc, 328
pcps_opencl_acquisition_cc, 331
pcps_quicksync_acquisition_cc, 335
pcps_tong_acquisition_cc, 339
initialize
Tracking_2nd_DLL_filter, 380
Tracking_2nd_PLL_filter, 381
interleaved_byte_to_complex_byte, 295
interleaved_byte_to_complex_short, 296
interleaved_short_to_complex_short, 296
iono
GpsL1CaSubframeFsm, 272
IshortToComplex, 297
implementation, 298
IshortToCshort, 298
implementation, 299
item_size
GalileoE1Observables, 135
GalileoE1Pvt, 151
GalileoE5aObservables, 159
GpsL1CaObservables, 247
GpsL1CaPvt, 269
GpsL2CObservables, 276
HybridObservables, 287
HybridPvt, 289

kernel_info_t, 299
Kml_Printer, 299
Kml_Printer_gpstk, 300

leastSquarePos
Ls_Pvt, 303
load_ephemeris_xml
gnss_sdr_supl_client, 166
load_iono_xml
gnss_sdr_supl_client, 166
load_ref_location_xml
gnss_sdr_supl_client, 166
load_ref_time_xml
gnss_sdr_supl_client, 166
load_utc_xml
gnss_sdr_supl_client, 166
lock_detectors.h
carrier_lock_detector, 522
cn0_svn_estimator, 522
log_rinex_nav
Rinex_Printer, 351, 352
log_rinex_obs
Rinex_Printer, 352
log_rinex_sbs
Rinex_Printer, 352
Long_Term_Correction, 301
d_daf0, 301
d_daf1, 301
d_dpos, 301
d_dvel, 301
d_trx, 302
i_tapp, 302
i_vel, 302
Ls_Pvt, 302
bancroftPos, 303
leastSquarePos, 303

M0_1

Galileo_Ephemeris, 106
Galileo_Navigation_Message, 125
MATH_CONSTANTS.h
PI_TWO_N19, 588
PI_TWO_N23, 588
PI_TWO_N31, 588
PI_TWO_N38, 588
PI_TWO_N43, 588
TWO_N10, 589
TWO_N11, 589
TWO_N14, 589
TWO_N15, 589
TWO_N16, 589
TWO_N19, 589
TWO_N2, 589
TWO_N20, 589
TWO_N21, 590
TWO_N24, 590
TWO_N25, 590
TWO_N27, 590
TWO_N29, 590
TWO_N30, 590
TWO_N31, 590
TWO_N32, 590
TWO_N33, 591
TWO_N34, 591
TWO_N35, 591
TWO_N38, 591
TWO_N43, 591
TWO_N44, 591
TWO_N46, 591
TWO_N48, 591
TWO_N5, 592
TWO_N50, 592
TWO_N51, 592
TWO_N55, 592
TWO_N57, 592
TWO_N59, 592
TWO_N60, 592
TWO_N8, 592
TWO_N9, 593
TWO_P11, 593
TWO_P12, 593
TWO_P14, 593
TWO_P16, 593
TWO_P19, 593
TWO_P31, 593
TWO_P32, 593
TWO_P4, 594
TWO_P56, 594
TWO_P57, 594
MAX_TOA_DELAY_MS
GPS_L1_CA.h, 576
mag
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 96
galileo_pcps_8ms_acquisition_cc, 128
GalileoE1Pcps8msAmbiguousAcquisition, 137
GalileoE1PcpsAmbiguousAcquisition, 140
GalileoE1PcpsCccwsrAmbiguousAcquisition, 143
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 146
GalileoE1PcpsTongAmbiguousAcquisition, 149
GalileoE5aNoncoherentIQAcquisitionCaf, 156
GpsL1CaPcpsAcquisition, 249
GpsL1CaPcpsAcquisitionFineDoppler, 252
GpsL1CaPcpsAssistedAcquisition, 255
GpsL1CaPcpsMultithreadAcquisition, 257
GpsL1CaPcpsOpenClAcquisition, 260
GpsL1CaPcpsQuickSyncAcquisition, 263
GpsL1CaPcpsTongAcquisition, 267
GpsL2MPcpsAcquisition, 280
pcps_acquisition_cc, 310
pcps_acquisition_fine_doppler_cc, 313
pcps_acquisition_sc, 317
pcps_assisted_acquisition_cc, 321
pcps_cccwsr_acquisition_cc, 324
pcps_multithread_acquisition_cc, 328
pcps_opencl_acquisition_cc, 332
pcps_quicksync_acquisition_cc, 335
pcps_tong_acquisition_cc, 339

Nmea_Printer, 303
~Nmea_Printer, 304
Nmea_Printer, 304
Print_Nmea_Line, 304
NsrFileSignalSource, 304
implementation, 305

OMEGA_0_2
Galileo_Ephemeris, 106
Galileo_Navigation_Message, 125
OMEGA_EARTH_DOT
GPS_L1_CA.h, 576
OMEGA_dot_3
Galileo_Ephemeris, 107
Galileo_Navigation_Message, 125
ObservablesInterface, 305
omega_2
Galileo_Ephemeris, 106
Galileo_Navigation_Message, 125
operator<<
Gnss_Satellite, 165
Gnss_Signal, 169
operator==
Gnss_Satellite, 165
Gnss_Signal, 169
OsmosdrSignalSource, 306
implementation, 307

PI_TWO_N19
MATH_CONSTANTS.h, 588
PI_TWO_N23
MATH_CONSTANTS.h, 588
PI_TWO_N31
MATH_CONSTANTS.h, 588
PI_TWO_N38

MATH_CONSTANTS.h, 588
PI_TWO_N43
MATH_CONSTANTS.h, 588
PRN
Gnss_Synchro, 172
ParseError
INIReader, 294
Pass_Through, 307
implementation, 308
pcps_acquisition_cc, 308
~pcps_acquisition_cc, 309
general_work, 309
init, 309
mag, 310
set_active, 310
set_channel, 310
set_doppler_max, 310
set_doppler_step, 310
set_gnss_synchro, 311
set_local_code, 311
set_state, 311
set_threshold, 311
pcps_acquisition_fine_doppler_cc, 312
~pcps_acquisition_fine_doppler_cc, 313
general_work, 313
init, 313
mag, 313
set_active, 313
set_channel, 314
set_doppler_max, 314
set_doppler_step, 314
set_gnss_synchro, 314
set_local_code, 315
set_threshold, 315
pcps_acquisition_sc, 315
~pcps_acquisition_sc, 316
general_work, 317
init, 317
mag, 317
set_active, 317
set_channel, 317
set_doppler_max, 317
set_doppler_step, 318
set_gnss_synchro, 318
set_local_code, 318
set_state, 318
set_threshold, 319
pcps_assisted_acquisition_cc, 319
~pcps_assisted_acquisition_cc, 320
general_work, 320
init, 320
mag, 321
set_active, 321
set_channel, 321
set_doppler_max, 321
set_doppler_step, 321
set_gnss_synchro, 322
set_local_code, 322
set_threshold, 322
pcps_cccwsr_acquisition_cc, 323
~pcps_cccwsr_acquisition_cc, 324
general_work, 324
init, 324
mag, 324
set_active, 324
set_channel, 325
set_doppler_max, 325
set_doppler_step, 325
set_gnss_synchro, 325
set_local_code, 325
set_state, 326
set_threshold, 326
pcps_multithread_acquisition_cc, 326
~pcps_multithread_acquisition_cc, 327
general_work, 328
init, 328
mag, 328
set_active, 328
set_channel, 328
set_doppler_max, 328
set_doppler_step, 329
set_gnss_synchro, 329
set_local_code, 329
set_state, 329
set_threshold, 330
pcps_opencl_acquisition_cc, 330
~pcps_opencl_acquisition_cc, 331
general_work, 331
init, 331
mag, 332
set_active, 332
set_channel, 332
set_doppler_max, 332
set_doppler_step, 332
set_gnss_synchro, 333
set_local_code, 333
set_state, 333
set_threshold, 333
pcps_quicksync_acquisition_cc, 334
~pcps_quicksync_acquisition_cc, 335
general_work, 335
init, 335
mag, 335
set_active, 335
set_channel, 336
set_doppler_max, 336
set_doppler_step, 336
set_gnss_synchro, 336
set_local_code, 337
set_state, 337
set_threshold, 337
pcps_tong_acquisition_cc, 337
~pcps_tong_acquisition_cc, 339
general_work, 339
init, 339
mag, 339

set_active, 339
set_channel, 339
set_doppler_max, 340
set_doppler_step, 340
set_gnss_synchro, 340
set_local_code, 340
set_state, 341
set_threshold, 341
pll_cloop_two_quadrant_atan
tracking_discriminators.h, 527
pll_four_quadrant_atan
tracking_discriminators.h, 527
print_MSM_1
Rtcm, 356
print_MSM_2
Rtcm, 356
print_MSM_3
Rtcm, 357
print_MSM_4
Rtcm, 357
print_MSM_5
Rtcm, 357
print_MSM_6
Rtcm, 357
print_MSM_7
Rtcm, 357
print_MT1001
Rtcm, 357
print_MT1002
Rtcm, 357
print_MT1003
Rtcm, 358
print_MT1004
Rtcm, 358
print_MT1005
Rtcm, 358
print_MT1006
Rtcm, 358
print_MT1008
Rtcm, 358
print_MT1019
Rtcm, 358
print_MT1029
Rtcm, 358
print_MT1045
Rtcm, 358
Print_Nmea_Line
Nmea_Printer, 304
Prn_timestamp_at_preamble_ms
Gnss_Synchro, 172
Prn_timestamp_ms
Gnss_Synchro, 172
Prompt_I
Gnss_Synchro, 172
Prompt_Q
Gnss_Synchro, 172
Pvt_Solution, 341
cart2geo, 343
compute_DOP, 343
d_averaging_depth, 345
d_avg_height_m, 345
d_avg_latitude_d, 345
d_avg_longitude_d, 345
d_height_m, 345
d_latitude_d, 345
d_longitude_d, 345
d_rx_dt_s, 345
d_valid_observations, 346
d_visible_satellites_Az, 346
d_visible_satellites_CN0_dB, 346
d_visible_satellites_Distance, 346
d_visible_satellites_El, 346
d_visible_satellites_IDs, 346
togeod, 343
topocent, 344
tropo, 344
PvtInterface, 347

RTL_TCP_COMMAND
rtl_tcp_commands.h, 491
RawArraySignalSource, 347
implementation, 348
read_MT1005
Rtcm, 358
read_MT1019
Rtcm, 359
read_MT1045
Rtcm, 359
Region1_flag_5
Galileo_Iono, 113
Galileo_Navigation_Message, 125
Region2_flag_5
Galileo_Iono, 113
Galileo_Navigation_Message, 125
Region3_flag_5
Galileo_Iono, 113
Galileo_Navigation_Message, 125
Region4_flag_5
Galileo_Iono, 113
Galileo_Navigation_Message, 126
Region5_flag_5
Galileo_Iono, 114
Galileo_Navigation_Message, 126
Rem_code_phase_secs
Gnss_Synchro, 172
resampler
gnss_signal_processing.h, 443
reset
GalileoE1Pcps8msAmbiguousAcquisition, 137
GalileoE1PcpsAmbiguousAcquisition, 140
GalileoE1PcpsCccwsrAmbiguousAcquisition, 143
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 146
GalileoE1PcpsTongAmbiguousAcquisition, 149
GalileoE5aNoncoherentIQAcquisitionCaf, 156
GpsL1CaPcpsAcquisition, 249
GpsL1CaPcpsAcquisitionFineDoppler, 252

GpsL1CaPcpsAssistedAcquisition, 255
GpsL1CaPcpsMultithreadAcquisition, 258
GpsL1CaPcpsOpenClAcquisition, 261
GpsL1CaPcpsQuickSyncAcquisition, 264
GpsL1CaPcpsTongAcquisition, 267
GpsL2MPcpsAcquisition, 280
Rinex_Printer, 348
~Rinex_Printer, 351
compute_GPS_time, 351
compute_Galileo_time, 351
compute_UTC_time, 351
log_rinex_nav, 351, 352
log_rinex_obs, 352
log_rinex_sbs, 352
Rinex_Printer, 351
rinex_nav_header, 352, 353
rinex_obs_header, 353
rinex_sbs_header, 354
to_date_time, 354
rinex_nav_header
Rinex_Printer, 352, 353
rinex_obs_header
Rinex_Printer, 353
rinex_sbs_header
Rinex_Printer, 354
Rtcm, 354
print_MSM_1, 356
print_MSM_2, 356
print_MSM_3, 357
print_MSM_4, 357
print_MSM_5, 357
print_MSM_6, 357
print_MSM_7, 357
print_MT1001, 357
print_MT1002, 357
print_MT1003, 358
print_MT1004, 358
print_MT1005, 358
print_MT1006, 358
print_MT1008, 358
print_MT1019, 358
print_MT1029, 358
print_MT1045, 358
read_MT1005, 358
read_MT1019, 359
read_MT1045, 359
Rtcm_Printer, 359
~Rtcm_Printer, 360
Rtcm_Printer, 360
rtl_tcp_command
rtl_tcp_commands.h, 491
rtl_tcp_commands.h
RTL_TCP_COMMAND, 491
rtl_tcp_command, 491
rtl_tcp_dongle_info, 360
rtl_tcp_signal_source_c, 361
RtlTcpSignalSource, 362
implementation, 362
run
ControlThread, 72
satelliteBlock
Gps_CNAV_Navigation_Message, 197
Gps_Ephemeris, 209
Gps_Navigation_Message, 236
satellitePosition
Galileo_Ephemeris, 102
Gps_CNAV_Ephemeris, 185
Gps_Ephemeris, 202
Gps_Navigation_Message, 227
save_ephemeris_map_xml
gnss_sdr_supl_client, 166
save_iono_map_xml
gnss_sdr_supl_client, 166
save_ref_location_map_xml
gnss_sdr_supl_client, 167
save_ref_time_map_xml
gnss_sdr_supl_client, 167
save_utc_map_xml
gnss_sdr_supl_client, 167
Sbas_Ephemeris, 363
b_sv_do_not_use, 363
d_acc, 363
d_af0, 364
d_af1, 364
d_pos, 364
d_vel, 364
i_prn, 364
i_sv_ura, 364
Sbas_Ionosphere_Correction, 365
apply, 365
Sbas_Raw_Msg, 367
get_sample_stamp, 368
Sbas_Satellite_Correction, 368
Sbas_Telemetry_Data, 369
~Sbas_Telemetry_Data, 369
Sbas_Telemetry_Data, 369
Sbas_Time, 369
Sbas_Time_Relation, 370
sbas_l1_telemetry_decoder_cc, 366
forecast, 366
general_work, 366
set_channel, 367
set_satellite, 367
SbasL1TelemetryDecoder, 370
implementation, 371
send_telemetry_msg
GNSSFlowgraph, 176
serialize
Galileo_Ephemeris, 102
Gps_CNAV_Ephemeris, 185
Gps_CNAV_Iono, 193
Gps_Ephemeris, 202
Gps_Iono, 211
Gps_Ref_Location, 238
Gps_Ref_Time, 239
set_active

galileo_e5a_noncoherentIQ_acquisition_caf_cc, 96
galileo_pcps_8ms_acquisition_cc, 128
pcps_acquisition_cc, 310
pcps_acquisition_fine_doppler_cc, 313
pcps_acquisition_sc, 317
pcps_assisted_acquisition_cc, 321
pcps_cccwsr_acquisition_cc, 324
pcps_multithread_acquisition_cc, 328
pcps_opencl_acquisition_cc, 332
pcps_quicksync_acquisition_cc, 335
pcps_tong_acquisition_cc, 339
set_channel
galileo_e1b_telemetry_decoder_cc, 93
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 96
galileo_e5a_telemetry_decoder_cc, 99
galileo_pcps_8ms_acquisition_cc, 129
GalileoE1DllPllVemlTracking, 134
GalileoE1Pcps8msAmbiguousAcquisition, 137
GalileoE1PcpsAmbiguousAcquisition, 140
GalileoE1PcpsCccwsrAmbiguousAcquisition, 143
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 146
GalileoE1PcpsTongAmbiguousAcquisition, 149
GalileoE1TcpConnectorTracking, 153
GalileoE5aDllPllTracking, 154
GalileoE5aNoncoherentIQAcquisitionCaf, 156
gps_l1_ca_telemetry_decoder_cc, 221
gps_l2c_telemetry_decoder_cc, 223
GpsL1CaDllPllCAidTracking, 242
GpsL1CaDllPllTracking, 244
GpsL1CaDllPllTrackingGPU, 245
GpsL1CaPcpsAcquisition, 249
GpsL1CaPcpsAcquisitionFineDoppler, 252
GpsL1CaPcpsAssistedAcquisition, 255
GpsL1CaPcpsMultithreadAcquisition, 258
GpsL1CaPcpsOpenClAcquisition, 261
GpsL1CaPcpsQuickSyncAcquisition, 264
GpsL1CaPcpsTongAcquisition, 267
GpsL1CaTcpConnectorTracking, 273
GpsL2MDllPllTracking, 278
GpsL2MPcpsAcquisition, 281
pcps_acquisition_cc, 310
pcps_acquisition_fine_doppler_cc, 314
pcps_acquisition_sc, 317
pcps_assisted_acquisition_cc, 321
pcps_cccwsr_acquisition_cc, 325
pcps_multithread_acquisition_cc, 328
pcps_opencl_acquisition_cc, 332
pcps_quicksync_acquisition_cc, 336
pcps_tong_acquisition_cc, 339
sbas_l1_telemetry_decoder_cc, 367
set_configuration
FrontEndCal, 85
set_control_queue
ControlThread, 72
set_decimation
galileo_e1b_telemetry_decoder_cc, 93
gps_l1_ca_telemetry_decoder_cc, 221
set_doppler_max
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 96
galileo_pcps_8ms_acquisition_cc, 129
GalileoE1Pcps8msAmbiguousAcquisition, 137
GalileoE1PcpsAmbiguousAcquisition, 140
GalileoE1PcpsCccwsrAmbiguousAcquisition, 143
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 146
GalileoE1PcpsTongAmbiguousAcquisition, 149
GalileoE5aNoncoherentIQAcquisitionCaf, 157
GpsL1CaPcpsAcquisition, 249
GpsL1CaPcpsAcquisitionFineDoppler, 252
GpsL1CaPcpsAssistedAcquisition, 255
GpsL1CaPcpsMultithreadAcquisition, 258
GpsL1CaPcpsOpenClAcquisition, 261
GpsL1CaPcpsQuickSyncAcquisition, 264
GpsL1CaPcpsTongAcquisition, 267
GpsL2MPcpsAcquisition, 281
pcps_acquisition_cc, 310
pcps_acquisition_fine_doppler_cc, 314
pcps_acquisition_sc, 317
pcps_assisted_acquisition_cc, 321
pcps_cccwsr_acquisition_cc, 325
pcps_multithread_acquisition_cc, 328
pcps_opencl_acquisition_cc, 332
pcps_quicksync_acquisition_cc, 336
pcps_tong_acquisition_cc, 340
set_doppler_step
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 97
galileo_pcps_8ms_acquisition_cc, 129
GalileoE1Pcps8msAmbiguousAcquisition, 138
GalileoE1PcpsAmbiguousAcquisition, 141
GalileoE1PcpsCccwsrAmbiguousAcquisition, 144
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 147
GalileoE1PcpsTongAmbiguousAcquisition, 150
GalileoE5aNoncoherentIQAcquisitionCaf, 157
GpsL1CaPcpsAcquisition, 249
GpsL1CaPcpsAcquisitionFineDoppler, 252
GpsL1CaPcpsAssistedAcquisition, 255
GpsL1CaPcpsMultithreadAcquisition, 258
GpsL1CaPcpsOpenClAcquisition, 261
GpsL1CaPcpsQuickSyncAcquisition, 264
GpsL1CaPcpsTongAcquisition, 267
GpsL2MPcpsAcquisition, 281
pcps_acquisition_cc, 310
pcps_acquisition_fine_doppler_cc, 314
pcps_acquisition_sc, 318
pcps_assisted_acquisition_cc, 321
pcps_cccwsr_acquisition_cc, 325
pcps_multithread_acquisition_cc, 329
pcps_opencl_acquisition_cc, 332
pcps_quicksync_acquisition_cc, 336
pcps_tong_acquisition_cc, 340

set_gnss_synchro
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 97
galileo_pcps_8ms_acquisition_cc, 129
GalileoE1DllPllVemlTracking, 134
GalileoE1Pcps8msAmbiguousAcquisition, 138
GalileoE1PcpsAmbiguousAcquisition, 141
GalileoE1PcpsCccwsrAmbiguousAcquisition, 144
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 147
GalileoE1PcpsTongAmbiguousAcquisition, 150
GalileoE1TcpConnectorTracking, 153
GalileoE5aDllPllTracking, 154
GalileoE5aNoncoherentIQAcquisitionCaf, 157
GpsL1CaDllPllCAidTracking, 242
GpsL1CaDllPllTracking, 244
GpsL1CaDllPllTrackingGPU, 245
GpsL1CaPcpsAcquisition, 249
GpsL1CaPcpsAcquisitionFineDoppler, 252
GpsL1CaPcpsAssistedAcquisition, 255
GpsL1CaPcpsMultithreadAcquisition, 258
GpsL1CaPcpsOpenClAcquisition, 261
GpsL1CaPcpsQuickSyncAcquisition, 264
GpsL1CaPcpsTongAcquisition, 267
GpsL1CaTcpConnectorTracking, 274
GpsL2MDllPllTracking, 278
GpsL2MPcpsAcquisition, 281
pcps_acquisition_cc, 311
pcps_acquisition_fine_doppler_cc, 314
pcps_acquisition_sc, 318
pcps_assisted_acquisition_cc, 322
pcps_cccwsr_acquisition_cc, 325
pcps_multithread_acquisition_cc, 329
pcps_opencl_acquisition_cc, 333
pcps_quicksync_acquisition_cc, 336
pcps_tong_acquisition_cc, 340
set_local_code
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 97
galileo_pcps_8ms_acquisition_cc, 129
GalileoE1Pcps8msAmbiguousAcquisition, 138
GalileoE1PcpsAmbiguousAcquisition, 141
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 147
GalileoE1PcpsTongAmbiguousAcquisition, 150
GalileoE5aNoncoherentIQAcquisitionCaf, 157
GpsL1CaPcpsAcquisition, 250
GpsL1CaPcpsMultithreadAcquisition, 258
GpsL1CaPcpsOpenClAcquisition, 261
GpsL1CaPcpsQuickSyncAcquisition, 264
GpsL1CaPcpsTongAcquisition, 268
GpsL2MPcpsAcquisition, 281
pcps_acquisition_cc, 311
pcps_acquisition_fine_doppler_cc, 315
pcps_acquisition_sc, 318
pcps_assisted_acquisition_cc, 322
pcps_cccwsr_acquisition_cc, 325
pcps_multithread_acquisition_cc, 329
pcps_opencl_acquisition_cc, 333
pcps_quicksync_acquisition_cc, 337
pcps_tong_acquisition_cc, 340
set_pdi
Tracking_2nd_DLL_filter, 380
Tracking_2nd_PLL_filter, 381
set_satellite
galileo_e1b_telemetry_decoder_cc, 93
galileo_e5a_telemetry_decoder_cc, 99
gps_l1_ca_telemetry_decoder_cc, 221
gps_l2c_telemetry_decoder_cc, 223
sbas_l1_telemetry_decoder_cc, 367
set_signal
Channel, 61
set_state
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 97
galileo_pcps_8ms_acquisition_cc, 130
GalileoE1PcpsAmbiguousAcquisition, 141
GalileoE1PcpsCccwsrAmbiguousAcquisition, 144
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 147
GalileoE1PcpsTongAmbiguousAcquisition, 150
GalileoE5aNoncoherentIQAcquisitionCaf, 157
GpsL1CaPcpsAcquisition, 250
GpsL1CaPcpsQuickSyncAcquisition, 265
GpsL1CaPcpsTongAcquisition, 268
GpsL2MPcpsAcquisition, 281
pcps_acquisition_cc, 311
pcps_acquisition_sc, 318
pcps_cccwsr_acquisition_cc, 326
pcps_multithread_acquisition_cc, 329
pcps_opencl_acquisition_cc, 333
pcps_quicksync_acquisition_cc, 337
pcps_tong_acquisition_cc, 341
set_threshold
galileo_e5a_noncoherentIQ_acquisition_caf_cc, 98
galileo_pcps_8ms_acquisition_cc, 130
GalileoE1Pcps8msAmbiguousAcquisition, 138
GalileoE1PcpsAmbiguousAcquisition, 141
GalileoE1PcpsCccwsrAmbiguousAcquisition, 144
GalileoE1PcpsQuickSyncAmbiguousAcquisition, 147
GalileoE1PcpsTongAmbiguousAcquisition, 150
GalileoE5aNoncoherentIQAcquisitionCaf, 158
GpsL1CaPcpsAcquisition, 250
GpsL1CaPcpsAcquisitionFineDoppler, 253
GpsL1CaPcpsAssistedAcquisition, 256
GpsL1CaPcpsMultithreadAcquisition, 259
GpsL1CaPcpsOpenClAcquisition, 262
GpsL1CaPcpsQuickSyncAcquisition, 265
GpsL1CaPcpsTongAcquisition, 268
GpsL2MPcpsAcquisition, 282
pcps_acquisition_cc, 311
pcps_acquisition_fine_doppler_cc, 315
pcps_acquisition_sc, 319
pcps_assisted_acquisition_cc, 322

pcps_cccwsr_acquisition_cc, 326
pcps_multithread_acquisition_cc, 330
pcps_opencl_acquisition_cc, 333
pcps_quicksync_acquisition_cc, 337
pcps_tong_acquisition_cc, 341
short_x2_to_cshort, 371
Signal
Gnss_Synchro, 173
signal_generator_c, 372
signal_make_generator_c, 373
signal_generator_c.h
signal_make_generator_c, 473
signal_make_generator_c
signal_generator_c, 373
signal_generator_c.h, 473
SignalConditioner, 373
~SignalConditioner, 374
implementation, 374
SignalConditioner, 374
SignalGenerator, 375
implementation, 375
SpirFileSignalSource, 376
implementation, 376
src/algorithms/PVT/adapters/galileo_e1_pvt.h, 453
src/algorithms/PVT/adapters/gps_l1_ca_pvt.h, 454
src/algorithms/PVT/adapters/hybrid_pvt.h, 455
src/algorithms/PVT/gnuradio_blocks/galileo_e1_pvt_cc.h, 456
src/algorithms/PVT/gnuradio_blocks/gps_l1_ca_pvt_cc.h, 457
src/algorithms/PVT/gnuradio_blocks/hybrid_pvt_cc.h, 458
src/algorithms/PVT/libs/galileo_e1_ls_pvt.h, 459
src/algorithms/PVT/libs/geojson_printer.h, 459
src/algorithms/PVT/libs/gps_l1_ca_ls_pvt.h, 460
src/algorithms/PVT/libs/kml_printer.h, 461
src/algorithms/PVT/libs/ls_pvt.h, 462
src/algorithms/PVT/libs/nmea_printer.h, 463
src/algorithms/PVT/libs/pvt_solution.h, 464
src/algorithms/PVT/libs/rinex_printer.h, 465
src/algorithms/PVT/libs/rtcm_printer.h, 466
src/algorithms/acquisition/adapters/galileo_e1_pcps_8ms_ambiguous_acquisition.h, 393
src/algorithms/acquisition/adapters/galileo_e1_pcps_ambiguous_acquisition.h, 394
src/algorithms/acquisition/adapters/galileo_e1_pcps_cccwsr_ambiguous_acquisition.h, 395
src/algorithms/acquisition/adapters/galileo_e1_pcps_quicksync_ambiguous_acquisition.h, 395
src/algorithms/acquisition/adapters/galileo_e1_pcps_tong_ambiguous_acquisition.h, 396
src/algorithms/acquisition/adapters/galileo_e5a_noncoherent_iq_acquisition_caf.h, 397
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition.h, 398
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_acquisition_fine_doppler.h, 399
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_assisted_acquisition.h, 400
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_multithread_acquisition.h, 401
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_opencl_acquisition.h, 402
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_quicksync_acquisition.h, 403
src/algorithms/acquisition/adapters/gps_l1_ca_pcps_tong_acquisition.h, 404
src/algorithms/acquisition/adapters/gps_l2_m_pcps_acquisition.h, 405
src/algorithms/acquisition/gnuradio_blocks/galileo_e5a_noncoherent_iq_acquisition_caf_cc.h, 406
src/algorithms/acquisition/gnuradio_blocks/galileo_pcps_8ms_acquisition_cc.h, 407
src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_cc.h, 408
src/algorithms/acquisition/gnuradio_blocks/pcps_acquisition_sc.h, 410
src/algorithms/acquisition/gnuradio_blocks/pcps_assisted_acquisition_cc.h, 411
src/algorithms/acquisition/gnuradio_blocks/pcps_cccwsr_acquisition_cc.h, 412
src/algorithms/acquisition/gnuradio_blocks/pcps_multithread_acquisition_cc.h, 414
src/algorithms/acquisition/gnuradio_blocks/pcps_opencl_acquisition_cc.h, 415
src/algorithms/acquisition/gnuradio_blocks/pcps_quicksync_acquisition_cc.h, 417
src/algorithms/acquisition/gnuradio_blocks/pcps_tong_acquisition_cc.h, 418
src/algorithms/channel/adapters/channel.h, 420
src/algorithms/channel/libs/channel_fsm.h, 421
src/algorithms/channel/libs/channel_msg_receiver_cc.h, 421
src/algorithms/conditioner/adapters/array_signal_conditioner.h, 422
src/algorithms/conditioner/adapters/signal_conditioner.h, 423
src/algorithms/data_type_adapter/adapters/byte_to_short.h, 424
src/algorithms/data_type_adapter/adapters/ibyte_to_cbyte.h, 425
src/algorithms/data_type_adapter/adapters/ibyte_to_complex.h, 425
src/algorithms/data_type_adapter/adapters/ibyte_to_cshort.h, 426
src/algorithms/data_type_adapter/adapters/ishort_to_complex.h, 427
src/algorithms/data_type_adapter/adapters/ishort_to_cshort.h, 428
src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_byte.h, 428
src/algorithms/data_type_adapter/gnuradio_blocks/interleaved_byte_to_complex_short.h, 429
src/algorithms/data_type_adapter/gnuradio_blocks/interleaved

_short_to_complex_short.h, 430
src/algorithms/input_filter/adapters/beamformer_filter.h, 431
src/algorithms/input_filter/adapters/fir_filter.h, 432
src/algorithms/input_filter/adapters/freq_xlating_fir_filter.h, 433
src/algorithms/input_filter/gnuradio_blocks/beamformer.h, 434
src/algorithms/libs/byte_x2_to_complex_byte.h, 435
src/algorithms/libs/complex_byte_to_float_x2.h, 436
src/algorithms/libs/complex_float_to_complex_byte.h, 437
src/algorithms/libs/cshort_to_float_x2.h, 438
src/algorithms/libs/galileo_e1_signal_processing.h, 439
src/algorithms/libs/gnss_sdr_valve.h, 441
src/algorithms/libs/gnss_signal_processing.h, 442
src/algorithms/libs/gps_l2c_signal.h, 443
src/algorithms/libs/gps_sdr_signal_processing.h, 444
src/algorithms/libs/pass_through.h, 445
src/algorithms/libs/short_x2_to_cshort.h, 446
src/algorithms/observables/adapters/galileo_e1_observables.h, 447
src/algorithms/observables/adapters/galileo_e5a_observables.h, 447
src/algorithms/observables/adapters/gps_l1_ca_observables.h, 448
src/algorithms/observables/adapters/gps_l2c_observables.h, 449
src/algorithms/observables/adapters/hybrid_observables.h, 450
src/algorithms/observables/gnuradio_blocks/galileo_e1_observables_cc.h, 450
src/algorithms/observables/gnuradio_blocks/gps_l1_ca_observables_cc.h, 451
src/algorithms/observables/gnuradio_blocks/hybrid_observables_cc.h, 452
src/algorithms/resampler/adapters/direct_resampler_conditioner.h, 467
src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cb.h, 468
src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cc.h, 469
src/algorithms/resampler/gnuradio_blocks/direct_resampler_conditioner_cs.h, 470
src/algorithms/signal_generator/adapters/signal_generator.h, 471
src/algorithms/signal_generator/gnuradio_blocks/signal_generator_c.h, 472
src/algorithms/signal_source/adapters/file_signal_source.h, 474
src/algorithms/signal_source/adapters/gen_signal_source.h, 474
src/algorithms/signal_source/adapters/gn3s_signal_source.h, 475
src/algorithms/signal_source/adapters/nsr_file_signal_source.h, 476
src/algorithms/signal_source/adapters/osmosdr_signal_source.h, 477
src/algorithms/signal_source/adapters/raw_array_signal_source.h, 478
src/algorithms/signal_source/adapters/rtl_tcp_signal_source.h, 479
src/algorithms/signal_source/adapters/spir_file_signal_source.h, 480
src/algorithms/signal_source/adapters/two_bit_cpx_file_signal_source.h, 481
src/algorithms/signal_source/adapters/two_bit_packed_file_signal_source.h, 482
src/algorithms/signal_source/adapters/uhd_signal_source.h, 483
src/algorithms/signal_source/gnuradio_blocks/rtl_tcp_signal_source_c.h, 484
src/algorithms/signal_source/gnuradio_blocks/unpack_2bit_samples.h, 485
src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_cpx_samples.h, 487
src/algorithms/signal_source/gnuradio_blocks/unpack_byte_2bit_samples.h, 488
src/algorithms/signal_source/gnuradio_blocks/unpack_intspir_1bit_samples.h, 489
src/algorithms/signal_source/libs/rtl_tcp_commands.h, 490
src/algorithms/signal_source/libs/rtl_tcp_dongle_info.h, 491
src/algorithms/telemetry_decoder/adapters/galileo_e1b_telemetry_decoder.h, 492
src/algorithms/telemetry_decoder/adapters/galileo_e5a_telemetry_decoder.h, 492
src/algorithms/telemetry_decoder/adapters/gps_l1_ca_telemetry_decoder.h, 493
src/algorithms/telemetry_decoder/adapters/gps_l2c_telemetry_decoder.h, 494
src/algorithms/telemetry_decoder/adapters/sbas_l1_telemetry_decoder.h, 495
src/algorithms/telemetry_decoder/gnuradio_blocks/galileo_e1b_telemetry_decoder_cc.h, 495
src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l1_ca_telemetry_decoder_cc.h, 496
src/algorithms/telemetry_decoder/gnuradio_blocks/gps_l2c_telemetry_decoder_cc.h, 497
src/algorithms/telemetry_decoder/gnuradio_blocks/sbas_l1_telemetry_decoder_cc.h, 498
src/algorithms/telemetry_decoder/libs/convolutional.h, 499
src/algorithms/telemetry_decoder/libs/gps_l1_ca_subframe_fsm.h, 500
src/algorithms/telemetry_decoder/libs/viterbi_decoder.h, 501
src/algorithms/tracking/adapters/galileo_e1_dll_pll_veml_tracking.h, 502
src/algorithms/tracking/adapters/galileo_e1_tcp_connector_tracking.h, 502
src/algorithms/tracking/adapters/galileo_e5a_dll_pll_tracking.h, 503
src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_c_aid_tracking.h, 504

src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_↩
tracking.h, 505
src/algorithms/tracking/adapters/gps_l1_ca_dll_pll_↩
tracking_gpu.h, 506
src/algorithms/tracking/adapters/gps_l1_ca_tcp_↩
connector_tracking.h, 507
src/algorithms/tracking/adapters/gps_l2_m_dll_pll_↩
tracking.h, 508
src/algorithms/tracking/gnuradio_blocks/galileo_e1_↩
dll_pll_veml_tracking_cc.h, 509
src/algorithms/tracking/gnuradio_blocks/galileo_e1_↩
tcp_connector_tracking_cc.h, 510
src/algorithms/tracking/gnuradio_blocks/galileo_e5a_↩
dll_pll_tracking_cc.h, 511
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_↩
dll_pll_c_aid_tracking_cc.h, 512
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_↩
dll_pll_c_aid_tracking_sc.h, 514
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_↩
dll_pll_tracking_cc.h, 515
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_↩
dll_pll_tracking_gpu_cc.h, 516
src/algorithms/tracking/gnuradio_blocks/gps_l1_ca_↩
tcp_connector_tracking_cc.h, 517
src/algorithms/tracking/gnuradio_blocks/gps_l2_m_dll↩
_pll_tracking_cc.h, 518
src/algorithms/tracking/libs/cpu_multicorrelator.h, 519
src/algorithms/tracking/libs/cpu_multicorrelator_16sc.h,
519
src/algorithms/tracking/libs/cuda_multicorrelator.h, 520
src/algorithms/tracking/libs/lock_detectors.h, 521
src/algorithms/tracking/libs/tcp_communication.h, 523
src/algorithms/tracking/libs/tcp_packet_data.h, 523
src/algorithms/tracking/libs/tracking_2nd_DLL_filter.h,
524
src/algorithms/tracking/libs/tracking_2nd_PLL_filter.h,
525
src/algorithms/tracking/libs/tracking_FLL_PLL_filter.h,
528
src/algorithms/tracking/libs/tracking_discriminators.h,
526
src/algorithms/tracking/libs/tracking_loop_filter.h, 528
src/core/interfaces/acquisition_interface.h, 529
src/core/interfaces/channel_interface.h, 530
src/core/interfaces/configuration_interface.h, 531
src/core/interfaces/gnss_block_interface.h, 532
src/core/interfaces/observables_interface.h, 533
src/core/interfaces/pvt_interface.h, 534
src/core/interfaces/telemetry_decoder_interface.h, 535
src/core/interfaces/tracking_interface.h, 536
src/core/libs/INIReader.h, 539
src/core/libs/gnss_sdr_supl_client.h, 537
src/core/libs/ini.h, 538
src/core/libs/string_converter.h, 540
src/core/receiver/concurrent_map.h, 541
src/core/receiver/concurrent_queue.h, 542
src/core/receiver/control_message.h, 543
src/core/receiver/control_message_factory.h, 543
src/core/receiver/control_thread.h, 544
src/core/receiver/file_configuration.h, 545
src/core/receiver/gnss_block_factory.h, 546
src/core/receiver/gnss_flowgraph.h, 547
src/core/receiver/in_memory_configuration.h, 548
src/core/system_parameters/GPS_L1_CA.h, 569
src/core/system_parameters/GPS_L2C.h, 576
src/core/system_parameters/Galileo_E1.h, 550
src/core/system_parameters/MATH_CONSTANTS.h,
585
src/core/system_parameters/galileo_almanac.h, 549
src/core/system_parameters/galileo_ephemeris.h, 557
src/core/system_parameters/galileo_fnav_message.h,
558
src/core/system_parameters/galileo_iono.h, 559
src/core/system_parameters/galileo_navigation_↩
message.h, 560
src/core/system_parameters/galileo_utc_model.h, 561
src/core/system_parameters/gnss_satellite.h, 562
src/core/system_parameters/gnss_signal.h, 562
src/core/system_parameters/gnss_synchro.h, 563
src/core/system_parameters/gps_acq_assist.h, 564
src/core/system_parameters/gps_almanac.h, 565
src/core/system_parameters/gps_cnav_ephemeris.h,
565
src/core/system_parameters/gps_cnav_iono.h, 566
src/core/system_parameters/gps_cnav_navigation_↩
message.h, 567
src/core/system_parameters/gps_ephemeris.h, 567
src/core/system_parameters/gps_iono.h, 568
src/core/system_parameters/gps_navigation_message.↩
h, 582
src/core/system_parameters/gps_ref_location.h, 583
src/core/system_parameters/gps_ref_time.h, 584
src/core/system_parameters/gps_utc_model.h, 585
src/core/system_parameters/rtcm.h, 594
src/core/system_parameters/sbas_ephemeris.h, 595
src/core/system_parameters/sbas_ionospheric_↩
correction.h, 596
src/core/system_parameters/sbas_satellite_correction.↩
h, 597
src/core/system_parameters/sbas_telemetry_data.h,
597
src/core/system_parameters/sbas_time.h, 599
src/tests/common-files/signal_generator_flags.h, 600
src/tests/unit-tests/signal-processing-blocks/libs/tlm_↩
dump_reader.h, 601
src/tests/unit-tests/signal-processing-blocks/libs/tracking↩
_dump_reader.h, 602
src/tests/unit-tests/signal-processing-blocks/libs/tracking↩
_true_obs_reader.h, 602
src/utils/front-end-cal/front_end_cal.h, 603
start
GNSSFlowgraph, 176
start_acquisition
Channel, 61
stop
GNSSFlowgraph, 176

StringConverter, 377
subframe_decoder
Gps_Navigation_Message, 228
sv_clock_correction
Gps_Navigation_Message, 228
sv_clock_drift
Galileo_Ephemeris, 102
Gps_CNAV_Ephemeris, 185
Gps_Ephemeris, 203
sv_clock_relativistic_term
Galileo_Ephemeris, 102
Gps_CNAV_Ephemeris, 186
Gps_Ephemeris, 204
System
Gnss_Synchro, 173

t0c_4
Galileo_Ephemeris, 107
Galileo_Navigation_Message, 126
t0e_1
Galileo_Ephemeris, 107
Galileo_Navigation_Message, 126
t0t_6
Galileo_Utc_Model, 131
t_0G_10
Galileo_Navigation_Message, 126
T_OA
GPS_L1_CA.h, 573
TOW_5
Galileo_Ephemeris, 107
Galileo_Iono, 114
TWO_N10
MATH_CONSTANTS.h, 589
TWO_N11
MATH_CONSTANTS.h, 589
TWO_N14
MATH_CONSTANTS.h, 589
TWO_N15
MATH_CONSTANTS.h, 589
TWO_N16
MATH_CONSTANTS.h, 589
TWO_N19
MATH_CONSTANTS.h, 589
TWO_N2
MATH_CONSTANTS.h, 589
TWO_N20
MATH_CONSTANTS.h, 589
TWO_N21
MATH_CONSTANTS.h, 590
TWO_N24
MATH_CONSTANTS.h, 590
TWO_N25
MATH_CONSTANTS.h, 590
TWO_N27
MATH_CONSTANTS.h, 590
TWO_N29
MATH_CONSTANTS.h, 590
TWO_N30
MATH_CONSTANTS.h, 590
TWO_N31
MATH_CONSTANTS.h, 590
TWO_N32
MATH_CONSTANTS.h, 590
TWO_N33
MATH_CONSTANTS.h, 591
TWO_N34
MATH_CONSTANTS.h, 591
TWO_N35
MATH_CONSTANTS.h, 591
TWO_N38
MATH_CONSTANTS.h, 591
TWO_N43
MATH_CONSTANTS.h, 591
TWO_N44
MATH_CONSTANTS.h, 591
TWO_N46
MATH_CONSTANTS.h, 591
TWO_N48
MATH_CONSTANTS.h, 591
TWO_N5
MATH_CONSTANTS.h, 592
TWO_N50
MATH_CONSTANTS.h, 592
TWO_N51
MATH_CONSTANTS.h, 592
TWO_N55
MATH_CONSTANTS.h, 592
TWO_N57
MATH_CONSTANTS.h, 592
TWO_N59
MATH_CONSTANTS.h, 592
TWO_N60
MATH_CONSTANTS.h, 592
TWO_N8
MATH_CONSTANTS.h, 592
TWO_N9
MATH_CONSTANTS.h, 593
TWO_P11
MATH_CONSTANTS.h, 593
TWO_P12
MATH_CONSTANTS.h, 593
TWO_P14
MATH_CONSTANTS.h, 593
TWO_P16
MATH_CONSTANTS.h, 593
TWO_P19
MATH_CONSTANTS.h, 593
TWO_P31
MATH_CONSTANTS.h, 593
TWO_P32
MATH_CONSTANTS.h, 593
TWO_P4
MATH_CONSTANTS.h, 594
TWO_P56
MATH_CONSTANTS.h, 594
TWO_P57
MATH_CONSTANTS.h, 594

tcp_communication, 377
tcp_packet_data, 378
TelemetryDecoderInterface, 378
tlm_dump_reader, 379
to_date_time
    Rinex_Printer, 354
togeod
    Pvt_Solution, 343
topocent
    Pvt_Solution, 344
Tracking_2nd_DLL_filter, 379
    get_code_nco, 380
    initialize, 380
    set_pdi, 380
    Tracking_2nd_DLL_filter, 380
Tracking_2nd_PLL_filter, 381
    initialize, 381
    set_pdi, 381
Tracking_FLL_PLL_filter, 382
tracking_discriminators.h
    dll_nc_e_minus_l_normalized, 527
    dll_nc_vemlp_normalized, 527
    fll_four_quadrant_atan, 527
    pll_cloop_two_quadrant_atan, 527
    pll_four_quadrant_atan, 527
tracking_dump_reader, 381
Tracking_loop_filter, 383
Tracking_timestamp_secs
    Gnss_Synchro, 173
tracking_true_obs_reader, 383
TrackingInterface, 384
tropo
    Pvt_Solution, 344
TwoBitCpxFileSignalSource, 385
    implementation, 386
TwoBitPackedFileSignalSource, 386
    implementation, 387

UhdSignalSource, 387
    implementation, 388
unpack_2bit_samples, 388
unpack_byte_2bit_cpx_samples, 389
unpack_byte_2bit_samples, 390
unpack_intspir_1bit_samples, 390
utc_model
    GpsL1CaSubframeFsm, 272
utc_time
    Gps_Navigation_Message, 228
    Gps_Utc_Model, 240

valid
    Gps_CNAV_Iono, 194
    Gps_Iono, 212
Viterbi_Decoder, 391
    decode_block, 391

WN_0G_10
    Galileo_Navigation_Message, 126
WN_5
    Galileo_Ephemeris, 107
    Galileo_Iono, 114
WNot_6
    Galileo_Utc_Model, 131
what_block
    Gnss_Satellite, 165